

# Simulateur de Circuits — Code source complet

Listing intégral de tous les fichiers de production (hors tests)

**70** fichiers · **22 512** lignes

Généré le 20/06/2026

## Sommaire des fichiers

1. [Simulateur/app.js](#) (982 lignes)
2. [Simulateur/arduino-editor.js](#) (463 lignes)
3. [Simulateur/arduino-lib-commands.mjs](#) (128 lignes)
4. [Simulateur/arduino-lib-popup.js](#) (146 lignes)
5. [Simulateur/arduino-sketch-sync.js](#) (26 lignes)
6. [Simulateur/arduino-syntax-highlight.js](#) (151 lignes)
7. [Simulateur/arduino-uno-layout.js](#) (108 lignes)
8. [Simulateur/bargraph-dc10h-layout.js](#) (72 lignes)
9. [Simulateur/bode-popup.js](#) (369 lignes)
10. [Simulateur/cd4511-layout.js](#) (51 lignes)
11. [Simulateur/dht22-layout.js](#) (39 lignes)
12. [Simulateur/Engine/arduino-analog-ideal.mjs](#) (201 lignes)
13. [Simulateur/Engine/arduino-avr-registers.mjs](#) (238 lignes)
14. [Simulateur/Engine/arduino-gpio-ideal.mjs](#) (221 lignes)
15. [Simulateur/Engine/arduino-sketch-parse.mjs](#) (1200 lignes)
16. [Simulateur/Engine/arduino-uart-wave.mjs](#) (157 lignes)
17. [Simulateur/Engine/arduino-uno.mjs](#) (219 lignes)
18. [Simulateur/Engine/bcd-seg7.mjs](#) (49 lignes)
19. [Simulateur/Engine/build-bridge-json.mjs](#) (178 lignes)
20. [Simulateur/Engine/dht22-ideal.mjs](#) (140 lignes)
21. [Simulateur/Engine/dht22-sketch-parse.mjs](#) (111 lignes)
22. [Simulateur/Engine/grove-lcd-ideal.mjs](#) (238 lignes)
23. [Simulateur/Engine/grove-lcd-sketch-parse.mjs](#) (515 lignes)
24. [Simulateur/Engine/hc90-cascade.mjs](#) (555 lignes)
25. [Simulateur/Engine/i2c-bus-ideal.mjs](#) (112 lignes)
26. [Simulateur/Engine/i2c-protocol.mjs](#) (224 lignes)
27. [Simulateur/Engine/lcd-pcf8574-i2c.mjs](#) (192 lignes)
28. [Simulateur/Engine/logic-74hc90.mjs](#) (211 lignes)
29. [Simulateur/Engine/logic-cd4511-bsource.mjs](#) (125 lignes)
30. [Simulateur/Engine/logic-cd4511-xspice.mjs](#) (194 lignes)
31. [Simulateur/Engine/logic-rails.mjs](#) (43 lignes)
32. [Simulateur/Engine/logic-sequential.mjs](#) (542 lignes)
33. [Simulateur/Engine/logic-xspice.mjs](#) (295 lignes)
34. [Simulateur/Engine/ngspice-xspice-probe.mjs](#) (92 lignes)
35. [Simulateur/Engine/ripple-mod10.mjs](#) (84 lignes)
36. [Simulateur/Engine/schematic-to-spice.mjs](#) (2919 lignes)
37. [Simulateur/Engine/scope-arduino-ideal.mjs](#) (160 lignes)
38. [Simulateur/Engine/simulate-smoke.mjs](#) (62 lignes)
39. [Simulateur/Engine/spice-netlist-v2.mjs](#) (93 lignes)
40. [Simulateur/Engine/v2/result-parser.mjs](#) (1335 lignes)
41. [Simulateur/Engine/voltmeter-display.mjs](#) (23 lignes)
42. [Simulateur/geometry.js](#) (326 lignes)
43. [Simulateur/gimp-panel.js](#) (10 lignes)
44. [Simulateur/grove-lcd-layout.js](#) (52 lignes)
45. [Simulateur/hd44780-font.js](#) (125 lignes)
46. [Simulateur/ic74hc90-layout.js](#) (90 lignes)
47. [Simulateur/index.html](#) (703 lignes)
48. [Simulateur/json-utils.js](#) (72 lignes)
49. [Simulateur/led-animation.js](#) (1004 lignes)
50. [Simulateur/modal-ui.js](#) (22 lignes)
51. [Simulateur/renderer.js](#) (1690 lignes)
52. [Simulateur/scope-animation.js](#) (271 lignes)
53. [Simulateur/scope-panel.js](#) (151 lignes)
54. [Simulateur/scope-popup.js](#) (233 lignes)
55. [Simulateur/serial-monitor-popup.js](#) (234 lignes)
56. [Simulateur/simulation.js](#) (551 lignes)
57. [Simulateur/source-panel.js](#) (226 lignes)
58. [Simulateur/speaker-audio.js](#) (366 lignes)
59. [Simulateur/state.js](#) (74 lignes)
60. [Simulateur/style.css](#) (1203 lignes)
61. [Simulateur/theme.js](#) (75 lignes)
62. [Simulateur/value-prompt.js](#) (67 lignes)
63. [SimulateurH/main.js](#) (116 lignes)
64. [SimulateurH/simulate-server.cjs](#) (625 lignes)
65. [tools/arduino-api.cjs](#) (427 lignes)
66. [tools/arduino-routes.cjs](#) (125 lignes)
67. [tools/check-ngspice.cjs](#) (101 lignes)
68. [tools/ngspice-bundle.cjs](#) (200 lignes)
69. [tools/read-json-safe.cjs](#) (32 lignes)
70. [tools/simulate-engine-loader.cjs](#) (78 lignes)

```

1 // app.js (Nouveau fichier principal)
2 import { canvas, GRID_SIZE, scale, pan, flags, counters, circuit, interaction, zone, menuDrag, clipboard, undoStack, redoStack, emptyDragImage, snapToGrid, toGridCoords, saveState } from './state.js';
3 import { isPointOnSegment, findWireIntersection, getComponentJunctions, componentHitTest, potentiometerControlHit, switchSpdtToggleHit, pushButtonToggleHit } from './geometry.js';
4 import { resizeCanvas, draw } from './renderer.js';
5 import { triggerSimulation, stopSimulation, requestLiveSimulation } from './simulation.js';
6 import { openSourcePanel, closeSourcePanel, onSourceRemoved, initSourcePanel } from './source-panel.js';
7 import { openScopePanel, closeScopePanelFully, onScopeRemoved, initScopePanel, onScopePopupClosed, refreshScopePanelFields } from './scope-panel.js';
8 import { initScopePopup, refreshScopePopup, setScopePopupCloseCallback, setScopeViewChangeCallback } from './scope-popup.js';
9 import { initSerialMonitor, openSerialMonitor, refreshSerialMonitor } from './serial-monitor-popup.js';
10 import { initArduinoLibPopup } from './arduino-lib-popup.js';
11 import { initBodePopup, openBodePopup } from './bode-popup.js';
12 import { bindLedAnimationRedraw, onCircuitLoaded } from './led-animation.js';
13 import { applyArduinoSketchToComponent } from './Engine/arduino-sketch-parse.mjs';
14 import { bindSpeakerAudioRedraw } from './speaker-audio.js';
15 import { bindScopeAnimationRedraw, bindScopePopupRedraw } from './scope-animation.js';
16 import { initEditorTheme, setEditorTheme } from './theme.js';
17 import { initValuePrompt, showValuePrompt } from './value-prompt.js';
18 import {
19   initArduinoEditor,
20   openArduinoEditor,
21   openArduinoEditorForCircuit,
22   getActiveArduinoBoard,
23   onArduinoBoardRemoved,
24   resetArduinoEditorOnCircuitLoad,
25   flushArduinoSketchesBeforeSave,
26 } from './arduino-editor.js';
27 import { DEFAULT_ARDUINO_SKETCH } from './arduino-uno-layout.js';
28 import { showModal, hideModal, initModalUi } from './modal-ui.js';
29 import { parseJsonText, normalizeCircuitPayload, migrateLoadedComponents } from './json-utils.js';
30 import { DC10H_COLOR_IDS, DC10H_COLORS } from './bargraph-dc10h-layout.js';
31
32 /** « Simulateur H » (application Windows) ou site web. */
33 const APP_PRODUCT_NAME =
34   new URLSearchParams(window.location.search).get('app') === 'h'
35     ? 'Simulateur H'
36     : 'Simulateur de Circuits';
37
38 const COMPONENT_PREFIX = {
39   battery: 'VDC', resistor: 'R', potentiometer: 'POT', switch_spdt: 'SW', push_button: 'BP', capacitor: 'C', inductor: 'L', diode: 'D',
40   npn: 'Q', opamp: 'AOP',
41   not: 'NOT', and: 'AND', nand: 'NAND', or: 'OR', nor: 'NOR', xor: 'XOR', xnor: 'XNOR',
42   d_flipflop: 'DFF', jk_flipflop: 'JKFF', cd4511: 'CD4511', ic_74hc90: 'HC90', arduino_uno: 'UNO', led: 'LED', seg7: 'SEG', bargraph_dc10h: 'BAR', grove_lcd16x2: 'LCD', grove_dht22: 'DHT',
43   voltmeter: 'V', ammeter: 'A', ohmmeter: 'OHM', oscilloscope: 'Osci', bode_analyzer: 'Bode', speaker: 'HP', gnd: 'GND', vcc: 'VCC', logic_terminal: 'LOGIC', gimp: 'GImp', gsin: 'Sin', gsqr: 'Sq',
44 };
45 const NON_ROTATABLE = new Set(['d_flipflop', 'jk_flipflop', 'cd4511', 'ic_74hc90', 'arduino_uno', 'gimp', 'gsin', 'gsqr', 'oscilloscope', 'npn', 'opamp', 'seg7', 'bargraph_dc10h', 'grove_lcd16x2', 'grove_dht22']);
46 function ensureComponentCounter(type) {
47   if (counters[type] == null || !Number.isFinite(counters[type])) counters[type] = 0;
48   counters[type]++;
49   return counters[type];
50 }
51 function newComponentLabel(type) {
52   const pfx = COMPONENT_PREFIX[type] || 'U';
53   return `${pfx}${ensureComponentCounter(type)}`;
54 }
55 function ensureAllCounters() {
56   for (const type of Object.keys(COMPONENT_PREFIX)) {
57     if (counters[type] == null || !Number.isFinite(counters[type])) counters[type] = 0;
58   }
59   if (counters.junction == null || !Number.isFinite(counters.junction)) counters.junction = 0;
60 }
61 let fileHandle = null;
62 let circuitDisplayName = 'Sans titre';
63
64 function formatCircuitDisplayName(raw) {
65   const s = String(raw || '').trim();
66   if (!s) return 'Sans titre';
67   return s.replace(/\.json$/i, '');
68 }
69
70 function setCircuitDisplayName(name) {
71   circuitDisplayName = formatCircuitDisplayName(name);
72   const el = document.getElementById('circuit-name');
73   if (el) {

```

```

74     el.textContent = circuitDisplayName;
75     el.title = `Montage : ${circuitDisplayName}`;
76 }
77 document.title = `${circuitDisplayName} - ${APP_PRODUCT_NAME}`;
78 }
79 let lastMouseGridPos = { x: 0, y: 0 };
80 let liveDragMoved = false;
81 let heldPushButton = null;
82
83 // --- HISTORIQUE & SÉCURITÉ ---
84 function undo() {
85     if (flags.isSimulating) { alert("Veuillez arrêter la simulation avant de modifier le schéma."); return; }
86     if (undoStack.length === 0) return;
87     redoStack.push(JSON.stringify({ components: circuit.components, wires: circuit.wires, autoJunctions: circuit.autoJunctions, counters }));
88     const prev = JSON.parse(undoStack.pop());
89     circuit.components = prev.components; circuit.wires = prev.wires; circuit.autoJunctions = prev.autoJunctions; Object.assign(counters, prev.counters);
90     interaction.selectedComponents = []; interaction.selectedAutoJunctions = []; interaction.selectedWire = null; draw();
91 }
92 function redo() {
93     if (flags.isSimulating) { alert("Veuillez arrêter la simulation avant de modifier le schéma."); return; }
94     if (redoStack.length === 0) return;
95     undoStack.push(JSON.stringify({ components: circuit.components, wires: circuit.wires, autoJunctions: circuit.autoJunctions, counters }));
96     const next = JSON.parse(redoStack.pop());
97     circuit.components = next.components; circuit.wires = next.wires; circuit.autoJunctions = next.autoJunctions; Object.assign(counters, next.counters);
98     interaction.selectedComponents = []; interaction.selectedAutoJunctions = []; interaction.selectedWire = null; draw();
99 }
100
101 // --- FICHIERS ET SAUVEGARDE ---
102 function getCircuitDataJSON() {
103     flushArduinoSketchesBeforeSave();
104     return JSON.stringify(
105         {
106             name: circuitDisplayName,
107             components: circuit.components,
108             wires: circuit.wires,
109             autoJunctions: circuit.autoJunctions,
110             counters,
111         },
112         null,
113         4
114     );
115 }
116 async function saveAs() {
117     try {
118         fileHandle = await window.showSaveFilePicker({ suggestedName: 'schema_circuit.json', types: [{ description: 'Fichier JSON Circuit', accept: { 'application/json': ['.json'] } }] });
119         setCircuitDisplayName(fileHandle.name);
120         const writable = await fileHandle.createWritable(); await writable.write(getCircuitDataJSON()); await writable.close();
121     } catch (err) { fallbackDownload(getCircuitDataJSON()); }
122 }
123 async function saveFile() {
124     if (fileHandle) { try { const w = await fileHandle.createWritable(); await w.write(getCircuitDataJSON()); await w.close(); } catch (e) { await saveAs(); } } else { await saveAs(); }
125 }
126 function fallbackDownload(content) {
127     const a = document.createElement('a'); a.href = URL.createObjectURL(new Blob([content], { type: 'application/json' })); a.download = 'schema_export.json'; a.click();
128 }
129 async function openFile() {
130     if (circuit.components.length > 0 && confirm("Voulez-vous sauvegarder votre schéma actuel ?")) { await saveFile(); }
131     try {
132         const [handle] = await window.showOpenFilePicker({ types: [{ accept: { 'application/json': ['.json'] } }] });
133         fileHandle = handle;
134         setCircuitDisplayName(handle.name);
135         loadCircuitFromJSON(await (await handle.getFile()).text());
136     } catch (err) {
137         if (err?.name === 'AbortError') return;
138         const input = document.createElement('input'); input.type = 'file'; input.accept = '.json,application/json';
139         input.onchange = e => {
140             const file = e.target.files?.[0];
141             if (!file) return;
142             fileHandle = null;
143             setCircuitDisplayName(file.name);
144             const r = new FileReader();
145             r.onload = ev => loadCircuitFromJSON(ev.target.result);
146             r.onerror = () => alert('Impossible de lire le fichier sélectionné.');
```



```

150     }
151 }
152 function syncCountersFromLabels(components) {
153     for (const comp of components) {
154         const pfx = COMPONENT_PREFIX[comp.type];
155         if (!pfx || !comp.label?.startsWith(pfx)) continue;
156         const n = parseInt(comp.label.slice(pfx.length), 10);
157         if (Number.isFinite(n) && n > (counters[comp.type] || 0)) {
158             counters[comp.type] = n;
159         }
160     }
161 }
162
163 function repairComponentsAfterLoad(components) {
164     migrateLoadedComponents(components);
165     for (const comp of components) {
166         if (comp.type === 'arduino_uno') {
167             if (typeof comp.sketch !== 'string') comp.sketch = DEFAULT_ARDUINO_SKETCH;
168             if (!comp.fqbn) comp.fqbn = 'arduino:avr:uno';
169             comp.pinModes = comp.pinModes || {};
170             comp.pinLevels = comp.pinLevels || {};
171             comp.lastCompileLog = typeof comp.lastCompileLog === 'string' ? comp.lastCompileLog : '';
172             applyArduinoSketchToComponent(comp);
173         }
174     }
175 }
176
177 function resetUiAfterCircuitLoad() {
178     flags.isDraggingFromMenu = false;
179     menuDrag.draggedComponentType = null;
180     flags.isPanning = false;
181     interaction.activeWire = null;
182     document.querySelectorAll('.navbare > .menu-item.open').forEach((m) => m.classList.remove('open'));
183     document.querySelectorAll('.dropdown-submenu.open').forEach((s) => s.classList.remove('open'));
184     hideModal(document.getElementById('value-prompt-modal'));
185 }
186
187 function loadCircuitFromJSON(jsonText) {
188     try {
189         const data = normalizeCircuitPayload(parseJsonText(jsonText));
190         if (data.name) setCircuitDisplayName(data.name);
191         resetArduinoEditorOnCircuitLoad();
192         for (const k of Object.keys(counters)) counters[k] = 0;
193         circuit.components = data.components;
194         circuit.wires = data.wires;
195         circuit.autoJunctions = data.autoJunctions;
196         Object.assign(counters, data.counters);
197         repairComponentsAfterLoad(circuit.components);
198         ensureAllCounters();
199         syncCountersFromLabels(circuit.components);
200         interaction.selectedComponents = [];
201         interaction.selectedAutoJunctions = [];
202         interaction.selectedWire = null;
203         undoStack.length = 0;
204         redoStack.length = 0;
205         stopSimulation();
206         closeSourcePanel();
207         closeScopePanelFully();
208         resetUiAfterCircuitLoad();
209         onCircuitLoaded();
210         draw();
211     } catch (e) {
212         console.error('loadCircuitFromJSON', e);
213         alert('Erreur lors de la lecture du fichier JSON.\n\n${e?.message || e}');
214     }
215 }
216
217 // --- CLAVIER & RECOPIE ---
218 function isTextFieldFocused() {
219     const el = document.activeElement;
220     if (!el) return false;
221     const tag = el.tagName;
222     if (tag === 'TEXTAREA' || tag === 'INPUT' || tag === 'SELECT') return true;
223     return el.isContentEditable === true;
224 }
225
226 window.addEventListener('keydown', (e) => {
227     const key = e.key.toLowerCase(); if (e.key === 'Shift') flags.isShiftPressed = true;
228     if (isTextFieldFocused()) return;
229     if (e.code === 'Space' && interaction.activeWire) {

```

```

230     e.preventDefault(); const last = interaction.activeWire.points[interaction.activeWire.points.length - 1]; interaction.activeWire.poin
ts.push({ x: last.x, y: last.y }); draw(); return;
231 }
232 if (key === 'r' && !interaction.activeWire && interaction.selectedComponents.length > 0) {
233     if (flags.isSimulating) { alert("Arrêtez la simulation avant de pivoter."); return; }
234     const rotatable = interaction.selectedComponents.filter(comp => !NON_ROTATABLE.has(comp.type));
235     if (rotatable.length === 0) { alert("La rotation est verrouillée pour les bascules, le GImp, le transistor et l'AOP."); return; }
236     saveState(); rotatable.forEach(comp => comp.rotation = ((comp.rotation || 0) + 90) % 360); draw(); return;
237 }
238 if (key === 'x' && !interaction.activeWire && interaction.selectedComponents.length > 0) {
239     if (flags.isSimulating) { alert("Arrêtez la simulation avant de retourner."); return; }
240     const flippable = interaction.selectedComponents.filter(comp =>
241         comp.type === 'gimp' || comp.type === 'npn' || comp.type === 'opamp' || comp.type === 'grove_lcd16x2' || comp.type === 'grove_dht
22' || comp.type === 'bargraph_dc10h');
242     if (flippable.length === 0) return;
243     saveState(); flippable.forEach(comp => { comp.flipX = !comp.flipX; }); draw(); return;
244 }
245 if (e.key === 'Delete' || e.key === 'Backspace') {
246     if (interaction.selectedComponents.length > 0 || interaction.selectedAutoJunctions.length > 0 || interaction.selectedWire !== null) {
247         if (flags.isSimulating) { alert("Arrêtez la simulation avant de supprimer."); return; }
248         saveState();
249         if (interaction.selectedComponents.length > 0) {
250             interaction.selectedComponents.forEach(comp => {
251                 onSourceRemoved(comp);
252                 onScopeRemoved(comp);
253                 onArduinoBoardRemoved(comp);
254                 const idx = circuit.components.indexOf(comp); if (idx > -1) circuit.components.splice(idx, 1);
255                 circuit.wires = circuit.wires.filter(w => !w.fromJunctionId.startsWith(comp.label) && !w.toJunctionId.startsWith(comp.lab
el));
256             }); interaction.selectedComponents = [];
257         }
258         if (interaction.selectedAutoJunctions.length > 0) {
259             interaction.selectedAutoJunctions.forEach(aj => {
260                 const idx = circuit.autoJunctions.indexOf(aj); if (idx > -1) circuit.autoJunctions.splice(idx, 1);
261                 circuit.wires = circuit.wires.filter(w => w.fromJunctionId !== aj.id && w.toJunctionId !== aj.id);
262             }); interaction.selectedAutoJunctions = [];
263         }
264         if (interaction.selectedWire) { const idx = circuit.wires.indexOf(interaction.selectedWire); if (idx > -1) circuit.wires.splice(i
dx, 1); interaction.selectedWire = null; }
265         circuit.autoJunctions = circuit.autoJunctions.filter(aj => circuit.wires.some(w => w.fromJunctionId === aj.id || w.toJunctionId =
=== aj.id)); draw(); return;
266     }
267 }
268 if (e.ctrlKey && key === 'c' && (interaction.selectedComponents.length > 0 || interaction.selectedAutoJunctions.length > 0)) {
269     const internalWires = circuit.wires.filter(w => {
270         const fC = interaction.selectedComponents.some(c => w.fromJunctionId.startsWith(c.label)), fJ = interaction.selectedAutoJunction
s.some(aj => w.fromJunctionId === aj.id);
271         const tC = interaction.selectedComponents.some(c => w.toJunctionId.startsWith(c.label)), tJ = interaction.selectedAutoJunctions.s
ome(aj => w.toJunctionId === aj.id);
272         return (fC || fJ) && (tC || tJ);
273     });
274     clipboard.data = { components: JSON.parse(JSON.stringify(interaction.selectedComponents)), autoJunctions: JSON.parse(JSON.stringify(i
nteraction.selectedAutoJunctions)), wires: JSON.parse(JSON.stringify(internalWires)) };
275 }
276 if (e.ctrlKey && key === 'v' && clipboard.data) {
277     if (flags.isSimulating) { alert("Arrêtez la simulation avant de coller."); return; }
278     saveState(); const labelMap = {}, juncMap = {}, newC = [], newJ = [], newW = [];
279     clipboard.data.components.forEach(comp => {
280         const n1 = newComponentLabel(comp.type); labelMap[comp.label] = n1;
281         const cloned = { ...comp, x: comp.x + 40, y: comp.y + 40, label: n1 }; circuit.components.push(cloned); newC.push(cloned);
282     });
283     clipboard.data.autoJunctions.forEach(aj => {
284         counters.junction++; const njid = `auto_junc_${counters.junction}`; juncMap[aj.id] = njid;
285         const cloned = { id: njid, x: aj.x + 40, y: aj.y + 40 }; circuit.autoJunctions.push(cloned); newJ.push(cloned);
286     });
287     clipboard.data.wires.forEach(w => {
288         let fid = w.fromJunctionId, tid = w.toJunctionId;
289         for (let ol in labelMap) { if (fid.startsWith(ol)) fid = fid.replace(ol, labelMap[ol]); if (tid.startsWith(ol)) tid = tid.replace
(ol, labelMap[ol]); }
290         if (juncMap[fid]) fid = juncMap[fid]; if (juncMap[tid]) tid = juncMap[tid];
291         const cloned = { fromJunctionId: fid, toJunctionId: tid, points: w.points.map(pt => ({ x: pt.x + 40, y: pt.y + 40 })); circuit.
wires.push(cloned); newW.push(cloned);
292     });
293     interaction.selectedComponents = newC; interaction.selectedAutoJunctions = newJ; interaction.selectedWire = null;
294     clipboard.data = { components: JSON.parse(JSON.stringify(newC)), autoJunctions: JSON.parse(JSON.stringify(newJ)), wires: JSON.parse(J
SON.stringify(newW)) }; draw();
295 }
296 if (e.ctrlKey && key === 'z') { e.preventDefault(); undo(); }
297 if (e.ctrlKey && key === 'y') { e.preventDefault(); redo(); }
298 });
299 window.addEventListener('keyup', (e) => { if (e.key === 'Shift') flags.isShiftPressed = false; });
300

```

```

301 // --- GESTION DES MOUSE EVENTS ---
302 function updateMouseState(e) {
303     const mousePos = toGridCoords(e.clientX, e.clientY);
304     interaction.hoverJunction = null; interaction.hoveredComponent = null; interaction.hoveredWire = null;
305     if (flags.isPanning || flags.isSelectingZone) return;
306
307     interaction.hoveredComponent = circuit.components.find(comp => componentHitTest(comp, mousePos.x, mousePos.y));
308     for (let comp of circuit.components) {
309         for (let j of getComponentJunctions(comp)) { if (Math.hypot(mousePos.x - j.x, mousePos.y - j.y) < 12) { interaction.hoverJunction = j; break; } }
310         if (interaction.hoverJunction) break;
311     }
312     if (!interaction.hoverJunction) {
313         for (let aj of circuit.autoJunctions) { if (Math.hypot(mousePos.x - aj.x, mousePos.y - aj.y) < 12) { interaction.hoverJunction = aj; break; } }
314     }
315     if (!interaction.hoveredComponent && !interaction.hoverJunction && !interaction.activeWire) {
316         for (let w of circuit.wires) {
317             for (let i = 0; i < w.points.length - 1; i++) {
318                 if (isPointOnSegment(mousePos.x, mousePos.y, w.points[i], w.points[i+1], 5)) { interaction.hoveredWire = w; break; }
319             }
320             if (interaction.hoveredWire) break;
321         }
322     }
323     canvas.style.cursor = flags.isShiftPressed ? 'cell' : (interaction.activeWire || interaction.hoverJunction ? 'crosshair' : (flags.isDraggingComponent ? 'grabbing' : (interaction.hoveredComponent ? 'pointer' : (interaction.hoveredWire ? 'help' : 'grab'))));
324 }
325
326 canvas.addEventListener('mousedown', (e) => {
327     const mousePos = toGridCoords(e.clientX, e.clientY); const sX = snapToGrid(mousePos.x), sY = snapToGrid(mousePos.y);
328     lastMouseGridPos = { x: sX, y: sY }; updateMouseState(e);
329     if (e.button === 0) {
330         const hitComp = circuit.components.find((c) => componentHitTest(c, mousePos.x, mousePos.y));
331         if (hitComp?.type === 'potentiometer') {
332             const ctrl = potentiometerControlHit(hitComp, mousePos.x, mousePos.y);
333             if (ctrl) {
334                 if (!flags.isSimulating) saveState();
335                 const step = 5;
336                 const pos = hitComp.position ?? 50;
337                 hitComp.position = ctrl === 'inc' ? Math.min(100, pos + step) : Math.max(0, pos - step);
338                 draw();
339                 if (flags.isSimulating) requestLiveSimulation();
340                 return;
341             }
342         }
343         if (hitComp?.type === 'switch_spdt' && !interaction.hoverJunction && switchSpdtToggleHit(hitComp, mousePos.x, mousePos.y)) {
344             if (!flags.isSimulating) saveState();
345             hitComp.state = hitComp.state === 1 ? 0 : 1;
346             draw();
347             if (flags.isSimulating) requestLiveSimulation();
348             return;
349         }
350         if (hitComp?.type === 'push_button' && !interaction.hoverJunction && pushButtonToggleHit(hitComp, mousePos.x, mousePos.y)) {
351             if (!flags.isSimulating) saveState();
352             if (hitComp.maintained) {
353                 hitComp.state = hitComp.state === 1 ? 0 : 1;
354             } else {
355                 hitComp.state = 1;
356                 heldPushButton = hitComp;
357             }
358             draw();
359             if (flags.isSimulating) requestLiveSimulation();
360             return;
361         }
362         if (interaction.hoveredComponent && interaction.hoveredComponent.type === 'logic_terminal' && flags.isSimulating) {
363             saveState(); interaction.hoveredComponent.state = interaction.hoveredComponent.state === 1 ? 0 : 1; draw();
364             triggerSimulation(true); return;
365         }
366         if (flags.isShiftPressed) { flags.isSelectingZone = true; zone.start = { x: mousePos.x, y: mousePos.y }; zone.end = { x: mousePos.x, y: mousePos.y }; draw(); return; }
367         if (interaction.hoverJunction) {
368             if (flags.isSimulating) { alert("Arrêtez la simulation avant d'éditer."); return; }
369             const matchedaj = circuit.autoJunctions.find(aj => aj.id === interaction.hoverJunction.id);
370             if (matchedaj && !interaction.activeWire) { interaction.selectedComponents = []; interaction.selectedAutoJunctions = [matchedaj]; flags.isDraggingComponent = true; saveState(); draw(); return; }
371             if (!interaction.activeWire) { interaction.activeWire = { fromJunctionId: interaction.hoverJunction.id, points: [{ x: interaction.hoverJunction.x, y: interaction.hoverJunction.y }, { x: sX, y: sY }] }; }
372             else { if (interaction.hoverJunction.id !== interaction.activeWire.fromJunctionId) { saveState(); interaction.activeWire.points[interaction.activeWire.points.length - 1] = { x: interaction.hoverJunction.x, y: interaction.hoverJunction.y }; interaction.activeWire.toJunctionId = interaction.hoverJunction.id; circuit.wires.push(interaction.activeWire); } interaction.activeWire = null; }
373             interaction.selectedWire = null; draw(); return;

```

```

374     }
375     if (interaction.activeWire) {
376         const intersect = findWireIntersection(mousePos.x, mousePos.y);
377         if (intersect) {
378             saveState(); counters.junction++; const njid = `auto_junc_${counters.junction}`; circuit.autoJunctions.push({ id: njid, x: in
tersect.x, y: intersect.y });
379             interaction.activeWire.points[interaction.activeWire.points.length - 1] = { x: intersect.x, y: intersect.y }; interaction.act
iveWire.toJunctionId = njid; circuit.wires.push(interaction.activeWire); interaction.activeWire = null; interaction.selectedWire = null; draw
(); return;
380         }
381     }
382     if (interaction.hoveredWire) { interaction.selectedWire = interaction.hoveredWire; interaction.selectedComponents = []; interaction.s
electedAutoJunctions = []; draw(); return; } else { interaction.selectedWire = null; }
383 }
384 if (interaction.activeWire && e.button === 2) { interaction.activeWire = null; draw(); return; }
385 if (interaction.hoveredComponent) {
386     const hc = interaction.hoveredComponent;
387     if (flags.isSimulating) {
388         if (hc.type === 'oscilloscope') {
389             closeSourcePanel();
390             openScopePanel(hc);
391             interaction.selectedComponents = [hc];
392             draw();
393             return;
394         }
395         if (hc.type === 'bode_analyzer') {
396             closeSourcePanel();
397             closeScopePanelFully();
398             openBodePopup(hc);
399             interaction.selectedComponents = [hc];
400             draw();
401             return;
402         }
403         if (hc.type === 'gimp' || hc.type === 'gsin' || hc.type === 'gsqr') {
404             closeScopePanelFully();
405             openSourcePanel(hc);
406             interaction.selectedComponents = [hc];
407             draw();
408             return;
409         }
410         if (hc.type === 'arduino_uno') {
411             openArduinoEditor(hc);
412             interaction.selectedComponents = [hc];
413             draw();
414             return;
415         }
416         if (!interaction.selectedComponents.includes(hc)) {
417             interaction.selectedComponents = [hc];
418             interaction.selectedAutoJunctions = [];
419         }
420         flags.isDraggingComponent = true;
421         liveDragMoved = false;
422         draw();
423         return;
424     }
425     if (!interaction.selectedComponents.includes(hc)) { interaction.selectedComponents = [hc]; interaction.selectedAutoJunctions = []; }
426     if (hc.type === 'gimp' || hc.type === 'gsin' || hc.type === 'gsqr') { closeScopePanelFully(); openSourcePanel(hc); }
427     else if (hc.type === 'oscilloscope') { closeSourcePanel(); openScopePanel(hc); }
428     else if (hc.type === 'arduino_uno') { openArduinoEditor(hc); }
429     else { closeSourcePanel(); closeScopePanelFully(); }
430     flags.isDraggingComponent = true; saveState();
431 } else { interaction.selectedComponents = []; interaction.selectedAutoJunctions = []; if (!flags.isSimulating) { closeSourcePanel(); clos
eScopePanelFully(); } flags.isPanning = true; flags.startX = e.clientX - pan.x; flags.startY = e.clientY - pan.y; }
432 draw();
433 });
434
435 canvas.addEventListener('mouseleave', () => {
436     if (heldPushButton) {
437         heldPushButton.state = 0;
438         heldPushButton = null;
439         draw();
440         if (flags.isSimulating) requestLiveSimulation();
441     }
442 });
443 canvas.addEventListener('mousemove', (e) => {
444     const mousePos = toGridCoords(e.clientX, e.clientY);
445     if (flags.isSelectingZone) { zone.end = { x: mousePos.x, y: mousePos.y }; draw(); return; }
446     updateMouseState(e);
447     if (interaction.activeWire) { interaction.activeWire.points[interaction.activeWire.points.length - 1] = { x: snapToGrid(mousePos.x), y: s
napToGrid(mousePos.y) }; draw(); }
448     else if (flags.isPanning) { pan.x = e.clientX - flags.startX; pan.y = e.clientY - flags.startY; draw(); }

```

```

449     else if (flags.isDraggingComponent && (interaction.selectedComponents.length > 0 || interaction.selectedAutoJunctions.length > 0)) {
450         const sX = snapToGrid(mousePos.x), sY = snapToGrid(mousePos.y), dX = sX - lastMouseGridPos.x, dY = sY - lastMouseGridPos.y;
451         if (dX !== 0 || dY !== 0) {
452             interaction.selectedComponents.forEach(c => { c.x += dX; c.y += dY; }); interaction.selectedAutoJunctions.forEach(aj => { aj.x +=
dX; aj.y += dY; });
453             circuit.wires.forEach(w => {
454                 const fMoved = interaction.selectedComponents.some(c => w.fromJunctionId.startsWith(c.label)) || interaction.selectedAutoJunc
tions.some(aj => w.fromJunctionId === aj.id);
455                 const tMoved = interaction.selectedComponents.some(c => w.toJunctionId.startsWith(c.label)) || interaction.selectedAutoJuncti
ons.some(aj => w.toJunctionId === aj.id);
456                 if (fMoved && tMoved) { w.points.forEach(pt => { pt.x += dX; pt.y += dY; }); } else if (fMoved) { w.points[0].x += dX; w.poin
ts[0].y += dY; } else if (tMoved) { w.points[w.points.length - 1].x += dX; w.points[w.points.length - 1].y += dY; }
457             });
458             lastMouseGridPos = { x: sX, y: sY };
459             if (flags.isSimulating) liveDragMoved = true;
460             } draw();
461         } else draw();
462     });
463
464     canvas.addEventListener('mouseup', () => {
465         if (heldPushButton) {
466             heldPushButton.state = 0;
467             heldPushButton = null;
468             draw();
469             if (flags.isSimulating) requestLiveSimulation();
470         }
471         if (flags.isSelectingZone) {
472             flags.isSelectingZone = false; const xMi = Math.min(zone.start.x, zone.end.x), xMa = Math.max(zone.start.x, zone.end.x), yMi = Math.m
in(zone.start.y, zone.end.y), yMa = Math.max(zone.start.y, zone.end.y);
473             interaction.selectedComponents = circuit.components.filter(c => c.x >= xMi && c.x <= xMa && c.y >= yMi && c.y <= yMa); interaction.se
lectedAutoJunctions = circuit.autoJunctions.filter(aj => aj.x >= xMi && aj.x <= xMa && aj.y >= yMi && aj.y <= yMa);
474         }
475         const wasDragging = flags.isDraggingComponent;
476         flags.isPanning = false; flags.isDraggingComponent = false;
477         if (wasDragging && flags.isSimulating && liveDragMoved) {
478             requestLiveSimulation();
479         }
480         draw();
481     });
482
483     function openCd4511DocModal(componentLabel) {
484         const modal = document.getElementById('cd4511-doc-modal');
485         const title = document.getElementById('cd4511-doc-title');
486         if (!modal) return;
487         if (title) {
488             title.textContent = componentLabel
489                 ? `CD4511 - ${componentLabel}`
490                 : 'CD4511 - Décodeur BCD / 7 segments';
491         }
492         showModal(modal);
493     }
494
495     function closeCd4511DocModal() {
496         hideModal(document.getElementById('cd4511-doc-modal'));
497     }
498
499     function openHc90DocModal(componentLabel) {
500         const modal = document.getElementById('hc90-doc-modal');
501         const title = document.getElementById('hc90-doc-title');
502         if (!modal) return;
503         if (title) {
504             title.textContent = componentLabel
505                 ? `74HC90 - ${componentLabel}`
506                 : '74HC90 - Compteur décade asynchrone';
507         }
508         showModal(modal);
509     }
510
511     function closeHc90DocModal() {
512         hideModal(document.getElementById('hc90-doc-modal'));
513     }
514
515     function openUnoDocModal(componentLabel) {
516         const modal = document.getElementById('uno-doc-modal');
517         const title = document.getElementById('uno-doc-title');
518         if (!modal) return;
519         if (title) {
520             title.textContent = componentLabel
521                 ? `Arduino UNO R3 - ${componentLabel}`
522                 : 'Arduino UNO R3';
523         }

```

```

524     showModal(modal);
525 }
526
527 function closeUnoDocModal() {
528     hideModal(document.getElementById('uno-doc-modal'));
529 }
530
531 let activeBargraphDocComp = null;
532
533 function refreshBargraphColorPicker(selectedId) {
534     const picker = document.getElementById('bargraph-color-picker');
535     if (!picker) return;
536     picker.innerHTML = '';
537     for (const id of DC10H_COLOR_IDS) {
538         const pal = DC10H_COLORS[id];
539         const btn = document.createElement('button');
540         btn.type = 'button';
541         btn.className = 'bargraph-color-btn' + (id === selectedId ? ' is-active' : '');
542         btn.dataset.colorId = id;
543         btn.title = pal.label;
544         const swatch = document.createElement('span');
545         swatch.className = 'bargraph-color-swatch';
546         swatch.style.background = pal.lit;
547         btn.appendChild(swatch);
548         btn.appendChild(document.createTextNode(pal.label));
549         btn.addEventListener('click', () => {
550             if (!activeBargraphDocComp) return;
551             activeBargraphDocComp.barColor = id;
552             refreshBargraphColorPicker(id);
553             draw();
554         });
555         picker.appendChild(btn);
556     }
557 }
558
559 function openBargraphDocModal(comp) {
560     const modal = document.getElementById('bargraph-doc-modal');
561     const title = document.getElementById('bargraph-doc-title');
562     if (!modal || !comp) return;
563     activeBargraphDocComp = comp;
564     if (title) {
565         title.textContent = comp.label
566             ? `Bargraph DC10H - ${comp.label}`
567             : 'Bargraph DC10H - Lite-On';
568     }
569     refreshBargraphColorPicker(comp.barColor || 'red');
570     showModal(modal);
571 }
572
573 function closeBargraphDocModal() {
574     activeBargraphDocComp = null;
575     hideModal(document.getElementById('bargraph-doc-modal'));
576 }
577
578 canvas.addEventListener('dblclick', async (e) => {
579     const mousePos = toGridCoords(e.clientX, e.clientY); const target = circuit.components.find(c => componentHitTest(c, mousePos.x, mousePos.y));
580     if (target) {
581         if (target.type === 'cd4511') {
582             openCd4511DocModal(target.label);
583             return;
584         }
585         if (target.type === 'bargraph_dc10h') {
586             openBargraphDocModal(target);
587             return;
588         }
589         if (target.type === 'ic_74hc90') {
590             openHc90DocModal(target.label);
591             return;
592         }
593         if (target.type === 'arduino_uno') {
594             openUnoDocModal(target.label);
595             return;
596         }
597         if (target.type === 'oscilloscope') {
598             closeSourcePanel();
599             openScopePanel(target);
600             return;
601         }
602         if (target.type === 'bode_analyzer') {

```

```

603         closeSourcePanel();
604         closeScopePanelFully();
605         openBodePopup(target);
606         return;
607     }
608     if (target.type === 'gimp' || target.type === 'gsin' || target.type === 'gsqr') {
609         closeScopePanelFully();
610         openSourcePanel(target);
611         return;
612     }
613     const live = flags.isSimulating;
614     if (target.type === 'resistor') {
615         let v = await showValuePrompt(`Valeur de ${target.label} :`, target.value || "1k");
616         if (v) { if (!live) saveState(); target.value = v.trim(); draw(); if (live) requestLiveSimulation(); }
617     }
618     else if (target.type === 'potentiometer') {
619         let v = await showValuePrompt(`Résistance totale de ${target.label} (ex. 10k) :`, target.value || '10k');
620         if (v) {
621             if (!live) saveState();
622             target.value = v.trim();
623             const p = await showValuePrompt(`Position du curseur (0-100 %) :`, String(target.position ?? 50));
624             if (p !== null && p.trim() !== '' && !isNaN(parseFloat(p))) {
625                 target.position = Math.min(100, Math.max(0, parseFloat(p)));
626             }
627             draw();
628             if (live) requestLiveSimulation();
629         }
630     }
631     else if (target.type === 'capacitor') {
632         let v = await showValuePrompt(`Capacité de ${target.label} (ex. 1u, 100n, 10p, 1m) :`, target.value || "1u");
633         if (v) { if (!live) saveState(); target.value = v.trim(); draw(); if (live) requestLiveSimulation(); }
634     }
635     else if (target.type === 'speaker') {
636         let v = await showValuePrompt(`Impédance de ${target.label} (Ω, ex. 8) :`, target.value || '8');
637         if (v) { if (!live) saveState(); target.value = v.trim(); draw(); if (live) requestLiveSimulation(); }
638     }
639     else if (target.type === 'inductor') {
640         let v = await showValuePrompt(`Inductance de ${target.label} (ex. 1m, 10u) :`, target.value || "1m");
641         if (v) { if (!live) saveState(); target.value = v.trim(); draw(); if (live) requestLiveSimulation(); }
642     }
643     else if (target.type === 'diode') {
644         let v = await showValuePrompt(`Modèle diode ${target.label} (ex. 1N4148) :`, target.value || "1N4148");
645         if (v) { if (!live) saveState(); target.value = v.trim(); draw(); if (live) requestLiveSimulation(); }
646     }
647     else if (target.type === 'opamp') {
648         let vp = await showValuePrompt(`Alimentation + (V) :`, String(target.vp ?? 15));
649         if (vp === null) return;
650         let vn = await showValuePrompt(`Alimentation - (V) :`, String(target.vn ?? -15));
651         if (vn === null) return;
652         if (!live) saveState();
653         target.vp = parseFloat(vp) || 15;
654         target.vn = parseFloat(vn) || -15;
655         draw();
656         if (live) requestLiveSimulation();
657     }
658     else if (['battery', 'vcc'].includes(target.type)) {
659         let v = await showValuePrompt(`Tension (Volts) :`, target.value !== undefined ? String(target.value) : "5");
660         if (v && !isNaN(parseFloat(v))) { if (!live) saveState(); target.value = parseFloat(v); draw(); if (live) requestLiveSimulation
661         (); }
662     }
663     else if (target.type === 'logic_terminal') {
664         if (live) { alert("Arrêtez la simulation."); return; }
665         let v = await showValuePrompt(`Tension Niveau Haut (Volts) :`, target.highVoltage !== undefined ? String(target.highVoltage) :
666         "5");
667         if (v && !isNaN(parseFloat(v))) { saveState(); target.highVoltage = parseFloat(v); draw(); }
668     }
669     else if (target.type === 'grove_dht22') {
670         const curT = target.temperature ?? 24;
671         const curH = target.humidity ?? 55;
672         const t = await showValuePrompt(`Température simulée ${target.label} (°C) :`, String(curT));
673         if (t === null || t.trim() === '') return;
674         const tVal = parseFloat(t);
675         if (!Number.isFinite(tVal)) return;
676         const h = await showValuePrompt(`Humidité simulée ${target.label} (%) :`, String(curH));
677         if (h === null || h.trim() === '') return;
678         const hVal = parseFloat(h);
679         if (!Number.isFinite(hVal)) return;
680         if (!live) saveState();
681         target.temperature = Math.max(-40, Math.min(80, tVal));
682         target.humidity = Math.max(0, Math.min(100, hVal));

```

```

681         draw();
682         if (live) requestLiveSimulation();
683     }
684     else if (target.type === 'push_button') {
685         const cur = target.maintained ? 'maintenu' : 'momentane';
686         let v = await showValuePrompt(
687             `Mode de ${target.label} – "momentane" (fermé pendant l'appui) ou "maintenu" (1er clic ferme, 2e clic ouvre) :`,
688             cur
689         );
690         if (v) {
691             if (!live) saveState();
692             target.maintained = v.trim().toLowerCase().startsWith('maint');
693             target.state = 0;
694             heldPushButton = null;
695             draw();
696             if (live) requestLiveSimulation();
697         }
698     }
699 }
700 });
701
702 canvas.addEventListener('wheel', (e) => {
703     e.preventDefault(); const factor = 1.1, mousePos = toGridCoords(e.clientX, e.clientY);
704     if (e.deltaY < 0) scale.value *= factor; else scale.value /= factor; scale.value = Math.max(0.3, Math.min(scale.value, 4));
705     const rect = canvas.getBoundingClientRect(); pan.x = e.clientX - rect.left - mousePos.x * scale.value; pan.y = e.clientY - rect.top - mousePos.y * scale.value; draw();
706 }, { passive: false });
707
708 canvas.addEventListener('contextmenu', e => e.preventDefault());
709
710 // --- DRAG & DROP DEPUIS MENU ---
711 function finishMenuDrag() {
712     flags.isDraggingFromMenu = false;
713     menuDrag.draggedComponentType = null;
714     draw();
715 }
716
717 function onMenuDragOver(e) {
718     e.preventDefault();
719     if (e.dataTransfer) e.dataTransfer.dropEffect = 'copy';
720     if (!flags.isDraggingFromMenu || !menuDrag.draggedComponentType) return;
721     const gp = toGridCoords(e.clientX, e.clientY);
722     menuDrag.x = gp.x;
723     menuDrag.y = gp.y;
724     draw();
725 }
726
727 function applyNewComponentDefaults(nc, type) {
728     if (type === 'gimp') {
729         nc.frequency = 2;
730         nc.dutyCycle = 50;
731         nc.voltageRail = 5;
732         nc.flipX = false;
733     } else if (type === 'potentiometer') {
734         nc.value = '10k';
735         nc.position = 50;
736     } else if (type === 'switch_spdt') {
737         nc.state = 0;
738     } else if (type === 'push_button') {
739         nc.state = 0;
740         nc.maintained = false;
741     } else if (type === 'speaker') {
742         nc.value = '8';
743     } else if (type === 'gsin') {
744         nc.peakAmplitude = 5;
745         nc.frequency = 440;
746         nc.offset = 0;
747     } else if (type === 'gsqr') {
748         nc.peakAmplitude = 5;
749         nc.frequency = 1000;
750         nc.offset = 0;
751     } else if (type === 'capacitor') {
752         nc.value = '1u';
753     } else if (type === 'inductor') {
754         nc.value = '1m';
755     } else if (type === 'diode') {
756         nc.value = '1N4148';
757     } else if (type === 'npn') {
758         nc.value = '2N2222';
759         nc.flipX = false;

```



```

760     } else if (type === 'opamp') {
761         nc.value = 'uA741';
762         nc.vp = 15;
763         nc.vn = -15;
764         nc.flipX = false;
765         nc.flipY = false;
766     } else if (type === 'oscilloscope') {
767         nc.timeDivSec = 0.001;
768         nc.ch1VoltsPerDiv = 1;
769         nc.ch2VoltsPerDiv = 1;
770         nc.ch1PositionDiv = 0;
771         nc.ch2PositionDiv = 0;
772         nc.timePositionDiv = 0;
773     } else if (type === 'arduino_uno') {
774         nc.sketch = DEFAULT_ARDUINO_SKETCH;
775         nc.fqbn = 'arduino:avr:uno';
776         nc.pinModes = {};
777         nc.pinLevels = {};
778         nc.avrRegisters = null;
779         nc.lastCompileOk = null;
780         nc.lastCompileLog = '';
781     } else if (type === 'grove_lcd16x2') {
782         nc.i2cAddress = 0x3e;
783         nc.flipX = false;
784     } else if (type === 'bargraph_dc10h') {
785         nc.barColor = 'red';
786         nc.flipX = false;
787     } else if (type === 'grove_dht22') {
788         nc.flipX = false;
789         nc.temperature = 24;
790         nc.humidity = 55;
791     }
792 }
793
794 function onMenuDrop(e) {
795     e.preventDefault();
796     const type = menuDrag.draggedComponentType
797         || e.dataTransfer?.getData('text/plain')
798         || '';
799     if (!type) return;
800     if (flags.isSimulating) {
801         alert('Arrêtez la simulation pour ajouter un composant.');
```

```

840 .replace(/[\\u{2190}-\\u{21FF}\\u{2300}-\\u{23FF}\\u{2460}-\\u{24FF}\\u{25A0}-\\u{25FF}]/gu, '')
841 .replace(/\\s+/g, ' ')
842 .trim();
843 }
844
845 function normalizeSubmenuLabels() {
846   document.querySelectorAll('.dropdown .dropdown-item').forEach((el) => {
847     const label = el.getAttribute('data-label') || el.textContent;
848     const clean = stripMenuDecorations(label);
849     if (clean) el.textContent = clean;
850   });
851 }
852
853 // --- CHARGEMENT INITIAL ---
854 function initApp() {
855   initModalUi();
856   initEditorTheme();
857   document.getElementById('btn-theme-dark')?.addEventListener('click', () => { setEditorTheme('dark'); draw(); });
858   document.getElementById('btn-theme-light')?.addEventListener('click', () => { setEditorTheme('light'); draw(); });
859   normalizeSubmenuLabels();
860   ensureAllCounters();
861   setCircuitDisplayName('Sans titre');
862   resizeCanvas(); window.addEventListener('resize', resizeCanvas);
863   document.querySelectorAll('.dropdown-item[draggable=true]').forEach(item => {
864     item.addEventListener('dragstart', (e) => {
865       if (flags.isSimulating) { alert("Arrêtez la simulation pour ajouter."); e.preventDefault(); return; }
866       menuDrag.draggedComponentType = e.currentTarget.getAttribute('data-component');
867       flags.isDraggingFromMenu = true;
868       if (e.dataTransfer) {
869         e.dataTransfer.effectAllowed = 'copy';
870         e.dataTransfer.setData('text/plain', menuDrag.draggedComponentType);
871         e.dataTransfer.setDragImage(emptyDragImage, 0, 0);
872       }
873     });
874     item.addEventListener('dragend', finishMenuDrag);
875   });
876   document.getElementById('menu-serial-monitor')?.addEventListener('pointerdown', (e) => {
877     if (e.button !== 0) return;
878     e.preventDefault();
879     e.stopPropagation();
880     document.querySelectorAll('.navbar > .menu-item.open').forEach((m) => m.classList.remove('open'));
881     openSerialMonitor(getActiveArduinoBoard()?.label);
882   });
883   document.getElementById('menu-arduino-editor')?.addEventListener('pointerdown', (e) => {
884     if (e.button !== 0) return;
885     e.preventDefault();
886     e.stopPropagation();
887     document.querySelectorAll('.navbar > .menu-item.open').forEach((m) => m.classList.remove('open'));
888     openArduinoEditorForCircuit();
889   });
890   document.getElementById('arduino-btn-serial-monitor')?.addEventListener('click', () => {
891     openSerialMonitor(getActiveArduinoBoard()?.label);
892   });
893   document.querySelectorAll('.dropdown-submenu .submenu-title').forEach(title => {
894     title.addEventListener('click', (e) => {
895       e.stopPropagation();
896       const sub = title.closest('.dropdown-submenu');
897       const wasOpen = sub.classList.contains('open');
898       document.querySelectorAll('.dropdown-submenu.open').forEach(s => s.classList.remove('open'));
899       if (!wasOpen) sub.classList.add('open');
900     });
901   });
902   document.querySelectorAll('.dropdown-submenu .submenu').forEach(sub => {
903     sub.addEventListener('click', (e) => e.stopPropagation());
904   });
905   document.querySelectorAll('.navbar > .menu-item').forEach((item) => {
906     if (!item.querySelector('.dropdown')) return;
907     item.addEventListener('pointerdown', (e) => {
908       if (e.button !== 0) return;
909       if (e.target.closest('.dropdown-item')) return;
910       e.stopPropagation();
911       const wasOpen = item.classList.contains('open');
912       document.querySelectorAll('.navbar > .menu-item.open').forEach((m) => m.classList.remove('open'));
913       if (!wasOpen) item.classList.add('open');
914     });
915     item.querySelector('.dropdown')?.addEventListener('pointerdown', (e) => e.stopPropagation());
916   });
917   document.addEventListener('pointerdown', (e) => {
918     if (e.target.closest('.navbar')) return;
919     document.querySelectorAll('.navbar > .menu-item.open').forEach((m) => m.classList.remove('open'));

```

```

920     document.querySelectorAll('.dropdown-submenu.open').forEach(s => s.classList.remove('open'));
921   });
922   document.getElementById('btn-new').addEventListener('click', () => {
923     if (circuit.components.length > 0 && confirm("Tout effacer ?")) {
924       saveState();
925       circuit.components = [];
926       circuit.wires = [];
927       circuit.autoJunctions = [];
928       Object.keys(counters).forEach(k => counters[k] = 0);
929       fileHandle = null;
930       setCircuitDisplayName('Sans titre');
931       stopSimulation();
932     }
933   });
934   document.getElementById('btn-open').addEventListener('click', () => {
935     document.querySelectorAll('.navbar > .menu-item.open').forEach(m => m.classList.remove('open'));
936     openFile();
937   }); document.getElementById('btn-save').addEventListener('click', saveFile); document.getElementById('btn-save-as').addEventListener('click', saveAs);
938   document.getElementById('btn-simulate').addEventListener('click', () => { if (!flags.isSimulating) triggerSimulation(); }); document.getElementById('btn-stop').addEventListener('click', stopSimulation);
939   initSourcePanel();
940   initValuePrompt();
941   initArduinoEditor();
942   initScopePanel();
943   initScopePopup();
944   initSerialMonitor();
945   initArduinoLibPopup();
946   initBodePopup();
947   setScopePopupCloseCallback(onScopePopupClosed);
948   setScopeViewChangeCallback(() => refreshScopePanelFields());
949   document.getElementById('source-panel-close')?.addEventListener('click', () => {
950     closeSourcePanel();
951     closeScopePanelFully();
952   });
953   bindLedAnimationRedraw(() => {
954     draw();
955     refreshSerialMonitor();
956   });
957   bindSpeakerAudioRedraw(draw);
958   bindScopeAnimationRedraw(draw);
959   bindScopePopupRedraw(refreshScopePopup);
960   const m = document.getElementById('commands-modal');
961   document.getElementById('btn-commands')?.addEventListener('click', () => showModal(m));
962   document.getElementById('close-commands')?.addEventListener('click', () => hideModal(m));
963   window.addEventListener('click', (e) => { if (e.target === m) hideModal(m); });
964   const cdDoc = document.getElementById('cd4511-doc-modal');
965   document.getElementById('close-cd4511-doc')?.addEventListener('click', closeCd4511DocModal);
966   window.addEventListener('click', (e) => { if (e.target === cdDoc) closeCd4511DocModal(); });
967   const hc90Doc = document.getElementById('hc90-doc-modal');
968   document.getElementById('close-hc90-doc')?.addEventListener('click', closeHc90DocModal);
969   window.addEventListener('click', (e) => { if (e.target === hc90Doc) closeHc90DocModal(); });
970   const unoDoc = document.getElementById('uno-doc-modal');
971   document.getElementById('close-uno-doc')?.addEventListener('click', closeUnoDocModal);
972   window.addEventListener('click', (e) => { if (e.target === unoDoc) closeUnoDocModal(); });
973   const bargraphDoc = document.getElementById('bargraph-doc-modal');
974   document.getElementById('close-bargraph-doc')?.addEventListener('click', closeBargraphDocModal);
975   window.addEventListener('click', (e) => { if (e.target === bargraphDoc) closeBargraphDocModal(); });
976 }
977
978 if (document.readyState === 'loading') {
979   document.addEventListener('DOMContentLoaded', initApp);
980 } else {
981   initApp();
982 }

```

```

1  /** Panneau latéral droit - éditeur sketch Arduino + compilation arduino-cli. */
2  import { saveState, circuit, flags } from './state.js';
3  import { draw, resizeCanvas } from './renderer.js';
4  import { onArduinoSketchUpdated } from './led-animation.js';
5  import { refreshGroveLcdDisplayCache } from './Engine/grove-lcd-ideal.mjs';
6  import { DEFAULT_ARDUINO_SKETCH } from './arduino-uno-layout.js';
7  import { applyArduinoSketchToComponent } from './Engine/arduino-sketch-parse.mjs';
8  import { registerArduinoSketchSync, syncArduinoSketchesFromEditor } from './arduino-sketch-sync.js';
9  import { openArduinoLibPopup } from './arduino-lib-popup.js';
10 import { initArduinoSyntaxHighlight, refreshArduinoSyntaxHighlight } from './arduino-syntax-highlight.js';
11 import { showModal } from './modal-ui.js';
12
13 let activeBoard = null;
14
15 const panel = () => document.getElementById('arduino-panel');
16 const sketchEl = () => document.getElementById('arduino-sketch-input');
17 const logEl = () => document.getElementById('arduino-compile-log');
18 const compileStatusEl = () => document.getElementById('arduino-compile-status');
19 const showErrorsBtn = () => document.getElementById('arduino-btn-show-errors');
20 const titleEl = () => document.getElementById('arduino-panel-title');
21 const statusEl = () => document.getElementById('arduino-cli-status');
22 const libSearchEl = () => document.getElementById('arduino-lib-search');
23 const libSearchResultsEl = () => document.getElementById('arduino-lib-search-results');
24 const libInstalledEl = () => document.getElementById('arduino-lib-installed');
25 const libMsgEl = () => document.getElementById('arduino-lib-msg');
26
27 function resolveApiBaseUrl() {
28   const { protocol, pathname, origin } = window.location;
29   if (protocol === 'file:') return 'http://127.0.0.1:43721';
30   if (pathname.startsWith('/Simulateur')) return origin;
31   return origin;
32 }
33
34 function setLibMsg(text, kind = '') {
35   const el = libMsgEl();
36   if (!el) return;
37   el.textContent = text || '';
38   el.className = 'arduino-lib-msg' + (kind ? ` arduino-lib-msg--${kind}` : '');
39 }
40
41 function setCompileLogVisible(visible) {
42   const log = logEl();
43   const btn = showErrorsBtn();
44   if (log) log.classList.toggle('hidden', !visible);
45   if (btn) btn.textContent = visible ? 'Masquer les erreurs' : 'Voir les erreurs';
46 }
47
48 function updateCompileStatusUi(board) {
49   const status = compileStatusEl();
50   const btn = showErrorsBtn();
51   const log = logEl();
52   if (!status || !btn) return;
53
54   if (!board) {
55     status.textContent = '';
56     status.className = 'arduino-compile-status';
57     btn.classList.add('hidden');
58     if (log) {
59       log.textContent = '';
60       log.classList.add('hidden');
61     }
62     return;
63   }
64
65   if (board.lastCompileOk) {
66     status.textContent = 'OK';
67     status.className = 'arduino-compile-status arduino-compile-status--ok';
68     btn.classList.add('hidden');
69     setCompileLogVisible(false);
70     return;
71   }
72
73   status.textContent = '';
74   status.className = 'arduino-compile-status';
75   const hasLog = !(board.lastCompileLog || '').trim();
76   if (hasLog) {
77     btn.classList.remove('hidden');
78     if (log) log.textContent = board.lastCompileLog;

```

```

79     setCompileLogVisible(false);
80 } else {
81     btn.classList.add('hidden');
82     if (log) log.classList.add('hidden');
83 }
84 }
85
86 function escapeHtml(s) {
87     return String(s || '')
88         .replace(/&/g, '&amp;')
89         .replace(/</g, '&lt;')
90         .replace(/>/g, '&gt;')
91         .replace(/"/g, '&quot;');
92 }
93
94 function renderLibraryItem(lib, mode) {
95     const li = document.createElement('li');
96     li.className = 'arduino-lib-item';
97     const meta = [lib.author, lib.version].filter(Boolean).join(' · ');
98     const desc = lib.description ? `<div class="arduino-lib-item-meta">${escapeHtml(lib.description)}</div>` : '';
99     const btnClass = mode === 'install' ? 'arduino-btn--install' : 'arduino-btn--uninstall';
100    const btnLabel = mode === 'install' ? 'Installer' : 'Retirer';
101    const btn = mode === 'local'
102        ? `<span class="arduino-lib-item-meta">local</span>`
103        : `<button type="button" class="arduino-btn arduino-btn--small ${btnClass}" data-lib-name="${escapeHtml(lib.name)}">${btnLabel}</butt
on>`;
104    li.innerHTML = `
105        <div class="arduino-lib-item-info">
106            <button type="button" class="arduino-lib-item-name" title="Voir les commandes">${escapeHtml(lib.name)}</button>
107            ${meta ? `<div class="arduino-lib-item-meta">${escapeHtml(meta)}</div>` : ''}
108            ${desc}
109        </div>
110        ${btn}
111    `;
112    li.querySelector('.arduino-lib-item-name')?.addEventListener('click', (e) => {
113        e.stopPropagation();
114        openArduinoLibPopup(lib.name);
115    });
116    if (mode !== 'local') {
117        li.querySelector('button')?.addEventListener('click', () => {
118            if (mode === 'install') installLibraryByName(lib.name);
119            else uninstallLibraryByName(lib.name);
120        });
121    }
122    return li;
123 }
124
125 export function isArduinoPanelOpen() {
126     const p = panel();
127     return p && !p.classList.contains('hidden');
128 }
129
130 export function getArduinoPanelWidth() {
131     const p = panel();
132     return isArduinoPanelOpen() && p ? p.offsetWidth : 0;
133 }
134
135 export async function refreshInstalledLibraries() {
136     const list = libInstalledEl();
137     if (!list) return;
138     list.innerHTML = '';
139     setLibMsg('Chargement des bibliothèques...');
140     try {
141         const base = resolveApiBaseUrl();
142         const r = await fetch(`${base}/api/arduino/libraries`);
143         const data = await r.json();
144         const libs = Array.isArray(data.libraries) ? data.libraries : [];
145         if (libs.length === 0) {
146             setLibMsg('Aucune bibliothèque installée. Recherchez « LiquidCrystal I2C » ci-dessus.', '');
147         } else {
148             setLibMsg(`${libs.length} bibliothèque(s) disponible(s).`, 'ok');
149         }
150         for (const lib of libs) {
151             const mode = lib.source === 'local' ? 'local' : 'uninstall';
152             list.appendChild(renderLibraryItem(lib, mode));
153         }
154         await refreshArduinoCliStatus();
155     } catch (err) {
156         setLibMsg(err?.message || 'Impossible de charger les bibliothèques.', 'err');
157     }

```

```

158 }
159
160 export async function searchLibraries(query) {
161   const q = String(query || '').trim();
162   const list = libSearchResultsEl();
163   if (!list) return;
164   list.innerHTML = '';
165   if (!q) {
166     setLibMsg('Saisissez un nom à rechercher.', 'err');
167     return;
168   }
169   setLibMsg(`Recherche « ${q} »...`);
170   try {
171     const base = resolveApiBaseUrl();
172     const r = await fetch(`${base}/api/arduino/lib/search?q=${encodeURIComponent(q)}`);
173     const data = await r.json();
174     const libs = Array.isArray(data.libraries) ? data.libraries : [];
175     if (!data.ok) {
176       setLibMsg((data.errors || []).join(' ') || 'Recherche échouée.', 'err');
177       return;
178     }
179     if (libs.length === 0) {
180       setLibMsg('Aucun résultat. Essayez « LiquidCrystal » ou mettez à jour l\'index (U).', 'err');
181       return;
182     }
183     setLibMsg(`${libs.length} résultat(s).`, 'ok');
184     for (const lib of libs.slice(0, 12)) {
185       list.appendChild(renderLibraryItem(lib, 'install'));
186     }
187   } catch (err) {
188     setLibMsg(err?.message || 'Recherche impossible.', 'err');
189   }
190 }
191
192 export async function installLibraryByName(name) {
193   const libName = String(name || '').trim();
194   if (!libName) return;
195   setLibMsg(`Installation de « ${libName} »...`);
196   try {
197     const base = resolveApiBaseUrl();
198     const r = await fetch(`${base}/api/arduino/lib/install`, {
199       method: 'POST',
200       headers: { 'Content-Type': 'application/json' },
201       body: JSON.stringify({ name: libName }),
202     });
203     const data = await r.json();
204     if (!data.ok) {
205       setLibMsg((data.errors || []).join('\n') || data.log || 'Installation échouée.', 'err');
206       return;
207     }
208     setLibMsg(`« ${libName} » installée.`, 'ok');
209     libSearchResultsEl()?.replaceChildren();
210     await refreshInstalledLibraries();
211   } catch (err) {
212     setLibMsg(err?.message || 'Installation impossible.', 'err');
213   }
214 }
215
216 export async function uninstallLibraryByName(name) {
217   const libName = String(name || '').trim();
218   if (!libName) return;
219   if (!confirm(`Retirer la bibliothèque « ${libName} » ?`)) return;
220   setLibMsg(`Retrait de « ${libName} »...`);
221   try {
222     const base = resolveApiBaseUrl();
223     const r = await fetch(`${base}/api/arduino/lib/uninstall`, {
224       method: 'POST',
225       headers: { 'Content-Type': 'application/json' },
226       body: JSON.stringify({ name: libName }),
227     });
228     const data = await r.json();
229     if (!data.ok) {
230       setLibMsg((data.errors || []).join('\n') || data.log || 'Retrait échoué.', 'err');
231       return;
232     }
233     setLibMsg(`« ${libName} » retirée.`, 'ok');
234     await refreshInstalledLibraries();
235   } catch (err) {
236     setLibMsg(err?.message || 'Retrait impossible.', 'err');
237   }

```

```

238 }
239
240 export function getActiveArduinoBoard() {
241     return activeBoard;
242 }
243
244 export function openArduinoEditor(comp) {
245     if (!comp || comp.type !== 'arduino_uno') return;
246     activeBoard = comp;
247     if (titleEl()) titleEl().textContent = `${comp.label} - Arduino UNO`;
248     if (sketchEl()) sketchEl().value = comp.sketch || DEFAULT_ARDUINO_SKETCH;
249     refreshArduinoSyntaxHighlight();
250     applyArduinoSketchToComponent(comp);
251     updateCompileStatusUi(comp);
252     panel()?.classList.remove('hidden');
253     refreshArduinoCliStatus();
254     refreshInstalledLibraries();
255     resizeCanvas();
256     requestAnimationFrame(() => sketchEl()?.focus({ preventScroll: true }));
257 }
258
259 /** Ouvre l'éditeur sur la première carte UNO du schéma (ou la carte déjà active). */
260 export function openArduinoEditorForCircuit() {
261     if (activeBoard && circuit.components.includes(activeBoard)) {
262         openArduinoEditor(activeBoard);
263         return true;
264     }
265     const uno = circuit.components.find((c) => c.type === 'arduino_uno');
266     if (!uno) {
267         alert('Ajoutez d'abord une carte Arduino UNO au schéma (menu Arduino).');
268         return false;
269     }
270     openArduinoEditor(uno);
271     return true;
272 }
273
274 export function closeArduinoEditor() {
275     applySketchToBoard();
276     panel()?.classList.add('hidden');
277     activeBoard = null;
278     resizeCanvas();
279 }
280
281 function applySketchToBoard() {
282     if (!activeBoard || !sketchEl() || !circuit.components.includes(activeBoard)) return;
283     activeBoard.sketch = sketchEl().value;
284 }
285
286 /** Vide l'éditeur sans réécrire le textarea dans un composant détaché (chargement JSON). */
287 export function resetArduinoEditorOnCircuitLoad() {
288     activeBoard = null;
289     panel()?.classList.add('hidden');
290     if (sketchEl()) sketchEl().value = '';
291     updateCompileStatusUi(null);
292     resizeCanvas();
293 }
294
295 /** Enregistre le sketch visible dans le composant UNO actif avant export fichier. */
296 export function flushArduinoSketchesBeforeSave() {
297     applySketchToBoard();
298 }
299
300 export async function refreshArduinoCliStatus() {
301     const el = statusEl();
302     if (!el) return;
303     el.textContent = 'Vérification arduino-cli...';
304     try {
305         const base = resolveApiBaseUrl();
306         const r = await fetch(`${base}/api/arduino/status`);
307         const data = await r.json();
308         if (data.ok && data.version) {
309             el.textContent = `arduino-cli OK - ${String(data.version).split('\n')[0]}`;
310             el.className = 'arduino-cli-status arduino-cli-status--ok';
311         } else {
312             el.textContent = data.hint || 'arduino-cli introuvable sur ce serveur.';
313             el.className = 'arduino-cli-status arduino-cli-status--warn';
314         }
315     } catch {
316         el.textContent = 'Serveur Arduino injoignable (lancez npm start ou Simulateur H).';
317         el.className = 'arduino-cli-status arduino-cli-status--warn';
318     }
319 }

```

```

318     }
319 }
320
321 export async function compileActiveSketch() {
322     if (!activeBoard) return;
323     applySketchToBoard();
324     saveState();
325     const log = logEl();
326     const status = compileStatusEl();
327     const btn = showErrorsBtn();
328     if (status) {
329         status.textContent = 'Compilation...';
330         status.className = 'arduino-compile-status arduino-compile-status--busy';
331     }
332     if (btn) btn.classList.add('hidden');
333     if (log) {
334         log.textContent = 'Compilation en cours...';
335         log.classList.add('hidden');
336     }
337     const base = resolveApiBaseUrl();
338     try {
339         const r = await fetch(`${base}/api/arduino/compile`, {
340             method: 'POST',
341             headers: { 'Content-Type': 'application/json' },
342             body: JSON.stringify({
343                 sketch: activeBoard.sketch,
344                 sketchName: activeBoard.label,
345                 fqbn: activeBoard.fqbn || 'arduino:avr:uno',
346             }),
347         });
348         const data = await r.json();
349         activeBoard.lastCompileOk = !!data.ok;
350         activeBoard.lastCompileLog = data.log || (data.errors || []).join('\n');
351         if (data.ok) {
352             applyArduinoSketchToComponent(activeBoard);
353             onArduinoSketchUpdated();
354             if (flags.isSimulating) {
355                 import('./simulation.js').then((m) => m.requestLiveSimulation());
356             }
357         }
358         if (log) log.textContent = activeBoard.lastCompileLog;
359         updateCompileStatusUi(activeBoard);
360         draw();
361     } catch (err) {
362         activeBoard.lastCompileOk = false;
363         activeBoard.lastCompileLog = err?.message || String(err);
364         if (log) log.textContent = activeBoard.lastCompileLog;
365         updateCompileStatusUi(activeBoard);
366         draw();
367     }
368 }
369
370 export function prepareArduinoForSimulation() {
371     syncArduinoSketchesFromEditor();
372     for (const comp of circuit.components) {
373         if (comp.type === 'arduino_uno') applyArduinoSketchToComponent(comp);
374     }
375     refreshGrovelCircuitCache(circuit.components, circuit.wires, circuit.autoJunctions);
376 }
377
378 export function initArduinoEditor() {
379     registerArduinoSketchSync(() => activeBoard);
380     document.getElementById('arduino-panel-close')?.addEventListener('click', () => {
381         saveState();
382         closeArduinoEditor();
383         draw();
384     });
385     document.getElementById('arduino-panel-doc')?.addEventListener('click', () => {
386         if (!activeBoard) return;
387         const modal = document.getElementById('uno-doc-modal');
388         const title = document.getElementById('uno-doc-title');
389         if (title) title.textContent = `Arduino UNO R3 - ${activeBoard.label}`;
390         showModal(modal);
391     });
392     document.getElementById('arduino-btn-compile')?.addEventListener('click', () => {
393         compileActiveSketch();
394     });
395     document.getElementById('arduino-btn-show-errors')?.addEventListener('click', () => {
396         const log = logEl();
397         if (!log) return;

```



```

398     setCompileLogVisible(log.classList.contains('hidden'));
399   });
400   document.getElementById('arduino-lib-search-btn')?.addEventListener('click', () => {
401     searchLibraries(libSearchEl()?.value || '');
402   });
403   libSearchEl()?.addEventListener('keydown', (e) => {
404     if (e.key === 'Enter') {
405       e.preventDefault();
406       searchLibraries(libSearchEl()?.value || '');
407     }
408   });
409   document.getElementById('arduino-lib-refresh')?.addEventListener('click', async () => {
410     setLibMsg('Mise à jour de l\'index Library Manager...');
411     try {
412       const base = resolveApiBaseUrl();
413       const r = await fetch(`${base}/api/arduino/lib/update-index`, { method: 'POST' });
414       const data = await r.json();
415       if (!data.ok) {
416         setLibMsg((data.errors || []).join(' ') || 'Échec.', 'err');
417       } else {
418         setLibMsg('Index à jour.', 'ok');
419       }
420       await refreshInstalledLibraries();
421     } catch (err) {
422       setLibMsg(err?.message || 'Échec.', 'err');
423     }
424   });
425   sketchEl()?.addEventListener('keydown', (e) => {
426     e.stopPropagation();
427     if (e.key === 'Tab') {
428       e.preventDefault();
429       const ta = sketchEl();
430       if (!ta) return;
431       const start = ta.selectionStart;
432       const end = ta.selectionEnd;
433       const val = ta.value;
434       ta.value = `${val.slice(0, start)} ${val.slice(end)} `;
435       ta.selectionStart = ta.selectionEnd = start + 4;
436       ta.dispatchEvent(new Event('input', { bubbles: true }));
437     }
438   });
439   document.querySelector('.arduino-sketch-wrap')?.addEventListener('mousedown', (e) => {
440     if (e.target === sketchEl()) return;
441     e.preventDefault();
442     sketchEl()?.focus({ preventScroll: true });
443   });
444   let sketchDrawTimer = 0;
445   sketchEl()?.addEventListener('input', () => {
446     applySketchToBoard();
447     if (activeBoard) applyArduinoSketchToComponent(activeBoard);
448     refreshArduinoSyntaxHighlight();
449     clearTimeout(sketchDrawTimer);
450     sketchDrawTimer = setTimeout(() => draw(), 200);
451   });
452   sketchEl()?.addEventListener('blur', () => {
453     applySketchToBoard();
454     applyArduinoSketchToComponent(activeBoard);
455     saveState();
456   });
457   initArduinoSyntaxHighlight();
458 }
459
460 export function onArduinoBoardRemoved(comp) {
461   if (activeBoard && comp === activeBoard) closeArduinoEditor();
462 }
463

```

```

1  /**
2   * Référence des commandes par bibliothèque Arduino – affichée dans la fenêtre flottante.
3   * simSupported : interprété par le simulateur (LCD, GPIO, délais...).
4   */
5
6  /** @typedef {{ sig: string; desc: string; simSupported?: boolean }} LibCommand */
7
8  /** @type {{ id: string; match: RegExp; title: string; header?: string; ctor?: string; note?: string; commands: LibCommand[] }} */
9  export const ARDUINO_LIB_DOCS = [
10   {
11     id: "liquidcrystal_i2c",
12     match: /liquid\s*crystal/i,
13     title: "LiquidCrystal I2C",
14     header: "#include <LiquidCrystal_I2C.h>",
15     ctor: "LiquidCrystal_I2C lcd(0x3E, 16, 2); // adresse Grove Seed",
16     note: "Compatible Grove LCD 16x2 (PCF8574). Relier SDA→A4, SCL→A5, 5 V et GND.",
17     commands: [
18       { sig: "lcd.init()", desc: "Initialise l’afficheur HD44780 (4 bits via I²C).", simSupported: true },
19       { sig: "lcd.begin(cols, rows)", desc: "Alias de init() avec taille optionnelle.", simSupported: true },
20       { sig: "lcd.backlight()", desc: "Rétroéclairage ON.", simSupported: true },
21       { sig: "lcd.noBacklight()", desc: "Rétroéclairage OFF.", simSupported: true },
22       { sig: "lcd.clear()", desc: "Efface l’écran.", simSupported: true },
23       { sig: "lcd.home()", desc: "Curseur en (0, 0).", simSupported: true },
24       { sig: "lcd.setCursor(col, row)", desc: "Position du curseur (colonne, ligne).", simSupported: true },
25       { sig: "lcd.print('texte')", desc: "Affiche une chaîne ou un nombre.", simSupported: true },
26       { sig: "lcd.print(F('texte'))", desc: "Chaîne en mémoire programme (FLASH).", simSupported: true },
27       { sig: "lcd.write(byte)", desc: "Écrit un caractère (code ASCII).", simSupported: true },
28       { sig: "delay(ms)", desc: "Pause en millisecondes (animation LCD en simulation).", simSupported: true },
29       { sig: "lcd.scrollDisplayLeft()", desc: "Décale l’affichage d’une colonne vers la gauche.", simSupported: true },
30       { sig: "lcd.scrollDisplayRight()", desc: "Décale l’affichage d’une colonne vers la droite.", simSupported: true },
31       { sig: "lcd.cursor() / noCursor()", desc: "Curseur clignotant.", simSupported: false },
32       { sig: "lcd.blink() / noBlink()", desc: "Clignotement du curseur.", simSupported: false },
33       { sig: "lcd.createChar(id, data[])", desc: "Caractère personnalisé CGRAM.", simSupported: false },
34     ],
35   },
36   {
37     id: "wire",
38     match: /^wire$/i,
39     title: "Wire (I²C)",
40     header: "#include <Wire.h>",
41     note: "Requis pour la compilation LiquidCrystal_I2C. Le bus I²C est modélisé automatiquement si le LCD Grove est câblé.",
42     commands: [
43       { sig: "Wire.begin()", desc: "Initialise le maître I²C (UNO : A4=SDA, A5=SCL).", simSupported: false },
44       { sig: "Wire.beginTransmission(addr)", desc: "Début de transmission vers l’esclave.", simSupported: false },
45       { sig: "Wire.write(data)", desc: "Octet à envoyer.", simSupported: false },
46       { sig: "Wire.endTransmission()", desc: "Fin de transmission.", simSupported: false },
47       { sig: "Wire.requestFrom(addr, n)", desc: "Lecture de n octets.", simSupported: false },
48       { sig: "Wire.read()", desc: "Lit un octet reçu.", simSupported: false },
49     ],
50   },
51   {
52     id: "dht_sensor",
53     match: /\bDHT\b|dht\.h/i,
54     title: "DHT sensor library",
55     header: "#include <DHT.h>",
56     ctor: "#define DHTPIN 2\n#define DHTTYPE DHT22\nDHT dht(DHTPIN, DHTTYPE);",
57     note: "Grove DHT22 : DATA→broche digitale UNO, VCC→5V, GND. La broche NC n’est pas utilisée.",
58     commands: [
59       { sig: "dht.begin()", desc: "Initialise le capteur.", simSupported: true },
60       { sig: "dht.readTemperature()", desc: "Température en °C (simulation).", simSupported: true },
61       { sig: "dht.readHumidity()", desc: "Humidité relative en % (simulation).", simSupported: true },
62       { sig: "dht.read()", desc: "Lecture brute (non simulée).", simSupported: false },
63     ],
64   },
65   {
66     id: "grove_rgb_lcd",
67     match: /grove.*lcd.*rgb|rgb_lcd/i,
68     title: "Grove – LCD RGB Backlight",
69     header: "#include <rgb_lcd.h>",
70     ctor: "rgb_lcd lcd;",
71     note: "Bibliothèque Seeed pour l’écran RGB (même câblage I²C que le Grove 16x2).",
72     commands: [
73       { sig: "lcd.begin(cols, rows, charsize)", desc: "Initialisation.", simSupported: true },
74       { sig: "lcd.setRGB(r, g, b)", desc: "Couleur du rétroéclairage RGB (0-255).", simSupported: true },
75       { sig: "lcd.scrollDisplayLeft()", desc: "Décale l’affichage d’une colonne vers la gauche.", simSupported: true },
76       { sig: "lcd.scrollDisplayRight()", desc: "Décale l’affichage d’une colonne vers la droite.", simSupported: true },
77       { sig: "lcd.setCursor / print / clear", desc: "Comme LiquidCrystal (texte 16x2).", simSupported: true },
78     ],
79   },
80 ]

```

```

79     },
80     {
81         id: "serial_uart",
82         match: /\bserial\b/i,
83         title: "Serial (UART)",
84         note: "UART matériel UNO : TX = broche D1, RX = broche D0. Menu Mesures → Moniteur série.",
85         commands: [
86             { sig: "Serial.begin(baud)", desc: "Initialise le port série (ex. 9600).", simSupported: true },
87             { sig: "Serial.print('texte')", desc: "Écrit dans le moniteur série (sans retour ligne).", simSupported: true },
88             { sig: "Serial.println(val)", desc: "Écrit avec saut de ligne.", simSupported: true },
89             { sig: "Serial.write(byte)", desc: "Envoie un octet brut.", simSupported: true },
90             { sig: "Serial.available()", desc: "Nombre d'octets reçus sur RX (D0).", simSupported: true },
91             { sig: "Serial.read()", desc: "Lit un octet (-1 si aucune donnée).", simSupported: true },
92         ],
93     },
94     {
95         id: "arduino_core",
96         match: /^(arduino|avr|standard)$/i,
97         title: "Arduino (noyau AVR)",
98         note: "Fonctions de base du sketch – sorties GPIO simulées sur broches digitales.",
99         commands: [
100             { sig: "pinMode(pin, OUTPUT)", desc: "Configure une broche en sortie.", simSupported: true },
101             { sig: "pinMode(pin, INPUT)", desc: "Configure une broche en entrée.", simSupported: true },
102             { sig: "digitalWrite(pin, HIGH|LOW)", desc: "Niveau logique sur une sortie.", simSupported: true },
103             { sig: "digitalRead(pin)", desc: "Lit une entrée digitale (boutons, interrupteurs).", simSupported: true },
104             { sig: "analogRead(pin)", desc: "Lit une entrée analogique A0-A5 (0-1023, tension du circuit).", simSupported: true },
105             { sig: "analogWrite(pin, val)", desc: "PWM – partiel selon broche.", simSupported: false },
106             { sig: "delay(ms)", desc: "Pause bloquante.", simSupported: true },
107             { sig: "delayMicroseconds(us)", desc: "Pause courte.", simSupported: false },
108             { sig: "millis() / micros()", desc: "Temps écoulé depuis le démarrage.", simSupported: false },
109         ],
110     },
111 ];
112
113 /**
114  * @param {string} libName
115  * @returns {typeof ARDUINO_LIB_DOCS[0] | null}
116  */
117 export function resolveArduinoLibDoc(libName) {
118     const n = String(libName || "").trim();
119     if (!n) return null;
120     for (const doc of ARDUINO_LIB_DOCS) {
121         if (doc.match.test(n)) return doc;
122     }
123     if (/i2c/i.test(n) && /lcd/i.test(n)) {
124         return ARDUINO_LIB_DOCS.find((d) => d.id === "liquidcrystal_i2c") ?? null;
125     }
126     return null;
127 }
128

```

```

1 // arduino-lib-popup.js – fenêtre flottante : commandes d'une bibliothèque Arduino
2 import { resolveArduinoLibDoc } from './arduino-lib-commands.mjs';
3
4 let floatX = 160;
5 let floatY = 90;
6 let dragging = false;
7 let dragOffX = 0;
8 let dragOffY = 0;
9
10 const popupEl = () => document.getElementById('arduino-lib-popup');
11 const titleEl = () => document.getElementById('arduino-lib-popup-title');
12 const bodyEl = () => document.getElementById('arduino-lib-popup-body');
13
14 function escapeHtml(s) {
15   return String(s | '|')
16     .replace(/&/g, '&amp;')
17     .replace(/</g, '&lt;')
18     .replace(/>/g, '&gt;')
19     .replace(/"/g, '&quot;');
20 }
21
22 function applyFloatPosition() {
23   const el = popupEl();
24   if (!el) return;
25   el.style.left = `${floatX}px`;
26   el.style.top = `${floatY}px`;
27 }
28
29 function clampFloatPosition() {
30   const el = popupEl();
31   if (!el) return;
32   const w = el.offsetWidth || 480;
33   const h = el.offsetHeight || 400;
34   floatX = Math.max(4, Math.min(window.innerWidth - w - 4, floatX));
35   floatY = Math.max(48, Math.min(window.innerHeight - h - 4, floatY));
36 }
37
38 function renderUnknownLib(libName) {
39   return `
40     <p class="arduino-lib-doc-note">
41       Aucune fiche détaillée pour <strong>${escapeHtml(libName)}</strong>.
42       Consultez la documentation du Library Manager ou le fichier <code>README</code> de la bibliothèque.
43     </p>
44     <p class="arduino-lib-doc-note">
45       Bibliothèques documentées dans le simulateur :
46       <strong>LiquidCrystal I2C</strong>, <strong>Wire</strong>, <strong>Grove LCD RGB</strong>.
47     </p>
48   `;
49 }
50
51 function renderLibDoc(doc) {
52   const header = doc.header
53     ? `<pre class="arduino-lib-doc-code">${escapeHtml(doc.header)}</pre>`
54     : '';
55   const ctor = doc.ctor
56     ? `<pre class="arduino-lib-doc-code">${escapeHtml(doc.ctor)}</pre>`
57     : '';
58   const note = doc.note
59     ? `<p class="arduino-lib-doc-note">${escapeHtml(doc.note)}</p>`
60     : '';
61   const rows = doc.commands.map((cmd) => {
62     const badge = cmd.simSupported
63       ? `<span class="arduino-lib-doc-badge arduino-lib-doc-badge--sim">simulateur</span>`
64       : `<span class="arduino-lib-doc-badge">compilation</span>`;
65     return `<tr>
66       <td><code class="arduino-lib-doc-sig">${escapeHtml(cmd.sig)}</code>${badge}</td>
67       <td>${escapeHtml(cmd.desc)}</td>
68     </tr>`;
69   }).join('');
70   return `
71     ${note}
72     ${header}
73     ${ctor}
74     <table class="arduino-lib-doc-table">
75       <thead>
76         <tr><th>Commande</th><th>Description</th></tr>
77       </thead>
78       <tbody>${rows}</tbody>

```

```

79     </table>
80     <p class="arduino-lib-doc-legend">
81         <span class="arduino-lib-doc-badge arduino-lib-doc-badge--sim">simulateur</span> effet visible en simulation
82         · <span class="arduino-lib-doc-badge">compilation</span> compile uniquement (arduino-cli)
83     </p>
84 `;
85 }
86
87 export function isArduinoLibPopupOpen() {
88     const el = popupEl();
89     return el && !el.classList.contains('hidden');
90 }
91
92 export function openArduinoLibPopup(libName) {
93     const name = String(libName || '').trim();
94     if (!name) return;
95     const doc = resolveArduinoLibDoc(name);
96     if (titleEl()) titleEl().textContent = doc ? doc.title : name;
97     if (bodyEl()) {
98         bodyEl().innerHTML = doc ? renderLibDoc(doc) : renderUnknownLib(name);
99     }
100     applyFloatPosition();
101     popupEl()?.classList.remove('hidden');
102 }
103
104 export function closeArduinoLibPopup() {
105     popupEl()?.classList.add('hidden');
106 }
107
108 export function initArduinoLibPopup() {
109     document.getElementById('arduino-lib-popup-close')?.addEventListener('click', (e) => {
110         e.stopPropagation();
111         closeArduinoLibPopup();
112     });
113
114     const handle = document.getElementById('arduino-lib-popup-drag-handle');
115     handle?.addEventListener('mousedown', (e) => {
116         if (e.button !== 0) return;
117         if (e.target.closest('#arduino-lib-popup-close')) return;
118         dragging = true;
119         const rect = popupEl()?.getBoundingClientRect();
120         if (rect) {
121             dragOffX = e.clientX - rect.left;
122             dragOffY = e.clientY - rect.top;
123         }
124         e.preventDefault();
125     });
126
127     window.addEventListener('mousemove', (e) => {
128         if (!dragging) return;
129         floatX = e.clientX - dragOffX;
130         floatY = e.clientY - dragOffY;
131         clampFloatPosition();
132         applyFloatPosition();
133     });
134
135     window.addEventListener('mouseup', () => {
136         dragging = false;
137     });
138
139     window.addEventListener('resize', () => {
140         if (isArduinoLibPopupOpen()) {
141             clampFloatPosition();
142             applyFloatPosition();
143         }
144     });
145 }
146

```

```
1  /** Synchronise le sketch éditeur → composants UNO (sans dépendre de renderer.js). */
2  import { circuit } from './state.js';
3  import { applyArduinoSketchToComponent } from './Engine/arduino-sketch-parse.mjs';
4
5  let getActiveBoard = () => null;
6
7  export function registerArduinoSketchSync(getBoardFn) {
8    getActiveBoard = typeof getBoardFn === 'function' ? getBoardFn : () => null;
9  }
10
11 function isActiveBoardOnCanvas(board) {
12   return !!board && circuit.components.includes(board);
13 }
14
15 export function syncArduinoSketchesFromEditor() {
16   const board = getActiveBoard();
17   const el = document.getElementById('arduino-sketch-input');
18   const panel = document.getElementById('arduino-panel');
19   if (board && el && panel && !panel.classList.contains('hidden') && isActiveBoardOnCanvas(board)) {
20     board.sketch = el.value;
21   }
22   for (const comp of circuit.components) {
23     if (comp.type === 'arduino_uno') applyArduinoSketchToComponent(comp);
24   }
25 }
26
```

```

1  /** Coloration syntaxique sketch Arduino (style proche de l'IDE). */
2
3  const KEYWORDS = new Set([
4    'if', 'else', 'for', 'while', 'do', 'switch', 'case', 'default', 'break', 'continue', 'return', 'goto',
5    'void', 'boolean', 'byte', 'char', 'int', 'long', 'float', 'double', 'short', 'unsigned', 'signed',
6    'const', 'static', 'volatile', 'sizeof', 'true', 'false', 'NULL', 'struct', 'enum', 'typedef',
7    'uint8_t', 'uint16_t', 'uint32_t', 'int8_t', 'int16_t', 'int32_t', 'word', 'string', 'String',
8  ]);
9
10 const TYPE_WORDS = new Set(['boolean', 'byte', 'char', 'int', 'long', 'float', 'double', 'short', 'void', 'word', 'String']);
11
12 function escapeHtml(s) {
13   return String(s)
14     .replace(/&/g, '&amp;')
15     .replace(/</g, '&lt;')
16     .replace(/>/g, '&gt;');
17 }
18
19 function span(cls, text) {
20   return `<span class="${cls}">${escapeHtml(text)}</span>`;
21 }
22
23 function skipString(src, i, quote) {
24   i++;
25   while (i < src.length) {
26     if (src[i] === '\\' && i + 1 < src.length) { i += 2; continue; }
27     if (src[i] === quote) return i + 1;
28     i++;
29   }
30   return src.length;
31 }
32
33 function skipBlockComment(src, i) {
34   i += 2;
35   while (i < src.length - 1) {
36     if (src[i] === '*' && src[i + 1] === '/') return i + 2;
37     i++;
38   }
39   return src.length;
40 }
41
42 /** @param {string} code */
43 export function highlightArduinoSource(code) {
44   const src = String(code);
45   let out = '';
46   let i = 0;
47   while (i < src.length) {
48     const ch = src[i];
49     const next = src[i + 1];
50
51     if (ch === '/' && next === '/') {
52       const end = src.indexOf('\n', i);
53       const slice = end === -1 ? src.slice(i) : src.slice(i, end);
54       out += span('ard-hl-cmt', slice);
55       i += slice.length;
56       continue;
57     }
58     if (ch === '/' && next === '*') {
59       const end = skipBlockComment(src, i);
60       out += span('ard-hl-cmt', src.slice(i, end));
61       i = end;
62       continue;
63     }
64     if (ch === '#') {
65       const end = src.indexOf('\n', i);
66       const slice = end === -1 ? src.slice(i) : src.slice(i, end);
67       const inc = slice.match(/^#include\s+([<">"])[^">"]+/);
68       if (inc) {
69         const head = slice.slice(0, inc.index + inc[0].indexOf(inc[1]));
70         const lib = inc[1];
71         out += span('ard-hl-pre', head) + span('ard-hl-lib', lib) + span('ard-hl-pre', slice.slice(head.length + lib.length));
72       } else {
73         out += span('ard-hl-pre', slice);
74       }
75       i += slice.length;
76       continue;
77     }
78     if (ch === '"' || ch === "'") {

```

```

79         const end = skipString(src, i, ch);
80         out += span('ard-hl-str', src.slice(i, end));
81         i = end;
82         continue;
83     }
84     if (/[\0-9]/.test(ch) || (ch === '.' && /\0-9]/.test(next))) {
85         let j = i;
86         while (j < src.length && /\0-9.xXa-fA-FuUll]/.test(src[j])) j++;
87         out += span('ard-hl-num', src.slice(i, j));
88         i = j;
89         continue;
90     }
91     if (/[A-Za-z_]/.test(ch)) {
92         let j = i;
93         while (j < src.length && /[A-Za-z\0-9_]/.test(src[j])) j++;
94         const word = src.slice(i, j);
95         let k = j;
96         while (k < src.length && /\s/.test(src[k])) k++;
97         const isCall = src[k] === '(';
98         if (word === 'F' && src[k] === '(') {
99             out += span('ard-hl-macro', word);
100         } else if (KEYWORDS.has(word)) {
101             out += span(TYPE_WORDS.has(word) ? 'ard-hl-type' : 'ard-hl-kw', word);
102         } else if (isCall) {
103             out += span('ard-hl-fn', word);
104         } else {
105             out += escapeHtml(word);
106         }
107         i = j;
108         continue;
109     }
110     out += escapeHtml(ch);
111     i++;
112 }
113 return out;
114 }
115
116 let highlightEl = null;
117 let textareaEl = null;
118
119 /** Ajuste la hauteur des deux calques pour qu'ils restent superposés. */
120 function syncEditorHeight() {
121     if (!textareaEl || !highlightEl) return;
122     textareaEl.style.height = '0px';
123     const h = Math.max(240, textareaEl.scrollHeight);
124     textareaEl.style.height = `${h}px`;
125     highlightEl.style.height = `${h}px`;
126 }
127
128 function syncHighlight() {
129     if (!highlightEl || !textareaEl) return;
130     highlightEl.innerHTML = highlightArduinoSource(textareaEl.value);
131     syncEditorHeight();
132 }
133
134 export function initArduinoSyntaxHighlight() {
135     textareaEl = document.getElementById('arduino-sketch-input');
136     highlightEl = document.getElementById('arduino-sketch-highlight');
137     if (!textareaEl || !highlightEl) return;
138
139     const refresh = () => syncHighlight();
140
141     textareaEl.addEventListener('input', refresh);
142     textareaEl.addEventListener('focus', refresh);
143     window.addEventListener('resize', refresh);
144
145     refresh();
146 }
147
148 export function refreshArduinoSyntaxHighlight() {
149     syncHighlight();
150 }
151

```



```

1  /** Géométrie Arduino UNO R3 – brochage réel (headers power/analog + digital). */
2  import { GRID_SIZE as G } from './state.js';
3
4  export const UNO_BOX_L = -4 * G;
5  export const UNO_BOX_R = 5 * G;
6  export const UNO_BOX_T = -7 * G;
7  export const UNO_BOX_B = 8 * G;
8  export const UNO_JUNC_L = -5 * G;
9  export const UNO_JUNC_R = 6 * G;
10 export const UNO_LABEL_L = UNO_BOX_L + G / 2;
11 export const UNO_LABEL_R = UNO_BOX_R - G / 2;
12 export const UNO_HIT_DX = 6 * G + G / 2;
13 export const UNO_HIT_DY = 8 * G + G / 2;
14
15 /** Broches header gauche (haut → bas) : alimentation + analogique. */
16 export const UNO_LEFT_PINS = [
17   'IOREF', 'RESET', '3V3', '5V', 'GND', 'GND2', 'VIN',
18   'A0', 'A1', 'A2', 'A3', 'A4', 'A5',
19 ];
20
21 /** Broches header droit (haut → bas) : D13 → D0. */
22 export const UNO_RIGHT_PINS = [
23   'D13', 'D12', 'D11', 'D10', 'D9', 'D8', 'D7', 'D6',
24   'D5', 'D4', 'D3', 'D2', 'D1', 'D0',
25 ];
26
27 export const UNO_LEFT_PIN_Y = UNO_LEFT_PINS.map((_, i) => (i - 6) * G);
28
29 /** 14 broches : positions entières sur la grille (évite le décalage i - 6.5). */
30 export const UNO_RIGHT_PIN_Y = UNO_RIGHT_PINS.map((_, i) => (i - 6) * G);
31
32 /** Indice SPICE #0..#26 (ordre : gauche puis droite). */
33 export const UNO_PIN = {};
34 UNO_LEFT_PINS.forEach((name, i) => {
35   UNO_PIN[name] = i;
36 });
37 UNO_RIGHT_PINS.forEach((name, i) => {
38   UNO_PIN[name] = UNO_LEFT_PINS.length + i;
39 });
40
41 export const UNO_PIN_COUNT = UNO_LEFT_PINS.length + UNO_RIGHT_PINS.length;
42
43 export const UNO_JUNCTION_SUFFIX = { ...UNO_PIN };
44
45 export const UNO_DIGITAL_PINS = UNO_RIGHT_PINS.slice().reverse();
46 export const UNO_ANALOG_PINS = ['A0', 'A1', 'A2', 'A3', 'A4', 'A5'];
47
48 /**
49  * Numéro de broche du boîtier DIP-28 ATmega328P (datasheet Microchip).
50  * Correspond aux signaux GPIO du header Arduino UNO R3.
51  */
52 export const UNO_ATMEGA328P_DIP = {
53   RESET: 1,
54   D0: 2,
55   D1: 3,
56   D2: 4,
57   D3: 5,
58   D4: 6,
59   D5: 11,
60   D6: 12,
61   D7: 13,
62   D8: 14,
63   D9: 15,
64   D10: 16,
65   D11: 17,
66   D12: 18,
67   D13: 19,
68   A0: 23,
69   A1: 24,
70   A2: 25,
71   A3: 26,
72   A4: 27,
73   A5: 28,
74 };
75
76 /** Libellé broche header : « D0 (2) » si numéro MCU connu. */
77 export function formatUnoPinLabel(pinName) {
78   const n = UNO_ATMEGA328P_DIP[pinName];

```

```

79     return n !== null ? `${pinName} (${n})` : pinName;
80 }
81
82 export function arduinoUnoJunctionToTerminalKey(label, junctionId) {
83     if (!label || !junctionId?.startsWith(`${label}_`)) return null;
84     const suffix = junctionId.slice(label.length + 1);
85     const idx = UNO_JUNCTION_SUFFIX[suffix];
86     if (idx === undefined) return null;
87     return `${label}#${idx}`;
88 }
89
90 export function arduinoUnoTerminalKeys(label) {
91     const keys = [];
92     for (let i = 0; i < UNO_PIN_COUNT; i++) keys.push(`${label}#${i}`);
93     return keys;
94 }
95
96 export const DEFAULT_ARDUINO_SKETCH = `// Arduino UNO - sketch minimal
97 void setup() {
98     pinMode(LED_BUILTIN, OUTPUT);
99 }
100
101 void loop() {
102     digitalWrite(LED_BUILTIN, HIGH);
103     delay(500);
104     digitalWrite(LED_BUILTIN, LOW);
105     delay(500);
106 }
107 `;
108

```

```

1  /** Bargraph DC10H (Lite-On) – 10 segments, cathode commune. s1 = barre du bas, s10 = barre du haut. */
2  /** Jonctions sur multiples de 20 (GRID_SIZE) pour coïncider avec snapToGrid. */
3
4  export const DC10H_SEG_COUNT = 10;
5  export const DC10H_SEG_NAMES = Array.from({ length: DC10H_SEG_COUNT }, (_, i) => `s${i + 1}`);
6
7  export const DC10H_BOX_L = -20;
8  export const DC10H_BOX_R = 40;
9  export const DC10H_BOX_T = -100;
10 /** Cadre blanc jusqu'au segment du bas (s1, y=100) inclus. */
11 export const DC10H_BOX_B = 120;
12
13 export const DC10H_JUNC_L = -40;
14 /** Jonctions à droite (flip X) – même écart de 20 px que JUNC_L → BOX_L. */
15 export const DC10H_JUNC_R = 60;
16 export const DC10H_STUB_LEN = DC10H_BOX_L - DC10H_JUNC_L;
17 export const DC10H_BOX_CX = (DC10H_BOX_L + DC10H_BOX_R) / 2;
18
19 /** s1 (bas, y=100) ... s10 (haut, y=-80) – pas de 20 px. */
20 export const DC10H_PIN_Y = [100, 80, 60, 40, 20, 0, -20, -40, -60, -80];
21
22 export const DC10H_COM_X = 20;
23 export const DC10H_COM_Y = 140;
24
25 export const DC10H_BAR_H = 12;
26
27 /** Décalage du nom composant au-dessus du cadre (repère local, côté s10). */
28 export const DC10H_COMP_LABEL_OFFSET = 22;
29 /** Décalage « DC10H » sous la jonction COM (repère local). */
30 export const DC10H_TYPE_LABEL_OFFSET = 16;
31 /** Décalage des noms de broches hors des fils (repère local, côté jonctions). */
32 export const DC10H_PIN_LABEL_OFFSET_X = -12;
33 export const DC10H_COM_LABEL_OFFSET_X = 10;
34
35 export const DC10H_SEL_L = DC10H_JUNC_L - 4;
36 export const DC10H_SEL_T = DC10H_BOX_T - DC10H_COMP_LABEL_OFFSET - 10;
37 export const DC10H_SEL_W = DC10H_JUNC_R + 4 - DC10H_SEL_L;
38 export const DC10H_SEL_H = DC10H_COM_Y + DC10H_TYPE_LABEL_OFFSET + 10 - DC10H_SEL_T;
39
40 export const DC10H_HIT_DX = Math.max(-DC10H_SEL_L, DC10H_SEL_L + DC10H_SEL_W) + 4;
41 export const DC10H_HIT_DY = DC10H_SEL_H / 2 + 4;
42
43 export const DC10H_COLOR_IDS = ['red', 'green', 'amber', 'blue', 'yellow'];
44
45 export const DC10H_COLORS = {
46   red: { lit: '#ff1744', dim: '#3d0a0a', pin: '#ff1744', label: 'Rouge' },
47   green: { lit: '#00e676', dim: '#0a2e14', pin: '#00e676', label: 'Vert' },
48   amber: { lit: '#ffab00', dim: '#3a2800', pin: '#ffab00', label: 'Ambre' },
49   blue: { lit: '#448aff', dim: '#0a1a3a', pin: '#448aff', label: 'Bleu' },
50   yellow: { lit: '#ffea00', dim: '#3a3500', pin: '#ffea00', label: 'Jaune' },
51 };
52
53 export function dc10hPalette(colorId) {
54   return DC10H_COLORS[colorId] || DC10H_COLORS.red;
55 }
56
57 export function nextDc10hColor(colorId) {
58   const idx = DC10H_COLOR_IDS.indexOf(colorId);
59   const next = idx < 0 ? 0 : (idx + 1) % DC10H_COLOR_IDS.length;
60   return DC10H_COLOR_IDS[next];
61 }
62
63 /** Y du haut de la barre pour le segment s(segIndex+1), centrée sur la patte. */
64 export function dc10hBarTopY(segIndex) {
65   return DC10H_PIN_Y[segIndex] - DC10H_BAR_H / 2;
66 }
67
68 /** Position X de COM (symétrique par rapport au centre du boîtier). */
69 export function dc10hComX(flipX) {
70   return flipX ? 2 * DC10H_BOX_CX - DC10H_COM_X : DC10H_COM_X;
71 }
72

```

```

1 // bode-popup.js - fenêtre diagramme de Bode (gain en dB, axe fréquence semi-log)
2 import { flags, simulationResults } from './state.js';
3
4 let popupComp = null;
5 let popupCanvas = null;
6 let popupCtx = null;
7
8 let floatX = 140;
9 let floatY = 90;
10 let dragging = false;
11 let dragOffX = 0;
12 let dragOffY = 0;
13
14 /** Curseur souris : fréquence (Hz) sous le pointeur, null = masqué */
15 let cursorFreqHz = null;
16
17 /** Géométrie du dernier tracé (pour convertir souris ↔ fréquence) */
18 let plotGeom = null;
19
20 const popupEl = () => document.getElementById('bode-popup');
21 const titleEl = () => document.getElementById('bode-popup-title');
22 const infoEl = () => document.getElementById('bode-popup-info');
23
24 function getBodePlotForComp(comp) {
25   if (!comp) return null;
26   const plots = simulationResults.bodePlots;
27   if (!plots || typeof plots !== 'object') return null;
28   if (plots[comp.label]) return plots[comp.label];
29   const keys = Object.keys(plots);
30   if (keys.length === 1) return plots[keys[0]];
31   return null;
32 }
33
34 function applyFloatPosition() {
35   const el = popupEl();
36   if (!el) return;
37   el.style.left = `${floatX}px`;
38   el.style.top = `${floatY}px`;
39 }
40
41 function formatFreqHz(f) {
42   if (!Number.isFinite(f) || f <= 0) return '?';
43   if (f >= 1e6) return `${(f / 1e6).toFixed(f >= 10e6 ? 0 : 2)} MHz`;
44   if (f >= 1000) return `${(f / 1000).toFixed(f >= 10e3 ? 1 : 2)} kHz`;
45   return `${f < 10 ? f.toFixed(2) : Math.round(f * 10) / 10} Hz`;
46 }
47
48 function freqToX(freq, fMin, fMax, x0, plotW) {
49   const lo = Math.log10(Math.max(fMin, 1e-6));
50   const hi = Math.log10(Math.max(fMax, fMin * 1.01));
51   const f = Math.max(freq, fMin);
52   return x0 + ((Math.log10(f) - lo) / (hi - lo)) * plotW;
53 }
54
55 function xToFreq(x, fMin, fMax, x0, plotW) {
56   const lo = Math.log10(Math.max(fMin, 1e-6));
57   const hi = Math.log10(Math.max(fMax, fMin * 1.01));
58   const t = Math.max(0, Math.min(1, (x - x0) / plotW));
59   return Math.pow(10, lo + t * (hi - lo));
60 }
61
62 function gainToY(gainDb, gMin, gMax, y0, plotH) {
63   const t = (gainDb - gMin) / Math.max(gMax - gMin, 1e-6);
64   return y0 + plotH - t * plotH;
65 }
66
67 function interpolateGainDb(frequency, gainDb, f) {
68   if (!frequency?.length || f <= 0) return NaN;
69   if (f <= frequency[0]) return gainDb[0];
70   if (f >= frequency[frequency.length - 1]) return gainDb[gainDb.length - 1];
71   for (let i = 1; i < frequency.length; i++) {
72     if (f <= frequency[i]) {
73       const f0 = frequency[i - 1];
74       const f1 = frequency[i];
75       const g0 = gainDb[i - 1];
76       const g1 = gainDb[i];
77       if (f1 <= f0) return g0;
78       const t = (Math.log10(f) - Math.log10(f0)) / (Math.log10(f1) - Math.log10(f0));

```

```

79         return g0 + t * (g1 - g0);
80     }
81 }
82 return gainDb[gainDb.length - 1];
83 }
84
85 function drawBodeGrid(ctx, x0, y0, plotW, plotH, fMin, fMax, gMin, gMax) {
86     ctx.strokeStyle = '#1a3a4a';
87     ctx.lineWidth = 1;
88     const decades = Math.max(1, Math.ceil(Math.log10(fMax / Math.max(fMin, 1e-6))));
89     for (let d = 0; d <= decades; d++) {
90         const f = fMin * Math.pow(10, (d / decades) * Math.log10(fMax / fMin));
91         const x = freqToX(f, fMin, fMax, x0, plotW);
92         ctx.beginPath();
93         ctx.moveTo(x, y0);
94         ctx.lineTo(x, y0 + plotH);
95         ctx.stroke();
96     }
97     for (let i = 0; i <= 4; i++) {
98         const y = y0 + (i / 4) * plotH;
99         ctx.beginPath();
100        ctx.moveTo(x0, y);
101        ctx.lineTo(x0 + plotW, y);
102        ctx.stroke();
103    }
104 }
105
106 function drawFreqLabels(ctx, x0, y0, plotW, plotH, fMin, fMax) {
107     ctx.fillStyle = '#8899aa';
108     ctx.font = '10px Arial';
109     ctx.textAlign = 'center';
110     ctx.textBaseline = 'top';
111     const ticks = [fMin, fMin * 10, fMin * 100, fMin * 1000, fMax].filter((f) => f >= fMin && f <= fMax * 1.001);
112     const seen = new Set();
113     for (const f of ticks) {
114         const key = Math.round(Math.log10(f) * 100);
115         if (seen.has(key)) continue;
116         seen.add(key);
117         ctx.fillText(formatFreqHz(f), freqToX(f, fMin, fMax, x0, plotW), y0 + plotH + 6);
118     }
119     ctx.textAlign = 'right';
120     ctx.textBaseline = 'middle';
121     ctx.save();
122     ctx.translate(x0 - 8, y0 + plotH / 2);
123     ctx.rotate(-Math.PI / 2);
124     ctx.fillText('Gain (dB)', 0, 0);
125     ctx.restore();
126     ctx.textAlign = 'center';
127     ctx.fillText('Fréquence', x0 + plotW / 2, y0 + plotH + 22);
128 }
129
130 function drawGainLabels(ctx, x0, y0, plotH, gMin, gMax) {
131     ctx.fillStyle = '#8899aa';
132     ctx.font = '10px Arial';
133     ctx.textAlign = 'right';
134     ctx.textBaseline = 'middle';
135     for (let i = 0; i <= 4; i++) {
136         const g = gMin + (i / 4) * (gMax - gMin);
137         ctx.fillText(`${g.toFixed(0)}`, x0 - 6, gainToY(g, gMin, gMax, y0, plotH));
138     }
139 }
140
141 function drawCursor(ctx, plot, geom) {
142     if (cursorFreqHz == null || !plot?.frequency?.length || !geom) return;
143     const { x0, y0, plotW, plotH, fMin, fMax, gMin, gMax } = geom;
144     const f = Math.max(fMin, Math.min(fMax, cursorFreqHz));
145     const gain = interpolateGainDb(plot.frequency, plot.gainDb, f);
146     if (!Number.isFinite(gain)) return;
147     const cx = freqToX(f, fMin, fMax, x0, plotW);
148     const cy = gainToY(gain, gMin, gMax, y0, plotH);
149
150     ctx.save();
151     ctx.beginPath();
152     ctx.rect(x0, y0, plotW, plotH);
153     ctx.clip();
154     ctx.strokeStyle = '#ff9800';
155     ctx.lineWidth = 1;
156     ctx.setLineDash([4, 4]);
157     ctx.beginPath();
158     ctx.moveTo(cx, y0);

```

```

159 ctx.lineTo(cx, y0 + plotH);
160 ctx.stroke();
161 ctx.setLineDash([]);
162 ctx.fillStyle = '#ff9800';
163 ctx.beginPath();
164 ctx.arc(cx, cy, 5, 0, Math.PI * 2);
165 ctx.fill();
166 ctx.strokeStyle = '#fff';
167 ctx.lineWidth = 1.5;
168 ctx.stroke();
169 ctx.restore();
170
171 if (infoEl()) {
172     let txt = `f = ${formatFreqHz(f)} | Gain = ${gain.toFixed(2)} dB`;
173     if (plot.responseHint?.includes('passe-haut')) {
174         txt += ` - ${plot.responseHint}`;
175     }
176     infoEl().textContent = txt;
177 }
178 }
179
180 function updateCursorFromMouse(clientX, clientY) {
181     if (!plotGeom || !popupCanvas) return;
182     const rect = popupCanvas.getBoundingClientRect();
183     const scaleX = popupCanvas.width / rect.width;
184     const scaleY = popupCanvas.height / rect.height;
185     const mx = (clientX - rect.left) * scaleX;
186     const my = (clientY - rect.top) * scaleY;
187     const { x0, y0, plotW, plotH, fMin, fMax } = plotGeom;
188     if (mx < x0 || mx > x0 + plotW || my < y0 || my > y0 + plotH) return;
189     cursorFreqHz = xToFreq(mx, fMin, fMax, x0, plotW);
190     renderBodePopup();
191 }
192
193 export function isBodePopupOpen() {
194     const el = popupEl();
195     return popupComp != null && el && !el.classList.contains('hidden');
196 }
197
198 export function renderBodePopup() {
199     if (!popupComp || !popupCtx || !popupCanvas) return;
200     const w = popupCanvas.width;
201     const h = popupCanvas.height;
202     const margin = { l: 52, r: 20, t: 28, b: 44 };
203     const plotW = w - margin.l - margin.r;
204     const plotH = h - margin.t - margin.b;
205     const x0 = margin.l;
206     const y0 = margin.t;
207
208     popupCtx.fillStyle = '#050a0e';
209     popupCtx.fillRect(0, 0, w, h);
210
211     const plot = getBodePlotForComp(popupComp);
212     plotGeom = null;
213
214     if (!flags.isSimulating || !plot?.frequency?.length) {
215         if (infoEl()) {
216             infoEl().textContent = flags.isSimulating
217                 ? 'Sin requis - + sur la sortie du filtre, - sur la masse'
218                 : 'Lancez la simulation';
219         }
220         popupCtx.fillStyle = '#8899aa';
221         popupCtx.font = '13px Arial';
222         popupCtx.textAlign = 'center';
223         popupCtx.textBaseline = 'middle';
224         const lines = flags.isSimulating
225             ? ['Aucune courbe', 'Générateur Sin + filtre RC/RL/RLC']
226             : ['Lancez la simulation pour voir le diagramme de Bode'];
227         lines.forEach((line, i) => {
228             popupCtx.fillText(line, w / 2, h / 2 - 10 + i * 20);
229         });
230         return;
231     }
232
233     const { frequency, gainDb, fMin, fMax } = plot;
234     let gMin = Math.min(...gainDb);
235     let gMax = Math.max(...gainDb);
236     if (gMax - gMin < 6) {
237         const mid = (gMax + gMin) / 2;
238         gMin = mid - 6;

```

```

239     gMax = mid + 6;
240 } else {
241     gMin -= 2;
242     gMax += 2;
243 }
244
245 plotGeom = { x0, y0, plotW, plotH, fMin, fMax, gMin, gMax };
246
247 if (cursorFreqHz == null && frequency.length > 0) {
248     const midIdx = Math.floor(frequency.length / 2);
249     cursorFreqHz = frequency[midIdx];
250 }
251
252 drawBodeGrid(popupCtx, x0, y0, plotW, plotH, fMin, fMax, gMin, gMax);
253 drawGainLabels(popupCtx, x0, y0, plotH, gMin, gMax);
254 drawFreqLabels(popupCtx, x0, y0, plotW, plotH, fMin, fMax);
255
256 popupCtx.save();
257 popupCtx.beginPath();
258 popupCtx.rect(x0, y0, plotW, plotH);
259 popupCtx.clip();
260 popupCtx.strokeStyle = '#7cff6b';
261 popupCtx.lineWidth = 2;
262 popupCtx.beginPath();
263 let started = false;
264 for (let i = 0; i < frequency.length; i++) {
265     const x = freqToX(frequency[i], fMin, fMax, x0, plotW);
266     const y = gainToY(gainDb[i], gMin, gMax, y0, plotH);
267     if (!started) {
268         popupCtx.moveTo(x, y);
269         started = true;
270     } else {
271         popupCtx.lineTo(x, y);
272     }
273 }
274 if (started) popupCtx.stroke();
275 popupCtx.restore();
276
277 drawCursor(popupCtx, plot, plotGeom);
278
279 if (infoEl() && cursorFreqHz == null && plot.responseHint?.includes('passe-haut')) {
280     infoEl().textContent = plot.responseHint;
281 }
282
283 popupCtx.fillStyle = plot.responseHint?.includes('passe-haut') ? '#ff9800' : '#7cff6b';
284 popupCtx.font = '12px Arial';
285 popupCtx.textAlign = 'left';
286 const title =
287     plot.responseHint?.includes('passe-haut')
288         ? '⚠ Passe-haut – vérifiez le câblage (+ jonction R/C, – GND)'
289         : '|H(jw)| – déplacez la souris sur la courbe';
290 popupCtx.fillText(title, margin.l, 16);
291 }
292
293 export function refreshBodePopup() {
294     if (isBodePopupOpen()) renderBodePopup();
295 }
296
297 export function openBodePopup(comp) {
298     if (!comp || comp.type !== 'bode_analyzer') return;
299     popupComp = comp;
300     popupCanvas = document.getElementById('bode-popup-canvas');
301     popupCtx = popupCanvas?.getContext('2d') ?? null;
302     cursorFreqHz = null;
303     plotGeom = null;
304     if (titleEl()) titleEl().textContent = `${comp.label} – Analyse fréquentielle`;
305     applyFloatPosition();
306     popupEl()?.classList.remove('hidden');
307     renderBodePopup();
308 }
309
310 export function closeBodePopup(clearComp = false) {
311     popupEl()?.classList.add('hidden');
312     if (clearComp) popupComp = null;
313     cursorFreqHz = null;
314     plotGeom = null;
315 }
316
317 function clampFloatPosition() {
318     const el = popupEl();

```

```

319     if (!el) return;
320     const pw = el.offsetWidth || 680;
321     const ph = el.offsetHeight || 440;
322     floatX = Math.max(4, Math.min(window.innerWidth - pw - 4, floatX));
323     floatY = Math.max(48, Math.min(window.innerHeight - ph - 4, floatY));
324 }
325
326 export function initBodePopup() {
327     document.getElementById('bode-popup-close')?.addEventListener('click', (e) => {
328         e.stopPropagation();
329         closeBodePopup();
330     });
331
332     const handle = document.getElementById('bode-popup-drag-handle');
333     handle?.addEventListener('mousedown', (e) => {
334         if (e.button !== 0) return;
335         if (e.target.closest('#bode-popup-close')) return;
336         dragging = true;
337         const rect = popupEl()?.getBoundingClientRect();
338         if (rect) {
339             dragOffX = e.clientX - rect.left;
340             dragOffY = e.clientY - rect.top;
341         }
342         e.preventDefault();
343     });
344
345     window.addEventListener('mousemove', (e) => {
346         if (dragging) {
347             floatX = e.clientX - dragOffX;
348             floatY = e.clientY - dragOffY;
349             clampFloatPosition();
350             applyFloatPosition();
351             return;
352         }
353         if (isBodePopupOpen() && plotGeom) {
354             updateCursorFromMouse(e.clientX, e.clientY);
355         }
356     });
357
358     window.addEventListener('mouseup', () => {
359         dragging = false;
360     });
361
362     window.addEventListener('resize', () => {
363         if (isBodePopupOpen()) {
364             clampFloatPosition();
365             applyFloatPosition();
366         }
367     });
368 }
369

```



```

1  /** Géométrie CD4511 : boîtier 4x8 pas, broches ±3 pas, espacement vertical 1 pas. */
2  import { GRID_SIZE as G } from './state.js';
3
4  /** Rectangle central : 4 pas (L) × 8 pas (H), centré sur le composant. */
5  export const CD4511_BOX_L = -2 * G;
6  export const CD4511_BOX_R = 2 * G;
7  export const CD4511_BOX_T = -4 * G;
8  export const CD4511_BOX_B = 4 * G;
9
10 /** Libellés à ½ pas à l'intérieur du cadre. */
11 export const CD4511_LABEL_L = CD4511_BOX_L + G / 2;
12 export const CD4511_LABEL_R = CD4511_BOX_R - G / 2;
13
14 /** Jonctions à 1 pas à l'extérieur du boîtier (colonne grille). */
15 export const CD4511_JUNC_L = -3 * G;
16 export const CD4511_JUNC_R = 3 * G;
17
18 /** 7 broches, 1 pas entre chaque niveau, centrées verticalement (±3 pas). */
19 export const CD4511_PIN_Y = [-3, -2, -1, 0, 1, 2, 3].map((n) => n * G);
20
21 export const CD4511_HIT_DX = 3 * G + G / 2;
22 export const CD4511_HIT_DY = 4 * G + G / 2;
23
24 /** Noms de jonctions schéma (suffixe après le label) → indice broche SPICE. */
25 export const CD4511_JONCTION_SUFFIX = {
26   A: 0,
27   B: 1,
28   C: 2,
29   D: 3,
30   LE: 4,
31   BI: 5,
32   LT: 6,
33   a: 7,
34   b: 8,
35   c: 9,
36   d: 10,
37   e: 11,
38   f: 12,
39   g: 13,
40 };
41
42 /** @param {string} label ex. CD45111 */
43 /** @param {string} jonctionId ex. CD45111_A */
44 export function cd4511JonctionToTerminalKey(label, jonctionId) {
45   if (!label || !jonctionId?.startsWith(`${label}_`)) return null;
46   const suffix = jonctionId.slice(label.length + 1);
47   const idx = CD4511_JONCTION_SUFFIX[suffix];
48   if (idx === undefined) return null;
49   return `${label}#${idx}`;
50 }
51

```

```
1  /** Grove DHT22 (T° / humidité) – géométrie calée grille, origine = centre du module. */
2  import { GRID_SIZE as G } from './state.js';
3
4  export const GROVE_DHT22_PINS = ['DATA', 'VCC', 'NC', 'GND'];
5  export const GROVE_DHT22_PIN_COUNT = GROVE_DHT22_PINS.length;
6
7  /** PCB bleu ~ 8×3,5 pas. */
8  export const GROVE_DHT22_BOX_W = 8 * G;
9  export const GROVE_DHT22_BOX_H = 3.5 * G;
10 export const GROVE_DHT22_BOX_L = -GROVE_DHT22_BOX_W / 2;
11 export const GROVE_DHT22_BOX_R = GROVE_DHT22_BOX_W / 2;
12
13 const GROVE_DHT22_Y_NUDGE = G / 2;
14 export const GROVE_DHT22_PIN_Y = [-1.5, -0.5, 0.5, 1.5].map((f) => f * G + GROVE_DHT22_Y_NUDGE);
15
16 const _wireCenterY = (GROVE_DHT22_PIN_Y[0] + GROVE_DHT22_PIN_Y[GROVE_DHT22_PIN_COUNT - 1]) / 2;
17 export const GROVE_DHT22_BOX_T = _wireCenterY - GROVE_DHT22_BOX_H / 2;
18 export const GROVE_DHT22_BOX_B = _wireCenterY + GROVE_DHT22_BOX_H / 2;
19
20 export const GROVE_DHT22_CONNECTOR_W = G;
21 export const GROVE_DHT22_CONN_L = GROVE_DHT22_BOX_L;
22 export const GROVE_DHT22_STUB_LEN = 4 * G;
23 export const GROVE_DHT22_JUNC_X = GROVE_DHT22_CONN_L - GROVE_DHT22_STUB_LEN;
24
25 export const GROVE_DHT22_SEL_L = GROVE_DHT22_JUNC_X - 4;
26 export const GROVE_DHT22_SEL_T = GROVE_DHT22_BOX_T - 4;
27 export const GROVE_DHT22_SEL_W = GROVE_DHT22_BOX_R - GROVE_DHT22_SEL_L + 4;
28 export const GROVE_DHT22_SEL_H = GROVE_DHT22_BOX_H + 8;
29
30 export const GROVE_DHT22_PIN_LABEL_X = (GROVE_DHT22_JUNC_X + GROVE_DHT22_CONN_L) / 2;
31 export const GROVE_DHT22_HIT_DX = Math.max(-GROVE_DHT22_SEL_L, GROVE_DHT22_BOX_R) + 4;
32 export const GROVE_DHT22_HIT_DY = GROVE_DHT22_BOX_H / 2 + 14;
33
34 /** Zone capteur blanc (grille perforée) à droite du connecteur. */
35 export const GROVE_DHT22_SENSOR_L = GROVE_DHT22_BOX_L + GROVE_DHT22_CONNECTOR_W + 4;
36 export const GROVE_DHT22_SENSOR_R = GROVE_DHT22_BOX_R - 6;
37 export const GROVE_DHT22_SENSOR_T = GROVE_DHT22_BOX_T + 4;
38 export const GROVE_DHT22_SENSOR_B = GROVE_DHT22_BOX_B - 4;
39
```

```

1  /**
2   * Entrées analogiques Arduino (A0-A5) – tension du circuit → ADC 10 bits.
3   */
4
5  import { reachableJunctions } from "../hc90-cascade.mjs";
6
7  const VREF = 5;
8  const ADC_MAX = 1023;
9
10 export function voltageToAdc(volts) {
11     if (!Number.isFinite(volts)) return 0;
12     const v = Math.max(0, Math.min(VREF, volts));
13     return Math.round((v / VREF) * ADC_MAX);
14 }
15
16 /** Tension fixe d'une jonction d'alimentation (broches UNO, GND, VCC...). */
17 export function fixedVoltageAtJunction(jid) {
18     if (!jid) return null;
19     if (jid.startsWith("GND") || /^GND\d+/.test(jid)) return 0;
20     if (/_GND2?$/ .test(jid)) return 0;
21     if (/_5V$/ .test(jid) || jid.endsWith("_VCC")) return 5;
22     if (/_3V3$/ .test(jid)) return 3.3;
23     return null;
24 }
25
26 function isGndJunction(jid) {
27     return fixedVoltageAtJunction(jid) === 0;
28 }
29
30 function parseResistorOhms(value) {
31     const s = String(value || "1k").trim().toLowerCase();
32     const m = s.match(/^([\d.]+)\s*([kmg]?)$/);
33     if (!m) return 1000;
34     let n = parseFloat(m[1]);
35     if (!Number.isFinite(n) || n <= 0) return 1000;
36     if (m[2] === "k") n *= 1e3;
37     else if (m[2] === "m") n *= 1e6;
38     else if (m[2] === "g") n *= 1e9;
39     return n;
40 }
41
42 function gsinVoltageAt(comp, tSec) {
43     const a = comp.peakAmplitude ?? 5;
44     const f = comp.frequency ?? 1000;
45     const o = comp.offset ?? 0;
46     return o + a * Math.sin(2 * Math.PI * f * Math.max(0, tSec));
47 }
48
49 function gimpVoltageAt(comp, tSec) {
50     const rail = comp.voltageRail ?? 5;
51     const f = comp.frequency ?? 2;
52     const duty = (comp.dutyCycle ?? 50) / 100;
53     if (f <= 0) return 0;
54     const period = 1 / f;
55     const phase = ((tSec % period) + period) % period;
56     return phase < period * duty ? rail : 0;
57 }
58
59 function gsqrVoltageAt(comp, tSec) {
60     const a = comp.peakAmplitude ?? 5;
61     const f = comp.frequency ?? 1000;
62     const o = comp.offset ?? 0;
63     if (f <= 0) return o;
64     const period = 1 / f;
65     const phase = ((tSec % period) + period) % period;
66     return o + (phase < period / 2 ? a : -a);
67 }
68
69 function voltageFromVoltsmetersOnNet(net, voltmeters) {
70     if (!voltmeters || typeof voltmeters !== "object") return null;
71     for (const [name, measure] of Object.entries(voltmeters)) {
72         const v = typeof measure === "object" ? measure?.voltage : measure;
73         if (!Number.isFinite(v)) continue;
74         const inn = `${name}_in`;
75         const out = `${name}_out`;
76         if (net.has(inn) || net.has(out)) return v;
77     }
78     return null;

```

```

79 }
80
81 /** Pont diviseur : deux résistances sur le même nœud. */
82 function tryResistorDivider(jonctionId, ctx, visiting) {
83     const { components, wires, autoJunctions = [] } = ctx;
84     const net = reachableJunctions(jonctionId, wires, autoJunctions);
85     const legs = [];
86     for (const comp of components) {
87         if (comp.type !== "resistor" || !comp.label) continue;
88         const inn = `${comp.label}_in`;
89         const out = `${comp.label}_out`;
90         if (!net.has(inn) && !net.has(out)) continue;
91         const touch = net.has(inn) ? inn : out;
92         const other = touch === inn ? out : inn;
93         const branch = new Set(visiting);
94         const vOther = resolveNetVoltage(other, ctx, branch);
95         if (!Number.isFinite(vOther)) continue;
96         legs.push({ ohm: parseResistorOhms(comp.value), vOther });
97     }
98     if (legs.length !== 2) return null;
99     const [a, b] = legs;
100     if (a.vOther === b.vOther) return a.vOther;
101     const high = a.vOther > b.vOther ? a : b;
102     const low = a.vOther > b.vOther ? b : a;
103     const rTop = high.ohm;
104     const rBot = low.ohm;
105     if (rTop + rBot <= 0) return null;
106     return low.vOther + (high.vOther - low.vOther) * (rBot / (rTop + rBot));
107 }
108
109 /**
110  * Tension (V) sur le réseau câblé à une jonction.
111  */
112 export function resolveNetVoltage(jonctionId, ctx, visiting = new Set()) {
113     if (!jonctionId || visiting.has(jonctionId)) return null;
114     visiting.add(jonctionId);
115
116     const { components, wires, autoJunctions = [], tSec = 0, getVoltageAtJunction, voltmeters } = ctx;
117     const net = reachableJunctions(jonctionId, wires, autoJunctions);
118
119     for (const j of net) {
120         const fixed = fixedVoltageAtJunction(j);
121         if (fixed !== null) return fixed;
122     }
123
124     for (const comp of components) {
125         if (!comp.label) continue;
126         if (comp.type === "vcc") {
127             for (const j of net) {
128                 if (j.startsWith(`${comp.label}_`)) return Number(comp.value) || 5;
129             }
130         }
131         if (comp.type === "battery" && net.has(`${comp.label}_in`)) {
132             return Number(comp.value) || 5;
133         }
134         if (comp.type === "logic_terminal") {
135             if (net.has(`${comp.label}_out`) || net.has(`${comp.label}_in`)) {
136                 return comp.state ? 5 : 0;
137             }
138         }
139     }
140
141     for (const comp of components) {
142         if (comp.type !== "potentiometer" || !comp.label) continue;
143         const wip = `${comp.label}_wip`;
144         if (!net.has(wip)) continue;
145         const branch = new Set(visiting);
146         const vIn = resolveNetVoltage(`${comp.label}_in`, ctx, branch);
147         const vOut = resolveNetVoltage(`${comp.label}_out`, ctx, branch);
148         const pos = Math.min(100, Math.max(0, Number(comp.position) || 50)) / 100;
149         const vi = Number.isFinite(vIn) ? vIn : 5;
150         const vo = Number.isFinite(vOut) ? vOut : 0;
151         return vi * (1 - pos) + vo * pos;
152     }
153
154     const dividerV = tryResistorDivider(jonctionId, ctx, visiting);
155     if (Number.isFinite(dividerV)) return dividerV;
156
157     for (const comp of components) {
158         if (!comp.label) continue;

```

```

159     const outJ = `${comp.label}_out`;
160     if (!net.has(outJ)) continue;
161     if (comp.type === "gsin") return gsinVoltageAt(comp, tSec);
162     if (comp.type === "gimp") return gimpVoltageAt(comp, tSec);
163     if (comp.type === "gsqr") return gsqrVoltageAt(comp, tSec);
164 }
165
166 for (const comp of components) {
167     if (comp.type !== "arduino_uno" || !comp.label) continue;
168     for (const j of net) {
169         if (!j.startsWith(`${comp.label}_`)) continue;
170         const suffix = j.slice(comp.label.length + 1);
171         if (!/^A\d+$|^D\d+$/.test(suffix)) continue;
172         const modes = comp.pinModes || {};
173         const levels = comp.liveLevels || comp.pinLevels || {};
174         if (modes[suffix] === "OUTPUT") return levels[suffix] ? 5 : 0;
175     }
176 }
177
178 const vmV = voltageFromVoltmetersOnNet(net, voltmeters);
179 if (Number.isFinite(vmV)) return vmV;
180
181 if (typeof getVoltageAtJunction === "function") {
182     const v = getVoltageAtJunction(junctionId);
183     if (Number.isFinite(v)) return v;
184 }
185
186 return null;
187 }
188
189 /** Valeurs ADC 0-1023 pour A0-A5 d'une carte UNO. */
190 export function readUnoAnalogInputs(uno, ctx) {
191     const out = {};
192     if (!uno?.label) return out;
193     for (let i = 0; i <= 5; i++) {
194         const label = `A${i}`;
195         const jid = `${uno.label}_${label}`;
196         const v = resolveNetVoltage(jid, ctx);
197         out[label] = voltageToAdc(Number.isFinite(v) ? v : 0);
198     }
199     return out;
200 }
201

```

```

1  /**
2   * Registres GPIO ATmega328P - DDRx / PORTx (Arduino UNO).
3   * Port D : D0-D7 | Port B : D8-D13 | Port C : A0-A5 (D14-D19).
4   */
5
6  export const AVR_GPIO_PORTS = ["B", "C", "D"];
7
8  /** @typedef {{ DDRB: number, DDRC: number, DDRD: number, PORTB: number, PORTC: number, PORTD: number }} AvrRegisters */
9
10 export function createEmptyAvrRegisters() {
11   return { DDRB: 0, DDRC: 0, DDRD: 0, PORTB: 0, PORTC: 0, PORTD: 0 };
12 }
13
14 export function cloneAvrRegisters(reg) {
15   return { ...createEmptyAvrRegisters(), ...(reg || {}) };
16 }
17
18 /** Broches GPIO câblables (D0-D13, A0-A5). */
19 export function allArduinoGpioLabels() {
20   const labels = [];
21   for (let i = 0; i <= 13; i++) labels.push(`D${i}`);
22   for (let i = 0; i <= 5; i++) labels.push(`A${i}`);
23   return labels;
24 }
25
26 /**
27  * @param {number|string} pin - 0-13, 14-19, Dn, An, A0...
28  * @returns {{ port: 'B'|'C'|'D', bit: number, label: string }|null}
29  */
30 export function arduinoPinToPortBit(pin) {
31   if (typeof pin === "string") {
32     const a = pin.match(/^A(\d)$/i);
33     if (a) {
34       const bit = parseInt(a[1], 10);
35       if (bit >= 0 && bit <= 5) return { port: "C", bit, label: `A${bit}` };
36       return null;
37     }
38     const d = pin.match(/^D?(\d+)$/i);
39     if (d) return arduinoPinToPortBit(parseInt(d[1], 10));
40     return null;
41   }
42   const n = pin;
43   if (n >= 0 && n <= 7) return { port: "D", bit: n, label: `D${n}` };
44   if (n >= 8 && n <= 13) return { port: "B", bit: n - 8, label: `D${n}` };
45   if (n >= 14 && n <= 19) return { port: "C", bit: n - 14, label: `A${n - 14}` };
46   return null;
47 }
48
49 export function labelToPortBit(label) {
50   return arduinoPinToPortBit(String(label || ""));
51 }
52
53 export function isPinOutput(reg, label) {
54   const pb = labelToPortBit(label);
55   if (!pb || !reg) return false;
56   return ((reg[`DDR${pb.port}`] || 0) >> pb.bit) & 1;
57 }
58
59 export function isPinInputPullup(reg, label) {
60   const pb = labelToPortBit(label);
61   if (!pb || !reg) return false;
62   if (isPinOutput(reg, label)) return false;
63   return ((reg[`PORT${pb.port}`] || 0) >> pb.bit) & 1;
64 }
65
66 export function getPinPortLevel(reg, label) {
67   const pb = labelToPortBit(label);
68   if (!pb || !reg) return 0;
69   return ((reg[`PORT${pb.port}`] || 0) >> pb.bit) & 1;
70 }
71
72 /** @param {'OUTPUT'|'INPUT'|'INPUT_PULLUP'} mode */
73 export function applyPinModeToRegisters(reg, label, mode) {
74   const pb = labelToPortBit(label);
75   if (!pb || !reg) return;
76   const ddrKey = `DDR${pb.port}`;
77   const portKey = `PORT${pb.port}`;
78   const mask = 1 << pb.bit;

```

```

79   if (mode === "OUTPUT") {
80       reg[ddrKey] = (reg[ddrKey] | mask) & 0xff;
81   } else if (mode === "INPUT_PULLUP") {
82       reg[ddrKey] = reg[ddrKey] & ~mask & 0xff;
83       reg[portKey] = (reg[portKey] | mask) & 0xff;
84   } else {
85       reg[ddrKey] = reg[ddrKey] & ~mask & 0xff;
86       reg[portKey] = reg[portKey] & ~mask & 0xff;
87   }
88 }
89
90 export function applyDigitalWriteToRegisters(reg, label, high) {
91     const pb = labelToPortBit(label);
92     if (!pb || !reg) return;
93     applyPinModeToRegisters(reg, label, "OUTPUT");
94     const portKey = `PORT${pb.port}`;
95     const mask = 1 << pb.bit;
96     if (high) reg[portKey] = (reg[portKey] | mask) & 0xff;
97     else reg[portKey] = reg[portKey] & ~mask & 0xff;
98 }
99
100 export function registersToPinModes(reg) {
101     const modes = {};
102     for (const label of allArduinoGpioLabels()) {
103         if (isPinOutput(reg, label)) modes[label] = "OUTPUT";
104         else if (isPinInputPullup(reg, label)) modes[label] = "INPUT_PULLUP";
105         else modes[label] = "INPUT";
106     }
107     return modes;
108 }
109
110 export function registersToPinLevels(reg) {
111     const levels = {};
112     for (const label of allArduinoGpioLabels()) {
113         if (isPinOutput(reg, label)) levels[label] = getPinPortLevel(reg, label);
114     }
115     return levels;
116 }
117
118 export function formatRegByte(n) {
119     const v = (Number(n) || 0) & 0xff;
120     return `0x${v.toString(16).toUpperCase().padStart(2, "0")}`;
121 }
122
123 export function formatAvrRegistersSummary(reg) {
124     if (!reg) return "";
125     return `DDB${formatRegByte(reg.DDB)} PORTB${formatRegByte(reg.PORTB)} DDRC${formatRegByte(reg.DDRC)} PORTC${formatRegByte(reg.PORTC)}
126     DDBD${formatRegByte(reg.DDBD)} PORTD${formatRegByte(reg.PORTD)}`;
127 }
128
129 function resolveAvrBitToken(token, port) {
130     const t = String(token || "").trim();
131     const num = t.match(/^\d+$/);
132     if (num) {
133         const b = parseInt(num[1], 10);
134         return b >= 0 && b <= 7 ? b : null;
135     }
136     const px = t.match(/^P([BCD])(\d)$/i);
137     if (px && px[1].toUpperCase() === port) return parseInt(px[2], 10);
138     const ddx = t.match(/^DD([BCD])(\d)$/i);
139     if (ddx && ddx[1].toUpperCase() === port) return parseInt(ddx[2], 10);
140     return null;
141 }
142
143 function evalAvrMaskExpr(expr, port) {
144     const s = String(expr || "").trim();
145     if (!s) return null;
146     if (/^0[xX][0-9a-fA-F]+$/.test(s)) return parseInt(s, 16) & 0xff;
147     if (/^0[bB][01]+$/.test(s)) return parseInt(s.slice(2), 2) & 0xff;
148     if (/^B[01]+$/.test(s)) return parseInt(s.slice(1), 2) & 0xff;
149     if (/^\d+$/.test(s)) return parseInt(s, 10) & 0xff;
150
151     const invert = /^~\s*(.+)$/i.test(s) ? true : false;
152     const inner = invert ? s.replace(/^~\s*/i, "").trim() : s;
153
154     let mask = 0;
155     const shiftRe = /(?:\(\s*\)?1\s*<\s*([^\s,|&]+)(?:\s*\)))/gi;
156     let sm;
157     while ((sm = shiftRe.exec(inner)) !== null) {
158         const bit = resolveAvrBitToken(sm[1], port);

```

```

158     if (bit == null) return null;
159     mask |= 1 << bit;
160 }
161 if (mask) return (invert ? ~mask : mask) & 0xff;
162
163 const bv = inner.match(/_BV\s*(\s*[P(BCD)](\d)\s*)/i);
164 if (bv && bv[1].toUpperCase() === port) {
165     const v = (1 << parseInt(bv[2], 10)) & 0xff;
166     return invert ? (~v & 0xff) : v;
167 }
168
169 return null;
170 }
171
172 function avrRegPortFromName(regName) {
173     const m = String(regName || "").match(/^(?:DDR|PORT)([BCD])$/i);
174     return m?.[1]?.toUpperCase() || null;
175 }
176
177 function applyRegOp(reg, regName, op, value) {
178     const port = avrRegPortFromName(regName);
179     if (!port || !AVR_GPIO_PORTS.includes(port)) return;
180     const cur = reg[regName] & 0xff;
181     const v = value & 0xff;
182     if (op === "=") reg[regName] = v;
183     else if (op === "|=") reg[regName] = (cur | v) & 0xff;
184     else if (op === "&=") reg[regName] = (cur & v) & 0xff;
185     else if (op === "^=") reg[regName] = (cur ^ v) & 0xff;
186 }
187
188 /** Parse écritures directes DDRx / PORTx dans le sketch (AVR). */
189 export function parseAvrRegisterWrites(sketch) {
190     const reg = createEmptyAvrRegisters();
191     const src = String(sketch || "");
192     const re = /\b(DDR[BCD]|PORT[BCD])\s*(\|=|\&=|\^=|=\s*{[^\;]+})/gi;
193     let m;
194     while ((m = re.exec(src)) !== null) {
195         const regName = m[1].toUpperCase();
196         const port = avrRegPortFromName(regName);
197         if (!port) continue;
198         const mask = evalAvrMaskExpr(m[3], port);
199         if (mask == null) continue;
200         applyRegOp(reg, regName, m[2], mask);
201     }
202     return reg;
203 }
204
205 /**
206  * Fusionne pinModes / pinLevels / phases dans les registres AVR.
207  * @param {object} parsed – sortie parseArduinoSketch (sans avr)
208  */
209 export function buildAvrRegistersFromParsed(parsed, sketch) {
210     const reg = parseAvrRegisterWrites(sketch);
211
212     for (const [label, mode] of Object.entries(parsed?.pinModes || {})) {
213         if (mode === "OUTPUT" || mode === "INPUT" || mode === "INPUT_PULLUP") {
214             applyPinModeToRegisters(reg, label, mode);
215         }
216     }
217
218     for (const [label, level] of Object.entries(parsed?.pinLevels || {})) {
219         applyDigitalWriteToRegisters(reg, label, !!level);
220     }
221
222     return reg;
223 }
224
225 /** Applique niveaux dynamiques (phase / pulse) sur une copie PORT sans toucher DDR. */
226 export function applyDynamicLevelsToRegisters(reg, levels) {
227     const out = cloneAvrRegisters(reg);
228     for (const [label, level] of Object.entries(levels || {})) {
229         if (!isPinOutput(out, label) && level !== undefined) {
230             applyPinModeToRegisters(out, label, "OUTPUT");
231         }
232         if (isPinOutput(out, label)) {
233             applyDigitalWriteToRegisters(out, label, !!level);
234         }
235     }
236     return out;
237 }

```





```

1  /**
2   * Valeurs idéales GPIO Arduino (5 V / 0 V) pour voltmètre / BCD / 7 segments.
3   */
4
5  import { bcdDigitToSeg7Segments } from "./bcd-seg7.mjs";
6  import {
7    applyArduinoSketchToComponent,
8    arduinoGpioIsTimeVarying,
9    resolvePinLevelsAt,
10 } from "./arduino-sketch-parse.mjs";
11 import { isPinOutput } from "./arduino-avr-registers.mjs";
12 import { reachableJunctions } from "./hc90-cascade.mjs";
13
14 const CD4511_BCD_INPUTS = { A: 0, B: 1, C: 2, D: 3 };
15 const SEG7_OFF = { a: false, b: false, c: false, d: false, e: false, f: false, g: false };
16
17 function isGndJunction(jid) {
18   if (!jid) return false;
19   if (jid.startsWith("GND")) return true;
20   const prefix = jid.split("_")[0];
21   return /^GND\d+$/i.test(prefix);
22 }
23
24 function parseArduinoPinAtJunction(jid, components, pinLevelsByUno) {
25   if (!jid) return null;
26   for (const comp of components) {
27     if (comp.type !== "arduino_uno" || !jid.startsWith(`${comp.label}_`)) continue;
28     const suffix = jid.slice(comp.label.length + 1);
29     if (!/^D\d+|^A\d+$/i.test(suffix)) continue;
30     const out =
31       (comp.avrRegisters && isPinOutput(comp.avrRegisters, suffix)) ||
32       comp.pinModes?.[suffix] === "OUTPUT";
33     if (!out) continue;
34     const levels = pinLevelsByUno?.get(comp.label) || comp.pinLevels || {};
35     const high = levels[suffix] === 1 || levels[suffix] === "1" || levels[suffix] === true;
36     return { pinLabel: suffix, high, volts: high ? 5 : 0 };
37   }
38   return null;
39 }
40
41 function buildPinLevelsByUno(components, tSec = 0) {
42   const map = new Map();
43   for (const comp of components) {
44     if (comp.type !== "arduino_uno") continue;
45     applyArduinoSketchToComponent(comp);
46     map.set(comp.label, resolvePinLevelsAt(comp, tSec));
47   }
48   return map;
49 }
50
51 /** Classifie une jonction : Arduino OUTPUT, masse, ou flottant (inclut jonctions T). */
52 function classifyJunction(jid, components, wires, autoJunctions = [], tSec = 0) {
53   if (!jid) return { kind: "unknown" };
54   const pinLevelsByUno = buildPinLevelsByUno(components, tSec);
55   const net = reachableJunctions(jid, wires, autoJunctions);
56   for (const cur of net) {
57     if (isGndJunction(cur)) return { kind: "gnd", volts: 0 };
58     const ar = parseArduinoPinAtJunction(cur, components, pinLevelsByUno);
59     if (ar) return { kind: "arduino", volts: ar.volts, high: ar.high };
60   }
61   return { kind: "unknown" };
62 }
63
64 export function traceJunctionToIdealVolts(jid, components, wires, tSec = 0, autoJunctions = []) {
65   const c = classifyJunction(jid, components, wires, autoJunctions, tSec);
66   if (c.kind === "gnd") return 0;
67   if (c.kind === "arduino") return c.volts;
68   return null;
69 }
70
71 /** Voltmètre : détecte Arduino vs GND sur _in ou _out (câblage +/- libre). */
72 export function getIdealVoltmeterVoltage(vmLabel, components, wires, tSec = 0, autoJunctions = []) {
73   const out = classifyJunction(`${vmLabel}_out`, components, wires, autoJunctions, tSec);
74   const inn = classifyJunction(`${vmLabel}_in`, components, wires, autoJunctions, tSec);
75
76   if (out.kind === "arduino" && inn.kind === "gnd") return out.volts;
77   if (inn.kind === "arduino" && out.kind === "gnd") return inn.volts;
78   if (out.kind === "arduino" && inn.kind !== "arduino") return out.volts;

```

```

79   if (inn.kind === "arduino" && out.kind !== "arduino") return inn.volts;
80   if (out.kind === "gnd" && inn.kind === "gnd") return 0;
81   return null;
82 }
83
84 function cd4511LabelWiredToSeg7(segLabel, wires, autoJunctions = [], components = []) {
85   for (const seg of ["a", "b", "c", "d", "e", "f", "g"]) {
86     const segJ = `${segLabel}_${seg}`;
87     const net = reachableJunctions(segJ, wires, autoJunctions);
88     for (const jid of net) {
89       for (const comp of components) {
90         if (comp.type !== "cd4511" || !comp.label) continue;
91         if (!jid.startsWith(`${comp.label}_`)) continue;
92         const suffix = jid.slice(comp.label.length + 1);
93         if (/^[a-g]$/.test(suffix)) return comp.label;
94       }
95     }
96   }
97   return null;
98 }
99
100 function resolveCd4511ForSeg7(segLabel, components, wires, autoJunctions = []) {
101   const linked = cd4511LabelWiredToSeg7(segLabel, wires, autoJunctions, components);
102   if (linked) return linked;
103   const seg7s = components.filter((c) => c.type === "seg7");
104   const cd4511s = components.filter((c) => c.type === "cd4511");
105   if (seg7s.length === 1 && cd4511s.length === 1) return cd4511s[0].label;
106   return null;
107 }
108
109 /**
110  * Détecte le câblage CD4511 A...D → UNO D0...D3 (permutation).
111  * @returns {Record<'A' | 'B' | 'C' | 'D', number> | null} bit PORTD (0-3) par entrée BCD
112  */
113 export function detectPortDMappingForCd4511(cd4511Label, unoLabel, wires, autoJunctions = []) {
114   const map = {};
115   for (const name of Object.keys(CD4511_BCD_INPUTS)) {
116     const jid = `${cd4511Label}_${name}`;
117     const net = reachableJunctions(jid, wires, autoJunctions);
118     let dPin = null;
119     for (const j of net) {
120       const m = new RegExp(`^${unoLabel}_D([0-3])$`).exec(j);
121       if (m) {
122         dPin = parseInt(m[1], 10);
123         break;
124       }
125     }
126     if (dPin == null) return null;
127     map[name] = dPin;
128   }
129   return map;
130 }
131
132 function findUnoPortDMappingForCd4511(cd4511Label, components, wires, autoJunctions) {
133   for (const comp of components) {
134     if (comp.type !== "arduino_uno" || !comp.label) continue;
135     applyArduinoSketchToComponent(comp);
136     const map = detectPortDMappingForCd4511(cd4511Label, comp.label, wires, autoJunctions);
137     if (map) return { uno: comp, map };
138   }
139   return null;
140 }
141
142 /** BCD à partir de PORTD et du câblage (A→Dx, B→Dy...). */
143 export function bcdFromPortDRegister(portD, inputToBit) {
144   const port = (Number(portD) || 0) & 0xff;
145   let bcd = 0;
146   for (const [name, bcdBit] of Object.entries(CD4511_BCD_INPUTS)) {
147     const dPin = inputToBit[name];
148     if (dPin == null) return null;
149     if ((port >> dPin) & 1) bcd |= 1 << bcdBit;
150   }
151   return bcd;
152 }
153
154 function bcdFromMappedPortDATime(uno, map, tSec) {
155   if (uno?.liveLevels || arduinoGpioIsTimeVarying(uno)) {
156     const levels = resolvePinLevelsAt(uno, tSec);
157     let bcd = 0;
158     for (const [name, bcdBit] of Object.entries(CD4511_BCD_INPUTS)) {

```

```

159     const dPin = map[name];
160     const label = `D${dPin}`;
161     if (levels[label] === 1 || levels[label] === "1" || levels[label] === true) {
162         bcd |= 1 << bcdBit;
163     }
164 }
165 return bcd;
166 }
167 return bcdFromPortDRegister(uno.avrRegisters?.PORTD ?? 0, map);
168 }
169
170 function bcdFromTrace(cd4511Label, components, wires, tSec, autoJunctions) {
171     let bcd = 0;
172     let any = false;
173     for (const [name, idx] of Object.entries(CD4511_BCD_INPUTS)) {
174         const jid = `${cd4511Label}_${name}`;
175         const v = traceJunctionToIdealVolts(jid, components, wires, tSec, autoJunctions);
176         if (v == null) continue;
177         any = true;
178         if (v >= 2.5) bcd |= 1 << idx;
179     }
180     return any ? bcd : null;
181 }
182
183 export function getIdealArduinoBcdForCd4511(cd4511Label, components, wires, tSec = 0, autoJunctions = []) {
184     const linked = findUnoPortDMappingForCd4511(cd4511Label, components, wires, autoJunctions);
185     if (linked) {
186         return bcdFromMappedPortDATime(linked.uno, linked.map, tSec);
187     }
188     return bcdFromTrace(cd4511Label, components, wires, tSec, autoJunctions);
189 }
190
191 export function getIdealSeg7FromArduino(segLabel, components, wires, tSec = 0, autoJunctions = []) {
192     const cdLabel = resolveCd4511ForSeg7(segLabel, components, wires, autoJunctions);
193     if (!cdLabel) return null;
194     const bcd = getIdealArduinoBcdForCd4511(cdLabel, components, wires, tSec, autoJunctions);
195     if (bcd == null) return null;
196     if (bcd > 9) return { segments: { ...SEG7_OFF }, bcd, blank: true };
197     return { segments: bcdDigitToSeg7Segments(bcd), bcd };
198 }
199
200 const BARGRAPH_LIT_DELTA_V = 0.35;
201
202 /** Bargraph DC10H câblé directement sur des sorties GPIO Arduino (s1...s10, COM→GND). */
203 export function getIdealBargraphFromArduino(barLabel, components, wires, tSec = 0, autoJunctions = []) {
204     if (!barLabel) return null;
205     const vCom = traceJunctionToIdealVolts(`${barLabel}_COM`, components, wires, tSec, autoJunctions);
206     const vc = vCom != null ? vCom : 0;
207     const segments = {};
208     let anyDrive = false;
209     for (let i = 1; i <= 10; i++) {
210         const name = `s${i}`;
211         const v = traceJunctionToIdealVolts(`${barLabel}_${name}`, components, wires, tSec, autoJunctions);
212         if (v == null) {
213             segments[name] = false;
214             continue;
215         }
216         anyDrive = true;
217         segments[name] = v - vc >= BARGRAPH_LIT_DELTA_V;
218     }
219     return anyDrive ? { segments } : null;
220 }
221

```

```

1  /**
2   * Interprétation minimale d'un sketch Arduino (.ino) pour la co-simulation.
3   * pinMode(n, OUTPUT|INPUT|INPUT_PULLUP), digitalWrite(n, HIGH|LOW|1|0), delay(ms).
4   * Registres AVR : DDRB/C/D, PORTB/C/D (D0-D13, A0-A5).
5   */
6
7  import {
8    buildAvrRegistersFromParsed,
9    applyDynamicLevelsToRegisters,
10   registersToPinLevels,
11   registersToPinModes,
12 } from "./arduino-avr-registers.mjs";
13
14 export function resolvePinToken(token) {
15   const t = String(token || "").trim();
16   if (/LED_BUILTIN/i.test(t)) return 13;
17   const a = t.match(/^A(\d+)\$/i);
18   if (a) return 14 + parseInt(a[1], 10);
19   const m = t.match(/\bD?s*(\d+)\b/i);
20   if (m) return parseInt(m[1], 10);
21   return null;
22 }
23
24 function stripComments(src) {
25   return String(src || "")
26     .replace(/\/\[/[^\n]*/g, "")
27     .replace(/\/\[/[^\n]*/g, "");
28 }
29
30 function extractLoopBody(src) {
31   const idx = src.search(/void\s+loop\s*\(\s*\)\s*\{/i);
32   if (idx < 0) return src;
33   let i = src.indexOf("{", idx);
34   if (i < 0) return src;
35   let depth = 0;
36   const start = i + 1;
37   for (; i < src.length; i++) {
38     const ch = src[i];
39     if (ch === "{") depth++;
40     else if (ch === "}") {
41       depth--;
42       if (depth === 0) return src.slice(start, i);
43     }
44   }
45   return src.slice(start);
46 }
47
48 function pinLabel(pin) {
49   if (pin == null || pin < 0) return null;
50   if (pin <= 13) return `D${pin}`;
51   if (pin <= 19) return `A${pin - 14}`;
52   return null;
53 }
54
55 /* -----
56  * Mini-interpréteur de sketch : variables, delay, incréments, écritures PORTx.
57  * Génère des phases temporelles (compteur, séquences) pour l'animation.
58  * ----- */
59
60 function extractFunctionBody(src, name) {
61   const idx = src.search(new RegExp(`\\b(?:void|int|byte)\\s+${name}\\s*\\([^\)]*\\)\\s*\\{`, "i"));
62   if (idx < 0) return "";
63   let i = src.indexOf("{", idx);
64   if (i < 0) return "";
65   let depth = 0;
66   const start = i + 1;
67   for (; i < src.length; i++) {
68     if (src[i] === "{") depth++;
69     else if (src[i] === "}") {
70       depth--;
71       if (depth === 0) return src.slice(start, i);
72     }
73   }
74   return src.slice(start);
75 }
76
77 function parseNumberLiteral(t) {
78   const s = String(t).trim();

```

```

79   if (/^0[xX][0-9a-fA-F]+$/.test(s)) return { v: parseInt(s, 16), f: false };
80   if (/^0[bB][01]+$/.test(s)) return { v: parseInt(s.slice(2), 2), f: false };
81   if (/^B[01]+$/.test(s)) return { v: parseInt(s.slice(1), 2), f: false };
82   if (/^(\d+\.\d*|\.\d+)([eE][+-]?\d+)?$/i.test(s)) return { v: parseFloat(s), f: true };
83   if (/^\d+$/i.test(s)) return { v: parseInt(s, 10), f: false };
84   return null;
85 }
86
87 function evVal(x) {
88   return x && typeof x === "object" && "v" in x ? x.v : (x ?? 0);
89 }
90
91 function evFloat(x) {
92   return !(x && typeof x === "object" && x.f);
93 }
94
95 function evPack(v, f) {
96   return f ? { v, f: true } : v;
97 }
98
99 function tokenizeExpr(expr) {
100   const toks = [];
101   const s = String(expr);
102   let i = 0;
103   const two = ["<<", ">>", "<=", ">=", "==", "!=", "&&", "||"];
104   while (i < s.length) {
105     const c = s[i];
106     if (/\\s/.test(c)) { i++; continue; }
107     if (/[0-9.]/.test(c)) {
108       let j = i + 1;
109       if (c === "0" && (s[i + 1] === "x" || s[i + 1] === "X")) {
110         j = i + 2;
111         while (j < s.length && /[0-9a-fA-F]/.test(s[j])) j++;
112       } else if (c === "0" && (s[i + 1] === "b" || s[i + 1] === "B")) {
113         j = i + 2;
114         while (j < s.length && /[01]/.test(s[j])) j++;
115       } else {
116         while (j < s.length && /[0-9.]/.test(s[j])) j++;
117       }
118       const lit = parseNumberLiteral(s.slice(i, j));
119       if (!lit) return null;
120       toks.push({ t: "num", v: lit.v, f: lit.f });
121       i = j;
122       continue;
123     }
124     if (/[A-Za-z_]/.test(c)) {
125       let j = i + 1;
126       while (j < s.length && /[A-Za-z0-9_]/.test(s[j])) j++;
127       toks.push({ t: "id", v: s.slice(i, j) });
128       i = j;
129       continue;
130     }
131     const pair = s.slice(i, i + 2);
132     if (two.includes(pair)) { toks.push({ t: "op", v: pair }); i += 2; continue; }
133     if ("+-*/%(>|&|^~".includes(c)) { toks.push({ t: "op", v: c }); i++; continue; }
134     return null;
135   }
136   return toks;
137 }
138
139 const BIN_PREC = {
140   "**": 10, "/": 10, "%": 10,
141   "+": 9, "-": 9,
142   "<<": 8, ">>": 8,
143   "<": 7, "<=": 7, ">": 7, ">=": 7,
144   "==" : 6, "!=": 6,
145   "&": 5, "^": 4, "|": 3,
146   "&&": 2, "||": 1,
147 };
148
149 function evalExpr(expr, vars, floatVars = new Set(), regs = {}) {
150   const toks = tokenizeExpr(expr);
151   if (!toks) return 0;
152   const out = [];
153   const ops = [];
154   let prevValue = false;
155   const applyTop = () => {
156     const op = ops.pop();
157     if (op.unary) {
158       const a = out.pop() ?? 0;

```

```

159     const av = evVal(a);
160     if (op.v === "-") out.push(evPack(-av, evFloat(a)));
161     else if (op.v === "!") out.push(av ? 0 : 1);
162     else if (op.v === "~") out.push(~av);
163     return;
164 }
165 const b = out.pop() ?? 0;
166 const a = out.pop() ?? 0;
167 const av = evVal(a);
168 const bv = evVal(b);
169 const fp = evFloat(a) || evFloat(b);
170 switch (op.v) {
171     case "*": out.push(evPack(av * bv, fp)); break;
172     case "/":
173         out.push(evPack(fp ? (bv === 0 ? 0 : av / bv) : (bv === 0 ? 0 : Math.trunc(av / bv)), fp));
174         break;
175     case "%": out.push(bv === 0 ? 0 : av % bv); break;
176     case "+": out.push(evPack(av + bv, fp)); break;
177     case "-": out.push(evPack(av - bv, fp)); break;
178     case "<<": out.push(av << bv); break;
179     case ">>": out.push(av >> bv); break;
180     case "<": out.push(av < bv ? 1 : 0); break;
181     case "<=": out.push(av <= bv ? 1 : 0); break;
182     case ">": out.push(av > bv ? 1 : 0); break;
183     case ">=": out.push(av >= bv ? 1 : 0); break;
184     case "==": out.push(av === bv ? 1 : 0); break;
185     case "!=": out.push(av !== bv ? 1 : 0); break;
186     case "&": out.push(av & bv); break;
187     case "^": out.push(av ^ bv); break;
188     case "|": out.push(av | bv); break;
189     case "&&": out.push(av && bv ? 1 : 0); break;
190     case "||": out.push(av || bv ? 1 : 0); break;
191     default: out.push(0);
192 }
193 };
194 for (const tok of toks) {
195     if (tok.t === "num") { out.push(evPack(tok.v, tok.f)); prevValue = true; continue; }
196     if (tok.t === "id") {
197         const name = tok.v;
198         if (/^(HIGH|true)$/i.test(name)) out.push(1);
199         else if (/^(LOW|false)$/i.test(name)) out.push(0);
200         else if (/^(DDR|PORT)[BCD]$/.test(name)) {
201             out.push((regs[name] ?? 0) & 0xff);
202         } else {
203             const pin = resolvePinToken(name);
204             if (pin !== null) out.push(pin);
205             else {
206                 const raw = vars[name];
207                 const n = Number.isFinite(raw) ? raw : 0;
208                 const asFloat =
209                     floatVars.has(name) ||
210                     (typeof raw === "number" && !Number.isInteger(raw));
211                 out.push(asFloat ? { v: n, f: true } : n);
212             }
213         }
214         prevValue = true;
215         continue;
216     }
217     const v = tok.v;
218     if (v === "(") { ops.push({ v }); prevValue = false; continue; }
219     if (v === ")") {
220         while (ops.length && ops[ops.length - 1].v !== "(") applyTop();
221         ops.pop();
222         prevValue = true;
223         continue;
224     }
225     const unary = !prevValue && (v === "-" || v === "!" || v === "~");
226     if (unary) { ops.push({ v, unary: true, prec: 11 }); prevValue = false; continue; }
227     const prec = BIN_PREC[v] ?? 0;
228     while (
229         ops.length &&
230         ops[ops.length - 1].v !== "(" &&
231         (ops[ops.length - 1].prec ?? BIN_PREC[ops[ops.length - 1].v] ?? 0) >= prec
232     ) {
233         applyTop();
234     }
235     ops.push({ v, prec });
236     prevValue = false;
237 }
238 while (ops.length) {

```

```

239         if (ops[ops.length - 1].v === "(") { ops.pop(); continue; }
240         applyTop();
241     }
242     const r = out[out.length - 1];
243     if (r == null) return 0;
244     return evFloat(r) ? evVal(r) : Math.trunc(evVal(r));
245 }
246
247 const DECL_RE = /\b(?:const\s+)?(?:unsigned\s+)?(?:int|long|byte|char|short|float|double|uint8_t|uint16_t|volatile\s+int|volatile\s+byte))\s
+([A-Za-z_][A-Za-z0-9_]*)\s*=\s*([^\s,;]+)/g;
248 const FLOAT_DECL_RE = /\b(?:const\s+)?(?:float|double)\s+([A-Za-z_][A-Za-z0-9_]*)\s*/g;
249
250 function collectInitialVars(src) {
251     const vars = {};
252     const floatVars = new Set();
253     const globalPart = String(src || "").split(/bvoid\s+setup\s*\(/i)[0];
254     let m;
255     FLOAT_DECL_RE.lastIndex = 0;
256     while ((m = FLOAT_DECL_RE.exec(globalPart)) !== null) {
257         floatVars.add(m[1]);
258         if (!(m[1] in vars)) vars[m[1]] = 0;
259     }
260     DECL_RE.lastIndex = 0;
261     while ((m = DECL_RE.exec(globalPart)) !== null) {
262         const name = m[2];
263         vars[name] = evalExpr(m[3], vars, floatVars);
264         if (/b(?:float|double)\b/i.test(m[1])) floatVars.add(name);
265     }
266     return { vars, floatVars };
267 }
268
269 function parseParenCondition(s, startIdx) {
270     let i = startIdx;
271     const n = s.length;
272     while (i < n && /\s/.test(s[i])) i++;
273     if (s[i] !== "(") return null;
274     let depth = 0;
275     let j = i;
276     for (; j < n; j++) {
277         if (s[j] === "(") depth++;
278         else if (s[j] === ")") { depth--; if (depth === 0) break; }
279     }
280     return { cond: s.slice(i + 1, j), next: j + 1 };
281 }
282
283 const MAX_WHILE_ITER = 10000;
284
285 function parseStatementOrBlock(s, i) {
286     const n = s.length;
287     while (i < n && /\s/.test(s[i])) i++;
288     if (s[i] === "{") {
289         let depth = 0;
290         let j = i;
291         for (; j < n; j++) {
292             if (s[j] === "{") depth++;
293             else if (s[j] === "}") { depth--; if (depth === 0) break; }
294         }
295         return { stmt: parseStatements(s.slice(i + 1, j)), next: j + 1 };
296     }
297     let j = i;
298     let depth = 0;
299     for (; j < n; j++) {
300         const c = s[j];
301         if (c === "(") depth++;
302         else if (c === ")") depth--;
303         else if (c === ";" && depth === 0) break;
304     }
305     const text = s.slice(i, j).trim();
306     return { stmt: text ? [{ type: "expr", text }] : [], next: j + 1 };
307 }
308
309 function parseStatements(body) {
310     const stmts = [];
311     const s = String(body);
312     const n = s.length;
313     let i = 0;
314     while (i < n) {
315         while (i < n && /\s/.test(s[i])) i++;
316         if (i >= n) break;
317         if (s.slice(i, i + 5) === "while" && ![A-Za-z0-9_]/.test(s[i + 5] || "")) {

```



```

118     const condPart = parseParenCondition(s, i + 5);
119     if (!condPart) { i++; continue; }
120     const bodyPart = parseStatementOrBlock(s, condPart.next);
121     stmts.push({ type: "while", cond: condPart.cond, body: bodyPart.stmt });
122     i = bodyPart.next;
123     continue;
124 }
125 if (s.slice(i, i + 2) === "if" && !/[A-Za-z0-9_]/.test(s[i + 2] || "")) {
126     i += 2;
127     const condPart = parseParenCondition(s, i);
128     if (!condPart) continue;
129     const thenPart = parseStatementOrBlock(s, condPart.next);
130     let elsePart = null;
131     let k = thenPart.next;
132     while (k < n && /\s/.test(s[k])) k++;
133     if (s.slice(k, k + 4) === "else" && !/[A-Za-z0-9_]/.test(s[k + 4] || "")) {
134         const r = parseStatementOrBlock(s, k + 4);
135         elsePart = r.stmt;
136         i = r.next;
137     } else {
138         i = thenPart.next;
139     }
140     stmts.push({ type: "if", cond: condPart.cond, body: thenPart.stmt, elseBody: elsePart });
141     continue;
142 }
143 let j = i;
144 let depth = 0;
145 for (; j < n; j++) {
146     const c = s[j];
147     if (c === "(") depth++;
148     else if (c === ")") depth--;
149     else if (c === ";" && depth === 0) break;
150 }
151 const text = s.slice(i, j).trim();
152 if (text) stmts.push({ type: "expr", text });
153 i = j + 1;
154 }
155 return stmts;
156 }
157
158 function resolvePinLabelFromExpr(expr, state) {
159     const t = String(expr || "").trim();
160     if (!t) return null;
161     try {
162         return pinLabel(evalExprState(substituteCalls(t, state), state));
163     } catch {
164         return pinLabel(resolvePinToken(t));
165     }
166 }
167
168 /**
169  * Remplace les appels de fonction connus par leur valeur avant évaluation :
170  * digitalRead(pin), analogRead(pin), millis()/micros(), Serial...
171  */
172 function substituteCalls(expr, state) {
173     let s = String(expr);
174     s = s.replace(/\\bdigitalRead\\s*(\\s*(\\^[\\s\\S]*?)\\s*)\\s*/gi, (_, p) => {
175         const label = resolvePinLabelFromExpr(p, state);
176         const v = label && state.inputs ? state.inputs[label] : undefined;
177         return v === 0 ? "0" : "1";
178     });
179     s = s.replace(/\\banalogRead\\s*(\\s*(\\^[\\s\\S]*?)\\s*)\\s*/gi, (_, p) => {
180         const label = resolvePinLabelFromExpr(p, state);
181         const adc = label && state.analogInputs ? state.analogInputs[label] : undefined;
182         return String(Number.isFinite(adc) ? Math.round(adc) : 0);
183     });
184     s = s.replace(/\\b(?:millis|micros)\\s*(\\s*(\\^[\\s\\S]*?)\\s*)\\s*/gi, String(Math.trunc(state.simTimeMs || 0)));
185     s = s.replace(/\\bSerial\\.available\\s*(\\s*(\\^[\\s\\S]*?)\\s*)\\s*/gi, String(serialAvailable(state)));
186     s = s.replace(/\\bSerial\\.read\\s*(\\s*(\\^[\\s\\S]*?)\\s*)\\s*/gi, String(serialRead(state)));
187     s = s.replace(/\\bSerial\\b/g, "1");
188     return s;
189 }
190
191 function createSerialState() {
192     return { tx: "", rx: [], baud: 9600, begun: false, schedule: [] };
193 }
194
195 function ensureSerial(state) {
196     if (!state.serial) state.serial = createSerialState();
197     return state.serial;
198 }

```

```

398 }
399
400 /** UART matériel UNO : TX=D1 (sortie), RX=D0 (entrée). */
401 function configUartPins(state) {
402     state.regs.DDRD = ((state.regs.DDRD || 0) & ~0x01) | 0x02;
403     state.regs.PORTD = (state.regs.PORTD || 0) | 0x02;
404     state.pins.D1 = 1;
405 }
406
407 function serialAvailable(state) {
408     const ser = state.serial;
409     if (!ser?.begun) return 0;
410     if (ser.rx.length > 0) return ser.rx.length;
411     return 1;
412 }
413
414 function serialRead(state) {
415     const ser = ensureSerial(state);
416     if (!ser.rx.length) return -1;
417     return ser.rx.shift();
418 }
419
420 function unescapeCppString(s) {
421     return String(s)
422         .replace(/\\n/g, "\n")
423         .replace(/\\r/g, "\r")
424         .replace(/\\t/g, "\t")
425         .replace(/\\\\/g, "\\")
426         .replace(/\"/g, '"')
427         .replace(/\'/g, "'");
428 }
429
430 function skipQuotedArg(src, i) {
431     const q = src[i];
432     i++;
433     while (i < src.length) {
434         if (src[i] === "\\") { i += 2; continue; }
435         if (src[i] === q) return i + 1;
436         i++;
437     }
438     return src.length;
439 }
440
441 function extractFirstCallArg(text) {
442     const open = text.indexOf("(");
443     if (open < 0) return null;
444     let depth = 0;
445     let i = open + 1;
446     const start = i;
447     while (i < text.length) {
448         const c = text[i];
449         if (c === '"' || c === "'") { i = skipQuotedArg(text, i); continue; }
450         if (c === "(") { depth++; i++; continue; }
451         if (c === ")") {
452             if (depth === 0) return text.slice(start, i).trim();
453             depth--;
454             i++;
455             continue;
456         }
457         if (c === "," && depth === 0) return text.slice(start, i).trim();
458         i++;
459     }
460     return null;
461 }
462
463 function evalExprState(expr, state) {
464     return evalExpr(substituteCalls(expr, state), state.vars, state.floatVars, state.regs);
465 }
466
467 function formatSerialNumber(n, state, exprText) {
468     const name = String(exprText || "").trim();
469     const isFloat =
470         evFloat(n) ||
471         state.floatVars?.has(name) ||
472         (typeof n === "number" && !Number.isInteger(n));
473     const v = evVal(n);
474     if (isFloat) return v.toFixed(2);
475     return String(Math.trunc(v));
476 }
477

```

```

478 function evalSerialPrintArg(arg, state) {
479     const t = String(arg || "").trim();
480     if (!t) return "";
481     const strM = t.match(/^String\s*\(\s*([\s\S]+?)\s*\)$/i);
482     if (strM) return evalSerialPrintArg(strM[1], state);
483     const concatM = t.match(/^([\s\S]+?)\s*\+\s*([\s\S]+)$/);
484     if (concatM) {
485         return evalSerialPrintArg(concatM[1].trim(), state) + evalSerialPrintArg(concatM[2].trim(), state);
486     }
487     const fM = t.match(/^F\s*\(\s*([\s\S]+?)\s*\)$/i);
488     if (fM) return unescapeCppString(fM[2]);
489     if (t.startsWith('') && t.endsWith('')) return unescapeCppString(t.slice(1, -1));
490     if (t.startsWith('') && t.endsWith('') && t.length > 2) return unescapeCppString(t.slice(1, -1));
491     if (t.startsWith('') && t.length === 3) return t[1];
492     try {
493         const n = evalExprState(t, state);
494         return formatSerialNumber(n, state, t);
495     } catch {
496         return t;
497     }
498 }
499
500 function appendSerialTx(state, text, newline = false) {
501     const ser = ensureSerial(state);
502     if (!ser.begun) {
503         ser.begun = true;
504         ser.baud = ser.baud || 9600;
505         configUartPins(state);
506     }
507     const chunk = String(text) + (newline ? "\n" : "");
508     ser.tx += chunk;
509     ser.schedule.push({ startMs: state.simTimeMs || 0, data: chunk });
510     configUartPins(state);
511 }
512
513 /** Exécute une instruction simple ; renvoie la durée delay() en ms (0 sinon). */
514 function execExprStatement(text, state, onDelay) {
515     const delayM = text.match(/^delay\s*\(\s*(.+?)\s*\)$/i);
516     if (delayM) {
517         const ms = Math.max(0, evalExprState(delayM[1], state));
518         if (onDelay) onDelay(ms);
519         return ms;
520     }
521     if (/^Serial\.begin\s*\(/i.test(text)) {
522         const ser = ensureSerial(state);
523         const arg = extractFirstCallArg(text);
524         ser.baud = arg ? Math.max(300, evalExprState(arg, state)) : 9600;
525         ser.begun = true;
526         configUartPins(state);
527         return 0;
528     }
529     if (/^Serial\.print(?:ln)?\s*\(/i.test(text)) {
530         const ln = /^Serial\.println\s*\(/i.test(text);
531         const arg = extractFirstCallArg(text);
532         appendSerialTx(state, evalSerialPrintArg(arg, state), ln);
533         return 0;
534     }
535     if (/^Serial\.write\s*\(/i.test(text)) {
536         const arg = extractFirstCallArg(text);
537         const code = evalExprState(arg || "0", state) & 0xff;
538         appendSerialTx(state, String.fromCharCode(code), false);
539         return 0;
540     }
541     if (/^digitalWrite\s*\(/i.test(text)) {
542         const m = text.match(/^digitalWrite\s*\(\s*([\^,]+\s*\s*\s*(.+)\s*\)$/i);
543         if (m) {
544             const label = pinLabel(resolvePinToken(m[1]));
545             if (label) state.pins[label] = evalExprState(m[2], state) ? 1 : 0;
546         }
547         return 0;
548     }
549     if (/^pinMode\s*\(/i.test(text)) return 0;
550
551     const localDecl = text.match(
552         /^(?:const\s+)?(?:unsigned\s+)?(int|long|byte|short|float|double)\s+(\[A-Za-z_\][A-Za-z0-9_]*)\s*=\s*([\s\S]+)$/i
553     );
554     if (localDecl) {
555         const type = localDecl[1];
556         const name = localDecl[2];
557         const isFloat = /^(?:float|double)$/i.test(type);

```

```

558     if (isFloat) {
559         state.floatVars = state.floatVars || new Set();
560         state.floatVars.add(name);
561     }
562     const val = evalExprState(localDecl[3], state);
563     state.vars[name] = isFloat ? val : Math.trunc(val);
564     return 0;
565 }
566
567 const localFloatOnly = text.match(/^(?:const\s+)?(?:float|double)\s+([A-Za-z_][A-Za-z0-9_]*)\s*$/i);
568 if (localFloatOnly) {
569     state.floatVars = state.floatVars || new Set();
570     state.floatVars.add(localFloatOnly[1]);
571     if (!(localFloatOnly[1] in state.vars)) state.vars[localFloatOnly[1]] = 0;
572     return 0;
573 }
574
575 let m = text.match(/^[A-Za-z_][A-Za-z0-9_]*\s*(\+|\+|\-|\-|\%|\%|\<=<=>|>=>|>=>|>=>)\s*$/i) ||
576     text.match(/^[A-Za-z_][A-Za-z0-9_]*\s*$/i);
577 if (m) {
578     const name = /[A-Za-z_]/.test(m[1][0]) ? m[1] : m[2];
579     const op = /[A-Za-z_]/.test(m[1][0]) ? m[2] : m[1];
580     state.vars[name] = (state.vars[name] || 0) + (op === "+" ? 1 : -1);
581     return 0;
582 }
583
584 m = text.match(/^(DDR[BCD]|PORT[BCD]|([A-Za-z_][A-Za-z0-9_]*)\s*(\+|\+|\-|\-|\%|\%|\<=<=>|>=>|>=>|>=>)\s*$/i);
585 if (!m) return 0;
586 const target = m[1];
587 const op = m[2];
588 const rhs = evalExprState(m[3], state);
589 const cur = /^(DDR|PORT)[BCD]$/i.test(target)
590     ? state.regs[target] || 0
591     : state.vars[target] || 0;
592 let val;
593 switch (op) {
594     case "=": val = rhs; break;
595     case "|=": val = cur | rhs; break;
596     case "&=": val = cur & rhs; break;
597     case "^=": val = cur ^ rhs; break;
598     case "+=": val = cur + rhs; break;
599     case "-=": val = cur - rhs; break;
600     case "*=": val = cur * rhs; break;
601     case "/=": val = rhs === 0 ? cur : Math.trunc(cur / rhs); break;
602     case "%=": val = rhs === 0 ? cur : cur % rhs; break;
603     case "<<=": val = cur << rhs; break;
604     case ">>=": val = cur >> rhs; break;
605     default: val = rhs;
606 }
607 if (/^(DDR|PORT)[BCD]$/i.test(target)) {
608     state.regs[target] = val & 0xff;
609     if (state.assignedRegs) state.assignedRegs.add(target);
610 } else {
611     const isFloat = state.floatVars?.has(target);
612     state.vars[target] = isFloat ? val : Math.trunc(val);
613 }
614 return 0;
615 }
616
617 function execStatements(stmts, state, onDelay) {
618     for (const st of stmts) {
619         if (st.type === "expr") execExprStatement(st.text, state, onDelay);
620         else if (st.type === "if") {
621             if (evalExprState(st.cond, state)) execStatements(st.body, state, onDelay);
622             else if (st.elseBody) execStatements(st.elseBody, state, onDelay);
623         } else if (st.type === "while") {
624             let guard = 0;
625             while (evalExprState(st.cond, state) && guard < MAX_WHILE_ITER) {
626                 guard++;
627                 execStatements(st.body, state, onDelay);
628             }
629         }
630     }
631 }
632
633 /**
634  * Exécution pas-à-pas du loop() sous forme de générateur : chaque delay(ms)
635  * rendu (yield) permet de suspendre l'exécution et de l'aligner sur le temps réel.
636  */
637 function* runLoopGen(stmts, state) {

```

```

638     for (const st of stmts) {
639         if (st.type === "expr") {
640             const d = execExprStatement(st.text, state);
641             if (d > 0) yield d;
642         } else if (st.type === "if") {
643             if (evalExprState(st.cond, state)) yield* runLoopGen(st.body, state);
644             else if (st.elseBody) yield* runLoopGen(st.elseBody, state);
645         } else if (st.type === "while") {
646             let guard = 0;
647             while (evalExprState(st.cond, state) && guard < MAX_WHILE_ITER) {
648                 guard++;
649                 yield* runLoopGen(st.body, state);
650             }
651         }
652     }
653 }
654
655 const PORT_TO_LABELS = {
656     PORTD: ["D0", "D1", "D2", "D3", "D4", "D5", "D6", "D7"],
657     PORTB: ["D8", "D9", "D10", "D11", "D12", "D13"],
658     PORTC: ["A0", "A1", "A2", "A3", "A4", "A5"],
659 };
660 const DDR_FOR_PORT = { PORTD: "DDRD", PORTB: "DDRB", PORTC: "DDRC" };
661
662 function resolveOutputLevels(state) {
663     const levels = {};
664     for (const [port, labels] of Object.entries(PORT_TO_LABELS)) {
665         const ddr = state.regs[DDR_FOR_PORT[port]] || 0;
666         const val = state.regs[port] || 0;
667         labels.forEach((label, bit) => {
668             if ((ddr >> bit) & 1) levels[label] = (val >> bit) & 1;
669         });
670     }
671     for (const [label, lv] of Object.entries(state.pins)) levels[label] = lv ? 1 : 0;
672     return levels;
673 }
674
675 /* -----
676  * Runtime temps réel : exécute setup() une fois puis loop() en boucle pendant
677  * la simulation (delay(), digitalWrite, registres PORT, digitalRead...).
678  * ----- */
679
680 /** Le sketch définit-il une fonction loop() non vide ? */
681 export function sketchHasLoop(sketch) {
682     const loop = extractFunctionBody(stripComments(sketch || ""), "loop");
683     return !!loop.trim();
684 }
685
686 /** Le loop() lit-il une entrée (digitalRead) ? */
687 export function sketchUsesLiveInput(sketch) {
688     const loop = extractFunctionBody(stripComments(sketch || ""), "loop");
689     return /\bdigitalRead\s*\(/i.test(loop);
690 }
691
692 /** Le loop() utilise-t-il analogRead ? */
693 export function sketchUsesAnalogInput(sketch) {
694     const loop = extractFunctionBody(stripComments(sketch || ""), "loop");
695     return /\banalogRead\s*\(/i.test(loop);
696 }
697
698 export function createArduinoRuntime(uno) {
699     const src = stripComments(uno?.sketch || "");
700     const setupBody = extractFunctionBody(src, "setup");
701     const loopBody = extractFunctionBody(src, "loop");
702     const init = collectInitialVars(src);
703     const state = {
704         vars: init.vars,
705         floatVars: init.floatVars,
706         regs: {},
707         pins: {},
708         inputs: {},
709         analogInputs: {},
710         simTimeMs: 0,
711         serial: createSerialState(),
712     };
713     execStatements(parseStatements(setupBody), state, (ms) => { state.simTimeMs += ms; });
714     return {
715         sketch: uno?.sketch || "",
716         state,
717         loopStmts: parseStatements(loopBody),

```

```

718     gen: null,
719     sleepMs: 0,
720     passDelays: 0,
721     idle: false,
722     lastInputs: null,
723   };
724 }
725
726 /**
727  * Avance le runtime de deltaMs (temps réel), en consommant les delay().
728  * @param {object} rt runtime créé par createArduinoRuntime
729  * @param {number} deltaMs durée écoulée depuis le dernier pas
730  * @param {Record<string, number>} inputs niveaux d'entrée live (ex. { D13: 0 })
731  * @param {Record<string, number>} [analogInputs] valeurs ADC 0-1023 (ex. { A0: 512 })
732  */
733 export function stepArduinoRuntime(rt, deltaMs, inputs, analogInputs) {
734   if (!rt) return;
735   rt.state.inputs = inputs || {};
736   rt.state.analogInputs = analogInputs || {};
737   const inKey = JSON.stringify(rt.state.inputs);
738   if (rt.lastInputs !== inKey) rt.idle = false;
739   rt.lastInputs = inKey;
740   if (rt.idle) return;
741   let budget = Math.min(Math.max(0, deltaMs), 2000);
742   let guard = 0;
743   while (budget > 0 && guard < 200000) {
744     guard++;
745     if (rt.sleepMs > 0) {
746       const c = Math.min(budget, rt.sleepMs);
747       rt.sleepMs -= c;
748       budget -= c;
749       rt.state.simTimeMs += c;
750       if (rt.sleepMs > 0) break;
751       continue;
752     }
753     if (!rt.gen) { rt.gen = runLoopGen(rt.loopStmts, rt.state); rt.passDelays = 0; }
754     const r = rt.gen.next();
755     if (r.done) {
756       rt.gen = null;
757       if (!rt.loopStmts.length) {
758         rt.idle = true;
759         break;
760       }
761       if (rt.passDelays > 0) {
762         rt.passDelays = 0;
763         continue;
764       }
765       // loop() sans delay : une itération par frame (effets cumulatifs, digitalRead...)
766       break;
767     }
768     rt.sleepMs = Math.max(0, Number(r.value) || 0);
769     rt.passDelays++;
770   }
771 }
772
773 export function arduinoRuntimeLevels(rt) {
774   return rt ? resolveOutputLevels(rt.state) : {};
775 }
776
777 export function getRuntimeSerialTx(rt) {
778   return rt?.state?.serial?.tx ?? "";
779 }
780
781 export function injectRuntimeSerialRx(rt, text) {
782   if (!rt) return;
783   const ser = ensureSerial(rt.state);
784   for (const ch of String(text)) ser.rx.push(ch.charCodeAt(0) & 0xff);
785 }
786
787 export function getRuntimeSerialMeta(rt) {
788   const ser = rt?.state?.serial;
789   return { begun: !!ser?.begun, baud: ser?.baud ?? 0 };
790 }
791
792 function formatBindingValue(name, value, floatVars) {
793   const v = typeof value === "number" ? value : Number(value);
794   if (!Number.isFinite(v)) return "0";
795   if (floatVars?.has(name) || !Number.isInteger(v)) return v.toFixed(2);
796   return String(Math.trunc(v));
797 }

```

```

798
799 /**
800  * Évalue les variables locales du loop() (analogRead, calculs float...) pour lcd.print(var).
801  * @param {string} sketch
802  * @param {Record<string, number>} [analogInputs] ex. { A0: 512 }
803  * @returns {Record<string, string>}
804  */
805 export function evaluateLoopVarBindings(sketch, analogInputs = {}) {
806   const src = stripComments(sketch || "");
807   const loopBody = extractFunctionBody(src, "loop");
808   if (!loopBody.trim()) return {};
809   const init = collectInitialVars(src);
810   const state = {
811     vars: { ...init.vars },
812     floatVars: new Set(init.floatVars),
813     regs: {},
814     pins: {},
815     inputs: {},
816     analogInputs: analogInputs || {},
817     simTimeMs: 0,
818   };
819   const bindings = {};
820   for (const st of parseStatements(loopBody)) {
821     if (st.type !== "expr") continue;
822     const text = st.text.trim();
823     const isLocalDecl = /^(?:const\s+)?(?:unsigned\s+)?(?:int|long|byte|short|float|double)\s+[A-Za-z_]\w*\s*=/i.test(text);
824     if (!isLocalDecl) continue;
825     execExprStatement(text, state, null);
826     const m = text.match(
827       /^(?:const\s+)?(?:unsigned\s+)?(?:int|long|byte|short|float|double)\s+([A-Za-z_][A-Za-z0-9_]*)\s*=/i
828     );
829     if (m && Object.prototype.hasOwnProperty.call(state.vars, m[1])) {
830       bindings[m[1]] = formatBindingValue(m[1], state.vars[m[1]], state.floatVars);
831     }
832   }
833   return bindings;
834 }
835
836 /**
837  * Exécute setup() puis loop() une fois et renvoie l'état final des registres.
838  * Résout les variables et expressions (PORTD = x, PORTD = x << 1, etc.).
839  */
840 function interpretRegisterState(src) {
841   const setupBody = extractFunctionBody(src, "setup");
842   const loopBody = extractFunctionBody(src, "loop");
843   if (!setupBody && !loopBody) return null;
844   if (!/\b(?:DDR|PORT)\[BCD]\s*(?:\|=|\^=|\/=|<=<|>=)/.test(`${setupBody};${loopBody}`)) return null;
845   const init = collectInitialVars(src);
846   const state = { vars: init.vars, floatVars: init.floatVars, regs: {}, pins: {}, assignedRegs: new Set() };
847   execStatements(parseStatements(setupBody), state, () => {});
848   execStatements(parseStatements(loopBody), state, () => {});
849   return { regs: state.regs, assigned: state.assignedRegs };
850 }
851
852 /** Exécute le sketch (setup + loop) pour générer des phases temporelles. */
853 function interpretRegisterPhases(src) {
854   const setupBody = extractFunctionBody(src, "setup");
855   const loopBody = extractFunctionBody(src, "loop");
856   if (!loopBody) return null;
857   if (!/\bdelay\s*\(/i.test(loopBody)) return null;
858   // Réserve aux écritures registre PORTx dans loop() (compteurs, séquences).
859   // Les clignotements digitalWrite restent gérés par la logique dédiée.
860   if (!/\bPORT\[BCD]\s*(?:\|=|\^=|\/=|<=<|>=)/.test(loopBody)) return null;
861
862   const init = collectInitialVars(src);
863   const state = { vars: init.vars, floatVars: init.floatVars, regs: {}, pins: {} };
864   execStatements(parseStatements(setupBody), state, () => {});
865
866   const loopStmts = parseStatements(loopBody);
867   const phases = [];
868   const MAX_PHASES = 64;
869   const seen = new Set();
870
871   for (let iter = 0; iter < 256 && phases.length < MAX_PHASES; iter++) {
872     let emitted = false;
873     execStatements(loopStmts, state, (ms) => {
874       if (ms <= 0) return;
875       phases.push({ durationMs: ms, levels: resolveOutputLevels(state) });
876       emitted = true;
877     });

```

```

878     if (!emitted) break;
879     const key = JSON.stringify(state.vars) + "|" + JSON.stringify(state.regs) + "|" + JSON.stringify(state.pins);
880     if (seen.has(key) && phases.length >= 2) break;
881     seen.add(key);
882 }
883
884 return phases.length >= 2 ? phases : null;
885 }
886
887 /**
888  * Niveau logique d'une broche à l'instant tSec (phases / pulsations du sketch).
889  * N'utilise pas liveLevels – adapté à l'oscilloscope et aux courbes idéales.
890  */
891 export function resolveArduinoPinLevelAt(uno, pinLabel, tSec = 0) {
892     if (!uno || !pinLabel) return 0;
893     const dynamic = computeDynamicPinLevels(uno, tSec);
894     if (Object.prototype.hasOwnProperty.call(dynamic, pinLabel)) {
895         return dynamic[pinLabel] ? 1 : 0;
896     }
897     const lv = uno.pinLevels?.[pinLabel];
898     if (lv === 1 || lv === "1" || lv === true) return 1;
899     if (lv === 0 || lv === "0" || lv === false) return 0;
900     return 0;
901 }
902
903 function computeDynamicPinLevels(uno, tSec = 0) {
904     const levels = {};
905     const pinPhases = uno?.pinPhases;
906     if (Array.isArray(pinPhases) && pinPhases.length >= 2) {
907         const totalMs = pinPhases.reduce((s, p) => s + (p.durationMs || 0), 0);
908         if (totalMs > 0) {
909             let rem = (((tSec * 1000) % totalMs) + totalMs) % totalMs;
910             for (const ph of pinPhases) {
911                 const d = ph.durationMs || 0;
912                 if (rem < d) return { ...(ph.levels || {}) };
913                 rem -= d;
914             }
915             return { ...(pinPhases[pinPhases.length - 1].levels || {}) };
916         }
917     }
918     const pinPulses = uno?.pinPulses || {};
919     for (const [pin, pulse] of Object.entries(pinPulses)) {
920         if (pulse?.highSec > 0 && pulse?.lowSec > 0) {
921             const period = pulse.highSec + pulse.lowSec;
922             const phase = ((tSec % period) + period) % period;
923             levels[pin] = phase < pulse.highSec ? 1 : 0;
924         }
925     }
926     return levels;
927 }
928
929 function parseLoopPhases(loopBody) {
930     const phases = [];
931     let current = {};
932     let hadWrite = false;
933     const re =
934         /digitalWrite\s*\(\s*(\[^\,]+\s*\s*)\s*,\s*(HIGH|LOW|1|0)\s*\)\s*|delay\s*\(\s*(\d+)\s*\)/gi;
935     let m;
936     while ((m = re.exec(loopBody)) !== null) {
937         if (/digitalWrite/i.test(m[0])) {
938             const pin = resolvePinToken(m[1]);
939             const label = pinLabel(pin);
940             if (!label) continue;
941             current[label] = /HIGH|1/i.test(m[2]) ? 1 : 0;
942             hadWrite = true;
943             continue;
944         }
945         const delayMs = parseInt(m[3], 10);
946         if (hadWrite && delayMs > 0) {
947             phases.push({ durationMs: delayMs, levels: { ...current } });
948         }
949         hadWrite = false;
950     }
951     return phases;
952 }
953
954 /** Deux phases ou plus sur une seule broche → pulsation carrée (clignotement). */
955 function singlePinPulseFromPhases(phases) {
956     const pins = new Set();
957     for (const ph of phases) {

```



```

958     for (const label of Object.keys(ph.levels || {})) pins.add(label);
959   }
960   if (pins.size !== 1) return null;
961   const label = [...pins][0];
962   let highMs = 0;
963   let lowMs = 0;
964   for (const ph of phases) {
965     if (ph.levels?.[label]) highMs += ph.durationMs || 0;
966     else lowMs += ph.durationMs || 0;
967   }
968   if (highMs <= 0 || lowMs <= 0) return null;
969   return { label, pulse: { highSec: highMs / 1000, lowSec: lowMs / 1000 } };
970 }
971
972 /**
973  * Niveaux GPIO D0-D13 à l'instant tSec (phases loop, pulsations ou statique).
974  * @returns {Record<string, number>}
975  */
976 export function resolvePinLevelsAt(uno, tSec = 0) {
977   if (uno && uno.liveLevels) return { ...uno.liveLevels };
978   const base =
979     uno?.avrRegisters ||
980     buildAvrRegistersFromParsed(
981       {
982         pinModes: uno?.pinModes,
983         pinLevels: uno?.pinLevels,
984         pinPulses: uno?.pinPulses,
985         pinPhases: uno?.pinPhases,
986       },
987       uno?.sketch || ""
988     );
989   const dynamic = computeDynamicPinLevels(uno, tSec);
990   const hasDynamic = Object.keys(dynamic).length > 0;
991   if (!hasDynamic) return registersToPinLevels(base);
992   return registersToPinLevels(applyDynamicLevelsToRegisters(base, dynamic));
993 }
994
995 export function arduinoGpioIsTimeVarying(uno) {
996   if (!uno || uno.type !== "arduino_uno") return false;
997   if (Array.isArray(uno.pinPhases) && uno.pinPhases.length >= 2) return true;
998   return !(uno.pinPulses && Object.keys(uno.pinPulses).length > 0);
999 }
1000
1001 /**
1002  * @returns {{ pinModes: Record<string, string>, pinLevels: Record<string, number>, pinPulses: Record<string, { highSec: number, lowSec: number }>, pinPhases: Array<{ durationMs: number, levels: Record<string, number> }> }}
1003  */
1004 export function parseArduinoSketch(sketch) {
1005   const src = stripComments(sketch);
1006   const pinModes = {};
1007   const pinLevels = {};
1008   const pinPulses = {};
1009   let pinPhases = [];
1010
1011   for (const m of src.matchAll(/pinMode\s*(\s*(\[^\,]+\s*)\s*,\s*(OUTPUT|INPUT_PULLUP|INPUT)\s*\)/gi)) {
1012     const pin = resolvePinToken(m[1]);
1013     const label = pinLabel(pin);
1014     if (!label) continue;
1015     const mode = String(m[2]).toUpperCase();
1016     if (mode === "INPUT_PULLUP") pinModes[label] = "INPUT_PULLUP";
1017     else if (/INPUT/i.test(mode)) pinModes[label] = "INPUT";
1018     else pinModes[label] = "OUTPUT";
1019   }
1020
1021   const loopBody = extractLoopBody(src);
1022   const writes = [];
1023   const writeRe = /digitalWrite\s*(\s*(\[^\,]+\s*)\s*,\s*(HIGH|LOW|1|0)\s*\)/gi;
1024   for (const m of loopBody.matchAll(writeRe)) {
1025     const pin = resolvePinToken(m[1]);
1026     const label = pinLabel(pin);
1027     if (!label) continue;
1028     writes.push({ label, level: /HIGH|1/i.test(m[2]) });
1029   }
1030
1031   const loopPhases = parseLoopPhases(loopBody);
1032   let blinkApplied = false;
1033
1034   if (loopPhases.length >= 2) {
1035     const singlePulse = singlePinPulseFromPhases(loopPhases);
1036     if (singlePulse) {

```

```

1037     pinModes[singlePulse.label] = "OUTPUT";
1038     pinPulses[singlePulse.label] = singlePulse.pulse;
1039     delete pinLevels[singlePulse.label];
1040     blinkApplied = true;
1041   } else {
1042     pinPhases = loopPhases;
1043     for (const ph of pinPhases) {
1044       for (const label of Object.keys(ph.levels || {})) {
1045         pinModes[label] = "OUTPUT";
1046         delete pinLevels[label];
1047         delete pinPulses[label];
1048       }
1049     }
1050   }
1051 } else {
1052   const stepRe =
1053     /digitalWrite\s*\(\s*([^\s,]+)\s*,\s*(HIGH|LOW|1|0)\s*\)\s*;\s*delay\s*\(\s*(\d+)\s*\)/gi;
1054   const steps = [];
1055   let match;
1056   while ((match = stepRe.exec(loopBody)) !== null) {
1057     const pin = resolvePinToken(match[1]);
1058     const label = pinLabel(pin);
1059     if (!label) continue;
1060     steps.push({
1061       label,
1062       level: /HIGH|1/i.test(match[2]),
1063       delayMs: parseInt(match[3], 10),
1064     });
1065   }
1066
1067   if (steps.length >= 2) {
1068     const labels = [...new Set(steps.map((s) => s.label))];
1069     if (labels.length === 1) {
1070       const label = labels[0];
1071       let highMs = 0;
1072       let lowMs = 0;
1073       for (const s of steps) {
1074         if (s.level) highMs += s.delayMs;
1075         else lowMs += s.delayMs;
1076       }
1077       if (highMs > 0 && lowMs > 0) {
1078         pinModes[label] = "OUTPUT";
1079         pinPulses[label] = { highSec: highMs / 1000, lowSec: lowMs / 1000 };
1080         delete pinLevels[label];
1081         blinkApplied = true;
1082       }
1083     }
1084   }
1085
1086   if (!blinkApplied && loopPhases.length === 1) {
1087     for (const [label, level] of Object.entries(loopPhases[0].levels || {})) {
1088       pinModes[label] = "OUTPUT";
1089       pinLevels[label] = level ? 1 : 0;
1090       delete pinPulses[label];
1091     }
1092   } else if (!blinkApplied && writes.length > 0) {
1093     const lastByPin = {};
1094     for (const w of writes) lastByPin[w.label] = w.level;
1095     for (const [label, level] of Object.entries(lastByPin)) {
1096       pinModes[label] = "OUTPUT";
1097       pinLevels[label] = level ? 1 : 0;
1098       delete pinPulses[label];
1099     }
1100   }
1101 }
1102
1103 if (pinPhases.length < 2) {
1104   const regPhases = interpretRegisterPhases(src);
1105   if (regPhases && regPhases.length >= 2) {
1106     pinPhases = regPhases;
1107     for (const ph of pinPhases) {
1108       for (const label of Object.keys(ph.levels || {})) {
1109         pinModes[label] = "OUTPUT";
1110         delete pinLevels[label];
1111         delete pinPulses[label];
1112       }
1113     }
1114   }
1115 }
1116

```

```

1117     for (const label of Object.keys(pinModes)) {
1118         if (pinModes[label] !== "OUTPUT") {
1119             delete pinLevels[label];
1120             delete pinPulses[label];
1121             continue;
1122         }
1123         if (pinLevels[label] === undefined && !pinPulses[label] && pinPhases.length < 2) {
1124             pinLevels[label] = 0;
1125         }
1126     }
1127
1128     if (/^Serial\.begin\s*\(/.test(src)) {
1129         pinModes.D1 = "OUTPUT";
1130         pinLevels.D1 = 1;
1131         if (!pinModes.D0) pinModes.D0 = "INPUT_PULLUP";
1132     }
1133
1134     const avrRegisters = buildAvrRegistersFromParsed(
1135         { pinModes, pinLevels, pinPulses, pinPhases },
1136         sketch
1137     );
1138
1139     // L'interpréteur résout les variables/expressions (PORTD = x, PORTD = x << 1...)
1140     // que l'analyse statique ne peut pas évaluer. On écrase les registres touchés.
1141     // Ne pas écraser les registres si des phases temporelles existent (sinon bargraph figé).
1142     const execState = interpretRegisterState(src);
1143     if (execState && pinPhases.length < 2) {
1144         for (const reg of execState.assigned) {
1145             avrRegisters[reg] = (execState.regs[reg] || 0) & 0xff;
1146         }
1147     }
1148
1149     const syncedModes = registersToPinModes(avrRegisters);
1150     const syncedLevels = registersToPinLevels(avrRegisters);
1151     for (const label of Object.keys(syncedModes)) {
1152         if (syncedModes[label] !== "OUTPUT") {
1153             delete syncedLevels[label];
1154             continue;
1155         }
1156         if (pinPulses[label] || (pinPhases.length >= 2 && pinPhases.some((ph) => label in (ph.levels || {})))) {
1157             delete syncedLevels[label];
1158         }
1159     }
1160
1161     return {
1162         pinModes: syncedModes,
1163         pinLevels: syncedLevels,
1164         pinPulses,
1165         pinPhases,
1166         avrRegisters,
1167         usesLiveInput: sketchUsesLiveInput(sketch),
1168     };
1169 }
1170
1171 export function applyArduinoSketchToComponent(comp) {
1172     if (!comp || comp.type !== "arduino_uno") return;
1173     const parsed = parseArduinoSketch(comp.sketch || "");
1174     comp.pinModes = parsed.pinModes;
1175     comp.pinLevels = parsed.pinLevels;
1176     comp.pinPulses = parsed.pinPulses;
1177     comp.pinPhases = parsed.pinPhases || [];
1178     comp.avrRegisters = parsed.avrRegisters;
1179     comp.usesLiveInput = parsed.usesLiveInput;
1180 }
1181
1182 export function arduinoUnoMinPulsePeriodSec(components) {
1183     let min = Infinity;
1184     for (const c of components) {
1185         if (c.type !== "arduino_uno") continue;
1186         if (Array.isArray(c.pinPhases) && c.pinPhases.length >= 2) {
1187             for (const ph of c.pinPhases) {
1188                 const d = (ph.durationMs || 0) / 1000;
1189                 if (d > 0) min = Math.min(min, d);
1190             }
1191         }
1192         if (!c.pinPulses) continue;
1193         for (const pulse of Object.values(c.pinPulses)) {
1194             const period = (pulse.highSec || 0) + (pulse.lowSec || 0);
1195             if (period > 0) min = Math.min(min, period);
1196         }
1197     }
1198 }

```

```
1197     }  
1198     return Number.isFinite(min) ? min : 0;  
1199 }  
1200
```

```

1  /**
2   * Forme d'onde UART 8N1 (TX=D1) pour l'oscilloscope.
3   */
4
5  import {
6    createArduinoRuntime,
7    stepArduinoRuntime,
8    sketchHasLoop,
9  } from "../arduino-sketch-parse.mjs";
10
11 export function sketchUsesSerial(sketch) {
12   return /\bSerial\.begin\s*\(/i.test(String(sketch || ""));
13 }
14
15 /** Bits 8N1 LSB-first : start 0, 8 data, stop 1. */
16 export function uartByteBits(byte) {
17   const code = byte & 0xff;
18   const bits = [0];
19   for (let i = 0; i < 8; i++) bits.push((code >> i) & 1);
20   bits.push(1);
21   return bits;
22 }
23
24 /**
25  * Niveau TX (0/1) à l'instant tMs dans une trame.
26  * @param {Array<{ startMs: number, data: string }>} schedule
27  */
28 export function uartTxBitAt(schedule, baud, tMs) {
29   if (!schedule?.length || !baud || baud <= 0) return 1;
30   const bitMs = 1000 / baud;
31   for (const ev of schedule) {
32     let cursor = ev.startMs;
33     const data = String(ev.data || "");
34     for (let i = 0; i < data.length; i++) {
35       for (const bit of uartByteBits(data.charCodeAt(i))) {
36         if (tMs >= cursor && tMs < cursor + bitMs) return bit;
37         cursor += bitMs;
38       }
39     }
40   }
41   return 1;
42 }
43
44 /** Niveau TX avec période de boucle (affichage stable sur l'oscillo). */
45 export function uartTxBitAtLoop(schedule, baud, tMs, loopPeriodMs) {
46   if (!loopPeriodMs || loopPeriodMs <= 0) return uartTxBitAt(schedule, baud, tMs);
47   const tMod = ((tMs % loopPeriodMs) + loopPeriodMs) % loopPeriodMs;
48   const bitMs = 1000 / baud;
49   for (const ev of schedule) {
50     const start = ((ev.startMs % loopPeriodMs) + loopPeriodMs) % loopPeriodMs;
51     let cursor = start;
52     const data = String(ev.data || "");
53     for (let i = 0; i < data.length; i++) {
54       for (const bit of uartByteBits(data.charCodeAt(i))) {
55         let end = cursor + bitMs;
56         if (end <= loopPeriodMs) {
57           if (tMod >= cursor && tMod < end) return bit;
58         } else if (tMod >= cursor || tMod < end - loopPeriodMs) {
59           return bit;
60         }
61         cursor = end;
62         if (cursor >= loopPeriodMs) cursor -= loopPeriodMs;
63       }
64     }
65   }
66   return 1;
67 }
68
69 /** Somme des delay() dans void loop() – sans exécuter le sketch. */
70 export function uartSketchLoopPeriodMs(sketch) {
71   const loopBody = String(sketch || "").match(/void\s+loop\s*\(\s*\)\s*\{\s*([\s\S]*?)\s*\}/i)?.[1] || "";
72   let delaySum = 0;
73   for (const m of loopBody.matchAll(/delay\s*\(\s*(\d+)\s*\)/gi)) {
74     delaySum += parseInt(m[1], 10);
75   }
76   return delaySum;
77 }
78

```

```

79 function parseSerialBaud(sketch) {
80   const m = String(sketch || "").match(/bSerial\.begin\s*(\s*(\d+)/i);
81   return m ? parseInt(m[1], 10) : 0;
82 }
83
84 /** Période de boucle UART pour réglage auto scope (s), sans rejouer le runtime. */
85 export function arduinoUartScopePeriodSec(components) {
86   let min = Infinity;
87   for (const c of components) {
88     if (c.type !== "arduino_uno") continue;
89     if (!sketchUsesSerial(c.sketch)) continue;
90     const loopMs = uartSketchLoopPeriodMs(c.sketch);
91     if (loopMs > 0) {
92       min = Math.min(min, loopMs / 1000);
93       continue;
94     }
95     const baud = parseSerialBaud(c.sketch) || 9600;
96     min = Math.min(min, uartCharDurationSec(baud) * 2);
97   }
98   return Number.isFinite(min) ? min : 0;
99 }
100
101 function inferLoopPeriodMs(sketch, schedule) {
102   const delaySum = uartSketchLoopPeriodMs(sketch);
103   if (delaySum > 0) return delaySum;
104
105   const loopEvents = schedule.filter((ev) => ev.loopPhase);
106   if (loopEvents.length >= 2) {
107     const d = loopEvents[1].startMs - loopEvents[0].startMs;
108     if (d > 0) return d;
109   }
110   if (schedule.length >= 2) {
111     const d = schedule[schedule.length - 1].startMs - schedule[0].startMs;
112     if (d > 0) return d;
113   }
114   return 0;
115 }
116
117 /**
118  * Rejoue le sketch pour construire le planning des émissions UART.
119  * @returns {{ schedule: Array<{startMs:number,data:string}>, baud: number, loopPeriodMs: number }}
120  */
121 export function buildUartScopeModel(unso, inputs = {}, replayMs = 3500) {
122   if (!unso || !sketchUsesSerial(unso.sketch)) {
123     return { schedule: [], baud: 9600, loopPeriodMs: 0 };
124   }
125   const rt = createArduinoRuntime(unso);
126   const ser = rt.state.serial;
127   const setupEndMs = rt.state.simTimeMs;
128   let guard = 0;
129   while (rt.state.simTimeMs < replayMs && guard < 50000) {
130     stepArduinoRuntime(rt, 2, inputs);
131     guard++;
132   }
133   const schedule = (ser?.schedule || []).map((ev) => ({
134     ...ev,
135     loopPhase: ev.startMs >= setupEndMs - 0.5,
136   }));
137   const baud = ser?.baud > 0 ? ser.baud : 9600;
138   const loopPeriodMs = sketchHasLoop(unso.sketch)
139     ? inferLoopPeriodMs(unso.sketch, schedule)
140     : 0;
141   return { schedule, baud, loopPeriodMs };
142 }
143
144 export function uartScopeBitAt(model, tMs) {
145   if (!model?.schedule?.length) return 1;
146   if (model.loopPeriodMs > 0) {
147     return uartTxBitAtLoop(model.schedule, model.baud, tMs, model.loopPeriodMs);
148   }
149   return uartTxBitAt(model.schedule, model.baud, tMs);
150 }
151
152 /** Durée d'une trame 8N1 (s) pour réglage de la base de temps. */
153 export function uartCharDurationSec(baud) {
154   if (!baud || baud <= 0) return 0;
155   return 10 / baud;
156 }
157

```

```

1  /**
2   * Arduino UNO R3 – modèle SPICE simplifié pour co-simulation avec le circuit.
3   * Alimentation : 5 V / 3,3 V / masse sur les broches correspondantes.
4   * GPIO sortie : source DC ou PULSE selon le sketch interprété (pinModes / pinLevels / pinPulses).
5   */
6
7  import { appendI2cBusNetlist } from "./i2c-bus-ideal.mjs";
8
9  function formatSpiceTime(seconds) {
10     const s = Math.abs(Number(seconds));
11     if (!Number.isFinite(s)) return "1n";
12     if (s === 0) return "0";
13     if (s >= 1) return String(s);
14     if (s >= 1e-3) return `${(s * 1e3).toFixed(6)}m`;
15     if (s >= 1e-6) return `${(s * 1e6).toFixed(6)}u`;
16     if (s >= 1e-9) return `${(s * 1e9).toFixed(6)}n`;
17     return `${s.toExponential(3)}`;
18 }
19
20 function pinVoltageFromPhases(pinPhases, pinName) {
21     return pinPhases.map((ph) => (ph.levels?.[pinName] ? 5 : 0));
22 }
23
24 /** Carré périodique dérivé des phases (HIGH/LOW cumulés sur une période). */
25 function pinPhaseToPulse(pinPhases, pinName) {
26     const levels = pinVoltageFromPhases(pinPhases, pinName);
27     if (levels.length < 2 || levels.every((v) => v === levels[0])) return null;
28     let highMs = 0;
29     let lowMs = 0;
30     for (let i = 0; i < pinPhases.length; i++) {
31         const d = pinPhases[i].durationMs || 0;
32         if (levels[i]) highMs += d;
33         else lowMs += d;
34     }
35     if (highMs <= 0 || lowMs <= 0) return null;
36     const highSec = highMs / 1000;
37     const lowSec = lowMs / 1000;
38     const period = highSec + lowSec;
39     const delaySec = levels[0] ? 0 : lowSec;
40     return { highSec, lowSec, period, delaySec };
41 }
42
43 function appendPinPhasePulse(c, pinName, nPin, pinPhases, lines, spiceBranchName) {
44     const pulse = pinPhaseToPulse(pinPhases, pinName);
45     if (!pulse) return false;
46     lines.push(
47         `${spiceBranchName("V", `${c.id}_${pinName}`)} ${nPin} 0 PULSE(0 5 ${formatSpiceTime(pulse.delaySec)} 1n 1n ${formatSpiceTime(pulse.highSec)} ${formatSpiceTime(pulse.period)})`
48     );
49     return true;
50 }
51
52 /** Séquence PWL pour motifs non périodiques (compteurs PORTD, bargraph...). */
53 function appendPinPhasePwl(c, pinName, nPin, pinPhases, lines, spiceBranchName) {
54     const levels = pinVoltageFromPhases(pinPhases, pinName);
55     if (!levels.length) return false;
56     if (levels.every((v) => v === levels[0])) {
57         lines.push(`${spiceBranchName("V", `${c.id}_${pinName}`)} ${nPin} 0 DC ${levels[0]}`);
58         return true;
59     }
60     if (appendPinPhasePulse(c, pinName, nPin, pinPhases, lines, spiceBranchName)) return true;
61     let t = 0;
62     const parts = [];
63     for (let i = 0; i < pinPhases.length; i++) {
64         parts.push(formatSpiceTime(t), String(levels[i]));
65         t += (pinPhases[i].durationMs || 0) / 1000;
66     }
67     if (t <= 0) return false;
68     parts.push(formatSpiceTime(t), String(levels[levels.length - 1]));
69     lines.push(
70         `${spiceBranchName("V", `${c.id}_${pinName}`)} ${nPin} 0 PWL(${parts.join(" ")}`
71     );
72     return true;
73 }
74 export const UNO_PIN = {
75     IOREF: 0,
76     RESET: 1,
77     "3V3": 2,

```

```

78     "5V": 3,
79     GND: 4,
80     GND2: 5,
81     VIN: 6,
82     A0: 7,
83     A1: 8,
84     A2: 9,
85     A3: 10,
86     A4: 11,
87     A5: 12,
88     D13: 13,
89     D12: 14,
90     D11: 15,
91     D10: 16,
92     D9: 17,
93     D8: 18,
94     D7: 19,
95     D6: 20,
96     D5: 21,
97     D4: 22,
98     D3: 23,
99     D2: 24,
100    D1: 25,
101    D0: 26,
102 };
103
104 export const UNO_PIN_COUNT = 27;
105
106 export function isArduinoUnoType(t) {
107     return t === "arduino_uno";
108 }
109
110 export function arduinoUnoTerminalKeys(c) {
111     const keys = [];
112     for (let i = 0; i < UNO_PIN_COUNT; i++) keys.push(`${c.id}#${i}`);
113     return keys;
114 }
115
116 /** Broches mesurées pour animation logique (D0..D13). */
117 export function arduinoUnoDigitalPinIndices() {
118     return [
119         UNO_PIN.D0, UNO_PIN.D1, UNO_PIN.D2, UNO_PIN.D3, UNO_PIN.D4, UNO_PIN.D5,
120         UNO_PIN.D6, UNO_PIN.D7, UNO_PIN.D8, UNO_PIN.D9, UNO_PIN.D10, UNO_PIN.D11,
121         UNO_PIN.D12, UNO_PIN.D13,
122     ];
123 }
124
125 export function arduinoUnoDigitalPinName(pinIndex) {
126     const map = {
127         [UNO_PIN.D0]: "D0", [UNO_PIN.D1]: "D1", [UNO_PIN.D2]: "D2", [UNO_PIN.D3]: "D3",
128         [UNO_PIN.D4]: "D4", [UNO_PIN.D5]: "D5", [UNO_PIN.D6]: "D6", [UNO_PIN.D7]: "D7",
129         [UNO_PIN.D8]: "D8", [UNO_PIN.D9]: "D9", [UNO_PIN.D10]: "D10", [UNO_PIN.D11]: "D11",
130         [UNO_PIN.D12]: "D12", [UNO_PIN.D13]: "D13",
131     };
132     return map[pinIndex] || `P${pinIndex}`;
133 }
134
135 /** Broches GPIO simulées (D0..D13, A0..A5). */
136 export function arduinoUnoGpioPinIndices() {
137     return [
138         ...arduinoUnoDigitalPinIndices(),
139         UNO_PIN.A0, UNO_PIN.A1, UNO_PIN.A2, UNO_PIN.A3, UNO_PIN.A4, UNO_PIN.A5,
140     ];
141 }
142
143 export function arduinoUnoGpioPinName(pinIndex) {
144     const digital = arduinoUnoDigitalPinName(pinIndex);
145     if (digital.startsWith("D")) return digital;
146     const analog = {
147         [UNO_PIN.A0]: "A0", [UNO_PIN.A1]: "A1", [UNO_PIN.A2]: "A2",
148         [UNO_PIN.A3]: "A3", [UNO_PIN.A4]: "A4", [UNO_PIN.A5]: "A5",
149     };
150     return analog[pinIndex] || `P${pinIndex}`;
151 }
152
153 /**
154  * @param {object} c composant moteur
155  * @param {(key: string) => string} nodeFor
156  * @param {string[]} lines
157  * @param {(prefix: string, id: string) => string} spiceBranchName

```



```

158 */
159 export function appendArduinoUnoNetlist(c, nodeFor, lines, spiceBranchName, opts = {}) {
160     const id = c.id;
161     const n5 = nodeFor(`${id}#${UNO_PIN["5V"]}`);
162     const n33 = nodeFor(`${id}#${UNO_PIN["3V3"]}`);
163     const ng1 = nodeFor(`${id}#${UNO_PIN.GND}`);
164     const ng2 = nodeFor(`${id}#${UNO_PIN.GND2}`);
165     const nioref = nodeFor(`${id}#${UNO_PIN.IOREF}`);
166     const nreset = nodeFor(`${id}#${UNO_PIN.RESET}`);
167
168     lines.push(`* ${id} Arduino UNO (rails + GPIO passifs)`);
169     lines.push(`${spiceBranchName("R", `${id}_g1`) ${ng1} 0 1m`);
170     lines.push(`${spiceBranchName("R", `${id}_g2`) ${ng2} 0 1m`);
171     lines.push(`${spiceBranchName("V", `${id}_5v`) ${n5} 0 DC 5`);
172     lines.push(`${spiceBranchName("V", `${id}_3v3`) ${n33} 0 DC 3.3`);
173     lines.push(`${spiceBranchName("R", `${id}_ioref`) ${nioref} ${n5} 100`);
174     lines.push(`${spiceBranchName("R", `${id}_rstpu`) ${nreset} ${n5} 10000`);
175
176     const pinModes = c.pinModes && typeof c.pinModes === "object" ? c.pinModes : {};
177     const pinLevels = c.pinLevels && typeof c.pinLevels === "object" ? c.pinLevels : {};
178     const pinPulses = c.pinPulses && typeof c.pinPulses === "object" ? c.pinPulses : {};
179     const pinPhases = Array.isArray(c.pinPhases) ? c.pinPhases : [];
180     const hasMultiPhase = pinPhases.length >= 2;
181
182     for (const pinIdx of arduinoUnoGpioPinIndices()) {
183         const pinName = arduinoUnoGpioPinName(pinIdx);
184         if (c.i2cBus?.active && (pinName === "A4" || pinName === "A5")) continue;
185         const nPin = nodeFor(`${id}#${pinIdx}`);
186         if (pinModes[pinName] === "OUTPUT") {
187             const pulse = pinPulses[pinName];
188             if (pulse && pulse.highSec > 0 && pulse.lowSec > 0) {
189                 const period = pulse.highSec + pulse.lowSec;
190                 const ton = pulse.highSec;
191                 lines.push(
192                     `${spiceBranchName("V", `${id}_${pinName}`) ${nPin} 0 PULSE(0 5 0 1n 1n ${formatSpiceTime(ton)} ${formatSpiceTime(perio
193 d))}`
194                 );
195             } else if (hasMultiPhase) {
196                 const levels = pinVoltageFromPhases(pinPhases, pinName);
197                 const constant = levels.length > 0 && levels.every((v) => v === levels[0]);
198                 if (constant) {
199                     lines.push(`${spiceBranchName("V", `${id}_${pinName}`) ${nPin} 0 DC ${levels[0]}`);
200                 } else if (!appendPinPhasePwl(c, pinName, nPin, pinPhases, lines, spiceBranchName)) {
201                     const high = pinLevels[pinName] === 1 || pinLevels[pinName] === "1" || pinLevels[pinName] === true;
202                     lines.push(`${spiceBranchName("V", `${id}_${pinName}`) ${nPin} 0 DC ${high ? 5 : 0}`);
203                 }
204             } else {
205                 const high = pinLevels[pinName] === 1 || pinLevels[pinName] === "1" || pinLevels[pinName] === true;
206                 const v = high ? 5 : 0;
207                 lines.push(`${spiceBranchName("V", `${id}_${pinName}`) ${nPin} 0 DC ${v}`);
208             }
209         } else if (pinModes[pinName] === "INPUT_PULLUP") {
210             lines.push(`${spiceBranchName("R", `${id}_${pinName}_pu`) ${n5} ${nPin} 30000`);
211         } else {
212             lines.push(`${spiceBranchName("R", `${id}_${pinName}_z`) ${nPin} 0 10Meg`);
213         }
214     }
215
216     if (c.i2cBus?.active) {
217         appendI2cBusNetlist(c, nodeFor, lines, spiceBranchName, opts.i2cRepeatSec ?? 0.02);
218     }
219 }

```

```

1  /** BCD 0-9 → segments 7 segments (cathode commune, 1 = segment allumé). */
2
3  const SEG7_NAMES = ["a", "b", "c", "d", "e", "f", "g"];
4
5  const BCD_SEGMENTS = [
6    [1, 1, 1, 1, 1, 1, 0],
7    [0, 1, 1, 0, 0, 0, 0],
8    [1, 1, 0, 1, 1, 0, 1],
9    [1, 1, 1, 1, 0, 0, 1],
10   [0, 1, 1, 0, 0, 1, 1],
11   [1, 0, 1, 1, 0, 1, 1],
12   [1, 0, 1, 1, 1, 1, 1],
13   [1, 1, 1, 0, 0, 0, 0],
14   [1, 1, 1, 1, 1, 1, 1],
15   [1, 1, 1, 1, 0, 1, 1],
16 ];
17
18 export function bcdDigitToSeg7Segments(digit) {
19   const d = Math.min(9, Math.max(0, Math.floor(Number(digit) || 0)));
20   const row = BCD_SEGMENTS[d];
21   const segments = {};
22   SEG7_NAMES.forEach((name, i) => {
23     segments[name] = !!row[i];
24   });
25   return segments;
26 }
27
28 /** @param {number[]} qVoltages [Q0,Q1,Q2,Q3] */
29 export function bcdFromQVoltages(qVoltages, vth = 2.5) {
30   let n = 0;
31   for (let i = 0; i < 4; i++) {
32     const v = qVoltages[i];
33     if (Number.isFinite(v) && v > vth) n |= 1 << i;
34   }
35   return n;
36 }
37
38 /** Retrouve le chiffre 0-9 à partir des segments allumés (afficheur). */
39 export function seg7PatternToDigit(segments) {
40   if (!segments || typeof segments !== "object") return null;
41   for (let d = 0; d <= 9; d++) {
42     const ref = bcdDigitToSeg7Segments(d);
43     if (SEG7_NAMES.every((n) => !!segments[n] === ref[n])) return d;
44   }
45   return null;
46 }
47
48 export { SEG7_NAMES };
49

```

```

1  /**
2   * Génère pont-diode-double-alternance.json et valide la netlist.
3   * node Simulateur/Engine/build-bridge-json.mjs
4   */
5  import { writeFileSync } from "fs";
6  import { join } from "path";
7  import { buildNetlistFromGraphicalState } from "../schematic-to-spice.mjs";
8
9  const GRID = 20;
10 const snap = (v) => Math.round(v / GRID) * GRID;
11
12 function jonctions(comp) {
13   const rad = ((comp.rotation || 0) * Math.PI) / 180;
14   const local = [];
15   const label = comp.label;
16   if (comp.type === "gsin") {
17     local.push({ id: `${label}_in`, x: 0, y: 40 });
18     local.push({ id: `${label}_out`, x: 40, y: 0 });
19   } else if (comp.type === "gnd") {
20     local.push({ id: `${label}_out`, x: GRID, y: 0 });
21   } else if (comp.type === "diode") {
22     local.push({ id: `${label}_in`, x: -40, y: 0 });
23     local.push({ id: `${label}_out`, x: 40, y: 0 });
24   } else if (["resistor", "capacitor", "voltmeter"].includes(comp.type)) {
25     local.push({ id: `${label}_in`, x: -40, y: 0 });
26     local.push({ id: `${label}_out`, x: 40, y: 0 });
27   } else if (comp.type === "oscilloscope") {
28     local.push({ id: `${label}_CH1`, x: -60, y: -20 });
29     local.push({ id: `${label}_CH2`, x: -60, y: 20 });
30     local.push({ id: `${label}_GND`, x: 0, y: 60 });
31   }
32   const out = {};
33   for (const pt of local) {
34     let lx = pt.x;
35     let ly = pt.y;
36     if (!["gsin", "oscilloscope"].includes(comp.type)) {
37       const rx = lx * Math.cos(rad) - ly * Math.sin(rad);
38       const ry = lx * Math.sin(rad) + ly * Math.cos(rad);
39       lx = rx;
40       ly = ry;
41     }
42     out[pt.id] = { x: snap(comp.x + lx), y: snap(comp.y + ly) };
43   }
44   return out;
45 }
46
47 function wire(fromId, toId, jmap, extra = []) {
48   const a = jmap[fromId];
49   const b = jmap[toId];
50   const pts = [{ x: a.x, y: a.y }, ...extra, { x: b.x, y: b.y }];
51   return { fromJunctionId: fromId, toJunctionId: toId, points: pts };
52 }
53
54 const components = [
55   { type: "gsin", label: "Sin_1", x: 0, y: 120, rotation: 0, peakAmplitude: 5, frequency: 50, offset: 0 },
56   { type: "gnd", label: "GND_1", x: 0, y: 200, rotation: 0 },
57   { type: "diode", label: "D_1", x: 240, y: 110, rotation: -90, value: "1N4007" },
58   { type: "diode", label: "D_2", x: 200, y: 110, rotation: -90, value: "1N4007" },
59   { type: "diode", label: "D_3", x: 240, y: 170, rotation: -90, value: "1N4007" },
60   { type: "diode", label: "D_4", x: 200, y: 170, rotation: -90, value: "1N4007" },
61   { type: "resistor", label: "R_1", x: 360, y: 120, rotation: 90, value: "1k" },
62   { type: "capacitor", label: "C_1", x: 360, y: 180, rotation: 0, value: "100u" },
63   { type: "voltmeter", label: "V_1", x: 420, y: 150, rotation: 90, value: "" },
64   { type: "oscilloscope", label: "Osci_1", x: 520, y: 140, rotation: 0, timeDivSec: 0.005, ch1VoltsPerDiv: 2, ch2VoltsPerDiv: 2 },
65 ];
66
67 const jmap = {};
68 for (const c of components) Object.assign(jmap, jonctions(c));
69
70 // Nœuds communs (jonctions auto sur bus)
71 const autoJunctions = [
72   { id: "__t#120#140", x: 120, y: 140 }, // A (Sin+)
73   { id: "__t#120#180", x: 120, y: 180 }, // B (Sin- / GND AC)
74   { id: "__t#300#80", x: 300, y: 80 }, // V+
75   { id: "__t#300#200", x: 300, y: 200 }, // V- (flottante)
76 ];
77 jmap["__t#120#140"] = { x: 120, y: 140 };
78 jmap["__t#120#180"] = { x: 120, y: 180 };

```

```

79 jmap["__t#300#80"] = { x: 300, y: 80 };
80 jmap["__t#300#200"] = { x: 300, y: 200 };
81
82 const wires = [
83   // Entrée AC + masse SPICE (Sin- = GND)
84   wire("Sin_1_out", "__t#120#140", jmap),
85   wire("Sin_1_in", "GND_1_out", jmap),
86   wire("GND_1_out", "__t#120#180", jmap),
87   // Pont : D1 A→V+, D2 B→V+, D3 V→A, D4 V→B
88   wire("__t#120#140", "D_1_in", jmap),
89   wire("D_1_out", "__t#300#80", jmap),
90   wire("__t#120#180", "D_2_in", jmap),
91   wire("D_2_out", "__t#300#80", jmap, [{ x: 200, y: 80 }]),
92   wire("__t#300#200", "D_3_in", jmap),
93   wire("D_3_out", "__t#120#140", jmap),
94   wire("__t#300#200", "D_4_in", jmap),
95   wire("D_4_out", "__t#120#180", jmap),
96   // Charge R//C entre V+ et V- (masse flottante côté V-)
97   wire("__t#300#80", "R_1_in", jmap),
98   wire("R_1_out", "__t#300#200", jmap),
99   wire("__t#300#80", "C_1_in", jmap, [{ x: 340, y: 80 }]),
100  wire("C_1_out", "__t#300#200", jmap, [{ x: 340, y: 200 }]),
101  // Voltmètre sur la charge (V+ / V-)
102  wire("__t#300#80", "V_1_in", jmap, [{ x: 380, y: 80 }]),
103  wire("V_1_out", "__t#300#200", jmap, [{ x: 380, y: 200 }]),
104  // Oscilloscope : GND = référence AC, CH1 = V+, CH2 = V-
105  wire("Osci_1_GND", "__t#120#180", jmap, [{ x: 520, y: 180 }]),
106  wire("__t#300#80", "Osci_1_CH1", jmap, [{ x: 460, y: 80 }, { x: 460, y: 120 }]),
107  wire("__t#300#200", "Osci_1_CH2", jmap, [{ x: 460, y: 200 }, { x: 460, y: 160 }]),
108 ];
109
110 const counters = {
111   battery: 0, resistor: 1, capacitor: 1, inductor: 0, diode: 4, npn: 0, opamp: 0,
112   not: 0, and: 0, nand: 0, or: 0, nor: 0, xor: 0, xnor: 0, d_flipflop: 0, jk_flipflop: 0,
113   cd4511: 0, ic_74hc90: 0, led: 0, seg7: 0, voltmeter: 1, ammeter: 0, ohmmeter: 0,
114   oscilloscope: 1, bode_analyzer: 0, junction: 0, gnd: 1, vcc: 0, logic_terminal: 0,
115   gimp: 0, gsin: 1, gsqr: 0,
116 };
117
118 const json = { components, wires, autoJunctions, counters };
119
120 // Validation netlist (format moteur)
121 const simComponents = components.map((c) => ({
122   id: c.label,
123   type: c.type === "gsin" ? "vsin" : c.type === "gnd" ? "ground" : c.type,
124   x: c.x,
125   y: c.y,
126   rotation: c.rotation || 0,
127   value:
128     c.type === "gsin"
129       ? `${c.peakAmplitude}V ${c.frequency}Hz ${c.offset}V`
130       : c.value,
131   ...(c.type === "oscilloscope"
132     ? {
133         timeDivSec: c.timeDivSec,
134         ch1VoltsPerDiv: c.ch1VoltsPerDiv,
135         ch2VoltsPerDiv: c.ch2VoltsPerDiv,
136       }
137     : {}),
138 }));
139
140 const junctionToKey = (jid) => {
141   if (jid.startsWith("__t#")) return jid;
142   const m = jid.match(/^(.+)_([in|out|CH1|CH2|GND])$/);
143   if (!m) return null;
144   const [, label, pin] = m;
145   const comp = components.find((c) => c.label === label);
146   if (!comp) return null;
147   if (pin === "in") return `${label}#0`;
148   if (pin === "out") return `${label}#1`;
149   if (pin === "CH1") return `${label}#0`;
150   if (pin === "CH2") return `${label}#1`;
151   if (pin === "GND") return `${label}#2`;
152   return null;
153 };
154
155 const simWires = wires
156   .map((w) => ({
157     solid: true,
158     fromKey: junctionToKey(w.fromJunctionId),

```

```
159         toKey: junctionToKey(w.toJunctionId),
160         points: w.points,
161     )))
162     .filter((w) => w.fromKey && w.toKey);
163
164 const built = buildNetlistFromGraphicalState({ components: simComponents, wires: simWires });
165 if (!built.ok) {
166     console.error("Netlist FAILED:", built.errors);
167     process.exit(1);
168 }
169 if (!built.analysisTran) {
170     console.error("Expected .tran analysis");
171     process.exit(1);
172 }
173
174 const outPath = join(import.meta.dirname, "..", "circuits", "pont-diode-double-alternance.json");
175 writeFileSync(outPath, JSON.stringify(json, null, 4), "utf8");
176 console.log("Written:", outPath);
177 console.log("Netlist OK – diodes:", (built.netlist.match(/^D/gim) || []).length);
178
```

```

1  /**
2   * Grove DHT22 – câblage DATA/VCC/GND vers Arduino UNO.
3   */
4
5  import { applyArduinoSketchToComponent } from "./arduino-sketch-parse.mjs";
6  import { parseDht22FromSketch, buildDhtVarBindingsFromBody } from "./dht22-sketch-parse.mjs";
7  import { reachableJunctions } from "./hc90-cascade.mjs";
8
9  function isDhtPowered(dhtLabel, components, wires, autoJunctions) {
10     for (const pin of ["VCC", "GND"]) {
11         const j = `${dhtLabel}_${pin}`;
12         const net = reachableJunctions(j, wires, autoJunctions);
13         let ok = false;
14         for (const comp of components) {
15             if (pin === "VCC") {
16                 if (comp.type === "vcc" && net.has(`${comp.label}_out`)) ok = true;
17                 if (comp.type === "battery" && net.has(`${comp.label}_out`)) ok = true;
18                 if (comp.type === "arduino_uno" && (net.has(`${comp.label}_5V`) || net.has(`${comp.label}_3V3`))) {
19                     ok = true;
20                 }
21             } else {
22                 if (comp.type === "gnd" && net.has(`${comp.label}_out`)) ok = true;
23                 if (comp.type === "battery" && net.has(`${comp.label}_in`)) ok = true;
24                 if (comp.type === "arduino_uno" && (net.has(`${comp.label}_GND`) || net.has(`${comp.label}_GND2`))) {
25                     ok = true;
26                 }
27             }
28         }
29         if (!ok) return false;
30     }
31     return true;
32 }
33
34 function dataPinOnUno(dhtLabel, unoLabel, wires, autoJunctions) {
35     const net = reachableJunctions(`${dhtLabel}_DATA`, wires, autoJunctions);
36     for (let d = 0; d <= 13; d++) {
37         const label = `D${d}`;
38         if (net.has(`${unoLabel}_${label}`)) return label;
39     }
40     return null;
41 }
42
43 /**
44  * @returns {{ uno: object | null, wired: boolean, dataPin: string | null }}
45  */
46 export function findUnoForGroveDht22(dhtLabel, components, wires, autoJunctions = []) {
47     const dht = components.find((c) => c.label === dhtLabel && c.type === "grove_dht22");
48     if (!dht || !isDhtPowered(dhtLabel, components, wires, autoJunctions)) {
49         return { uno: null, wired: false, dataPin: null };
50     }
51
52     const candidates = components.filter((c) => c.type === "arduino_uno");
53     candidates.sort((a, b) => (b.lastCompileOk ? 1 : 0) - (a.lastCompileOk ? 1 : 0));
54
55     for (const uno of candidates) {
56         const dataPin = dataPinOnUno(dhtLabel, uno.label, wires, autoJunctions);
57         if (dataPin) return { uno, wired: true, dataPin };
58     }
59     return { uno: null, wired: false, dataPin: null };
60 }
61
62 export function isGroveDht22WiredToUno(dhtLabel, components, wires, autoJunctions = []) {
63     return findUnoForGroveDht22(dhtLabel, components, wires, autoJunctions).wired;
64 }
65
66 /** Grove DHT22 câblé sur une broche DATA donnée de l'UNO. */
67 export function findGroveDht22ForUnoDataPin(unoLabel, dataPin, components, wires, autoJunctions = []) {
68     if (!unoLabel || !dataPin) return null;
69     for (const comp of components) {
70         if (comp.type !== "grove_dht22") continue;
71         const link = findUnoForGroveDht22(comp.label, components, wires, autoJunctions);
72         if (link.wired && link.uno?.label === unoLabel && link.dataPin === dataPin) return comp;
73     }
74     return null;
75 }
76
77 /**
78  * Résout dht.readTemperature() / readHumidity() dans lcd.print() – valeurs du composant câblé.

```

```

79  * @returns {string | null}
80  */
81  export function resolveDhtPrintArg(arg, sketch, unoLabel, components, wires, autoJunctions = []) {
82    const t = String(arg || "").trim();
83    if (!t || !sketch || !unoLabel) return null;
84    const parsed = parseDht22FromSketch(sketch);
85    if (!parsed?.pinLabel) return null;
86    const { varName, pinLabel } = parsed;
87    const tempRe = new RegExp(`\\b${varName}\\\\.readTemperature\\s*\\(\\s*\\)` , "i");
88    const humRe = new RegExp(`\\b${varName}\\\\.readHumidity\\s*\\(\\s*\\)` , "i");
89    if (!tempRe.test(t) && !humRe.test(t)) return null;
90
91    const dht = findGroveDht22ForUnoDataPin(unoLabel, pinLabel, components, wires, autoJunctions);
92    if (!dht) return null;
93
94    const temperature = Number.isFinite(dht.temperature) ? dht.temperature : 24;
95    const humidity = Number.isFinite(dht.humidity) ? dht.humidity : 55;
96    if (tempRe.test(t)) return String(Math.round(temperature * 10) / 10);
97    if (humRe.test(t)) return String(Math.round(humidity));
98    return null;
99  }
100
101  /** Variables locales (float t = dht.readTemperature(); lcd.print(t);). */
102  export function buildDhtVarBindings(body, sketch, unoLabel, components, wires, autoJunctions = []) {
103    const parsed = parseDht22FromSketch(sketch);
104    if (!parsed?.pinLabel) return {};
105    const dht = findGroveDht22ForUnoDataPin(unoLabel, parsed.pinLabel, components, wires, autoJunctions);
106    if (!dht) return {};
107    const temperature = Number.isFinite(dht.temperature) ? dht.temperature : 24;
108    const humidity = Number.isFinite(dht.humidity) ? dht.humidity : 55;
109    return buildDhtVarBindingsFromBody(body, parsed.varName, temperature, humidity);
110  }
111
112  /**
113   * @returns {{ wired: boolean, active: boolean, temperature: number | null, humidity: number | null, dataPin: string | null, sensorType: string | null }}
114   */
115  export function getIdealDht22Reading(dhtLabel, components, wires, autoJunctions = [], _elapsedSec = 0) {
116    const dht = components.find((c) => c.label === dhtLabel && c.type === "grove_dht22");
117    const temperature = Number.isFinite(dht?.temperature) ? dht.temperature : 24;
118    const humidity = Number.isFinite(dht?.humidity) ? dht.humidity : 55;
119
120    const { uno, wired, dataPin } = findUnoForGroveDht22(dhtLabel, components, wires, autoJunctions);
121    if (!wired || !uno) {
122      return { wired: false, active: false, temperature: null, humidity: null, dataPin: null, sensorType: null };
123    }
124
125    applyArduinoSketchToComponent(uno);
126    const parsed = parseDht22FromSketch(uno.sketch || "");
127    if (!parsed) {
128      return { wired: true, active: false, temperature: null, humidity: null, dataPin, sensorType: null };
129    }
130
131    return {
132      wired: true,
133      active: true,
134      temperature,
135      humidity,
136      dataPin,
137      sensorType: parsed.sensorType,
138    };
139  }
140

```

```

1  /**
2   * Interprétation minimale DHT.h (DHT22 / Grove T° humidité).
3   */
4
5  function stripComments(src) {
6      return String(src || "").
7          .replace(/\n/[\^n]*g, "")
8          .replace(/\n*\s\S]*?*\n/g, "");
9  }
10
11  const PIN_MAP = {
12      0: "D0", 1: "D1", 2: "D2", 3: "D3", 4: "D4", 5: "D5", 6: "D6", 7: "D7",
13      8: "D8", 9: "D9", 10: "D10", 11: "D11", 12: "D12", 13: "D13",
14  };
15
16  function resolvePinToken(tok) {
17      const t = String(tok || "").trim();
18      if (/^D\d+$/i.test(t)) return t.toUpperCase();
19      const n = parseInt(t, 10);
20      if (Number.isFinite(n) && PIN_MAP[n]) return PIN_MAP[n];
21      if (/^A(\d+)$/i.test(t)) return t.toUpperCase();
22      return null;
23  }
24
25  function resolveSketchToken(src, tok) {
26      let t = String(tok || "").trim();
27      for (let i = 0; i < 4; i++) {
28          if (!/^[A-Za-z_]\w*$/i.test(t)) break;
29          const def = src.match(new RegExp(`#define\s+${t}\s+(\s+)`));
30          if (!def) break;
31          t = def[1].trim();
32      }
33      return t;
34  }
35
36  export function sketchUsesDht(sketch) {
37      const src = stripComments(sketch);
38      return /#include\s*<"DHT.h">/i.test(src) || /\bDHT\s+\w+\s*/i.test(src);
39  }
40
41  /** Sketch avec lcd.print(dht.readTemperature|readHumidity()). */
42  export function sketchLcdUsesDhtReads(sketch) {
43      if (!sketchUsesDht(sketch)) return false;
44      const src = stripComments(sketch);
45      return /\s.read(Temperature|Humidity)\s*\(\s*\)/i.test(src);
46  }
47
48  /**
49   * @param {string} sketch
50   * @returns {{ varName: string, pinLabel: string | null, sensorType: string, baseTemp: number, baseHum: number } | null}
51   */
52  export function parseDht22FromSketch(sketch) {
53      const src = stripComments(sketch);
54      if (!sketchUsesDht(src)) return null;
55
56      const ctor = src.match(/\bDHT\s+(\w+)\s*\(\s*([^\,]+)\s*,\s*([^\,]+)\s*\)/i);
57      const varName = ctor?.[1] ?? "dht";
58      const pinRaw = ctor ? resolveSketchToken(src, ctor[2]) : null;
59      const pinLabel = pinRaw ? resolvePinToken(pinRaw) : null;
60      const typeRaw = resolveSketchToken(src, String(ctor?.[3] ?? "DHT22").trim());
61      const sensorType = /22/.test(typeRaw) ? "DHT22" : "DHT11";
62
63      let baseTemp = 24.0;
64      let baseHum = 55.0;
65      const tempConst = src.match(new RegExp(`\\b${varName}\\s\\.readTemperature\\s*\\(\\s*\\s*\\)[^;]*;`, "i"));
66      const humConst = src.match(new RegExp(`\\b${varName}\\s\\.readHumidity\\s*\\(\\s*\\s*\\)[^;]*;`, "i"));
67      const tempAssign = src.match(/(?:float|double)\s+\w+\s*=\s*(-?\d+(?:\.\d+)?)\s*;/);
68      const humAssign = src.match(/(?:float|double)\s+\w+\s*=\s*(-?\d+(?:\.\d+)?)\s*;[\s\S]{0,80}readHumidity/i);
69      if (tempAssign) baseTemp = parseFloat(tempAssign[1]);
70      if (humAssign) baseHum = parseFloat(humAssign[1]);
71      if (tempConst && /=\s*(-?\d+(?:\.\d+)?)/.test(tempConst[0])) {
72          const m = tempConst[0].match(/=\s*(-?\d+(?:\.\d+)?)/);
73          if (m) baseTemp = parseFloat(m[1]);
74      }
75      if (humConst && /=\s*(-?\d+(?:\.\d+)?)/.test(humConst[0])) {
76          const m = humConst[0].match(/=\s*(-?\d+(?:\.\d+)?)/);
77          if (m) baseHum = parseFloat(m[1]);
78      }
79  }

```



```

79     return { varName, pinLabel, sensorType, baseTemp, baseHum };
80 }
81
82
83 /** Variables locales assignées depuis dht.readTemperature() / readHumidity(). */
84 export function buildDhtVarBindingsFromBody(body, dhtVarName, temperature, humidity) {
85     const bindings = {};
86     const tempStr = String(Math.round(temperature * 10) / 10);
87     const humStr = String(Math.round(humidity));
88     const tempRe = new RegExp(
89         `(?:float|double|int|unsigned\\s+int)?\\s*(\\w+)\\s*=\\s*${dhtVarName}\\s\\.readTemperature\\s*\\((\\s*\\|)\\s*`;
90         "gi"
91     );
92     const humRe = new RegExp(
93         `(?:float|double|int|unsigned\\s+int)?\\s*(\\w+)\\s*=\\s*${dhtVarName}\\s\\.readHumidity\\s*\\((\\s*\\|)\\s*`;
94         "gi"
95     );
96     let m;
97     while ((m = tempRe.exec(body)) !== null) bindings[m[1]] = tempStr;
98     while ((m = humRe.exec(body)) !== null) bindings[m[1]] = humStr;
99     return bindings;
100 }
101
102 export function dht22ReadingAt(parsed, elapsedSec = 0) {
103     if (!parsed) return null;
104     const t = Math.max(0, elapsedSec);
105     return {
106         temperature: parsed.baseTemp + 2.2 * Math.sin(t * 0.07),
107         humidity: Math.max(0, Math.min(100, parsed.baseHum + 9 * Math.sin(t * 0.045 + 0.8))),
108         sensorType: parsed.sensorType,
109     };
110 }
111

```

```

1  /**
2   * Affichage idéal Grove LCD 2x16 I2C - câblage SDA/SCL/VCC/GND vers Arduino UNO.
3   */
4
5  import { applyArduinoSketchToComponent, evaluateLoopVarBindings, sketchUsesAnalogInput } from "./arduino-sketch-parse.mjs";
6  import { parseGroveLcdFromSketch, pickLcdPhaseAt, resolveLcdDisplayAt, effectiveLcdLoopCycleMs, emptyLcdBuffer } from "./grove-lcd-sketch-parse.mjs";
7  import { resolveDhtPrintArg, buildDhtVarBindings } from "./dht22-ideal.mjs";
8  import { sketchLcdUsesDhtReads } from "./dht22-sketch-parse.mjs";
9  import { readUnoAnalogInputs } from "./arduino-analog-ideal.mjs";
10 import { reachableJunctions } from "./hc90-cascade.mjs";
11
12 const GROVE_LCD_DEFAULT_I2C = 0x3e;
13
14 const I2C_UNO_PINS = { SDA: "A4", SCL: "A5" };
15
16 function isLcdI2cWiredToUno(lcdLabel, unoLabel, wires, autoJunctions) {
17   for (const [lcdPin, unoPin] of Object.entries(I2C_UNO_PINS)) {
18     const lcdJ = `${lcdLabel}_${lcdPin}`;
19     const unoJ = `${unoLabel}_${unoPin}`;
20     const net = reachableJunctions(lcdJ, wires, autoJunctions);
21     if (!net.has(unoJ)) return false;
22   }
23   return true;
24 }
25
26 function padHint(s) {
27   return String(s).slice(0, 16).padEnd(16, " ");
28 }
29
30 function isLcdPowered(lcdLabel, components, wires, autoJunctions) {
31   for (const lcdPin of ["VCC", "GND"]) {
32     const lcdJ = `${lcdLabel}_${lcdPin}`;
33     const net = reachableJunctions(lcdJ, wires, autoJunctions);
34     let ok = false;
35     for (const comp of components) {
36       if (lcdPin === "VCC") {
37         if (comp.type === "vcc" && net.has(`${comp.label}_out`)) ok = true;
38         if (comp.type === "battery" && net.has(`${comp.label}_out`)) ok = true;
39         if (comp.type === "arduino_uno" && (net.has(`${comp.label}_5V`) || net.has(`${comp.label}_3V3`))) {
40           ok = true;
41         }
42       } else {
43         if (comp.type === "gnd" && net.has(`${comp.label}_out`)) ok = true;
44         if (comp.type === "battery" && net.has(`${comp.label}_in`)) ok = true;
45         if (comp.type === "arduino_uno" && (net.has(`${comp.label}_GND`) || net.has(`${comp.label}_GND2`))) {
46           ok = true;
47         }
48       }
49     }
50     if (!ok) return false;
51   }
52   return true;
53 }
54
55 /**
56  * Trouve l'UNO câblé sur ce LCD Grove (I2C + alim).
57  * @returns {{ uno: object, wired: boolean }}
58  */
59 export function findUnoForGroveLcd(lcdLabel, components, wires, autoJunctions = []) {
60   const lcd = components.find((c) => c.label === lcdLabel && c.type === "grove_lcd16x2");
61   if (!lcd) return { uno: null, wired: false };
62
63   if (!isLcdPowered(lcdLabel, components, wires, autoJunctions)) {
64     return { uno: null, wired: false };
65   }
66
67   const candidates = components.filter((c) => c.type === "arduino_uno");
68   candidates.sort((a, b) => (b.lastCompileOk ? 1 : 0) - (a.lastCompileOk ? 1 : 0));
69
70   for (const comp of candidates) {
71     if (isLcdI2cWiredToUno(lcdLabel, comp.label, wires, autoJunctions)) {
72       return { uno: comp, wired: true };
73     }
74   }
75   return { uno: null, wired: false };
76 }
77

```

```

78 export function isGroveLcdWiredToUno(lcdLabel, components, wires, autoJunctions = []) {
79   return findUnoForGroveLcd(lcdLabel, components, wires, autoJunctions).wired;
80 }
81
82 function buildLcdPrintCtx(uno, components, wires, autoJunctions, elapsedSec = 0, opts = {}) {
83   const sketch = uno?.sketch || "";
84   const hasDht = sketchLcdUsesDhtReads(sketch);
85   const hasAnalog = sketchUsesAnalogInput(sketch);
86   if (!hasDht && !hasAnalog) return null;
87
88   const analogInputs = () => readUnoAnalogInputs(uno, {
89     components,
90     wires,
91     autoJunctions,
92     tSec: elapsedSec,
93     getVoltageAtJunction: opts.getVoltageAtJunction,
94     voltmeters: opts.voltmeters,
95   });
96
97   return {
98     resolveDht: hasDht
99       ? (arg) => resolveDhtPrintArg(arg, sketch, uno.label, components, wires, autoJunctions)
100       : undefined,
101     collectVarBindings: (body) => {
102       let bindings = hasAnalog ? evaluateLoopVarBindings(sketch, analogInputs()) : {};
103       if (hasDht) {
104         bindings = {
105           ...bindings,
106           ...buildDhtVarBindings(body, sketch, uno.label, components, wires, autoJunctions),
107         };
108       }
109       return bindings;
110     },
111   };
112 }
113
114 function needsRuntimeLcdCtx(uno) {
115   const sketch = uno?.sketch || "";
116   return sketchLcdUsesDhtReads(sketch) || sketchUsesAnalogInput(sketch);
117 }
118
119 /**
120  * Met à jour le cache d'affichage LCD après compilation / édition sketch.
121  */
122 export function refreshGroveLcdDisplayCache(components, wires, autoJunctions = []) {
123   for (const lcd of components) {
124     if (lcd.type !== "grove_lcd16x2") continue;
125     delete lcd.lcdDisplayCache;
126     const { uno, wired } = findUnoForGroveLcd(lcd.label, components, wires, autoJunctions);
127     if (!wired || !uno) continue;
128     applyArduinoSketchToComponent(uno);
129     const parsed = parseGroveLcdFromSketch(uno.sketch || "");
130     if (parsed) {
131       lcd.lcdDisplayCache = {
132         lines: parsed.lines,
133         phases: parsed.phases,
134         hasTiming: parsed.hasTiming,
135         setupDurationMs: parsed.setupDurationMs,
136         loopCycleMs: parsed.loopCycleMs,
137         effectiveLoopCycleMs: parsed.effectiveLoopCycleMs,
138         setupEndState: parsed.setupEndState,
139         loopEvents: parsed.loopEvents,
140         sketchSrc: parsed.sketchSrc,
141         backlight: parsed.backlight,
142         rgb: parsed.rgb,
143         address: parsed.address,
144         blank: !parsed.lines.some((l) => l.trim()),
145       };
146     } else if (uno.lastCompileOk) {
147       lcd.lcdDisplayCache = {
148         lines: [padHint("LiquidCrystal"), padHint("_I2C requis")],
149         backlight: true,
150         address: lcd.i2cAddress ?? GROVE_LCD_DEFAULT_I2C,
151         blank: false,
152       };
153     } else {
154       lcd.lcdDisplayCache = {
155         lines: emptyLcdBuffer(),
156         backlight: true,
157         address: lcd.i2cAddress ?? GROVE_LCD_DEFAULT_I2C,

```

```

158         blank: true,
159     };
160 }
161 }
162 }
163
164 /**
165  * Texte à afficher sur le LCD (sketch Arduino interprété).
166  * @param {number} [elapsedSec] temps écoulé depuis le début de la simulation (pour delay()).
167  * @returns {{ lines: string[], backlight: boolean, rgb: object | null, wired: boolean, address: number, blank: boolean }}
168  */
169 export function getIdealGroveLcdDisplay(lcdLabel, components, wires, autoJunctions = [], elapsedSec = 0, opts = {}) {
170     const lcd = components.find((c) => c.label === lcdLabel && c.type === "grove_lcd16x2");
171     if (!lcd) {
172         return { lines: emptyLcdBuffer(), backlight: false, rgb: null, wired: false, address: GROVE_LCD_DEFAULT_I2C, blank: true };
173     }
174
175     const { uno, wired } = findUnoForGroveLcd(lcdLabel, components, wires, autoJunctions);
176     if (!wired || !uno) {
177         return {
178             lines: [padHint("SDA->A4 SCL->"), padHint("A5 5V GND")],
179             backlight: false,
180             rgb: null,
181             wired: false,
182             address: lcd.i2cAddress ?? GROVE_LCD_DEFAULT_I2C,
183             blank: false,
184         };
185     }
186
187     applyArduinoSketchToComponent(uno);
188     const printCtx = buildLcdPrintCtx(uno, components, wires, autoJunctions, elapsedSec, opts);
189     const needsRuntime = needsRuntimeLcdCtx(uno);
190
191     const pickDisplay = (parsedOrCache) => {
192         const elapsedMs = Math.max(0, elapsedSec * 1000);
193         const loopCycleMs = effectiveLcdLoopCycleMs(parsedOrCache);
194         const resolveOpts = needsRuntime ? { ctx: printCtx } : {};
195         const phase = loopCycleMs > 0 && parsedOrCache.loopEvents?.length
196             ? resolveLcdDisplayAt(parsedOrCache, elapsedMs, resolveOpts)
197             : (parsedOrCache.hasTiming && parsedOrCache.phases?.length
198                 ? pickLcdPhaseAt(parsedOrCache.phases, elapsedMs, {
199                     loopCycleMs: 0,
200                     setupDurationMs: 0,
201                 })
202                 : null);
203         const lines = phase?.lines ?? parsedOrCache.lines;
204         const backlight = phase?.backlight ?? parsedOrCache.backlight;
205         const rgb = phase?.rgb ?? parsedOrCache.rgb ?? null;
206         return {
207             lines,
208             backlight,
209             rgb: rgb ? { ...rgb } : null,
210             blank: !lines.some((l) => l.trim()),
211             address: parsedOrCache.address ?? lcd.i2cAddress ?? GROVE_LCD_DEFAULT_I2C,
212         };
213     };
214
215     if (lcd.lcdDisplayCache && !needsRuntime) {
216         const d = pickDisplay(lcd.lcdDisplayCache);
217         return { ...d, wired: true };
218     }
219
220     const parsed = parseGroveLcdFromSketch(uno.sketch || "", printCtx);
221     if (!parsed) {
222         const hint = uno.lastCompileOk
223             ? [padHint("LiquidCrystal"), padHint("_I2C requis")]
224             : emptyLcdBuffer();
225         return {
226             lines: hint,
227             backlight: true,
228             rgb: null,
229             wired: true,
230             address: lcd.i2cAddress ?? GROVE_LCD_DEFAULT_I2C,
231             blank: !hint.some((l) => l.trim()),
232         };
233     }
234
235     const d = pickDisplay(parsed);
236     return { ...d, wired: true };
237 }

```



```

1  /**
2   * Interprétation minimale LiquidCrystal_I2C / rgb_lcd (Seeed Grove 104020112).
3   * init/begin, backlight, setCursor, print, clear, scroll, setRGB, delay().
4   */
5
6  export const LCD_COLS = 16;
7  export const LCD_ROWS = 2;
8  /** Dernière adresse DDRAM par ligne (HD44780 : 0x27 = index 39, pas « 27 caractères »). */
9  export const LCD_DDRAM_LAST_ADDR = 0x27;
10 /** 40 colonnes DDRAM par ligne (adresses 0x00-0x27). */
11 export const LCD_DDRAM_COLS = LCD_DDRAM_LAST_ADDR + 1;
12
13 /** Défilement max. pour afficher l'adresse 0x27 en colonne 15 (40 - 16 = 24 pas). */
14 export function maxLcdDisplayShift(visibleCols = LCD_COLS) {
15   const vis = Math.max(1, visibleCols || LCD_COLS);
16   return LCD_DDRAM_LAST_ADDR - (vis - 1);
17 }
18
19 export function emptyLcdBuffer() {
20   return ["", ""];
21 }
22
23 function emptyDdramBuffer(rows = LCD_ROWS) {
24   return Array.from({ length: rows }, () => " ".repeat(LCD_DDRAM_COLS));
25 }
26
27 function padDdramLine(s) {
28   return String(s ?? "").slice(0, LCD_DDRAM_COLS).padEnd(LCD_DDRAM_COLS, " ");
29 }
30
31 /** Fenêtre visible (16 colonnes) à partir du tampon DDRAM et du décalage scroll. */
32 export function visibleLcdLines(st) {
33   const vis = st.cols || LCD_COLS;
34   const maxShift = maxLcdDisplayShift(vis);
35   const shift = Math.max(0, Math.min(maxShift, st.displayShift || 0));
36   return st.ddram.map((row) => padDdramLine(row).slice(shift, shift + vis).padEnd(vis, " "));
37 }
38
39 function cloneLines(st) {
40   return visibleLcdLines(st);
41 }
42
43 function stripComments(src) {
44   return String(src || "").
45     .replace(/\/\[/[^\n]*\/g, "")
46     .replace(/\/\[/[^\s\S]*?\/\[/g, "");
47 }
48
49 /** Durée par défaut d'un cycle loop() sans delay() (scroll instantané, etc.). */
50 export const LCD_DEFAULT_INSTANT_LOOP_MS = 1000;
51
52 export function effectiveLcdLoopCycleMs(parsed) {
53   if (!parsed) return 0;
54   const ms = parsed.loopCycleMs ?? 0;
55   if (ms > 0) return ms;
56   if (parsed.loopEvents?.length) return LCD_DEFAULT_INSTANT_LOOP_MS;
57   return 0;
58 }
59
60 function extractFunctionBody(src, name) {
61   const idx = src.search(new RegExp(`\\b(?:void|int)\\s+${name}\\s*\\([^\)]*\\)\\s*\\{`, "i"));
62   if (idx < 0) return "";
63   let i = src.indexOf("{", idx);
64   if (i < 0) return "";
65   let depth = 0;
66   const start = i + 1;
67   for (; i < src.length; i++) {
68     if (src[i] === "{") depth++;
69     else if (src[i] === "}") {
70       depth--;
71       if (depth === 0) return src.slice(start, i);
72     }
73   }
74   return src.slice(start);
75 }
76
77 function parsePrintArg(arg, ctx) {
78   const t = String(arg || "").trim();

```

```

79     if (!t) return "";
80     const fMacro = t.match(/^(F\s*(\s*(['"])([\s\S]*?)\1\s*)\/)/);
81     if (fMacro) return fMacro[2];
82     if ((t.startsWith('') && t.endsWith('')) || (t.startsWith('') && t.endsWith('')))) {
83         return t.slice(1, -1);
84     }
85     if (/^0[xX][0-9a-fA-F]+$/.test(t)) return String(parseInt(t, 16));
86     if (/^-?\d+$/.test(t)) return String(parseInt(t, 10));
87     if (ctx?.resolveDht) {
88         const dhtVal = ctx.resolveDht(t);
89         if (dhtVal != null) return formatLcdDecimalText(dhtVal);
90     }
91     if (ctx?.varBindings && Object.prototype.hasOwnProperty.call(ctx.varBindings, t)) {
92         return formatLcdDecimalText(ctx.varBindings[t]);
93     }
94     return "";
95 }
96
97 /** Affichage décimal français sur LCD : 2.50 → 2,50 */
98 function formatLcdDecimalText(text) {
99     const s = String(text);
100     if (/^-?\d+\.\d+$/.test(s)) return s.replace(".", ",");
101     return s;
102 }
103
104 function parseDelayMs(arg) {
105     const t = String(arg || "").trim();
106     const n = parseInt(t, 10);
107     return Number.isFinite(n) ? Math.max(0, n) : 0;
108 }
109
110 function resolveNumericConst(src, token) {
111     const t = String(token || "").trim();
112     if (!t) return null;
113     if (/^0[xX][0-9a-fA-F]+$/.test(t) || /^-?\d+$/.test(t)) {
114         return parseInt(t, /^0x/i.test(t) ? 16 : 10);
115     }
116     const def = src.match(new RegExp(`#define\\s+${t}\\s+(0x[0-9A-Fa-f]+|\\d+)`));
117     if (def) return parseInt(def[1], /^0x/i.test(def[1]) ? 16 : 10);
118     return null;
119 }
120
121 function detectLcdVariable(src) {
122     const i2c = src.match(/LiquidCrystal_I2C\\s+(\\w+)\\s*(\\s*([^(,)]+)?(\\s*\\s*([^(,)]+)?(\\s*\\s*([^(,)]+)?\\s*)\\s*)/i);
123     if (i2c) {
124         const addr = resolveNumericConst(src, i2c[2].trim()) ?? 0x3e;
125         const colsRaw = i2c[3] ? resolveNumericConst(src, i2c[3].trim()) : LCD_COLS;
126         const rowsRaw = i2c[4] ? resolveNumericConst(src, i2c[4].trim()) : LCD_ROWS;
127         return {
128             varName: i2c[1],
129             address: addr,
130             cols: Math.min(LCD_COLS, colsRaw || LCD_COLS),
131             rows: Math.min(LCD_ROWS, rowsRaw || LCD_ROWS),
132             supportsRgb: false,
133         };
134     }
135     const rgb = src.match(/\\brgb_lcd\\s+(\\w+)\\s*(?:\\s*(\\s*))?\\s*/i);
136     if (rgb) {
137         return {
138             varName: rgb[1],
139             address: 0x3e,
140             cols: LCD_COLS,
141             rows: LCD_ROWS,
142             supportsRgb: true,
143         };
144     }
145     return null;
146 }
147
148 function extractParenArg(body, openParenIndex) {
149     let i = openParenIndex + 1;
150     let depth = 1;
151     const start = i;
152     while (i < body.length && depth > 0) {
153         if (body[i] === "(") depth++;
154         else if (body[i] === ")") depth--;
155         i++;
156     }
157     return body.slice(start, i - 1);
158 }

```

```

159
160 /** Événements setup()/loop() dans l'ordre du source : appels lcd.* et delay(). */
161 function parseTimedEvents(body, varName) {
162     if (!body) return [];
163     const events = [];
164     const delayRe = /\bdelay\s*\(/gi;
165     let m;
166     while ((m = delayRe.exec(body)) !== null) {
167         const open = m.index + m[0].length - 1;
168         events.push({ type: "delay", index: m.index, ms: parseDelayMs(extractParenArg(body, open)) });
169     }
170     const lcdRe = new RegExp(`\\b${varName}\\.(\\w+)\\s*\\(`, "gi");
171     while ((m = lcdRe.exec(body)) !== null) {
172         const open = m.index + m[0].length - 1;
173         events.push({
174             type: "lcd",
175             index: m.index,
176             fn: m[1].toLowerCase(),
177             args: extractParenArg(body, open),
178         });
179     }
180     events.sort((a, b) => a.index - b.index);
181     return events;
182 }
183
184 function createLcdState(cols, rows, supportsRgb = false) {
185     return {
186         ddram: emptyDdramBuffer(rows),
187         displayShift: 0,
188         cursorCol: 0,
189         cursorRow: 0,
190         backlight: true,
191         rgb: supportsRgb ? { r: 255, g: 255, b: 255 } : null,
192         supportsRgb,
193         cols,
194         rows,
195     };
196 }
197
198 /** Avance le curseur après écriture (rgb_lcd = DDRAM 0x00-0x27 puis ligne 2). */
199 function advanceLcdCursor(st) {
200     if (st.supportsRgb) {
201         st.cursorCol++;
202         if (st.cursorCol >= LCD_DDRAM_COLS) {
203             st.cursorCol = 0;
204             if (st.cursorRow + 1 < st.rows) st.cursorRow++;
205         }
206         return;
207     }
208     // LiquidCrystal_I2C : retour à la ligne visible après cols caractères
209     st.cursorCol++;
210     if (st.cursorCol >= st.cols) {
211         st.cursorCol = 0;
212         if (st.cursorRow + 1 < st.rows) st.cursorRow++;
213         else st.cursorRow = 0;
214     }
215 }
216
217 function writeLcdChar(st, row, col, ch) {
218     if (row >= st.rows || col < 0 || col >= LCD_DDRAM_COLS) return;
219     const arr = st.ddram[row].split("");
220     arr[col] = ch;
221     st.ddram[row] = padDdramLine(arr.join(""));
222 }
223
224 function parseRgbArgs(args, src) {
225     const parts = String(args || "").split(",").map((x) => x.trim());
226     const resolve = (t) => {
227         if (!t) return 0;
228         const fromDef = resolveNumericConst(src, t);
229         if (fromDef !== null) return Math.max(0, Math.min(255, fromDef));
230         const n = parseInt(t, 10);
231         return Number.isFinite(n) ? Math.max(0, Math.min(255, n)) : 0;
232     };
233     return {
234         r: resolve(parts[0]),
235         g: resolve(parts[1] ?? "0"),
236         b: resolve(parts[2] ?? "0"),
237     };
238 }

```



```

239
240 function applyLcdCall(fn, args, st, src = "", ctx = null) {
241     if (fn === "clear") {
242         st.ddram = emptyDdramBuffer(st.rows);
243         st.displayShift = 0;
244         st.cursorCol = 0;
245         st.cursorRow = 0;
246         return true;
247     }
248     if (fn === "home") {
249         st.cursorCol = 0;
250         st.cursorRow = 0;
251         return true;
252     }
253     if (fn === "backlight") {
254         st.backlight = true;
255         return false;
256     }
257     if (fn === "noblacklight") {
258         st.backlight = false;
259         return false;
260     }
261     if (fn === "setrgb") {
262         st.rgb = parseRgbArgs(args, src);
263         st.backlight = st.rgb.r + st.rgb.g + st.rgb.b > 0;
264         return true;
265     }
266     if (fn === "scrolldisplayleft") {
267         const cap = maxLcdDisplayShift(st.cols || LCD_COLS);
268         st.displayShift = Math.min(cap, (st.displayShift || 0) + 1);
269         return true;
270     }
271     if (fn === "scrolldisplayright") {
272         st.displayShift = Math.max(0, (st.displayShift || 0) - 1);
273         return true;
274     }
275     if (fn === "init" || fn === "begin") {
276         const parts = args.split(",").map((x) => x.trim());
277         if (parts.length >= 2 && /^d+$/ .test(parts[0])) {
278             st.cols = Math.min(LCD_COLS, parseInt(parts[0], 10) || st.cols);
279             st.rows = Math.min(LCD_ROWS, parseInt(parts[1], 10) || st.rows);
280             st.ddram = emptyDdramBuffer(st.rows);
281             st.displayShift = 0;
282         }
283         return false;
284     }
285     if (fn === "setcursor") {
286         const parts = args.split(",").map((x) => x.trim());
287         if (parts.length >= 2) {
288             const col = parseInt(parts[0], 10) || 0;
289             const row = parseInt(parts[1], 10) || 0;
290             st.cursorCol = Math.max(0, Math.min(LCD_DDRAM_LAST_ADDR, col));
291             st.cursorRow = Math.max(0, Math.min(st.rows - 1, row));
292         }
293         return false;
294     }
295     if (fn === "print" || fn === "write") {
296         const text = parsePrintArg(args.split(",")[0], ctx);
297         if (!text) return true;
298         if (st.supportsRgb) {
299             for (const ch of text) {
300                 if (st.cursorRow >= st.rows) break;
301                 writeLcdChar(st, st.cursorRow, st.cursorCol, ch);
302                 advanceLcdCursor(st);
303             }
304         } else {
305             let col = st.cursorCol;
306             let row = st.cursorRow;
307             for (const ch of text) {
308                 if (row >= st.rows) break;
309                 writeLcdChar(st, row, col, ch);
310                 col++;
311                 if (col >= st.cols) {
312                     col = 0;
313                     if (row + 1 < st.rows) row++;
314                     else row = 0;
315                 }
316             }
317             st.cursorCol = col;
318             st.cursorRow = row;

```

```

319     }
320     return true;
321 }
322 return false;
323 }
324
325 function cloneLcdState(st) {
326     return {
327         ddram: st.ddram.map((row) => padDdramLine(row)),
328         displayShift: st.displayShift || 0,
329         cursorCol: st.cursorCol,
330         cursorRow: st.cursorRow,
331         backlight: st.backlight,
332         rgb: st.rgb ? { ...st.rgb } : null,
333         supportsRgb: !!st.supportsRgb,
334         cols: st.cols,
335         rows: st.rows,
336     };
337 }
338
339 function loopBodyDurationMs(events) {
340     let t = 0;
341     for (const ev of events) {
342         if (ev.type === "delay") t += ev.ms;
343     }
344     return t;
345 }
346
347 /** Rejoue les événements loop() jusqu'à maxMs (ms dans le cycle courant). */
348 function runLoopEvents(st, events, src, maxMs, ctx = null) {
349     let t = 0;
350     for (const ev of events) {
351         if (ev.type === "delay") {
352             if (t >= maxMs) return;
353             const next = t + ev.ms;
354             if (next > maxMs) return;
355             t = next;
356         } else {
357             if (t > maxMs) return;
358             applyLcdCall(ev.fn, ev.args, st, src, ctx);
359         }
360     }
361 }
362
363 function runFullLoop(st, events, src, ctx = null) {
364     runLoopEvents(st, events, src, Number.POSITIVE_INFINITY, ctx);
365 }
366
367 function snapshotPhase(phases, timeMs, st) {
368     phases.push({
369         atMs: timeMs,
370         lines: cloneLines(st),
371         backlight: st.backlight,
372         rgb: st.rgb ? { ...st.rgb } : null,
373     });
374 }
375
376 function runTimedBody(body, varName, phases, st, timeRef, src = "", ctx = null) {
377     if (!body?.trim()) return 0;
378     const bodyCtx = ctx?.collectVarBindings
379         ? { ...ctx, varBindings: ctx.collectVarBindings(body) }
380         : ctx;
381     const startMs = timeRef.ms;
382     for (const ev of parseTimedEvents(body, varName)) {
383         if (ev.type === "delay") {
384             timeRef.ms += ev.ms;
385         } else if (applyLcdCall(ev.fn, ev.args, st, src, bodyCtx)) {
386             snapshotPhase(phases, timeRef.ms, st);
387         }
388     }
389     return timeRef.ms - startMs;
390 }
391
392 /** Choisit la phase d'affichage pour un temps écoulé (ms). */
393 export function pickLcdPhaseAt(phases, elapsedMs, opts = {}) {
394     if (!Array.isArray(phases) || phases.length === 0) return null;
395     const loopCycleMs = opts.loopCycleMs ?? 0;
396     const setupDurationMs = opts.setupDurationMs ?? 0;
397     let t = Math.max(0, elapsedMs);
398     if (loopCycleMs > 0 && t > setupDurationMs) {

```

```

399     t = setupDurationMs + ((t - setupDurationMs) % loopCycleMs);
400 }
401 let picked = phases[0];
402 for (const p of phases) {
403     if (p.atMs <= t) picked = p;
404     else break;
405 }
406 return picked;
407 }
408
409 /**
410  * État LCD à un instant donné – rejoue loop() à chaque cycle (scroll cumulatif, etc.).
411  * @param {ReturnType<typeof parseGrovelcdFromSketch>} parsed
412  * @param {number} elapsedMs
413  */
414 export function resolveLcdDisplayAt(parsed, elapsedMs, opts = {}) {
415     if (!parsed) return null;
416     const ctx = opts.ctx ?? null;
417     const elapsed = Math.max(0, elapsedMs);
418     const setupDurationMs = parsed.setupDurationMs ?? 0;
419
420     if (elapsed < setupDurationMs) {
421         return pickLcdPhaseAt(parsed.phases, elapsed, { loopCycleMs: 0, setupDurationMs: 0 });
422     }
423
424     const loopCycleMs = effectiveLcdLoopCycleMs(parsed);
425     const loopEvents = parsed.loopEvents;
426     if (loopCycleMs <= 0 || !loopEvents?.length) {
427         const phase = pickLcdPhaseAt(parsed.phases, elapsed, { loopCycleMs: 0, setupDurationMs: 0 });
428         if (phase) return phase;
429         const end = parsed.setupEndState;
430         if (end) {
431             return {
432                 lines: cloneLines(end),
433                 backlight: end.backlight,
434                 rgb: end.rgb ? { ...end.rgb } : null,
435             };
436         }
437         return null;
438     }
439
440     const st = cloneLcdState(parsed.setupEndState);
441     const loopT = elapsed - setupDurationMs;
442     const cyclesCompleted = Math.floor(loopT / loopCycleMs);
443     const cyclePos = loopT % loopCycleMs;
444     const src = parsed.sketchSrc || "";
445     const loopCtx = ctx
446         ? { ...ctx, varBindings: parsed.loopVarBindings ?? ctx.varBindings ?? {} }
447         : null;
448
449     for (let c = 0; c < cyclesCompleted; c++) {
450         runFullLoop(st, loopEvents, src, loopCtx);
451     }
452     runLoopEvents(st, loopEvents, src, cyclePos, loopCtx);
453
454     return {
455         lines: cloneLines(st),
456         backlight: st.backlight,
457         rgb: st.rgb ? { ...st.rgb } : null,
458     };
459 }
460
461 /**
462  * @param {string} sketch
463  * @returns {{ address: number, cols: number, rows: number, lines: string[], backlight: boolean, rgb: object | null, varName: string, phases:
464  * object[], hasTiming: boolean, setupDurationMs: number, loopCycleMs: number } | null}
465  */
466 export function parseGrovelcdFromSketch(sketch, ctx = null) {
467     const src = stripComments(sketch);
468     const detected = detectLcdVariable(src);
469     if (!detected) return null;
470
471     const { varName, address, supportsRgb } = detected;
472     const st = createLcdState(detected.cols, detected.rows, supportsRgb);
473     const phases = [];
474     const timeRef = { ms: 0 };
475
476     const setupBody = extractFunctionBody(src, "setup");
477     const loopBody = extractFunctionBody(src, "loop");
478     const setupDurationMs = runTimedBody(setupBody, varName, phases, st, timeRef, src, ctx);

```

```

478     const setupEndState = cloneLcdState(st);
479     const loopEvents = parseTimedEvents(loopBody, varName);
480     const loopCycleMs = loopBody.trim() ? loopBodyDurationMs(loopEvents) : 0;
481     const effectiveLoopCycleMs = loopCycleMs > 0 ? loopCycleMs : (loopEvents.length ? LCD_DEFAULT_INSTANT_LOOP_MS : 0);
482     if (loopBody.trim() && loopCycleMs > 0) {
483         runTimedBody(loopBody, varName, phases, st, timeRef, src, ctx);
484     }
485
486     if (phases.length === 0) {
487         snapshotPhase(phases, 0, st);
488     }
489
490     const last = phases[phases.length - 1];
491     const hasTiming = effectiveLoopCycleMs > 0 || phases.length > 1 || phases.some((p) => p.atMs > 0);
492     const visible = visibleLcdLines(st);
493
494     const loopVarBindings = ctx?.collectVarBindings ? ctx.collectVarBindings(loopBody) : {};
495
496     return {
497         address,
498         cols: st.cols,
499         rows: st.rows,
500         lines: visible.slice(0, st.rows),
501         backlight: last.backlight,
502         rgb: last.rgb ? { ...last.rgb } : (st.rgb ? { ...st.rgb } : null),
503         varName,
504         phases,
505         hasTiming,
506         setupDurationMs,
507         loopCycleMs,
508         effectiveLoopCycleMs,
509         setupEndState,
510         loopEvents,
511         loopVarBindings,
512         sketchSrc: src,
513     };
514 }
515

```

```

1  /**
2   * Détection chaîne 74HC90 (unités + dizaines) et comptage idéal pour l'animation.
3   */
4
5  /** Indices broche HC90 (#pin SPICE) – local pour ne pas importer logic-74hc90 (fs) côté navigateur. */
6  const HC90_PIN_Q1 = 8;
7  const HC90_PIN_Q2 = 7;
8  const HC90_PIN_Q3 = 10;
9  const HC90_PIN_MR1 = 1;
10 const HC90_PIN_MR2 = 2;
11
12 const CLOCK_SOURCE_TYPES = new Set(["gimp", "gsin", "gsqr", "vpulse", "vsin", "vsquare"]);
13 const HC90_Q_RE = /^(.+)_Q([0-3])$/;
14 const UI_AND_TYPES = new Set(["and", "logic_and"]);
15 const UI_COMBINATORIAL_TYPES = new Set([
16   "and",
17   "logic_and",
18   "nand",
19   "logic_nand",
20   "or",
21   "logic_or",
22   "nor",
23   "logic_nor",
24 ]);
25
26 const WIRE_EPS = 1e-6;
27
28 function pointOnWireSegment(p, a, b) {
29   if (!p || !a || !b) return false;
30   if (Math.abs(a.x - b.x) < WIRE_EPS) {
31     return (
32       Math.abs(p.x - a.x) < WIRE_EPS &&
33       p.y >= Math.min(a.y, b.y) - WIRE_EPS &&
34       p.y <= Math.max(a.y, b.y) + WIRE_EPS
35     );
36   }
37   if (Math.abs(a.y - b.y) < WIRE_EPS) {
38     return (
39       Math.abs(p.y - a.y) < WIRE_EPS &&
40       p.x >= Math.min(a.x, b.x) - WIRE_EPS &&
41       p.x <= Math.max(a.x, b.x) + WIRE_EPS
42     );
43   }
44   return false;
45 }
46
47 /** Graphe électrique incluant les jonctions auto posées sur un fil (T). */
48 function buildJunctionGroups(wires, autoJunctions = []) {
49   const parent = new Map();
50   function touch(id) {
51     if (id && !parent.has(id)) parent.set(id, id);
52   }
53   function find(id) {
54     touch(id);
55     let root = id;
56     while (parent.get(root) !== root) root = parent.get(root);
57     let cur = id;
58     while (parent.get(cur) !== root) {
59       const next = parent.get(cur);
60       parent.set(cur, root);
61       cur = next;
62     }
63     return root;
64   }
65   function union(a, b) {
66     if (!a || !b) return;
67     const ra = find(a);
68     const rb = find(b);
69     if (ra !== rb) parent.set(ra, rb);
70   }
71
72   for (const w of wires || []) {
73     if (!w?.fromJunctionId || !w?.toJunctionId) continue;
74     union(w.fromJunctionId, w.toJunctionId);
75     const pts = w.points;
76     if (!Array.isArray(pts) || pts.length < 2) continue;
77     for (const aj of autoJunctions) {
78       if (!aj?.id) continue;

```

```

79         for (let i = 0; i < pts.length - 1; i++) {
80             if (pointOnWireSegment({ x: aj.x, y: aj.y }, pts[i], pts[i + 1])) {
81                 union(aj.id, w.fromJunctionId);
82                 union(aj.id, w.toJunctionId);
83                 break;
84             }
85         }
86     }
87 }
88 return parent;
89 }
90
91 /** Jonctions reliées électriquement à startId (graphe non orienté). */
92 export function reachableJunctions(startId, wires, autoJunctions = []) {
93     if (!startId) return new Set();
94     const parent = buildJunctionGroups(wires, autoJunctions);
95     if (!parent.has(startId)) return new Set([startId]);
96     const root = findJunctionRoot(startId, parent);
97     const visited = new Set();
98     for (const id of parent.keys()) {
99         if (findJunctionRoot(id, parent) === root) visited.add(id);
100     }
101     return visited;
102 }
103
104 function findJunctionRoot(id, parent) {
105     let root = id;
106     while (parent.get(root) !== root) root = parent.get(root);
107     return root;
108 }
109
110 function hc90Labels(components) {
111     return components.filter((c) => c.type === "ic_74hc90" && c.label).map((c) => c.label);
112 }
113
114 function clockOutJunctions(components) {
115     const outs = [];
116     for (const c of components) {
117         if (!CLOCK_SOURCE_TYPES.has(c.type) || !c.label) continue;
118         outs.push(`${c.label}_out`);
119     }
120     return outs;
121 }
122
123 function cp0ClockedBySource(label, clockOuts, wires, autoJunctions = []) {
124     const cp0 = `${label}_CP0`;
125     const net = reachableJunctions(cp0, wires, autoJunctions);
126     return clockOuts.some((out) => net.has(out));
127 }
128
129 /** Report unités → dizaines : Q3 direct ou AND(Q0,Q3). */
130 export function validateHc90Carry(unitsLabel, tensLabel, components, wires, autoJunctions = []) {
131     const cp0Net = reachableJunctions(`${tensLabel}_CP0`, wires, autoJunctions);
132     const q0 = `${unitsLabel}_Q0`;
133     const q3 = `${unitsLabel}_Q3`;
134     if (cp0Net.has(q3) && !hasAndBetween(unitsLabel, tensLabel, components, wires, autoJunctions)) {
135         return { valid: true, kind: "q3" };
136     }
137     for (const comp of components) {
138         if (!isAndGate(comp)) continue;
139         const out = `${comp.label}_out`;
140         if (!cp0Net.has(out)) continue;
141         const inANet = reachableJunctions(`${comp.label}_inA`, wires, autoJunctions);
142         const inBNet = reachableJunctions(`${comp.label}_inB`, wires, autoJunctions);
143         const okQ0Q3 =
144             (inANet.has(q0) && inBNet.has(q3)) || (inANet.has(q3) && inBNet.has(q0));
145         if (okQ0Q3) return { valid: true, kind: "and_q0_q3" };
146     }
147     return { valid: false, kind: "unknown" };
148 }
149
150 function hasAndBetween(unitsLabel, tensLabel, components, wires, autoJunctions = []) {
151     const cp0Net = reachableJunctions(`${tensLabel}_CP0`, wires, autoJunctions);
152     return components.some((c) => isAndGate(c) && cp0Net.has(`${c.label}_out`));
153 }
154
155 function isAndGate(comp) {
156     return !!comp?.label && UI_AND_TYPES.has(comp.type);
157 }
158

```

```

159 function isCombinatorialGate(comp) {
160     return !(comp?.label && UI_COMBINATORIAL_TYPES.has(comp.type));
161 }
162
163 function gateOutputJunction(comp) {
164     return `${comp.label}_out`;
165 }
166
167 function gateInputJunctions(comp) {
168     return [`${comp.label}_inA`, `${comp.label}_inB`];
169 }
170
171 function hc90MrJunctions(...labels) {
172     const out = [];
173     for (const label of labels) {
174         if (!label) continue;
175         out.push(`${label}_MR1`, `${label}_MR2`);
176     }
177     return out;
178 }
179
180 /** AND dont les entrées sont Q1 et Q2 du HC90 « dizaines ». */
181 function findMod60Q1Q2And(tensLabel, components, wires, autoJunctions = []) {
182     const q1 = `${tensLabel}_Q1`;
183     const q2 = `${tensLabel}_Q2`;
184     return components.filter((comp) => {
185         if (!isAndGate(comp)) return false;
186         const inANet = reachableJunctions(`${comp.label}_inA`, wires, autoJunctions);
187         const inBNet = reachableJunctions(`${comp.label}_inB`, wires, autoJunctions);
188         return (inANet.has(q1) && inBNet.has(q2)) || (inANet.has(q2) && inBNet.has(q1));
189     });
190 }
191
192 /**
193  * Sortie d'une porte combinatoire (directe ou via d'autres portes) reliée à MR1/MR2.
194  * Ex. AND(Q1,Q2) → ANDRST → MR (reset mod-60 complet).
195  */
196 function combinatorialOutputDrivesMr(outputJid, mrJids, components, wires, autoJunctions = [], visited = new Set()) {
197     const outNet = reachableJunctions(outputJid, wires, autoJunctions);
198     for (const mr of mrJids) {
199         if (outNet.has(mr)) return true;
200     }
201     for (const comp of components) {
202         if (!isCombinatorialGate(comp)) continue;
203         const inputs = gateInputJunctions(comp);
204         if (!inputs.some((inJ) => outNet.has(inJ))) continue;
205         if (visited.has(comp.label)) continue;
206         visited.add(comp.label);
207         if (combinatorialOutputDrivesMr(gateOutputJunction(comp), mrJids, components, wires, autoJunctions, visited)) {
208             return true;
209         }
210     }
211     return false;
212 }
213
214 /** Reset mod-60 : AND(Q1,Q2) du HC90 dizaines → MR (direct ou via portes intermédiaires). */
215 export function detectHc90Mod60Reset(tensLabel, components, wires, unitsLabel = null, autoJunctions = []) {
216     if (!tensLabel) return false;
217     const mrJids = hc90MrJunctions(tensLabel, unitsLabel);
218     for (const andGate of findMod60Q1Q2And(tensLabel, components, wires, autoJunctions)) {
219         if (combinatorialOutputDrivesMr(gateOutputJunction(andGate), mrJids, components, wires, autoJunctions)) {
220             return true;
221         }
222     }
223     return false;
224 }
225
226 /**
227  * @returns {{
228  *   mode: 'none'|'single'|'two_digit'|'multi',
229  *   units: string|null,
230  *   tens: string|null,
231  *   clockSource: boolean,
232  *   carryValid: boolean,
233  *   carryKind: string,
234  *   mod60: boolean
235  * }}
236  */
237 export function detectHc90Cascade(components, wires, autoJunctions = []) {
238     const labels = hc90Labels(components);

```

```

239     const clockOuts = clockOutJunctions(components);
240     const hasClock = clockOuts.length > 0;
241
242     if (labels.length === 0) {
243         return { mode: "none", units: null, tens: null, clockSource: false, carryValid: false, carryKind: "none", mod60: false };
244     }
245     if (labels.length === 1) {
246         const units = labels[0];
247         const clocked = hasClock && cp0ClockedBySource(units, clockOuts, wires, autoJunctions);
248         return {
249             mode: "single",
250             units,
251             tens: null,
252             clockSource: clocked,
253             carryValid: true,
254             carryKind: "single",
255             mod60: false,
256         };
257     }
258     if (labels.length > 2) {
259         return { mode: "multi", units: null, tens: null, clockSource: hasClock, carryValid: false, carryKind: "unknown", mod60: false };
260     }
261
262     const [a, b] = labels;
263     const aClock = cp0ClockedBySource(a, clockOuts, wires, autoJunctions);
264     const bClock = cp0ClockedBySource(b, clockOuts, wires, autoJunctions);
265     let units = null;
266     let tens = null;
267     if (aClock && !bClock) {
268         units = a;
269         tens = b;
270     } else if (bClock && !aClock) {
271         units = b;
272         tens = a;
273     } else {
274         const carryAB = validateHc90Carry(a, b, components, wires, autoJunctions);
275         const carryBA = validateHc90Carry(b, a, components, wires, autoJunctions);
276         if (carryAB.valid && !carryBA.valid) {
277             units = a;
278             tens = b;
279         } else if (carryBA.valid && !carryAB.valid) {
280             units = b;
281             tens = a;
282         } else {
283             return { mode: "multi", units: null, tens: null, clockSource: hasClock, carryValid: false, carryKind: "unknown", mod60: false };
284         }
285     }
286
287     const carry = validateHc90Carry(units, tens, components, wires, autoJunctions);
288     return {
289         mode: "two_digit",
290         units,
291         tens,
292         clockSource: true,
293         carryValid: carry.valid,
294         carryKind: carry.kind,
295         mod60: detectHc90Mod60Reset(tens, components, wires, units, autoJunctions),
296     };
297 }
298
299 /** Niveau logique haut sur une broche HC90 (logic_terminal à 1 ou VCC). */
300 function isHc90PinLogicHigh(pinJunctionId, components, wires, autoJunctions = []) {
301     const net = reachableJunctions(pinJunctionId, wires, autoJunctions);
302     for (const comp of components) {
303         if (comp.type === "logic_terminal" && comp.label) {
304             const out = `${comp.label}_out`;
305             if (net.has(out) && Number(comp.state) === 1) return true;
306         }
307         if (comp.type === "vcc" && comp.label) {
308             const out = `${comp.label}_out`;
309             if (net.has(out)) return true;
310         }
311     }
312     return false;
313 }
314
315 /**
316  * Master reset actif : MR1 ET MR2 au niveau haut sur au moins un 74HC90.
317  * (Broches reliées à la masse / logic_terminal à 0 → inactif.)
318  */

```



```

319 export function isHc90MasterResetActive(components, wires, autoJunctions = []) {
320   for (const comp of components) {
321     if (comp.type !== "ic_74hc90" || !comp.label) continue;
322     const mr1 = `${comp.label}_MR1`;
323     const mr2 = `${comp.label}_MR2`;
324     if (isHc90PinLogicHigh(mr1, components, wires, autoJunctions) && isHc90PinLogicHigh(mr2, components, wires, autoJunctions)) {
325       return true;
326     }
327   }
328   return false;
329 }
330
331 /** Nombre total de fronts horloge depuis t = 0 (1 impulsion GImp = +1). */
332 export function hc90PulseCount(elapsedSec, clockPeriodSec) {
333   if (!(clockPeriodSec > 0) || !(elapsedSec >= 0)) return 0;
334   return Math.floor(elapsedSec / clockPeriodSec);
335 }
336
337 /** Chiffre BCD 0-9 pour un HC90 donné (comptage idéal 0..99 ou 0..9). */
338 export function idealHc90BcdForLabel(complabel, cascade, elapsedSec, clockPeriodSec) {
339   if (!cascade || !complabel || !(clockPeriodSec > 0)) return null;
340   const pulses = hc90PulseCount(elapsedSec, clockPeriodSec);
341   if (cascade.mode === "single" && complabel === cascade.units) {
342     return pulses % 10;
343   }
344   if (cascade.mode === "two_digit" && cascade.units && cascade.tens) {
345     const mod = cascade.mod60 ? 60 : 100;
346     const total = pulses % mod;
347     if (complabel === cascade.units) return total % 10;
348     if (complabel === cascade.tens) return Math.floor(total / 10) % 10;
349   }
350   return null;
351 }
352
353 /**
354  * Temps .tran pour lire l'état stabilisé après N impulsions dans la fenêtre simulée.
355  * Quand totalPulses est un multiple de pulsesPerSpan, on lit la dernière impulsion
356  * du span (pas t=0 qui est l'état initial avant tout front).
357  */
358 export function hc90TranSampleTimeSec(elapsedSec, clockPeriodSec, plotSpanSec, phase = 0.49) {
359   if (!(clockPeriodSec > 0) || !(plotSpanSec > 0)) return elapsedSec;
360   const totalPulses = hc90PulseCount(elapsedSec, clockPeriodSec);
361   if (totalPulses === 0) return 0;
362   const pulsesPerSpan = Math.max(1, Math.floor(plotSpanSec / clockPeriodSec));
363   let pulseInSpan = totalPulses % pulsesPerSpan;
364   if (pulseInSpan === 0) pulseInSpan = pulsesPerSpan;
365   return Math.min(
366     plotSpanSec - 1e-12,
367     (pulseInSpan - 1) * clockPeriodSec + clockPeriodSec * phase
368   );
369 }
370
371 const CD4511_BCD_PINS = ["A", "B", "C", "D"];
372 const SEG7_SEG_PINS = ["a", "b", "c", "d", "e", "f", "g"];
373
374 /**
375  * LED dont l'anode est sur une sortie Q0..Q3 d'un 74HC90 (directement ou via résistance).
376  * @returns {{ hc90Label: string, qIndex: number } | null}
377  */
378 export function hc90QBitForLed(ledLabel, components, wires, autoJunctions = []) {
379   if (!ledLabel) return null;
380   const anodeNet = reachableJunctions(`${ledLabel}_in`, wires, autoJunctions);
381   for (const hc of components) {
382     if (hc.type !== "ic_74hc90" || !hc.label) continue;
383     for (let i = 0; i < 4; i++) {
384       if (anodeNet.has(`${hc.label}_Q${i}`)) {
385         return { hc90Label: hc.label, qIndex: i };
386       }
387     }
388   }
389   return null;
390 }
391
392 /** HC90 dont Q0..Q3 alimentent un CD4511 relié à cet afficheur SEG. */
393 export function hc90LabelForSeg7(segLabel, components, wires, autoJunctions = []) {
394   if (!segLabel) return null;
395   for (const cd of components) {
396     if (cd.type !== "cd4511" || !cd.label) continue;
397     let segLinked = false;
398     for (const s of SEG7_SEG_PINS) {

```

```

399     const segPin = `${segLabel}_${s}`;
400     const cdPin = `${cd.label}_${s}`;
401     if (reachableJunctions(segPin, wires, autoJunctions).has(cdPin)) {
402         seglinked = true;
403         break;
404     }
405 }
406 if (!seglinked) continue;
407 for (const hc of components) {
408     if (hc.type !== "ic_74hc90" || !hc.label) continue;
409     let matched = 0;
410     for (let i = 0; i < 4; i++) {
411         const qNet = reachableJunctions(`${hc.label}_Q${i}`, wires, autoJunctions);
412         const bcdPin = `${cd.label}_${CD4511_BCD_PINS[i]}`;
413         if (qNet.has(bcdPin)) matched++;
414     }
415     if (matched >= 4) return hc.label;
416 }
417 }
418 return null;
419 }
420
421 /** Animation fiable dès qu'un Gimp horloge le HC90 (toute fréquence). */
422 export function shouldUseIdealHc90Counting(cascade, clockPeriodSec) {
423     if (!cascade || !(clockPeriodSec > 0)) return false;
424     if (cascade.mode === "two_digit" && cascade.units && cascade.tens) {
425         return cascade.clockSource !== false;
426     }
427     // Un seul HC90 : comptage idéal dès qu'une horloge est présente (évite les erreurs
428     // de relecture .tran après le rollover 9→0).
429     if (cascade.mode === "single" && cascade.units) return true;
430     return false;
431 }
432
433 const ENGINE_AND_TYPES = new Set(["and", "logic_and"]);
434 const ENGINE_COMB_TYPES = new Set([
435     "and",
436     "logic_and",
437     "or",
438     "logic_or",
439     "nand",
440     "logic_nand",
441     "nor",
442     "logic_nor",
443 ]);
444
445 /** Graphe fils SPICE / moteur (clés terminal `id#pin`). */
446 export function reachableTerminalKeys(startKey, wires) {
447     if (!startKey) return new Set();
448     const visited = new Set([startKey]);
449     const queue = [startKey];
450     while (queue.length) {
451         const key = queue.shift();
452         for (const w of wires || []) {
453             if (!w?.solid) continue;
454             let next = null;
455             if (w.fromKey === key) next = w.toKey;
456             else if (w.toKey === key) next = w.fromKey;
457             if (next && !visited.has(next)) {
458                 visited.add(next);
459                 queue.push(next);
460             }
461         }
462     }
463     return visited;
464 }
465
466 function isEngineAndGate(comp) {
467     const id = comp?.id || comp?.label;
468     return id && ENGINE_AND_TYPES.has(comp.type);
469 }
470
471 function isEngineCombGate(comp) {
472     const id = comp?.id || comp?.label;
473     return id && ENGINE_COMB_TYPES.has(comp.type);
474 }
475
476 function engineGateOutKey(comp) {
477     const id = comp.id || comp.label;
478     return `${id}#2`;

```

```

479 }
480
481 function engineGateInKeys(comp) {
482     const id = comp.id || comp.label;
483     return [`${id}#0`, `${id}#1`];
484 }
485
486 function findMod60Q1Q2AndKeys(tensId, components, wires) {
487     const q1 = `${tensId}#${HC90_PIN_Q1}`;
488     const q2 = `${tensId}#${HC90_PIN_Q2}`;
489     return components.filter((comp) => {
490         if (!isEngineAndGate(comp)) return false;
491         const inANet = reachableTerminalKeys(`${comp.id || comp.label}#0`, wires);
492         const inBNet = reachableTerminalKeys(`${comp.id || comp.label}#1`, wires);
493         return (inANet.has(q1) && inBNet.has(q2)) || (inANet.has(q2) && inBNet.has(q1));
494     });
495 }
496
497 function combinatorialKeyDrivesMr(outputKey, mrKeys, components, wires, visited = new Set()) {
498     const outNet = reachableTerminalKeys(outputKey, wires);
499     for (const mr of mrKeys) {
500         if (outNet.has(mr)) return true;
501     }
502     for (const comp of components) {
503         const id = comp.id || comp.label;
504         if (!isEngineCombGate(comp)) continue;
505         if (visited.has(id)) continue;
506         if (!engineGateInKeys(comp).some((inK) => outNet.has(inK))) continue;
507         visited.add(id);
508         if (combinatorialKeyDrivesMr(engineGateOutKey(comp), mrKeys, components, wires, visited)) {
509             return true;
510         }
511     }
512     return false;
513 }
514
515 /**
516  * AND(Q1,Q3) du même HC90 qui pilote MR1/MR2 – provoque des resets pendant les transitions.
517  * (Le report dizaines utilise AND(Q0,Q3) → CP0 du 2e HC90, pas MR.)
518  */
519 export function detectHc90MrAndQ1Q3OnSameChip(hc90Id, components, wires) {
520     if (!hc90Id) return false;
521     const q1 = `${hc90Id}#${HC90_PIN_Q1}`;
522     const q3 = `${hc90Id}#${HC90_PIN_Q3}`;
523     const mrKeys = [`${hc90Id}#${HC90_PIN_MR1}`, `${hc90Id}#${HC90_PIN_MR2}`];
524     for (const comp of components) {
525         if (!isEngineAndGate(comp)) continue;
526         const cid = comp.id || comp.label;
527         const inA = reachableTerminalKeys(`${cid}#0`, wires);
528         const inB = reachableTerminalKeys(`${cid}#1`, wires);
529         const hasQ1Q3 = (inA.has(q1) && inB.has(q3)) || (inA.has(q3) && inB.has(q1));
530         if (!hasQ1Q3) continue;
531         if (combinatorialKeyDrivesMr(engineGateOutKey(comp), mrKeys, components, wires)) {
532             return true;
533         }
534     }
535     return false;
536 }
537
538 /** Détection mod-60 pour la netlist SPICE (clés `fromKey` / `toKey`). */
539 export function detectHc90Mod60FromGraphicalState(components, wires) {
540     const hc90s = components.filter((c) => c.type === "ic_74hc90" && c.id);
541     if (hc90s.length < 2) return false;
542     const mrKeys = hc90s.flatMap((c) => [
543         `${c.id}#${HC90_PIN_MR1}`,
544         `${c.id}#${HC90_PIN_MR2}`,
545     ]);
546     for (const tens of hc90s) {
547         for (const andGate of findMod60Q1Q2AndKeys(tens.id, components, wires)) {
548             if (combinatorialKeyDrivesMr(engineGateOutKey(andGate), mrKeys, components, wires)) {
549                 return true;
550             }
551         }
552     }
553     return false;
554 }
555

```

```

1  /**
2   * Bus I²C PCF8574 + HD44780 – formes d'onde réelles pour ngspice / oscilloscope.
3   */
4
5  import { parseGroveLcdFromSketch } from "../grove-lcd-sketch-parse.mjs";
6  import { reachableTerminalKeys } from "../hc90-cascade.mjs";
7  import {
8    I2C_SCL_PERIOD_SEC,
9    I2C_SCL_HZ,
10   formatSpiceTime,
11   pwlToSpiceString,
12 } from "../i2c-protocol.mjs";
13 import { buildLcdI2cWaveformFromSketch } from "../lcd-pcf8574-i2c.mjs";
14
15 /** Indices broches UNO (alignés sur arduino-uno.mjs). */
16 const UNO_A4 = 11;
17 const UNO_A5 = 12;
18 const UNO_5V = 3;
19 const UNO_GND = 4;
20 const UNO_GND2 = 5;
21
22 const LCD_SDA = 0;
23 const LCD_SCL = 1;
24 const LCD_VCC = 2;
25 const LCD_GND = 3;
26
27 export { I2C_SCL_HZ, I2C_SCL_PERIOD_SEC };
28
29 export function sketchUsesI2cLcd(sketch) {
30   return parseGroveLcdFromSketch(sketch || "") != null;
31 }
32
33 function isLcdPoweredEngine(lcdId, components, wires) {
34   const vccNet = reachableTerminalKeys(`${lcdId}#${LCD_VCC}`, wires);
35   const gndNet = reachableTerminalKeys(`${lcdId}#${LCD_GND}`, wires);
36   let vccOk = false;
37   let gndOk = false;
38   for (const comp of components) {
39     const id = comp.id || comp.label;
40     if (!id) continue;
41     if (vccNet.has(`${id}#0`) || vccNet.has(`${id}#1`)) {
42       if (comp.type === "vcc" || comp.type === "battery" || comp.type === "vsource" || comp.type === "vterm") {
43         vccOk = true;
44       }
45     }
46     if (comp.type === "arduino_uno") {
47       if (vccNet.has(`${id}#${UNO_5V}`) || vccNet.has(`${id}#2`)) vccOk = true;
48       if (gndNet.has(`${id}#${UNO_GND}`) || gndNet.has(`${id}#${UNO_GND2}`)) gndOk = true;
49     }
50     if (gndNet.has(`${id}#0`) || gndNet.has(`${id}#1`)) {
51       if (comp.type === "ground" || comp.type === "gnd" || comp.type === "battery") gndOk = true;
52     }
53   }
54   return vccOk && gndOk;
55 }
56
57 function isLcdI2cWiredToUnoEngine(unoId, lcdId, wires) {
58   const sdaNet = reachableTerminalKeys(`${lcdId}#${LCD_SDA}`, wires);
59   const sclNet = reachableTerminalKeys(`${lcdId}#${LCD_SCL}`, wires);
60   return sdaNet.has(`${unoId}#${UNO_A4}`) && sclNet.has(`${unoId}#${UNO_A5}`);
61 }
62
63 /**
64  * Marque l'UNO comme émetteur I²C si sketch LiquidCrystal_I2C + LCD Grove câblé.
65  */
66 export function annotateUnoI2cBusEngine(uno, components, wires) {
67   if (!uno || uno.type !== "arduino_uno") return;
68   const unoId = uno.id || uno.label;
69   if (!unoId || !sketchUsesI2cLcd(uno.sketch)) {
70     delete uno.i2cBus;
71     return;
72   }
73   for (const lcd of components) {
74     if (lcd.type !== "grove_lcd16x2") continue;
75     const lcdId = lcd.id || lcd.label;
76     if (!lcdId) continue;
77     if (!isLcdI2cWiredToUnoEngine(unoId, lcdId, wires)) continue;
78     if (!isLcdPoweredEngine(lcdId, components, wires)) continue;

```

```

79     uno.i2cBus = {
80         active: true,
81         address: lcd.i2cAddress ?? 0x3e,
82         lcdId,
83     };
84     return;
85 }
86 delete uno.i2cBus;
87 }
88
89 export function i2cBusMinPeriodSec(components) {
90     for (const c of components) {
91         if (c.type === "arduino_uno" && c.i2cBus?.active) return I2C_SCL_PERIOD_SEC;
92     }
93     return 0;
94 }
95
96 /**
97  * Pull-up I2C + pilotes PWL (START/STOP/ACK, 100 kHz) sur SDA/SCL.
98  */
99 export function appendI2cBusNetlist(c, nodeFor, lines, spiceBranchName, repeatUntilSec = 0.02) {
100     if (!c?.i2cBus?.active) return;
101     const wf = buildLcdI2cWaveformFromSketch(c.sketch || "", repeatUntilSec);
102     if (!wf?.sda?.length || !wf?.scl?.length) return;
103
104     const id = c.id;
105     const nSda = nodeFor(`${id}#${UNO_A4}`);
106     const nScl = nodeFor(`${id}#${UNO_A5}`);
107
108     lines.push(`* ${id} I2C PCF8574 - ${I2C_SCL_HZ / 1000} kHz, START/STOP/ACK, PWL ${formatSpiceTime(wf.durationSec)} s`);
109     lines.push(`${spiceBranchName("V", `${id}_SDA_DRV`) ${nSda} 0 ${pwlToSpiceString(wf.sda)}}`);
110     lines.push(`${spiceBranchName("V", `${id}_SCL_DRV`) ${nScl} 0 ${pwlToSpiceString(wf.scl)}}`);
111 }
112

```

```

1  /**
2   * Générateur de formes d'onde I²C (UM10204 mode standard 100 kHz).
3   * Modèle push-pull avec niveaux bus haut = 5 V (tireur relâché + pull-up), bas = 0 V.
4   */
5
6  export const I2C_V_HIGH = 5;
7  export const I2C_V_LOW = 0;
8  /** Mode standard 100 kHz –  $t_{\text{LOW}} / t_{\text{HIGH}} \geq 4,7 \mu\text{s}$ . */
9  export const I2C_SCL_HZ = 100_000;
10 export const I2C_SCL_PERIOD_SEC = 1 / I2C_SCL_HZ;
11 export const I2C_T_LOW = 4.7e-6;
12 export const I2C_T_HIGH = 4.0e-6;
13 export const I2C_T_SU_STA = 4.7e-6;
14 export const I2C_T_HD_STA = 4.0e-6;
15 export const I2C_T_SU_STO = 4.0e-6;
16 export const I2C_T_BUF = 4.7e-6;
17 export const I2C_T_SU_DAT = 0.25e-6;
18 export const I2C_T_RISE = 50e-9;
19
20 export function formatSpiceTime(seconds) {
21   const s = Math.abs(Number(seconds));
22   if (!Number.isFinite(s)) return "1n";
23   if (s === 0) return "0";
24   if (s >= 1) return String(s);
25   if (s >= 1e-3) return `${(s * 1e3).toPrecision(6)}m`;
26   if (s >= 1e-6) return `${(s * 1e6).toPrecision(6)}u`;
27   if (s >= 1e-9) return `${(s * 1e9).toPrecision(6)}n`;
28   return `${s.toExponential(3)}`;
29 }
30
31 function coalescePwl(points) {
32   if (!points.length) return [[0, I2C_V_HIGH]];
33   const out = [[points[0][0], points[0][1]]];
34   for (let i = 1; i < points.length; i++) {
35     const [t, v] = points[i];
36     const prev = out[out.length - 1];
37     if (Math.abs(t - prev[0]) < 1e-15 && Math.abs(v - prev[1]) < 1e-9) continue;
38     if (Math.abs(v - prev[1]) < 1e-9 && t > prev[0]) {
39       prev[0] = t;
40       continue;
41     }
42     out.push([t, v]);
43   }
44   return out;
45 }
46
47 /** Émetteur maître I²C – construction SDA/SCL synchrones. */
48 export class I2cMasterWaveform {
49   constructor() {
50     this.t = 0;
51     this.scl = I2C_V_HIGH;
52     this.sda = I2C_V_HIGH;
53     /** @type {[number, number][]} */
54     this.sdaPoints = [[0, I2C_V_HIGH]];
55     /** @type {[number, number][]} */
56     this.sclPoints = [[0, I2C_V_HIGH]];
57   }
58
59   durationSec() {
60     return this.t;
61   }
62
63   _snapshot() {
64     this.sdaPoints.push([this.t, this.sda]);
65     this.sclPoints.push([this.t, this.scl]);
66   }
67
68   _setSda(v) {
69     if (Math.abs(this.sda - v) > 1e-9) {
70       this.sda = v;
71       this._snapshot();
72     }
73   }
74
75   _setScl(v) {
76     if (Math.abs(this.scl - v) > 1e-9) {
77       this.scl = v;
78       this._snapshot();

```

```

79     }
80 }
81
82 _wait(dt) {
83     this.t += dt;
84 }
85
86 /** Condition START (t_HD;STA). */
87 start() {
88     this._setSda(I2C_V_HIGH);
89     this._setScl(I2C_V_HIGH);
90     this._wait(I2C_T_SU_STA);
91     this._setSda(I2C_V_LOW);
92     this._wait(I2C_T_HD_STA);
93     this._setScl(I2C_V_LOW);
94     this._wait(I2C_T_RISE);
95 }
96
97 /** Condition STOP (t_SU;STO). */
98 stop() {
99     this._setSda(I2C_V_LOW);
100    this._setScl(I2C_V_LOW);
101    this._wait(I2C_T_RISE);
102    this._setScl(I2C_V_HIGH);
103    this._wait(I2C_T_SU_STO);
104    this._setSda(I2C_V_HIGH);
105    this._wait(I2C_T_BUF);
106 }
107
108 /** Un bit de données (MSB→LSB), horloge 100 kHz. */
109 _writeBit(bit) {
110     this._setScl(I2C_V_LOW);
111     this._setSda(bit ? I2C_V_HIGH : I2C_V_LOW);
112     this._wait(I2C_T_SU_DAT);
113     this._setScl(I2C_V_HIGH);
114     this._wait(I2C_T_HIGH);
115     this._setScl(I2C_V_LOW);
116     this._wait(I2C_T_LOW);
117 }
118
119 /** Octet + créneau ACK (esclave tire SDA à 0). */
120 writeByte(byte) {
121     for (let i = 7; i >= 0; i--) {
122         this._writeBit((byte >> i) & 1);
123     }
124     this._setScl(I2C_V_LOW);
125     this._setSda(I2C_V_HIGH);
126     this._wait(I2C_T_SU_DAT);
127     this._setScl(I2C_V_HIGH);
128     this._wait(I2C_T_HIGH);
129     this._setSda(I2C_V_LOW);
130     this._setScl(I2C_V_LOW);
131     this._wait(I2C_T_LOW);
132 }
133
134 /** Transaction WRITE : START + adresse 8 bits (7b+R/W) + ACK + [data+ACK]* + STOP. */
135 writeTransaction(addr8, dataBytes = []) {
136     this.start();
137     this.writeByte(addr8 & 0xff);
138     for (const b of dataBytes) {
139         this.writeByte(b & 0xff);
140     }
141     this.stop();
142 }
143
144 toPwl() {
145     return {
146         sda: coalescePwl(this.sdaPoints),
147         scl: coalescePwl(this.sclPoints),
148         durationSec: this.t,
149     };
150 }
151 }
152
153 /**
154  * Répète une forme d'onde pour remplir la fenêtre SPICE.
155  * @param {{ sda: [number,number][], scl: [number,number][], durationSec: number }} once
156  * @param {number} repeatUntilSec
157  */
158 export function repeatI2cWaveform(once, repeatUntilSec) {

```

```

159     const dur = Math.max(once.durationSec, 1e-9);
160     const target = Math.max(repeatUntilSec, dur);
161     const sda = [...once.sda];
162     const scl = [...once.scl];
163     let t = dur;
164     while (t + dur <= target + 1e-12) {
165         const offset = t;
166         for (let i = 1; i < once.sda.length; i++) {
167             sda.push([once.sda[i][0] + offset, once.sda[i][1]]);
168         }
169         for (let i = 1; i < once.scl.length; i++) {
170             scl.push([once.scl[i][0] + offset, once.scl[i][1]]);
171         }
172         t += dur + I2C_T_BUF;
173     }
174     return {
175         sda: coalescePwl(sda),
176         scl: coalescePwl(scl),
177         durationSec: t,
178     };
179 }
180
181 /**
182  * ngspice interpole linéairement entre points PWL : chaque palier doit être
183  * (t_début, V) puis (t_fin, V) avant un saut, sinon la tension « rampe » sur toute la durée du palier.
184  */
185 export function expandPwlPlateaus(points) {
186     if (!points.length) return [[0, I2C_V_HIGH]];
187     const out = [[points[0][0], points[0][1]]];
188     for (let i = 1; i < points.length; i++) {
189         const [t, v] = points[i];
190         const [tPrev, vPrev] = points[i - 1];
191         if (Math.abs(v - vPrev) > 1e-9) {
192             if (t > tPrev + 1e-18) out.push([t, vPrev]);
193             out.push([t, v]);
194         } else if (t > tPrev + 1e-18) {
195             out.push([t, v]);
196         }
197     }
198     return out;
199 }
200
201 /** PWL ngspice : PWL(t1 v1 t2 v2 ...). */
202 export function pwlToSpiceString(points) {
203     const parts = [];
204     for (const [t, v] of expandPwlPlateaus(points)) {
205         parts.push(formatSpiceTime(t), String(v));
206     }
207     return `PWL(${parts.join(" ")}`;
208 }
209
210 /**
211  * Adresse 8 bits sur le bus (7 bits + R/W=0 pour écriture).
212  * LiquidCrystal_I2C(0x3E) Grove : octet d'adresse = 0x3E.
213  */
214 export function i2cWriteAddress8From7bit(addr7) {
215     return ((addr7 & 0x7f) << 1) & 0xfe;
216 }
217
218 export function groveLcdBusAddress8(addrFromSketch) {
219     const a = addrFromSketch & 0xff;
220     if (a >= 0x80) return a;
221     if (a <= 0x7f && (a & 1) === 0 && a >= 0x20) return a;
222     return i2cWriteAddress8From7bit(a);
223 }
224

```



```

1  /**
2   * Encode les opérations LiquidCrystal_I2C (PCF8574 + HD44780 4 bits) en transactions I²C.
3   * Compatible bibliothèque Frank de Brabander / Grove 104020112.
4   */
5
6  import { parseGroveLcdFromSketch } from "../grove-lcd-sketch-parse.mjs";
7  import { I2cMasterWaveform, groveLcdBusAddress8, repeatI2cWaveform } from "../i2c-protocol.mjs";
8
9  const RS = 0x01;
10 const EN = 0x04;
11 const BL = 0x08;
12
13 const LCD_CLEAR = 0x01;
14 const LCD_ENTRYMODE = 0x06;
15 const LCD_DISPLAYON = 0x0c;
16 const LCD_FUNCTIONSET = 0x28;
17 const LCD_SETDDRAM = 0x80;
18 const LCD_SHIFT_LEFT = 0x18;
19 const LCD_SHIFT_RIGHT = 0x1c;
20
21 function ddramAddr(col, row) {
22   return row === 0 ? col : 0x40 + col;
23 }
24
25 function extractParenArg(body, openParenIndex) {
26   let i = openParenIndex + 1;
27   let depth = 1;
28   const start = i;
29   while (i < body.length && depth > 0) {
30     if (body[i] === "(") depth++;
31     else if (body[i] === ")") depth--;
32     i++;
33   }
34   return body.slice(start, i - 1);
35 }
36
37 function parsePrintArg(arg) {
38   const t = String(arg || "").trim();
39   const fMacro = t.match(/^F\s*(\s*("[\s\S]*?")\s*)?(\s*\s*)/);
40   if (fMacro) return fMacro[2];
41   if ((t.startsWith("'") && t.endsWith("'")) || (t.startsWith('"') && t.endsWith('"'))) {
42     return t.slice(1, -1);
43   }
44   return "";
45 }
46
47 /** @returns {number[]} octets PCF8574 pour une impulsion EN (2 writes I²C). */
48 function expanderPulseBytes(value, backlight) {
49   const bl = backlight ? BL : 0;
50   const base = (value & 0xff) | bl;
51   return [base | EN, base & ~EN];
52 }
53
54 function encodeWrite4bits(wave, addr8, nibbleData, rs, backlight) {
55   const hi = (nibbleData & 0xf0) | rs;
56   const lo = ((nibbleData & 0x0f) << 4) | rs;
57   for (const val of [hi, lo]) {
58     for (const b of expanderPulseBytes(val, backlight)) {
59       wave.writeTransaction(addr8, [b]);
60     }
61   }
62 }
63
64 function encodeCommand(wave, addr8, cmd, backlight) {
65   encodeWrite4bits(wave, addr8, cmd, 0, backlight);
66 }
67
68 function encodeData(wave, addr8, data, backlight) {
69   encodeWrite4bits(wave, addr8, data, RS, backlight);
70 }
71
72 function encodeHd44780Init(wave, addr8, backlight) {
73   encodeWrite4bits(wave, addr8, 0x03, 0, backlight);
74   encodeWrite4bits(wave, addr8, 0x03, 0, backlight);
75   encodeWrite4bits(wave, addr8, 0x03, 0, backlight);
76   encodeWrite4bits(wave, addr8, 0x02, 0, backlight);
77   encodeCommand(wave, addr8, LCD_FUNCTIONSET, backlight);
78   encodeCommand(wave, addr8, LCD_DISPLAYON, backlight);

```

```

79   encodeCommand(wave, addr8, LCD_ENTRYMODE, backlight);
80   encodeCommand(wave, addr8, LCD_CLEAR, backlight);
81 }
82
83 function encodeSetCursor(wave, addr8, col, row, backlight) {
84   encodeCommand(wave, addr8, LCD_SETDDRAM | ddramAddr(col, row), backlight);
85 }
86
87 function encodePrintText(wave, addr8, text, backlight) {
88   for (const ch of String(text || "")) {
89     encodeData(wave, addr8, ch.charCodeAt(0) & 0xff, backlight);
90   }
91 }
92
93 function extractFunctionBody(src, name) {
94   const idx = src.search(new RegExp(`\\b(?:void|int)\\s+${name}\\s*\\([^\\)]*\\)\\s*\\{`, "i"));
95   if (idx < 0) return "";
96   let i = src.indexOf("{", idx);
97   if (i < 0) return "";
98   let depth = 0;
99   const start = i + 1;
100  for (; i < src.length; i++) {
101    if (src[i] === "{") depth++;
102    else if (src[i] === "}") {
103      depth--;
104      if (depth === 0) return src.slice(start, i);
105    }
106  }
107  return "";
108 }
109
110 /**
111  * Rejoue setup()/loop() du sketch et génère le bus I2C complet (100 kHz, START/ACK/STOP).
112  * @param {string} sketch
113  * @param {number} [repeatUntilSec=0.02]
114  */
115 export function buildLcdI2cWaveformFromSketch(sketch, repeatUntilSec = 0.02) {
116   const parsed = parseGroveLcdFromSketch(sketch || "");
117   if (!parsed) return null;
118
119   const addr8 = groveLcdBusAddress8(parsed.address);
120   const wave = new I2cMasterWaveform();
121   let backlight = true;
122   let initDone = false;
123
124   const src = String(sketch || "").replace(/\\/[^\\n]*/g, "").replace(/\\/\\s\\s*/g, "");
125   const varName = parsed.varName;
126
127   for (const fn of ["setup", "loop"]) {
128     const body = extractFunctionBody(src, fn);
129     if (!body) continue;
130     const lcdRe = new RegExp(`\\b${varName}\\.(\\w+)\\s*\\(`, "gi");
131     let m;
132     while ((m = lcdRe.exec(body)) !== null) {
133       const op = m[1].toLowerCase();
134       const open = m.index + m[0].length - 1;
135       const args = extractParenArg(body, open);
136       if (op === "init" || op === "begin") {
137         if (!initDone) {
138           encodeHd44780Init(wave, addr8, backlight);
139           initDone = true;
140         }
141         continue;
142       }
143       if (!initDone) {
144         encodeHd44780Init(wave, addr8, backlight);
145         initDone = true;
146       }
147       if (op === "backlight") {
148         backlight = true;
149         continue;
150       }
151       if (op === "noblacklight") {
152         backlight = false;
153         continue;
154       }
155       if (op === "clear") {
156         encodeCommand(wave, addr8, LCD_CLEAR, backlight);
157         continue;
158       }

```

```
159     if (op === "scrolldisplayleft") {
160         encodeCommand(wave, addr8, LCD_SHIFT_LEFT, backlight);
161         continue;
162     }
163     if (op === "scrolldisplayright") {
164         encodeCommand(wave, addr8, LCD_SHIFT_RIGHT, backlight);
165         continue;
166     }
167     if (op === "home") {
168         encodeSetCursor(wave, addr8, 0, 0, backlight);
169         continue;
170     }
171     if (op === "setcursor") {
172         const parts = args.split(",").map((x) => x.trim());
173         const col = Math.max(0, parseInt(parts[0], 10) || 0);
174         const row = Math.max(0, parseInt(parts[1], 10) || 0);
175         encodeSetCursor(wave, addr8, col, row, backlight);
176         continue;
177     }
178     if (op === "print" || op === "write") {
179         const text = parsePrintArg(args.split(",")[0]);
180         if (text) encodePrintText(wave, addr8, text, backlight);
181     }
182 }
183
184
185 if (!initDone) {
186     encodeHd44780Init(wave, addr8, backlight);
187 }
188
189 const once = wave.toPwl();
190 return repeatI2cWaveform(once, repeatUntilSec);
191 }
192
```

```

1  /**
2   * 74HC90 / 74LS90 – compteur décade asynchrone (÷2 + ÷5).
3   * Brochage TI : CP1=1, MR1=2, MR2=3, VCC=5, MS1=6, MS2=7, Q2=8, Q1=9, GND=10, Q3=11, Q0=12, CP0=14.
4   */
5   import {
6     appendLogicJkNetlist,
7     useLogicJkXspice,
8   } from "./logic-sequential.mjs";
9   import { logicJkXspiceInternalNodeKeys } from "./logic-xspice.mjs";
10
11  export const IC90_PIN = {
12    CP1: 0,
13    MR1: 1,
14    MR2: 2,
15    NC4: 3,
16    VCC: 4,
17    MS1: 5,
18    MS2: 6,
19    Q2: 7,
20    Q1: 8,
21    GND: 9,
22    Q3: 10,
23    Q0: 11,
24    NC13: 12,
25    CP0: 13,
26  };
27
28  export function isIc74hc90Type(t) {
29    return t === "ic_74hc90";
30  }
31
32  export function ic74hc90VccPinIndex() {
33    return IC90_PIN.VCC;
34  }
35
36  export function ic74hc90GndPinIndex() {
37    return IC90_PIN.GND;
38  }
39
40  /** Sections internes : bascule T (D=Q) par indice de broche Q et CLK. */
41  export function ic74hc90ToggleSlices() {
42    return [
43      { suffix: "u0", q: IC90_PIN.Q0, clk: IC90_PIN.CP0 },
44      { suffix: "u1", q: IC90_PIN.Q1, clk: IC90_PIN.CP1, rippleFromQ: true },
45      { suffix: "u2", q: IC90_PIN.Q2, clk: IC90_PIN.Q1, rippleFromQ: true },
46      { suffix: "u3", q: IC90_PIN.Q3, clk: IC90_PIN.Q2, rippleFromQ: true },
47    ];
48  }
49
50  export function ic74hc90OutputNodeKeys(c) {
51    return ic74hc90ToggleSlices().map((sl) => `${c.id}#${sl.q}`);
52  }
53
54  export function ic74hc90InputNodeKeys(c) {
55    return [
56      `${c.id}#${IC90_PIN.CP0}`,
57      `${c.id}#${IC90_PIN.CP1}`,
58      `${c.id}#${IC90_PIN.MR1}`,
59      `${c.id}#${IC90_PIN.MR2}`,
60      `${c.id}#${IC90_PIN.MS1}`,
61      `${c.id}#${IC90_PIN.MS2}`,
62    ];
63  }
64
65  /** Sous-bascules JK internes (suffixes) – voir appendIc74hc90Netlist. */
66  const IC90_JK_SUFFIXES = ["u0", "u1", "u2", "u3"];
67
68  /** Nœuds internes d'une sous-bascule JK selon le moteur (XSPICE d_jkff ou source B). */
69  function ic90JkSubInternalKeys(subId, opts) {
70    if (useLogicJkXspice(opts)) {
71      return logicJkXspiceInternalNodeKeys({ id: subId }, {});
72    }
73    return [`${subId}#__qi`, `${subId}#__clkedge`, `${subId}#__qbar`];
74  }
75
76  export function ic74hc90InternalNodeKeys(c, opts = {}) {
77    const keys = [
78      `${c.id}#__mr`,

```

```

79     `${c.id}#__ms`,
80     `${c.id}#__rst_mid`,
81     `${c.id}#__set_lo`,
82     `${c.id}#__one`,
83     `${c.id}#__jb`,
84     `${c.id}#__jd`,
85     `${c.id}#__clka`,
86     `${c.id}#__clkbd`,
87     `${c.id}#__clkc`,
88 ];
89 IC90_JK_SUFFIXES.forEach((sfx, i) => {
90     keys.push(`${c.id}#__qb${i}`);
91     for (const k of ic90JkSubInternalKeys(`${c.id}_${sfx}`, opts)) keys.push(k);
92 });
93 return keys;
94 }
95
96 function stepGt(node, th) {
97     return `u(V(${node})-${th})`;
98 }
99
100 /**
101  * 74HC90 modélisé avec 4 bascules JK (datasheet) :
102  *   - FFA (Q0) : diviseur par 2, J=K=1, horloge CP0 (front descendant).
103  *   - Section ÷5 SYNCHRON (CP1) :
104  *       FFB (Q1) : J=/Q3, K=1, horloge CP1↓
105  *       FFC (Q2) : J=K=1, horloge Q1↓ (ripple interne ÷5)
106  *       FFD (Q3) : J=Q1·Q2, K=1, horloge CP1↓
107  * Aucune remise à zéro « décade » asynchrone auto-effaçante : la section ÷5
108  * boucle naturellement (0→4→0), ce qui supprime la course qui faisait
109  * « repartir de 4 » dès qu'on prélevait Q0/Q3 (report cascade).
110  * MR1·MR2 force 0000 ; MS1·MS2 force 1001 (=9).
111  * @param {object} c composant ic_74hc90
112  */
113 export function appendIc74hc90Netlist(c, nodeFor, vhi, lines, spiceBranchName, deckOpts = {}) {
114     const th = vhi > 0 ? vhi / 2 : 2.5;
115     const v = vhi > 0 ? vhi : 5;
116     const B = (suffix) => `${spiceBranchName("B", c.id)}${suffix}`;
117
118     const nMr1 = nodeFor(`${c.id}#${IC90_PIN.MR1}`);
119     const nMr2 = nodeFor(`${c.id}#${IC90_PIN.MR2}`);
120     const nMs1 = nodeFor(`${c.id}#${IC90_PIN.MS1}`);
121     const nMs2 = nodeFor(`${c.id}#${IC90_PIN.MS2}`);
122     const nMr = nodeFor(`${c.id}#__mr`);
123     const nMs = nodeFor(`${c.id}#__ms`);
124     const nRstMid = nodeFor(`${c.id}#__rst_mid`);
125     const nSetLo = nodeFor(`${c.id}#__set_lo`);
126     const nOne = nodeFor(`${c.id}#__one`);
127
128     lines.push(`${B("__mr")}`) $ {nMr} 0 V = { ${stepGt(nMr1, th)}*${stepGt(nMr2, th)}*${v} }`);
129     lines.push(`${B("__ms")}`) $ {nMs} 0 V = { ${stepGt(nMs1, th)}*${stepGt(nMs2, th)}*${v} }`);
130     // Reset des bascules Q1/Q2 : MR OU MS (1e 9 a Q1=Q2=0).
131     lines.push(
132         `${B("__rstm")}`) $ {nRstMid} 0 V = { (${stepGt(nMr, th)}+${stepGt(nMs, th)})-${stepGt(nMr, th)}*${stepGt(nMs, th)}}*${v} }`
133     );
134     lines.push(`${spiceBranchName("V", c.id)}_setlo $ {nSetLo} 0 DC 0`);
135     lines.push(`${B("__one")}`) $ {nOne} 0 V = { ${v} }`);
136
137     const nCp0 = nodeFor(`${c.id}#${IC90_PIN.CP0}`);
138     const nCp1 = nodeFor(`${c.id}#${IC90_PIN.CP1}`);
139     const nQ0 = nodeFor(`${c.id}#${IC90_PIN.Q0}`);
140     const nQ1 = nodeFor(`${c.id}#${IC90_PIN.Q1}`);
141     const nQ2 = nodeFor(`${c.id}#${IC90_PIN.Q2}`);
142     const nQ3 = nodeFor(`${c.id}#${IC90_PIN.Q3}`);
143
144     // Entrées J combinatoires de la section ÷5.
145     const nJb = nodeFor(`${c.id}#__jb`); // FFB : J = /Q3
146     lines.push(`${B("__jb")}`) $ {nJb} 0 V = { (1-${stepGt(nQ3, th)})*${v} }`);
147     const nJd = nodeFor(`${c.id}#__jd`); // FFD : J = Q1·Q2
148     lines.push(`${B("__jd")}`) $ {nJd} 0 V = { ${stepGt(nQ1, th)}*${stepGt(nQ2, th)}*${v} }`);
149
150     // Horloges inversées : déclenchement sur front descendant (74HC90 = négatif).
151     const nClkA = nodeFor(`${c.id}#__clka`); // /CP0
152     const nClkBD = nodeFor(`${c.id}#__clkbd`); // /CP1
153     const nClkC = nodeFor(`${c.id}#__clkc`); // /Q1 (ripple FFC)
154     // Pendant MR actif, geler toutes les horloges : sinon Q1↓ lors du reset asynchrone
155     // horloge FFC (J=K=1) et le compteur se fige à 4 ; un front CP0 résiduel
156     // (report cascade) peut aussi corrompre l'état juste après le reset.
157     const mrOn = stepGt(nMr, th);
158     lines.push(

```

```

159   `${B("_clkA")} ${nClkA} 0 V = { (1-(${mrOn}))*(${v} - V(${nCp0})) + (${mrOn})*${v} }`
160   );
161   lines.push(
162     `${B("_clkBD")} ${nClkBD} 0 V = { (1-(${mrOn}))*(${v} - V(${nCp1})) + (${mrOn})*${v} }`
163   );
164   lines.push(
165     `${B("_clkC")} ${nClkC} 0 V = { (1-(${mrOn}))*(${v} - V(${nQ1})) + (${mrOn})*${v} }`
166   );
167
168   const slices = [
169     { sfx: "u0", q: nQ0, clk: nClkA, j: n0ne, k: n0ne, set: nMs, reset: nMr, srWired: { set: true, reset: true } },
170     { sfx: "u1", q: nQ1, clk: nClkBD, j: nJb, k: n0ne, set: nSetLo, reset: nRstMid, srWired: { set: false, reset: true } },
171     { sfx: "u2", q: nQ2, clk: nClkC, j: n0ne, k: n0ne, set: nSetLo, reset: nMr, srWired: { set: false, reset: true } },
172     { sfx: "u3", q: nQ3, clk: nClkBD, j: nJd, k: n0ne, set: nMs, reset: nMr, srWired: { set: true, reset: true } },
173   ];
174
175   slices.forEach((sl, i) => {
176     const subId = `${c.id}_${sl.sfx}`;
177     const nQbar = nodeFor(`${c.id}#_qb${i}`);
178     // Remap des broches JK (#0 J, #1 K, #2 CLK, #3 Q, #4 /Q, #5 Set, #6 Reset).
179     const sub = { id: subId, type: "logic_jk" };
180     const sliceNodeFor = (key) => {
181       if (key === `${subId}#0`) return sl.j;
182       if (key === `${subId}#1`) return sl.k;
183       if (key === `${subId}#2`) return sl.clk;
184       if (key === `${subId}#3`) return sl.q;
185       if (key === `${subId}#4`) return nQbar;
186       if (key === `${subId}#5`) return sl.set;
187       if (key === `${subId}#6`) return sl.reset;
188       return nodeFor(key);
189     };
190     appendLogicJkNetlist(sub, sliceNodeFor, v, lines, spiceBranchName, {
191       ...deckOpts,
192       srWired: sl.srWired,
193     });
194   });
195 }
196
197 export function resolveIc74hc90Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhiFn) {
198   const vccKey = `${c.id}#${ic74hc90VccPinIndex()}`;
199   let vhi = logicVhiByTerminal.get(vccKey) ?? 0;
200   for (const k of ic74hc90InputNodeKeys(c)) {
201     vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
202   }
203   for (const k of ic74hc90OutputNodeKeys(c)) {
204     vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
205   }
206   if (c.logicRail !== null && c.logicRail !== "") {
207     vhi = Math.max(vhi, logicVhiFn(parseLogicRail(c.logicRail)));
208   }
209   return vhi > 0 ? vhi : 5;
210 }
211

```

```

1  /**
2   * CD4511 via sources comportementales (sans XSPICE d_genlut / d_d latch).
3   * Compatible ngspice-39 Linux (Render, Docker).
4   */
5  import {
6    cd4511InputPinIndices,
7    cd4511OutputPinIndices,
8  } from "../logic-cd4511-xspice.mjs";
9
10 const SEG_NAMES = ["a", "b", "c", "d", "e", "f", "g"];
11
12 const BCD_SEGMENTS = [
13   [1, 1, 1, 1, 1, 1, 0],
14   [0, 1, 1, 0, 0, 0, 0],
15   [1, 1, 0, 1, 1, 0, 1],
16   [1, 1, 1, 1, 0, 0, 1],
17   [0, 1, 1, 0, 0, 1, 1],
18   [1, 0, 1, 1, 0, 1, 1],
19   [1, 0, 1, 1, 1, 1, 1],
20   [1, 1, 1, 0, 0, 0, 0],
21   [1, 1, 1, 1, 1, 1, 1],
22   [1, 1, 1, 1, 0, 1, 1],
23 ];
24
25 const LATCH_STATE_C_F = 100e-9;
26 const LATCH_Q_R_OHM = 10e3;
27
28 function stepGt(node, th) {
29   return `u(V(${node})-${th})`;
30 }
31
32 function bitMatchExpr(nPin, bitVal, th) {
33   return bitVal ? stepGt(nPin, th) : `(1-${stepGt(nPin, th)})`;
34 }
35
36 /** Somme de produits BCD → un segment (après latch). */
37 export function bcdLatchSegDecodeExpr(nA, nB, nC, nD, segIdx, th) {
38   const terms = [];
39   for (let d = 0; d <= 9; d++) {
40     if (!BCD_SEGMENTS[d][segIdx]) continue;
41     const t = [
42       bitMatchExpr(nA, d & 1, th),
43       bitMatchExpr(nB, d & 2, th),
44       bitMatchExpr(nC, d & 4, th),
45       bitMatchExpr(nD, d & 8, th),
46     ].join("*");
47     terms.push(`(${t})`);
48   }
49   return terms.length ? `(${terms.join("+)})` : "0";
50 }
51
52 export function logicCd4511BsourceInternalNodeKeys(c) {
53   if (!c?.id) return [];
54   const keys = [];
55   for (const s of ["a", "b", "c", "d"]) {
56     keys.push(`__inbuf_${s}`, `__lat_${s}`, `__lat_${s}_qi`);
57   }
58   return keys.map((k) => `${c.id}#${k}`);
59 }
60
61 /**
62  * @param {object} c composant logic_cd4511
63  */
64 export function appendLogicCd4511BsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
65   const th = vhi > 0 ? vhi / 2 : 2.5;
66   const v = vhi > 0 ? vhi : 5;
67   const pins = cd4511InputPinIndices();
68   const outs = cd4511OutputPinIndices();
69
70   const nA = nodeFor(`${c.id}#${pins.A}`);
71   const nB = nodeFor(`${c.id}#${pins.B}`);
72   const nC = nodeFor(`${c.id}#${pins.C}`);
73   const nD = nodeFor(`${c.id}#${pins.D}`);
74   const nLE = nodeFor(`${c.id}#${pins.LE}`);
75   if (!opts.leWired) {
76     lines.push(`${spiceBranchName("V", c.id)}_lep ${nLE} 0 DC 0`);
77   }
78   const nBI = nodeFor(`${c.id}#${pins.BI}`);

```

```

79     const nLT = nodeFor(`${c.id}#${pins.LT}`);
80     if (!opts.biWired) {
81         lines.push(`${spiceBranchName("V", c.id)}_bipu ${nBI} 0 DC ${v}`);
82     }
83     if (!opts.ltWired) {
84         lines.push(`${spiceBranchName("V", c.id)}_ltpu ${nLT} 0 DC ${v}`);
85     }
86
87     const leTransparent = `(1-${stepGt(nLE, th)})`;
88
89     // Entrées BCD lues via VCVS (haute impédance) : évite de perturber les Q des bascules
90     // ripple au reset asynchrone (symptôme 9→4 au lieu de 9→0 quand B/D partagent le fil AND).
91     const inputBuf = (suffix, nIn) => {
92         const nBuf = nodeFor(`${c.id}#__inbuf_${suffix}`);
93         lines.push(`${spiceBranchName("E", c.id)}_buf${suffix} ${nBuf} 0 ${nIn} 0 1`);
94         return nBuf;
95     };
96
97     const latchBit = (suffix, nIn) => {
98         const nLat = nodeFor(`${c.id}#__lat_${suffix}`);
99         const nQi = nodeFor(`${c.id}#__lat_${suffix}_qi`);
100         const inHi = stepGt(nIn, th);
101         lines.push(
102             `${spiceBranchName("B", c.id)}_lat${suffix} ${nQi} 0 V = { (${leTransparent})*${inHi}+(1-${leTransparent})*${stepGt(nLat, th)}}*
103             ${v} }`
104         );
105         lines.push(`${spiceBranchName("R", c.id)}_lat${suffix} ${nQi} ${nLat} ${LATCH_Q_R_OHM}`);
106         lines.push(`${spiceBranchName("C", c.id)}_lat${suffix} ${nLat} 0 ${LATCH_STATE_C_F}`);
107         return nLat;
108     };
109
110     const nLa = latchBit("a", inputBuf("a", nA));
111     const nLb = latchBit("b", inputBuf("b", nB));
112     const nLc = latchBit("c", inputBuf("c", nC));
113     const nLd = latchBit("d", inputBuf("d", nD));
114
115     const biOn = stepGt(nBI, th);
116     const ltActive = `(1-${stepGt(nLT, th)})`;
117
118     SEG_NAMES.forEach((s, segIdx) => {
119         const decode = bcdLatchSegDecodeExpr(nLa, nLb, nLc, nLd, segIdx, th);
120         const nOut = nodeFor(`${c.id}#${outs[s]}`);
121         lines.push(
122             `${spiceBranchName("B", c.id)}_seg${s} ${nOut} 0 V = { (${biOn})*(${ltActive}+${decode}*(1-${ltActive}))}*${v} }`
123         );
124     });
125 }

```



```

1  /**
2   * CD4511 (BCD → 7 segments, latch LE, BI, LT) via XSPICE :
3   * adc_bridge, d_d latch (x4), d_genlut, d_or / d_and, dac_bridge.
4   */
5   import { xspiceLogicThresholds } from "./logic-xspice.mjs";
6
7   /** Chiffres 0-9 : segments a,b,c,d,e,f,g (1 = segment actif, cathode commune). */
8   const BCD_SEGMENTS = [
9     [1, 1, 1, 1, 1, 1, 0],
10    [0, 1, 1, 0, 0, 0, 0],
11    [1, 1, 0, 1, 1, 0, 1],
12    [1, 1, 1, 1, 0, 0, 1],
13    [0, 1, 1, 0, 0, 1, 1],
14    [1, 0, 1, 1, 0, 1, 1],
15    [1, 0, 1, 1, 1, 1, 1],
16    [1, 1, 1, 0, 0, 0, 0],
17    [1, 1, 1, 1, 1, 1, 1],
18    [1, 1, 1, 1, 0, 1, 1],
19  ];
20
21  /** table_values pour d_genlut : 4 entrées [A B C D], 7 sorties a..g (concaténées). */
22  export function cd4511BcdToSegGenlutTable() {
23    let table = "";
24    for (let seg = 0; seg < 7; seg++) {
25      table += cd4511SegGenlutTable(seg);
26    }
27    return table;
28  }
29
30  /** Une sortie 7-seg : 16 bits (2^4 entrées BCD) pour compatibilité ngspice-39 (1 sortie / d_genlut). */
31  export function cd4511SegGenlutTable(segIndex) {
32    let table = "";
33    for (let val = 0; val < 16; val++) {
34      if (val <= 9 && BCD_SEGMENTS[val][segIndex]) table += "1";
35      else table += "0";
36    }
37    return table;
38  }
39
40  export function cd4511InputPinIndices() {
41    return { A: 0, B: 1, C: 2, D: 3, LE: 4, BI: 5, LT: 6 };
42  }
43
44  export function cd4511OutputPinIndices() {
45    return { a: 7, b: 8, c: 9, d: 10, e: 11, f: 12, g: 13 };
46  }
47
48  export function cd4511InputNodeKeys(c) {
49    const p = cd4511InputPinIndices();
50    return Object.values(p).map((i) => `${c.id}#${i}`);
51  }
52
53  export function cd4511OutputNodeKeys(c) {
54    const p = cd4511OutputPinIndices();
55    return Object.values(p).map((i) => `${c.id}#${i}`);
56  }
57
58  const SEG_NAMES = ["a", "b", "c", "d", "e", "f", "g"];
59
60  /**
61   * @param {object} c composant logic_cd4511
62   * @param {function} nodeFor
63   * @param {number} vhi
64   * @param {string[]} lines
65   * @param {function} spiceBranchName
66   * @param {{ biWired?: boolean; ltWired?: boolean; leWired?: boolean }} [opts]
67   */
68  export function appendLogicCd4511XspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
69    const th = xspiceLogicThresholds(vhi);
70    const v = vhi > 0 ? vhi : 5;
71    const pins = cd4511InputPinIndices();
72    const outs = cd4511OutputPinIndices();
73
74    const nA = nodeFor(`${c.id}#${pins.A}`);
75    const nB = nodeFor(`${c.id}#${pins.B}`);
76    const nC = nodeFor(`${c.id}#${pins.C}`);
77    const nD = nodeFor(`${c.id}#${pins.D}`);
78    const nLE = nodeFor(`${c.id}#${pins.LE}`);

```

```

79   if (!opts.leWired) {
80     lines.push(`${spiceBranchName("V", c.id)}_lep ${nLE} 0 DC 0`);
81   }
82   const nBI = nodeFor(`${c.id}#${pins.BI}`);
83   const nLT = nodeFor(`${c.id}#${pins.LT}`);
84
85   const mAdc = `${c.id}_m_adcio`;
86   const mDac = `${c.id}_m_dac`;
87   lines.push(`.model ${mAdc} adc_bridge(in_low=${th.inLow} in_high=${th.inHigh})`);
88   lines.push(
89     `.model ${mDac} dac_bridge(out_low=${th.outLow} out_high=${th.outHigh} t_rise=2e-9 t_fall=2e-9)`
90   );
91
92   const ndZero = nodeFor(`${c.id}#__xd_zero`);
93   const nZa = nodeFor(`${c.id}#__xa_zero`);
94   lines.push(`${spiceBranchName("V", c.id)}_xz ${nZa} 0 DC 0`);
95   lines.push(`${spiceBranchName("A", c.id)}_adc [${nZa}] [${ndZero}] [${mAdc}]`);
96
97   let ndPullHi = null;
98   const pullHighDigital = () => {
99     if (ndPullHi) return ndPullHi;
100    const nPullA = nodeFor(`${c.id}#__xa_pull`);
101    ndPullHi = nodeFor(`${c.id}#__xd_pull`);
102    lines.push(`${spiceBranchName("V", c.id)}_pull ${nPullA} 0 DC ${v}`);
103    lines.push(`${spiceBranchName("A", c.id)}_adcpull [${nPullA}] [${ndPullHi}] [${mAdc}]`);
104    return ndPullHi;
105  };
106
107  const adcIn = (suffix, nAnalog) => {
108    const nd = nodeFor(`${c.id}#__xd_in_${suffix}`);
109    lines.push(`${spiceBranchName("A", c.id)}_adc${suffix} [${nAnalog}] [${nd}] [${mAdc}]`);
110    return nd;
111  };
112
113  const ndA = adcIn("a", nA);
114  const ndB = adcIn("b", nB);
115  const ndC = adcIn("c", nC);
116  const ndD = adcIn("d", nD);
117
118  const ndBI = opts.biWired ? adcIn("bi", nBI) : pullHighDigital();
119  let ndLTbar;
120  if (opts.ltWired) {
121    const nLTinvA = nodeFor(`${c.id}#__xa_ltinv`);
122    lines.push(`${spiceBranchName("B", c.id)}_ltinv ${nLTinvA} 0 V = { ${v} - V(${nLT}) }`);
123    ndLTbar = nodeFor(`${c.id}#__xd_ltbar`);
124    lines.push(`${spiceBranchName("A", c.id)}_adcltbar [${nLTinvA}] [${ndLTbar}] [${mAdc}]`);
125  } else {
126    lines.push(`${spiceBranchName("V", c.id)}_ltpu ${nLT} 0 DC ${v}`);
127    ndLTbar = ndZero;
128  }
129
130  // LE actif bas → transparent ; d_d latch : enable=1 suit data, enable=0 mémorise.
131  const nLEenA = nodeFor(`${c.id}#__xa_le_en`);
132  lines.push(`${spiceBranchName("B", c.id)}_leen ${nLEenA} 0 V = { ${v} - V(${nLE}) }`);
133  const ndLEen = nodeFor(`${c.id}#__xd_le_en`);
134  lines.push(`${spiceBranchName("A", c.id)}_adcleen [${nLEenA}] [${ndLEen}] [${mAdc}]`);
135
136  const mLatch = `${c.id}_m_d latch`;
137  lines.push(
138    `.model ${mLatch} d_d latch(data_delay=2e-9 enable_delay=2e-9 rise_delay=2e-9 fall_delay=2e-9 ic=0)`
139  );
140
141  const latchBit = (bit, ndIn) => {
142    const ndLat = nodeFor(`${c.id}#__xd_lat_${bit}`);
143    const ndQb = nodeFor(`${c.id}#__xd_lat_${bit}qb`);
144    lines.push(
145      `${spiceBranchName("A", c.id)}_lt${bit} ${ndIn} [${ndLEen}] [${ndZero}] [${ndZero}] [${ndLat}] [${ndQb}] [${mLatch}]`
146    );
147    return ndLat;
148  };
149
150  const ndLa = latchBit("a", ndA);
151  const ndLb = latchBit("b", ndB);
152  const ndLc = latchBit("c", ndC);
153  const ndLd = latchBit("d", ndD);
154
155  // 7× d_genlut 4→1 (évite « Defaulted array parameter... » sur ngspice Linux / Render).
156  const ndSeg = SEG_NAMES.map((s, segIdx) => {
157    const mSeg = `${c.id}_m_lut_${s}`;
158    const ndOut = nodeFor(`${c.id}#__xd_seg_${s}`);

```

```

159     lines.push(`.model ${mSeg} d_genlut(table_values="${cd4511SegGenlutTable(segIdx)}")`);
160     lines.push(
161         `${spiceBranchName("A", c.id)}_dec${s} [${ndLa} ${ndLb} ${ndLc} ${ndLd}] [${ndOut}] ${mSeg}`
162     );
163     return ndOut;
164 });
165
166 const mOr = `${c.id}_m_or`;
167 const mAnd = `${c.id}_m_and`;
168 lines.push(`.model ${mOr} d_or(rise_delay=2e-9 fall_delay=2e-9)`);
169 lines.push(`.model ${mAnd} d_and(rise_delay=2e-9 fall_delay=2e-9)`);
170
171 SEG_NAMES.forEach((s, i) => {
172     const ndTmp = nodeFor(`${c.id}#__xd_${s}_pre`);
173     const ndOutD = nodeFor(`${c.id}#__xd_${s}_out`);
174     // LT actif bas : test lampe = (-LT) v segment
175     lines.push(`${spiceBranchName("A", c.id)}_or${s} [${ndLTbar} ${ndSeg[i]}] ${ndTmp} ${mOr}`);
176     lines.push(`${spiceBranchName("A", c.id)}_and${s} [${ndTmp} ${ndBI}] ${ndOutD} ${mAnd}`);
177     const nOut = nodeFor(`${c.id}#${outs[s]}`);
178     lines.push(`${spiceBranchName("A", c.id)}_dac${s} [${ndOutD}] [${nOut}] ${mDac}`);
179 });
180 }
181
182 export function logicCd4511XspiceInternalNodeKeys(c) {
183     if (!c?.id) return [];
184     const keys = ["__xd_zero", "__xa_zero", "__xa_pull", "__xd_pull", "__xa_le_en", "__xd_le_en"];
185     for (const s of ["a", "b", "c", "d"]) {
186         keys.push(`__xd_in_${s}`, `__xd_lat_${s}`, `__xd_lat_${s}qb`);
187     }
188     keys.push("__xd_in_bi", "__xa_ltin", "__xd_ltbar");
189     for (const s of SEG_NAMES) {
190         keys.push(`__xd_seg_${s}`, `__xd_${s}_pre`, `__xd_${s}_out`);
191     }
192     return keys.map((k) => `${c.id}#${k}`);
193 }
194

```

```

1  /** Niveaux logiques en tension (analogique ngspice) : 0 V et rail 3,3 V ou 5 V. */
2
3  export function parseLogicRail(raw) {
4    if (raw == null || raw === "") return 5;
5    const n = parseFloat(String(raw).replace(",", "."));
6    return Number.isFinite(n) && Math.abs(n - 3.3) < 0.05 ? 3.3 : 5;
7  }
8
9  export function toggleLogicRail(rail) {
10    return parseLogicRail(rail) === 3.3 ? 5 : 3.3;
11  }
12
13  export function logicVhi(rail) {
14    return parseLogicRail(rail);
15  }
16
17  export function logicVth(rail) {
18    return logicVhi(rail) / 2;
19  }
20
21  export function logicLevelFromVoltage(v, railOrVth) {
22    if (!Number.isFinite(v)) return null;
23    const th =
24      typeof railOrVth === "number" && railOrVth > 0 && railOrVth < 3
25        ? railOrVth
26        : logicVth(railOrVth);
27    return v >= th ? "1" : "0";
28  }
29
30  export function formatLogicRailLabel(rail) {
31    return parseLogicRail(rail) === 3.3 ? "3,3 V" : "5 V";
32  }
33
34  export function parseLogicStateVolts(value, rail) {
35    const t = String(value ?? "")
36      .trim()
37      .toLowerCase();
38    if (t === "1" || t === "haut" || t === "high" || t === "true" || t === "on" || t === "vcc") {
39      return logicVhi(rail);
40    }
41    return 0;
42  }
43

```

```

1  /** Bascules et circuits intégrés logiques – ngspice (XSPICE d_dff ou sources B). */
2
3  import {
4    appendLogicCd4511BsourceNetlist,
5    logicCd4511BsourceInternalNodeKeys,
6  } from "./logic-cd4511-bsource.mjs";
7  import {
8    appendLogicCd4511XspiceNetlist,
9    cd4511InputNodeKeys,
10   cd4511OutputNodeKeys,
11   logicCd4511XspiceInternalNodeKeys,
12 } from "./logic-cd4511-xspice.mjs";
13 import {
14   appendLogicDffXspiceNetlist,
15   appendLogicJkXspiceNetlist,
16   isXspiceDffAvailable,
17   logicDffXspiceInternalNodeKeys,
18   logicJkXspiceInternalNodeKeys,
19   xspiceCodemodelLines,
20 } from "./logic-xspice.mjs";
21 import { ngspiceHasXspice } from "./ngspice-xspice-probe.mjs";
22
23 export { cd4511InputNodeKeys, cd4511OutputNodeKeys };
24
25 export function isLogicSequentialType(t) {
26   return t === "logic_dff" || t === "logic_jk";
27 }
28
29 export function isIc74ls00Type(t) {
30   return t === "ic_74ls00";
31 }
32
33 export function isIc74ls74Type(t) {
34   return t === "ic_74ls74";
35 }
36
37 export function isLogicCd4511Type(t) {
38   return t === "logic_cd4511";
39 }
40
41 export function isIc74hc90Type(t) {
42   return t === "ic_74hc90";
43 }
44
45 export function isLogicIcType(t) {
46   return isIc74ls00Type(t) || isIc74ls74Type(t) || isLogicCd4511Type(t) || isIc74hc90Type(t);
47 }
48
49 export function isLogicDigitalSimType(t) {
50   return isLogicSequentialType(t) || isLogicIcType(t);
51 }
52
53 /** Nœuds internes (pré-déclaration union-find dans schematic-to-spice). */
54 export function logicSequentialInternalNodeKeys(c, opts = {}) {
55   if (!c?.id) return [];
56   if (c.type === "logic_dff" && useLogicDffXspice(opts)) {
57     return logicDffXspiceInternalNodeKeys(c, opts);
58   }
59   if (c.type === "logic_dff") {
60     return [`${c.id}#__qi`, `${c.id}#__clkedge`, `${c.id}#__qbar`];
61   }
62   if (c.type === "logic_jk") {
63     if (useLogicJkXspice(opts)) {
64       return logicJkXspiceInternalNodeKeys(c, opts);
65     }
66     return [`${c.id}#__qi`, `${c.id}#__clkedge`, `${c.id}#__qbar`];
67   }
68   if (isIc74ls74Type(c.type)) {
69     const keys = [];
70     for (const sl of ic74ls74DffSlices()) {
71       keys.push(`${c.id}#__qi${sl.suffix}`, `${c.id}#__clkedge${sl.suffix}`);
72     }
73     return keys;
74   }
75   if (isLogicCd4511Type(c.type)) {
76     return useLogicCd4511Xspice(opts)
77       ? logicCd4511XspiceInternalNodeKeys(c)
78       : logicCd4511BsourceInternalNodeKeys(c);

```

```

79     }
80     return [];
81 }
82
83 export function logicDffInputNodeKeys(c) {
84     return [`${c.id}#0`, `${c.id}#1`];
85 }
86
87 /** Broches Set (#4) / Reset (#5) asynchrones, optionnelles. */
88 export function logicDffSetResetNodeKeys(c) {
89     return [`${c.id}#4`, `${c.id}#5`];
90 }
91
92 export function logicDffOutputNodeKey(c) {
93     return `${c.id}#2`;
94 }
95
96 export function logicDffQbarOutputNodeKey(c) {
97     return `${c.id}#3`;
98 }
99
100 export function logicDffOutputNodeKeys(c) {
101     return [logicDffOutputNodeKey(c), logicDffQbarOutputNodeKey(c)];
102 }
103
104 export function logicJkInputNodeKeys(c) {
105     return [`${c.id}#0`, `${c.id}#1`, `${c.id}#2`];
106 }
107
108 /** Broches Set (#5) / Reset (#6) asynchrones, optionnelles. */
109 export function logicJkSetResetNodeKeys(c) {
110     return [`${c.id}#5`, `${c.id}#6`];
111 }
112
113 export function logicJkOutputNodeKey(c) {
114     return `${c.id}#3`;
115 }
116
117 export function logicJkQbarOutputNodeKey(c) {
118     return `${c.id}#4`;
119 }
120
121 export function logicJkOutputNodeKeys(c) {
122     return [logicJkOutputNodeKey(c), logicJkQbarOutputNodeKey(c)];
123 }
124
125 /** Quatre portes NAND du 74LS00 (indices de bornes #0...#13). */
126 export function ic74ls00NandGates() {
127     return [
128         { a: 0, b: 1, y: 2 },
129         { a: 3, b: 4, y: 5 },
130         { a: 8, b: 9, y: 7 },
131         { a: 11, b: 12, y: 10 },
132     ];
133 }
134
135 export function ic74ls00VccPinIndex() {
136     return 13;
137 }
138
139 export function ic74ls74VccPinIndex() {
140     return 13;
141 }
142
143 /** Double bascule D 74LS74 : broches D, CLK, Q (indices #0...#13). */
144 export function ic74ls74DffSlices() {
145     return [
146         { suffix: "1", d: 1, clk: 2, q: 4 },
147         { suffix: "2", d: 11, clk: 10, q: 8 },
148     ];
149 }
150
151 /** RC plus lents : évite « Timestep too small » sur __qi en .tran (surtout D relié à /Q). */
152 const CLK_EDGE_C_F = 1e-9;
153 const CLK_EDGE_R_OHM = 10e3;
154 const FF_STATE_C_F = 10e-9;
155 const FF_Q_R_OHM = 1e3;
156 const FF_QBAR_R_OHM = 1e3;
157
158 /** Encore plus lent si D et /Q sont court-circuités (diviseur par 2) – ngspice-46. */

```

```

159 const TOGGLE_CLK_EDGE_C_F = 10e-9;
160 const TOGGLE_CLK_EDGE_R_OHM = 100e3;
161 const TOGGLE_FF_STATE_C_F = 100e-9;
162 const TOGGLE_FF_Q_R_OHM = 10e3;
163 const TOGGLE_D_HOLD_C_F = 50e-12;
164
165 function stepGt(node, th) {
166   return `u(V(${node})-${th})`;
167 }
168
169 /**
170  * Front montant sans delay() ni ddt(clk) : RC entre horloge et nœud « edge ».
171  * Impulsion courte quand V(clk) >> V(edge) (condensateur en charge).
172  */
173 function risingEdgeFromRc(edgeNode, clkNode, th) {
174   const pulseTh = Math.max(0.25, th * 0.12);
175   return `(u(V(${clkNode})-${th})*u(V(${clkNode})-V(${edgeNode})-${pulseTh}))`;
176 }
177
178 /** Front descendant (compteur ripple : horloge = Q de l'étage précédent). */
179 function fallingEdgeFromRc(edgeNode, clkNode, th) {
180   const pulseTh = Math.max(0.25, th * 0.12);
181   return `(u(V(${edgeNode})-${th})*u(V(${edgeNode})-V(${clkNode})-${pulseTh}))`;
182 }
183
184 /** Horloge reliée à la sortie Q d'une autre bascule (chainage ripple). */
185 export function logicFlipFlopClockNodeKey(c) {
186   if (c.type === "logic_dff") return `${c.id}#1`;
187   if (c.type === "logic_jk") return `${c.id}#2`;
188   return null;
189 }
190
191 export function logicFlipFlopOutputNodeKey(c) {
192   if (c.type === "logic_dff") return `${c.id}#2`;
193   if (c.type === "logic_jk") return `${c.id}#3`;
194   return null;
195 }
196
197 export function logicFlipFlopQbarOutputNodeKey(c) {
198   if (c.type === "logic_dff") return logicDffQbarOutputNodeKey(c);
199   if (c.type === "logic_jk") return logicJkQbarOutputNodeKey(c);
200   return null;
201 }
202
203 /**
204  * Chainage ripple : horloge = Q ou /Q de l'étage précédent.
205  * @returns {'q'|'qbar'|null}
206  */
207 export function getRippleClockFromPrev(c, wires, components) {
208   if (!c?.id || !isLogicSequentialType(c.type)) return null;
209   const clkKey = logicFlipFlopClockNodeKey(c);
210   if (!clkKey) return null;
211   for (const w of wires) {
212     if (!w?.solid || !w.fromKey || !w.toKey) continue;
213     let otherKey = null;
214     if (w.fromKey === clkKey) otherKey = w.toKey;
215     else if (w.toKey === clkKey) otherKey = w.fromKey;
216     if (!otherKey) continue;
217     for (const prev of components) {
218       if (!isLogicSequentialType(prev.type) || prev.id === c.id) continue;
219       if (otherKey === logicFlipFlopOutputNodeKey(prev)) return "q";
220       if (otherKey === logicFlipFlopQbarOutputNodeKey(prev)) return "qbar";
221     }
222   }
223   return null;
224 }
225
226 /** @deprecated Utiliser getRippleClockFromPrev(c) === 'q' */
227 export function isRippleClockFromPrevQ(c, wires, components) {
228   return getRippleClockFromPrev(c, wires, components) === "q";
229 }
230
231 function appendClkEdgeDetector(clkNode, edgeNode, lines, spiceBranchName, idLabel, cF, rOhm) {
232   lines.push(`${spiceBranchName("C", idLabel)}_cedge ${clkNode} ${edgeNode} ${cF}`);
233   lines.push(`${spiceBranchName("R", idLabel)}_cedge ${edgeNode} 0 ${rOhm}`);
234 }
235
236 export function logicNandBsourceExpression(aNode, bNode, vhi) {
237   const th = vhi / 2;
238   return `{ (1-${stepGt(aNode, th)})*${stepGt(bNode, th)}*${vhi} `;

```

```

239 }
240
241 export function logicDffQiExpression(
242     dNode,
243     edgeNode,
244     clkNode,
245     qOutNode,
246     vhi,
247     dFromComplementOfQ = false,
248     setNode = null,
249     resetNode = null,
250     useFallingClockEdge = false
251 ) {
252     const th = vhi / 2;
253     const edge = useFallingClockEdge
254         ? fallingEdgeFromRc(edgeNode, clkNode, th)
255         : risingEdgeFromRc(edgeNode, clkNode, th);
256     // Si D et /Q sont sur le même nœud, ne pas lire V(D) dans B_qi : boucle avec B_qbar → nœud D.
257     // On impose alors D = ~Q au front (diviseur par 2), cohérent avec le câblage.
258     const dVal = dFromComplementOfQ
259         ? ~(1-$stepGt(qOutNode, th))*${vhi}`
260         : `${stepGt(dNode, th))*${vhi}`;
261     const hold = `${stepGt(qOutNode, th))*${vhi}`;
262     const clocked = `${edge})*(${dVal}) + (1-(${edge}))*(${hold})`;
263     if (setNode == null && resetNode == null) {
264         return `{ ${clocked} }`;
265     }
266     // Set / Reset asynchrones, actifs à l'état haut, Reset prioritaire sur Set.
267     const set = setNode != null ? stepGt(setNode, th) : "0";
268     const rst = resetNode != null ? stepGt(resetNode, th) : "0";
269     return `{ (1-(${rst}))* ( (${set})*${vhi} + (1-(${set}))*(${clocked}) ) }`;
270 }
271
272 export function logicJkQiExpression(jNode, kNode, edgeNode, clkNode, qOutNode, vhi, setNode = null, resetNode = null, useFallingClockEdge = false) {
273     const th = vhi / 2;
274     const edge = useFallingClockEdge
275         ? fallingEdgeFromRc(edgeNode, clkNode, th)
276         : risingEdgeFromRc(edgeNode, clkNode, th);
277     const j = stepGt(jNode, th);
278     const k = stepGt(kNode, th);
279     const q = stepGt(qOutNode, th);
280     const toggle = `(1-${q})*${vhi}`;
281     const hold = `${q}*${vhi}`;
282     const next = `((${j})*(${k})*(${toggle}) + (${j})*(1-(${k}))*${vhi} + (1-(${j}))*(${k})*0 + (1-(${j}))* (1-(${k}))*(${hold})`;
283     const clocked = `${edge})*(${next}) + (1-(${edge}))*(${hold})`;
284     if (setNode == null && resetNode == null) {
285         return `{ ${clocked} }`;
286     }
287     const set = setNode != null ? stepGt(setNode, th) : "0";
288     const rst = resetNode != null ? stepGt(resetNode, th) : "0";
289     return `{ (1-(${rst}))* ( (${set})*${vhi} + (1-(${set}))*(${clocked}) ) }`;
290 }
291
292 function appendQbarDriver(c, nQ, nQbarPin, nodeFor, vhi, lines, spiceBranchName, rSeries = FF_QBAR_R_OHM) {
293     const th = vhi / 2;
294     const nQbarDrv = nodeFor(`${c.id}#__qbar`);
295     lines.push(
296         `${spiceBranchName("B", c.id)}_qbar ${nQbarDrv} 0 V = { (1-${stepGt(nQ, th}))*${vhi} }`
297     );
298     if (nQbarDrv !== nQbarPin) {
299         lines.push(
300             `${spiceBranchName("R", c.id)}_qbr ${nQbarDrv} ${nQbarPin} ${rSeries}`
301         );
302     }
303 }
304
305 /**
306  * true si bascule D via XSPICE : digital.cm présent + binaire ngspice compilé avec XSPICE.
307  * @param {{ repoRoot?: string; ngspiceExe?: string; ngspiceEnv?: NodeJS.ProcessEnv; forceBsourceDff?: boolean; forceXspiceDff?: boolean }} o
308  * pts
309  */
310 export function useLogicDffXspice(opts = {}) {
311     if (opts.forceBsourceDff === true) return false;
312     if (!isXspiceDffAvailable(opts.repoRoot)) return false;
313     if (opts.forceXspiceDff === true) return true;
314     if (opts.ngspiceExe) return ngspiceHasXspice(opts.ngspiceExe, opts.ngspiceEnv);
315     return false;
316 }

```



```

317 export { xspiceCodemodelLines, isXspiceDffAvailable };
318
319 export function appendLogicDffNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
320   if (useLogicDffXspice(opts)) {
321     appendLogicDffXspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
322     return;
323   }
324   appendLogicDffBsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
325 }
326
327 function appendLogicDffBsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
328   const nD = nodeFor(`${c.id}#0`);
329   const nClk = nodeFor(`${c.id}#1`);
330   const nQ = nodeFor(`${c.id}#2`);
331   const nQbar = nodeFor(`${c.id}#3`);
332   const nSet = nodeFor(`${c.id}#4`);
333   const nReset = nodeFor(`${c.id}#5`);
334   const nQi = nodeFor(`${c.id}#_qi`);
335   const nEdge = nodeFor(`${c.id}#__clkedge`);
336   const dAndQbarShared = nD === nQbar;
337   const cEdge = dAndQbarShared ? TOGGLE_CLK_EDGE_C_F : CLK_EDGE_C_F;
338   const rEdge = dAndQbarShared ? TOGGLE_CLK_EDGE_R_OHM : CLK_EDGE_R_OHM;
339   const cSt = dAndQbarShared ? TOGGLE_FF_STATE_C_F : FF_STATE_C_F;
340   const rQ = dAndQbarShared ? TOGGLE_FF_Q_R_OHM : FF_Q_R_OHM;
341   // Tirage à 0 V des broches Set/Reset : inactives (état bas) si non câblées.
342   lines.push(`${spiceBranchName("R", c.id)}_setpd ${nSet} 0 1e9`);
343   lines.push(`${spiceBranchName("R", c.id)}_rstpd ${nReset} 0 1e9`);
344   const vClk = vhi > 0 ? vhi : 5;
345   let nClkDet = nClk;
346   if (opts.clockInvert && opts.rippleClockFromPrev !== "q") {
347     nClkDet = nodeFor(`${c.id}#__xa_clkinv`);
348     lines.push(
349       `${spiceBranchName("B", c.id)}_clkinv ${nClkDet} 0 V = { ${vClk} - V(${nClk}) }`
350     );
351   }
352   appendClkEdgeDetector(nClkDet, nEdge, lines, spiceBranchName, c.id, cEdge, rEdge);
353   lines.push(
354     `${spiceBranchName("B", c.id)}_qi ${nQi} 0 V = ${logicDffQiExpression(
355       nD,
356       nEdge,
357       nClkDet,
358       nQ,
359       vhi,
360       dAndQbarShared,
361       nSet,
362       nReset,
363       opts.rippleClockFromPrev === "q"
364     )}`
365   );
366   lines.push(`${spiceBranchName("R", c.id)}_q ${nQi} ${nQ} ${rQ}`);
367   lines.push(`${spiceBranchName("C", c.id)}_st ${nQi} 0 ${cSt}`);
368   if (dAndQbarShared) {
369     // D et /Q sur le même fil : piloter directement le nœud D (= /Q) depuis Q, sans nœud
370     // interne __qbar + R (sinon ngspice-46 peut bloquer sur n4 en .tran).
371     const th = vhi / 2;
372     lines.push(
373       `${spiceBranchName("B", c.id)}_qbar ${nD} 0 V = { (1-${stepGt(nQ, th)})*${vhi} }`
374     );
375     lines.push(`${spiceBranchName("C", c.id)}_dhold ${nD} 0 ${TOGGLE_D_HOLD_C_F}`);
376   } else {
377     appendQbarDriver(c, nQ, nQbar, nodeFor, vhi, lines, spiceBranchName);
378   }
379 }
380
381 /** true si D et /Q partagent le même nœud (fil entre broches 0 et 3). */
382 export function logicDffDAndQbarShareNode(c, nodeFor) {
383   return nodeFor(`${c.id}#0`) === nodeFor(`${c.id}#3`);
384 }
385
386 export function useLogicJkXspice(opts = {}) {
387   if (opts.forceBsourceJk === true) return false;
388   if (!isXspiceDffAvailable(opts.repoRoot)) return false;
389   if (opts.forceXspiceJk === true) return true;
390   if (opts.ngspiceExe) return ngspiceHasXspice(opts.ngspiceExe, opts.ngspiceEnv);
391   return false;
392 }
393
394 export function useLogicCd4511Xspice(opts = {}) {
395   if (opts.forceBsourceCd4511 === true) return false;
396   if (!isXspiceDffAvailable(opts.repoRoot)) return false;

```

```

397     if (opts.forceXspiceCd4511 === true) return true;
398     if (opts.ngspiceExe) return ngspiceHasXspice(opts.ngspiceExe, opts.ngspiceEnv);
399     return false;
400 }
401
402 export function appendLogicCd4511Netlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
403     if (useLogicCd4511Xspice(opts)) {
404         appendLogicCd4511XspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
405         return;
406     }
407     appendLogicCd4511BsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
408 }
409
410 export function resolveLogicCd4511Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhiFn) {
411     let vhi = 0;
412     for (const k of cd4511InputNodeKeys(c)) {
413         vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
414     }
415     for (const k of cd4511OutputNodeKeys(c)) {
416         vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
417     }
418     if (c.logicRail !== null && c.logicRail !== "") {
419         vhi = Math.max(vhi, logicVhiFn(parseLogicRail(c.logicRail)));
420     }
421     if (vhi <= 0) vhi = 5;
422     return vhi > 0 ? vhi : 5;
423 }
424
425 export function appendLogicJkNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
426     if (useLogicJkXspice(opts)) {
427         appendLogicJkXspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
428         return;
429     }
430     appendLogicJkBsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts);
431 }
432
433 function appendLogicJkBsourceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
434     const nJ = nodeFor(`${c.id}#0`);
435     const nK = nodeFor(`${c.id}#1`);
436     const nClk = nodeFor(`${c.id}#2`);
437     const nQ = nodeFor(`${c.id}#3`);
438     const nQbar = nodeFor(`${c.id}#4`);
439     const nSet = nodeFor(`${c.id}#5`);
440     const nReset = nodeFor(`${c.id}#6`);
441     const nQi = nodeFor(`${c.id}#_qi`);
442     const nEdge = nodeFor(`${c.id}#_clkedge`);
443     lines.push(`${spiceBranchName("R", c.id)}_setpd ${nSet} 0 1e9`);
444     lines.push(`${spiceBranchName("R", c.id)}_rstpd ${nReset} 0 1e9`);
445     appendClkEdgeDetector(nClk, nEdge, lines, spiceBranchName, c.id, TOGGLE_CLK_EDGE_C_F, TOGGLE_CLK_EDGE_R_OHM);
446     lines.push(
447         `${spiceBranchName("B", c.id)}_qi ${nQi} 0 V = ${logicJkQiExpression(nJ, nK, nEdge, nClk, nQ, vhi, nSet, nReset, opts.rippleClockFrom
Prev === "q")}`
448     );
449     lines.push(`${spiceBranchName("R", c.id)}_q ${nQi} ${nQ} ${TOGGLE_FF_Q_R_OHM}`);
450     lines.push(`${spiceBranchName("C", c.id)}_st ${nQi} 0 ${TOGGLE_FF_STATE_C_F}`);
451     appendQbarDriver(c, nQ, nQbar, nodeFor, vhi, lines, spiceBranchName);
452 }
453
454 export function appendIc74ls00Netlist(c, nodeFor, vhi, lines, spiceBranchName) {
455     for (let gi = 0; gi < ic74ls00NandGates().length; gi++) {
456         const g = ic74ls00NandGates()[gi];
457         const nA = nodeFor(`${c.id}#${g.a}`);
458         const nB = nodeFor(`${c.id}#${g.b}`);
459         const nY = nodeFor(`${c.id}#${g.y}`);
460         const expr = logicNandBsourceExpression(nA, nB, vhi);
461         lines.push(`${spiceBranchName("B", c.id)}_g${gi + 1} ${nY} 0 V = ${expr}`);
462     }
463 }
464
465 function appendLogicDffSlice(c, nodeFor, vhi, lines, spiceBranchName, suffix, dIdx, clkIdx, qIdx) {
466     const nD = nodeFor(`${c.id}#${dIdx}`);
467     const nClk = nodeFor(`${c.id}#${clkIdx}`);
468     const nQ = nodeFor(`${c.id}#${qIdx}`);
469     const nQi = nodeFor(`${c.id}#_qi${suffix}`);
470     const nEdge = nodeFor(`${c.id}#_clkedge${suffix}`);
471     appendClkEdgeDetector(nClk, nEdge, lines, spiceBranchName, `${c.id}${suffix}`, CLK_EDGE_C_F, CLK_EDGE_R_OHM);
472     lines.push(
473         `${spiceBranchName("B", c.id)}_qi${suffix} ${nQi} 0 V = ${logicDffQiExpression(
474             nD,
475             nEdge,

```

```

476         nClk,
477         nQ,
478         vhi,
479         false
480     )}`
481 );
482 lines.push(`${spiceBranchName("R", c.id)}_q${suffix} ${nQi} ${nQ} ${FF_Q_R_OHM}`);
483 lines.push(`${spiceBranchName("C", c.id)}_st${suffix} ${nQi} 0 ${FF_STATE_C_F}`);
484 }
485
486 export function appendIc74ls74Netlist(c, nodeFor, vhi, lines, spiceBranchName) {
487     for (const sl of ic74ls74DffSlices()) {
488         appendLogicDffSlice(c, nodeFor, vhi, lines, spiceBranchName, sl.suffix, sl.d, sl.clk, sl.q);
489     }
490 }
491
492 export function resolveIc74ls74Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhiFn) {
493     const vccKey = `${c.id}#${ic74ls74VccPinIndex()}`;
494     let vhi = logicVhiByTerminal.get(vccKey) ?? 0;
495     for (const sl of ic74ls74DffSlices()) {
496         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${sl.d}`) ?? 0);
497         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${sl.clk}`) ?? 0);
498         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${sl.q}`) ?? 0);
499     }
500     if (c.logicRail !== null && c.logicRail !== "") {
501         vhi = Math.max(vhi, logicVhiFn(parseLogicRail(c.logicRail)));
502     }
503     if (vhi <= 0) vhi = 5;
504     return vhi > 0 ? vhi : 5;
505 }
506
507 export function resolveSequentialVhi(c, logicVhiByTerminal, parseLogicRail, logicVhiFn, inputKeys) {
508     let vhi = 0;
509     for (const k of inputKeys) {
510         vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
511     }
512     if (c.logicRail !== null && c.logicRail !== "") {
513         vhi = Math.max(vhi, logicVhiFn(parseLogicRail(c.logicRail)));
514     }
515     const outKeys =
516         c.type === "logic_dff"
517             ? logicDffOutputNodeKeys(c)
518             : c.type === "logic_jk"
519                 ? logicJkOutputNodeKeys(c)
520                 : [];
521     for (const k of outKeys) {
522         vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
523     }
524     if (vhi <= 0) vhi = 5;
525     return vhi > 0 ? vhi : 5;
526 }
527
528 export function resolveIc74ls00Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhiFn) {
529     const vccKey = `${c.id}#${ic74ls00VccPinIndex()}`;
530     let vhi = logicVhiByTerminal.get(vccKey) ?? 0;
531     for (const g of ic74ls00NandGates()) {
532         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${g.a}`) ?? 0);
533         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${g.b}`) ?? 0);
534         vhi = Math.max(vhi, logicVhiByTerminal.get(`${c.id}#${g.y}`) ?? 0);
535     }
536     if (c.logicRail !== null && c.logicRail !== "") {
537         vhi = Math.max(vhi, logicVhiFn(parseLogicRail(c.logicRail)));
538     }
539     if (vhi <= 0) vhi = 5;
540     return vhi > 0 ? vhi : 5;
541 }
542

```

```

1  /**
2   * Bascules D via XSPICE (d_dff) + ponts adc_bridge / dac_bridge.
3   * Nécessite ngspice compilé avec XSPICE et digital.cm (Simulateur/lib/ngspice/digital.cm).
4   */
5   import { existsSync } from "fs";
6   import { join, dirname } from "path";
7   import { fileURLToPath } from "url";
8
9   const __engineDir = dirname(fileURLToPath(import.meta.url));
10
11  /** Chemins possibles pour digital.cm (relatif à la racine du dépôt). */
12  const DIGITAL_CM_REL_PATHS = [
13    ["Simulateur", "lib", "ngspice", "digital.cm"],
14    ["Simulateur", "share", "ngspice", "lib", "ngspice", "digital.cm"],
15  ];
16
17  /**
18   * @param {string} [repoRoot] racine du projet (dossier contenant Simulateur/)
19   * @returns {string | null} chemin absolu vers digital.cm
20   */
21  /** Chemin relatif (/) sous la racine du dépôt – détection de présence de digital.cm. */
22  export function resolveDigitalCmRelPath(repoRoot) {
23    const root = repoRoot || join(__engineDir, "..", "..");
24    for (const parts of DIGITAL_CM_REL_PATHS) {
25      const p = join(root, ...parts);
26      if (existsSync(p)) return parts.join("/");
27    }
28    return null;
29  }
30
31  /** @deprecated préférer resolveDigitalCmRelPath pour la netlist */
32  export function resolveDigitalCmPath(repoRoot) {
33    const rel = resolveDigitalCmRelPath(repoRoot);
34    if (!rel) return null;
35    const root = repoRoot || join(__engineDir, "..", "..");
36    return join(root, ...rel.split("/"));
37  }
38
39  export function isXspiceDffAvailable(repoRoot) {
40    return resolveDigitalCmRelPath(repoRoot) != null;
41  }
42
43  /**
44   * Seuils adc/dac pour signaux unipolaires 0..Vhi (Gimp, états logiques, carré 0..Vhi).
45   * Compatible aussi avec un carré bipolaire ±Vhi si l'amplitude dépasse les seuils.
46   * @param {number} vhi amplitude positive (ex. 5)
47   */
48  export function xspiceLogicThresholds(vhi) {
49    const v = vhi > 0 ? vhi : 5;
50    return {
51      inLow: 0.2 * v,
52      inHigh: 0.8 * v,
53      outLow: 0,
54      outHigh: v,
55    };
56  }
57
58  /**
59   * Marqueur en tête de netlist : le serveur charge digital.cm via ngspice -f (init.rc),
60   * pas via « codemodel » dans le .cir (non reconnu en batch ngspice-46 Windows).
61   */
62  export const XSPICE_DIGITAL_CM_PLACEHOLDER = "__XSPICE_DIGITAL_CM__";
63
64  /**
65   * @param {string} repoRoot
66   * @returns {string[]}
67   */
68  export function xspiceCodemodelLines(repoRoot) {
69    if (!resolveDigitalCmRelPath(repoRoot)) return [];
70    return [`* XSPICE digital.cm ${XSPICE_DIGITAL_CM_PLACEHOLDER}`];
71  }
72
73  /** @param {string} id préfixe modèle (ex. id du composant) */
74  function dffModelName(id) {
75    return `xdff_${String(id).replace(/^[a-zA-Z0-9_]/g, "_")}`;
76  }
77
78  /**

```

```

79 * Bascule D XSPICE : broches analogiques via ponts ; cœur numérique d_dff.
80 * @param {object} c composant logic_dff
81 * @param {function} nodeFor clé broche → nœud SPICE
82 * @param {number} vhi
83 * @param {string[]} lines
84 * @param {function} spiceBranchName
85 * @param {{srWired?: {set?: boolean, reset?: boolean}}} [opts]
86 */
87 export function appendLogicDffXspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
88   const th = xspiceLogicThresholds(vhi);
89   const srWired = opts.srWired || {};
90   const nD = nodeFor(`${c.id}#0`);
91   const nClk = nodeFor(`${c.id}#1`);
92   const nQ = nodeFor(`${c.id}#2`);
93   const nQbar = nodeFor(`${c.id}#3`);
94   const nSet = nodeFor(`${c.id}#4`);
95   const nReset = nodeFor(`${c.id}#5`);
96
97   const ndData = nodeFor(`${c.id}#__xd_d`);
98   const ndClk = nodeFor(`${c.id}#__xd_clk`);
99   const ndQ = nodeFor(`${c.id}#__xd_q`);
100  const ndQbar = nodeFor(`${c.id}#__xd_qbar`);
101
102  // Seuils mi-rail pour Set/Reset (niveaux logiques 0..Vhi) : actifs à l'état haut (1).
103  const v = vhi > 0 ? vhi : 5;
104  const srLow = 0.4 * v;
105  const srHigh = 0.6 * v;
106
107  const rippleFromQ = opts.rippleClockFromPrev === "q";
108  let nClkAdc = nClk;
109  if (rippleFromQ || opts.clockInvert) {
110    nClkAdc = nodeFor(`${c.id}#__xa_clkinv`);
111    lines.push(
112      `${spiceBranchName("B", c.id)}_clkinv ${nClkAdc} 0 V = { ${v} - V(${nClk}) }`
113    );
114  }
115
116  // Entrées Data / Clock : analogique -> numérique. Data passe toujours par un pont
117  // (même si D est relié à /Q) : la rétroaction traverse alors les délais dac+adc,
118  // ce qui casse la boucle algébrique numérique (sinon la sortie q reste UNKNOWN).
119  lines.push(`${spiceBranchName("A", c.id)}_adcd [${nD}] [${ndData}] ${c.id}_m_adcio`);
120  lines.push(`${spiceBranchName("A", c.id)}_adcc [${nClkAdc}] [${ndClk}] ${c.id}_m_adcio`);
121  lines.push(
122    `.model ${c.id}_m_adcio adc_bridge(in_low=${th.inLow} in_high=${th.inHigh})`
123  );
124
125  // Set / Reset asynchrones. Si une broche n'est PAS câblée, on ne la tire pas par une
126  // simple résistance (un nœud haute impédance reste « UNKNOWN » côté adc_bridge et
127  // contamine la sortie Q). On la relie à un nœud numérique ZÉRO franc, issu d'une
128  // source 0 V dédiée -> adc : état logique bas garanti dès t=0.
129  lines.push(`.model ${c.id}_m_adcsr adc_bridge(in_low=${srLow} in_high=${srHigh})`);
130  let ndZero = null;
131  const zeroNode = () => {
132    if (ndZero) return ndZero;
133    const nZa = nodeFor(`${c.id}#__xa_zero`);
134    ndZero = nodeFor(`${c.id}#__xd_zero`);
135    lines.push(`${spiceBranchName("V", c.id)}_xz ${nZa} 0 DC 0`);
136    lines.push(`${spiceBranchName("A", c.id)}_adcz [${nZa}] [${ndZero}] ${c.id}_m_adcsr`);
137    return ndZero;
138  };
139
140  let ndSet;
141  if (srWired.set) {
142    ndSet = nodeFor(`${c.id}#__xd_set`);
143    lines.push(`${spiceBranchName("A", c.id)}_adcs [${nSet}] [${ndSet}] ${c.id}_m_adcsr`);
144  } else {
145    ndSet = zeroNode();
146  }
147
148  let ndRst;
149  if (srWired.reset) {
150    ndRst = nodeFor(`${c.id}#__xd_rst`);
151    lines.push(`${spiceBranchName("A", c.id)}_adcr [${nReset}] [${ndRst}] ${c.id}_m_adcsr`);
152  } else {
153    ndRst = zeroNode();
154  }
155
156  const mname = dffModelName(c.id);
157  // Reset asynchrone ripple mod-10 : maintien plus long pour éviter 9→4 (course avec l'horloge ripple).
158  const srDelay = opts.rippleMod10 ? "100e-9" : "2e-9";

```

```

159     lines.push(
160         `.model ${mname} d_dff(clk_delay=2e-9 set_delay=${srDelay} reset_delay=${srDelay} ic=0 rise_delay=2e-9 fall_delay=2e-9)`
161     );
162
163     // d_dff : ports scalaires (data clk set reset out Nout), sans crochets [ ].
164     lines.push(
165         `${spiceBranchName("A", c.id)}_dff ${ndData} ${ndClk} ${ndSet} ${ndRst} ${ndQ} ${ndQbar} ${mname}`
166     );
167
168     // Sorties numériques -> analogique.
169     lines.push(`${spiceBranchName("A", c.id)}_dacq [ ${ndQ} ] [ ${nQ} ] ${c.id}_m_dac`);
170     lines.push(`${spiceBranchName("A", c.id)}_dacqb [ ${ndQbar} ] [ ${nQbar} ] ${c.id}_m_dac`);
171     lines.push(
172         `.model ${c.id}_m_dac dac_bridge(out_low=${th.outLow} out_high=${th.outHigh} t_rise=2e-9 t_fall=2e-9)`
173     );
174 }
175
176 export function logicDffXspiceInternalNodeKeys(c, opts = {}) {
177     if (!c?.id) return [];
178     const keys = [
179         `${c.id}#__xd_d`,
180         `${c.id}#__xd_clk`,
181         `${c.id}#__xd_set`,
182         `${c.id}#__xd_rst`,
183         `${c.id}#__xd_q`,
184         `${c.id}#__xd_qbar`,
185         `${c.id}#__xa_zero`,
186         `${c.id}#__xd_zero`,
187     ];
188     if (opts.rippleClockFromPrev === "q" || opts.clockInvert) keys.push(`${c.id}#__xa_clkinv`);
189     return keys;
190 }
191
192 function jkffModelName(id) {
193     return `xjkff_${String(id).replace(/^[a-zA-Z0-9_]/g, "_")}`;
194 }
195
196 /**
197  * Bascule JK XSPICE : broches analogiques via ponts ; cœur numérique d_jkff.
198  * Ports : j k clk set reset out Nout
199  */
200 export function appendLogicJkXspiceNetlist(c, nodeFor, vhi, lines, spiceBranchName, opts = {}) {
201     const th = xspiceLogicThresholds(vhi);
202     const srWired = opts.srWired || {};
203     const nJ = nodeFor(`${c.id}#0`);
204     const nK = nodeFor(`${c.id}#1`);
205     const nClk = nodeFor(`${c.id}#2`);
206     const nQ = nodeFor(`${c.id}#3`);
207     const nQbar = nodeFor(`${c.id}#4`);
208     const nSet = nodeFor(`${c.id}#5`);
209     const nReset = nodeFor(`${c.id}#6`);
210
211     const ndJ = nodeFor(`${c.id}#__xd_j`);
212     const ndK = nodeFor(`${c.id}#__xd_k`);
213     const ndClk = nodeFor(`${c.id}#__xd_clk`);
214     const ndQ = nodeFor(`${c.id}#__xd_q`);
215     const ndQbar = nodeFor(`${c.id}#__xd_qbar`);
216
217     const v = vhi > 0 ? vhi : 5;
218     const srLow = 0.4 * v;
219     const srHigh = 0.6 * v;
220
221     const rippleFromQ = opts.rippleClockFromPrev === "q";
222     let nClkAdc = nClk;
223     if (rippleFromQ) {
224         nClkAdc = nodeFor(`${c.id}#__xa_clkinv`);
225         lines.push(
226             `${spiceBranchName("B", c.id)}_clkinv ${nClkAdc} 0 V = { ${v} - V(${nClk}) }`
227         );
228     }
229
230     lines.push(`${spiceBranchName("A", c.id)}_adcj [ ${nJ} ] [ ${ndJ} ] ${c.id}_m_adcio`);
231     lines.push(`${spiceBranchName("A", c.id)}_adck [ ${nK} ] [ ${ndK} ] ${c.id}_m_adcio`);
232     lines.push(`${spiceBranchName("A", c.id)}_adcc [ ${nClkAdc} ] [ ${ndClk} ] ${c.id}_m_adcio`);
233     lines.push(
234         `.model ${c.id}_m_adcio adc_bridge(in_low=${th.inLow} in_high=${th.inHigh})`
235     );
236
237     lines.push(`${c.id}_m_adcsr adc_bridge(in_low=${srLow} in_high=${srHigh})`);
238     let ndZero = null;

```

```

239     const zeroNode = () => {
240         if (ndZero) return ndZero;
241         const nZa = nodeFor(`${c.id}#__xa_zero`);
242         ndZero = nodeFor(`${c.id}#__xd_zero`);
243         lines.push(`${spiceBranchName("V", c.id)}_xz ${nZa} 0 DC 0`);
244         lines.push(`${spiceBranchName("A", c.id)}_adcz [${nZa}] [${ndZero}] ${c.id}_m_adcsr`);
245         return ndZero;
246     };
247
248     let ndSet;
249     if (srWired.set) {
250         ndSet = nodeFor(`${c.id}#__xd_set`);
251         lines.push(`${spiceBranchName("A", c.id)}_adcs [${nSet}] [${ndSet}] ${c.id}_m_adcsr`);
252     } else {
253         ndSet = zeroNode();
254     }
255
256     let ndRst;
257     if (srWired.reset) {
258         ndRst = nodeFor(`${c.id}#__xd_rst`);
259         lines.push(`${spiceBranchName("A", c.id)}_adcr [${nReset}] [${ndRst}] ${c.id}_m_adcsr`);
260     } else {
261         ndRst = zeroNode();
262     }
263
264     const mname = jkffModelName(c.id);
265     lines.push(
266         `.model ${mname} d_jkff(clk_delay=2e-9 set_delay=2e-9 reset_delay=2e-9 ic=0 rise_delay=2e-9 fall_delay=2e-9)`
267     );
268     lines.push(
269         `${spiceBranchName("A", c.id)}_jk ${ndJ} ${ndK} ${ndClk} ${ndSet} ${ndRst} ${ndQ} ${ndQbar} ${mname}`
270     );
271
272     lines.push(`${spiceBranchName("A", c.id)}_dacq [${ndQ}] [${nQ}] ${c.id}_m_dac`);
273     lines.push(`${spiceBranchName("A", c.id)}_dacqb [${ndQbar}] [${nQbar}] ${c.id}_m_dac`);
274     lines.push(
275         `.model ${c.id}_m_dac dac_bridge(out_low=${th.outLow} out_high=${th.outHigh} t_rise=2e-9 t_fall=2e-9)`
276     );
277 }
278
279 export function logicJkXspiceInternalNodeKeys(c, opts = {}) {
280     if (!c?.id) return [];
281     const keys = [
282         `${c.id}#__xd_j`,
283         `${c.id}#__xd_k`,
284         `${c.id}#__xd_clk`,
285         `${c.id}#__xd_set`,
286         `${c.id}#__xd_rst`,
287         `${c.id}#__xd_q`,
288         `${c.id}#__xd_qbar`,
289         `${c.id}#__xa_zero`,
290         `${c.id}#__xd_zero`,
291     ];
292     if (opts.rippleClockFromPrev === "q") keys.push(`${c.id}#__xa_clkinv`);
293     return keys;
294 }
295

```

```

1  /**
2   * Détecte si le binaire ngspice supporte XSPICE (codemodel + devices A).
3   * ngspice -v n'affiche pas toujours "XSPICE" même si devhelp d_dff fonctionne.
4   */
5  import { existsSync, mkdtempSync, writeFileSync, readFileSync, rmSync } from "fs";
6  import { execFileSync } from "child_process";
7  import { tmpdir } from "os";
8  import { join } from "path";
9
10 let cachedByExe = new Map();
11
12 const D_DFF_HELP = /d_dff\s*-\s*digital/i;
13
14 /**
15  * @param {string} exe
16  * @param {NodeJS.ProcessEnv} [env]
17  * @returns {boolean}
18  */
19 function probeXspiceViaDevhelp(exe, env) {
20   const dir = mkdtempSync(join(tmpdir(), "ngspice-xspice-probe-"));
21   const cir = join(dir, "probe.cir");
22   const log = join(dir, "out.log");
23   writeFileSync(
24     cir,
25     `* xspice probe\n.control\ndevhelp d_dff\n.quit\n.endc\n`,
26     "utf8"
27   );
28   let combined = "";
29   try {
30     try {
31       combined = execFileSync(exe, ["-b", "-o", log, cir], {
32         encoding: "utf8",
33         timeout: 20000,
34         windowsHide: true,
35         env: env || process.env,
36         maxBuffer: 4 * 1024 * 1024,
37       });
38     } catch (err) {
39       /* devhelp s'affiche souvent sur stdout/stderr, pas dans le .log */
40       combined =
41         (err && typeof err.stdout === "string" ? err.stdout : "") +
42         (err && typeof err.stderr === "string" ? err.stderr : "");
43     }
44     if (D_DFF_HELP.test(combined)) return true;
45     if (existsSync(log)) {
46       const text = readFileSync(log, "utf8");
47       if (D_DFF_HELP.test(text)) return true;
48     }
49   } finally {
50     try {
51       rmSync(dir, { recursive: true, force: true });
52     } catch {
53       /* ignore */
54     }
55   }
56   return false;
57 }
58
59 /**
60  * @param {string | null | undefined} exe chemin vers ngspice / ngspice_con
61  * @param {NodeJS.ProcessEnv} [env] PATH étendu (bin/lib) si bundle portable
62  * @returns {boolean}
63  */
64 function cacheKeyForExe(exe, env) {
65   const pathEnv = env && (env.PATH || env.Path) ? String(env.PATH || env.Path) : "";
66   return `${String(exe).toLowerCase()}${pathEnv}`;
67 }
68
69 export function ngspiceHasXspice(exe, env) {
70   if (!exe || !existsSync(exe)) return false;
71   const key = cacheKeyForExe(exe, env);
72   if (cachedByExe.has(key)) return cachedByExe.get(key);
73
74   let ok = false;
75   try {
76     const out = execFileSync(exe, ["-v"], {
77       encoding: "utf8",
78       timeout: 8000,

```



```
79         windowsHide: true,
80         env: env || process.env,
81     });
82     ok = /\bxspice\b/i.test(out);
83 } catch {
84     ok = false;
85 }
86
87 if (!ok) ok = probeXspiceViaDevhelp(exe, env);
88
89 cachedByExe.set(key, ok);
90 return ok;
91 }
92
```

```

1  /**
2   * Compteur ripple mod-10 (4x DFF, AND Q2+Q4 → reset) – détection UI + animation idéale.
3   */
4  import { reachableJunctions } from "../hc90-cascade.mjs";
5
6  const CD4511_BCD_PINS = ["A", "B", "C", "D"];
7  const SEG7_SEG_PINS = ["a", "b", "c", "d", "e", "f", "g"];
8
9  function dffLabelIndex(label) {
10     const m = /(\d+)\$/.exec(String(label || ""));
11     return m ? parseInt(m[1], 10) : null;
12 }
13
14 /**
15  * Détecte un compteur ripple décade correctement câblé (AND sur DFF2.Q et DFF4.Q → RESET).
16  * @returns {{ mode: "mod10" } | null}
17  */
18 export function detectRippleMod10ForAnim(components, wires, autoJunctions = []) {
19     const dffs = components.filter((c) => c.type === "d_flipflop" && c.label);
20     const ands = components.filter((c) => c.type === "and" && c.label);
21     if (dffs.length < 4 || ands.length === 0) return null;
22
23     for (const and of ands) {
24         const outNet = reachableJunctions(`\${and.label}_out`, wires, autoJunctions);
25         if (dffs.filter((d) => outNet.has(`\${d.label}_RESET`)).length < 4) continue;
26
27         const inANet = reachableJunctions(`\${and.label}_inA`, wires, autoJunctions);
28         const inBNet = reachableJunctions(`\${and.label}_inB`, wires, autoJunctions);
29         const qIdx = [];
30         for (const d of dffs) {
31             const qPin = `\${d.label}_Q`;
32             if (inANet.has(qPin) || inBNet.has(qPin)) {
33                 const idx = dffLabelIndex(d.label);
34                 if (idx != null && !qIdx.includes(idx)) qIdx.push(idx);
35             }
36         }
37         if (qIdx.length !== 2) continue;
38         qIdx.sort((a, b) => a - b);
39         if (qIdx[0] === 2 && qIdx[1] === 4) return { mode: "mod10" };
40     }
41     return null;
42 }
43
44 /** Afficheur 7 seg alimenté par CD4511 lui-même relié aux Q des bascules ripple mod-10. */
45 export function rippleMod10Seg7Linked(segLabel, components, wires, autoJunctions = []) {
46     if (!segLabel || !detectRippleMod10ForAnim(components, wires, autoJunctions)) return false;
47
48     for (const cd of components) {
49         if (cd.type !== "cd4511" || !cd.label) continue;
50         let segLinked = false;
51         for (const s of SEG7_SEG_PINS) {
52             if (reachableJunctions(`\${segLabel}_\${s}`, wires, autoJunctions).has(`\${cd.label}_\${s}`)) {
53                 segLinked = true;
54                 break;
55             }
56         }
57         if (!segLinked) continue;
58
59         const dffs = components.filter((c) => c.type === "d_flipflop" && c.label);
60         let matched = 0;
61         for (const d of dffs) {
62             const idx = dffLabelIndex(d.label);
63             if (idx == null || idx < 1 || idx > 4) continue;
64             const bcdPin = `\${cd.label}_\${CD4511_BCD_PINS[idx - 1]}`;
65             if (reachableJunctions(`\${d.label}_Q`, wires, autoJunctions).has(bcdPin)) matched++;
66         }
67         if (matched >= 4) return true;
68     }
69     return false;
70 }
71
72 /** Chiffre 0..9 : 1 impulsion GImp = +1 (modulo 10). */
73 export function idealRippleMod10Bcd(elapsedSec, clockPeriodSec) {
74     if (!clockPeriodSec > 0 || !elapsedSec >= 0) return 0;
75     return Math.floor(elapsedSec / clockPeriodSec) % 10;
76 }
77
78 export function shouldUseIdealRippleMod10Seg7(segLabel, components, wires, autoJunctions, clockPeriodSec) {

```

```
79     return (  
80         clockPeriodSec > 0 &&  
81         rippleMod10Seg7Linked(segLabel, components, wires, autoJunctions)  
82     );  
83 }  
84
```

```

1  /**
2   * Construit une netlist ngspice (.op) à partir du JSON de l'éditeur graphique
3   * (résistances, pile DC, générateurs, voltmètres, fils avec clés __t / __p).
4   */
5
6  import { dirname, join } from "path";
7  import { fileURLToPath } from "url";
8
9  const __schematicDir = dirname(fileURLToPath(import.meta.url));
10 const DEFAULT_REPO_ROOT = join(__schematicDir, "..", "..");
11
12 import {
13   logicVhi,
14   logicVth,
15   parseLogicRail,
16   parseLogicStateVolts,
17 } from "../logic-rails.mjs";
18 import {
19   appendIc74ls00Netlist,
20   appendIc74ls74Netlist,
21   appendLogicCd4511Netlist,
22   appendLogicDffNetlist,
23   appendLogicJkNetlist,
24   cd4511InputNodeKeys,
25   cd4511OutputNodeKeys,
26   getRippleClockFromPrev,
27   logicSequentialInternalNodeKeys,
28   ic74ls00NandGates,
29   ic74ls00VccPinIndex,
30   ic74ls74DffSlices,
31   ic74ls74VccPinIndex,
32   isIc74hc90Type,
33   isIc74ls00Type,
34   isIc74ls74Type,
35   isLogicCd4511Type,
36   isLogicDigitalSimType,
37   isLogicIcType,
38   isLogicSequentialType,
39   logicDffInputNodeKeys,
40   logicDffSetResetNodeKeys,
41   logicDffOutputNodeKey,
42   logicDffOutputNodeKeys,
43   logicDffQbarOutputNodeKey,
44   logicDffDAndQbarShareNode,
45   logicJkInputNodeKeys,
46   logicJkSetResetNodeKeys,
47   logicJkOutputNodeKey,
48   logicJkOutputNodeKeys,
49   logicJkQbarOutputNodeKey,
50   resolveIc74ls00Vhi,
51   resolveIc74ls74Vhi,
52   resolveLogicCd4511Vhi,
53   resolveSequentialVhi,
54   useLogicCd4511Xspice,
55   useLogicDffXspice,
56   useLogicJkXspice,
57   xspiceCodemodelLines,
58   isXspiceDffAvailable,
59 } from "../logic-sequential.mjs";
60 import {
61   appendIc74hc90Netlist,
62   IC90_PIN,
63   ic74hc90GndPinIndex,
64   ic74hc90InternalNodeKeys,
65   ic74hc90OutputNodeKeys,
66   ic74hc90ToggleSlices,
67   ic74hc90VccPinIndex,
68   resolveIc74hc90Vhi,
69 } from "../logic-74hc90.mjs";
70 import {
71   appendArduinoUnoNetlist,
72   arduinoUnoDigitalPinIndices,
73   arduinoUnoDigitalPinName,
74   arduinoUnoTerminalKeys,
75   isArduinoUnoType,
76 } from "../arduino-uno.mjs";
77 import {
78   arduinoUnoMinPulsePeriodSec,

```

```

79     applyArduinoSketchToComponent,
80     arduinoGpioIsTimeVarying,
81 } from "./arduino-sketch-parse.mjs";
82 import { annotateUnoI2cBusEngine, i2cBusMinPeriodSec } from "./i2c-bus-ideal.mjs";
83 import {
84     detectHc90Mod60FromGraphicalState,
85     detectHc90MrAndQ1Q3OnSameChip,
86 } from "./hc90-cascade.mjs";
87
88 function isLedType(t) {
89     return t === "led" || t === "diode_led";
90 }
91
92 /** Nœud SPICE relié à une masse ou une source « 0 » (borne logique, VCC à 0 V, etc.). */
93 function isNodeLikelyLogicLow(nodeName, components, nodeFor) {
94     if (!nodeName || nodeName === "0") return true;
95     for (const g of components) {
96         if (g.type === "ground" && nodeFor(`${g.id}#0`) === nodeName) return true;
97     }
98     for (const g of components) {
99         if ((g.type === "vsin" || g.type === "vsquare") && nodeFor(`${g.id}#0`) === nodeName) {
100             return true;
101         }
102     }
103     for (const g of components) {
104         const key = `${g.id}#0`;
105         if (nodeFor(key) !== nodeName) continue;
106         if (g.type === "logic_state" && parseLogicStateVolts(g.value, g.logicRail) < 0.5) return true;
107         if (g.type === "logic_terminal" && Number(g.state) === 0) return true;
108         if (g.type === "vterm" && parseDcVolts(g.value) < 0.5) return true;
109     }
110     return false;
111 }
112
113 /** Nœud relié à +5 V (VCC, borne à 1, etc.). */
114 function isNodeLikelyLogicHigh(nodeName, components, nodeFor) {
115     if (!nodeName) return false;
116     for (const g of components) {
117         const key = `${g.id}#0`;
118         if (nodeFor(key) !== nodeName) continue;
119         if (g.type === "logic_state" && parseLogicStateVolts(g.value, g.logicRail) > 2.5) return true;
120         if (g.type === "logic_terminal" && Number(g.state) === 1) return true;
121         if (g.type === "vterm" && parseDcVolts(g.value) > 2.5) return true;
122     }
123     return false;
124 }
125
126 function isTwoTerminalType(t) {
127     return (
128         t === "resistor" ||
129         t === "capacitor" ||
130         t === "inductor" ||
131         t === "diode" ||
132         isLedType(t) ||
133         t === "npn" ||
134         t === "vsource" ||
135         t === "voltmeter" ||
136         t === "ammeter" ||
137         t === "voltmeter_rms" ||
138         t === "ammeter_rms" ||
139         t === "ohmmeter" ||
140         t === "bode_analyzer" ||
141         t === "speaker" ||
142         t === "push_button"
143     );
144 }
145
146 function isBodeAnalyzerType(t) {
147     return t === "bode_analyzer";
148 }
149
150 function isSpeakerType(t) {
151     return t === "speaker";
152 }
153
154 function isVoltmeterRmsType(t) {
155     return t === "voltmeter_rms";
156 }
157
158 function isAmmeterRmsType(t) {

```

```

159     return t === "ammeter_rms";
160 }
161
162 function isSignalGeneratorType(t) {
163     return t === "vsin" || t === "vsquare" || t === "vpulse";
164 }
165
166 function isOscilloscopeType(t) {
167     return t === "oscilloscope";
168 }
169
170 function isGroundType(t) {
171     return t === "ground";
172 }
173
174 function isVtermType(t) {
175     return t === "vterm";
176 }
177
178 function isLogicStateType(t) {
179     return t === "logic_state";
180 }
181
182 function isSingleTerminalRefType(t) {
183     return isGroundType(t) || isVtermType(t) || isLogicStateType(t);
184 }
185
186 function isOpampType(t) {
187     return t === "opamp";
188 }
189
190 function isThreeTerminalType(t) {
191     return t === "npn" || t === "opamp" || t === "potentiometer" || t === "switch_spdt";
192 }
193
194 function isSeg7Type(t) {
195     return t === "seg7";
196 }
197
198 function isBargraphDc10hType(t) {
199     return t === "bargraph_dc10h";
200 }
201
202 function isGroveLcdType(t) {
203     return t === "grove_lcd16x2";
204 }
205
206 function isGroveDht22Type(t) {
207     return t === "grove_dht22";
208 }
209
210 function seg7TerminalKeys(c) {
211     const keys = [];
212     for (let i = 0; i < 8; i++) keys.push(`${c.id}#${i}`);
213     return keys;
214 }
215
216 function bargraphTerminalKeys(c) {
217     const keys = [];
218     for (let i = 0; i < 11; i++) keys.push(`${c.id}#${i}`);
219     return keys;
220 }
221
222 /** Bornes SPICE à enregistrer pour le câblage (union-find). */
223 function terminalKeysForComponent(c) {
224     if (!c || !c.id) return [];
225     if (isSeg7Type(c.type)) return seg7TerminalKeys(c);
226     if (isBargraphDc10hType(c.type)) return bargraphTerminalKeys(c);
227     if (isGroveLcdType(c.type)) return [`${c.id}#0`, `${c.id}#1`, `${c.id}#2`, `${c.id}#3`];
228     if (isGroveDht22Type(c.type)) return [`${c.id}#0`, `${c.id}#1`, `${c.id}#2`, `${c.id}#3`];
229     if (isOscilloscopeType(c.type)) return [`${c.id}#0`, `${c.id}#1`, `${c.id}#2`];
230     if (isThreeTerminalType(c.type)) return [`${c.id}#0`, `${c.id}#1`, `${c.id}#2`];
231     if (isLogicGateComponentType(c.type)) {
232         return [...logicGateInputNodeKeys(c), logicGateOutputNodeKey(c)];
233     }
234     if (c.type === "logic_dff") {
235         return [
236             ...logicDffInputNodeKeys(c),
237             ...logicDffSetResetNodeKeys(c),
238             ...logicDffOutputNodeKeys(c),

```

```

239     ];
240   }
241   if (c.type === "logic_jk") {
242     return [
243       ...logicJkInputNodeKeys(c),
244       ...logicJkSetResetNodeKeys(c),
245       ...logicJkOutputNodeKeys(c),
246     ];
247   }
248   if (isLogicIcType(c.type)) {
249     const keys = [];
250     for (let i = 0; i < 14; i++) keys.push(`${c.id}#${i}`);
251     return keys;
252   }
253   if (isArduinoUnoType(c.type)) return arduinoUnoTerminalKeys(c);
254   if (isSingleTerminalRefType(c.type)) return [`${c.id}#0`];
255   if (isTwoTerminalType(c.type) || isSignalGeneratorType(c.type)) {
256     return [`${c.id}#0`, `${c.id}#1`];
257   }
258   return [];
259 }
260
261 /** Fréquence de résonance d'un circuit LC parallèle (bobine + condensateur). */
262 function estimateParallelLcResonantHz(components) {
263   let cFarad = 0;
264   let lHenry = 0;
265   for (const c of components) {
266     if (c.type === "capacitor") cFarad = Math.max(cFarad, parseCapacitanceFarad(c.value));
267     if (c.type === "inductor") lHenry = Math.max(lHenry, parseInductanceHenry(c.value));
268   }
269   if (cFarad <= 0 || lHenry <= 0) return 0;
270   return 1 / (2 * Math.PI * Math.sqrt(lHenry * cFarad));
271 }
272
273 /** AOP + LC + oscilloscope, sans générateur externe : oscillateur autonome. */
274 function circuitHasAutonomousLcOscillator(components) {
275   return (
276     components.some((c) => c.type === "oscilloscope") &&
277     components.some((c) => isOpampType(c.type)) &&
278     components.some((c) => c.type === "inductor") &&
279     components.some((c) => c.type === "capacitor")
280   );
281 }
282
283 /** Comparateur / Schmitt (boucle + ou boucle ouverte) : bascule dure. Amplificateur (boucle -) : tanh raide. */
284 function opampUsesComparatorModel(c, parent, components) {
285   const topo = new Map(parent);
286   ufUnionPassiveInternals(topo, components);
287   const outKey = `${c.id}#2`;
288   const posKey = `${c.id}#0`;
289   const negKey = `${c.id}#1`;
290   const outRoot = ufFind(topo, outKey);
291   const posRoot = ufFind(topo, posKey);
292   const negRoot = ufFind(topo, negKey);
293   const positiveFeedback = outRoot === posRoot;
294   const negativeFeedback = outRoot === negRoot;
295   if (
296     positiveFeedback &&
297     negativeFeedback &&
298     components.some((x) => x.type === "inductor") &&
299     components.some((x) => x.type === "capacitor")
300   ) {
301     return false;
302   }
303   return positiveFeedback || !negativeFeedback;
304 }
305
306 /** Source comportementale AOP : saturation aux rails (comparateur, hystérésis, amplif.). */
307 function formatOpampBsourceLine(c, nOut, nPlus, nMinus, { comparatorMode = false, gain = 1e5 } = {}) {
308   const vp = parseOpampVp(c);
309   const vn = parseOpampVn(c);
310   const bname = spiceBranchName("BAOP", c.id);
311   const diff = spiceVoltageDiffExpr(nPlus, nMinus);
312   if (comparatorMode) {
313     return `${bname} ${nOut} 0 V=${vn}+(${vp}-${vn})*u(${diff})`;
314   }
315   const vmid = (vp + vn) / 2;
316   const vhalf = (vp - vn) / 2;
317   return `${bname} ${nOut} 0 V=${vmid}+${vhalf}*tanh(${gain}*(${diff}))`;
318 }

```

```

319
320 function isTerminalComponentType(t) {
321     return (
322         isTwoTerminalType(t) ||
323         isOpampType(t) ||
324         isSignalGeneratorType(t) ||
325         isOscilloscopeType(t) ||
326         isSingleTerminalRefType(t)
327     );
328 }
329
330 function isPowerSourceType(t) {
331     return t === "vsources" || t === "vsin" || t === "vsquare" || t === "vpulse" || t === "vterm" || t === "logic_state";
332 }
333
334 /** Courant de test pour l'ohmmètre (mesure  $R = \Delta V / I$ ). */
335 const OHMMETER_TEST_CURRENT_A = 0.001;
336
337 /** Met à 0 les sources indépendantes pour mesure  $\Omega$  (comme un multimètre). */
338 function zeroIndependentSourceLine(line) {
339     const t = String(line || "").trim();
340     if (/^IOHM/i.test(t)) return line;
341     if (/^VI/i.test(t)) return line;
342     if (/^V[^\s]+\s+\s+DC\s+/i.test(t)) {
343         return line.replace(/(\sDC\s+)[^\s]+/i, "$10");
344     }
345     if (/^V[^\s]+\s+\s+(SIN|PULSE)\(/i.test(t)) {
346         return line.replace(/(\s+(SIN|PULSE)\([^\)]*\)/i, " DC 0");
347     }
348     return line;
349 }
350
351 /**
352  * Netlist .op avec sources coupées – mesure de résistance entre les bornes de l'ohmmètre.
353  * @param {string} fullNetlist
354  * @param {{ id: string; nodePlus: string; nodeMinus: string }[]} ohmmeters
355  */
356 export function buildOhmmeterIsolationNetlist(fullNetlist, ohmmeters) {
357     if (!fullNetlist || !Array.isArray(ohmmeters) || ohmmeters.length === 0) return "";
358     const out = [];
359     let skipControl = false;
360     for (const raw of fullNetlist.split(/\r?\n/)) {
361         const t = raw.trim();
362         if (/^\.(tran|op)\b/i.test(t)) continue;
363         if (t === ".control") {
364             skipControl = true;
365             continue;
366         }
367         if (t === ".endc") {
368             skipControl = false;
369             continue;
370         }
371         if (skipControl) continue;
372         if (t.startsWith("wrdata")) continue;
373         if (t === ".end") continue;
374         if (/^\.options\b/i.test(t)) continue;
375         out.push(zeroIndependentSourceLine(raw));
376     }
377     out.push(".op");
378     out.push(".control");
379     out.push("op");
380     for (const om of ohmmeters) {
381         out.push(`echo @@OH:${om.id}@@`);
382         if (om.nodePlus !== "0") out.push(`print v(${om.nodePlus})`);
383         if (om.nodeMinus !== "0") out.push(`print v(${om.nodeMinus})`);
384     }
385     out.push("quit");
386     out.push(".endc");
387     out.push(".end");
388     return out.join("\n");
389 }
390
391 function uFind(parent, x) {
392     if (!parent.has(x)) parent.set(x, x);
393     const p = parent.get(x);
394     if (p !== x) {
395         const r = uFind(parent, p);
396         parent.set(x, r);
397         return r;
398     }

```



```

399     return x;
400 }
401
402 function ufUnion(parent, a, b) {
403     const ra = ufFind(parent, a);
404     const rb = ufFind(parent, b);
405     if (ra !== rb) parent.set(ra, rb);
406 }
407
408 /**
409  * Détecte un AND de reset mod-10 sur deux sorties Q de bascules D (indices DFF1...DFF4).
410  * @returns {{ a: number, b: number } | null}
411  */
412 function getRippleMod10ResetAndIndices(components, parent) {
413     const dffs = components.filter((c) => c.type === "logic_dff");
414     const ands = components.filter((c) => c.type === "logic_and" || c.type === "and");
415     if (dffs.length < 4 || ands.length === 0) return null;
416
417     const dffIndex = (c) => {
418         const m = /(\d+)\$/.exec(String(c.id || ""));
419         return m ? parseInt(m[1], 10) : null;
420     };
421     const sameNet = (a, b) => ufFind(parent, a) === ufFind(parent, b);
422
423     for (const and of ands) {
424         const outKey = `${and.id}#2`;
425         if (!parent.has(outKey)) continue;
426         if (!dffs.some((d) => sameNet(outKey, `${d.id}#5`))) continue;
427
428         const inputIdx = [];
429         for (const inKey of [`${and.id}#0`, `${and.id}#1`]) {
430             if (!parent.has(inKey)) continue;
431             for (const d of dffs) {
432                 const qKey = `${d.id}#2`;
433                 if (parent.has(qKey) && sameNet(inKey, qKey)) {
434                     const idx = dffIndex(d);
435                     if (idx !== null && !inputIdx.includes(idx)) inputIdx.push(idx);
436                 }
437             }
438         }
439         if (inputIdx.length !== 2) continue;
440         inputIdx.sort((a, b) => a - b);
441         return { a: inputIdx[0], b: inputIdx[1] };
442     }
443     return null;
444 }
445
446 /**
447  * AND de reset mal câblé sur compteur ripple 4 bits (symptôme typique : 4...9 au lieu de 0...9).
448  * @returns {string|null}
449  */
450 function detectRippleMod10ResetAndWarning(components, parent) {
451     const pair = getRippleMod10ResetAndIndices(components, parent);
452     if (!pair) return null;
453     const { a, b } = pair;
454     if (a === 2 && b === 4) return null;
455     if (a === 1 && b === 4) {
456         return (
457             "Compteur ripple mod-10 : AND sur la Q de DFF1 et DFF4 – reset au 9 (1001) au lieu du 10 (1010), " +
458             "comptage souvent bloqué entre 4 et 8. Corrigez : AND sur DFF2.Q et DFF4.Q (bits de poids 2 et 8). "
459         );
460     }
461     if (a === 3 && b === 4) {
462         return (
463             "Compteur ripple mod-10 : AND sur DFF3.Q et DFF4.Q – reset au 12 (1100), pas une décade 0...9. " +
464             "Corrigez : AND sur DFF2.Q et DFF4.Q pour détecter 1010. "
465         );
466     }
467     return (
468         `Compteur ripple mod-10 : AND de reset sur DFF${a} et DFF${b} – pour le 10 décimal (1010), ` +
469         "reliez AND à DFF2.Q et DFF4.Q seulement."
470     );
471 }
472
473 /** Résistances / passifs : même nœud électrique aux deux bornes (pour détecter la rétroaction AOP). */
474 function ufUnionPassiveInternals(parent, components) {
475     for (const c of components) {
476         if (c.type === "resistor" || c.type === "capacitor" || c.type === "inductor") {
477             ufUnion(parent, `${c.id}#0`, `${c.id}#1`);
478         }
479     }
480 }

```

```

479     }
480 }
481
482 function spiceVoltageExpr(nodeName) {
483     const n = String(nodeName || "").trim();
484     if (n === "0") return "0";
485     return `V(${n})`;
486 }
487
488 function spiceVoltageDiffExpr(nPlus, nMinus) {
489     const p = String(nPlus || "").trim();
490     const m = String(nMinus || "").trim();
491     if (m === "0") return spiceVoltageExpr(p);
492     if (p === "0") return `-V(${m})`;
493     return `V(${p})-V(${m})`;
494 }
495
496 function parseResistanceOhm(s) {
497     if (s == null) return 1000;
498     let t = String(s).trim().toLowerCase().replace(/\s/g, "");
499     if (!t) return 1000;
500     let mult = 1;
501     if (t.endsWith("meg")) {
502         mult = 1e6;
503         t = t.slice(0, -3);
504     } else if (t.endsWith("k")) {
505         mult = 1e3;
506         t = t.slice(0, -1);
507     } else if (t.endsWith("m") && /^d/.test(t.slice(0, -1))) {
508         mult = 1e-3;
509         t = t.slice(0, -1);
510     }
511     const n = parseFloat(t.replace(",", "."));
512     return Number.isFinite(n) && n > 0 ? n * mult : 1000;
513 }
514
515 function parseCapacitanceFarad(s) {
516     if (s == null) return 1e-6;
517     let t = String(s).trim().toLowerCase().replace(/\s/g, "").replace(",", ".").replace("μ", "u");
518     if (!t) return 1e-6;
519     let mult = 1;
520     if (t.endsWith("uf")) {
521         mult = 1e-6;
522         t = t.slice(0, -2);
523     } else if (t.endsWith("u")) {
524         mult = 1e-6;
525         t = t.slice(0, -1);
526     } else if (t.endsWith("pf")) {
527         mult = 1e-12;
528         t = t.slice(0, -2);
529     } else if (t.endsWith("nf")) {
530         mult = 1e-9;
531         t = t.slice(0, -2);
532     } else if (t.endsWith("mf")) {
533         mult = 1e-3;
534         t = t.slice(0, -2);
535     } else if (t.endsWith("p")) {
536         mult = 1e-12;
537         t = t.slice(0, -1);
538     } else if (t.endsWith("n")) {
539         mult = 1e-9;
540         t = t.slice(0, -1);
541     } else if (t.endsWith("m")) {
542         mult = 1e-3;
543         t = t.slice(0, -1);
544     } else if (t.endsWith("f")) {
545         mult = 1;
546         t = t.slice(0, -1);
547     }
548     const n = parseFloat(t);
549     return Number.isFinite(n) && n > 0 ? n * mult : 1e-6;
550 }
551
552 function parseInductanceHenry(s) {
553     if (s == null) return 1e-3;
554     let t = String(s).trim().toLowerCase().replace(/\s/g, "").replace(",", ".").replace("μ", "u");
555     if (!t) return 1e-3;
556     let mult = 1;
557     if (t.endsWith("mh")) {
558         mult = 1e-3;

```

```

559     t = t.slice(0, -2);
560 } else if (t.endsWith("m")) {
561     mult = 1e-3;
562     t = t.slice(0, -1);
563 } else if (t.endsWith("uh")) {
564     mult = 1e-6;
565     t = t.slice(0, -2);
566 } else if (t.endsWith("kh")) {
567     mult = 1e3;
568     t = t.slice(0, -2);
569 } else if (t.endsWith("h")) {
570     mult = 1;
571     t = t.slice(0, -1);
572 }
573 const n = parseFloat(t);
574 return Number.isFinite(n) && n > 0 ? n * mult : 1e-3;
575 }
576
577 /** Nom de modèle SPICE pour un BJT (ex. 2N2222 → Q2N2222). */
578 function spiceBjtModelName(value) {
579     const raw = String(value || "2N2222")
580         .trim()
581         .replace(/\s+/g, "");
582     const safe = raw.replace(/^[a-zA-Z0-9_]/g, "");
583     const base = safe.length ? safe : "2N2222";
584     return /^[a-zA-Z_]/.test(base) ? `Q${base}` : `Q_${base}`;
585 }
586
587 /** Nom de modèle SPICE pour une diode (ex. 1N4148 → D1N4148). */
588 function spiceDiodeModelName(value) {
589     const raw = String(value || "1N4148")
590         .trim()
591         .replace(/\s+/g, "");
592     const safe = raw.replace(/^[a-zA-Z0-9_]/g, "");
593     const base = safe.length ? safe : "1N4148";
594     return /^[a-zA-Z_]/.test(base) ? `D${base}` : `D_${base}`;
595 }
596
597 function parseDcVolts(s) {
598     if (s == null) return 5;
599     const t = String(s).trim().replace(/\s/g, "").replace(",", ".");
600     const m = /^([-+]?[\d.])\s*v?$/i.exec(t);
601     if (m) return parseFloat(m[1]) || 5;
602     const n = parseFloat(t.replace(/v$/i, ""));
603     return Number.isFinite(n) ? n : 5;
604 }
605
606 /** Carré : « 5V 5V 1kHz 0V » – amplitudes + et - au-dessus / en dessous de l'offset. */
607 function parseSquareAmplitudes(s) {
608     const raw = String(s || "").trim();
609     const pair = /^([-+]?[\d.])\s*v\s+([-+]?[\d.])\s*v$/i.exec(raw);
610     if (pair) {
611         return {
612             ampPos: Math.abs(parseFloat(pair[1])) || 5,
613             ampNeg: Math.abs(parseFloat(pair[2])) || 5,
614         };
615     }
616     const vpk = parseDcVolts(s);
617     return { ampPos: vpk, ampNeg: vpk };
618 }
619
620 /** Amplitude crête sinus : premier « ..V » en tête de chaîne. */
621 function parseSinusAmplitudeVolts(s) {
622     const raw = String(s || "").trim();
623     const m = /^([-+]?[\d.])\s*v$/i.exec(raw);
624     if (!m) return 5;
625     const n = parseFloat(m[1]);
626     return Number.isFinite(n) ? Math.abs(n) : 5;
627 }
628
629 /** Phase sinus (°) : « 90° », « phase 45 », etc. */
630 function parsePhaseDeg(s) {
631     const raw = String(s || "").trim();
632     let m = /(?:phase|phi)\s*([-+]?[\d.])\s*(?:°|deg)?/i.exec(raw);
633     if (m) {
634         const n = parseFloat(m[1]);
635         return Number.isFinite(n) ? n : 0;
636     }
637     m = /([-+]?[\d.])\s*(?:°|deg)\s*$/i.exec(raw);
638     if (m) {

```

```

639     const n = parseFloat(m[1]);
640     return Number.isFinite(n) ? n : 0;
641 }
642 return 0;
643 }
644
645 function parseOpampVp(c) {
646     const n = Number(c?.vp);
647     return Number.isFinite(n) ? n : 15;
648 }
649
650 function parseOpampVn(c) {
651     const n = Number(c?.vn);
652     return Number.isFinite(n) ? n : -15;
653 }
654
655 /** Offset DC (V) : dernier « ...V » après la fréquence (sinus ou carré), ou mot offset. */
656 function parseOffsetVolts(s) {
657     if (s == null) return 0;
658     const raw = String(s).trim();
659     const mOff = /(?:offset|off|dc)\s*([-\+]?\d\.+)\s*v?/i.exec(raw);
660     if (mOff) {
661         const n = parseFloat(mOff[1]);
662         return Number.isFinite(n) ? n : 0;
663     }
664     const volts = [...raw.matchAll(/([-\+]?\d\.+)\s*v/gi)];
665     const isSquarePair = /^([-+]?\d\.+)\s*v\s+([-+]?\d\.+)\s*v/i.test(raw);
666     if (isSquarePair) {
667         if (volts.length >= 3) {
668             const n = parseFloat(volts[volts.length - 1][1]);
669             return Number.isFinite(n) ? n : 0;
670         }
671         return 0;
672     }
673     if (volts.length >= 2) {
674         const n = parseFloat(volts[volts.length - 1][1]);
675         return Number.isFinite(n) ? n : 0;
676     }
677     return 0;
678 }
679
680 /** Extrait la fréquence (Hz) d'une chaîne du type « 5V 1kHz », « 5V 60s », « 5V 1/60Hz ». */
681 function parseFreqHz(s) {
682     if (s == null) return 1000;
683     const t = String(s).toLowerCase().replace(/\\s/g, "").replace(",", ".");
684     let m = /period[e]?=([\\d\.]+)s/.exec(t);
685     if (m) {
686         const p = parseFloat(m[1]);
687         return Number.isFinite(p) && p > 0 ? 1 / p : 1000;
688     }
689     m = /([\\d\.]+)s(?:period|per)?$/i.exec(t) || /([\\d\.]+)s(?:!\\/)/i.exec(t);
690     if (m) {
691         const p = parseFloat(m[1]);
692         return Number.isFinite(p) && p > 0 ? 1 / p : 1000;
693     }
694     m = /1\\/([\\d\.]+)s*hz/i.exec(t);
695     if (m) {
696         const p = parseFloat(m[1]);
697         return Number.isFinite(p) && p > 0 ? 1 / p : 1000;
698     }
699     m = /([\\d\.]+)s*khz/i.exec(t);
700     if (m) {
701         const n = parseFloat(m[1]);
702         return Number.isFinite(n) && n > 0 ? n * 1000 : 1000;
703     }
704     m = /([\\d\.]+)s*mhz/i.exec(t);
705     if (m) {
706         const n = parseFloat(m[1]);
707         return Number.isFinite(n) && n > 0 ? n * 1e6 : 1000;
708     }
709     m = /([\\d\.]+)s*hz/i.exec(t);
710     if (m) {
711         const n = parseFloat(m[1]);
712         return Number.isFinite(n) && n > 0 ? n : 1000;
713     }
714     return 1000;
715 }
716
717 /** Amplitude haute d'un générateur d'impulsions : « 5V 1kHz 10% ». */
718 function parsePulseHighVolts(s) {

```

```

719     if (s == null) return 5;
720     const m = /[[-+]?[\\d.]+)\\s*v/i.exec(String(s));
721     if (m) {
722         const n = parseFloat(m[1]);
723         return Number.isFinite(n) && n > 0 ? n : 5;
724     }
725     return 5;
726 }
727
728 /** Rapport cyclique (%) : « 5V 1kHz 10% » ou « duty 25 ». */
729 function parseDutyPercent(s) {
730     if (s == null) return 10;
731     const raw = String(s).trim();
732     let m = /[\\d.]+)\\s*%/i.exec(raw);
733     if (m) {
734         const n = parseFloat(m[1]);
735         return Number.isFinite(n) ? Math.min(99, Math.max(1, n)) : 10;
736     }
737     m = /duty\\s*([\\d.]+)/i.exec(raw);
738     if (m) {
739         const n = parseFloat(m[1]);
740         return Number.isFinite(n) ? Math.min(99, Math.max(1, n)) : 10;
741     }
742     return 10;
743 }
744
745 function spiceBranchName(prefix, id) {
746     const safe = String(id).replace(/^[a-zA-Z0-9_]/g, "_");
747     return `${prefix}_${safe}`;
748 }
749
750 const LOGIC_VLO = 0;
751
752 function isLogicGateComponentType(t) {
753     return (
754         t === "logic_not" ||
755         t === "logic_and" ||
756         t === "logic_or" ||
757         t === "logic_nand" ||
758         t === "logic_nor" ||
759         t === "logic_xor" ||
760         t === "logic_xnor"
761     );
762 }
763
764 /** Expression ngspice (source comportementale B) : niveaux 0 / rail, seuil à mi-rail. */
765 function logicGateBsourceExpression(type, inputNodes, vhi) {
766     const hi = vhi;
767     const lo = LOGIC_VLO;
768     const th = vhi / 2;
769     const a = inputNodes[0];
770     const b = inputNodes[1];
771     const gt = n => `u(V(${n})-${th})`;
772     switch (type) {
773         case "logic_not":
774             return `{ (1-${gt(a)})*${hi} }`;
775         case "logic_and":
776             return `{ (${gt(a)})*(${gt(b)})*${hi} }`;
777         case "logic_nand":
778             return `{ (1-(${gt(a)})*(${gt(b)}))*${hi} }`;
779         case "logic_or":
780             return `{ ((${gt(a)})+(${gt(b)})>0)*${hi} }`;
781         case "logic_nor":
782             return `{ (1-((${gt(a)})+(${gt(b)})>0))*${hi} }`;
783         case "logic_xor":
784             return `{ abs(${gt(a)}-${gt(b)})*${hi} }`;
785         case "logic_xnor":
786             return `{ (1-abs(${gt(a)}-${gt(b)}))*${hi} }`;
787         default:
788             return `{ ${lo} }`;
789     }
790 }
791
792 function logicGateOutputNodeKey(c) {
793     return c.type === "logic_not" ? `${c.id}#1` : `${c.id}#2`;
794 }
795
796 function logicGateInputNodeKeys(c) {
797     if (c.type === "logic_not") return [`${c.id}#0`];
798     return [`${c.id}#0`, `${c.id}#1`];

```

```

799 }
800
801 /** Rail de sortie d'une porte : max des entrées (et override logicRail), pas la borne sortie (peut être à la masse). */
802 function resolveLogicGateVhi(c, logicVhiByTerminal) {
803   let vhi = 0;
804   for (const k of logicGateInputNodeKeys(c)) {
805     vhi = Math.max(vhi, logicVhiByTerminal.get(k) ?? 0);
806   }
807   if (c.logicRail != null && c.logicRail !== "") {
808     vhi = Math.max(vhi, logicVhi(parseLogicRail(c.logicRail)));
809   }
810   if (vhi <= 0) {
811     const outKey = logicGateOutputNodeKey(c);
812     vhi = logicVhiByTerminal.get(outKey) ?? 5;
813   }
814   return vhi > 0 ? vhi : 5;
815 }
816
817 /** Rail logique (V) par borne : états logiques, propagation réseau, sorties de portes. */
818 function computeLogicVhiByTerminalKey(components, parent) {
819   const vhiByKey = new Map();
820
821   const touchKey = (k, v) => {
822     if (!k || !(v > 0)) return;
823     const prev = vhiByKey.get(k);
824     if (prev == null || v > prev) vhiByKey.set(k, v);
825   };
826
827   for (const c of components) {
828     if (c.type === "logic_state") {
829       touchKey(`${c.id}#0`, logicVhi(parseLogicRail(c.logicRail)));
830     }
831     if (c.type === "vsquare") {
832       const { ampPos } = parseSquareAmplitudes(c.value);
833       const voff = parseOffsetVolts(c.value);
834       touchKey(`${c.id}#0`, voff + ampPos);
835     }
836     if (c.type === "vpulse") {
837       touchKey(`${c.id}#0`, parsePulseHighVolts(c.value));
838     }
839     if (c.type === "vsin") {
840       const voff = parseOffsetVolts(c.value);
841       const vpk = parseSinusAmplitudeVolts(c.value);
842       touchKey(`${c.id}#0`, voff + vpk);
843     }
844     if (isLogicGateComponentType(c.type) && c.logicRail != null && c.logicRail !== "") {
845       const v = logicVhi(parseLogicRail(c.logicRail));
846       for (const k of logicGateInputNodeKeys(c)) touchKey(k, v);
847     }
848     if (c.type === "logic_dff" && c.logicRail != null && c.logicRail !== "") {
849       const v = logicVhi(parseLogicRail(c.logicRail));
850       for (const k of logicDffInputNodeKeys(c)) touchKey(k, v);
851     }
852     if (c.type === "logic_jk" && c.logicRail != null && c.logicRail !== "") {
853       const v = logicVhi(parseLogicRail(c.logicRail));
854       for (const k of logicJkInputNodeKeys(c)) touchKey(k, v);
855     }
856     if (isLogicIcType(c.type) && c.logicRail != null && c.logicRail !== "") {
857       const vccIdx = isIc74ls74Type(c.type)
858         ? ic74ls74VccPinIndex()
859         : isIc74hc90Type(c.type)
860           ? ic74hc90VccPinIndex()
861           : ic74ls00VccPinIndex();
862       touchKey(`${c.id}#${vccIdx}`, logicVhi(parseLogicRail(c.logicRail)));
863     }
864   }
865
866   const propagateNets = () => {
867     const netMax = new Map();
868     for (const [k] of parent) {
869       const r = uFind(parent, k);
870       const v = vhiByKey.get(k);
871       if (v != null && v > 0) netMax.set(r, Math.max(netMax.get(r) ?? 0, v));
872     }
873     let changed = false;
874     for (const [k] of parent) {
875       const r = uFind(parent, k);
876       const target = netMax.get(r);
877       if (target == null || target <= 0) continue;
878       if ((vhiByKey.get(k) ?? 0) < target) {

```

```

879         vhiByKey.set(k, target);
880         changed = true;
881     }
882 }
883 return changed;
884 };
885
886 for (let iter = 0; iter < 48; iter++) {
887     let changed = false;
888     for (const c of components) {
889         if (!isLogicGateComponentType(c.type)) continue;
890         let inVhi = 0;
891         for (const k of logicGateInputNodeKeys(c)) {
892             inVhi = Math.max(inVhi, vhiByKey.get(k) ?? 0);
893         }
894         if (inVhi <= 0) inVhi = 5;
895         const outKey = logicGateOutputNodeKey(c);
896         if ((vhiByKey.get(outKey) ?? 0) < inVhi) {
897             vhiByKey.set(outKey, inVhi);
898             changed = true;
899         }
900     }
901     while (propagateNets()) changed = true;
902     if (!changed) break;
903 }
904
905 return vhiByKey;
906 }
907
908 /** Durée SPICE (ex. 2.5u, 4m) pour .tran / wrdata. */
909 function formatSpiceTime(seconds) {
910     const s = Math.abs(Number(seconds));
911     if (!Number.isFinite(s) || s === 0) return "1n";
912     if (s >= 1) return String(s);
913     if (s >= 1e-3) return `${(s * 1e3).toFixed(6)}m`;
914     if (s >= 1e-6) return `${(s * 1e6).toFixed(6)}u`;
915     if (s >= 1e-9) return `${(s * 1e9).toFixed(6)}n`;
916     return `${s.toExponential(3)}`;
917 }
918
919 /** Aligné sur l'oscilloscope UI (8 div. horizontales, jusqu'à 100 ms/div). */
920 const TRAN_SCOPE_H_DIVS = 8;
921 const TRAN_MAX_TIME_DIV_SEC = 0.1;
922 const TRAN_SAMPLES_PER_PERIOD = 200;
923 const TRAN_MAX_POINTS = 50000;
924 /** Au-delà de cette période (f ≤ 1 Hz), .tran court + animation temps réel côté client. */
925 const SLOW_CLOCK_PERIOD_SEC = 1;
926 /** Un seul HC90 : 2 cycles décade (0.9 × 2). */
927 const SLOW_CLOCK_TRAN_PERIODS = 20;
928 /** Compteur ripple mod-10 : 2 décades complètes (0.9 × 2) + marge reset. */
929 const RIPPLE_MOD10_TRAN_PERIODS = 22;
930 /** Deux HC90 en cascade : cycle complet 0.99 (100 impulsions) ou 0.59 (60) si reset mod-60. */
931 const SLOW_CLOCK_TWO_DIGIT_PERIODS = 100;
932 const SLOW_CLOCK_MOD60_PERIODS = 60;
933 const SLOW_CLOCK_MAX_TSTOP_SEC = 120;
934
935 /** Pas et durée de simulation transitoire selon les générateurs AC et la base de temps du scope. */
936 function computeTranTiming(components, deckOpts = {}) {
937     let minPeriod = Infinity;
938     for (const c of components) {
939         if (c.type !== "vsin" && c.type !== "vsquare" && c.type !== "vpulse") continue;
940         const f = parseFreqHz(c.value);
941         if (f > 0) minPeriod = Math.min(minPeriod, 1 / f);
942     }
943     const arduinoPeriod = arduinoUnoMinPulsePeriodSec(components);
944     if (arduinoPeriod > 0) minPeriod = Math.min(minPeriod, arduinoPeriod);
945     const i2cPeriod = i2cBusMinPeriodSec(components);
946     if (i2cPeriod > 0) minPeriod = Math.min(minPeriod, i2cPeriod);
947     if (!Number.isFinite(minPeriod) || minPeriod <= 0) minPeriod = 1;
948
949     const lcHz = estimateParallelLcResonantHz(components);
950     const hasAcGenerator = components.some(
951         (c) => c.type === "vsin" || c.type === "vsquare" || c.type === "vpulse"
952     );
953     const opampOscillator = lcHz > 0 && !hasAcGenerator && components.some((c) => isOpampType(c.type));
954
955     if (opampOscillator) {
956         minPeriod = 1 / lcHz;
957     }
958 }

```

```

959     let scopeWindowSec = 0;
960     for (const c of components) {
961         if (c.type === "oscilloscope" && Number(c.timeDivSec) > 0) {
962             scopeWindowSec = Math.max(scopeWindowSec, c.timeDivSec * TRAN_SCOPE_H_DIVS);
963         }
964     }
965     if (scopeWindowSec <= 0) scopeWindowSec = TRAN_SCOPE_H_DIVS * 0.001;
966
967     let tstep = minPeriod / TRAN_SAMPLES_PER_PERIOD;
968     const hasScope = components.some((c) => c.type === "oscilloscope");
969     const hasSpeaker = components.some((c) => isSpeakerType(c.type));
970     let tstop = hasScope
971         ? Math.max(scopeWindowSec, minPeriod * 2)
972         : Math.max(minPeriod * 8, scopeWindowSec);
973     if (hasSpeaker) {
974         tstop = Math.max(tstop, minPeriod * 20, 0.25);
975         // Durée = nombre entier de périodes du générateur → boucle audio sans « toc ».
976         if (minPeriod > 0 && Number.isFinite(minPeriod)) {
977             const periods = Math.max(20, Math.ceil(tstop / minPeriod));
978             tstop = periods * minPeriod;
979         }
980     }
981
982     const numFf =
983         components.filter((c) => c.type === "logic_dff" || c.type === "logic_jk").length +
984         components.filter((c) => isIc74hc90Type(c.type)).length * 4;
985     const hc90Count = components.filter((c) => isIc74hc90Type(c.type)).length;
986     const hc90Mod60 = deckOpts.hc90Mod60 === true;
987     const twoDigitPeriods = hc90Mod60 ? SLOW_CLOCK_MOD60_PERIODS : SLOW_CLOCK_TWO_DIGIT_PERIODS;
988     if (numFf > 0) {
989         const ripplePeriods = (1 << numFf) + 2;
990         tstop = Math.max(tstop, minPeriod * ripplePeriods);
991     }
992     const slowClock = minPeriod >= SLOW_CLOCK_PERIOD_SEC;
993     if (hc90Count > 0) {
994         if (slowClock) {
995             const slowPeriods =
996                 hc90Count >= 2 ? twoDigitPeriods : SLOW_CLOCK_TRAN_PERIODS;
997             tstop = Math.min(
998                 SLOW_CLOCK_MAX_TSTOP_SEC,
999                 Math.max(minPeriod * slowPeriods, minPeriod * 8)
1000             );
1001         } else {
1002             const minRipplePeriods = hc90Count >= 2 ? twoDigitPeriods : 24;
1003             tstop = Math.max(tstop, minPeriod * minRipplePeriods);
1004         }
1005     }
1006
1007     if (tstop / tstep > TRAN_MAX_POINTS) {
1008         tstep = tstop / TRAN_MAX_POINTS;
1009         const minStep = minPeriod / 40;
1010         if (tstep > minStep) {
1011             tstep = minStep;
1012             if (tstop / tstep > TRAN_MAX_POINTS) {
1013                 tstop = tstep * TRAN_MAX_POINTS;
1014             }
1015         }
1016     }
1017     if (hc90Count > 0 && !slowClock) {
1018         const hc90MinTstop = minPeriod * 12;
1019         if (tstop < hc90MinTstop) {
1020             tstop = hc90MinTstop;
1021             if (tstop / tstep > TRAN_MAX_POINTS) {
1022                 tstep = tstop / TRAN_MAX_POINTS;
1023             }
1024         }
1025     }
1026     if (opampOscillator) {
1027         tstop = Math.max(tstop, minPeriod * 50);
1028         if (tstop / tstep > TRAN_MAX_POINTS) {
1029             tstep = tstop / TRAN_MAX_POINTS;
1030         }
1031     }
1032     if (deckOpts.rippleMod10) {
1033         tstop = Math.max(tstop, minPeriod * RIPPLE_MOD10_TRAN_PERIODS);
1034         if (tstop / tstep > TRAN_MAX_POINTS) {
1035             tstep = tstop / TRAN_MAX_POINTS;
1036         }
1037     }
1038     // quickTran (serveur Linux) : fenêtre .tran réduite pour limiter le temps ngspice.

```



```

1039 // Décade seule : au moins 10 impulsions (0..9) - 6 provoquait un comptage bloqué à 0..5 en réseau.
1040 // Deux chiffres + horloge lente : ne pas tronquer (sinon 1 Hz → 8 s → affichage ~07).
1041 if (deckOpts.quickTran && hc90Count > 0 && !(slowClock && hc90Count >= 2)) {
1042     const quickPeriods = hc90Count >= 2 ? 8 : 10;
1043     tstop = Math.min(tstop, minPeriod * quickPeriods);
1044     if (tstop / tstep > TRAN_MAX_POINTS) {
1045         tstep = tstop / TRAN_MAX_POINTS;
1046     }
1047 }
1048 if (deckOpts.quickTran && deckOpts.rippleMod10) {
1049     tstop = Math.max(tstop, minPeriod * RIPPLE_MOD10_TRAN_PERIODS);
1050     if (tstop / tstep > TRAN_MAX_POINTS) {
1051         tstep = tstop / TRAN_MAX_POINTS;
1052     }
1053 }
1054 const hasArduinoPulse = components.some(
1055     (c) =>
1056         isArduinoUnoType(c.type) &&
1057         ((c.pinPulses && Object.keys(c.pinPulses).length > 0) ||
1058             (Array.isArray(c.pinPhases) && c.pinPhases.length >= 2))
1059 );
1060 if (deckOpts.quickTran && hasArduinoPulse && minPeriod > 0) {
1061     const arduinoQuickPeriods = 12;
1062     tstop = Math.min(tstop, minPeriod * arduinoQuickPeriods);
1063     if (tstop / tstep > TRAN_MAX_POINTS) {
1064         tstep = tstop / TRAN_MAX_POINTS;
1065     }
1066 }
1067 return {
1068     tstep,
1069     tstop,
1070     tstepStr: formatSpiceTime(tstep),
1071     tstopStr: formatSpiceTime(tstop),
1072     clockPeriodSec: minPeriod,
1073     slowClock,
1074 };
1075 }
1076
1077 const WIRE_EPS = 1e-6;
1078
1079 function parseVirtualWirePointKey(key) {
1080     const m = /^_(?:t|p|#[^#]+)(#[^#]+)$/.exec(String(key || ""));
1081     if (!m) return null;
1082     const x = Number(m[1]);
1083     const y = Number(m[2]);
1084     return Number.isFinite(x) && Number.isFinite(y) ? { x, y } : null;
1085 }
1086
1087 function pointOnWireSegment(p, a, b) {
1088     if (!p || !a || !b) return false;
1089     if (Math.abs(a.x - b.x) < WIRE_EPS) {
1090         return (
1091             Math.abs(p.x - a.x) < WIRE_EPS &&
1092             p.y >= Math.min(a.y, b.y) - WIRE_EPS &&
1093             p.y <= Math.max(a.y, b.y) + WIRE_EPS
1094         );
1095     }
1096     if (Math.abs(a.y - b.y) < WIRE_EPS) {
1097         return (
1098             Math.abs(p.y - a.y) < WIRE_EPS &&
1099             p.x >= Math.min(a.x, b.x) - WIRE_EPS &&
1100             p.x <= Math.max(a.x, b.x) + WIRE_EPS
1101         );
1102     }
1103     return false;
1104 }
1105
1106 /**
1107  * UNO + bargraph / LED / 7 segments : l'animation GPIO est gérée côté client ;
1108  * inutile de lancer un .tran ngspice (lent sur Render).
1109  */
1110 function isArduinoIdealOnlyCircuit(components) {
1111     const arduinos = components.filter((c) => isArduinoUnoType(c.type));
1112     if (!arduinos.length) return false;
1113     if (!arduinos.some((c) => arduinoGpioIsTimeVarying(c))) return false;
1114     for (const c of components) {
1115         if (isArduinoUnoType(c.type)) continue;
1116         if (
1117             c.type === "bargraph_dc10h" ||
1118             c.type === "seg7" ||

```

```

1119         c.type === "led" ||
1120         c.type === "grove_lcd16x2" ||
1121         c.type === "grove_dht22" ||
1122         c.type === "ground" ||
1123         c.type === "gnd" ||
1124         c.type === "push_button" ||
1125         c.type === "switch_spdt" ||
1126         c.type === "voltmeter" ||
1127         c.type === "ammeter" ||
1128         c.type === "resistor" ||
1129         c.type === "capacitor" ||
1130         c.type === "inductor" ||
1131         c.type === "diode" ||
1132         c.type === "potentiometer"
1133     ) {
1134         continue;
1135     }
1136     return false;
1137 }
1138 return true;
1139 }
1140
1141 /**
1142  * @param {{ components: any[]; wires: any[] }} state
1143  * @param {{ gridStep?: number }} [opts]
1144  */
1145 export function buildNetlistFromGraphicalState(state, opts = {}) {
1146     const deckOpts = { repoRoot: opts.repoRoot || DEFAULT_REPO_ROOT, ...opts };
1147     const warnings = [];
1148     const components = Array.isArray(state.components) ? state.components : [];
1149     const wires = Array.isArray(state.wires) ? state.wires : [];
1150
1151     for (const c of components) {
1152         if (isArduinoUnoType(c.type)) {
1153             applyArduinoSketchToComponent(c);
1154             annotateUnoI2cBusEngine(c, components, wires);
1155         }
1156     }
1157
1158     const parent = new Map();
1159     const terminalWireCount = new Map();
1160
1161     function touch(k) {
1162         if (!parent.has(k)) parent.set(k, k);
1163     }
1164
1165     for (const c of components) {
1166         for (const k of terminalKeysForComponent(c)) touch(k);
1167     }
1168     for (const w of wires) {
1169         if (!w || !w.solid || !w.fromKey || !w.toKey) continue;
1170         touch(w.fromKey);
1171         touch(w.toKey);
1172         terminalWireCount.set(w.fromKey, (terminalWireCount.get(w.fromKey) || 0) + 1);
1173         terminalWireCount.set(w.toKey, (terminalWireCount.get(w.toKey) || 0) + 1);
1174         ufUnion(parent, w.fromKey, w.toKey);
1175     }
1176     const virtualKeys = new Set();
1177     for (const w of wires) {
1178         if (!w || !w.solid) continue;
1179         if (parseVirtualWirePointKey(w.fromKey)) virtualKeys.add(w.fromKey);
1180         if (parseVirtualWirePointKey(w.toKey)) virtualKeys.add(w.toKey);
1181     }
1182     for (const key of virtualKeys) {
1183         const p = parseVirtualWirePointKey(key);
1184         for (const w of wires) {
1185             if (!w || !w.solid || !w.fromKey || !Array.isArray(w.points) || w.points.length < 2) continue;
1186             for (let i = 0; i < w.points.length - 1; i++) {
1187                 if (pointOnWireSegment(p, w.points[i], w.points[i + 1])) {
1188                     ufUnion(parent, key, w.fromKey);
1189                     break;
1190                 }
1191             }
1192         }
1193     }
1194
1195     const powerSrc =
1196         components.find(c => c.type === "vsource") ||
1197         components.find(c => c.type === "vsin") ||
1198         components.find(c => c.type === "vsquare") ||

```

```

1199     components.find(c => c.type === "vpulse");
1200     const vtermComponents = components.filter(c => c.type === "vterm" || c.type === "logic_state");
1201     const groundComponents = components.filter(c => c.type === "ground");
1202     const hasVtermPower = vtermComponents.length > 0;
1203     const ohmeterComponents = components.filter(c => c.type === "ohmmeter");
1204     const needsDcSupply = components.some(
1205         c =>
1206             c.type === "voltmeter" ||
1207             c.type === "ammeter" ||
1208             c.type === "voltmeter_rms" ||
1209             c.type === "ammeter_rms" ||
1210             c.type === "oscilloscope" ||
1211             isBodeAnalyzerType(c.type) ||
1212             isSpeakerType(c.type)
1213     );
1214
1215     const hasAutonomousLcOscillator = circuitHasAutonomousLcOscillator(components);
1216     const hasArduinoUno = components.some((c) => isArduinoUnoType(c.type));
1217     if (needsDcSupply && !powerSrc && !hasVtermPower && !hasAutonomousLcOscillator && !hasArduinoUno) {
1218         return {
1219             ok: false,
1220             errors: [
1221                 "Voltmètre, ampèremètre (DC ou efficace), oscilloscope : ajoutez une source (pile, borne, état logique, sinus, carré ou impulsions GImp).",
1222             ],
1223             warnings,
1224             netlist: "",
1225             voltmeters: [],
1226             ammeters: [],
1227             ohmeters: [],
1228             oscilloscopes: [],
1229             nodeMeasures: [],
1230             scopesTranMeta: [],
1231             analysisTran: false,
1232         };
1233     }
1234     if (!powerSrc && !hasVtermPower && !hasArduinoUno && ohmeterComponents.length === 0 && groundComponents.length === 0) {
1235         return {
1236             ok: false,
1237             errors: [
1238                 "Ajoutez une pile DC, une borne, un état logique, un générateur (sinus/carré), une masse, ou un ohmmètre pour définir le circuit.",
1239             ],
1240             warnings,
1241             netlist: "",
1242             voltmeters: [],
1243             ammeters: [],
1244             ohmeters: [],
1245             oscilloscopes: [],
1246             nodeMeasures: [],
1247             scopesTranMeta: [],
1248             analysisTran: false,
1249         };
1250     }
1251
1252     let gndKey;
1253     if (groundComponents.length > 0) {
1254         gndKey = `${groundComponents[0].id}#0`;
1255     } else if (powerSrc) {
1256         if (powerSrc.type === "vsin" || powerSrc.type === "vsquare") {
1257             gndKey = `${powerSrc.id}#0`;
1258         } else {
1259             gndKey = `${powerSrc.id}#1`;
1260         }
1261     } else if (ohmeterComponents.length > 0) {
1262         gndKey = `${ohmeterComponents[0].id}#1`;
1263     } else {
1264         gndKey = "__spice_gnd__";
1265     }
1266
1267     touch(gndKey);
1268     for (const g of groundComponents) {
1269         ufUnion(parent, `${g.id}#0`, gndKey);
1270     }
1271     if (
1272         powerSrc &&
1273         (powerSrc.type === "vsin" || powerSrc.type === "vsquare") &&
1274         groundComponents.length > 0
1275     ) {
1276         ufUnion(parent, `${powerSrc.id}#0`, gndKey);

```

```

1277     }
1278     const gndRoot = uFind(parent, gndKey);
1279     if (powerSrc) {
1280         const vsrcP = `${powerSrc.id}#0`;
1281         const vsrcM = `${powerSrc.id}#1`;
1282         if ((terminalWireCount.get(vsrcP) || 0) === 0 || (terminalWireCount.get(vsrcM) || 0) === 0) {
1283             warnings.push(`Source ${powerSrc.id} : au moins une borne n'est reliée à aucun fil.`);
1284         }
1285     } else if (ohmeterComponents.length > 0) {
1286         warnings.push(
1287             "Mode ohmmètre : pas de pile – référence sur la borne « - » du premier ohmmètre ; courant de test injecté pour calculer R."
1288         );
1289     } else if (hasVtermPower) {
1290         for (const vt of vtermComponents) {
1291             const kp = `${vt.id}#0`;
1292             if ((terminalWireCount.get(kp) || 0) === 0) {
1293                 const label = isLogicStateType(vt.type) ? "État logique" : "Borne";
1294                 warnings.push(`${label} ${vt.id} : la connexion n'est reliée à aucun fil.`);
1295             }
1296         }
1297     }
1298
1299     /* Nœuds internes LED / bascules : doivent exister avant l'attribution n1, n2... */
1300     for (const c of components) {
1301         if (isLedType(c.type)) touch(`${c.id}#__ledint`);
1302         for (const k of logicSequentialInternalNodeKeys(c, {
1303             ...deckOpts,
1304             rippleClockFromPrev: getRippleClockFromPrev(c, wires, components),
1305         })) touch(k);
1306         if (isIc74hc90Type(c.type)) {
1307             for (const k of ic74hc90InternalNodeKeys(c, deckOpts)) touch(k);
1308         }
1309     }
1310
1311     const roots = new Set();
1312     for (const k of parent.keys()) roots.add(uFind(parent, k));
1313
1314     const rootToSpice = new Map();
1315     rootToSpice.set(gndRoot, "0");
1316     let ni = 1;
1317     const sortedRoots = [...roots].sort();
1318     for (const r of sortedRoots) {
1319         if (r === gndRoot) continue;
1320         rootToSpice.set(r, `n${ni++}`);
1321     }
1322
1323     function nodeFor(key) {
1324         touch(key);
1325         const root = uFind(parent, key);
1326         if (!rootToSpice.has(root)) {
1327             rootToSpice.set(root, `n${ni++}`);
1328         }
1329         return rootToSpice.get(root);
1330     }
1331
1332     const logicVhiByTerminal = computeLogicVhiByTerminalKey(components, parent);
1333
1334     const rippleMod10Pair = getRippleMod10ResetAndIndices(components, parent);
1335     if (rippleMod10Pair?.a === 2 && rippleMod10Pair?.b === 4) {
1336         deckOpts.rippleMod10 = true;
1337     }
1338
1339     const bodeAnalyzerComponents = components.filter((c) => isBodeAnalyzerType(c.type));
1340     const hasBodeAnalyzer = bodeAnalyzerComponents.length > 0;
1341     const hasBlockingTranForBode =
1342         components.some((c) => c.type === "vpulse") ||
1343         components.some((c) => isLogicGateComponentType(c.type) || isLogicDigitalSimType(c.type)) ||
1344         hasAutonomousLcOscillator;
1345     const useAcForBode = hasBodeAnalyzer && !hasBlockingTranForBode;
1346     if (hasBodeAnalyzer && hasBlockingTranForBode) {
1347         warnings.push(
1348             "Analyse fréquentielle : incompatible avec les impulsions GImp, la logique numérique ou l'oscillateur LC autonome."
1349         );
1350     }
1351     if (useAcForBode && !components.some((c) => c.type === "vsin")) {
1352         return {
1353             ok: false,
1354             errors: [
1355                 "Analyse fréquentielle : ajoutez un générateur sinus (Sin) comme source d'entrée du filtre.",
1356             ],
1357         };
1358     }

```

```

1357     warnings,
1358     netlist: "",
1359     voltmeters: [],
1360     ammeters: [],
1361     ohmeters: [],
1362     oscilloscopes: [],
1363     bodeAnalyzers: [],
1364     nodeMeasures: [],
1365     scopesTranMeta: [],
1366     bodeAcMeta: [],
1367     analysisTran: false,
1368     analysisAc: false,
1369 };
1370 }
1371
1372 const hasLogicDff = components.some((c) => c.type === "logic_dff");
1373 const hasLogicJk = components.some((c) => c.type === "logic_jk");
1374 const hasLogicCd4511 = components.some((c) => isLogicCd4511Type(c.type));
1375 const usesXspiceFf =
1376     (hasLogicDff && useLogicDffXspice(deckOpts)) || (hasLogicJk && useLogicJkXspice(deckOpts));
1377 const usesXspiceCd4511 = hasLogicCd4511 && useLogicCd4511Xspice(deckOpts);
1378 const usesBsourceCd4511 = hasLogicCd4511 && !usesXspiceCd4511;
1379 const usesXspiceDigital = usesXspiceFf || usesXspiceCd4511;
1380 const lines = [];
1381 lines.push("* Circuit Designer - netlist SPICE (.op)");
1382 if (usesXspiceDigital) {
1383     for (const cmLine of xspiceCodemodellines(deckOpts.repoRoot)) {
1384         lines.push(cmLine);
1385     }
1386     const xParts = [];
1387     if (hasLogicDff && useLogicDffXspice(deckOpts)) xParts.push("D (d_dff)");
1388     if (hasLogicJk && useLogicJkXspice(deckOpts)) xParts.push("JK (d_jkff)");
1389     if (usesXspiceCd4511) xParts.push("CD4511 (d_d latch + d_genlut)");
1390     warnings.push(
1391         `${xParts.length ? xParts.join(", ") : "Circuits logiques"} : modèle XSPICE (digital.cm) – simulation mixte analogique/numérique.
1392     );
1393 } else if (usesBsourceCd4511) {
1394     warnings.push(
1395         "CD4511 : modèle sources B (compatibilité ngspice Linux / serveur distant – pas de d_genlut).\"
1396     );
1397 } else if (
1398     (hasLogicDff || hasLogicJk) &&
1399     isXspiceDffAvailable(deckOpts.repoRoot)
1400 ) {
1401     warnings.push(
1402         "Bascule(s) : digital.cm présent mais ngspice sans XSPICE – modèle sources B (voir Simulateur/lib/ngspice/README.txt).\"
1403     );
1404 }
1405
1406 const declaredDiodeModels = new Set();
1407 const declaredBjtModels = new Set();
1408
1409 let oscillatorCapKickDone = false;
1410 let oscillatorTankNode = null;
1411 let i2cRepeatSec = 0.02;
1412 for (const c of components) {
1413     if (c.type === "oscilloscope" && Number(c.timeDivSec) > 0) {
1414         i2cRepeatSec = Math.max(i2cRepeatSec, c.timeDivSec * TRAN_SCOPE_H_DIVS);
1415     }
1416 }
1417 for (const c of components) {
1418     if (c.type === "resistor") {
1419         const n0 = nodeFor(`${c.id}#0`);
1420         const n1 = nodeFor(`${c.id}#1`);
1421         const ohms = parseResistanceOhm(c.value);
1422         lines.push(`${spiceBranchName("R", c.id)} ${n0} ${n1} ${ohms}`);
1423     } else if (c.type === "potentiometer") {
1424         const n0 = nodeFor(`${c.id}#0`);
1425         const n1 = nodeFor(`${c.id}#1`);
1426         const n2 = nodeFor(`${c.id}#2`);
1427         const total = Math.max(parseResistanceOhm(c.value), 1);
1428         const pos = Math.min(100, Math.max(0, Number(c.position) || 50)) / 100;
1429         const r1 = Math.max(1e-3, total * pos);
1430         const r2 = Math.max(1e-3, total * (1 - pos));
1431         lines.push(`${spiceBranchName("R", c.id)}a ${n0} ${n1} ${r1}`);
1432         lines.push(`${spiceBranchName("R", c.id)}b ${n1} ${n2} ${r2}`);
1433     } else if (c.type === "switch_spdt") {
1434         const nCom = nodeFor(`${c.id}#0`);
1435         const nA = nodeFor(`${c.id}#1`);

```

```

1436     const nB = nodeFor(`${c.id}#2`);
1437     const rOn = 0.01;
1438     const rOff = 1e9;
1439     const toB = Number(c.state) === 1;
1440     lines.push(`${spiceBranchName("R", c.id)}a ${nCom} ${nA} ${toB ? rOff : rOn}`);
1441     lines.push(`${spiceBranchName("R", c.id)}b ${nCom} ${nB} ${toB ? rOn : rOff}`);
1442 } else if (c.type === "push_button") {
1443     const n0 = nodeFor(`${c.id}#0`);
1444     const n1 = nodeFor(`${c.id}#1`);
1445     const pressed = Number(c.state) === 1;
1446     lines.push(`${spiceBranchName("R", c.id)} ${n0} ${n1} ${pressed ? 0.01 : 1e9}`);
1447 } else if (c.type === "capacitor") {
1448     const n0 = nodeFor(`${c.id}#0`);
1449     const n1 = nodeFor(`${c.id}#1`);
1450     const farad = parseCapacitanceFarad(c.value);
1451     let icSuffix = "";
1452     if (hasAutonomousLcOscillator && !oscillatorCapKickDone) {
1453         icSuffix = " IC=50m";
1454         oscillatorTankNode = n0 !== "0" ? n0 : n1;
1455         oscillatorCapKickDone = true;
1456     }
1457     lines.push(`${spiceBranchName("C", c.id)} ${n0} ${n1} ${farad}${icSuffix}`);
1458 } else if (c.type === "inductor") {
1459     const n0 = nodeFor(`${c.id}#0`);
1460     const n1 = nodeFor(`${c.id}#1`);
1461     const henry = parseInductanceHenry(c.value);
1462     lines.push(`${spiceBranchName("L", c.id)} ${n0} ${n1} ${henry}`);
1463 } else if (isSpeakerType(c.type)) {
1464     const nMinus = nodeFor(`${c.id}#0`);
1465     const nPlus = nodeFor(`${c.id}#1`);
1466     const kp = `${c.id}#0`;
1467     const km = `${c.id}#1`;
1468     if ((terminalWireCount.get(kp) || 0) === 0 || (terminalWireCount.get(km) || 0) === 0) {
1469         warnings.push(
1470             `Haut-parleur ${c.id} : reliez les deux bornes (+ et -) au circuit.`
1471         );
1472     }
1473     if (nPlus === nMinus) {
1474         warnings.push(
1475             `Haut-parleur ${c.id} : les deux bornes sont sur le même nœud (${nPlus}).`
1476         );
1477     }
1478     const ohms = parseResistanceOhm(c.value) > 0 ? parseResistanceOhm(c.value) : 8;
1479     lines.push(`${spiceBranchName("R", c.id)} ${nMinus} ${nPlus} ${ohms}`);
1480 } else if (c.type === "diode") {
1481     const n0 = nodeFor(`${c.id}#0`);
1482     const n1 = nodeFor(`${c.id}#1`);
1483     const model = spiceDiodeModelName(c.value);
1484     if (!declaredDiodeModels.has(model)) {
1485         declaredDiodeModels.add(model);
1486         const v = String(c.value || "").toLowerCase();
1487         if (v.includes("4148")) {
1488             lines.push(
1489                 `.model ${model} D (IS=2.52n RS=0.568 N=1.752 TT=4n CJO=4p M=0.4 BV=100)`
1490             );
1491         } else {
1492             lines.push(`.model ${model} D`);
1493         }
1494     }
1495     lines.push(`${spiceBranchName("D", c.id)} ${n0} ${n1} ${model}`);
1496 } else if (isLedType(c.type)) {
1497     const anode = nodeFor(`${c.id}#0`);
1498     const cathode = nodeFor(`${c.id}#1`);
1499     touch(`${c.id}#__ledint`);
1500     const mid = nodeFor(`${c.id}#__ledint`);
1501     if (!declaredDiodeModels.has("DLED")) {
1502         declaredDiodeModels.add("DLED");
1503         lines.push(
1504             `.model DLED D (IS=1.05E-15 N=1.8 RS=15 BV=50 IBV=10u CJO=10p)`
1505         );
1506     }
1507     /* Source 0 V en série : ngspice ne fournit pas toujours i(D_*), mais i(VIL_*) oui. */
1508     lines.push(`${spiceBranchName("VIL", c.id)} ${anode} ${mid} 0`);
1509     lines.push(`${spiceBranchName("D", c.id)} ${mid} ${cathode} DLED`);
1510 } else if (c.type === "seg7") {
1511     const nCom = nodeFor(`${c.id}#7`);
1512     if (!declaredDiodeModels.has("DLED")) {
1513         declaredDiodeModels.add("DLED");
1514         lines.push(
1515             `.model DLED D (IS=1.05E-15 N=1.8 RS=15 BV=50 IBV=10u CJO=10p)`

```

```

1516     });
1517   }
1518   for (let i = 0; i < 7; i++) {
1519     const nSeg = nodeFor(`${c.id}#${i}`);
1520     touch(`${c.id}#__seg${i}`);
1521     const mid = nodeFor(`${c.id}#__seg${i}`);
1522     lines.push(`${spiceBranchName("VIL", `${c.id}_s${i}`)} ${nSeg} ${mid} 0`);
1523     lines.push(`${spiceBranchName("D", `${c.id}_s${i}`)} ${mid} ${nCom} DLED`);
1524   }
1525   } else if (c.type === "bargraph_dc10h") {
1526     const nCom = nodeFor(`${c.id}#10`);
1527     if (!declaredDiodeModels.has("DLED")) {
1528       declaredDiodeModels.add("DLED");
1529       lines.push(
1530         `.model DLED D (IS=1.05E-15 N=1.8 RS=15 BV=50 IBV=10u CJO=10p)`
1531       );
1532     }
1533     for (let i = 0; i < 10; i++) {
1534       const nSeg = nodeFor(`${c.id}#${i}`);
1535       touch(`${c.id}#__seg${i}`);
1536       const mid = nodeFor(`${c.id}#__seg${i}`);
1537       lines.push(`${spiceBranchName("VIL", `${c.id}_bg${i}`)} ${nSeg} ${mid} 0`);
1538       lines.push(`${spiceBranchName("D", `${c.id}_bg${i}`)} ${mid} ${nCom} DLED`);
1539     }
1540   } else if (c.type === "npn") {
1541     const nb = nodeFor(`${c.id}#0`);
1542     const nc = nodeFor(`${c.id}#1`);
1543     const ne = nodeFor(`${c.id}#2`);
1544     const model = spiceBjtModelName(c.value);
1545     if (!declaredBjtModels.has(model)) {
1546       declaredBjtModels.add(model);
1547       const v = String(c.value || "").toLowerCase();
1548       if (v.includes("2222")) {
1549         lines.push(
1550           `.model ${model} NPN (IS=1e-14 BF=200 VAF=100 IKF=0.3 CJE=2.4p CJC=8p TF=0.5n TR=50n)`
1551         );
1552       } else {
1553         lines.push(`.model ${model} NPN`);
1554       }
1555     }
1556     lines.push(`${spiceBranchName("Q", c.id)} ${nc} ${nb} ${ne} ${model}`);
1557   } else if (c.type === "opamp") {
1558     const nPlus = nodeFor(`${c.id}#0`);
1559     const nMinus = nodeFor(`${c.id}#1`);
1560     const nOut = nodeFor(`${c.id}#2`);
1561     const comparatorMode = opampUsesComparatorModel(c, parent, components);
1562     const topo = new Map(parent);
1563     ufUnionPassiveInternals(topo, components);
1564     const outEqualsPlus = ufFind(topo, `${c.id}#2`) === ufFind(topo, `${c.id}#0`);
1565     let bOut = nOut;
1566     if (outEqualsPlus) {
1567       touch(`${c.id}#__obuf`);
1568       bOut = nodeFor(`${c.id}#__obuf`);
1569       lines.push(`${spiceBranchName("R", c.id)}_iso ${nOut} ${bOut} 1`);
1570     }
1571     lines.push(formatOpampBsourceLine(c, bOut, nPlus, nMinus, { comparatorMode }));
1572   } else if (isLogicGateComponentType(c.type)) {
1573     const inKeys = logicGateInputNodeKeys(c);
1574     const inNodes = inKeys.map(k => nodeFor(k));
1575     const outKey = logicGateOutputNodeKey(c);
1576     const nOut = nodeFor(outKey);
1577     const vhi = resolveLogicGateVhi(c, logicVhiByTerminal);
1578     const expr = logicGateBsourceExpression(c.type, inNodes, vhi);
1579     lines.push(`${spiceBranchName("B", c.id)} ${nOut} 0 V = ${expr}`);
1580   } else if (c.type === "logic_dff") {
1581     const vhi = resolveSequentialVhi(
1582       c,
1583       logicVhiByTerminal,
1584       parseLogicRail,
1585       logicVhi,
1586       logicDffInputNodeKeys(c)
1587     );
1588     const srWired = {
1589       set: (terminalWireCount.get(`${c.id}#4`) || 0) > 0,
1590       reset: (terminalWireCount.get(`${c.id}#5`) || 0) > 0,
1591     };
1592     appendLogicDffNetlist(c, nodeFor, vhi, lines, spiceBranchName, {
1593       ...deckOpts,
1594       srWired,
1595       rippleClockFromPrev: getRippleClockFromPrev(c, wires, components),

```

```

1596     });
1597     } else if (c.type === "logic_jk") {
1598         const vhi = resolveSequentialVhi(
1599             c,
1600             logicVhiByTerminal,
1601             parseLogicRail,
1602             logicVhi,
1603             logicJkInputNodeKeys(c)
1604         );
1605         const srWired = {
1606             set: (terminalWireCount.get(`${c.id}#5`) || 0) > 0,
1607             reset: (terminalWireCount.get(`${c.id}#6`) || 0) > 0,
1608         };
1609         appendLogicJkNetlist(c, nodeFor, vhi, lines, spiceBranchName, {
1610             ...deckOpts,
1611             srWired,
1612             rippleClockFromPrev: getRippleClockFromPrev(c, wires, components),
1613         });
1614     } else if (isIc74ls00Type(c.type)) {
1615         const vhi = resolveIc74ls00Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
1616         appendIc74ls00Netlist(c, nodeFor, vhi, lines, spiceBranchName);
1617     } else if (isIc74ls74Type(c.type)) {
1618         const vhi = resolveIc74ls74Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
1619         appendIc74ls74Netlist(c, nodeFor, vhi, lines, spiceBranchName);
1620     } else if (isIc74hc90Type(c.type)) {
1621         const vhi = resolveIc74hc90Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
1622         appendIc74hc90Netlist(c, nodeFor, vhi, lines, spiceBranchName, deckOpts);
1623     } else if (isArduinoUnoType(c.type)) {
1624         appendArduinoUnoNetlist(c, nodeFor, lines, spiceBranchName, { i2cRepeatSec });
1625     } else if (isLogicCd4511Type(c.type)) {
1626         const vhi = resolveLogicCd4511Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
1627         appendLogicCd4511Netlist(c, nodeFor, vhi, lines, spiceBranchName, {
1628             ...deckOpts,
1629             leWired: (terminalWireCount.get(`${c.id}#4`) || 0) > 0,
1630             biWired: (terminalWireCount.get(`${c.id}#5`) || 0) > 0,
1631             ltWired: (terminalWireCount.get(`${c.id}#6`) || 0) > 0,
1632         });
1633     } else if (c.type === "vsource") {
1634         const nPlus = nodeFor(`${c.id}#0`);
1635         const nMinus = nodeFor(`${c.id}#1`);
1636         const v = parseDcVolts(c.value);
1637         lines.push(`${spiceBranchName("V", c.id)} ${nPlus} ${nMinus} DC ${v}`);
1638     } else if (c.type === "vterm") {
1639         const n0 = nodeFor(`${c.id}#0`);
1640         const v = parseDcVolts(c.value);
1641         lines.push(`${spiceBranchName("V", c.id)} ${n0} 0 DC ${v}`);
1642     } else if (c.type === "logic_state") {
1643         const n0 = nodeFor(`${c.id}#0`);
1644         const v = parseLogicStateVolts(c.value, c.logicRail);
1645         lines.push(`${spiceBranchName("V", c.id)} ${n0} 0 DC ${v}`);
1646     } else if (c.type === "vsin") {
1647         const nMinus = nodeFor(`${c.id}#0`);
1648         const nPlus = nodeFor(`${c.id}#1`);
1649         const voff = parseOffsetVolts(c.value);
1650         if (useAcForBode) {
1651             lines.push(`${spiceBranchName("V", c.id)} ${nPlus} ${nMinus} DC ${voff} AC 1`);
1652         } else {
1653             const vpk = parseSinusAmplitudeVolts(c.value);
1654             const freq = parseFreqHz(c.value);
1655             const phi = parsePhaseDeg(c.value);
1656             lines.push(`${spiceBranchName("V", c.id)} ${nPlus} ${nMinus} SIN(${voff} ${vpk} ${freq} 0 0 ${phi})`);
1657         }
1658     } else if (c.type === "vsquare") {
1659         const nMinus = nodeFor(`${c.id}#0`);
1660         const nPlus = nodeFor(`${c.id}#1`);
1661         const freq = parseFreqHz(c.value);
1662         const voff = parseOffsetVolts(c.value);
1663         const { ampPos, ampNeg } = parseSquareAmplitudes(c.value);
1664         const period = 1 / freq;
1665         const ton = period / 2;
1666         const vlow = voff - ampNeg;
1667         const vhi = voff + ampPos;
1668         lines.push(
1669             `${spiceBranchName("V", c.id)} ${nPlus} ${nMinus} PULSE(${vlow} ${vhi} 0 1n ${ton} ${period})`
1670         );
1671     } else if (c.type === "vpulse") {
1672         const n0 = nodeFor(`${c.id}#0`);
1673         const n1 = nodeFor(`${c.id}#1`);
1674         const freq = parseFreqHz(c.value);
1675         const vhi = parsePulseHighVolts(c.value);

```



```

1676     const duty = parseDutyPercent(c.value);
1677     const period = 1 / freq;
1678     const pw = period * duty / 100;
1679     lines.push(
1680         `${spiceBranchName("V", c.id)} ${n0} ${n1} PULSE(0 ${vhi} 0 1n ${formatSpiceTime(pw)} ${formatSpiceTime(period)})`
1681     );
1682 } else if (c.type === "ammeter" || isAmmeterRmsType(c.type)) {
1683     const n0 = nodeFor(`${c.id}#0`);
1684     const n1 = nodeFor(`${c.id}#1`);
1685     const kp = `${c.id}#0`;
1686     const km = `${c.id}#1`;
1687     const label = isAmmeterRmsType(c.type) ? "Ampèremètre (eff.)" : "Ampèremètre";
1688     if ((terminalWireCount.get(kp) || 0) === 0 || (terminalWireCount.get(km) || 0) === 0) {
1689         warnings.push(`${label} ${c.id} : au moins une borne n'est reliée à aucun fil (branchement en série requis).`);
1690     }
1691     if (n0 === n1) {
1692         warnings.push(`${label} ${c.id} : les deux bornes sont sur le même nœud (${n0}).`);
1693     } else {
1694         lines.push(`${spiceBranchName("VI", c.id)} ${n0} ${n1} DC 0`);
1695     }
1696 }
1697 }
1698
1699 for (const c of components) {
1700     if (c.type !== "voltmeter" && !isVoltmeterRmsType(c.type) && !isBodeAnalyzerType(c.type)) continue;
1701     const kp = `${c.id}#0`;
1702     const km = `${c.id}#1`;
1703     const label = isBodeAnalyzerType(c.type)
1704         ? "Analyse fréquentielle"
1705         : isVoltmeterRmsType(c.type)
1706         ? "Voltmètre (eff.)"
1707         : "Voltmètre";
1708     if ((terminalWireCount.get(kp) || 0) === 0 || (terminalWireCount.get(km) || 0) === 0) {
1709         warnings.push(`${label} ${c.id} : au moins une borne n'est reliée à aucun fil.`);
1710     }
1711     const n0 = nodeFor(kp);
1712     const n1 = nodeFor(km);
1713     if (n0 !== n1) {
1714         lines.push(`${spiceBranchName("RVM", c.id)} ${n0} ${n1} 1e12`);
1715     }
1716 }
1717
1718 const ohmmeters = [];
1719 for (const c of components) {
1720     if (c.type !== "ohmmeter") continue;
1721     const kp = `${c.id}#0`;
1722     const km = `${c.id}#1`;
1723     if ((terminalWireCount.get(kp) || 0) === 0 || (terminalWireCount.get(km) || 0) === 0) {
1724         warnings.push(`Ohmmètre ${c.id} : au moins une borne n'est reliée à aucun fil.`);
1725     }
1726     const np = nodeFor(kp);
1727     const nm = nodeFor(km);
1728     if (np === nm) {
1729         warnings.push(`Ohmmètre ${c.id} : les deux bornes sont sur le même nœud (${np}).`);
1730         continue;
1731     }
1732     lines.push(`${spiceBranchName("IOHM", c.id)} ${np} ${nm} DC ${OHMMETER_TEST_CURRENT_A}`);
1733     ohmmeters.push({
1734         id: c.id,
1735         nodePlus: np,
1736         nodeMinus: nm,
1737         testCurrent: OHMMETER_TEST_CURRENT_A,
1738     });
1739 }
1740 if (ohmmeters.length > 0 && (powerSrc || hasVtermPower)) {
1741     warnings.push(
1742         "Ohmmètre : la résistance affichée est mesurée avec les sources du circuit coupées (mode Ω d'un multimètre).",
1743     );
1744 }
1745
1746 const oscilloscopes = [];
1747 for (const c of components) {
1748     if (!isOscilloscopeType(c.type)) continue;
1749     const kCh1 = `${c.id}#0`;
1750     const kCh2 = `${c.id}#1`;
1751     const kGnd = `${c.id}#2`;
1752     if (
1753         (terminalWireCount.get(kCh1) || 0) === 0 ||
1754         (terminalWireCount.get(kCh2) || 0) === 0 ||
1755         (terminalWireCount.get(kGnd) || 0) === 0

```

```

1756     ) {
1757         warnings.push(
1758             `Oscilloscope ${c.id} : reliez CH1, CH2 et la borne commune (masse) au circuit.`
1759         );
1760     }
1761     const nCh1 = nodeFor(kCh1);
1762     const nCh2 = nodeFor(kCh2);
1763     const nGnd = nodeFor(kGnd);
1764     if (nCh1 !== nGnd) {
1765         lines.push(`${spiceBranchName("RVM", `${c.id}_ch1`)}` ${nCh1} ${nGnd} 1e12`);
1766     }
1767     if (nCh2 !== nGnd) {
1768         lines.push(`${spiceBranchName("RVM", `${c.id}_ch2`)}` ${nCh2} ${nGnd} 1e12`);
1769     }
1770     oscilloscopes.push({
1771         id: c.id,
1772         ch1NodePlus: nCh1,
1773         ch1NodeMinus: nGnd,
1774         ch2NodePlus: nCh2,
1775         ch2NodeMinus: nGnd,
1776     });
1777 }
1778
1779 const bodeAnalyzers = [];
1780 const vsinForBode = components.find((c) => c.type === "vsin");
1781 for (const c of bodeAnalyzerComponents) {
1782     const kp = `${c.id}#0`;
1783     const km = `${c.id}#1`;
1784     if ((terminalWireCount.get(kp) || 0) === 0 || (terminalWireCount.get(km) || 0) === 0) {
1785         warnings.push(
1786             `Analyse fréquentielle ${c.id} : reliez les deux bornes (+ et -) au circuit.`
1787         );
1788     }
1789     const n0 = nodeFor(kp);
1790     const n1 = nodeFor(km);
1791     if (n0 === n1) {
1792         warnings.push(
1793             `Analyse fréquentielle ${c.id} : les deux bornes sont sur le même nœud (${n0}).`
1794         );
1795     }
1796     const n0IsGnd = n0 === "0" || isNodeLikelyLogicLow(n0, components, nodeFor);
1797     const n1IsGnd = n1 === "0" || isNodeLikelyLogicLow(n1, components, nodeFor);
1798     if (!n0IsGnd && !n1IsGnd) {
1799         warnings.push(
1800             `Analyse fréquentielle ${c.id} : reliez la borne - à la masse (symbole GND relié au même fil). Sans référence masse, la courb
1801 e peut ressembler à un passe-haut.`
1802         );
1803     }
1804     let nodeOutPlus;
1805     let nodeOutMinus;
1806     if (n1IsGnd && !n0IsGnd) {
1807         nodeOutPlus = n0;
1808         nodeOutMinus = n1;
1809     } else if (n0IsGnd && !n1IsGnd) {
1810         nodeOutPlus = n1;
1811         nodeOutMinus = n0;
1812     } else {
1813         nodeOutPlus = n1;
1814         nodeOutMinus = n0;
1815     }
1816     let nodeInPlus = "0";
1817     let nodeInMinus = "0";
1818     if (vsinForBode) {
1819         const sin0 = nodeFor(`${vsinForBode.id}#0`);
1820         const sin1 = nodeFor(`${vsinForBode.id}#1`);
1821         const sin0Gnd = sin0 === "0" || isNodeLikelyLogicLow(sin0, components, nodeFor);
1822         const sin1Gnd = sin1 === "0" || isNodeLikelyLogicLow(sin1, components, nodeFor);
1823         if (sin1Gnd && !sin0Gnd) {
1824             nodeInPlus = sin0;
1825             nodeInMinus = sin1;
1826         } else if (sin0Gnd && !sin1Gnd) {
1827             nodeInPlus = sin1;
1828             nodeInMinus = sin0;
1829         } else {
1830             nodeInPlus = sin1;
1831             nodeInMinus = sin0;
1832         }
1833     }
1834     bodeAnalyzers.push({
1835         id: c.id,

```

```

1835         nodeOutPlus,
1836         nodeOutMinus,
1837         nodeInPlus,
1838         nodeInMinus,
1839     });
1840 }
1841
1842 for (const c of components) {
1843     if (!isOpampType(c.type)) continue;
1844     const kOut = `${c.id}#2`;
1845     const kPlus = `${c.id}#0`;
1846     const kMinus = `${c.id}#1`;
1847     if ((terminalWireCount.get(kOut) || 0) === 0) {
1848         warnings.push(
1849             `AOP ${c.id} : reliez la sortie à une charge (résistance vers la masse) ou à une boucle de rétroaction – indispensable en com
parateur.`
1850         );
1851     }
1852     if ((terminalWireCount.get(kPlus) || 0) === 0 || (terminalWireCount.get(kMinus) || 0) === 0) {
1853         warnings.push(`AOP ${c.id} : les entrées + et - doivent être câblées.`);
1854     }
1855 }
1856 const hasOpamp = components.some(c => isOpampType(c.type));
1857 const acSources = components.filter(c => c.type === "vsin" || c.type === "vsquare" || c.type === "vpulse");
1858 const hasPulseSource = components.some(c => c.type === "vpulse");
1859
1860 const voltmetersRms = [];
1861 for (const c of components) {
1862     if (!isVoltmeterRmsType(c.type)) continue;
1863     const kp = `${c.id}#0`;
1864     const km = `${c.id}#1`;
1865     const np = nodeFor(kp);
1866     const nm = nodeFor(km);
1867     if (np === nm) {
1868         warnings.push(
1869             `Voltmètre (eff.) ${c.id} : les deux bornes sont sur le même nœud (${np}). Vérifiez le câblage.`
1870         );
1871     }
1872     voltmetersRms.push({ id: c.id, nodePlus: np, nodeMinus: np });
1873 }
1874
1875 const ammetersRms = [];
1876 for (const c of components) {
1877     if (!isAmmeterRmsType(c.type)) continue;
1878     const np = nodeFor(`${c.id}#0`);
1879     const nm = nodeFor(`${c.id}#1`);
1880     ammetersRms.push({
1881         id: c.id,
1882         branch: spiceBranchName("VI", c.id),
1883         nodePlus: np,
1884         nodeMinus: nm,
1885     });
1886 }
1887
1888 const hasLeds = components.some(c => isLedType(c.type));
1889 const hasSpeakers = components.some(c => isSpeakerType(c.type));
1890 const hasLogicGates = components.some(
1891     c => isLogicGateComponentType(c.type) || isLogicDigitalSimType(c.type)
1892 );
1893 const hasArduinoPulse = components.some(
1894     (c) =>
1895         isArduinoUnoType(c.type) &&
1896         ((c.pinPulses && Object.keys(c.pinPulses).length > 0) ||
1897             (Array.isArray(c.pinPhases) && c.pinPhases.length >= 2))
1898 );
1899 const hasI2cBus = components.some((c) => isArduinoUnoType(c.type) && c.i2cBus?.active);
1900 let useTran =
1901     hasPulseSource ||
1902     hasArduinoPulse ||
1903     hasI2cBus ||
1904     hasLogicGates ||
1905     hasAutonomousLcOscillator ||
1906     (acSources.length > 0 &&
1907         (oscilloscopes.length > 0 ||
1908             voltmetersRms.length > 0 ||
1909             ammetersRms.length > 0 ||
1910             hasLeds ||
1911             hasSpeakers));
1912 if (isArduinoIdealOnlyCircuit(components)) {
1913     useTran = false;

```

```

1914     warnings.push(
1915         "Arduino (GPIO animés) : pas de transitoire SPICE – bargraph / LED / 7 segments animés en temps réel (plus rapide).";
1916     );
1917 }
1918 if (hasSpeakers && acSources.length === 0) {
1919     warnings.push(
1920         "Haut-parleur : ajoutez un générateur sinus ou carré pour activer la restitution sonore.";
1921     );
1922 }
1923 if (hasSpeakers && useAcForBode) {
1924     warnings.push(
1925         "Haut-parleur inactif en mode analyse fréquentielle (Bode) – retirez l'analyseur Bode pour entendre le signal.";
1926     );
1927 }
1928 if (useAcForBode && useTran) {
1929     warnings.push(
1930         "Analyse fréquentielle active : l'oscilloscope et les mesures temporelles sont ignorés pour cette simulation.";
1931     );
1932     useTran = false;
1933 }
1934 if (hasOpamp && acSources.length > 0 && oscilloscopes.length === 0) {
1935     warnings.push(
1936         "Comparateur / hystérésis dynamique : ajoutez un oscilloscope sur la sortie de l'AOP et un générateur sinus (ou carré) sur l'entrée pour voir les seuils en simulation transitoire.";
1937     );
1938 }
1939 if (hasOpamp && acSources.length === 0 && oscilloscopes.length > 0 && !hasAutonomousLcOscillator) {
1940     warnings.push(
1941         "AOP en régime DC (.op) : seuils de comparateur visibles au voltmètre ; pour l'hystérésis en fonction du temps, excitez l'entrée avec un générateur sinus.";
1942     );
1943 }
1944 let lcKickPeriodSec = null;
1945 if (hasAutonomousLcOscillator) {
1946     const fHz = estimateParallelLcResonantHz(components);
1947     const periodMs = fHz > 0 ? (1000 / fHz).toFixed(2) : "?";
1948     if (fHz > 0) lcKickPeriodSec = 1 / fHz;
1949     warnings.push(
1950         `Oscillateur LC autonome : fréquence LC théorique = ${fHz > 0 ? `${Math.round(fHz)} Hz` : "?"} (période = ${periodMs} ms). SPICE est lancé en transitoire réel avec impulsion d'excitation périodique de test.`
1951     );
1952 }
1953 if ((voltmetersRms.length > 0 || ammetersRms.length > 0) && !useTran) {
1954     warnings.push(
1955         "Voltmètre / ampèremètre efficace : ajoutez un générateur sinus ou carré (et un oscilloscope si besoin de .tran) pour activer l'analyse transitoire et la valeur efficace.";
1956     );
1957 }
1958 const scopesTranMeta = [];
1959 const ledsTranMeta = [];
1960 const seg7TranMeta = [];
1961 const bargraphTranMeta = [];
1962 const logicGatesTranMeta = [];
1963 const metersTranMeta = { voltmetersRms: [], ammetersRms: [], voltmeters: [], ammeters: [], ohmmeters: [], speakers: [] };
1964 let analysisTran = false;
1965 let analysisAc = false;
1966 const bodeAcMeta = [];
1967
1968 lines.push("");
1969
1970 const voltmeters = [];
1971 for (const c of components) {
1972     if (c.type !== "voltmeter") continue;
1973     const nMinus = nodeFor(`${c.id}#0`);
1974     const nPlus = nodeFor(`${c.id}#1`);
1975     if (nPlus === nMinus) {
1976         warnings.push(
1977             `Voltmètre ${c.id} : les deux bornes sont sur le même nœud (${nPlus}). Vérifiez le câblage des deux fils.`
1978         );
1979     }
1980     voltmeters.push({
1981         id: c.id,
1982         nodePlus: nPlus,
1983         nodeMinus: nMinus,
1984     });
1985 }
1986
1987 const speakers = [];
1988 for (const c of components) {
1989     if (!isSpeakerType(c.type)) continue;
1990     const nMinus = nodeFor(`${c.id}#0`);

```

```

1991     const nPlus = nodeFor(`${c.id}#1`);
1992     speakers.push({
1993         id: c.id,
1994         nodePlus: nPlus,
1995         nodeMinus: nMinus,
1996     });
1997 }
1998
1999 const ammeters = [];
2000 for (const c of components) {
2001     if (c.type !== "ammeter") continue;
2002     const np = nodeFor(`${c.id}#0`);
2003     const nm = nodeFor(`${c.id}#1`);
2004     const branch = spiceBranchName("VI", c.id);
2005     ammeters.push({
2006         id: c.id,
2007         branch,
2008         nodePlus: np,
2009         nodeMinus: nm,
2010     });
2011 }
2012
2013 if (useAcForBode) {
2014     analysisAc = true;
2015     if (lines[0]) lines[0] = "* Circuit Designer - netlist SPICE (.ac)";
2016     const acFmin = 1;
2017     const acFmax = 1e6;
2018     const acPtsPerDec = 50;
2019     lines.push(`.ac dec ${acPtsPerDec} ${acFmin} ${acFmax}`);
2020     lines.push(".control");
2021     lines.push("run");
2022     const wrVars = ["frequency"];
2023     function pushAcNodeWr(nodeName) {
2024         if (!nodeName || nodeName === "0") {
2025             return { dbCol: null, phCol: null, isGnd: true };
2026         }
2027         wrVars.push(`vdb(${nodeName})`);
2028         const dbCol = wrVars.length * 2;
2029         wrVars.push(`vp(${nodeName})`);
2030         const phCol = wrVars.length * 2;
2031         return { dbCol, phCol, isGnd: false, nodeName };
2032     }
2033     for (const ba of bodeAnalyzers) {
2034         const meta = {
2035             id: ba.id,
2036             freqCol: 0,
2037             fMin: acFmin,
2038             fMax: acFmax,
2039             outPlus: pushAcNodeWr(ba.nodeOutPlus),
2040             outMinus: pushAcNodeWr(ba.nodeOutMinus),
2041             inPlus: pushAcNodeWr(ba.nodeInPlus),
2042             inMinus: pushAcNodeWr(ba.nodeInMinus),
2043         };
2044         bodeAcMeta.push(meta);
2045     }
2046     lines.push(`wdata __AC_WAVE_PATH__ ${wrVars.join(" ")}`);
2047     warnings.push(
2048         `Analyse fréquentielle : balayage ${acFmin} Hz - ${acFmax} >= 1e6 ? "1 MHz" : acFmax + " Hz" (diagramme de Bode, gain en dB).`
2049     );
2050     if (oscilloscopes.length > 0) {
2051         warnings.push(
2052             "Oscilloscope présent : non alimenté en mode analyse fréquentielle – retirez l'analyseur Bode ou l'oscilloscope pour une simulation temporelle."
2053         );
2054     }
2055 } else if (useTran) {
2056     const hc90Mod60 = detectHc90Mod60FromGraphicalState(components, wires);
2057     const tranTiming = computeTranTiming(components, { ...deckOpts, hc90Mod60 });
2058     const { tstepStr, tstopStr, slowClock, clockPeriodSec } = tranTiming;
2059     analysisTran = true;
2060     const hc90InCircuit = components.some((c) => isIc74hc90Type(c.type));
2061     if (slowClock && hc90InCircuit) {
2062         const periodLabel =
2063             clockPeriodSec >= 3600
2064                 ? `${(clockPeriodSec / 3600).toFixed(clockPeriodSec % 3600 === 0 ? 0 : 2)} h`
2065                 : clockPeriodSec >= 60
2066                     ? `${(clockPeriodSec / 60).toFixed(clockPeriodSec % 60 === 0 ? 0 : 2)} min`
2067                     : `${clockPeriodSec.toFixed(2)} s`;
2068         warnings.push(
2069             `Horloge lente (${periodLabel} par impulsion) : SPICE simule ${tstopStr} s de transitoire.`

```

```

2070     );
2071 }
2072 if (lines[0]) lines[0] = "* Circuit Designer - netlist SPICE (.tran)";
2073 if (hasAutonomousLcOscillator && oscillatorTankNode) {
2074     const kickPeriod = lcKickPeriodSec && lcKickPeriodSec > 0 ? lcKickPeriodSec : 1e-3;
2075     const kickPw = Math.max(1e-6, Math.min(100e-6, kickPeriod * 0.05));
2076     lines.push(
2077         `${spiceBranchName("V", "lckick")} ${oscillatorTankNode} 0 PULSE(0 3 0 1n 1n ${formatSpiceTime(kickPw)} ${formatSpiceTime(kic
kPeriod)})`
2078     );
2079 }
2080 // Bascules internes (D, JK, 74HC90 ÷2÷5, 74LS74) : boucles logiques raides →
2081 // options de convergence (method=gear). Ne PAS forcer UIC ici : les ponts
2082 // numériques XSPICE (d_dff/d_jkff) gèrent mal les conditions initiales et
2083 // produiraient une séquence fausse.
2084 const hasInternalFlipFlops = components.some(
2085     (c) =>
2086         isLogicSequentialType(c.type) ||
2087         isIc74hc90Type(c.type) ||
2088         isIc74ls74Type(c.type)
2089 );
2090 if (hasInternalFlipFlops) {
2091     lines.push(".options method=gear reltol=1e-3 abstol=1e-9 gmin=1e-15 trtol=50");
2092 } else if (hasAutonomousLcOscillator) {
2093     lines.push(".options method=gear reltol=1e-3 abstol=1e-12 gmin=1e-12 trtol=10");
2094 }
2095 lines.push(
2096     hasAutonomousLcOscillator
2097     ? `.tran ${tstepStr} ${tstopStr} UIC`
2098     : `.tran ${tstepStr} ${tstopStr}`
2099 );
2100 lines.push(".control");
2101 if (hasInternalFlipFlops) {
2102     lines.push("set method=gear");
2103 }
2104 lines.push("set wr_singlescale");
2105 lines.push(
2106     hasAutonomousLcOscillator
2107     ? `tran ${tstepStr} ${tstopStr} UIC`
2108     : `tran ${tstepStr} ${tstopStr}`
2109 );
2110
2111 const wrVars = ["time"];
2112 const nodeCol = new Map();
2113 const currentCol = new Map();
2114 function isSpiceGndNode(nodeName) {
2115     return String(nodeName || "").trim() === "0";
2116 }
2117 function addWrNode(nodeName) {
2118     const n = String(nodeName || "").trim();
2119     // ngspice n'expose pas v(0) – la masse vaut 0 V par définition.
2120     if (!n || isSpiceGndNode(n) || nodeCol.has(n)) return;
2121     nodeCol.set(n, wrVars.length);
2122     wrVars.push(`v(${n})`);
2123 }
2124 function addWrCurrent(branchName) {
2125     const b = String(branchName || "").trim();
2126     if (!b || currentCol.has(b)) return;
2127     currentCol.set(b, wrVars.length);
2128     wrVars.push(`i(${b})`);
2129 }
2130 function channelWrMeta(plusNode, minusNode) {
2131     const plusIsGnd = isSpiceGndNode(plusNode);
2132     const minusIsGnd = isSpiceGndNode(minusNode);
2133     const plusWrIndex = plusIsGnd ? null : nodeCol.get(plusNode);
2134     const minusWrIndex = minusIsGnd ? null : nodeCol.get(minusNode);
2135     return {
2136         plusNode,
2137         minusNode,
2138         wrIndex: plusWrIndex,
2139         minusWrIndex,
2140         minusIsGnd,
2141         plusIsGnd,
2142     };
2143 }
2144 for (const osc of oscilloscopes) {
2145     addWrNode(osc.ch1NodePlus);
2146     addWrNode(osc.ch1NodeMinus);
2147     addWrNode(osc.ch2NodePlus);
2148     addWrNode(osc.ch2NodeMinus);

```

```

2149         scopesTranMeta.push({
2150             id: osc.id,
2151             timeCol: 0,
2152             wrVarCount: wrVars.length,
2153             ch1: channelWrMeta(osc.ch1NodePlus, osc.ch1NodeMinus),
2154             ch2: channelWrMeta(osc.ch2NodePlus, osc.ch2NodeMinus),
2155         });
2156     }
2157     for (const vm of voltmetersRms) {
2158         addWrNode(vm.nodePlus);
2159         addWrNode(vm.nodeMinus);
2160         const ch = channelWrMeta(vm.nodePlus, vm.nodeMinus);
2161         metersTranMeta.voltmetersRms.push({
2162             id: vm.id,
2163             timeCol: 0,
2164             wrVarCount: wrVars.length,
2165             nodePlus: vm.nodePlus,
2166             nodeMinus: vm.nodeMinus,
2167             channel: ch,
2168         });
2169     }
2170     for (const am of ammetersRms) {
2171         addWrCurrent(am.branch);
2172         metersTranMeta.ammetersRms.push({
2173             id: am.id,
2174             timeCol: 0,
2175             wrVarCount: wrVars.length,
2176             branch: am.branch,
2177             currentWrIndex: currentCol.get(am.branch),
2178         });
2179     }
2180     for (const vm of voltmeters) {
2181         addWrNode(vm.nodePlus);
2182         addWrNode(vm.nodeMinus);
2183         metersTranMeta.voltmeters.push({
2184             id: vm.id,
2185             timeCol: 0,
2186             wrVarCount: wrVars.length,
2187             nodePlus: vm.nodePlus,
2188             nodeMinus: vm.nodeMinus,
2189             channel: channelWrMeta(vm.nodePlus, vm.nodeMinus),
2190         });
2191     }
2192     for (const sp of speakers) {
2193         addWrNode(sp.nodePlus);
2194         addWrNode(sp.nodeMinus);
2195         metersTranMeta.speakers.push({
2196             id: sp.id,
2197             timeCol: 0,
2198             wrVarCount: wrVars.length,
2199             nodePlus: sp.nodePlus,
2200             nodeMinus: sp.nodeMinus,
2201             channel: channelWrMeta(sp.nodePlus, sp.nodeMinus),
2202         });
2203     }
2204     for (const am of ammeters) {
2205         addWrCurrent(am.branch);
2206         metersTranMeta.ammeters.push({
2207             id: am.id,
2208             timeCol: 0,
2209             wrVarCount: wrVars.length,
2210             branch: am.branch,
2211             currentWrIndex: currentCol.get(am.branch),
2212             nodePlus: am.nodePlus,
2213             nodeMinus: am.nodeMinus,
2214         });
2215     }
2216     for (const om of ohmmeters) {
2217         addWrNode(om.nodePlus);
2218         addWrNode(om.nodeMinus);
2219         metersTranMeta.ohmmeters.push({
2220             id: om.id,
2221             timeCol: 0,
2222             wrVarCount: wrVars.length,
2223             nodePlus: om.nodePlus,
2224             nodeMinus: om.nodeMinus,
2225             testCurrent: om.testCurrent,
2226             channel: channelWrMeta(om.nodePlus, om.nodeMinus),
2227         });
2228     }

```

```

2229     for (const c of components) {
2230       if (!isLogicGateComponentType(c.type) && !isLogicDigitalSimType(c.type) && !isArduinoUnoType(c.type)) continue;
2231       let outKeys = [];
2232       if (isLogicGateComponentType(c.type)) outKeys = [logicGateOutputNodeKey(c)];
2233       else if (c.type === "logic_dff") {
2234         outKeys = logicDffDAndQbarShareNode(c, nodeFor)
2235           ? [logicDffOutputNodeKey(c)]
2236           : logicDffOutputNodeKeys(c);
2237       } else if (c.type === "logic_jk") outKeys = logicJkOutputNodeKeys(c);
2238       else if (isIc74ls00Type(c.type)) {
2239         outKeys = ic74ls00NandGates().map(g => `${c.id}#${g.y}`);
2240       } else if (isIc74ls74Type(c.type)) {
2241         outKeys = ic74ls74DffSlices().map(sl => `${c.id}#${sl.q}`);
2242       } else if (isIc74hc90Type(c.type)) {
2243         outKeys = ic74hc90OutputNodeKeys(c);
2244       } else if (isLogicCd4511Type(c.type)) {
2245         outKeys = cd4511OutputNodeKeys(c);
2246       } else if (isArduinoUnoType(c.type)) {
2247         outKeys = arduinoUnoDigitalPinIndices().map((i) => `${c.id}#${i}`);
2248       }
2249       let vhiTran = 5;
2250       if (isLogicGateComponentType(c.type)) {
2251         vhiTran = resolveLogicGateVhi(c, logicVhiByTerminal);
2252       } else if (c.type === "logic_dff") {
2253         vhiTran = resolveSequentialVhi(
2254           c,
2255           logicVhiByTerminal,
2256           parseLogicRail,
2257           logicVhi,
2258           logicDffInputNodeKeys(c)
2259         );
2260       } else if (c.type === "logic_jk") {
2261         vhiTran = resolveSequentialVhi(
2262           c,
2263           logicVhiByTerminal,
2264           parseLogicRail,
2265           logicVhi,
2266           logicJkInputNodeKeys(c)
2267         );
2268       } else if (isIc74ls00Type(c.type)) {
2269         vhiTran = resolveIc74ls00Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2270       } else if (isIc74ls74Type(c.type)) {
2271         vhiTran = resolveIc74ls74Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2272       } else if (isIc74hc90Type(c.type)) {
2273         vhiTran = resolveIc74hc90Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2274       } else if (isLogicCd4511Type(c.type)) {
2275         vhiTran = resolveLogicCd4511Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2276       } else if (isArduinoUnoType(c.type)) {
2277         vhiTran = 5;
2278       }
2279       for (const outKey of outKeys) {
2280         const nOut = nodeFor(outKey);
2281         addWrNode(nOut);
2282         let icOutId = c.id;
2283         if (isIc74ls00Type(c.type)) {
2284           icOutId = `${c.id}_Y${outKeys.indexOf(outKey) + 1}`;
2285         } else if (isIc74ls74Type(c.type)) {
2286           icOutId = `${c.id}_Q${outKeys.indexOf(outKey) + 1}`;
2287         } else if (isIc74hc90Type(c.type)) {
2288           const qLabels = ["Q0", "Q1", "Q2", "Q3"];
2289           const idx = outKeys.indexOf(outKey);
2290           if (idx >= 0 && idx < qLabels.length) icOutId = `${c.id}_${qLabels[idx]}`;
2291         } else if (c.type === "logic_dff") {
2292           icOutId =
2293             outKey === logicDffOutputNodeKey(c) ? `${c.id}_Q` : `${c.id}_Qbar`;
2294         } else if (c.type === "logic_jk") {
2295           icOutId =
2296             outKey === logicJkOutputNodeKey(c) ? `${c.id}_Q` : `${c.id}_Qbar`;
2297         } else if (isLogicCd4511Type(c.type)) {
2298           const segNames = ["a", "b", "c", "d", "e", "f", "g"];
2299           const idx = outKeys.indexOf(outKey);
2300           if (idx >= 0 && idx < segNames.length) icOutId = `${c.id}_${segNames[idx]}`;
2301         } else if (isArduinoUnoType(c.type)) {
2302           const pinIdx = Number(String(outKey).split("#")[1]);
2303           if (Number.isFinite(pinIdx)) icOutId = `${c.id}_${arduinoUnoDigitalPinName(pinIdx)}`;
2304         }
2305         logicGatesTranMeta.push({
2306           id: icOutId,
2307           timeCol: 0,
2308           wrVarCount: wrVars.length,

```



```

2309         nodeOut: nOut,
2310         wrIndex: nodeCol.get(nOut),
2311         vhi: vhiTran,
2312         vth: vhiTran / 2,
2313     });
2314 }
2315 if (isLogicGateComponentType(c.type)) {
2316     for (const k of logicGateInputNodeKeys(c)) addWrNode(nodeFor(k));
2317 } else if (c.type === "logic_dff") {
2318     for (const k of logicDffInputNodeKeys(c)) addWrNode(nodeFor(k));
2319 } else if (c.type === "logic_jk") {
2320     for (const k of logicJkInputNodeKeys(c)) addWrNode(nodeFor(k));
2321 }
2322 }
2323 for (const c of components) {
2324     if (!isLedType(c.type)) continue;
2325     const branch = spiceBranchName("VIL", c.id);
2326     addWrCurrent(branch);
2327     ledsTranMeta.push({
2328         id: c.id,
2329         timeCol: 0,
2330         branch,
2331         currentWrIndex: currentCol.get(branch),
2332         nodePlus: nodeFor(`${c.id}#0`),
2333         nodeMinus: nodeFor(`${c.id}#1`),
2334     });
2335 }
2336 const seg7TranMetaLocal = [];
2337 for (const c of components) {
2338     if (c.type !== "seg7") continue;
2339     const segWr = [];
2340     for (let i = 0; i < 7; i++) {
2341         const n = nodeFor(`${c.id}#${i}`);
2342         addWrNode(n);
2343         segWr.push(nodeCol.get(n));
2344     }
2345     const nCom = nodeFor(`${c.id}#7`);
2346     addWrNode(nCom);
2347     seg7TranMetaLocal.push({
2348         id: c.id,
2349         timeCol: 0,
2350         segmentWrIndex: segWr,
2351         commonWrIndex: nodeCol.get(nCom),
2352     });
2353 }
2354 seg7TranMeta.push(...seg7TranMetaLocal);
2355 const bargraphTranMetaLocal = [];
2356 for (const c of components) {
2357     if (c.type !== "bargraph_dc10h") continue;
2358     const segWr = [];
2359     for (let i = 0; i < 10; i++) {
2360         const n = nodeFor(`${c.id}#${i}`);
2361         addWrNode(n);
2362         segWr.push(nodeCol.get(n));
2363     }
2364     const nCom = nodeFor(`${c.id}#10`);
2365     addWrNode(nCom);
2366     bargraphTranMetaLocal.push({
2367         id: c.id,
2368         timeCol: 0,
2369         segmentWrIndex: segWr,
2370         commonWrIndex: nodeCol.get(nCom),
2371     });
2372 }
2373 bargraphTranMeta.push(...bargraphTranMetaLocal);
2374 const finalWrVarCount = wrVars.length;
2375 for (const m of ledsTranMeta) {
2376     m.wrVarCount = finalWrVarCount;
2377     if (m.branch) m.currentWrIndex = currentCol.get(m.branch);
2378 }
2379 for (const m of logicGatesTranMeta) {
2380     m.wrVarCount = finalWrVarCount;
2381     if (m.nodeOut) m.wrIndex = nodeCol.get(m.nodeOut);
2382 }
2383 for (const m of scopesTranMeta) {
2384     const osc = oscilloscopes.find((o) => o.id === m.id);
2385     if (osc) {
2386         m.ch1 = channelWrMeta(osc.ch1NodePlus, osc.ch1NodeMinus);
2387         m.ch2 = channelWrMeta(osc.ch2NodePlus, osc.ch2NodeMinus);
2388     }

```

```

2389         m.wrVarCount = finalWrVarCount;
2390     }
2391     for (const m of seg7TranMeta) {
2392         m.wrVarCount = finalWrVarCount;
2393     }
2394     for (const m of bargraphTranMeta) {
2395         m.wrVarCount = finalWrVarCount;
2396     }
2397     for (const vm of metersTranMeta.voltmetersRms) {
2398         const def = voltmetersRms.find((v) => v.id === vm.id);
2399         if (def) vm.channel = channelWrMeta(def.nodePlus, def.nodeMinus);
2400         vm.wrVarCount = finalWrVarCount;
2401     }
2402     for (const m of metersTranMeta.ammetersRms) {
2403         m.wrVarCount = finalWrVarCount;
2404         if (m.branch) m.currentWrIndex = currentCol.get(m.branch);
2405     }
2406     for (const vm of metersTranMeta.voltmeters) {
2407         const def = voltmeters.find((v) => v.id === vm.id);
2408         if (def) vm.channel = channelWrMeta(def.nodePlus, def.nodeMinus);
2409         vm.wrVarCount = finalWrVarCount;
2410     }
2411     for (const m of metersTranMeta.ammeters) {
2412         m.wrVarCount = finalWrVarCount;
2413         if (m.branch) m.currentWrIndex = currentCol.get(m.branch);
2414     }
2415     for (const om of metersTranMeta.ohmmeters) {
2416         const def = ohmmeters.find((o) => o.id === om.id);
2417         if (def) om.channel = channelWrMeta(def.nodePlus, def.nodeMinus);
2418         om.wrVarCount = finalWrVarCount;
2419     }
2420     lines.push(`wdata __TRAN_WAVE_PATH__ ${wrVars.join(" ")}`);
2421     if (oscilloscopes.length > 0) {
2422         warnings.push(
2423             `Oscilloscope : analyse transitoire (${tstopStr}) – les courbes s’ouvrent dans la fenêtre dédiée.`
2424         );
2425     } else if (hasLeds || hasLogicGates) {
2426         warnings.push(
2427             `Signal alternatif : analyse transitoire (${tstopStr}) pour l’animation des LED et le fonctionnement des portes logiques.`
2428         );
2429     }
2430     if (voltmetersRms.length > 0) {
2431         warnings.push(
2432             `Voltmètre(s) efficace : Vrms calculée sur ${tstopStr} (générateur alternatif requis).`
2433         );
2434     }
2435     if (ammetersRms.length > 0) {
2436         warnings.push(
2437             `Ampèremètre(s) efficace : Arms calculé sur ${tstopStr} (générateur alternatif requis).`
2438         );
2439     }
2440     if (voltmetersRms.length > 0 && acSources.length === 0) {
2441         warnings.push(
2442             "Voltmètre (eff.) : ajoutez un générateur sinus ou carré pour une mesure Vrms non nulle."
2443         );
2444     }
2445     if (ammetersRms.length > 0 && acSources.length === 0) {
2446         warnings.push(
2447             "Ampèremètre (eff.) : ajoutez un générateur sinus ou carré pour une mesure Arms non nulle."
2448         );
2449     }
2450 } else {
2451     lines[0] = "* Circuit Designer - netlist SPICE (.op)";
2452     lines.push(".op");
2453     lines.push(".control");
2454     lines.push("op");
2455
2456     for (const c of components) {
2457         if (c.type !== "voltmeter") continue;
2458         const np = nodeFor(`${c.id}#0`);
2459         const nm = nodeFor(`${c.id}#1`);
2460         lines.push(`echo @@VM:${c.id}@@`);
2461         if (np !== "0") lines.push(`print v(${np})`);
2462         if (nm !== "0") lines.push(`print v(${nm})`);
2463     }
2464     for (const c of components) {
2465         if (c.type !== "ammeter") continue;
2466         const branch = spiceBranchName("VI", c.id);
2467         lines.push(`echo @@AM:${c.id}@@`);
2468         lines.push(`print i(${branch})`);

```

```

2469     }
2470     for (const c of components) {
2471         if (!isLedType(c.type)) continue;
2472         const branch = spiceBranchName("VIL", c.id);
2473         const np = nodeFor(`${c.id}#0`);
2474         const nm = nodeFor(`${c.id}#1`);
2475         lines.push(`echo @@LD:${c.id}@@`);
2476         lines.push(`print i${branch}`);
2477         if (np !== "0") lines.push(`print v${np}`);
2478         if (nm !== "0") lines.push(`print v${nm}`);
2479     }
2480     for (const c of components) {
2481         if (!isLogicGateComponentType(c.type)) continue;
2482         const nOut = nodeFor(logicGateOutputNodeKey(c));
2483         lines.push(`echo @@LG:${c.id}@@`);
2484         if (nOut !== "0") lines.push(`print v${nOut}`);
2485         for (const k of logicGateInputNodeKeys(c)) {
2486             const n = nodeFor(k);
2487             if (n !== "0") lines.push(`print v${n}`);
2488         }
2489     }
2490     for (const om of ohmmeters) {
2491         lines.push(`echo @@OH:${om.id}@@`);
2492         if (om.nodePlus !== "0") lines.push(`print v${om.nodePlus}`);
2493         if (om.nodeMinus !== "0") lines.push(`print v${om.nodeMinus}`);
2494     }
2495     for (const osc of oscilloscopes) {
2496         lines.push(`echo @@SC:${osc.id}:CH1@@`);
2497         if (osc.ch1NodePlus !== "0") lines.push(`print v${osc.ch1NodePlus}`);
2498         if (osc.ch1NodeMinus !== "0") lines.push(`print v${osc.ch1NodeMinus}`);
2499         lines.push(`echo @@SC:${osc.id}:CH2@@`);
2500         if (osc.ch2NodePlus !== "0") lines.push(`print v${osc.ch2NodePlus}`);
2501         if (osc.ch2NodeMinus !== "0") lines.push(`print v${osc.ch2NodeMinus}`);
2502     }
2503     for (const c of components) {
2504         if (c.type !== "seg7") continue;
2505         const nodes = new Set();
2506         for (let i = 0; i < 7; i++) {
2507             const n = nodeFor(`${c.id}#${i}`);
2508             if (n !== "0") nodes.add(n);
2509         }
2510         const nCom = nodeFor(`${c.id}#7`);
2511         if (nCom !== "0") nodes.add(nCom);
2512         lines.push(`echo @@S7:${c.id}@@`);
2513         for (const n of nodes) lines.push(`print v${n}`);
2514     }
2515     for (const c of components) {
2516         if (c.type !== "bargraph_dc10h") continue;
2517         const nodes = new Set();
2518         for (let i = 0; i < 10; i++) {
2519             const n = nodeFor(`${c.id}#${i}`);
2520             if (n !== "0") nodes.add(n);
2521         }
2522         const nCom = nodeFor(`${c.id}#10`);
2523         if (nCom !== "0") nodes.add(nCom);
2524         lines.push(`echo @@BG:${c.id}@@`);
2525         for (const n of nodes) lines.push(`print v${n}`);
2526     }
2527     if (oscilloscopes.length > 0 && acSources.length === 0) {
2528         warnings.push(
2529             "Oscilloscope avec source DC : valeurs affichées au point de repos (.op). Pour voir des signaux alternatifs, utilisez un géné-
rateur sinus ou carré."
2530         );
2531     }
2532 }
2533
2534 lines.push(".endc");
2535 lines.push(".end");
2536
2537 for (const c of components) {
2538     if (isLogicGateComponentType(c.type)) {
2539         const labels = {
2540             logic_not: "Inverseur",
2541             logic_and: "ET",
2542             logic_or: "OU",
2543             logic_nand: "NON-ET",
2544             logic_nor: "NON-OU",
2545             logic_xor: "OU exclusif",
2546             logic_xnor: "NON-OU exclusif",
2547         };

```

```

2548     const label = labels[c.type] || c.type;
2549     const kp = logicGateInputNodeKeys(c);
2550     const unwired = kp.filter(k => (terminalWireCount.get(k) || 0) === 0);
2551     if (unwired.length > 0) {
2552         warnings.push(`${label} ${c.id} : borne(s) non reliée(s) au circuit.`);
2553     }
2554 }
2555 if (c.type === "logic_dff") {
2556     const kp = logicDffInputNodeKeys(c);
2557     const unwired = kp.filter(k => (terminalWireCount.get(k) || 0) === 0);
2558     if (unwired.length > 0) {
2559         warnings.push(`Bascule D ${c.id} : borne(s) non reliée(s) au circuit.`);
2560     }
2561     if (logicDffDAndQbarShareNode(c, nodeFor)) {
2562         warnings.push(
2563             useLogicDffXspice(deckOpts)
2564             ? `Bascule D ${c.id} : D et /Q sur le même fil – diviseur par 2 (XSPICE : entrée D = sortie /Q numérique).`
2565             : `Bascule D ${c.id} : D et /Q sur le même fil – en simulation, à chaque front d’horloge on prend D = complément de
Q.`
2566         );
2567     }
2568 }
2569 if (c.type === "logic_jk") {
2570     const kp = logicJKInputNodeKeys(c);
2571     const unwired = kp.filter(k => (terminalWireCount.get(k) || 0) === 0);
2572     if (unwired.length > 0) {
2573         warnings.push(`Bascule JK ${c.id} : borne(s) non reliée(s) au circuit.`);
2574     }
2575 }
2576 if (isIc74ls00Type(c.type)) {
2577     const vccKey = `${c.id}#${ic74ls00VccPinIndex()}`;
2578     const gndKey = `${c.id}#6`;
2579     if ((terminalWireCount.get(vccKey) || 0) === 0) {
2580         warnings.push(`74HC00 ${c.id} : reliez VCC (broche 14) au rail d’alimentation.`);
2581     }
2582     if ((terminalWireCount.get(gndKey) || 0) === 0) {
2583         warnings.push(`74HC00 ${c.id} : reliez GND (broche 7) à la masse.`);
2584     }
2585 }
2586 if (isIc74ls74Type(c.type)) {
2587     const vccKey = `${c.id}#${ic74ls74VccPinIndex()}`;
2588     const gndKey = `${c.id}#6`;
2589     if ((terminalWireCount.get(vccKey) || 0) === 0) {
2590         warnings.push(`74LS74 ${c.id} : reliez VCC (broche 14) au rail d’alimentation.`);
2591     }
2592     if ((terminalWireCount.get(gndKey) || 0) === 0) {
2593         warnings.push(`74LS74 ${c.id} : reliez GND (broche 7) à la masse.`);
2594     }
2595 }
2596 if (isIc74hc90Type(c.type)) {
2597     const vccKey = `${c.id}#${ic74hc90VccPinIndex()}`;
2598     const gndKey = `${c.id}#${ic74hc90GndPinIndex()}`;
2599     if ((terminalWireCount.get(vccKey) || 0) === 0) {
2600         warnings.push(`74HC90 ${c.id} : reliez VCC (broche 5) au +5 V.`);
2601     }
2602     if ((terminalWireCount.get(gndKey) || 0) === 0) {
2603         warnings.push(`74HC90 ${c.id} : reliez GND (broche 10) à la masse.`);
2604     }
2605     const q0Key = `${c.id}#11`;
2606     const cp1Key = `${c.id}#0`;
2607     const q0ToCp1 = nodeFor(q0Key) === nodeFor(cp1Key);
2608     if (q0ToCp1) {
2609         warnings.push(
2610             `74HC90 ${c.id} : Q0 relié à CP1 – comptage décade 0...9 (horloge sur CP0, broche 14).`
2611         );
2612     }
2613     const cp0Key = `${c.id}#13`;
2614     const cp0Wired = (terminalWireCount.get(cp0Key) || 0) > 0;
2615     const cp1Wired = (terminalWireCount.get(cp1Key) || 0) > 0;
2616     if (!cp0Wired && cp1Wired) {
2617         warnings.push(
2618             `74HC90 ${c.id} : horloge détectée sur CP1 alors que CP0 n’est pas relié – compteur en mode /5 (0→4). Pour 0→9 : horloge
sur CP0 + Q0 relié à CP1.`
2619         );
2620     }
2621     if (cp1Wired && !q0ToCp1) {
2622         warnings.push(
2623             `74HC90 ${c.id} : CP1 n’est pas relié à Q0 – le mode décade 0→9 est invalide (comptage typiquement bloqué à 0→4). Reliez
Q0 à CP1.`
2624         );

```

```

2625     }
2626     const mr1Key = `${c.id}#1`;
2627     const mr2Key = `${c.id}#2`;
2628     const mr1High =
2629         (terminalWireCount.get(mr1Key) || 0) > 0 &&
2630         isNodeLikelyLogicHigh(nodeFor(mr1Key), components, nodeFor);
2631     const mr2High =
2632         (terminalWireCount.get(mr2Key) || 0) > 0 &&
2633         isNodeLikelyLogicHigh(nodeFor(mr2Key), components, nodeFor);
2634     if (mr1High && mr2High) {
2635         warnings.push(
2636             `74HC90 ${c.id} : MR1 et MR2 au niveau haut – compteur maintenu à 0. Reliez MR1 et MR2 à 0 V (masse).`
2637         );
2638     }
2639     if (detectHc90MrAndQ1Q3OnSameChip(c.id, components, wires)) {
2640         warnings.push(
2641             `74HC90 ${c.id} : AND(Q1,Q3) relié à MR1/MR2 – comptage erratique (reset pendant les transitions). Pour 0...9 : MR1 et MR2
à la masse. AND(Q0,Q3) sert au report vers un 2e HC90 (dizaines), pas au reset.`
2642         );
2643     }
2644     const ms1Key = `${c.id}#${IC90_PIN.MS1}`;
2645     const ms2Key = `${c.id}#${IC90_PIN.MS2}`;
2646     const ms1High =
2647         (terminalWireCount.get(ms1Key) || 0) > 0 &&
2648         isNodeLikelyLogicHigh(nodeFor(ms1Key), components, nodeFor);
2649     const ms2High =
2650         (terminalWireCount.get(ms2Key) || 0) > 0 &&
2651         isNodeLikelyLogicHigh(nodeFor(ms2Key), components, nodeFor);
2652     if (ms1High !== ms2High && (ms1High || ms2High)) {
2653         warnings.push(
2654             `74HC90 ${c.id} : MS1/MS2 asymétriques – pour le mode décade 0...9, reliez MS1 et MS2 tous deux à 0 V (masse).`
2655         );
2656     }
2657 }
2658 if (isLogicCd4511Type(c.type)) {
2659     const inp = cd4511InputNodeKeys(c);
2660     const unwired = inp.filter((k) => (terminalWireCount.get(k) || 0) === 0);
2661     if (unwired.length > 0) {
2662         warnings.push(
2663             `CD4511 ${c.id} : entrée(s) non reliée(s) (A-D, LE, BI, LT) – valeurs par défaut appliquées.`
2664         );
2665     }
2666     const biKey = `${c.id}#5`;
2667     const ltKey = `${c.id}#6`;
2668     const leKey = `${c.id}#4`;
2669     if ((terminalWireCount.get(biKey) || 0) > 0 && isNodeLikelyLogicLow(nodeFor(biKey), components, nodeFor)) {
2670         warnings.push(
2671             `CD4511 ${c.id} : BI relié à la masse (0 V) – afficheur éteint. Reliez BI au +5 V (même rail que VCC).`
2672         );
2673     }
2674     if ((terminalWireCount.get(ltKey) || 0) > 0 && isNodeLikelyLogicLow(nodeFor(ltKey), components, nodeFor)) {
2675         warnings.push(
2676             `CD4511 ${c.id} : LT relié à la masse (0 V) – mode test lampe (tous segments allumés). Reliez LT au +5 V en utilisation n
ormale.`
2677         );
2678     }
2679     if ((terminalWireCount.get(leKey) || 0) > 0 && isNodeLikelyLogicHigh(nodeFor(leKey), components, nodeFor)) {
2680         warnings.push(
2681             `CD4511 ${c.id} : LE au niveau haut – affichage verrouillé (ne suit plus le compteur). Reliez LE à 0 V pour le suivi en d
irect.`
2682         );
2683     }
2684 }
2685 if (c.type === "lamp" || c.type === "lcd") {
2686     const labels = { lamp: "Lampe", lcd: "LCD" };
2687     warnings.push(`${labels[c.type] || c.type} ${c.id} : non simulé pour l'instant.`);
2688 }
2689 if (c.type === "grove_lcd16x2") {
2690     warnings.push(
2691         `${c.id} (Grove LCD I²C) : protocole PCF8574/HD44780 simulé sur SDA/SCL (100 kHz). Scope : CH1=SDA, CH2=SCL, GND commun, 20-5
0 µs/div.`
2692     );
2693 }
2694 if (c.type === "grove_dht22") {
2695     warnings.push(
2696         `${c.id} (Grove DHT22) : lecture T°/humidité simulée via bibliothèque DHT.h sur broche DATA (câbler DATA→Dx, VCC→5V, GND).`
2697     );
2698 }
2699 if (c.type === "seg7") {
2700     const comKey = `${c.id}#7`;
2701     if ((terminalWireCount.get(comKey) || 0) === 0) {

```

```

2702         warnings.push(
2703             `7 Segments ${c.id} : reliez la cathode commune (broche C, bas) à la masse.`
2704         );
2705     }
2706 }
2707 if (c.type === "bargraph_dc10h") {
2708     const comKey = `${c.id}#10`;
2709     if ((terminalWireCount.get(comKey) || 0) === 0) {
2710         warnings.push(
2711             `Bargraph ${c.id} : reliez la cathode commune (COM) à la masse.`
2712         );
2713     }
2714 }
2715 }
2716
2717 const rippleMod10Warn = detectRippleMod10ResetAndWarning(components, parent);
2718 if (rippleMod10Warn) warnings.push(rippleMod10Warn);
2719
2720 const logicGates = [];
2721 for (const c of components) {
2722     if (isLogicGateComponentType(c.type)) {
2723         const outKey = logicGateOutputNodeKey(c);
2724         const vhi = resolveLogicGateVhi(c, logicVhiByTerminal);
2725         logicGates.push({
2726             id: c.id,
2727             type: c.type,
2728             nodeOut: nodeFor(outKey),
2729             inputs: logicGateInputNodeKeys(c).map(k => nodeFor(k)),
2730             vhi,
2731             vth: vhi / 2,
2732         });
2733     } else if (c.type === "logic_dff") {
2734         const vhi = resolveSequentialVhi(
2735             c,
2736             logicVhiByTerminal,
2737             parseLogicRail,
2738             logicVhi,
2739             logicDffInputNodeKeys(c)
2740         );
2741         const inputs = logicDffInputNodeKeys(c).map(k => nodeFor(k));
2742         logicGates.push({
2743             id: `${c.id}/Q`,
2744             type: c.type,
2745             nodeOut: nodeFor(logicDffOutputNodeKey(c)),
2746             inputs,
2747             vhi,
2748             vth: vhi / 2,
2749         });
2750         logicGates.push({
2751             id: `${c.id}/Qbar`,
2752             type: c.type,
2753             nodeOut: nodeFor(logicDffQbarOutputNodeKey(c)),
2754             inputs,
2755             vhi,
2756             vth: vhi / 2,
2757         });
2758     } else if (c.type === "logic_jk") {
2759         const vhi = resolveSequentialVhi(
2760             c,
2761             logicVhiByTerminal,
2762             parseLogicRail,
2763             logicVhi,
2764             logicJkInputNodeKeys(c)
2765         );
2766         const inputs = logicJkInputNodeKeys(c).map(k => nodeFor(k));
2767         logicGates.push({
2768             id: `${c.id}/Q`,
2769             type: c.type,
2770             nodeOut: nodeFor(logicJkOutputNodeKey(c)),
2771             inputs,
2772             vhi,
2773             vth: vhi / 2,
2774         });
2775         logicGates.push({
2776             id: `${c.id}/Qbar`,
2777             type: c.type,
2778             nodeOut: nodeFor(logicJkQbarOutputNodeKey(c)),
2779             inputs,
2780             vhi,
2781             vth: vhi / 2,

```

```

2782     });
2783     } else if (isIc74ls00Type(c.type)) {
2784         const vhi = resolveIc74ls00Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2785         ic74ls00NandGates().forEach((g, i) => {
2786             logicGates.push({
2787                 id: `${c.id}/Y${i + 1}`,
2788                 type: c.type,
2789                 nodeOut: nodeFor(`${c.id}#${g.y}`),
2790                 inputs: [nodeFor(`${c.id}#${g.a}`), nodeFor(`${c.id}#${g.b}`)],
2791                 vhi,
2792                 vth: vhi / 2,
2793             });
2794         });
2795     } else if (isIc74hc90Type(c.type)) {
2796         const vhi = resolveIc74hc90Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2797         const qNames = ["Q0", "Q1", "Q2", "Q3"];
2798         ic74hc90ToggleSlices().forEach((sl, i) => {
2799             logicGates.push({
2800                 id: `${c.id}/${qNames[i]}`,
2801                 type: c.type,
2802                 nodeOut: nodeFor(`${c.id}#${sl.q}`),
2803                 inputs: [nodeFor(`${c.id}#${sl.clk}`)],
2804                 vhi,
2805                 vth: vhi / 2,
2806             });
2807         });
2808     } else if (isArduinoUnoType(c.type)) {
2809         const vhi = 5;
2810         for (const pinIdx of arduinoUnoDigitalPinIndices()) {
2811             const pinName = arduinoUnoDigitalPinName(pinIdx);
2812             logicGates.push({
2813                 id: `${c.id}_${pinName}`,
2814                 type: c.type,
2815                 nodeOut: nodeFor(`${c.id}#${pinIdx}`),
2816                 inputs: [],
2817                 vhi,
2818                 vth: 2.5,
2819             });
2820         }
2821     } else if (isIc74ls74Type(c.type)) {
2822         const vhi = resolveIc74ls74Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2823         ic74ls74DffSlices().forEach((sl) => {
2824             logicGates.push({
2825                 id: `${c.id}/Q${sl.suffix}`,
2826                 type: c.type,
2827                 nodeOut: nodeFor(`${c.id}#${sl.q}`),
2828                 inputs: [nodeFor(`${c.id}#${sl.d}`), nodeFor(`${c.id}#${sl.clk}`)],
2829                 vhi,
2830                 vth: vhi / 2,
2831             });
2832         });
2833     } else if (isLogicCd4511Type(c.type)) {
2834         const vhi = resolveLogicCd4511Vhi(c, logicVhiByTerminal, parseLogicRail, logicVhi);
2835         const segNames = ["a", "b", "c", "d", "e", "f", "g"];
2836         segNames.forEach((s, i) => {
2837             logicGates.push({
2838                 id: `${c.id}/${s}`,
2839                 type: c.type,
2840                 nodeOut: nodeFor(`${c.id}#${7 + i}`),
2841                 inputs: cd4511InputNodeKeys(c).map((k) => nodeFor(k)),
2842                 vhi,
2843                 vth: vhi / 2,
2844             });
2845         });
2846     }
2847 }
2848
2849 const netlistText = lines.join("\n");
2850
2851 if (hasLogicCd4511 && !usesXspiceCd4511 && !usesBsourceCd4511) {
2852     return {
2853         ok: false,
2854         errors: [
2855             "CD4511 : modèle de simulation indisponible (XSPICE ou sources B).",
2856         ],
2857         warnings,
2858         netlist: netlistText,
2859         voltmeters: [],
2860         ammeters: [],
2861         ohmmeters: [],

```

```

2862         oscilloscopes: [],
2863         nodeMeasures: [],
2864         scopesTranMeta: [],
2865         analysisTran: false,
2866         analysisAc: false,
2867     };
2868 }
2869
2870 return {
2871     ok: true,
2872     netlist: netlistText,
2873     warnings,
2874     voltmeters,
2875     speakers,
2876     ammeters,
2877     leds: components
2878         .filter(c => isLedType(c.type))
2879         .map(c => ({
2880             id: c.id,
2881             branch: spiceBranchName("VIL", c.id),
2882             nodePlus: nodeFor(`${c.id}#0`),
2883             nodeMinus: nodeFor(`${c.id}#1`),
2884         })),
2885     voltmetersRms,
2886     ammetersRms,
2887     ohmmeters,
2888     ohmmeterIsolationNetlist:
2889         ohmmeters.length > 0 ? buildOhmmeterIsolationNetlist(netlistText, ohmmeters) : "",
2890     oscilloscopes,
2891     nodeMeasures: [],
2892     scopesTranMeta,
2893     ledsTranMeta,
2894     logicGates,
2895     logicGatesTranMeta,
2896     metersTranMeta,
2897     seg7TranMeta,
2898     bargraphTranMeta,
2899     analysisTran,
2900     analysisAc,
2901     bodeAnalyzers,
2902     bodeAcMeta,
2903     seg7Displays: components
2904         .filter(c => c.type === "seg7")
2905         .map(c => ({
2906             id: c.id,
2907             segmentNodes: [0, 1, 2, 3, 4, 5, 6].map(i => nodeFor(`${c.id}#${i}`)),
2908             commonNode: nodeFor(`${c.id}#7`),
2909         })),
2910     bargraphDisplays: components
2911         .filter(c => c.type === "bargraph_dc10h")
2912         .map(c => ({
2913             id: c.id,
2914             segmentNodes: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9].map(i => nodeFor(`${c.id}#${i}`)),
2915             commonNode: nodeFor(`${c.id}#10`),
2916         })),
2917     };
2918 }
2919

```



```

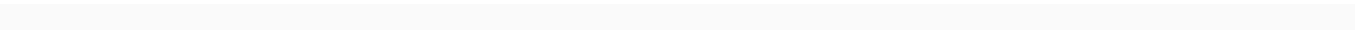
1  /**
2   * Oscilloscope : signaux idéaux 0/5 V depuis GPIO Arduino (comme LED / voltmètre).
3   */
4
5  import { reachableJonctions } from "./hc90-cascade.mjs";
6  import {
7    applyArduinoSketchToComponent,
8    resolveArduinoPinLevelAt,
9    sketchHasLoop,
10   createArduinoRuntime,
11   stepArduinoRuntime,
12   arduinoRuntimeLevels,
13 } from "./arduino-sketch-parse.mjs";
14 import {
15   buildUartScopeModel,
16   uartScopeBitAt,
17   sketchUsesSerial,
18   uartCharDurationSec,
19   uartSketchLoopPeriodMs,
20 } from "./arduino-uart-wave.mjs";
21
22 const uartModelCache = new WeakMap();
23
24 function pinHasSketchTiming(uno, pinLabel) {
25   if (uno?.pinPulses?.[pinLabel]) return true;
26   if (Array.isArray(uno?.pinPhases) && uno.pinPhases.length >= 2) {
27     return uno.pinPhases.some((ph) => Object.prototype.hasOwnProperty.call(ph.levels || {}, pinLabel));
28   }
29   return false;
30 }
31
32 function isUartTxPin(uno, pinLabel) {
33   return pinLabel === "D1" && sketchUsesSerial(uno?.sketch || "");
34 }
35
36 function getUartScopeModel(uno) {
37   const sketch = uno?.sketch || "";
38   const cached = uartModelCache.get(uno);
39   if (cached && cached.sketch === sketch) return cached;
40   const model = { sketch, ...buildUartScopeModel(uno) };
41   uartModelCache.set(uno, model);
42   return model;
43 }
44
45 /** Période (s) du signal sur une broche, si connue depuis le sketch. */
46 export function arduinoPinPeriodSec(uno, pinLabel) {
47   if (!uno || !pinLabel) return 0;
48   if (isUartTxPin(uno, pinLabel)) {
49     const model = getUartScopeModel(uno);
50     if (model.loopPeriodMs > 0) return model.loopPeriodMs / 1000;
51     if (model.baud > 0) return uartCharDurationSec(model.baud) * 4;
52   }
53   const pulse = uno.pinPulses?.[pinLabel];
54   if (pulse?.highSec > 0 && pulse?.lowSec > 0) {
55     return pulse.highSec + pulse.lowSec;
56   }
57   if (Array.isArray(uno.pinPhases) && uno.pinPhases.length >= 2) {
58     const usesPin = uno.pinPhases.some((ph) =>
59       Object.prototype.hasOwnProperty.call(ph.levels || {}, pinLabel)
60     );
61     if (usesPin) {
62       const totalMs = uno.pinPhases.reduce((s, p) => s + (p.durationMs || 0), 0);
63       if (totalMs > 0) return totalMs / 1000;
64     }
65   }
66   return 0;
67 }
68
69 /**
70  * UNO en sortie câblé sur CH1 ou CH2 de l'oscilloscope.
71  * @param {'CH1'|'CH2'} channelSuffix
72  */
73 export function findArduinoDriveForScopeChannel(
74   oscLabel,
75   channelSuffix,
76   components,
77   wires,
78   autoJonctions = []

```

```

79 } {
80   const net = reachableJunctions(`${oscLabel}_${channelSuffix}`, wires, autoJunctions);
81   for (const comp of components) {
82     if (comp.type !== "arduino_uno") continue;
83     applyArduinoSketchToComponent(comp);
84     for (const jid of net) {
85       if (!jid.startsWith(`${comp.label}_`)) continue;
86       const pinLabel = jid.slice(comp.label.length + 1);
87       if (!/^D\d+$|^A\d+$/.test(pinLabel)) continue;
88       if (isUartTxPin(comp, pinLabel)) {
89         return { uno: comp, pinLabel, uart: true };
90       }
91       if (comp.pinModes?.[pinLabel] === "OUTPUT") {
92         return { uno: comp, pinLabel, uart: false };
93       }
94     }
95   }
96   return null;
97 }
98
99 function runtimePinLevelAt(uno, pinLabel, tSec, inputs = {}) {
100   const rt = createArduinoRuntime(uno);
101   stepArduinoRuntime(rt, Math.max(0, tSec) * 1000, inputs);
102   const lv = arduinoRuntimeLevels(rt)[pinLabel];
103   return lv === 1 || lv === "1" || lv === true ? 1 : 0;
104 }
105
106 function synthesizeUartScopeTrace(uno, windowSec, timeOffsetSec = 0) {
107   const model = getUartScopeModel(uno);
108   if (!model.schedule?.length || !model.baud) return [];
109   const samplesPerBit = 4;
110   const n = Math.max(
111     128,
112     Math.min(4000, Math.ceil(windowSec * model.baud * samplesPerBit))
113   );
114   const points = [];
115   for (let i = 0; i < n; i++) {
116     const frac = n > 1 ? i / (n - 1) : 0;
117     const tScreenSec = frac * windowSec;
118     const tMs = timeOffsetSec * 1000 + tScreenSec * 1000;
119     const bit = uartScopeBitAt(model, tMs);
120     points.push({ t: tScreenSec, v: bit ? 5 : 0 });
121   }
122   return points;
123 }
124
125 /**
126  * Courbe numérique 0/5 V.
127  * Signaux périodiques (delay / blink / UART) : fenêtre verrouillée sur la phase.
128  */
129 export function synthesizeArduinoScopeTrace(uno, pinLabel, windowSec, elapsedSec, inputs = {}, timeOffsetSec = 0) {
130   if (!uno || !pinLabel || windowSec <= 0) return [];
131
132   if (isUartTxPin(uno, pinLabel)) {
133     return synthesizeUartScopeTrace(uno, windowSec, timeOffsetSec);
134   }
135
136   const period = arduinoPinPeriodSec(uno, pinLabel);
137   const synced = period > 0;
138   const useRuntime = sketchHasLoop(uno.sketch || "") && !pinHasSketchTiming(uno, pinLabel);
139   const n = Math.max(64, Math.min(800, Math.ceil(windowSec * 4000)));
140
141   const points = [];
142   for (let i = 0; i < n; i++) {
143     const frac = n > 1 ? i / (n - 1) : 0;
144     const tScreen = frac * windowSec;
145     let tAbs;
146     if (synced) {
147       tAbs = ((timeOffsetSec + tScreen) % period + period) % period;
148     } else {
149       const tEnd = Math.max(0, elapsedSec) + timeOffsetSec;
150       tAbs = Math.max(0, tEnd - windowSec) + tScreen;
151     }
152     const lv = useRuntime
153       ? runtimePinLevelAt(uno, pinLabel, tAbs, inputs)
154       : resolveArduinoPinLevelAt(uno, pinLabel, tAbs);
155     points.push({ t: tScreen, v: lv ? 5 : 0 });
156   }
157   return points;
158 }

```



```

1  /**
2   * Fumée : pile + R + voltmètre → ngspice + merge voltmètres.
3   * node Simulateur/Engine/simulate-smoke.mjs
4   */
5  import { buildNgspiceDeck } from "./spice-netlist-v2.mjs";
6  import { execFileSync } from "child_process";
7  import { writeFileSync, readFileSync, mkdtempSync, existsSync } from "fs";
8  import { join, dirname } from "path";
9  import { fileURLToPath } from "url";
10 import { tmpdir } from "os";
11 import { mergeVoltmeterMeasurements } from "./v2/result-parser.mjs";
12
13 const __dirname = dirname(fileURLToPath(import.meta.url));
14 const ng = join(__dirname, "..", "bin", "ngspice_con.exe");
15
16 const state = {
17   components: [
18     { id: "G", type: "ground", x: 0, y: 0, orient: 0 },
19     { id: "V1", type: "vsource", x: 0, y: 0, value: "5V", orient: 0 },
20     { id: "R1", type: "resistor", x: 0, y: 0, value: "1k", orient: 0 },
21     { id: "VM1", type: "voltmeter", x: 0, y: 0, orient: 0 },
22   ],
23   wires: [
24     { id: "w0", solid: true, fromKey: "G#0", toKey: "V1#1", points: [] },
25     { id: "w1", solid: true, fromKey: "V1#0", toKey: "R1#0", points: [] },
26     { id: "w2", solid: true, fromKey: "R1#1", toKey: "G#0", points: [] },
27     { id: "w3", solid: true, fromKey: "VM1#0", toKey: "R1#0", points: [] },
28     { id: "w4", solid: true, fromKey: "VM1#1", toKey: "G#0", points: [] },
29   ],
30 };
31
32 const built = await buildNgspiceDeck(state);
33 if (!built.ok) {
34   console.error(built);
35   process.exit(1);
36 }
37 if (!existsSync(ng)) {
38   console.log("SKIP (pas de ngspice_con.exe)");
39   process.exit(0);
40 }
41 const dir = mkdtempSync(join(tmpdir(), "smoke-"));
42 const cir = join(dir, "c.cir");
43 const logp = join(dir, "l.log");
44 writeFileSync(cir, built.netlist, "utf8");
45 try {
46   execFileSync(ng, ["-b", "-o", logp, cir]);
47 } catch (e) {
48   console.error("ngspice exit", e?.status);
49 }
50 const logText = readFileSync(logp, "utf8");
51 const vm = mergeVoltmeterMeasurements(logText, built.voltmeters, []);
52 const vm1 = vm.VM1;
53 const v = vm1 && typeof vm1 === "object" ? vm1.voltage : vm1;
54 console.log("netlist prefix:", JSON.stringify(built.netlist.slice(0, 120)));
55 console.log("VM1 merged:", vm1);
56 if (!Number.isFinite(v) || Math.abs(v - 5) > 0.01) {
57   console.error("FAIL voltmètre attendu ~5 V");
58   console.error(logText.slice(-800));
59   process.exit(1);
60 }
61 console.log("simulate-smoke.mjs : OK");
62

```

```

1  /** Import à chaque build pour éviter le cache ESM de schematic-to-spice (rechargé via importFresh côté serveur). */
2  async function loadSchematicBuilder() {
3      const mod = await import(`./schematic-to-spice.mjs?fresh=${Date.now()}`);
4      if (typeof mod.buildNetlistFromGraphicalState !== "function") {
5          throw new Error("buildNetlistFromGraphicalState introuvable dans schematic-to-spice.mjs");
6      }
7      return mod.buildNetlistFromGraphicalState;
8  }
9
10 /**
11  * Mode batch ngspice (-b) : un bloc .control sans analyse au niveau circuit
12  * (.op, .tran, .dc, ...) échoue souvent. On insère .op avant .control si besoin,
13  * et « quit » avant .endc pour une sortie propre.
14  * @param {string} text
15  */
16 function normalizeSpiceNetlistForNgspiceBatch(text) {
17     let s = String(text || "").replace(/\uFEFF/, "");
18     // SPICE / ngspice : la **première ligne non vide** du fichier est toujours le « titre »
19     // et n'est **pas** un composant. Si on supprime notre ligne * ..., la 1re ligne devient
20     // par ex. « V_V1 ... » : elle est alors ignorée comme titre → circuit faux (0 V, etc.).
21     // Remplacer par une ligne titre minimale (ASCII) – ngspice accepte tout commentaire *.
22     s = s.replace(
23         /\^* Circuit Designer - netlist SPICE \\\.(?:op|tran|ac|dc|noise|sens|tf)\b/im,
24         "* CD\n"
25     );
26     // Ne PAS forcer CRLF tout le fichier : sur certaines installs ngspice-46 sous Windows,
27     // ça combiné au bloc .control / wrdata donne une analyse transitoire « réussie » mais
28     // sans données exploitables dans l'interface.
29     if (!/\/S/.test(s) || !/^s*\..control\b/im.test(s)) return s;
30
31     const hasCircuitAnalysis = /\^s*\..(op|tran|dc|ac|pz|noise|sens|tf)\b/im.test(s);
32     if (!hasCircuitAnalysis) {
33         s = s.replace(/\^s*\..control\b/im, ".op\n.control");
34     }
35
36     const controlBlock = s.match(/\.control\b([\s\S]*)\.endc\b/i);
37     if (controlBlock && !/quit\b/i.test(controlBlock[1])) {
38         s = s.replace(/(\r?\n)(\s*\..endc\b)/i, "$1quit$1$2");
39     }
40     return s;
41 }
42
43 // Netlist brute (éditeur Simulateur) ou état JSON avec champ .netlist
44 export async function buildNgspiceDeck(state, opts = {}) {
45     if (
46         state &&
47         typeof state === "object" &&
48         !state.netlist &&
49         Array.isArray(state.components) &&
50         Array.isArray(state.wires)
51     ) {
52         const buildNetlistFromGraphicalState = await loadSchematicBuilder();
53         const built = buildNetlistFromGraphicalState(state, opts);
54         if (!built.ok) return built;
55         const nrm = normalizeSpiceNetlistForNgspiceBatch(built.netlist);
56         return {
57             ...built,
58             netlist: nrm,
59             warnings: built.warnings || [],
60         };
61     }
62
63     const raw =
64         typeof state === "string"
65             ? state
66             : typeof state?.netlist === "string"
67                 ? state.netlist
68                 : "";
69     const netlist = typeof raw === "string" ? normalizeSpiceNetlistForNgspiceBatch(raw) : "";
70     if (!netlist.trim()) {
71         return {
72             ok: false,
73             errors: ["Netlist vide ou invalide : fournir une chaîne SPICE ou un objet { netlist: string }."],
74             warnings: [],
75             netlist: "",
76         };
77     }
78     const warnings = [];

```

```
79     if (typeof raw === "string" && raw !== netlist) {
80         warnings.push("Netlist ajustée pour le mode batch ngspice (-b) : analyse .op ou commande quit ajoutée si nécessaire.");
81     }
82     return {
83         ok: true,
84         netlist,
85         analysisTran: /\b\.tran\b/i.test(netlist),
86         voltmeters: [],
87         ammeters: [],
88         ohmmeters: [],
89         nodeMeasures: [],
90         scopesTranMeta: [],
91         warnings,
92     };
93 }
```

```

1  /**
2
3  * Extraction des tensions / courants ngspice (.op) : plusieurs formats selon version / sortie.
4
5  */
6
7  import { logicLevelFromVoltage } from "../logic-rails.mjs";
8  import { bcdDigitToSeg7Segments, bcdFromQVoltages, SEG7_NAMES as BCD_SEG7_NAMES } from "../bcd-seg7.mjs";
9  import { quantizeVoltmeterReading } from "../voltmeter-display.mjs";
10
11
12
13 /**
14
15  * @param {string} log
16
17  * @returns {Record<string, number>}
18
19  */
20
21 function collectNodeVoltagesFromLog(log) {
22
23     const map = Object.create(null);
24
25     if (!log || typeof log !== "string") return map;
26
27
28
29     const reEquals = /\bV\s*\(\s*([^\)]+)\s*\)\s*=\s*([+-]eE0-9.]+)/gi;
30
31     let m;
32
33     while ((m = reEquals.exec(log)) !== null) {
34
35         const node = String(m[1]).trim().toLowerCase();
36
37         const v = parseFloat(m[2]);
38
39         if (node && Number.isFinite(v)) map[node] = v;
40
41     }
42
43
44
45     const reSpaced = /\bV\s*\(\s*([^\)]+)\s*\)\s+([+-]eE0-9.+)/gi;
46
47     while ((m = reSpaced.exec(log)) !== null) {
48
49         const node = String(m[1]).trim().toLowerCase();
50
51         const v = parseFloat(m[2]);
52
53         if (node && Number.isFinite(v)) map[node] = v;
54
55     }
56
57
58
59     const lines = log.split(/\r?\n/);
60
61     for (let i = 0; i < lines.length; i++) {
62
63         if (!Node.*Voltage/i.test(lines[i])) continue;
64
65         let j = i + 1;
66
67         while (j < lines.length && /^s*(-+|\.{3,})/.test(lines[j])) j++;
68
69         for (; j < lines.length; j++) {
70
71             const line = lines[j];
72
73             if (!line || !line.trim()) break;
74
75             if (/^Reference value/i.test(line)) continue;
76
77             const row = /^s*(\s+)\s+([+-])?(?:\d+\.\d*|\.\d+)(?:[eE][+-]?\d+)?\s*$/i.exec(line.trim());
78

```

```

79         if (!row) continue;
80
81         const node = row[1].toLowerCase();
82
83         if (node === "node" || node === "----" || /^\. {2,}/.test(row[1])) continue;
84
85         const v = parseFloat(row[2]);
86
87         if (Number.isFinite(v)) map[node] = v;
88
89     }
90
91 }
92
93
94
95 return map;
96
97 }
98
99
100
101 /** Courants de branches : i(vi_e1) = ... */
102
103 function collectBranchCurrentsFromLog(log) {
104
105     const map = Object.create(null);
106
107     if (!log || typeof log !== "string") return map;
108
109
110
111     const reEquals = /\bI\s*(\s*([^\s]+?)\s*)\s*=\s*([-\+eE0-9.]+)/gi;
112
113     let m;
114
115     while ((m = reEquals.exec(log)) !== null) {
116
117         const branch = String(m[1]).trim().toLowerCase();
118
119         const i = parseFloat(m[2]);
120
121         if (branch && Number.isFinite(i)) map[branch] = i;
122
123     }
124
125
126
127     const reSpaced = /\bI\s*(\s*([^\s]+?)\s*)\s+(\s+([-+eE0-9.]+))/gi;
128
129     while ((m = reSpaced.exec(log)) !== null) {
130
131         const branch = String(m[1]).trim().toLowerCase();
132
133         const i = parseFloat(m[2]);
134
135         if (branch && Number.isFinite(i)) map[branch] = i;
136
137     }
138
139     const reRev = /\b([-+eE0-9.]+\s*=\s*i\s*(\s*([^\s]+?)\s*)/gi;
140
141     while ((m = reRev.exec(log)) !== null) {
142
143         const branch = String(m[2]).trim().toLowerCase();
144
145         const i = parseFloat(m[1]);
146
147         if (branch && Number.isFinite(i)) map[branch] = i;
148
149     }
150
151     const reDev = /\b((?:vil|vi[d]_[a-z0-9_]+)(?:#branchcurrent)?\s*[:=]\s+([-+eE0-9.]+))/gi;
152
153     while ((m = reDev.exec(log)) !== null) {
154
155         const branch = String(m[1]).trim().toLowerCase();
156
157         const i = parseFloat(m[2]);
158

```



```

159         if (branch && Number.isFinite(i)) map[branch] = i;
160
161     }
162
163     return map;
164 }
165
166
167
168
169 function isSpiceReferenceNode(name) {
170
171     const n = String(name || "")
172
173     .trim()
174
175     .toLowerCase();
176
177     return n === "0" || n === "gnd";
178 }
179
180
181
182
183 function nodeVoltageFromMap(name, map) {
184
185     if (isSpiceReferenceNode(name)) return 0;
186
187     const n = String(name || "")
188
189     .trim()
190
191     .toLowerCase();
192
193     const v = map[n];
194
195     return v != null && Number.isFinite(v) ? v : null;
196 }
197
198
199
200
201 function branchCurrentFromMap(branchName, map) {
202
203     const b = String(branchName || "")
204
205     .trim()
206
207     .toLowerCase();
208
209     const i = map[b];
210
211     return i != null && Number.isFinite(i) ? i : null;
212 }
213
214
215
216
217 /**
218  * @param {string} log
219  * @param {{ id: string; nodePlus: string; nodeMinus: string }[]} voltmeters
220  * @param {unknown[]} _nodeMeasures
221  */
222
223 export function mergeVoltmeterMeasurements(log, voltmeters, _nodeMeasures) {
224
225     const out = {};
226
227     if (!log || typeof log !== "string" || !Array.isArray(voltmeters) || voltmeters.length === 0) return out;
228
229
230
231     const map = collectNodeVoltagesFromLog(log);
232
233
234
235
236
237
238

```

```

239   for (const vm of voltmeters) {
240
241       const np = String(vm.nodePlus || "").trim().toLowerCase();
242
243       const nm = String(vm.nodeMinus || "").trim().toLowerCase();
244
245       if (!np || !nm) continue;
246
247       if (isSpiceReferenceNode(vm.nodePlus) && isSpiceReferenceNode(vm.nodeMinus)) continue;
248
249
250
251       const vp = nodeVoltageFromMap(vm.nodePlus, map);
252
253       const vmv = nodeVoltageFromMap(vm.nodeMinus, map);
254
255       if (vp == null || vmv == null) continue;
256
257
258
259       out[vm.id] = {
260
261           voltage: vp - vmv,
262
263           unit: "V",
264
265           nodePlus: vm.nodePlus,
266
267           nodeMinus: vm.nodeMinus,
268
269       };
270
271   }
272
273
274
275   return out;
276
277 }
278
279
280
281 /**
282  * @param {string} log
283  * @param {{ id: string; branch: string; nodePlus?: string; nodeMinus?: string }[]} ammeters
284  */
285
286
287
288 export function mergeAmmeterMeasurements(log, ammeters) {
289
290     const out = {};
291
292     if (!log || typeof log !== "string" || !Array.isArray(ammeters) || ammeters.length === 0) return out;
293
294
295
296
297     const map = collectBranchCurrentsFromLog(log);
298
299
300
301     for (const am of ammeters) {
302
303         if (!am.branch) continue;
304
305         const i = branchCurrentFromMap(am.branch, map);
306
307         if (i == null) continue;
308
309         out[am.id] = {
310
311             current: i,
312
313             unit: "A",
314
315             branch: am.branch,
316
317             nodePlus: am.nodePlus,
318

```

```

319         nodeMinus: am.nodeMinus,
320     };
321 };
322
323 }
324
325
326
327     return out;
328 }
329 }
330
331 /**
332  * @param {string} log
333  * @param {{ id: string; branch: string }[]} leds
334  */
335 function ledForwardCurrentFromRow(row, meta, vmap) {
336     if (!row || typeof row.current !== "number" || !Number.isFinite(row.current)) return 0;
337     const i = row.current;
338     const np = meta?.nodePlus;
339     const nm = meta?.nodeMinus;
340     if (np && nm && vmap && Object.keys(vmap).length > 0) {
341         const vp = nodeVoltageFromMap(np, vmap);
342         const vm = nodeVoltageFromMap(nm, vmap);
343         if (vp !== null && vm !== null) {
344             if (vp <= vm + 0.15) return 0;
345             return Math.abs(i);
346         }
347     }
348     return i > 0 ? i : 0;
349 }
350
351 export function mergeLedMeasurements(log, leds) {
352     const out = {};
353     if (!log || typeof log !== "string" || !Array.isArray(leds) || leds.length === 0) return out;
354
355     const map = collectBranchCurrentsFromLog(log);
356     const vmap = collectNodeVoltagesFromLog(log);
357
358     for (const ld of leds) {
359         if (!ld.branch) continue;
360         const i = branchCurrentFromMap(ld.branch, map);
361         if (i == null) continue;
362         const iFwd = ledForwardCurrentFromRow({ current: i }, ld, vmap);
363         out[ld.id] = {
364             current: iFwd,
365             unit: "A",
366             branch: ld.branch,
367             nodePlus: ld.nodePlus,
368             nodeMinus: ld.nodeMinus,
369         };
370     }
371
372     return out;
373 }
374
375 /**
376  * @param {string} log
377  * @param {{ id: string; nodePlus: string; nodeMinus: string; testCurrent?: number }[]} ohmmeters
378  */
379 export function mergeOhmmeterMeasurements(log, ohmmeters) {
380     const out = {};
381     if (!log || typeof log !== "string" || !Array.isArray(ohmmeters) || ohmmeters.length === 0) return out;
382
383     const map = collectNodeVoltagesFromLog(log);
384
385     for (const om of ohmmeters) {
386         const iTest = om.testCurrent > 0 ? om.testCurrent : 0.001;
387         const vp = nodeVoltageFromMap(om.nodePlus, map);
388         const vm = nodeVoltageFromMap(om.nodeMinus, map);
389         if (vp == null || vm == null) continue;
390         const r = Math.abs(vp - vm) / iTest;
391         if (!Number.isFinite(r) || r <= 0) continue;
392
393         out[om.id] = {
394             resistance: r,
395             unit: "Ohm",
396             nodePlus: om.nodePlus,
397             nodeMinus: om.nodeMinus,
398         };

```

```

399     }
400
401     return out;
402 }
403
404 /**
405  * @param {string} log
406  * @param {{ id: string; ch1NodePlus: string; ch1NodeMinus: string; ch2NodePlus: string; ch2NodeMinus: string }[]} oscilloscopes
407  */
408 export function mergeOscilloscopeMeasurements(log, oscilloscopes) {
409     const out = {};
410     if (!log || typeof log !== "string" || !Array.isArray(oscilloscopes) || oscilloscopes.length === 0) {
411         return out;
412     }
413
414     const map = collectNodeVoltagesFromLog(log);
415
416     function channelVoltage(nodePlus, nodeMinus) {
417         const np = String(nodePlus || "").trim().toLowerCase();
418         const nm = String(nodeMinus || "").trim().toLowerCase();
419         if (!np || !nm) return null;
420         if (isSpiceReferenceNode(nodePlus) && isSpiceReferenceNode(nodeMinus)) return null;
421         const vp = nodeVoltageFromMap(nodePlus, map);
422         const vm = nodeVoltageFromMap(nodeMinus, map);
423         if (vp == null || vm == null) return null;
424         return vp - vm;
425     }
426
427     for (const osc of oscilloscopes) {
428         const v1 = channelVoltage(osc.ch1NodePlus, osc.ch1NodeMinus);
429         const v2 = channelVoltage(osc.ch2NodePlus, osc.ch2NodeMinus);
430         if (v1 == null && v2 == null) continue;
431         const row = { id: osc.id };
432         if (v1 != null) {
433             row.ch1 = {
434                 voltage: v1,
435                 unit: "V",
436                 nodePlus: osc.ch1NodePlus,
437                 nodeMinus: osc.ch1NodeMinus,
438             };
439         }
440         if (v2 != null) {
441             row.ch2 = {
442                 voltage: v2,
443                 unit: "V",
444                 nodePlus: osc.ch2NodePlus,
445                 nodeMinus: osc.ch2NodeMinus,
446             };
447         }
448         out[osc.id] = row;
449     }
450
451     return out;
452 }
453
454 /**
455  * @param {string} waveTxt
456  * @returns {number[][]}
457  */
458 function parseWrdDataNumericRows(waveTxt) {
459     const rows = [];
460     if (!waveTxt || typeof waveTxt !== "string") return rows;
461     for (const line of waveTxt.split(/\r?\n/)) {
462         const t = line.trim();
463         if (!t || t.startsWith("#")) continue;
464         const nums = t.split(/\s+/).map(s => parseFloat(s)).filter(n => Number.isFinite(n));
465         if (nums.length >= 2) rows.push(nums);
466     }
467     return rows;
468 }
469
470 function waveformPeakToPeak(values) {
471     if (!Array.isArray(values) || values.length === 0) return NaN;
472     let min = Infinity;
473     let max = -Infinity;
474     for (const v of values) {
475         if (!Number.isFinite(v)) continue;
476         if (v < min) min = v;
477         if (v > max) max = v;
478     }

```

```

479     return Number.isFinite(min) && Number.isFinite(max) ? max - min : NaN;
480 }
481
482 function waveformRms(values) {
483     if (!Array.isArray(values) || values.length === 0) return NaN;
484     let sum = 0;
485     let n = 0;
486     for (const v of values) {
487         if (!Number.isFinite(v)) continue;
488         sum += v * v;
489         n++;
490     }
491     return n > 0 ? Math.sqrt(sum / n) : NaN;
492 }
493
494 /**
495  * @param {string} waveTxt - sortie ngspice wrdata
496  * @param {{ id: string; timeCol?: number; wrVarCount?: number; ch1: { wrIndex?: number; minusWrIndex?: number | null; minusIsGnd?: boolean }; ch2: { wrIndex?: number; minusWrIndex?: number | null; minusIsGnd?: boolean } }[]} meta
497  */
498 export function mergeScopePlotsFromTranWrdata(waveTxt, meta) {
499     const out = {};
500     const rows = parseWrdataNumericRows(waveTxt);
501     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
502
503     let wrVarCount = 0;
504     for (const m of meta) {
505         if (m.wrVarCount > wrVarCount) wrVarCount = m.wrVarCount;
506         const idxs = [m.ch1?.wrIndex, m.ch2?.wrIndex, m.ch1?.minusWrIndex, m.ch2?.minusWrIndex];
507         for (const ix of idxs) {
508             if (ix != null && ix + 1 > wrVarCount) wrVarCount = ix + 1;
509         }
510     }
511     if (wrVarCount < 2) wrVarCount = 2;
512     const ncol = rows[0].length;
513     const colOffset = Math.max(0, ncol - wrVarCount);
514
515     function channelDiff(row, ch) {
516         if (!ch) return NaN;
517         const plusIsGnd = ch.plusIsGnd ?? isSpiceReferenceNode(ch.plusNode);
518         const minusIsGnd = ch.minusIsGnd ?? isSpiceReferenceNode(ch.minusNode);
519         let vp = 0;
520         let vm = 0;
521         if (!plusIsGnd) {
522             if (ch.wrIndex == null || ch.wrIndex === undefined) return NaN;
523             vp = row[ch.wrIndex + colOffset];
524             if (!Number.isFinite(vp)) return NaN;
525         }
526         if (!minusIsGnd) {
527             if (ch.minusWrIndex == null) return NaN;
528             vm = row[ch.minusWrIndex + colOffset];
529             if (!Number.isFinite(vm)) return NaN;
530         }
531         return vp - vm;
532     }
533
534     for (const m of meta) {
535         if (!m || !m.id) continue;
536         const timeCol = m.timeCol ?? 0;
537         const tArr = [];
538         const v1Arr = [];
539         const v2Arr = [];
540         for (const row of rows) {
541             if (row.length <= timeCol) continue;
542             const d1 = channelDiff(row, m.ch1);
543             const d2 = channelDiff(row, m.ch2);
544             if (!Number.isFinite(row[timeCol])) continue;
545             const ok1 = Number.isFinite(d1);
546             const ok2 = Number.isFinite(d2);
547             if (!ok1 && !ok2) continue;
548             tArr.push(row[timeCol]);
549             v1Arr.push(ok1 ? d1 : 0);
550             v2Arr.push(ok2 ? d2 : 0);
551         }
552         if (tArr.length === 0) continue;
553         out[m.id] = {
554             ch1: { time: tArr, voltage: v1Arr, unit: "V", label: "CH1" },
555             ch2: { time: tArr, voltage: v2Arr, unit: "V", label: "CH2" },
556         };
557     }

```

```

558     return out;
559 }
560
561
562 /**
563  * Valeurs résumées (crête à crête) pour le tableau lorsque l'analyse est .tran.
564  * @param {Record<string, { ch1?: { voltage?: number[] }; ch2?: { voltage?: number[] } }>} scopePlots
565  */
566 function channelDiffFromRow(row, ch, colOffset) {
567     if (!ch) return NaN;
568     const plusIsGnd = ch.plusIsGnd ?? isSpiceReferenceNode(ch.plusNode);
569     const minusIsGnd = ch.minusIsGnd ?? isSpiceReferenceNode(ch.minusNode);
570     let vp = 0;
571     let vm = 0;
572     if (!plusIsGnd) {
573         if (ch.wrIndex == null || ch.wrIndex === undefined) return NaN;
574         vp = row[ch.wrIndex + colOffset];
575         if (!Number.isFinite(vp)) return NaN;
576     }
577     if (!minusIsGnd) {
578         if (ch.minusWrIndex == null) return NaN;
579         vm = row[ch.minusWrIndex + colOffset];
580         if (!Number.isFinite(vm)) return NaN;
581     }
582     return vp - vm;
583 }
584
585 /**
586  * @param {string} waveTxt
587  * @param {{ id: string; wrVarCount?: number; channel: { wrIndex?: number; minusWrIndex?: number | null; minusIsGnd?: boolean }; nodePlus?: string; nodeMinus?: string }[]} meta
588  */
589 export function mergeVoltmeterRmsFromTranWrdata(waveTxt, meta) {
590     const out = {};
591     const rows = parseWrdataNumericRows(waveTxt);
592     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
593
594     for (const m of meta) {
595         if (!m?.id || !m.channel) continue;
596         let wrVarCount = m.wrVarCount || 2;
597         if (m.channel.wrIndex != null && m.channel.wrIndex + 1 > wrVarCount) {
598             wrVarCount = m.channel.wrIndex + 1;
599         }
600         const ncol = rows[0].length;
601         const colOffset = Math.max(0, ncol - wrVarCount);
602         const vals = [];
603         for (const row of rows) {
604             const d = channelDiffFromRow(row, m.channel, colOffset);
605             if (Number.isFinite(d)) vals.push(d);
606         }
607         const vrms = waveformRms(vals);
608         if (!Number.isFinite(vrms)) continue;
609         out[m.id] = {
610             voltage: vrms,
611             unit: "V",
612             measure: "Vrms",
613             nodePlus: m.nodePlus,
614             nodeMinus: m.nodeMinus,
615         };
616     }
617     return out;
618 }
619
620 /**
621  * @param {string} waveTxt
622  * @param {{ id: string; wrVarCount?: number; currentWrIndex?: number; branch?: string }[]} meta
623  */
624 export function mergeAmmeterRmsFromTranWrdata(waveTxt, meta) {
625     const out = {};
626     const rows = parseWrdataNumericRows(waveTxt);
627     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
628
629     for (const m of meta) {
630         if (!m?.id || m.currentWrIndex == null || m.currentWrIndex === undefined) continue;
631         const wrVarCount = m.wrVarCount || m.currentWrIndex + 1;
632         const ncol = rows[0].length;
633         const colOffset = Math.max(0, ncol - wrVarCount);
634         const vals = [];
635         for (const row of rows) {
636             const i = row[m.currentWrIndex + colOffset];

```

```

637         if (Number.isFinite(i)) vals.push(i);
638     }
639     const irms = waveformRms(vals);
640     if (!Number.isFinite(irms)) continue;
641     out[m.id] = {
642         current: irms,
643         unit: "A",
644         measure: "Arms",
645         branch: m.branch,
646     };
647 }
648 return out;
649 }
650
651 /**
652  * Voltmètre en .tran : dernière valeur instantanée (adapté aux sorties logiques / bascules).
653  * @param {string} waveTxt
654  * @param {{ id: string; wrVarCount?: number; channel: { wrIndex?: number; minusWrIndex?: number | null; minusIsGnd?: boolean }; nodePlus?: s
tring; nodeMinus?: string }[]} meta
655  */
656 export function mergeVoltmeterFromTranWrdata(waveTxt, meta) {
657     const out = {};
658     const rows = parseWrdataNumericRows(waveTxt);
659     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
660
661     for (const m of meta) {
662         if (!m?.id || !m.channel) continue;
663         let wrVarCount = m.wrVarCount || 2;
664         if (m.channel.wrIndex !== null && m.channel.wrIndex + 1 > wrVarCount) {
665             wrVarCount = m.channel.wrIndex + 1;
666         }
667         const ncol = rows[0].length;
668         const colOffset = Math.max(0, ncol - wrVarCount);
669         const vals = [];
670         for (const row of rows) {
671             const d = channelDiffFromRow(row, m.channel, colOffset);
672             if (Number.isFinite(d)) vals.push(d);
673         }
674         if (!vals.length) continue;
675         const raw = vals[vals.length - 1];
676         out[m.id] = {
677             voltage: quantizeVoltmeterReading(raw, vals),
678             unit: "V",
679             nodePlus: m.nodePlus,
680             nodeMinus: m.nodeMinus,
681         };
682     }
683     return out;
684 }
685
686 /**
687  * Courbes voltmètre pendant .tran (animation affichage).
688  * @param {string} waveTxt
689  * @param {{ id: string; wrVarCount?: number; channel: object; nodePlus?: string; nodeMinus?: string }[]} meta
690  */
691 export function mergeVoltmeterTranPlotsFromWrdata(waveTxt, meta) {
692     const out = {};
693     const rows = parseWrdataNumericRows(waveTxt);
694     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
695
696     for (const m of meta) {
697         if (!m?.id || !m.channel) continue;
698         let wrVarCount = m.wrVarCount || 2;
699         if (m.channel.wrIndex !== null && m.channel.wrIndex + 1 > wrVarCount) {
700             wrVarCount = m.channel.wrIndex + 1;
701         }
702         const ncol = rows[0].length;
703         const colOffset = Math.max(0, ncol - wrVarCount);
704         const timeCol = m.timeCol ?? 0;
705         const tArr = [];
706         const vArr = [];
707         for (const row of rows) {
708             if (row.length <= timeCol) continue;
709             const d = channelDiffFromRow(row, m.channel, colOffset);
710             if (!Number.isFinite(row[timeCol]) || !Number.isFinite(d)) continue;
711             tArr.push(row[timeCol]);
712             vArr.push(d);
713         }
714         if (!tArr.length) continue;
715         out[m.id] = {

```

```

716         time: tArr,
717         voltage: vArr,
718         nodePlus: m.nodePlus,
719         nodeMinus: m.nodeMinus,
720     };
721 }
722 return out;
723 }
724
725 /**
726  * Ampèremètre en .tran : dernier courant mesuré sur la branche VI.
727  * @param {string} waveTxt
728  * @param {{ id: string; wrVarCount?: number; currentWrIndex?: number; branch?: string; nodePlus?: string; nodeMinus?: string }} meta
729  */
730 export function mergeAmmeterFromTranWrdata(waveTxt, meta) {
731     const out = {};
732     const rows = parseWrdataNumericRows(waveTxt);
733     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
734
735     for (const m of meta) {
736         if (!m?.id || m.currentWrIndex == null || m.currentWrIndex === undefined) continue;
737         const wrVarCount = m.wrVarCount || m.currentWrIndex + 1;
738         const ncol = rows[0].length;
739         const colOffset = Math.max(0, ncol - wrVarCount);
740         const vals = [];
741         let peakI = 0;
742         for (const row of rows) {
743             const i = row[m.currentWrIndex + colOffset];
744             if (Number.isFinite(i)) {
745                 vals.push(i);
746                 if (Math.abs(i) > Math.abs(peakI)) peakI = i;
747             }
748         }
749         if (!vals.length) continue;
750         out[m.id] = {
751             current: peakI,
752             unit: "A",
753             branch: m.branch,
754             nodePlus: m.nodePlus,
755             nodeMinus: m.nodeMinus,
756         };
757     }
758     return out;
759 }
760
761 /**
762  * Ohmmètre en .tran :  $R = |V| / I_{\text{test}}$  à la fin de la simulation.
763  * @param {string} waveTxt
764  * @param {{ id: string; wrVarCount?: number; testCurrent?: number; channel: { wrIndex?: number; minusWrIndex?: number | null; minusIsGnd?: boolean }; nodePlus?: string; nodeMinus?: string }} meta
765  */
766 export function mergeOhmmeterFromTranWrdata(waveTxt, meta) {
767     const out = {};
768     const rows = parseWrdataNumericRows(waveTxt);
769     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
770
771     for (const m of meta) {
772         if (!m?.id || !m.channel) continue;
773         const iTest = m.testCurrent > 0 ? m.testCurrent : 0.001;
774         let wrVarCount = m.wrVarCount || 2;
775         if (m.channel.wrIndex !== null && m.channel.wrIndex + 1 > wrVarCount) {
776             wrVarCount = m.channel.wrIndex + 1;
777         }
778         const ncol = rows[0].length;
779         const colOffset = Math.max(0, ncol - wrVarCount);
780         const vals = [];
781         for (const row of rows) {
782             const d = channelDiffFromRow(row, m.channel, colOffset);
783             if (Number.isFinite(d)) vals.push(Math.abs(d));
784         }
785         if (!vals.length) continue;
786         const r = vals[vals.length - 1] / iTest;
787         if (!Number.isFinite(r) || r <= 0) continue;
788         out[m.id] = {
789             resistance: r,
790             unit: "Ohm",
791             nodePlus: m.nodePlus,
792             nodeMinus: m.nodeMinus,
793         };
794     }

```



```

795     return out;
796 }
797
798 /**
799  * Courants LED pendant .tran (wrdata i(VIL_*)).
800  * @param {string} waveTxt
801  * @param {{ id: string; timeCol?: number; wrVarCount?: number; currentWrIndex?: number; branch?: string; nodePlus?: string; nodeMinus?: string }} meta
802  */
803 export function mergeLedTranPlotsFromWrdata(waveTxt, meta) {
804     const out = {};
805     const rows = parseWrdataNumericRows(waveTxt);
806     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
807
808     let wrVarCount = 0;
809     for (const m of meta) {
810         if (m.wrVarCount > wrVarCount) wrVarCount = m.wrVarCount;
811         if (m.currentWrIndex !== null && m.currentWrIndex + 1 > wrVarCount) {
812             wrVarCount = m.currentWrIndex + 1;
813         }
814     }
815     if (wrVarCount < 2) wrVarCount = 2;
816     const ncol = rows[0].length;
817     const colOffset = Math.max(0, ncol - wrVarCount);
818     const timeCol = meta[0]?.timeCol ?? 0;
819
820     for (const m of meta) {
821         if (!m?.id || m.currentWrIndex === null || m.currentWrIndex === undefined) continue;
822         const tArr = [];
823         const iArr = [];
824         for (const row of rows) {
825             if (row.length <= timeCol) continue;
826             const t = row[timeCol];
827             const iCol = m.currentWrIndex + colOffset;
828             if (iCol >= row.length) continue;
829             const iRaw = row[iCol];
830             if (!Number.isFinite(t) || !Number.isFinite(iRaw)) continue;
831             tArr.push(t);
832             iArr.push(iRaw > 0 ? iRaw : 0);
833         }
834         if (!tArr.length) continue;
835         out[m.id] = {
836             time: tArr,
837             current: iArr,
838             branch: m.branch,
839             nodePlus: m.nodePlus,
840             nodeMinus: m.nodeMinus,
841         };
842     }
843     return out;
844 }
845
846 /**
847  * @param {Record<string, { time: number[]; current: number[]; branch?: string; nodePlus?: string; nodeMinus?: string }>}} ledPlots
848  */
849 /**
850  * @param {string} log
851  * @param {{ id: string; nodeOut: string }} logicGates
852  */
853 export function mergeLogicGateMeasurements(log, logicGates) {
854     const out = {};
855     if (!log || typeof log !== "string" || !Array.isArray(logicGates) || logicGates.length === 0) {
856         return out;
857     }
858     const map = collectNodeVoltagesFromLog(log);
859     for (const lg of logicGates) {
860         if (!lg?.id || !lg.nodeOut) continue;
861         const v = nodeVoltageFromMap(lg.nodeOut, map);
862         if (v === null) continue;
863         const th = typeof lg.vth === "number" && lg.vth > 0 ? lg.vth : 2.5;
864         out[lg.id] = {
865             voltage: v,
866             unit: "V",
867             logic: logicLevelFromVoltage(v, th),
868             nodeOut: lg.nodeOut,
869             vhi: lg.vhi,
870             vth: th,
871         };
872     }
873     return out;

```

```

874 }
875
876 /**
877  * @param {string} waveTxt
878  * @param {{ id: string; wrVarCount?: number; wrIndex?: number; nodeOut?: string }[]} meta
879  */
880 export function mergeLogicGateTranFromWrdata(waveTxt, meta) {
881     const out = {};
882     const rows = parseWrdataNumericRows(waveTxt);
883     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
884
885     for (const m of meta) {
886         if (!m?.id || m.wrIndex == null || m.wrIndex === undefined) continue;
887         const wrVarCount = m.wrVarCount || m.wrIndex + 1;
888         const ncol = rows[0].length;
889         const colOffset = Math.max(0, ncol - wrVarCount);
890         const vals = [];
891         for (const row of rows) {
892             const v = row[m.wrIndex + colOffset];
893             if (Number.isFinite(v)) vals.push(v);
894         }
895         if (!vals.length) continue;
896         const v = vals[vals.length - 1];
897         const th = typeof m.vth === "number" && m.vth > 0 ? m.vth : 2.5;
898         out[m.id] = {
899             voltage: v,
900             unit: "V",
901             logic: logicLevelFromVoltage(v, th),
902             nodeOut: m.nodeOut,
903             vhi: m.vhi,
904             vth: th,
905         };
906     }
907     return out;
908 }
909
910 /** Courbes tension complètes (.tran) pour animation des sorties logiques (ex. Q0...Q3 du 74HC90). */
911 export function mergeLogicGateTranPlotsFromWrdata(waveTxt, meta) {
912     const out = {};
913     const rows = parseWrdataNumericRows(waveTxt);
914     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
915
916     for (const m of meta) {
917         if (!m?.id || m.wrIndex == null || m.wrIndex === undefined) continue;
918         const wrVarCount = m.wrVarCount || m.wrIndex + 1;
919         const ncol = rows[0].length;
920         const colOffset = Math.max(0, ncol - wrVarCount);
921         const time = [];
922         const voltage = [];
923         for (const row of rows) {
924             if (!Number.isFinite(row[0])) continue;
925             const v = row[m.wrIndex + colOffset];
926             if (!Number.isFinite(v)) continue;
927             time.push(row[0]);
928             voltage.push(v);
929         }
930         if (!time.length) continue;
931         const th = typeof m.vth === "number" && m.vth > 0 ? m.vth : 2.5;
932         out[m.id] = { time, voltage, vth: th, vhi: m.vhi, nodeOut: m.nodeOut };
933     }
934     return out;
935 }
936
937 export function mergeLedValuesFromTranPlots(ledPlots) {
938     const out = {};
939     if (!ledPlots || typeof ledPlots !== "object") return out;
940     for (const [id, plot] of Object.entries(ledPlots)) {
941         if (!plot?.current?.length) continue;
942         let peak = 0;
943         for (const i of plot.current) {
944             if (Number.isFinite(i) && i > peak) peak = i;
945         }
946         out[id] = {
947             current: peak,
948             unit: "A",
949             branch: plot.branch,
950             nodePlus: plot.nodePlus,
951             nodeMinus: plot.nodeMinus,
952             reverseBias: peak < 5e-7,
953         };

```

```

954     }
955     return out;
956 }
957
958 export function deriveOscilloscopeValuesFromScopePlots(scopePlots) {
959     const out = {};
960     if (!scopePlots || typeof scopePlots !== "object") return out;
961     for (const [id, plot] of Object.entries(scopePlots)) {
962         if (!plot) continue;
963         const row = {};
964         if (plot.ch1 && Array.isArray(plot.ch1.voltage) && plot.ch1.voltage.length) {
965             const vpp = waveformPeakToPeak(plot.ch1.voltage);
966             row.ch1 = {
967                 voltage: vpp,
968                 unit: "V",
969                 measure: "Vpp",
970             };
971         }
972         if (plot.ch2 && Array.isArray(plot.ch2.voltage) && plot.ch2.voltage.length) {
973             const vpp = waveformPeakToPeak(plot.ch2.voltage);
974             row.ch2 = {
975                 voltage: vpp,
976                 unit: "V",
977                 measure: "Vpp",
978             };
979         }
980         if (row.ch1 || row.ch2) out[id] = row;
981     }
982     return out;
983 }
984
985 const SEG7_NAMES = ["a", "b", "c", "d", "e", "f", "g"];
986
987 /** Seuil bas : avec modèle diode (anode→COM), V(segment) ≈ 0,6-1 V quand le segment conduit. */
988 const SEG7_LIT_DELTA_V = 0.35;
989
990 function seg7SegmentsFromVoltages(segmentV, vCom) {
991     const segments = {};
992     const vc = Number.isFinite(vCom) ? vCom : 0;
993     for (let i = 0; i < 7; i++) {
994         const v = segmentV[i];
995         segments[SEG7_NAMES[i]] = Number.isFinite(v) && v - vc >= SEG7_LIT_DELTA_V;
996     }
997     return segments;
998 }
999
1000 export function mergeSeg7Measurements(log, seg7Displays) {
1001     const out = {};
1002     if (!log || !Array.isArray(seg7Displays) || seg7Displays.length === 0) return out;
1003     const vmap = collectNodeVoltagesFromLog(log);
1004     for (const d of seg7Displays) {
1005         if (!d?.id) continue;
1006         const vCom = nodeVoltageFromMap(d.commonNode, vmap) ?? 0;
1007         const segmentV = (d.segmentNodes || []).map((n) => nodeVoltageFromMap(n, vmap));
1008         out[d.id] = { segments: seg7SegmentsFromVoltages(segmentV, vCom) };
1009     }
1010     return out;
1011 }
1012
1013 /** Courbes .tran pour animation 7 segments (tension segment vs cathode commune). */
1014 export function mergeSeg7TranPlotsFromWrdata(waveTxt, meta) {
1015     const out = {};
1016     const rows = parseWrdataNumericRows(waveTxt);
1017     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
1018
1019     for (const m of meta) {
1020         if (!m?.id) continue;
1021         let wrVarCount = m.wrVarCount || 2;
1022         for (const ix of m.segmentWrIndex || []) {
1023             if (ix !== null && ix + 1 > wrVarCount) wrVarCount = ix + 1;
1024         }
1025         if (m.commonWrIndex !== null && m.commonWrIndex + 1 > wrVarCount) {
1026             wrVarCount = m.commonWrIndex + 1;
1027         }
1028         const colOffset = Math.max(0, rows[0].length - wrVarCount);
1029         const timeCol = m.timeCol ?? 0;
1030         const time = [];
1031         const common = [];
1032         const segments = Object.fromEntries(SEG7_NAMES.map((n) => [n, []]));
1033         for (const row of rows) {

```

```

1034         if (row.length <= timeCol || !Number.isFinite(row[timeCol])) continue;
1035         time.push(row[timeCol]);
1036         common.push(
1037             m.commonWrIndex != null ? row[m.commonWrIndex + colOffset] : 0
1038         );
1039         SEG7_NAMES.forEach((name, i) => {
1040             const ix = m.segmentWrIndex?.[i];
1041             segments[name].push(ix != null ? row[ix + colOffset] : NaN);
1042         });
1043     }
1044     if (!time.length) continue;
1045     out[m.id] = { time, common, segments };
1046 }
1047 return out;
1048 }
1049
1050 export function mergeSeg7FromTranWrdata(waveTxt, meta) {
1051     const out = {};
1052     const rows = parseWrdataNumericRows(waveTxt);
1053     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
1054     const last = rows[rows.length - 1];
1055     let wrVarCount = 2;
1056     for (const m of meta) {
1057         if (m.wrVarCount > wrVarCount) wrVarCount = m.wrVarCount;
1058     }
1059     const colOffset = Math.max(0, last.length - wrVarCount);
1060
1061     for (const m of meta) {
1062         if (!m?.id) continue;
1063         const vCom = m.commonWrIndex != null ? last[m.commonWrIndex + colOffset] : 0;
1064         const segmentV = (m.segmentWrIndex || []).map((ix) =>
1065             ix != null ? last[ix + colOffset] : NaN
1066         );
1067         out[m.id] = { segments: seg7SegmentsFromVoltages(segmentV, vCom) };
1068     }
1069     return out;
1070 }
1071
1072 const BARGRAPH_DC10H_SEG_NAMES = Array.from({ length: 10 }, (_, i) => `s${i + 1}`);
1073
1074 function bargraphSegmentsFromVoltages(segmentV, vCom) {
1075     const segments = {};
1076     const vc = Number.isFinite(vCom) ? vCom : 0;
1077     for (let i = 0; i < 10; i++) {
1078         const v = segmentV[i];
1079         segments[BARGRAPH_DC10H_SEG_NAMES[i]] = Number.isFinite(v) && v - vc >= SEG7_LIT_DELTA_V;
1080     }
1081     return segments;
1082 }
1083
1084 export function mergeBargraphMeasurements(log, bargraphDisplays) {
1085     const out = {};
1086     if (!log || !Array.isArray(bargraphDisplays) || bargraphDisplays.length === 0) return out;
1087     const vmap = collectNodeVoltagesFromLog(log);
1088     for (const d of bargraphDisplays) {
1089         if (!d?.id) continue;
1090         const vCom = nodeVoltageFromMap(d.commonNode, vmap) ?? 0;
1091         const segmentV = (d.segmentNodes || []).map((n) => nodeVoltageFromMap(n, vmap));
1092         out[d.id] = { segments: bargraphSegmentsFromVoltages(segmentV, vCom) };
1093     }
1094     return out;
1095 }
1096
1097 export function mergeBargraphTranPlotsFromWrdata(waveTxt, meta) {
1098     const out = {};
1099     const rows = parseWrdataNumericRows(waveTxt);
1100     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
1101
1102     for (const m of meta) {
1103         if (!m?.id) continue;
1104         let wrVarCount = m.wrVarCount || 2;
1105         for (const ix of m.segmentWrIndex || []) {
1106             if (ix != null && ix + 1 > wrVarCount) wrVarCount = ix + 1;
1107         }
1108         if (m.commonWrIndex != null && m.commonWrIndex + 1 > wrVarCount) {
1109             wrVarCount = m.commonWrIndex + 1;
1110         }
1111         const colOffset = Math.max(0, rows[0].length - wrVarCount);
1112         const timeCol = m.timeCol ?? 0;
1113         const time = [];

```

```

1114     const common = [];
1115     const segments = Object.fromEntries(BARGRAPH_DC10H_SEG_NAMES.map((n) => [n, []]));
1116     for (const row of rows) {
1117         if (row.length <= timeCol || !Number.isFinite(row[timeCol])) continue;
1118         time.push(row[timeCol]);
1119         common.push(
1120             m.commonWrIndex != null ? row[m.commonWrIndex + colOffset] : 0
1121         );
1122         BARGRAPH_DC10H_SEG_NAMES.forEach((name, i) => {
1123             const ix = m.segmentWrIndex?.[i];
1124             segments[name].push(ix != null ? row[ix + colOffset] : NaN);
1125         });
1126     }
1127     if (!time.length) continue;
1128     out[m.id] = { time, common, segments };
1129 }
1130 return out;
1131 }
1132
1133 export function mergeBargraphFromTranWrdata(waveTxt, meta) {
1134     const out = {};
1135     const rows = parseWrdataNumericRows(waveTxt);
1136     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
1137     const last = rows[rows.length - 1];
1138     let wrVarCount = 2;
1139     for (const m of meta) {
1140         if (m.wrVarCount > wrVarCount) wrVarCount = m.wrVarCount;
1141     }
1142     const colOffset = Math.max(0, last.length - wrVarCount);
1143
1144     for (const m of meta) {
1145         if (!m?.id) continue;
1146         const vCom = m.commonWrIndex != null ? last[m.commonWrIndex + colOffset] : 0;
1147         const segmentV = (m.segmentWrIndex || []).map((ix) =>
1148             ix != null ? last[ix + colOffset] : NaN
1149         );
1150         out[m.id] = { segments: bargraphSegmentsFromVoltages(segmentV, vCom) };
1151     }
1152     return out;
1153 }
1154
1155 /** Regroupe les métadonnées tran Q0..Q3 d'un 74HC90 (ids « HC902_Q0 » ...). */
1156 export function groupHc90QTranMeta(logicGatesTranMeta) {
1157     const groups = {};
1158     if (!Array.isArray(logicGatesTranMeta)) return groups;
1159     for (const m of logicGatesTranMeta) {
1160         const match = /^(.+)_Q([0-3])$/i.exec(m?.id || "");
1161         if (!match) continue;
1162         const base = match[1];
1163         const qi = Number(match[2]);
1164         if (!groups[base]) groups[base] = [];
1165         groups[base][qi] = m;
1166     }
1167     return groups;
1168 }
1169
1170 /**
1171  * Afficheur 7 seg piloté par les sorties Q d'un 74HC90 (secours si CD4511 éteint / absent).
1172  * @param {string} waveTxt
1173  * @param {{ 0?: object; 1?: object; 2?: object; 3?: object }} qMetaByIndex
1174  * @param {string} seg7Id
1175  */
1176 export function mergeSeg7TranPlotsFromHc90Q(waveTxt, qMetaByIndex, seg7Id) {
1177     const out = {};
1178     if (!seg7Id || !qMetaByIndex?.[0]) return out;
1179     const rows = parseWrdataNumericRows(waveTxt);
1180     if (!rows.length) return out;
1181
1182     let wrVarCount = 2;
1183     for (let i = 0; i < 4; i++) {
1184         const m = qMetaByIndex[i];
1185         if (m?.wrVarCount > wrVarCount) wrVarCount = m.wrVarCount;
1186     }
1187     const colOffset = Math.max(0, (rows[0]?.length || 0) - wrVarCount);
1188     const vth =
1189         typeof qMetaByIndex[0]?.vth === "number" && qMetaByIndex[0].vth > 0
1190             ? qMetaByIndex[0].vth
1191             : 2.5;
1192
1193     const time = [];

```

```

1194     const common = [];
1195     const segments = Object.fromEntries(BCD_SEG7_NAMES.map((n) => [n, []]));
1196     const vOn = 1.2;
1197     const vOff = 0.05;
1198
1199     for (const row of rows) {
1200         if (!Number.isFinite(row[0])) continue;
1201         const qV = [];
1202         for (let i = 0; i < 4; i++) {
1203             const m = qMetaByIndex[i];
1204             const ix = m?.wrIndex;
1205             qV.push(ix != null ? row[ix + colOffset] : 0);
1206         }
1207         const digit = bcdFromQVoltages(qV, vth);
1208         const seg = bcdDigitToSeg7Segments(digit);
1209         time.push(row[0]);
1210         common.push(0);
1211         BCD_SEG7_NAMES.forEach((name) => {
1212             segments[name].push(seg[name] ? vOn : vOff);
1213         });
1214     }
1215     if (!time.length) return out;
1216     out[seg7Id] = { time, common, segments };
1217     return out;
1218 }
1219
1220 /** Dernière valeur BCD → segments (pour affichage statique). */
1221 export function mergeSeg7FromHc90Q(waveTxt, qMetaByIndex, seg7Id) {
1222     const plots = mergeSeg7TranPlotsFromHc90Q(waveTxt, qMetaByIndex, seg7Id);
1223     const plot = plots[seg7Id];
1224     if (!plot?.time?.length) return {};
1225     const last = plot.time.length - 1;
1226     const segmentV = BCD_SEG7_NAMES.map((n) => plot.segments[n][last]);
1227     const vCom = plot.common[last] ?? 0;
1228     return {
1229         [seg7Id]: { segments: seg7SegmentsFromVoltages(segmentV, vCom) },
1230     };
1231 }
1232
1233 /** true si l'afficheur semble éteint (tous segments < seuil). */
1234 export function seg7DisplayAppearsBlank(seg7Entry) {
1235     const seg = seg7Entry?.segments;
1236     if (!seg || typeof seg !== "object") return true;
1237     return !Object.values(seg).some(Boolean);
1238 }
1239
1240 function vdbFromNodeChannel(row, ch) {
1241     if (!ch || ch.isGnd) return 0;
1242     if (ch.dbCol == null) return NaN;
1243     const db = row[ch.dbCol];
1244     return Number.isFinite(db) ? db : NaN;
1245 }
1246
1247 /** Gain en dB : différence des vdb si les références sont sur la masse. */
1248 function gainDbFromChannels(row, outPlus, outMinus, inPlus, inMinus) {
1249     const vdbOut = vdbFromNodeChannel(row, outPlus) - vdbFromNodeChannel(row, outMinus);
1250     const vdbIn = vdbFromNodeChannel(row, inPlus) - vdbFromNodeChannel(row, inMinus);
1251     if (!Number.isFinite(vdbOut) || !Number.isFinite(vdbIn)) return NaN;
1252     return vdbOut - vdbIn;
1253 }
1254
1255 /**
1256  * Fréquences de coupure à -3 dB (interpolation linéaire en échelle log).
1257  * @param {number[]} frequency
1258  * @param {number[]} gainDb
1259  * @returns {number[]}
1260  */
1261 export function computeCutoffFrequencies(frequency, gainDb) {
1262     if (!frequency?.length || frequency.length !== gainDb?.length || frequency.length < 2) return [];
1263     let refGain = gainDb[0];
1264     for (let i = 0; i < Math.min(5, gainDb.length); i++) {
1265         if (Number.isFinite(gainDb[i])) refGain = Math.max(refGain, gainDb[i]);
1266     }
1267     const target = refGain - 3;
1268     const cutoffs = [];
1269     for (let i = 1; i < gainDb.length; i++) {
1270         const g0 = gainDb[i - 1];
1271         const g1 = gainDb[i];
1272         const f0 = frequency[i - 1];
1273         const f1 = frequency[i];

```

```

1274     if (!Number.isFinite(g0) || !Number.isFinite(g1) || f0 <= 0 || f1 <= 0) continue;
1275     const crossesDown = g0 >= target && g1 < target;
1276     const crossesUp = g0 < target && g1 >= target;
1277     if (!crossesDown && !crossesUp) continue;
1278     const t = (target - g0) / (g1 - g0);
1279     const logF = Math.log10(f0) + t * (Math.log10(f1) - Math.log10(f0));
1280     cutoffs.push(Math.pow(10, logF));
1281 }
1282 return cutoffs;
1283 }
1284
1285 /**
1286  * @param {string} waveTxt - sortie ngspice wrdata (analyse .ac)
1287  * @param {{ id: string; freqCol?: number; fMin?: number; fMax?: number; outPlus: object; outMinus: object; inPlus: object; inMinus: object }
1288  []} meta
1289  */
1289 export function mergeBodePlotsFromAcWrdata(waveTxt, meta) {
1290     const out = {};
1291     const rows = parseWrdataNumericRows(waveTxt);
1292     if (!rows.length || !Array.isArray(meta) || meta.length === 0) return out;
1293
1294     for (const m of meta) {
1295         if (!m?.id) continue;
1296         const freqCol = m.freqCol ?? 0;
1297         const frequency = [];
1298         const gainDb = [];
1299         for (const row of rows) {
1300             if (row.length <= freqCol) continue;
1301             const f = row[freqCol];
1302             if (!Number.isFinite(f) || f <= 0) continue;
1303             const gain = gainDbFromChannels(row, m.outPlus, m.outMinus, m.inPlus, m.inMinus);
1304             if (!Number.isFinite(gain)) continue;
1305             frequency.push(f);
1306             gainDb.push(gain);
1307         }
1308         if (frequency.length === 0) continue;
1309         const cutoffHz = computeCutoffFrequencies(frequency, gainDb);
1310         let responseHint = null;
1311         const gLo = gainDb[0];
1312         const gHi = gainDb[gainDb.length - 1];
1313         if (Number.isFinite(gLo) && Number.isFinite(gHi)) {
1314             if (gHi > gLo + 6) {
1315                 responseHint =
1316                 "Courbe type passe-haut : reliez + sur la jonction R/C (sortie du filtre) et - sur la masse (GND), pas aux bornes de la r
1317                 ésistance.";
1318             } else if (gLo > 1 && gHi < gLo - 6) {
1319                 responseHint = "Filtre passe-bas détecté.";
1320             }
1321         }
1322         out[m.id] = {
1323             frequency,
1324             gainDb,
1325             fMin: m.fMin ?? frequency[0],
1326             fMax: m.fMax ?? frequency[frequency.length - 1],
1327             cutoffHz,
1328             unit: "dB",
1329             label: "Gain",
1330             responseHint,
1331         };
1332     }
1333     return out;
1334 }
1335

```

```

1 /**
2  * Affichage voltmètre : quantification 0/Vhi pour la logique, tensions négatives conservées.
3  * @param {number} v
4  * @param {number[]|null} [samples] échantillons .tran (détection signal logique unipolaire)
5  */
6 export function quantizeVoltmeterReading(v, samples) {
7   if (typeof v !== "number" || !Number.isFinite(v)) return v;
8   const vals = Array.isArray(samples) ? samples.filter(Number.isFinite) : [];
9   if (vals.length >= 4) {
10    const maxV = Math.max(...vals);
11    const minV = Math.min(...vals);
12    // Uniquement signaux unipolaires (0 / Vhi) – pas AC ni DC négatif.
13    if (maxV - minV > 1.5 && minV >= -0.5) {
14      const vhi = maxV >= 3 ? maxV : 5;
15      const vlo = minV <= 0.5 ? 0 : minV;
16      return v >= vhi / 2 ? vhi : vlo;
17    }
18   }
19   if (Math.abs(v) >= 4) return Math.round(v * 10) / 10;
20   if (v >= 0 && v <= 0.5) return 0;
21   return v;
22 }
23

```



```

1 // geometry.js
2 import { circuit, flags, simulationResults, snapToGrid, GRID_SIZE } from './state.js';
3 import { CD4511_JUNC_L, CD4511_JUNC_R, CD4511_PIN_Y, CD4511_HIT_DX, CD4511_HIT_DY } from './cd4511-layout.js';
4 import {
5     IC90_JUNC_L,
6     IC90_JUNC_R,
7     IC90_LEFT_PIN_Y,
8     IC90_RIGHT_PIN_Y,
9     IC90_HIT_DX,
10    IC90_HIT_DY,
11    ic74hc90JunctionToTerminalKey,
12 } from './ic74hc90-layout.js';
13 import {
14     UNO_JUNC_L,
15     UNO_JUNC_R,
16     UNO_LEFT_PINS,
17     UNO_RIGHT_PINS,
18     UNO_LEFT_PIN_Y,
19     UNO_RIGHT_PIN_Y,
20     UNO_HIT_DX,
21     UNO_HIT_DY,
22 } from './arduino-uno-layout.js';
23 import {
24     GROVE_LCD_PINS,
25     GROVE_LCD_PIN_Y,
26     GROVE_LCD_JUNC_X,
27     GROVE_LCD_HIT_DX,
28     GROVE_LCD_HIT_DY,
29 } from './grove-lcd-layout.js';
30 import {
31     GROVE_DHT22_PINS,
32     GROVE_DHT22_PIN_Y,
33     GROVE_DHT22_JUNC_X,
34     GROVE_DHT22_HIT_DX,
35     GROVE_DHT22_HIT_DY,
36 } from './dht22-layout.js';
37 import {
38     DC10H_PIN_Y,
39     DC10H_SEG_NAMES,
40     DC10H_JUNC_L,
41     DC10H_JUNC_R,
42     DC10H_COM_X,
43     DC10H_COM_Y,
44     DC10H_SEL_L,
45     DC10H_SEL_T,
46     DC10H_SEL_W,
47     DC10H_SEL_H,
48     dc10hComX,
49 } from './bargraph-dc10h-layout.js';
50
51 export function isPointOnSegment(px, py, p1, p2, tolerance = 1) {
52     const minX = Math.min(p1.x, p2.x) - tolerance, maxX = Math.max(p1.x, p2.x) + tolerance;
53     const minY = Math.min(p1.y, p2.y) - tolerance, maxY = Math.max(p1.y, p2.y) + tolerance;
54     if (px < minX || px > maxX || py < minY || py > maxY) return false;
55     if (Math.abs(p1.x - p2.x) < 2) return Math.abs(px - p1.x) <= tolerance;
56     if (Math.abs(p1.y - p2.y) < 2) return Math.abs(py - p1.y) <= tolerance;
57     const A = px - p1.x; const B = py - p1.y; const C = p2.x - p1.x; const D = p2.y - p1.y;
58     const dot = A * C + B * D; const lenSq = C * C + D * D;
59     const param = lenSq !== 0 ? dot / lenSq : -1;
60     if (param >= 0 && param <= 1) {
61         const xx = p1.x + param * C; const yy = p1.y + param * D;
62         return Math.hypot(px - xx, py - yy) <= tolerance;
63     }
64     return false;
65 }
66
67 export function findWireIntersection(x, y) {
68     for (let w of circuit.wires) {
69         for (let i = 0; i < w.points.length - 1; i++) {
70             if (isPointOnSegment(x, y, w.points[i], w.points[i+1], 1)) {
71                 return { x: snapToGrid(x), y: snapToGrid(y) };
72             }
73         }
74     }
75     return null;
76 }
77
78 export function getComponentJunctions(comp) {

```

```

79   const list = [];
80   const rad = (comp.rotation || 0) * Math.PI / 180;
81   let localPts = [];
82
83   if ([ 'battery', 'resistor', 'voltmeter', 'ammeter', 'ohmmeter', 'bode_analyzer', 'speaker', 'led', 'push_button' ].includes(comp.type)) {
84     localPts = [{ id: `${comp.label}_in`, x: -40, y: 0 }, { id: `${comp.label}_out`, x: 40, y: 0 }];
85   } else if (comp.type === 'potentiometer') {
86     localPts = [
87       { id: `${comp.label}_in`, x: -40, y: 0 },
88       { id: `${comp.label}_wip`, x: 0, y: -22 },
89       { id: `${comp.label}_out`, x: 40, y: 0 },
90     ];
91   } else if (comp.type === 'switch_spdt') {
92     localPts = [
93       { id: `${comp.label}_a`, x: -40, y: -18 },
94       { id: `${comp.label}_com`, x: -40, y: 0 },
95       { id: `${comp.label}_b`, x: -40, y: 18 },
96     ];
97   } else if (comp.type === 'gimp') {
98     localPts = [
99       { id: `${comp.label}_in`, x: 0, y: 40 },
100      { id: `${comp.label}_out`, x: comp.flipX ? -40 : 40, y: 0 },
101    ];
102   } else if (comp.type === 'gsin') {
103     localPts = [
104       { id: `${comp.label}_in`, x: 0, y: 40 },
105       { id: `${comp.label}_out`, x: 40, y: 0 },
106     ];
107   } else if (comp.type === 'oscilloscope') {
108     localPts = [
109       { id: `${comp.label}_CH1`, x: -60, y: -20 },
110       { id: `${comp.label}_CH2`, x: -60, y: 20 },
111       { id: `${comp.label}_GND`, x: 0, y: 60 },
112     ];
113   } else if (comp.type === 'seg7') {
114     const ys = [-60, -40, -20, 0, 20, 40, 60];
115     const names = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
116     localPts = names.map((n, i) => ({ id: `${comp.label}_${n}`, x: -40, y: ys[i] }));
117     localPts.push({ id: `${comp.label}_COM`, x: 20, y: 100 });
118   } else if (comp.type === 'bargraph_dc10h') {
119     const jx = comp.flipX ? DC10H_JUNC_R : DC10H_JUNC_L;
120     const comX = dc10hComX(!comp.flipX);
121     localPts = DC10H_SEG_NAMES.map((n, i) => ({ id: `${comp.label}_${n}`, x: jx, y: DC10H_PIN_Y[i] }));
122     localPts.push({ id: `${comp.label}_COM`, x: comX, y: DC10H_COM_Y });
123   } else if (comp.type === 'grove_lcd16x2') {
124     const jx = comp.flipX ? -GROVE_LCD_JUNC_X : GROVE_LCD_JUNC_X;
125     GROVE_LCD_PINS.forEach((n, i) => {
126       list.push({
127         id: `${comp.label}_${n}`,
128         x: snapToGrid(comp.x + jx),
129         y: snapToGrid(comp.y + GROVE_LCD_PIN_Y[i]),
130       });
131     });
132     return list;
133   } else if (comp.type === 'grove_dht22') {
134     const jx = comp.flipX ? -GROVE_DHT22_JUNC_X : GROVE_DHT22_JUNC_X;
135     GROVE_DHT22_PINS.forEach((n, i) => {
136       list.push({
137         id: `${comp.label}_${n}`,
138         x: snapToGrid(comp.x + jx),
139         y: snapToGrid(comp.y + GROVE_DHT22_PIN_Y[i]),
140       });
141     });
142     return list;
143   } else if ([ 'capacitor', 'inductor', 'diode' ].includes(comp.type)) {
144     localPts = [{ id: `${comp.label}_in`, x: -40, y: 0 }, { id: `${comp.label}_out`, x: 40, y: 0 }];
145   } else if (comp.type === 'gsqr') {
146     localPts = [
147       { id: `${comp.label}_in`, x: 0, y: 40 },
148       { id: `${comp.label}_out`, x: 40, y: 0 },
149     ];
150   } else if (comp.type === 'npn') {
151     localPts = [
152       { id: `${comp.label}_B`, x: -40, y: 0 },
153       { id: `${comp.label}_C`, x: 20, y: -40 },
154       { id: `${comp.label}_E`, x: 20, y: 40 },
155     ];
156   } else if (comp.type === 'opamp') {
157     localPts = [
158       { id: `${comp.label}_plus`, x: -40, y: -20 },

```

```

159         { id: `${comp.label}_minus`, x: -40, y: 20 },
160         { id: `${comp.label}_out`, x: 40, y: 0 },
161     ];
162     } else if ([ 'and', 'nand', 'or', 'nor', 'xor', 'xnor' ].includes(comp.type)) {
163         localPts = [{ id: `${comp.label}_inA`, x: -40, y: -20 }, { id: `${comp.label}_inB`, x: -40, y: 20 }, { id: `${comp.label}_out`, x: 40, y: 0 }];
164     } else if (comp.type === 'not') {
165         localPts = [{ id: `${comp.label}_inA`, x: -40, y: 0 }, { id: `${comp.label}_out`, x: 40, y: 0 }];
166     } else if (comp.type === 'd_flipflop') {
167         localPts = [
168             { id: `${comp.label}_D`, x: -40, y: -20 },
169             { id: `${comp.label}_CLK`, x: -40, y: 20 },
170             { id: `${comp.label}_Q`, x: 40, y: -20 },
171             { id: `${comp.label}_Qbar`, x: 40, y: 20 },
172             { id: `${comp.label}_SET`, x: 0, y: -60 },
173             { id: `${comp.label}_RESET`, x: 0, y: 60 }
174         ];
175     } else if (comp.type === 'jk_flipflop') {
176         localPts = [
177             { id: `${comp.label}_J`, x: -40, y: -20 },
178             { id: `${comp.label}_CLK`, x: -40, y: 0 },
179             { id: `${comp.label}_K`, x: -40, y: 20 },
180             { id: `${comp.label}_Q`, x: 40, y: -20 },
181             { id: `${comp.label}_Qbar`, x: 40, y: 20 },
182             { id: `${comp.label}_SET`, x: 0, y: -60 },
183             { id: `${comp.label}_RESET`, x: 0, y: 60 }
184         ];
185     } else if (comp.type === 'cd4511') {
186         const inNames = ['A', 'B', 'C', 'D', 'LE', 'BI', 'LT'];
187         localPts = inNames.map((n, i) => ({ id: `${comp.label}_${n}`, x: CD4511_JUNC_L, y: CD4511_PIN_Y[i] }));
188         const outNames = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
189         outNames.forEach((n, i) => {
190             localPts.push({ id: `${comp.label}_${n}`, x: CD4511_JUNC_R, y: CD4511_PIN_Y[i] });
191         });
192     } else if (comp.type === 'ic_74hc90') {
193         const left = ['CP1', 'MR1', 'MR2', null, 'VCC', 'MS1', 'MS2'];
194         const right = ['Q0', 'Q1', 'Q2', 'Q3', null, 'GND', 'CP0'];
195         left.forEach((n, i) => {
196             if (n) localPts.push({ id: `${comp.label}_${n}`, x: IC90_JUNC_L, y: IC90_LEFT_PIN_Y[i] });
197         });
198         right.forEach((n, i) => {
199             if (n) localPts.push({ id: `${comp.label}_${n}`, x: IC90_JUNC_R, y: IC90_RIGHT_PIN_Y[i] });
200         });
201     } else if (comp.type === 'arduino_uno') {
202         UNO_LEFT_PINS.forEach((n, i) => {
203             localPts.push({ id: `${comp.label}_${n}`, x: UNO_JUNC_L, y: UNO_LEFT_PIN_Y[i] });
204         });
205         UNO_RIGHT_PINS.forEach((n, i) => {
206             localPts.push({ id: `${comp.label}_${n}`, x: UNO_JUNC_R, y: UNO_RIGHT_PIN_Y[i] });
207         });
208     } else if ([ 'gnd', 'vcc', 'logic_terminal' ].includes(comp.type)) {
209         localPts = [{ id: `${comp.label}_out`, x: GRID_SIZE, y: 0 }];
210     }
211
212     localPts.forEach(pt => {
213         let lx = pt.x;
214         let ly = pt.y;
215         if (comp.type !== 'gimp' && comp.type !== 'gsin' && comp.type !== 'gsqr' && comp.type !== 'oscilloscope' && comp.type !== 'd_flipflop' && comp.type !== 'jk_flipflop' && comp.type !== 'cd4511' && comp.type !== 'ic_74hc90' && comp.type !== 'arduino_uno' && comp.type !== 'npn' && comp.type !== 'opamp' && comp.type !== 'seg7' && comp.type !== 'bargraph_dc10h' && comp.type !== 'grove_lcd16x2' && comp.type !== 'grove_dht22') {
216             const rx = lx * Math.cos(rad) - ly * Math.sin(rad);
217             const ry = lx * Math.sin(rad) + ly * Math.cos(rad);
218             lx = rx;
219             ly = ry;
220         }
221         if (comp.flipX && (comp.type === 'npn' || comp.type === 'opamp')) lx = -lx;
222         if (comp.flipY && comp.type === 'opamp') ly = -ly;
223         list.push({ id: pt.id, x: snapToGrid(comp.x + lx), y: snapToGrid(comp.y + ly) });
224     });
225     return list;
226 }
227
228 /** Zone cliquable pour sélectionner un composant (coords schéma). */
229 export function componentHitTest(comp, mx, my) {
230     const dx = Math.abs(mx - comp.x);
231     const dy = Math.abs(my - comp.y);
232     if (comp.type === 'logic_terminal') return dx < 24 && dy < 14;
233     if (comp.type === 'd_flipflop' || comp.type === 'jk_flipflop') return dx < 45 && dy < 68;
234     if (comp.type === 'cd4511') return dx < CD4511_HIT_DX && dy < CD4511_HIT_DY;
235     if (comp.type === 'ic_74hc90') return dx < IC90_HIT_DX && dy < IC90_HIT_DY;

```

```

236     if (comp.type === 'arduino_uno') return dx < UNO_HIT_DX && dy < UNO_HIT_DY;
237     if (comp.type === 'gimp' || comp.type === 'gsin' || comp.type === 'gsqr') return dx < 45 && dy < 50;
238     if (comp.type === 'oscilloscope') return dx < 52 && dy < 62;
239     if (comp.type === 'seg7') return dx < 60 && dy < 110;
240     if (comp.type === 'bargraph_dc10h') {
241         const x = mx - comp.x;
242         const y = my - comp.y;
243         return x >= DC10H_SEL_L && x <= DC10H_SEL_L + DC10H_SEL_W
244             && y >= DC10H_SEL_T && y <= DC10H_SEL_T + DC10H_SEL_H;
245     }
246     if (comp.type === 'grove_lcd16x2') return dx < GROVE_LCD_HIT_DX && dy < GROVE_LCD_HIT_DY;
247     if (comp.type === 'grove_dht22') return dx < GROVE_DHT22_HIT_DX && dy < GROVE_DHT22_HIT_DY;
248     if (comp.type === 'opamp') return dx < 44 && dy < 42;
249     if (comp.type === 'npn') return dx < 44 && dy < 44;
250     if (comp.type === 'switch_spdt') return dx < 38 && dy < 38;
251     if (comp.type === 'push_button') return dx < 34 && dy < 26;
252     if (comp.type === 'potentiometer') return dx < 34 && dy < 38;
253     return dx < 30 && dy < 30;
254 }
255
256 export function compLocalCoords(comp, mx, my) {
257     const dx = mx - comp.x;
258     const dy = my - comp.y;
259     const rad = -(comp.rotation || 0) * Math.PI / 180;
260     return {
261         x: dx * Math.cos(rad) - dy * Math.sin(rad),
262         y: dx * Math.sin(rad) + dy * Math.cos(rad),
263     };
264 }
265
266 /** Clic sur ◀ / ▶ du potentiomètre. */
267 export function potentiometerControlHit(comp, mx, my) {
268     if (comp.type !== 'potentiometer') return null;
269     const { x, y } = compLocalCoords(comp, mx, my);
270     if (x >= -16 && x <= -2 && y >= 11 && y <= 25) return 'dec';
271     if (x >= 2 && x <= 16 && y >= 11 && y <= 25) return 'inc';
272     return null;
273 }
274
275 /** Zone du levier (clic = bascule A ↔ B). */
276 export function switchSpdtToggleHit(comp, mx, my) {
277     if (comp.type !== 'switch_spdt') return false;
278     const { x, y } = compLocalCoords(comp, mx, my);
279     return x >= -12 && x <= 12 && y >= -18 && y <= 18;
280 }
281
282 /** Zone du capuchon du bouton poussoir (clic = appui ↔ relâche). */
283 export function pushButtonToggleHit(comp, mx, my) {
284     if (comp.type !== 'push_button') return false;
285     const { x, y } = compLocalCoords(comp, mx, my);
286     return x >= -16 && x <= 16 && y >= -20 && y <= 16;
287 }
288
289 export function isJunctionConnected(junctionId) {
290     return circuit.wires.some(w => w.fromJunctionId === junctionId || w.toJunctionId === junctionId);
291 }
292
293 export function getVoltageAtJunction(targetJunctionId) {
294     if (!flags.isSimulating || !simulationResults.voltmeters) return 0;
295     let openList = [targetJunctionId];
296     let connectedJunctions = new Set(openList);
297
298     while (openList.length > 0) {
299         let current = openList.pop();
300         for (let w of circuit.wires) {
301             if (w.fromJunctionId === current && !connectedJunctions.has(w.toJunctionId)) {
302                 connectedJunctions.add(w.toJunctionId); openList.push(w.toJunctionId);
303             }
304             if (w.toJunctionId === current && !connectedJunctions.has(w.fromJunctionId)) {
305                 connectedJunctions.add(w.fromJunctionId); openList.push(w.fromJunctionId);
306             }
307         }
308     }
309
310     for (let jId of connectedJunctions) {
311         if (jId.startsWith('GND')) return 0;
312         for (let vName in simulationResults.voltmeters) {
313             let measureData = simulationResults.voltmeters[vName];
314             let v = (typeof measureData === 'object') ? measureData.voltage : measureData;
315             if (jId === `${vName}_in`) return v || 0;

```

```
316     if (jId === `${vName}_out`) return 0;
317   }
318   for (let comp of circuit.components) {
319     if (comp.label && jId.startsWith(comp.label)) {
320       if (comp.type === 'vcc') return comp.value !== undefined ? comp.value : 5;
321       if (comp.type === 'battery' && jId.endsWith('_in')) return comp.value !== undefined ? comp.value : 5;
322     }
323   }
324 }
325 return 0;
326 }
```

```
1 export {  
2   isGimpPanelOpen,  
3   getGimpPanelHeight,  
4   openGimpPanel,  
5   closeGimpPanel,  
6   onGimpRemoved,  
7   formatGimpLabel,  
8   initGimpPanel,  
9 } from './source-panel.js';  
10
```

```

1  /** Grove LCD RGB 2x16 I2C – géométrie calée grille, centré sur comp.x/y (origine = centre du boîtier). */
2  import { GRID_SIZE as G } from './state.js';
3
4  export const GROVE_LCD_COLS = 16;
5  export const GROVE_LCD_ROWS = 2;
6  export const GROVE_LCD_DEFAULT_I2C = 0x3e;
7
8  export const GROVE_LCD_PINS = ['SDA', 'SCL', 'VCC', 'GND'];
9  export const GROVE_LCD_PIN_COUNT = GROVE_LCD_PINS.length;
10
11 /** Boîtier : 12 pas (long) × 4 pas (haut). */
12 export const GROVE_LCD_BOX_W = 12 * G;
13 export const GROVE_LCD_BOX_H = 4 * G;
14 export const GROVE_LCD_BOX_L = -GROVE_LCD_BOX_W / 2;
15 export const GROVE_LCD_BOX_R = GROVE_LCD_BOX_W / 2;
16
17 /** Translation Y commune : caler les 4 jonctions sur la grille sans déformer le bloc. */
18 const GROVE_LCD_Y_NUDGE = G / 2;
19
20 /** 4 broches équidistantes (pas = G), centrées verticalement sur le boîtier. */
21 export const GROVE_LCD_PIN_Y = [-1.5, -0.5, 0.5, 1.5].map((f) => f * G + GROVE_LCD_Y_NUDGE);
22
23 /** Boîtier centré sur le faisceau de 4 fils. */
24 const _wireCenterY = (GROVE_LCD_PIN_Y[0] + GROVE_LCD_PIN_Y[GROVE_LCD_PIN_COUNT - 1]) / 2;
25 export const GROVE_LCD_BOX_T = _wireCenterY - GROVE_LCD_BOX_H / 2;
26 export const GROVE_LCD_BOX_B = _wireCenterY + GROVE_LCD_BOX_H / 2;
27
28 /** Connecteur Grove : 1 pas de large, bord gauche du boîtier. */
29 export const GROVE_LCD_CONNECTOR_W = G;
30 export const GROVE_LCD_CONN_L = GROVE_LCD_BOX_L;
31
32 /** Pattes : 4 pas vers la gauche (jonctions sur la grille). */
33 export const GROVE_LCD_STUB_LEN = 4 * G;
34 export const GROVE_LCD_JUNC_X = GROVE_LCD_CONN_L - GROVE_LCD_STUB_LEN;
35
36 export const GROVE_LCD_BEZEL = 4;
37 export const GROVE_LCD_SEL_L = GROVE_LCD_JUNC_X - 4;
38 export const GROVE_LCD_SEL_T = GROVE_LCD_BOX_T - 4;
39 export const GROVE_LCD_SEL_W = GROVE_LCD_BOX_R - GROVE_LCD_SEL_L + 4;
40 export const GROVE_LCD_SEL_H = GROVE_LCD_BOX_H + 8;
41
42 export const GROVE_LCD_PIN_LABEL_X = (GROVE_LCD_JUNC_X + GROVE_LCD_CONN_L) / 2;
43
44 export const GROVE_LCD_HIT_DX = Math.max(-GROVE_LCD_SEL_L, GROVE_LCD_BOX_R) + 4;
45 export const GROVE_LCD_HIT_DY = GROVE_LCD_BOX_H / 2 + 16;
46
47 /** Zone verte (écran) : à droite du connecteur, dans le cadre. */
48 export const GROVE_LCD_SCREEN_L = GROVE_LCD_BOX_L + GROVE_LCD_CONNECTOR_W + GROVE_LCD_BEZEL;
49 export const GROVE_LCD_SCREEN_R = GROVE_LCD_BOX_R - GROVE_LCD_BEZEL;
50 export const GROVE_LCD_SCREEN_T = GROVE_LCD_BOX_T + GROVE_LCD_BEZEL;
51 export const GROVE_LCD_SCREEN_B = GROVE_LCD_BOX_B - GROVE_LCD_BEZEL;
52

```

```

1  /**
2   * CGROM HD44780 – matrice 5×8, ASCII 0x20-0x7F.
3   * 8 octets / caractère, 5 bits significatifs par rangée (bits 4→0, MSB = gauche).
4   * Source : ROM standard LiquidCrystal / Hitachi A00.
5   */
6
7  export const HD44780_CHAR_W = 5;
8  export const HD44780_CHAR_H = 8;
9  export const HD44780_LINE_GAP = 1;
10
11 /** Largeur native : 16 × 5 pixels. Hauteur : 2×8 + 1 interligne. */
12 export const HD44780_NATIVE_W = 80;
13 export const HD44780_NATIVE_H = HD44780_CHAR_H * 2 + HD44780_LINE_GAP;
14
15 const FONT = new Uint8Array([
16   0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // 0x20
17   0x00, 0x00, 0x5f, 0x00, 0x00, 0x00, 0x00, 0x00, // !
18   0x00, 0x07, 0x00, 0x07, 0x00, 0x00, 0x00, 0x00, // "
19   0x14, 0x7f, 0x14, 0x7f, 0x14, 0x00, 0x00, 0x00, // #
20   0x24, 0x2a, 0x7f, 0x2a, 0x12, 0x00, 0x00, 0x00, // $
21   0x00, 0x23, 0x13, 0x08, 0x12, 0x11, 0x00, 0x00, // % (0x25 – ROM A00, bas corrigé pour 5 bits)
22   0x36, 0x49, 0x55, 0x22, 0x50, 0x00, 0x00, 0x00, // &
23   0x00, 0x05, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00, // '
24   0x00, 0x1c, 0x22, 0x41, 0x00, 0x00, 0x00, 0x00, // (
25   0x00, 0x41, 0x22, 0x1c, 0x00, 0x00, 0x00, 0x00, // )
26   0x08, 0x2a, 0x1c, 0x2a, 0x08, 0x00, 0x00, 0x00, // *
27   0x08, 0x08, 0x3e, 0x08, 0x08, 0x00, 0x00, 0x00, // +
28   0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x08, // ,
29   0x08, 0x08, 0x08, 0x08, 0x08, 0x00, 0x00, 0x00, // -
30   0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0c, // .
31   0x20, 0x10, 0x08, 0x04, 0x02, 0x00, 0x00, 0x00, // /
32   0x0e, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0e, 0x00, // 0
33   0x04, 0x06, 0x04, 0x04, 0x04, 0x04, 0x0e, 0x00, // 1
34   0x0e, 0x11, 0x01, 0x02, 0x04, 0x08, 0x1f, 0x00, // 2
35   0x1f, 0x01, 0x02, 0x06, 0x01, 0x11, 0x0e, 0x00, // 3
36   0x02, 0x06, 0x0a, 0x12, 0x1f, 0x02, 0x02, 0x00, // 4
37   0x1f, 0x10, 0x1e, 0x01, 0x01, 0x11, 0x0e, 0x00, // 5
38   0x06, 0x08, 0x10, 0x1e, 0x11, 0x11, 0x0e, 0x00, // 6
39   0x1f, 0x01, 0x02, 0x04, 0x08, 0x08, 0x08, 0x00, // 7
40   0x0e, 0x11, 0x11, 0x0e, 0x11, 0x11, 0x0e, 0x00, // 8
41   0x0e, 0x11, 0x11, 0x0f, 0x01, 0x02, 0x0c, 0x00, // 9
42   0x00, 0x36, 0x36, 0x00, 0x00, 0x00, 0x00, 0x00, // :
43   0x00, 0x56, 0x36, 0x00, 0x00, 0x00, 0x00, 0x00, // ;
44   0x00, 0x08, 0x14, 0x22, 0x41, 0x00, 0x00, 0x00, // <
45   0x14, 0x14, 0x14, 0x14, 0x14, 0x00, 0x00, 0x00, // =
46   0x00, 0x41, 0x22, 0x14, 0x08, 0x00, 0x00, 0x00, // >
47   0x0e, 0x11, 0x01, 0x02, 0x04, 0x00, 0x04, 0x00, // ?
48   0x0e, 0x11, 0x17, 0x15, 0x17, 0x10, 0x0e, 0x00, // @
49   0x0e, 0x11, 0x11, 0x11, 0x1f, 0x11, 0x11, 0x00, // A
50   0x1e, 0x11, 0x11, 0x1e, 0x11, 0x11, 0x1e, 0x00, // B
51   0x0e, 0x11, 0x10, 0x10, 0x10, 0x11, 0x0e, 0x00, // C
52   0x1e, 0x11, 0x11, 0x11, 0x11, 0x11, 0x1e, 0x00, // D
53   0x1f, 0x10, 0x10, 0x1e, 0x10, 0x10, 0x1f, 0x00, // E
54   0x1f, 0x10, 0x10, 0x1e, 0x10, 0x10, 0x10, 0x00, // F
55   0x0e, 0x11, 0x10, 0x17, 0x11, 0x11, 0x0f, 0x00, // G
56   0x11, 0x11, 0x11, 0x1f, 0x11, 0x11, 0x11, 0x00, // H
57   0x0e, 0x04, 0x04, 0x04, 0x04, 0x04, 0x0e, 0x00, // I
58   0x07, 0x02, 0x02, 0x02, 0x02, 0x12, 0x0c, 0x00, // J
59   0x11, 0x12, 0x14, 0x18, 0x14, 0x12, 0x11, 0x00, // K
60   0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x1f, 0x00, // L
61   0x11, 0x1b, 0x15, 0x11, 0x11, 0x11, 0x11, 0x00, // M
62   0x11, 0x11, 0x19, 0x15, 0x13, 0x11, 0x11, 0x00, // N
63   0x0e, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0e, 0x00, // O
64   0x1e, 0x11, 0x11, 0x1e, 0x10, 0x10, 0x10, 0x00, // P
65   0x0e, 0x11, 0x11, 0x11, 0x15, 0x12, 0x0d, 0x00, // Q
66   0x1e, 0x11, 0x11, 0x1e, 0x14, 0x12, 0x11, 0x00, // R
67   0x0e, 0x11, 0x10, 0x0e, 0x01, 0x11, 0x0e, 0x00, // S
68   0x1f, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x00, // T
69   0x11, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0e, 0x00, // U
70   0x11, 0x11, 0x11, 0x11, 0x11, 0x0a, 0x04, 0x00, // V
71   0x11, 0x11, 0x11, 0x15, 0x15, 0x1b, 0x11, 0x00, // W
72   0x11, 0x11, 0x0a, 0x04, 0x0a, 0x11, 0x11, 0x00, // X
73   0x11, 0x11, 0x0a, 0x04, 0x04, 0x04, 0x04, 0x00, // Y
74   0x1f, 0x01, 0x02, 0x04, 0x08, 0x10, 0x1f, 0x00, // Z
75   0x0e, 0x08, 0x08, 0x08, 0x08, 0x08, 0x0e, 0x00, // [
76   0x00, 0x10, 0x08, 0x04, 0x02, 0x01, 0x00, 0x00, // \
77   0x0e, 0x02, 0x02, 0x02, 0x02, 0x0e, 0x00, // ]
78   0x04, 0x0a, 0x11, 0x00, 0x00, 0x00, 0x00, 0x00, // ^

```



```
79 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0x00, // _
80 0x08, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // `
81 0x00, 0x00, 0x0e, 0x01, 0x0f, 0x11, 0x0f, 0x00, // a
82 0x10, 0x10, 0x16, 0x19, 0x11, 0x11, 0x1e, 0x00, // b
83 0x00, 0x00, 0x0e, 0x10, 0x10, 0x11, 0x0e, 0x00, // c
84 0x01, 0x01, 0x0d, 0x13, 0x11, 0x11, 0x0f, 0x00, // d
85 0x00, 0x00, 0x0e, 0x11, 0x1f, 0x10, 0x0e, 0x00, // e
86 0x06, 0x09, 0x08, 0x1c, 0x08, 0x08, 0x08, 0x00, // f
87 0x00, 0x0f, 0x11, 0x11, 0x0f, 0x01, 0x0e, 0x00, // g
88 0x10, 0x10, 0x16, 0x19, 0x11, 0x11, 0x11, 0x00, // h
89 0x04, 0x00, 0x0c, 0x04, 0x04, 0x04, 0x0e, 0x00, // i
90 0x02, 0x00, 0x06, 0x02, 0x02, 0x12, 0x0c, 0x00, // j
91 0x10, 0x10, 0x12, 0x14, 0x18, 0x14, 0x12, 0x00, // k
92 0x0c, 0x04, 0x04, 0x04, 0x04, 0x04, 0x0e, 0x00, // l
93 0x00, 0x00, 0x1a, 0x15, 0x15, 0x11, 0x11, 0x00, // m
94 0x00, 0x00, 0x16, 0x19, 0x11, 0x11, 0x11, 0x00, // n
95 0x00, 0x00, 0x0e, 0x11, 0x11, 0x11, 0x0e, 0x00, // o
96 0x00, 0x00, 0x1e, 0x11, 0x11, 0x1e, 0x10, 0x10, // p
97 0x00, 0x00, 0x0d, 0x13, 0x11, 0x0f, 0x01, 0x01, // q
98 0x00, 0x00, 0x16, 0x19, 0x10, 0x10, 0x10, 0x00, // r
99 0x00, 0x00, 0x0e, 0x10, 0x0e, 0x01, 0x1e, 0x00, // s
100 0x08, 0x08, 0x1c, 0x08, 0x08, 0x09, 0x06, 0x00, // t
101 0x00, 0x00, 0x11, 0x11, 0x11, 0x13, 0x0d, 0x00, // u
102 0x00, 0x00, 0x11, 0x11, 0x11, 0x0a, 0x04, 0x00, // v
103 0x00, 0x00, 0x11, 0x11, 0x15, 0x15, 0x0a, 0x00, // w
104 0x00, 0x00, 0x11, 0x0a, 0x04, 0x0a, 0x11, 0x00, // x
105 0x00, 0x00, 0x11, 0x11, 0x0f, 0x01, 0x0e, 0x00, // y
106 0x00, 0x00, 0x1f, 0x02, 0x04, 0x08, 0x1f, 0x00, // z
107 0x02, 0x04, 0x04, 0x08, 0x04, 0x04, 0x02, 0x00, // {
108 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x00, // |
109 0x08, 0x04, 0x04, 0x02, 0x04, 0x04, 0x08, 0x00, // }
110 0x00, 0x08, 0x15, 0x02, 0x00, 0x00, 0x00, 0x00, // ~
111 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // 0x7F DEL
112 });
113
114 const SPACE_GLYPH = new Uint8Array(8);
115
116 /** @returns {Uint8Array} */
117 export function getHd44780Glyph(ch) {
118   const code = ch.charCodeAt(0);
119   if (code >= 0x20 && code <= 0x7f) {
120     const off = (code - 0x20) * 8;
121     return FONT.subarray(off, off + 8);
122   }
123   return SPACE_GLYPH;
124 }
125
```

```

1  /** Géométrie 74HC90 / 74LS90 – boîtier DIP 14 broches (datasheet TI). */
2  import { GRID_SIZE as G } from './state.js';
3
4  export const IC90_BOX_L = -2 * G;
5  export const IC90_BOX_R = 2 * G;
6  export const IC90_BOX_T = -3.5 * G;
7  export const IC90_BOX_B = 3.5 * G;
8  export const IC90_LABEL_L = IC90_BOX_L + G / 2;
9  export const IC90_LABEL_R = IC90_BOX_R - G / 2;
10 export const IC90_JUNC_L = -3 * G;
11 export const IC90_JUNC_R = 3 * G;
12
13 /** Broches 1-7 (gauche) et 14-8 (droite), espacement 1 pas. */
14 export const IC90_LEFT_PIN_Y = [-3, -2, -1, 0, 1, 2, 3].map((n) => n * G);
15 export const IC90_RIGHT_PIN_Y = [-3, -2, -1, 0, 1, 2, 3].map((n) => n * G);
16
17 /** Q0..Q3 empilés (même hauteur que A..D du CD4511) pour câblage direct. */
18 export const IC90_Q_STACK_INDICES = [0, 1, 2, 3];
19
20 /** Numéros de broches DIP 14 (TI 74HC90 / 74LS90). */
21 export const IC90_DIP = {
22   CP1: 1,
23   MR1: 2,
24   MR2: 3,
25   NC4: 4,
26   VCC: 5,
27   MS1: 6,
28   MS2: 7,
29   Q2: 8,
30   Q1: 9,
31   GND: 10,
32   Q3: 11,
33   Q0: 12,
34   NC13: 13,
35   CP0: 14,
36 };
37
38 export const IC90_HIT_DX = 3 * G + G / 2;
39 export const IC90_HIT_DY = 3.5 * G + G / 2;
40
41 /** Indice SPICE (#0..#13) = numéro de broche DIP - 1. */
42 export const IC90_PIN = {
43   CP1: 0,
44   MR1: 1,
45   MR2: 2,
46   NC4: 3,
47   VCC: 4,
48   MS1: 5,
49   MS2: 6,
50   Q2: 7,
51   Q1: 8,
52   GND: 9,
53   Q3: 10,
54   Q0: 11,
55   NC13: 12,
56   CP0: 13,
57 };
58
59 /** Suffixe jonction schéma → indice broche. */
60 export const IC90_JUNCTION_SUFFIX = {
61   CP1: IC90_PIN.CP1,
62   MR1: IC90_PIN.MR1,
63   MR2: IC90_PIN.MR2,
64   VCC: IC90_PIN.VCC,
65   MS1: IC90_PIN.MS1,
66   MS2: IC90_PIN.MS2,
67   Q2: IC90_PIN.Q2,
68   Q1: IC90_PIN.Q1,
69   GND: IC90_PIN.GND,
70   Q3: IC90_PIN.Q3,
71   Q0: IC90_PIN.Q0,
72   CP0: IC90_PIN.CP0,
73 };
74
75 export function ic74hc90JunctionToTerminalKey(label, junctionId) {
76   if (!label || !junctionId?.startsWith(`${label}_`)) return null;
77   const suffix = junctionId.slice(label.length + 1);
78   const idx = IC90_JUNCTION_SUFFIX[suffix];

```

```
79     if (idx === undefined) return null;
80     return `${label}#${idx}`;
81 }
82
83 export function ic74hc90OutputPinIndices() {
84     return [IC90_PIN.Q0, IC90_PIN.Q1, IC90_PIN.Q2, IC90_PIN.Q3];
85 }
86
87 export function ic74hc90InputPinIndices() {
88     return [IC90_PIN.CP1, IC90_PIN.MR1, IC90_PIN.MR2, IC90_PIN.MS1, IC90_PIN.MS2, IC90_PIN.CP0];
89 }
90
```

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Simulateur de Circuits Électroniques</title>
7   <link rel="stylesheet" href="style.css?v=theme4">
8 </head>
9 <body class="theme-dark">
10
11   <header class="navbar">
12     <div class="menu-item">
13       <span class="menu-icon">📁</span> Fichier
14       <div class="dropdown">
15         <div class="dropdown-item" id="btn-new">📄 Nouveau</div>
16         <div class="dropdown-item" id="btn-open">📂 Ouvrir...</div>
17         <div class="dropdown-item" id="btn-save">💾 Enregistrer</div>
18         <div class="dropdown-item" id="btn-save-as">💾 Enregistrer sous...</div>
19       </div>
20     </div>
21
22     <div class="menu-item">
23       <span class="menu-icon">⚙️</span> Éditeur
24       <div class="dropdown">
25         <div class="dropdown-item" id="btn-commands">Commandes</div>
26         <div class="dropdown-item" id="btn-theme-dark">Sombre</div>
27         <div class="dropdown-item" id="btn-theme-light">Clair</div>
28       </div>
29     </div>
30
31     <div class="menu-item">
32       <span class="menu-icon menu-icon--sources" aria-hidden="true"></span> Sources
33       <div class="dropdown">
34         <div class="dropdown-item" draggable="true" data-component="battery">Pile DC</div>
35         <div class="dropdown-item" draggable="true" data-component="vcc">Alimentation VCC</div>
36         <div class="dropdown-item" draggable="true" data-component="gnd">Masse (0 V)</div>
37         <div class="dropdown-item" draggable="true" data-component="gimp">GImp – horloge / impulsions</div>
38         <div class="dropdown-item" draggable="true" data-component="gsin">Générateur sinusoïdal</div>
39         <div class="dropdown-item" draggable="true" data-component="gsqr">Générateur carré</div>
40       </div>
41     </div>
42
43     <div class="menu-item">
44       <span class="menu-icon">🔍</span> Composants
45       <div class="dropdown">
46         <div class="dropdown-item" draggable="true" data-component="resistor">Résistance</div>
47         <div class="dropdown-item" draggable="true" data-component="capacitor">Condensateur</div>
48         <div class="dropdown-item" draggable="true" data-component="inductor">Bobine</div>
49         <div class="dropdown-item" draggable="true" data-component="diode">Diode</div>
50         <div class="dropdown-item" draggable="true" data-component="potentiometer">Potentiomètre</div>
51         <div class="dropdown-item" draggable="true" data-component="switch_spdt">Interrupteur 3 broches</div>
52         <div class="dropdown-item" draggable="true" data-component="push_button" title="Bouton poussoir – momentané par défaut (fermé pendant l'appui). Double-clic pour passer en mode maintenu (bascule).">Bouton poussoir</div>
53         <div class="dropdown-item" draggable="true" data-component="speaker" title="Charge 8 Ω – son via la carte son (GImp ou circuit importé)">Haut-parleur</div>
54         <div class="dropdown-submenu">
55           <span class="dropdown-item submenu-title">Transistors</span>
56           <div class="submenu">
57             <div class="dropdown-item" draggable="true" data-component="npn">2N2222</div>
58           </div>
59         </div>
60         <div class="dropdown-submenu">
61           <span class="dropdown-item submenu-title">AOP</span>
62           <div class="submenu">
63             <div class="dropdown-item" draggable="true" data-component="opamp">uA741</div>
64           </div>
65         </div>
66       </div>
67     </div>
68
69     <div class="menu-item">
70       <span class="menu-icon">💡</span> Logique
71       <div class="dropdown">
72         <div class="dropdown-item" draggable="true" data-component="logic_terminal">Borne Numérique (0/1)</div>
73         <div class="dropdown-submenu">
74           <span class="dropdown-item submenu-title">Portes</span>
75           <div class="submenu">
76             <div class="dropdown-item" draggable="true" data-component="not">Porte NOT (NON)</div>

```

```

77         <div class="dropdown-item" draggable="true" data-component="and">Porte AND (ET)</div>
78         <div class="dropdown-item" draggable="true" data-component="nand">Porte NAND (NON-ET)</div>
79         <div class="dropdown-item" draggable="true" data-component="or">Porte OR (OU)</div>
80         <div class="dropdown-item" draggable="true" data-component="nor">Porte NOR (NON-OU)</div>
81         <div class="dropdown-item" draggable="true" data-component="xor">Porte XOR</div>
82         <div class="dropdown-item" draggable="true" data-component="xnor">Porte XNOR</div>
83     </div>
84 </div>
85     <div class="dropdown-item" draggable="true" data-component="d_flipflop">Bascule D (Flip-Flop)</div>
86     <div class="dropdown-item" draggable="true" data-component="jk_flipflop">Bascule JK (Flip-Flop)</div>
87     <div class="dropdown-submenu">
88         <span class="dropdown-item submenu-title">Circuits</span>
89         <div class="submenu">
90             <div class="dropdown-item" draggable="true" data-component="cd4511">CD4511 (BCD → 7 seg)</div>
91             <div class="dropdown-item" draggable="true" data-component="ic_74hc90">74HC90 (compteur décade)</div>
92         </div>
93     </div>
94 </div>
95 </div>
96
97 <div class="menu-item">
98     <span class="menu-icon">📌</span> Arduino
99     <div class="dropdown">
100         <div class="dropdown-item" draggable="true" data-component="arduino_uno">Arduino UNO R3</div>
101         <div class="dropdown-item dropdown-item--action" id="menu-arduino-editor">Éditeur de programme</div>
102         <div class="dropdown-submenu">
103             <span class="dropdown-item submenu-title">Capteurs Grove</span>
104             <div class="submenu">
105                 <div class="dropdown-item" draggable="true" data-component="grove_dht22">Grove DHT22 (T° / humidité)</div>
106             </div>
107         </div>
108         <div class="dropdown-submenu">
109             <span class="dropdown-item submenu-title">Affichage</span>
110             <div class="submenu">
111                 <div class="dropdown-item" draggable="true" data-component="grove_lcd16x2">Grove LCD 2×16 I2C</div>
112             </div>
113         </div>
114     </div>
115 </div>
116
117 <div class="menu-item">
118     <span class="menu-icon">🔦</span> Opto
119     <div class="dropdown">
120         <div class="dropdown-item" draggable="true" data-component="led">LED (Rouge)</div>
121         <div class="dropdown-item" draggable="true" data-component="seg7">Afficheur 7 segments</div>
122         <div class="dropdown-item" draggable="true" data-component="bargraph_dc10h">Bargraph DC10H</div>
123     </div>
124 </div>
125
126 <div class="menu-item">
127     <span class="menu-icon">📊</span> Mesures
128     <div class="dropdown">
129         <div class="dropdown-item" draggable="true" data-component="voltmeter">Voltmètre</div>
130         <div class="dropdown-item" draggable="true" data-component="ammeter">Ampèremètre</div>
131         <div class="dropdown-item" draggable="true" data-component="ohmmeter">Ohmmètre</div>
132         <div class="dropdown-item" draggable="true" data-component="oscilloscope">Oscilloscope</div>
133         <div class="dropdown-item" draggable="true" data-component="bode_analyzer" title="Glisser-déposer sur le schéma, puis Lancer
Simulation (avec un générateur Sin)">Analyse fréquentielle</div>
134         <div class="dropdown-item dropdown-item--action" id="menu-serial-monitor" data-action="serial-monitor">Moniteur série</div>
135     </div>
136 </div>
137
138 <div class="simulation-controls">
139     <span id="circuit-name" class="circuit-name" title="Nom du montage">Sans titre</span>
140     <button id="btn-simulate" type="button">Lancer la simulation</button>
141     <button id="btn-stop" type="button" class="disabled">Arrêter</button>
142 </div>
143 </header>
144
145 <div id="app-root" class="app-root">
146 <div class="sim-workspace">
147     <main class="canvas-container">
148         <canvas id="circuitCanvas"></canvas>
149     </main>
150     <aside id="arduino-panel" class="right-panel hidden" aria-label="Éditeur Arduino">
151         <div class="panel-header panel-header--arduino">
152             <span id="arduino-panel-title">Arduino UNO</span>
153             <div class="arduino-panel-header-actions">
154                 <button type="button" id="arduino-panel-doc" class="arduino-btn arduino-btn--small" title="Documentation broches UNO">?</
button>

```

```

155         <button type="button" id="arduino-panel-close" class="panel-close" title="Fermer">&times;</button>
156     </div>
157 </div>
158 <div class="arduino-panel-body">
159     <section class="arduino-editor-section" aria-label="Code Arduino">
160         <label for="arduino-sketch-input" class="arduino-editor-label">Programme (.ino)</label>
161         <div class="arduino-sketch-wrap">
162             <div class="arduino-sketch-grid">
163                 <pre id="arduino-sketch-highlight" class="arduino-sketch-highlight" aria-hidden="true"></pre>
164                 <textarea id="arduino-sketch-input" class="arduino-sketch-input" spellcheck="false" wrap="off" autocomplete="off"
autocorrect="off" autocapitalize="off" aria-label="Code du sketch Arduino"></textarea>
165             </div>
166         </section>
167         <div class="arduino-panel-actions">
168             <button type="button" id="arduino-btn-compile" class="arduino-btn arduino-btn--primary">Compiler (arduino-cli)</button>
169             <button type="button" id="arduino-btn-serial-monitor" class="arduino-btn arduino-btn--small" title="Ouvrir le moniteur sé
rie (TX D1 / RX D0)">Moniteur série</button>
171             <span id="arduino-compile-status" class="arduino-compile-status" aria-live="polite"></span>
172             <button type="button" id="arduino-btn-show-errors" class="arduino-btn arduino-btn--small arduino-btn--errors hidden">Voir
les erreurs</button>
173         </div>
174         <pre id="arduino-compile-log" class="arduino-compile-log hidden" aria-label="Journal de compilation"></pre>
175         <details class="arduino-lib-details">
176             <summary class="arduino-lib-summary">Bibliothèques Arduino</summary>
177             <section class="arduino-lib-section" aria-label="Bibliothèques Arduino">
178                 <div class="arduino-lib-header">
179                     <button type="button" id="arduino-lib-refresh" class="arduino-btn arduino-btn--small" title="Actualiser la list
e">↻ Actualiser</button>
180                 </div>
181                 <p class="arduino-lib-hint">Cliquez sur le nom d'une bibliothèque pour voir les commandes (fenêtre flottante).</p>
182                 <div class="arduino-lib-search-row">
183                     <input type="search" id="arduino-lib-search" class="arduino-lib-search-input"
placeholder="Rechercher (ex. LiquidCrystal I2C)" autocomplete="off">
184                     <button type="button" id="arduino-lib-search-btn" class="arduino-btn arduino-btn--small">Chercher</button>
185                 </div>
186                 <ul id="arduino-lib-search-results" class="arduino-lib-list" aria-label="Résultats de recherche"></ul>
187                 <p class="arduino-lib-subtitle">Installées</p>
188                 <ul id="arduino-lib-installed" class="arduino-lib-list" aria-label="Bibliothèques installées"></ul>
189                 <p id="arduino-lib-msg" class="arduino-lib-msg" aria-live="polite"></p>
190             </section>
191         </details>
192         <p id="arduino-cli-status" class="arduino-cli-status">...</p>
193     </div>
194 </aside>
195 </div>
196
197 <div id="source-panel" class="bottom-panel hidden">
198     <div class="panel-header">
199         <span id="source-panel-title">Source – Réglages</span>
200         <button type="button" id="source-panel-close" class="panel-close" title="Fermer">&times;</button>
201     </div>
202     <div id="gimp-fields" class="panel-body" style="display:flex">
203         <label class="panel-field">
204             <span>Amplitude</span>
205             <select id="gimp-voltage">
206                 <option value="5">0 – 5 V</option>
207                 <option value="3.3">0 – 3,3 V</option>
208             </select>
209         </label>
210         <label class="panel-field">
211             <span>Fréquence</span>
212             <input type="number" id="gimp-freq" min="0.000011574" step="any" value="2" title="Hz – ex. 1 (secondes), 0.01667 (1/min), 0.0
00278 (1/h)">
213             <span class="panel-unit">Hz</span>
214         </label>
215         <label class="panel-field">
216             <span>Horloge</span>
217             <select id="gimp-clock-preset" title="Préréglages pour compteur lent">
218                 <option value="">Manuel</option>
219                 <option value="1">1 Hz – tick seconde</option>
220                 <option value="0.016666666666666666">1/60 Hz – tick minute</option>
221                 <option value="0.00027777777777777778">1/3600 Hz – tick heure</option>
222             </select>
223         </label>
224         <label class="panel-field">
225             <span>Rapport cyclique</span>
226             <input type="number" id="gimp-duty" min="1" max="99" step="1" value="10">
227             <span class="panel-unit">%</span>
228         </label>
229     </div>
230 </div>

```

```

231 <div id="gsin-fields" class="panel-body hidden">
232   <label class="panel-field">
233     <span>Amplitude crête</span>
234     <input type="number" id="gsin-amplitude" min="0" step="any" value="5">
235     <span class="panel-unit">V</span>
236   </label>
237   <label class="panel-field">
238     <span>Fréquence</span>
239     <input type="number" id="gsin-freq" min="0.1" step="any" value="1000">
240     <span class="panel-unit">Hz</span>
241   </label>
242   <label class="panel-field">
243     <span>Offset DC</span>
244     <input type="number" id="gsin-offset" step="any" value="0">
245     <span class="panel-unit">V</span>
246   </label>
247 </div>
248 <div id="gsqr-fields" class="panel-body hidden">
249   <label class="panel-field">
250     <span>Amplitude crête</span>
251     <input type="number" id="gsqr-amplitude" min="0" step="any" value="5">
252     <span class="panel-unit">V</span>
253   </label>
254   <label class="panel-field">
255     <span>Fréquence</span>
256     <input type="number" id="gsqr-freq" min="0.1" step="any" value="1000">
257     <span class="panel-unit">Hz</span>
258   </label>
259   <label class="panel-field">
260     <span>Offset DC</span>
261     <input type="number" id="gsqr-offset" step="any" value="0">
262     <span class="panel-unit">V</span>
263   </label>
264 </div>
265 <div id="scope-fields" class="panel-body hidden">
266   <label class="panel-field">
267     <span>Base de temps</span>
268     <select id="scope-time-div">
269       <option value="0.0005">500 µs/div</option>
270       <option value="0.0002">200 µs/div</option>
271       <option value="0.0001">100 µs/div</option>
272       <option value="0.00005" selected>50 µs/div</option>
273       <option value="0.00001">10 µs/div</option>
274       <option value="0.001">1 ms/div</option>
275       <option value="0.002">2 ms/div</option>
276       <option value="0.005">5 ms/div</option>
277       <option value="0.01">10 ms/div</option>
278       <option value="0.05">50 ms/div</option>
279       <option value="0.1">100 ms/div</option>
280     </select>
281   </label>
282   <label class="panel-field">
283     <span>Ampli vert. CH1</span>
284     <select id="scope-ch1-vdiv">
285       <option value="0.5">0,5 V/div</option>
286       <option value="1" selected>1 V/div</option>
287       <option value="2">2 V/div</option>
288       <option value="5">5 V/div</option>
289     </select>
290   </label>
291   <label class="panel-field">
292     <span>Ampli vert. CH2</span>
293     <select id="scope-ch2-vdiv">
294       <option value="0.5">0,5 V/div</option>
295       <option value="1" selected>1 V/div</option>
296       <option value="2">2 V/div</option>
297       <option value="5">5 V/div</option>
298     </select>
299   </label>
300   <label class="panel-field panel-field-range">
301     <span>Position horiz.</span>
302     <input type="range" id="scope-time-pos" min="-4" max="4" step="0.1" value="0">
303     <output id="scope-time-pos-val">0,0 div</output>
304   </label>
305   <label class="panel-field panel-field-range">
306     <span>Position CH1</span>
307     <input type="range" id="scope-ch1-pos" min="-4" max="4" step="0.1" value="0">
308     <output id="scope-ch1-pos-val">0,0 div</output>
309   </label>
310   <label class="panel-field panel-field-range">

```

```

311         <span>Position CH2</span>
312         <input type="range" id="scope-ch2-pos" min="-4" max="4" step="0.1" value="0">
313         <output id="scope-ch2-pos-val">0,0 div</output>
314     </label>
315     <button type="button" id="scope-open-window" class="panel-action-btn">🔍 Afficher fenêtre scope</button>
316 </div>
317 </div>
318
319 </div>
320
321 <div id="scope-popup" class="scope-floating hidden">
322     <div class="scope-floating-header" id="scope-popup-drag-handle">
323         <span id="scope-popup-title">Oscilloscope</span>
324         <button type="button" id="scope-popup-close" class="panel-close" title="Fermer">&times;</button>
325     </div>
326     <canvas id="scope-popup-canvas" width="640" height="360"></canvas>
327 </div>
328
329 <div id="serial-monitor-popup" class="scope-floating serial-monitor-floating hidden">
330     <div class="scope-floating-header" id="serial-monitor-drag-handle">
331         <span id="serial-monitor-title">Moniteur série</span>
332         <button type="button" id="serial-monitor-close" class="panel-close" title="Fermer">&times;</button>
333     </div>
334     <div class="serial-monitor-toolbar">
335         <label class="serial-monitor-board-label">
336             Carte
337             <select id="serial-monitor-board" class="serial-monitor-board-select"></select>
338         </label>
339         <span id="serial-monitor-baud" class="serial-monitor-baud">– baud</span>
340         <button type="button" id="serial-monitor-clear" class="arduino-btn arduino-btn--small">Effacer</button>
341     </div>
342     <pre id="serial-monitor-output" class="serial-monitor-output" aria-label="Sortie série"></pre>
343     <p id="serial-monitor-status" class="serial-monitor-status"></p>
344     <div class="serial-monitor-input-row">
345         <input type="text" id="serial-monitor-input" class="serial-monitor-input" placeholder="Envoyer vers RX (D0) – Entrée pour valide
r" autocomplete="off" spellcheck="false">
346         <button type="button" id="serial-monitor-send" class="arduino-btn arduino-btn--small arduino-btn--primary">Envoyer</button>
347     </div>
348 </div>
349
350 <div id="arduino-lib-popup" class="scope-floating arduino-lib-floating hidden">
351     <div class="scope-floating-header" id="arduino-lib-popup-drag-handle">
352         <span id="arduino-lib-popup-title">Bibliothèque Arduino</span>
353         <button type="button" id="arduino-lib-popup-close" class="panel-close" title="Fermer">&times;</button>
354     </div>
355     <div id="arduino-lib-popup-body" class="arduino-lib-popup-body"></div>
356 </div>
357
358 <div id="bode-popup" class="scope-floating hidden">
359     <div class="scope-floating-header" id="bode-popup-drag-handle">
360         <span id="bode-popup-title">Analyse fréquentielle</span>
361         <button type="button" id="bode-popup-close" class="panel-close" title="Fermer">&times;</button>
362     </div>
363     <div id="bode-popup-info" class="bode-popup-info"></div>
364     <canvas id="bode-popup-canvas" width="640" height="380"></canvas>
365 </div>
366
367 <div id="cd4511-doc-modal" class="modal hidden">
368     <div class="modal-content modal-content--doc">
369         <span class="close-modal" id="close-cd4511-doc">&times;</span>
370         <h2 id="cd4511-doc-title">CD4511 – Décodeur BCD / 7 segments</h2>
371         <div class="modal-body modal-body--doc">
372             <p>
373                 Le <strong>CD4511</strong> (ou équivalent CMOS) convertit un code <strong>BCD 4 bits</strong>
374                 (A = LSB) en sorties pour un afficheur <strong>7 segments</strong> en cathode commune.
375                 Il intègre une <strong>mémorisation</strong> (broche LE) et les fonctions
376                 <strong>extinction</strong> (BI) et <strong>test lampe</strong> (LT).
377             </p>
378             <h3>Broches du symbole (simulateur)</h3>
379             <table class="doc-pin-table">
380                 <thead>
381                     <tr><th>Broche</th><th>Rôle</th><th>Niveau actif</th></tr>
382                 </thead>
383                 <tbody>
384                     <tr><td>A, B, C, D</td><td>Entrées BCD (A = bit faible)</td><td>1 = logique haut (0..V<sub>CC</sub>)</td></tr>
385                     <tr><td>LE</td><td>Verrouillage – 0 : suivi des entrées ; 1 : mémorise</td><td>Relier à <strong>0 V</strong> pour sui
386                     <td>BI</td><td>Extinction de l’affichage</td><td><strong>0 V</strong> = éteint</td><td>– relier BI au <strong>+5 V</stron
387                     <td>LT</td><td>Test lampe (tous segments ON)</td><td><strong>0 V</strong> = test</td><td>– relier LT au <strong>+5 V</stron
ong> en usage normal</td></tr>

```



```

388         <tr><td>a ... g</td><td>Sorties vers les segments</td><td>1 = segment allumé (cathode commune)</td></tr>
389     </tbody>
390 </table>
391 <h3>Brochage réel (boîtier DIP 16 – repères courants)</h3>
392 <table class="doc-pin-table">
393     <thead>
394         <tr><th>Broche DIP</th><th>Symbole</th><th>Rôle</th></tr>
395     </thead>
396     <tbody>
397         <tr><td>7</td><td>A</td><td>Entrée BCD bit faible (LSB)</td></tr>
398         <tr><td>1</td><td>B</td><td>Entrée BCD</td></tr>
399         <tr><td>2</td><td>C</td><td>Entrée BCD</td></tr>
400         <tr><td>6</td><td>D</td><td>Entrée BCD bit fort (MSB)</td></tr>
401         <tr><td>1</td><td>LE</td><td>Verrouillage (0 = suivi direct)</td></tr>
402         <tr><td>2</td><td>BI</td><td>Extinction (1 = affichage actif)</td></tr>
403         <tr><td>3</td><td>LT</td><td>Test lampe (1 = fonctionnement normal)</td></tr>
404         <tr><td>12</td><td>V<sub>DD</sub></td><td>+5 V (alimentation du circuit, hors symbole)</td></tr>
405         <tr><td>8</td><td>V<sub>SS</sub></td><td>Masse</td></tr>
406         <tr><td>9, 10, 11, 13, 14, 15, 16</td><td>e, d, c, b, a, f, g</td><td>Sorties vers l’afficheur 7 segments</td></tr>
407     </tbody>
408 </table>
409 <h3>Câblage typique</h3>
410 <ul class="doc-list">
411     <li>Relier <strong>a ... g</strong> aux entrées du composant <em>Afficheur 7 segments</em> (<code>SEG_1</code>).</li>
412     <li>Relier la <strong>cathode commune (COM)</strong> de l’afficheur à la <strong>masse</strong> (<code>GND_1</code>).</li>
413     <li><strong>BI</strong> et <strong>LT</strong> au <strong>+5 V</strong> (<code>VCC_1</code> ou <code>LOGIC_1</code> à 1) – pas à la masse.</li>
414     <li><strong>LE</strong> à <strong>0 V</strong> (<code>GND_1</code> ou borne à 0) pour suivre un compteur.</li>
415 </ul>
416 <h3>Simulation</h3>
417 <p>
418     Modèle <strong>XSPICE</strong> : bascules <code>d_d latch</code>, table <code>d_genlut</code>,
419     ponts analogiques/numériques. Nécessite <code>digital.cm</code> et ngspice avec XSPICE
420     (<code>npm run check-ngspice</code>).
421 </p>
422 <p class="doc-hint">Double-clic sur le symbole pour rouvrir cette fiche.</p>
423 </div>
424 </div>
425 </div>
426
427 <div id="bargraph-doc-modal" class="modal hidden">
428     <div class="modal-content modal-content--doc">
429         <span class="close-modal" id="close-bargraph-doc">&times;</span>
430         <h2 id="bargraph-doc-title">Bargraph DC10H – Lite-On</h2>
431         <div class="modal-body modal-body--doc">
432             <p>
433                 Le <strong>DC10H</strong> (Lite-On, série LTA) est un afficheur à barres de
434                 <strong>10 segments LED</strong> en <strong>cathode commune</strong>.
435                 Chaque segment s’allume lorsque son anode est au niveau haut par rapport à la cathode commune (COM).
436             </p>
437             <h3>Couleur des segments</h3>
438             <p>Choisissez la couleur d’affichage (cosmétique – n’affecte pas la simulation SPICE) :</p>
439             <div id="bargraph-color-picker" class="bargraph-color-picker"></div>
440             <h3>Broches du symbole (simulateur)</h3>
441             <table class="doc-pin-table">
442                 <thead>
443                     <tr><th>Broche</th><th>Rôle</th><th>Connexion typique</th></tr>
444                 </thead>
445                 <tbody>
446                     <tr><td>s1 ... s10</td><td>Anodes des segments (s1 = barre du bas, s10 = barre du haut)</td><td>Sorties logiques ou rés
447 istances de limitation</td></tr>
448                     <tr><td>COM</td><td>Cathode commune</td><td>Relier à la <strong>masse (0 V)</strong></td></tr>
449                 </tbody>
450             </table>
451             <h3>Brochage réel (DIP 20 – repères courants LTA/DC10H)</h3>
452             <table class="doc-pin-table">
453                 <thead>
454                     <tr><th>Broches DIP</th><th>Segment</th><th>Broches DIP</th><th>Segment</th></tr>
455                 </thead>
456                 <tbody>
457                     <tr><td>1, 11</td><td>s1 (bas)</td><td>6, 16</td><td>s6</td></tr>
458                     <tr><td>2, 12</td><td>s2</td><td>7, 17</td><td>s7</td></tr>
459                     <tr><td>3, 13</td><td>s3</td><td>8, 18</td><td>s8</td></tr>
460                     <tr><td>4, 14</td><td>s4</td><td>9, 19</td><td>s9</td></tr>
461                     <tr><td>5, 15</td><td>s5</td><td>10, 20</td><td>s10 (haut)</td></tr>
462                 </tbody>
463             </table>
464             <p class="doc-hint">
465                 Les broches impaires (1–10) et paires (11–20) sont respectivement anode et cathode de chaque segment ;

```

```

465         toutes les cathodes sont reliées en interne (cathode commune).
466     </p>
467     <h3>Câblage typique</h3>
468     <ul class="doc-list">
469         <li>Relier <strong>COM</strong> à la masse du circuit.</li>
470         <li>Placer une résistance (220 Ω ... 1 kΩ) en série avec chaque anode si alimentation directe en 5 V.</li>
471         <li>Pour un niveau analogique : comparateur ou sorties numériques décimales (CD4511, compteur, Arduino).</li>
472     </ul>
473     <p class="doc-hint">Double-clic sur le symbole pour rouvrir cette fiche et changer la couleur.</p>
474 </div>
475 </div>
476 </div>
477
478 <div id="hc90-doc-modal" class="modal hidden">
479     <div class="modal-content modal-content--doc">
480         <span class="close-modal" id="close-hc90-doc">&times;</span>
481         <h2 id="hc90-doc-title">74HC90 – Compteur décade asynchrone</h2>
482         <div class="modal-body modal-body--doc">
483             <p>
484                 Le <strong>74HC90</strong> (ou 74LS90) est un compteur <strong>asynchrone</strong>
485                 en deux sections : <strong>÷2</strong> (bascule A, horloge <strong>CP0</strong>) et
486                 <strong>÷5</strong> (basculs B-D). En <strong>mode décade</strong>, la sortie
487                 <strong>Q0</strong> est reliée à <strong>CP1</strong> pour former un compteur
488                 <strong>0..9</strong> sur les sorties Q0-Q3 (code binaire naturel).
489             </p>
490             <h3>Brochage réel (boîtier DIP 14 – 74HC90)</h3>
491             <table class="doc-pin-table">
492                 <thead>
493                     <tr><th>Broche DIP</th><th>Symbole</th><th>Rôle</th></tr>
494                 </thead>
495                 <tbody>
496                     <tr><td>14</td><td>CP0</td><td>Horloge principale (GImp) – section ÷2</td></tr>
497                     <tr><td>1</td><td>CP1</td><td>Horloge section ÷5 – relié à <strong>Q0</strong> (décade)</td></tr>
498                     <tr><td>12</td><td>Q0</td><td>Sortie LSB – vers <strong>CD4511_1</strong> entrée <strong>A</strong></td></tr>
499                     <tr><td>9</td><td>Q1</td><td>Vers entrée <strong>B</strong></td></tr>
500                     <tr><td>8</td><td>Q2</td><td>Vers entrée <strong>C</strong></td></tr>
501                     <tr><td>11</td><td>Q3</td><td>MSB – vers entrée <strong>D</strong></td></tr>
502                     <tr><td>2, 3</td><td>MR1, MR2</td><td>Reset (niveau haut = compteur à 0)</td></tr>
503                     <tr><td>6, 7</td><td>MS1, MS2</td><td>Mode – laisser à 0 V (décade)</td></tr>
504                     <tr><td>5</td><td>VCC</td><td>+5 V (<code>VCC_1</code>)</td></tr>
505                     <tr><td>10</td><td>GND</td><td>Masse (<code>GND_1</code>)</td></tr>
506                     <tr><td>4, 13</td><td>NC</td><td>Non connectées</td></tr>
507                 </tbody>
508             </table>
509             <p class="doc-hint">Sur le schéma, <strong>Q0..Q3</strong> sont alignés verticalement (comme <strong>A..D</strong> du CD4511) p
our des fils horizontaux courts.</p>
510             <h3>Montage compteur → afficheur (noms du simulateur)</h3>
511             <table class="doc-pin-table">
512                 <thead>
513                     <tr><th>De</th><th>Vers</th><th>Fil</th></tr>
514                 </thead>
515                 <tbody>
516                     <tr><td><code>GND_1</code></td><td><code>HC90_1</code> GND, <code>CD4511_1</code> (masse circuit), <code>SEG_1</code>
COM</td><td>Masse commune</td></tr>
517                     <tr><td><code>VCC_1</code></td><td><code>HC90_1</code> VCC</td><td>+5 V</td></tr>
518                     <tr><td><code>VCC_1</code> ou <code>LOGIC_1</code> (1)</td><td><code>CD4511_1</code> BI, LT</td><td>Afficheur actif</
td></tr>
519                     <tr><td><code>GND_1</code> ou <code>LOGIC_2</code> (0)</td><td><code>CD4511_1</code> LE</td><td>Suivi direct du compt
eur</td></tr>
520                     <tr><td><code>GND_1</code> ou <code>LOGIC_3</code> (0)</td><td><code>HC90_1</code> MR1, MR2, MS1, MS2</td><td>Comptag
e décade</td></tr>
521                     <tr><td><code>GImp_1</code> sortie</td><td><code>HC90_1</code> CP0</td><td>Horloge</td></tr>
522                     <tr><td><code>HC90_1</code> Q0</td><td><code>HC90_1</code> CP1</td><td>Décade (aussi forcé en simulation)</td></tr>
523                     <tr><td><code>HC90_1</code> Q0</td><td><code>CD4511_1</code> A</td><td>BCD bit 0</td></tr>
524                     <tr><td><code>HC90_1</code> Q1</td><td><code>CD4511_1</code> B</td><td>BCD bit 1</td></tr>
525                     <tr><td><code>HC90_1</code> Q2</td><td><code>CD4511_1</code> C</td><td>BCD bit 2</td></tr>
526                     <tr><td><code>HC90_1</code> Q3</td><td><code>CD4511_1</code> D</td><td>BCD bit 3</td></tr>
527                     <tr><td><code>CD4511_1</code> a..g</td><td><code>SEG_1</code> a..g</td><td>Segments</td></tr>
528                 </tbody>
529             </table>
530             <h3>Câblage typique (mode décade 0..9)</h3>
531             <ul class="doc-list">
532                 <li>Les composants sont numérotés <code>HC90_1</code>, <code>CD4511_1</code>, <code>SEG_1</code>, <code>GND_1</code>, <co
de>VCC_1</code>, <code>GImp_1</code>, etc.</li>
533                 <li>Horloge <strong>1 Hz</strong> sur <code>GImp_1</code> → CP0 pour une seconde par pas (préréglage « tick seconde » dan
s le panneau GImp).</li>
534                 <li>Le fil <strong>Q0 → CP1</strong> reste recommandé sur la plaquette ; il est aussi appliqué automatiquement en simulat
ion.</li>
535             </ul>
536             <h3>Deux chiffres (unités + dizaines)</h3>
537             <ul class="doc-list">

```

```

538 >).</li>
539 <li>Horloge <code>GImp_1</code> uniquement sur <code>HC90_1</code> CP0 ; <code>HC90_2</code> CP0 via une porte <strong>ET
</strong>.</li>
540 <li><strong>Report au chiffre 9</strong> : relier <code>HC90_1</code> <strong>Q0</strong> et <strong>Q3</strong> aux entr
ées de l'ET, sortie → <code>HC90_2</code> CP0. Au comptage 9 (binaire 1001), Q0 et Q3 sont à 1 → un front horloge sur les dizaines.</li>
541 <li><strong>Ne pas utiliser Q1 + Q3</strong> : en BCD 0..9 ces sorties ne sont jamais hautes en même temps (seul l'état in
valide 10 = 1010 les active).</li>
542 <li><code>HC90_2</code> : même câblage décade (Q0 → CP1, MR/MS à 0, VCC/GND).</li>
543 <li><strong>Affichage 0..99</strong> : si le report est correct (AND Q0+Q3 ou Q3), l'animation utilise un comptage idéal e
n temps réel (1 impulsion = +1), fiable à toute fréquence du GImp.</li>
544 </ul>
545 <h3>Simulation</h3>
546 <p>
547 Quatre bascules <strong>D</strong> en mode bascule (D relié à /Q, horloges en cascade).
548 Modèle <strong>XSPICE</strong> (<code>d_dff</code>) si ngspice et <code>digital.cm</code> sont disponibles ;
549 sinon repli sur sources comportementales (comme la bascule D du simulateur).
550 Vérification : <code>node Simulateur/Engine/ic74hc90-decade.test.mjs</code>,
551 <code>node Simulateur/Engine/hc90-cascade.test.mjs</code> ou
552 <code>npm run check-ngspice</code>.
553 </p>
554 <p class="doc-hint">Double-clic sur le symbole pour rouvrir cette fiche.</p>
555 </div>
556 </div>
557 </div>
558
559 <div id="uno-doc-modal" class="modal hidden">
560 <div class="modal-content modal-content--doc">
561 <span class="close-modal" id="close-uno-doc">&times;</span>
562 <h2 id="uno-doc-title">Arduino UNO R3</h2>
563 <div class="modal-body modal-body--doc">
564 <p>
565 Carte <strong>Arduino UNO R3</strong> (ATmega328P) : alimentation <strong>5 V</strong> / <strong>3,3 V</strong>,
566 <strong>14 broches digitales</strong> D0-D13 et <strong>6 entrées analogiques</strong> A0-A5.
567 L'éditeur de sketch est dans le <strong>panneau à droite</strong> (ouvert à la pose du composant).
568 </p>
569 <h3>Broches du symbole (simulateur)</h3>
570 <table class="doc-pin-table">
571 <thead>
572 <tr><th>Broche</th><th>Rôle</th></tr>
573 </thead>
574 <tbody>
575 <tr><td>5V, 3V3</td><td>Alimentation sortie (rails simulés)</td></tr>
576 <tr><td>GND, GND2</td><td>Masse</td></tr>
577 <tr><td>VIN</td><td>Alimentation externe (entrée)</td></tr>
578 <tr><td>RESET</td><td>Reset (pull-up interne modélisé)</td></tr>
579 <tr><td>A0 ... A5</td><td>Entrées analogiques / I²C (A4/A5)</td></tr>
580 <tr><td>D0, D1</td><td>UART (RX/TX)</td></tr>
581 <tr><td>D2 ... D13</td><td>GPIO digitales (PWM : D3, D5, D6, D9, D10, D11)</td></tr>
582 <tr><td>D13</td><td>LED intégrée (<code>LED_BUILTIN</code> = 13)</td></tr>
583 </tbody>
584 </table>
585 <h3>Registres AVR – ports et broches (ATmega328P)</h3>
586 <p>
587 Chaque broche GPIO est pilotée par un couple <strong>DDR</strong> (direction) et
588 <strong>PORT</strong> (niveau ou pull-up). Le simulateur interprète
589 <code>pinMode</code> / <code>digitalWrite</code> et les accès directs aux registres
590 (<code>DDRD |= (1<math>\ll</math>5)</code>, <code>PORTB = 0xFF</code>, etc.).
591 Pendant la simulation, les valeurs hex des registres s'affichent sur le symbole UNO.
592 </p>
593 <table class="doc-pin-table">
594 <thead>
595 <tr><th>Port AVR</th><th>Registres</th><th>Bits</th><th>Broches Arduino</th><th>Repères MCU</th></tr>
596 </thead>
597 <tbody>
598 <tr>
599 <td><strong>Port D</strong></td>
600 <td><code>DDRD</code> / <code>PORTD</code></td>
601 <td>0 ... 7</td>
602 <td><strong>D0</strong> ... <strong>D7</strong></td>
603 <td>PD0 ... PD7</td>
604 </tr>
605 <tr>
606 <td><strong>Port B</strong></td>
607 <td><code>DDRB</code> / <code>PORTB</code></td>
608 <td>0 ... 5</td>
609 <td><strong>D8</strong> ... <strong>D13</strong></td>
610 <td>PB0 ... PB5 (<code>LED_BUILTIN</code> = PB5 / D13)</td>
611 </tr>
612 <tr>
613 <td><strong>Port C</strong></td>

```

```

614         <td><code>DDRC</code> / <code>PORTC</code></td>
615         <td>0 ... 5</td>
616         <td><strong>A0</strong> ... <strong>A5</strong></td>
617         <td>PC0 ... PC5 (pins Arduino 14 ... 19 en mode digital)</td>
618     </tr>
619 </tbody>
620 </table>
621 <h3>Correspondance broche ↔ bit (référence rapide)</h3>
622 <table class="doc-pin-table">
623     <thead>
624         <tr><th>Broche</th><th>Port / bit</th><th>Numéro Arduino</th><th>Usage courant</th></tr>
625     </thead>
626     <tbody>
627         <tr><td>D0</td><td>PORTD bit 0</td><td>0</td><td>UART RX</td></tr>
628         <tr><td>D1</td><td>PORTD bit 1</td><td>1</td><td>UART TX</td></tr>
629         <tr><td>D2 ... D7</td><td>PORTD bits 2 ... 7</td><td>2 ... 7</td><td>GPIO (PWM : D3, D5, D6)</td></tr>
630         <tr><td>D8 ... D12</td><td>PORTB bits 0 ... 4</td><td>8 ... 12</td><td>GPIO (PWM : D9, D10, D11)</td></tr>
631         <tr><td>D13</td><td>PORTB bit 5</td><td>13</td><td>GPIO, LED intégrée</td></tr>
632         <tr><td>A0 ... A3</td><td>PORTC bits 0 ... 3</td><td>14 ... 17</td><td>Analogique / GPIO digital</td></tr>
633         <tr><td>A4</td><td>PORTC bit 4</td><td>18</td><td>I²C SDA (SDA)</td></tr>
634         <tr><td>A5</td><td>PORTC bit 5</td><td>19</td><td>I²C SCL (SCL)</td></tr>
635     </tbody>
636 </table>
637 <p>
638     <strong>DDR</strong> : bit à <code>1</code> = sortie, à <code>0</code> = entrée.
639     <strong>PORT</strong> en sortie : bit à <code>1</code> = HIGH (5 V), à <code>0</code> = LOW.
640     En entrée avec <code>pinMode(n, INPUT_PULLUP)</code> : bit PORT à <code>1</code> active le pull-up interne.
641 </p>
642 <h3>CD4511 + PORTD (4 bits faibles)</h3>
643 <p>
644     Pour afficher le chiffre <strong>0..9</strong> avec <code>PORTD = n</code>, reliez
645     <strong>A→D0, B→D1, C→D2, D→D3</strong>, puis :
646 </p>
647 <pre class="doc-code">void setup() {
648     DDRD = 0b0001111; // D0..D3 en sortie
649 }
650 void loop() {
651     PORTD = 6; // affiche 6
652 }</pre>
653 <p>
654     Attention : écrire <code>DDRD = 6</code> dans <code>loop()</code> (au lieu de
655     <code>PORTD</code>) reconfigure les directions et peut bloquer l'affichage.
656     Seul <code>PORTD</code> fixe les niveaux logiques.
657 </p>
658 <h3>Programmation</h3>
659 <ul class="doc-list">
660     <li>Éditeur : sketch <code>.ino</code> + bouton <strong>Compiler (arduino-cli)</strong>.</li>
661     <li>Carte cible : <code>arduino:avr:uno</code>.</li>
662     <li>À la simulation, le moteur interprète <code>pinMode(..., OUTPUT|INPUT|INPUT_PULLUP)</code>, <code>digitalWrite</code>,
663     <code>delay(ms)</code> et les écritures <code>DDR</code> / <code>PORT</code> pour piloter D0-D13 et A0-A5.</li>
664     <li>Reliez <strong>GND</strong> de l'UNO à la masse du circuit si vous utilisez les rails 5 V / 3,3 V.</li>
665 </ul>
666 <p class="doc-hint">Double-clic sur le symbole pour rouvrir cette fiche.</p>
667 </div>
668 </div>
669
670 <div id="value-prompt-modal" class="modal hidden">
671     <div class="modal-content modal-content--prompt">
672         <span class="close-modal" id="close-value-prompt">&times;</span>
673         <label id="value-prompt-message" class="value-prompt-label" for="value-prompt-input"></label>
674         <input type="text" id="value-prompt-input" class="value-prompt-input" autocomplete="off">
675         <div class="value-prompt-actions">
676             <button type="button" id="value-prompt-ok" class="value-prompt-btn value-prompt-btn--ok">OK</button>
677             <button type="button" id="value-prompt-cancel" class="value-prompt-btn">Annuler</button>
678         </div>
679     </div>
680 </div>
681
682 <div id="commands-modal" class="modal hidden">
683     <div class="modal-content">
684         <span class="close-modal" id="close-commands">&times;</span>
685         <h2>Commandes de l'Éditeur</h2>
686         <div class="modal-body">
687             <p><strong>Ctrl + C / Ctrl + V</strong> Copier / Coller la sélection (Évite le double-copier)</p>
688             <p><strong>Clic simple (Simulation active)</strong> Commuter l'état (0 ou 1) d'une Borne Logique</p>
689             <p><strong>Clic simple (Hors simulation)</strong> Sélectionner un fil ou un composant</p>
690             <p><strong>Double-clic</strong> Modifier la valeur (R, C, L, VCC...) ; fiche technique <strong>CD4511</strong>, <strong>74HC90</strong>, <strong>Arduino UNO</strong></p>
691             <p><strong>[Shift] + Glisser cadre</strong> Sélectionner plusieurs composants</p>

```

```
692         <p><strong>[R] :</strong> Pivoter les composants sélectionnés de 90° (sauf bascules et GImp)</p>
693         <p><strong>[X] :</strong> Retourner horizontalement le GImp sélectionné</p>
694         <p><strong>[Suppr] ou [Backspace] :</strong> Supprimer la sélection (Composants ou fil choisi)</p>
695         <p><strong>[Espace] :</strong> Ajouter un point d'ancrage intermédiaire sur un fil en cours de tracé</p>
696         <p><strong>[Ctrl] + [Z] / [Ctrl] + [Y] :</strong> Annuler / Rétablir</p>
697     </div>
698 </div>
699 </div>
700
701     <script type="module" src="app.js?v=i2c-pro1"></script>
702 </body>
703 </html>
```

```

1  /**
2   * Parse JSON texte (fichiers circuit) - tolère BOM UTF-8 et espaces.
3   */
4
5  import { DC10H_COLOR_IDS } from './bargraph-dc10h-layout.js';
6
7  export function parseJsonText(raw) {
8      if (raw == null) {
9          throw new Error('Le fichier est vide.');

```

```

1 // led-animation.js - clignotement LED à partir des courbes .tran (wrdata)
2 import { circuit, flags, simulationResults } from './state.js';
3 import { bcdDigitToSeg7Segments, bcdFromQVoltages } from './Engine/bcd-seg7.mjs';
4 import { quantizeVoltmeterReading } from './Engine/voltmeter-display.mjs';
5
6 export { quantizeVoltmeterReading };
7 import {
8     detectHc90Cascade,
9     hc90LabelForSeg7,
10    hc90QBitForLed,
11    hc90TranSampleTimeSec,
12    idealHc90BcdForLabel,
13    isHc90MasterResetActive,
14    shouldUseIdealHc90Counting,
15 } from './Engine/hc90-cascade.mjs';
16 import {
17     idealRippleMod10Bcd,
18     shouldUseIdealRippleMod10Seg7,
19 } from './Engine/ripple-mod10.mjs';
20 import {
21     applyArduinoSketchToComponent,
22     arduinoGpioIsTimeVarying,
23     resolvePinLevelsAt,
24     createArduinoRuntime,
25     stepArduinoRuntime,
26     arduinoRuntimeLevels,
27     sketchHasLoop,
28     sketchUsesAnalogInput,
29     getRuntimeSerialTx,
30     injectRuntimeSerialRx,
31     getRuntimeSerialMeta,
32 } from './Engine/arduino-sketch-parse.mjs';
33 import { readUnoAnalogInputs } from './Engine/arduino-analog-ideal.mjs';
34 import { sketchUsesSerial } from './Engine/arduino-uart-wave.mjs';
35 import { getVoltageAtJunction } from './geometry.js';
36 import { reachableJunctions } from './Engine/hc90-cascade.mjs';
37 import { UNO_JUNCTION_SUFFIX } from './arduino-uno-layout.js';
38 import { syncArduinoSketchesFromEditor } from './arduino-sketch-sync.js';
39 import {
40     getIdealSeg7FromArduino,
41     getIdealBargraphFromArduino,
42     getIdealVoltmeterVoltage,
43 } from './Engine/arduino-gpio-ideal.mjs';
44 import { isGroveLcdWiredToUno, refreshGroveLcdDisplayCache, getIdealGroveLcdDisplay } from './Engine/grove-lcd-ideal.mjs';
45 import { isGroveDht22WiredToUno, getIdealDht22Reading } from './Engine/dht22-ideal.mjs';
46
47 /** Au-delà de cette fréquence, persistance rétinienne : LED fixe (courant moyen). */
48 export const PERSISTENCE_FREQ_HZ = 50;
49
50 const LED_ON_A = 1e-4;
51 /** Courant simulé pour une LED HC90 en mode comptage idéal (Render / .tran court). */
52 const HC90_IDEAL_LED_ON_A = 0.008;
53 /** Courant simulé pour une LED pilotée par GPIO Arduino (animation idéale). */
54 const ARDUINO_IDEAL_LED_ON_A = 0.008;
55
56 /** Courant maxi recommandé pour une LED standard (au-delà → grillée). */
57 export const LED_MAX_SAFE_CURRENT_A = 0.02;
58
59 export function isLedOvercurrent(current) {
60     return typeof current === 'number' && Number.isFinite(current) && Math.abs(current) > LED_MAX_SAFE_CURRENT_A;
61 }
62
63 let redraw = () => {};
64 const SEG7_NAMES = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
65 const BARGRAPH_SEG_NAMES = Array.from({ length: 10 }, (_, i) => `s${i + 1}`);
66 const SEG7_LIT_DELTA_V = 0.35;
67
68 let anim = {
69     rafId: null,
70     startMs: 0,
71     plots: {},
72     vmPlots: {},
73     seg7Plots: {},
74     bargraphPlots: {},
75     /** @type {Record<string, { q: Array< time: number[]; voltage: number[]; vth?: number } | null>; span: number }>} */
76     hc90QPlots: {},
77     /** Dernier BCD valide 0-9 par HC90 (filtre états transitoires > 9). */
78     hc90LastBcd: {},

```

```

79  /** MR1-MR2 actifs (reset asynchrone) sur le circuit courant. */
80  hc90MrActive: false,
81  /** @type {Record<string, number>} période (s) propre à chaque LED */
82  ledPeriods: {},
83  vmPeriods: {},
84  /** @type {Record<string, boolean>} persistance rétinienne par LED */
85  ledPersistence: {},
86  /** @type {Record<string, number>} courant moyen si persistance */
87  steadyCurrent: {},
88  /** @type {ReturnType<typeof detectHc90Cascade>|null} */
89  /** @type {Record<string, { pulse?: { highSec: number, lowSec: number }, level?: number }>} */
90  arduinoLedDrive: {},
91  };
92
93  export function bindLedAnimationRedraw(fn) {
94    redraw = typeof fn === 'function' ? fn : () => {};
95  }
96
97  let onArduinoRuntimeTick = () => {};
98
99  /** Callback après chaque pas runtime UNO (ex. rafraîchir le moniteur série). */
100 export function bindArduinoRuntimeTick(fn) {
101   onArduinoRuntimeTick = typeof fn === 'function' ? fn : () => {};
102 }
103
104 /* --- Runtime Arduino temps réel (sketches pilotés par digitalRead) ----- */
105
106 /** @type {Map<string, { rt: object, sketch: string }>} */
107 const arduinoRuntimes = new Map();
108 let lastRuntimeStepMs = 0;
109
110 function junctionNetReachesGnd(net) {
111   for (const j of net) {
112     if (typeof j === 'string' && (j.startsWith('GND') || /^GND\d*_/.test(j))) return true;
113   }
114   return false;
115 }
116
117 function junctionNetReachesHigh(net) {
118   for (const j of net) {
119     if (typeof j !== 'string') continue;
120     if (j.startsWith('VCC')) return true;
121     if (/^VDC\d*_in$/i.test(j)) return true;
122   }
123   return false;
124 }
125
126 /** Niveaux d'entrée live (boutons/interrupteurs vers une rampe) pour un UNO. */
127 function readUnoInputs(uno) {
128   const inputs = {};
129   const modes = uno.pinModes || {};
130   const wires = circuit.wires;
131   const aj = circuit.autoJunctions;
132   for (const [label, mode] of Object.entries(modes)) {
133     if (mode !== 'INPUT' && mode !== 'INPUT_PULLUP') continue;
134     const net = reachableJunctions(`${uno.label}_${label}`, wires, aj);
135     let value = 1;
136     if (junctionNetReachesGnd(net)) value = 0;
137     else if (junctionNetReachesHigh(net)) value = 1;
138     for (const comp of circuit.components) {
139       if (comp.type !== 'push_button' && comp.type !== 'switch_spdt') continue;
140       const a = `${comp.label}_in`;
141       const b = `${comp.label}_out`;
142       const pinSide = net.has(a) ? a : net.has(b) ? b : null;
143       if (!pinSide && comp.type === 'switch_spdt') continue;
144       if (comp.type === 'push_button') {
145         if (!pinSide) continue;
146         const otherSide = pinSide === a ? b : a;
147         const other = reachableJunctions(otherSide, wires, aj);
148         if (junctionNetReachesGnd(other)) value = comp.state === 1 ? 0 : 1;
149         else if (junctionNetReachesHigh(other)) value = comp.state === 1 ? 1 : 0;
150       }
151     }
152     inputs[label] = value;
153   }
154   return inputs;
155 }
156
157 /** Avance (une fois par frame) les runtimes des UNO dont le sketch a un loop(). */
158 function updateArduinoRuntimes() {

```



```

159     const now = performance.now();
160     if (lastRuntimeStepMs <= 0) lastRuntimeStepMs = now;
161     const deltaMs = now - lastRuntimeStepMs;
162     lastRuntimeStepMs = now;
163     const seen = new Set();
164     for (const comp of circuit.components) {
165         if (comp.type !== 'arduino_uno') continue;
166         applyArduinoSketchToComponent(comp);
167         if (!sketchHasLoop(comp.sketch || '')) {
168             comp.liveLevels = null;
169             continue;
170         }
171         seen.add(comp.label);
172         let entry = arduinoRuntimes.get(comp.label);
173         if (!entry || entry.sketch !== (comp.sketch || '')) {
174             entry = { rt: createArduinoRuntime(comp), sketch: comp.sketch || '', lastAnalogKey: '' };
175             arduinoRuntimes.set(comp.label, entry);
176         }
177         const analogInputs = readUnoAnalogInputs(comp, {
178             components: circuit.components,
179             wires: circuit.wires,
180             autoJunctions: circuit.autoJunctions,
181             tSec: getSimulationElapsedSec(),
182             getVoltageAtJunction,
183             voltmeters: simulationResults.voltmeters,
184         });
185         if (sketchUsesAnalogInput(comp.sketch || '')) {
186             const analogKey = JSON.stringify(analogInputs);
187             if (entry.lastAnalogKey && entry.lastAnalogKey !== analogKey) {
188                 entry.rt.gen = null;
189                 entry.rt.sleepMs = 0;
190                 entry.rt.idle = false;
191             }
192             entry.lastAnalogKey = analogKey;
193         }
194         stepArduinoRuntime(entry.rt, deltaMs, readUnoInputs(comp), analogInputs);
195         comp.liveLevels = arduinoRuntimeLevels(entry.rt);
196         comp.serialLog = getRuntimeSerialTx(entry.rt);
197         const serMeta = getRuntimeSerialMeta(entry.rt);
198         comp.serialBegun = serMeta.begun;
199         comp.serialBaud = serMeta.baud;
200         if (entry.rt.state.regs) {
201             comp.avrRegisters = { ...(comp.avrRegisters || {}), ...entry.rt.state.regs };
202         }
203     }
204     for (const key of [...arduinoRuntimes.keys()]) {
205         if (!seen.has(key)) arduinoRuntimes.delete(key);
206     }
207     onArduinoRuntimeTick();
208 }
209
210 function resetArduinoRuntimes() {
211     arduinoRuntimes.clear();
212     lastRuntimeStepMs = 0;
213     for (const comp of circuit.components) {
214         if (comp.type === 'arduino_uno') {
215             comp.liveLevels = null;
216             comp.serialLog = '';
217             comp.serialBegun = false;
218             comp.serialBaud = 0;
219         }
220     }
221 }
222
223 export { resetArduinoRuntimes };
224
225 /** Cartes UNO présentes sur le schéma. */
226 export function listUnoBoardLabels() {
227     return circuit.components
228         .filter((c) => c.type === 'arduino_uno' && c.label)
229         .map((c) => c.label);
230 }
231
232 export function getUnoSerialOutput(label) {
233     const entry = arduinoRuntimes.get(label);
234     if (entry) return getRuntimeSerialTx(entry.rt);
235     const comp = circuit.components.find((c) => c.type === 'arduino_uno' && c.label === label);
236     return comp?.serialLog ?? '';
237 }
238

```

```

239 export function sendUnoSerialInput(label, text) {
240     const entry = arduinoRuntimes.get(label);
241     if (entry) injectRuntimeSerialRx(entry.rt, text);
242 }
243
244 export function clearUnoSerialOutput(label) {
245     const entry = arduinoRuntimes.get(label);
246     if (entry?.rt?.state?.serial) entry.rt.state.serial.tx = '';
247     const comp = circuit.components.find((c) => c.type === 'arduino_uno' && c.label === label);
248     if (comp) comp.serialLog = '';
249 }
250
251 /** Après ouverture / chargement d'un schéma JSON (sketch depuis JSON, pas l'éditeur). */
252 export function onCircuitLoaded() {
253     for (const comp of circuit.components) {
254         if (comp.type === 'arduino_uno') applyArduinoSketchToComponent(comp);
255     }
256     refreshGroveLcdDisplayCache(circuit.components, circuit.wires, circuit.autoJunctions);
257     resetArduinoRuntimes();
258     indexArduinoLeds();
259 }
260
261 /** Sketch recompilé ou modifié : réinitialise runtime, LED pilotées et phase d'animation. */
262 export function onArduinoSketchUpdated() {
263     syncArduinoSketchesFromEditor();
264     for (const comp of circuit.components) {
265         if (comp.type === 'arduino_uno') applyArduinoSketchToComponent(comp);
266     }
267     refreshGroveLcdDisplayCache(circuit.components, circuit.wires, circuit.autoJunctions);
268     resetArduinoRuntimes();
269     indexArduinoLeds();
270     if (!flags.isSimulating) return;
271     if (anim.startMs > 0 || anim.rafId != null) anim.startMs = performance.now();
272     ensureArduinoLedsAnimation();
273 }
274
275 function getSourceFrequencyHz() {
276     for (const comp of circuit.components) {
277         if ((comp.type === 'gimp' || comp.type === 'gsin' || comp.type === 'gsqr') && comp.frequency > 0) return comp.frequency;
278     }
279     return 0;
280 }
281
282 function getGimpPeriodSec() {
283     const hz = getSourceFrequencyHz();
284     return hz > 0 ? 1 / hz : null;
285 }
286
287 function hasRippleCounter() {
288     const ffTypes = new Set(['d_flipflop', 'jk_flipflop']);
289     for (const comp of circuit.components) {
290         if (!ffTypes.has(comp.type)) continue;
291         const clkJon = `${comp.label}_CLK`;
292         for (const w of circuit.wires) {
293             const other = w.fromJunctionId === clkJon ? w.toJunctionId
294                 : w.toJunctionId === clkJon ? w.fromJunctionId : null;
295             if (!other) continue;
296             const mQ = /^[A-Za-z0-9_]+_Q$/ .exec(other);
297             const mQb = /^[A-Za-z0-9_]+_Qbar$/ .exec(other);
298             const prevLabel = (mQ || mQb)?.[1];
299             if (!prevLabel || prevLabel === comp.label) continue;
300             if (circuit.components.some((c) => ffTypes.has(c.type) && c.label === prevLabel)) return true;
301         }
302     }
303     return false;
304 }
305
306 /** 74HC90 en mode décade : l'animation doit balayer le .tran, pas figer une phase d'horloge. */
307 function hasHc90DecadeCounter() {
308     return circuit.components.some((c) => c.type === 'ic_74hc90');
309 }
310
311 function indexHc90QPlots(logicGateTranPlots) {
312     const byComp = {};
313     for (const [id, plot] of Object.entries(logicGateTranPlots || {})) {
314         const m = /^(.+)_Q([0-3])$/ .exec(id);
315         if (!m || !plot?.time?.length) continue;
316         const base = m[1];
317         const qi = Number(m[2]);
318         if (!byComp[base]) byComp[base] = { q: [null, null, null, null], span: 0 };

```

```

319     byComp[base].q[q1] = plot;
320     const span = plot.time[plot.time.length - 1] - plot.time[0];
321     if (span > byComp[base].span) byComp[base].span = span;
322 }
323 return byComp;
324 }
325
326 function hc90TranSpanSec() {
327     let max = 0;
328     for (const pack of Object.values(anim.hc90QPlots)) {
329         if (pack.span > max) max = pack.span;
330     }
331     return max;
332 }
333
334 /**
335  * Temps simulé pour l'échantillon HC90 : 1 impulsion GImp = 1 pas de comptage (temps réel).
336  * On lit la valeur stabilisée en fin de période d'horloge (comme les tests SPICE).
337  */
338 /**
339  * Synchronise l'horloge d'animation avec le reset MR :
340  * - MR actif → affichage 0 ;
341  * - MR relâché → repartir de t=0 (nouvelle .tran SPICE).
342  */
343 function syncHc90MasterResetClock() {
344     const mrActive = isHc90MasterResetActive(circuit.components, circuit.wires, circuit.autoJunctions);
345     if (anim.hc90MrActive && !mrActive) {
346         anim.startMs = performance.now();
347         anim.hc90LastBcd = {};
348     }
349     anim.hc90MrActive = mrActive;
350     return mrActive;
351 }
352
353 function hc90SampleTimeSec(elapsed, compLabel) {
354     const plotSpan = hc90TranSpanSec() || anim.hc90QPlots[compLabel]?.span || 0;
355     const clockPeriod = getGimpPeriodSec();
356     if (plotSpan > 0 && clockPeriod > 0) {
357         return hc90TranSampleTimeSec(elapsed, clockPeriod, plotSpan);
358     }
359     if (plotSpan > 0) return elapsed % plotSpan;
360     return elapsed;
361 }
362
363 function sampleHc90Bcd(compLabel, tSec) {
364     const pack = anim.hc90QPlots[compLabel];
365     if (!pack?.q?.[0]?.time?.length) return null;
366     const vth = pack.q[0].vth ?? 2.5;
367     const qV = pack.q.map((plot) =>
368         plot?.time?.length ? interpolateSeries(plot.time, plot.voltage, tSec) : 0
369     );
370     const raw = bcdFromQVoltages(qV, vth);
371     if (raw > 9) {
372         const last = anim.hc90LastBcd[compLabel];
373         return last !== null ? last : null;
374     }
375     anim.hc90LastBcd[compLabel] = raw;
376     return raw;
377 }
378
379 /** Valeur BCD 0-9 animée pour un 74HC90 (comptage idéal ou courbes Q0..Q3 du .tran). */
380 export function getAnimatedHc90Bcd(compLabel) {
381     if (!compLabel || !hasHc90DecadeCounter()) return null;
382     if (syncHc90MasterResetClock()) return 0;
383     const elapsed = (performance.now() - anim.startMs) / 1000;
384     const clockPeriod = getGimpPeriodSec();
385     const cascade = anim.hc90Cascade ?? detectHc90Cascade(circuit.components, circuit.wires, circuit.autoJunctions);
386     if (shouldUseIdealHc90Counting(cascade, clockPeriod)) {
387         const ideal = idealHc90BcdForLabel(compLabel, cascade, elapsed, clockPeriod);
388         if (ideal !== null) return ideal;
389     }
390     if (!anim.hc90QPlots[compLabel]) return null;
391     return sampleHc90Bcd(compLabel, hc90SampleTimeSec(elapsed, compLabel));
392 }
393
394 /** Instant stable dans la phase basse du GImp (ripple propagé). */
395 function getGimpStablePhase() {
396     for (const comp of circuit.components) {
397         if (comp.type === 'gimp') {
398             const duty = Math.min(99, Math.max(1, comp.dutyCycle ?? 50));

```

```

399         const lowStart = duty / 100;
400         return lowStart + (1 - lowStart) * 0.5;
401     }
402 }
403 return 0.75;
404 }
405
406 function sampleTimeSec(elapsed, ledPeriod) {
407     if (hasRippleCounter()) {
408         const master = getGimpPeriodSec() ?? ledPeriod;
409         const phase = getGimpStablePhase();
410         const tStable = Math.floor(elapsed / master) * master + master * phase;
411         return tStable;
412     }
413     return plotTimeOrigin(elapsed, ledPeriod);
414 }
415
416 function plotTimeOrigin(elapsed, period) {
417     return elapsed % period;
418 }
419
420
421 function arduinoPinLabelFromJunction(unoLabel, junctionId) {
422     if (!junctionId?.startsWith(`${unoLabel}_`)) return null;
423     const suffix = junctionId.slice(unoLabel.length + 1);
424     if (suffix in UNO_JUNCTION_SUFFIX && /^D\d+$/.test(suffix)) return suffix;
425     return null;
426 }
427
428 function indexArduinoLedDrives() {
429     anim.arduinoLedDrive = {};
430     for (const led of circuit.components.filter((c) => c.type === 'led')) {
431         const start = `${led.label}_in`;
432         for (const uno of circuit.components.filter((c) => c.type === 'arduino_uno')) {
433             applyArduinoSketchToComponent(uno);
434             const net = reachableJunctions(start, circuit.wires, circuit.autoJunctions);
435             let found = null;
436             for (const jid of net) {
437                 const pinLabel = arduinoPinLabelFromJunction(uno.label, jid);
438                 if (pinLabel && uno.pinModes?.[pinLabel] === 'OUTPUT') {
439                     found = {
440                         unoLabel: uno.label,
441                         pinLabel,
442                         pulse: uno.pinPulses?.[pinLabel],
443                         level: uno.pinLevels?.[pinLabel],
444                     };
445                     break;
446                 }
447             }
448             if (found && (found.pulse || found.level === 1 || found.level === 0)) {
449                 anim.arduinoLedDrive[led.label] = found;
450                 break;
451             }
452         }
453     }
454 }
455
456 function hasLcdTimeVaryingDisplay() {
457     for (const comp of circuit.components) {
458         if (comp.type !== 'grove_lcd16x2') continue;
459         if (comp.lcdDisplayCache?.hasTiming) return true;
460     }
461     return false;
462 }
463
464 /** LCD Grove avec delay() dans le sketch – affichage selon le temps de simulation. */
465 export function getAnimatedGroveLcdDisplay(label) {
466     const elapsed = anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
467     return getIdealGroveLcdDisplay(
468         label,
469         circuit.components,
470         circuit.wires,
471         circuit.autoJunctions,
472         elapsed,
473         {
474             getVoltageAtJunction,
475             voltmeters: simulationResults.voltmeters,
476         }
477     );
478 }

```

```

479
480 /** Temps écoulé depuis le début de la simulation animée (s). */
481 export function getSimulationElapsedSec() {
482     return anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
483 }
484
485 /** Mesures DHT22 animées pendant la simulation. */
486 export function getAnimatedDht22Reading(label) {
487     const elapsed = anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
488     return getIdealDht22Reading(
489         label,
490         circuit.components,
491         circuit.wires,
492         circuit.autoJunctions,
493         elapsed
494     );
495 }
496
497 function hasArduinoStaticIdealDisplay() {
498     syncArduinoSketchesFromEditor();
499     for (const comp of circuit.components) {
500         if (comp.type !== 'seg7') continue;
501         const ideal = getIdealSeg7FromArduino(
502             comp.label,
503             circuit.components,
504             circuit.wires,
505             0,
506             circuit.autoJunctions
507         );
508         if (ideal?.segments && !ideal.blank) return true;
509     }
510     for (const comp of circuit.components) {
511         if (comp.type !== 'bargraph_dc10h') continue;
512         const ideal = getIdealBargraphFromArduino(
513             comp.label,
514             circuit.components,
515             circuit.wires,
516             0,
517             circuit.autoJunctions
518         );
519         if (ideal?.segments && Object.values(ideal.segments).some(Boolean)) return true;
520     }
521     for (const comp of circuit.components) {
522         if (comp.type !== 'voltmeter') continue;
523         const v = getIdealVoltmeterVoltage(
524             comp.label,
525             circuit.components,
526             circuit.wires,
527             0,
528             circuit.autoJunctions
529         );
530         if (v !== null && Number.isFinite(v)) return true;
531     }
532     for (const comp of circuit.components) {
533         if (comp.type !== 'grove_lcd16x2') continue;
534         if (isGroveLcdWiredToUno(comp.label, circuit.components, circuit.wires, circuit.autoJunctions)) {
535             return true;
536         }
537     }
538     return false;
539 }
540
541 /** Affichage 7 segments piloté par sketch Arduino (sans attendre SPICE). */
542 export function getIdealSeg7Display(label) {
543     syncArduinoSketchesFromEditor();
544     return getIdealSeg7FromArduino(
545         label,
546         circuit.components,
547         circuit.wires,
548         0,
549         circuit.autoJunctions
550     );
551 }
552
553 /** Bargraph DC10H piloté directement par GPIO Arduino (sans attendre SPICE). */
554 export function getIdealBargraphDisplay(label, tSec) {
555     syncArduinoSketchesFromEditor();
556     const elapsed =
557         tSec ??
558         (anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0);

```

```

559     return getIdealBargraphFromArduino(
560         label,
561         circuit.components,
562         circuit.wires,
563         elapsed,
564         circuit.autoJunctions
565     );
566 }
567
568 export { hasArduinoStaticIdealDisplay };
569
570 function getIdealArduinoLedCurrent(label) {
571     const drive = anim.arduinoLedDrive[label];
572     if (!drive?.pinLabel || !drive?.unoLabel) return null;
573     const uno = circuit.components.find((c) => c.type === 'arduino_uno' && c.label === drive.unoLabel);
574     if (!uno) return null;
575     applyArduinoSketchToComponent(uno);
576     const elapsed = anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
577     const levels = resolvePinLevelsAt(uno, elapsed);
578     const lv = levels[drive.pinLabel];
579     if (lv === 1) return ARDUINO_IDEAL_LED_ON_A;
580     if (lv === 0) return 0;
581     return null;
582 }
583
584 function hasArduinoTimeVaryingGpio() {
585     syncArduinoSketchesFromEditor();
586     return circuit.components.some((c) => c.type === 'arduino_uno' && arduinoGpioIsTimeVarying(c));
587 }
588
589 function hasArduinoInteractiveSketch() {
590     syncArduinoSketchesFromEditor();
591     return circuit.components.some((c) => {
592         if (c.type !== 'arduino_uno') return false;
593         const sk = c.sketch || '';
594         return sketchHasLoop(sk) || sketchUsesAnalogInput(sk) || sketchUsesSerial(sk);
595     });
596 }
597
598 function hasArduinoLoopSimulation() {
599     return hasArduinoInteractiveSketch();
600 }
601
602 function hasDht22Simulation() {
603     syncArduinoSketchesFromEditor();
604     return circuit.components.some(
605         (c) => c.type === 'grove_dht22' && isGroveDht22WiredToUno(c.label, circuit.components, circuit.wires, circuit.autoJunctions)
606     );
607 }
608
609 export function ensureArduinoLedAnimation() {
610     if (!flags.isSimulating) return;
611     indexArduinoLedDrives();
612     if (
613         !hasArduinoTimeVaryingGpio() &&
614         !hasArduinoLoopSimulation() &&
615         !hasDht22Simulation() &&
616         !Object.keys(anim.arduinoLedDrive).length &&
617         !hasArduinoStaticIdealDisplay() &&
618         !hasLcdTimeVaryingDisplay()
619     ) {
620         return;
621     }
622     if (anim.startMs <= 0) anim.startMs = performance.now();
623     if (anim.rafId != null) return;
624     const tick = () => {
625         updateArduinoRuntimes();
626         redraw();
627         anim.rafId = requestAnimationFrame(tick);
628     };
629     anim.rafId = requestAnimationFrame(tick);
630 }
631
632 function fallbackPeriodSec(plots) {
633     const freqHz = getSourceFrequencyHz();
634     if (freqHz > 0) return 1 / freqHz;
635     const plot = Object.values(plots)[0];
636     if (plot?.time?.length > 1) {
637         const span = plot.time[plot.time.length - 1] - plot.time[0];
638         if (span > 0) return span;

```

```

639     }
640     return 1;
641 }
642
643 /** Période détectée dans le tracé SPICE (ex. diviseur par 2 → 2× l'horloge). */
644 function detectLedPeriodSec(plot) {
645     const { time, current } = plot;
646     if (!time?.length || !current?.length) return null;
647
648     const rising = [];
649     for (let i = 1; i < current.length; i++) {
650         if (current[i] > LED_ON_A && current[i - 1] <= LED_ON_A) rising.push(time[i]);
651     }
652     if (rising.length >= 2) return rising[1] - rising[0];
653
654     let toggles = 0;
655     for (let i = 1; i < current.length; i++) {
656         if ((current[i] > LED_ON_A) !== (current[i - 1] > LED_ON_A)) toggles++;
657     }
658     const span = time[time.length - 1] - time[0];
659     if (toggles >= 2 && span > 0) return (2 * span) / toggles;
660     return null;
661 }
662
663 function interpolateSeries(time, values, tSec) {
664     if (!time?.length || !values?.length) return NaN;
665     if (tSec <= time[0]) return values[0] ?? NaN;
666     const last = time.length - 1;
667     if (tSec >= time[last]) return values[last] ?? NaN;
668     let lo = 0;
669     let hi = last;
670     while (hi - lo > 1) {
671         const mid = (lo + hi) >> 1;
672         if (time[mid] <= tSec) lo = mid;
673         else hi = mid;
674     }
675     const t0 = time[lo];
676     const t1 = time[hi];
677     const v0 = values[lo];
678     const v1 = values[hi];
679     if (t1 <= t0) return v0;
680     const f = (tSec - t0) / (t1 - t0);
681     return v0 + f * (v1 - v0);
682 }
683
684 function seg7LitFromVoltages(segmentV, vCom) {
685     const lit = {};
686     const vc = Number.isFinite(vCom) ? vCom : 0;
687     for (let i = 0; i < 7; i++) {
688         const v = segmentV[i];
689         lit[SEG7_NAMES[i]] = Number.isFinite(v) && v - vc >= SEG7_LIT_DELTA_V;
690     }
691     return lit;
692 }
693
694 function bargraphLitFromVoltages(segmentV, vCom) {
695     const lit = {};
696     const vc = Number.isFinite(vCom) ? vCom : 0;
697     for (let i = 0; i < 10; i++) {
698         const v = segmentV[i];
699         lit[BARGRAPH_SEG_NAMES[i]] = Number.isFinite(v) && v - vc >= SEG7_LIT_DELTA_V;
700     }
701     return lit;
702 }
703
704 function interpolatePlot(plot, tSec) {
705     const { time, current } = plot;
706     if (!time?.length) return 0;
707     if (tSec <= time[0]) return current[0] ?? 0;
708     const last = time.length - 1;
709     if (tSec >= time[last]) return current[last] ?? 0;
710     let lo = 0;
711     let hi = last;
712     while (hi - lo > 1) {
713         const mid = (lo + hi) >> 1;
714         if (time[mid] <= tSec) lo = mid;
715         else hi = mid;
716     }
717     const t0 = time[lo];
718     const t1 = time[hi];

```

```

719     const i0 = current[lo] ?? 0;
720     const i1 = current[hi] ?? 0;
721     if (t1 <= t0) return i0;
722     const f = (tSec - t0) / (t1 - t0);
723     return i0 + f * (i1 - i0);
724 }
725
726 function averageCurrentOverPeriod(plot, periodSec) {
727     const { time, current } = plot;
728     if (!time?.length) return 0;
729     const tEnd = time[0] + periodSec;
730     let sum = 0;
731     let count = 0;
732     for (let i = 0; i < time.length; i++) {
733         if (time[i] >= tEnd) break;
734         sum += current[i] ?? 0;
735         count++;
736     }
737     return count > 0 ? sum / count : 0;
738 }
739
740 function prepareLedTiming(plots) {
741     const fallback = fallbackPeriodSec(plots);
742     anim.ledPeriods = {};
743     anim.ledPersistence = {};
744     anim.steadyCurrent = {};
745
746     for (const [id, plot] of Object.entries(plots)) {
747         const period = detectLedPeriodSec(plot) ?? fallback;
748         anim.ledPeriods[id] = period;
749         const freqHz = period > 0 ? 1 / period : 0;
750         if (freqHz > PERSISTENCE_FREQ_HZ) {
751             anim.ledPersistence[id] = true;
752             anim.steadyCurrent[id] = averageCurrentOverPeriod(plot, period);
753         }
754     }
755 }
756
757 function interpolateVoltagePlot(plot, tSec) {
758     const { time, voltage } = plot;
759     if (!time?.length) return 0;
760     if (tSec <= time[0]) return voltage[0] ?? 0;
761     const last = time.length - 1;
762     if (tSec >= time[last]) return voltage[last] ?? 0;
763     let lo = 0;
764     let hi = last;
765     while (hi - lo > 1) {
766         const mid = (lo + hi) >> 1;
767         if (time[mid] <= tSec) lo = mid;
768         else hi = mid;
769     }
770     const t0 = time[lo];
771     const t1 = time[hi];
772     const v0 = voltage[lo] ?? 0;
773     const v1 = voltage[hi] ?? 0;
774     if (t1 <= t0) return v0;
775     return v0 + ((tSec - t0) / (t1 - t0)) * (v1 - v0);
776 }
777
778 function detectPlotPeriodSec(time, values, threshold = 2.5) {
779     if (!time?.length || !values?.length) return null;
780     const rising = [];
781     for (let i = 1; i < values.length; i++) {
782         if (values[i] > threshold && values[i - 1] <= threshold) rising.push(time[i]);
783     }
784     if (rising.length >= 2) return rising[1] - rising[0];
785     let toggles = 0;
786     for (let i = 1; i < values.length; i++) {
787         if ((values[i] > threshold) !== (values[i - 1] > threshold)) toggles++;
788     }
789     const span = time[time.length - 1] - time[0];
790     if (toggles >= 2 && span > 0) return (2 * span) / toggles;
791     return null;
792 }
793
794 function prepareVmTiming(vmPlots) {
795     anim.vmPeriods = {};
796     const fallback = fallbackPeriodSec({});
797     for (const [id, plot] of Object.entries(vmPlots)) {
798         anim.vmPeriods[id] = detectPlotPeriodSec(plot.time, plot.voltage) ?? fallback;

```



```

799     }
800 }
801
802 export function getAnimatedVoltmeterVoltage(label) {
803     syncArduinoSketchesFromEditor();
804     const elapsed = anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
805     const ideal = getIdealVoltmeterVoltage(label, circuit.components, circuit.wires, elapsed, circuit.autoJunctions);
806     if (ideal !== null && Number.isFinite(ideal)) return ideal;
807     const plot = anim.vmPlots[label];
808     if (!plot?.time?.length) return null;
809     const period = anim.vmPeriods[label] ?? 1;
810     const elapsedPlot = (performance.now() - anim.startMs) / 1000;
811     // Temps réel : ne pas figer sur la phase stable des compteurs ripple (réservée aux LED).
812     const tSample = plotTimeOrigin(elapsedPlot, period);
813     const plotSpan = plot.time[plot.time.length - 1] - plot.time[0];
814     const tAbs = plot.time[0] + (plotSpan > 0 ? tSample % plotSpan : tSample);
815     return quantizeVoltmeterReading(interpolateVoltagePlot(plot, tAbs), plot.voltage);
816 }
817
818 export function hasVoltmeterAnimation() {
819     return Object.keys(anim.vmPlots).length > 0 && anim.rafId !== null;
820 }
821
822 export function getAnimatedLedCurrent(label) {
823     if (syncHc90MasterResetClock()) {
824         const qMap = hc90QBitForLed(label, circuit.components, circuit.wires, circuit.autoJunctions);
825         if (qMap) return 0;
826     }
827     const clockPeriod = getGimpPeriodSec();
828     const cascade = anim.hc90Cascade ?? detectHc90Cascade(circuit.components, circuit.wires, circuit.autoJunctions);
829     const qMap = hc90QBitForLed(label, circuit.components, circuit.wires, circuit.autoJunctions);
830     if (qMap && shouldUseIdealHc90Counting(cascade, clockPeriod)) {
831         const bcd = getAnimatedHc90Bcd(qMap.hc90Label);
832         if (bcd !== null) {
833             return (bcd >> qMap.qIndex) & 1 ? HC90_IDEAL_LED_ON_A : 0;
834         }
835     }
836     // LED câblée sur GPIO Arduino : le sketch interprété prime sur d'anciennes courbes .tran SPICE.
837     if (anim.arduinoLedDrive[label]) {
838         const ideal = getIdealArduinoLedCurrent(label);
839         if (ideal !== null) return ideal;
840     }
841     const plot = anim.plots[label];
842     if (!plot) {
843         const ideal = getIdealArduinoLedCurrent(label);
844         if (ideal !== null) return ideal;
845         return null;
846     }
847     if (anim.ledPersistence[label]) {
848         return anim.steadyCurrent[label] ?? 0;
849     }
850     const period = anim.ledPeriods[label] ?? 1;
851     const elapsed = (performance.now() - anim.startMs) / 1000;
852     const plotSpan = plot.time[plot.time.length - 1] - plot.time[0];
853     const tSample =
854         hasHc90DecadeCounter() && plotSpan > 0 && clockPeriod > 0
855         ? hc90TranSampleTimeSec(elapsed, clockPeriod, plotSpan)
856         : sampleTimeSec(elapsed, period);
857     const tAbs = plot.time[0] + (plotSpan > 0 ? Math.min(tSample, plotSpan - 1e-12) : tSample);
858     return interpolatePlot(plot, tAbs);
859 }
860
861 export function hasLedAnimation() {
862     return anim.rafId !== null && Object.keys(anim.plots).length > 0;
863 }
864
865 export function hasSeg7Animation() {
866     return anim.rafId !== null && Object.keys(anim.seg7Plots).length > 0;
867 }
868
869 /** Segments allumés à l'instant courant de la simulation live (.tran). */
870 export function getAnimatedSeg7Segments(label) {
871     syncArduinoSketchesFromEditor();
872     updateArduinoRuntimes();
873     const elapsed = anim.startMs > 0 ? (performance.now() - anim.startMs) / 1000 : 0;
874     const ideal = getIdealSeg7FromArduino(label, circuit.components, circuit.wires, elapsed, circuit.autoJunctions);
875     if (ideal?.segments) return ideal;
876     const plot = anim.seg7Plots[label];
877     if (!plot?.time?.length) return null;
878     if (syncHc90MasterResetClock()) return { segments: bcdDigitToSeg7Segments(0) };

```

```

879     const hc90Label = hc90LabelForSeg7(label, circuit.components, circuit.wires, circuit.autoJunctions);
880     if (hc90Label) {
881         const bcd = getAnimatedHc90Bcd(hc90Label);
882         if (bcd != null) return { segments: bcdDigitToSeg7Segments(bcd) };
883     }
884     const clockPeriod = getGimpPeriodSec() ?? 1;
885     const elapsedPlot = (performance.now() - anim.startMs) / 1000;
886     if (
887         shouldUseIdealRippleMod10Seg7(
888             label,
889             circuit.components,
890             circuit.wires,
891             circuit.autoJunctions,
892             clockPeriod
893         )
894     ) {
895         return { segments: bcdDigitToSeg7Segments(idealRippleMod10Bcd(elapsedPlot, clockPeriod)) };
896     }
897     const plotSpan = plot.time[plot.time.length - 1] - plot.time[0];
898     const tSample =
899         hasHc90DecadeCounter() && plotSpan > 0 && clockPeriod > 0
900         ? hc90TranSampleTimeSec(elapsedPlot, clockPeriod, plotSpan)
901         : sampleTimeSec(elapsedPlot, clockPeriod);
902     const tAbs = plot.time[0] + (plotSpan > 0 ? tSample % plotSpan : tSample);
903     const vCom = interpolateSeries(plot.time, plot.common, tAbs);
904     const segmentV = SEG7_NAMES.map((n) => interpolateSeries(plot.time, plot.segments[n], tAbs));
905     return { segments: seg7LitFromVoltages(segmentV, vCom) };
906 }
907
908 /** Segments bargraph allumés à l'instant courant de la simulation live (.tran). */
909 export function getAnimatedBargraphSegments(label) {
910     const plot = anim.bargraphPlots[label];
911     if (!plot?.time?.length) {
912         const ideal = getIdealBargraphDisplay(label);
913         return ideal?.segments ? { segments: ideal.segments } : null;
914     }
915     const clockPeriod = getGimpPeriodSec() ?? 1;
916     const elapsedPlot = (performance.now() - anim.startMs) / 1000;
917     const plotSpan = plot.time[plot.time.length - 1] - plot.time[0];
918     const tSample = sampleTimeSec(elapsedPlot, clockPeriod);
919     const tAbs = plot.time[0] + (plotSpan > 0 ? tSample % plotSpan : tSample);
920     const vCom = interpolateSeries(plot.time, plot.common, tAbs);
921     const segmentV = BARGRAPH_SEG_NAMES.map((n) => interpolateSeries(plot.time, plot.segments[n], tAbs));
922     return { segments: bargraphLitFromVoltages(segmentV, vCom) };
923 }
924
925 export function startLedAnimation(plots, vmPlots = {}, seg7Plots = {}, logicGateTranPlots = {}, opts = {}) {
926     const mrActive = isHc90MasterResetActive(circuit.components, circuit.wires, circuit.autoJunctions);
927     const mrReleased = anim.hc90MrActive && !mrActive;
928     const savedStartMs = opts.keepClock === true && !mrReleased ? anim.startMs : 0;
929     stopLedAnimation(opts.keepClock === true && !mrReleased);
930     const hasLeds = plots && Object.keys(plots).length > 0;
931     const hasVm = vmPlots && Object.keys(vmPlots).length > 0;
932     const hasSeg7 = seg7Plots && Object.keys(seg7Plots).length > 0;
933     const bargraphPlots = opts.bargraphPlots || {};
934     const hasBargraph = bargraphPlots && Object.keys(bargraphPlots).length > 0;
935     anim.hc90QPlots = indexHc90QPlots(logicGateTranPlots);
936     anim.hc90Cascade = detectHc90Cascade(circuit.components, circuit.wires, circuit.autoJunctions);
937     anim.hc90MrActive = mrActive;
938     const hasHc90Anim = Object.keys(anim.hc90QPlots).length > 0;
939     if (!hasLeds && !hasVm && !hasSeg7 && !hasBargraph && !hasHc90Anim) return;
940
941     anim.plots = plots || {};
942     anim.vmPlots = vmPlots || {};
943     anim.seg7Plots = seg7Plots || {};
944     anim.bargraphPlots = bargraphPlots || {};
945     anim.startMs = savedStartMs > 0 ? savedStartMs : performance.now();
946     if (hasLeds) prepareLedTiming(plots);
947     if (hasVm) prepareVmTiming(vmPlots);
948
949     const needsFrameLoop =
950         Object.keys(anim.plots).some((id) => !anim.ledPersistence[id]) ||
951         hasVm ||
952         hasSeg7 ||
953         hasBargraph ||
954         hasHc90Anim;
955     if (!needsFrameLoop) {
956         redraw();
957         return;
958     }

```

```

959
960     const tick = () => {
961         updateArduinoRuntimes();
962         redraw();
963         anim.rafId = requestAnimationFrame(tick);
964     };
965     anim.rafId = requestAnimationFrame(tick);
966 }
967
968 let smokeRafId = null;
969
970 export function startBurntLedSmokeLoop() {
971     if (smokeRafId !== null) return;
972     const tick = () => {
973         redraw();
974         smokeRafId = requestAnimationFrame(tick);
975     };
976     smokeRafId = requestAnimationFrame(tick);
977 }
978
979 function stopBurntLedSmokeLoop() {
980     if (smokeRafId !== null) cancelAnimationFrame(smokeRafId);
981     smokeRafId = null;
982 }
983
984 export function stopLedAnimation(preserveArduino = false) {
985     if (anim.rafId !== null) cancelAnimationFrame(anim.rafId);
986     anim.rafId = null;
987     anim.plots = {};
988     anim.vmPlots = {};
989     anim.seg7Plots = {};
990     anim.bargraphPlots = {};
991     anim.hc90QPlots = {};
992     anim.hc90LastBcd = {};
993     anim.hc90MrActive = false;
994     anim.hc90Cascade = null;
995     anim.ledPeriods = {};
996     anim.vmPeriods = {};
997     anim.ledPersistence = {};
998     anim.steadyCurrent = {};
999     anim.arduinoLedDrive = {};
1000     anim.startMs = 0;
1001     if (!preserveArduino) resetArduinoRuntimes();
1002     stopBurntLedSmokeLoop();
1003 }
1004

```

```
1  /** Affichage des modales sans bloquer la barre de menus. */
2
3  export function showModal(el) {
4      if (!el) return;
5      el.classList.add('is-open');
6      el.classList.remove('hidden');
7      el.style.removeProperty('display');
8  }
9
10 export function hideModal(el) {
11     if (!el) return;
12     el.classList.remove('is-open');
13     el.classList.add('hidden');
14     el.style.display = 'none';
15 }
16
17 export function initModalUi() {
18     for (const el of document.querySelectorAll('.modal')) {
19         if (!el.classList.contains('is-open')) hideModal(el);
20     }
21 }
22
```

```

1 // renderrer.js
2 import { canvas, ctx, GRID_SIZE, scale, pan, flags, circuit, interaction, zone, menuDrag, snapToGrid, simulationResults } from './state.js';
3 import {
4     CD4511_BOX_B,
5     CD4511_BOX_L,
6     CD4511_BOX_R,
7     CD4511_BOX_T,
8     CD4511_JUNC_L,
9     CD4511_JUNC_R,
10    CD4511_LABEL_L,
11    CD4511_LABEL_R,
12    CD4511_PIN_Y,
13    CD4511_HIT_DX,
14    CD4511_HIT_DY,
15 } from './cd4511-layout.js';
16 import {
17     IC90_BOX_B,
18     IC90_BOX_L,
19     IC90_BOX_R,
20     IC90_BOX_T,
21     IC90_JUNC_L,
22     IC90_JUNC_R,
23     IC90_LABEL_L,
24     IC90_LABEL_R,
25     IC90_HIT_DX,
26     IC90_HIT_DY,
27     IC90_LEFT_PIN_Y,
28     IC90_RIGHT_PIN_Y,
29     IC90_Q_STACK_INDICES,
30 } from './ic74hc90-layout.js';
31 import {
32     UNO_BOX_B,
33     UNO_BOX_L,
34     UNO_BOX_R,
35     UNO_BOX_T,
36     UNO_JUNC_L,
37     UNO_JUNC_R,
38     UNO_LABEL_L,
39     UNO_LABEL_R,
40     UNO_HIT_DX,
41     UNO_HIT_DY,
42     UNO_LEFT_PINS,
43     UNO_RIGHT_PINS,
44     UNO_LEFT_PIN_Y,
45     UNO_RIGHT_PIN_Y,
46     UNO_DIGITAL_PINS,
47     formatUnoPinLabel,
48 } from './arduino-uno-layout.js';
49 import { getComponentJunctions, isJunctionConnected, getVoltageAtJunction } from './geometry.js';
50 import { getBottomPanelHeight } from './source-panel.js';
51 import { getArduinoPanelWidth } from './arduino-editor.js';
52 import { isGroveLcdWiredToUno } from './Engine/grove-lcd-ideal.mjs';
53 import {
54     GROVE_LCD_CONN_L,
55     GROVE_LCD_BOX_L,
56     GROVE_LCD_BOX_R,
57     GROVE_LCD_BOX_T,
58     GROVE_LCD_BOX_B,
59     GROVE_LCD_JUNC_X,
60     GROVE_LCD_PIN_Y,
61     GROVE_LCD_HIT_DX,
62     GROVE_LCD_HIT_DY,
63     GROVE_LCD_PINS,
64     GROVE_LCD_COLS,
65     GROVE_LCD_ROWS,
66     GROVE_LCD_BEZEL,
67     GROVE_LCD_CONNECTOR_W,
68     GROVE_LCD_SCREEN_L,
69     GROVE_LCD_SCREEN_R,
70     GROVE_LCD_SCREEN_T,
71     GROVE_LCD_SCREEN_B,
72     GROVE_LCD_SEL_L,
73     GROVE_LCD_SEL_T,
74     GROVE_LCD_SEL_W,
75     GROVE_LCD_SEL_H,
76     GROVE_LCD_PIN_LABEL_X,
77     GROVE_LCD_STUB_LEN,
78 } from './grove-lcd-layout.js';

```

```

79 import {
80   GROVE_DHT22_CONN_L,
81   GROVE_DHT22_BOX_L,
82   GROVE_DHT22_BOX_R,
83   GROVE_DHT22_BOX_T,
84   GROVE_DHT22_BOX_B,
85   GROVE_DHT22_JUNC_X,
86   GROVE_DHT22_PIN_Y,
87   GROVE_DHT22_PINS,
88   GROVE_DHT22_CONNECTOR_W,
89   GROVE_DHT22_SENSOR_L,
90   GROVE_DHT22_SENSOR_R,
91   GROVE_DHT22_SENSOR_T,
92   GROVE_DHT22_SENSOR_B,
93   GROVE_DHT22_SEL_L,
94   GROVE_DHT22_SEL_T,
95   GROVE_DHT22_SEL_W,
96   GROVE_DHT22_SEL_H,
97   GROVE_DHT22_PIN_LABEL_X,
98 } from './dht22-layout.js';
99 import {
100   DC10H_SEG_COUNT,
101   DC10H_SEG_NAMES,
102   DC10H_PIN_Y,
103   DC10H_BOX_L,
104   DC10H_BOX_R,
105   DC10H_BOX_T,
106   DC10H_BOX_B,
107   DC10H_BAR_H,
108   DC10H_COM_X,
109   DC10H_COM_Y,
110   DC10H_JUNC_L,
111   DC10H_JUNC_R,
112   DC10H_SEL_L,
113   DC10H_SEL_T,
114   DC10H_SEL_W,
115   DC10H_SEL_H,
116   DC10H_COMP_LABEL_OFFSET,
117   DC10H_TYPE_LABEL_OFFSET,
118   DC10H_PIN_LABEL_OFFSET_X,
119   DC10H_COM_LABEL_OFFSET_X,
120   dc10hBarTopY,
121   dc10hComX,
122   dc10hPalette,
123 } from './bargraph-dc10h-layout.js';
124 import {
125   getHd44780Glyph,
126   HD44780_CHAR_W,
127   HD44780_CHAR_H,
128   HD44780_NATIVE_W,
129 } from './hd44780-font.js';
130 import { getAnimatedHc90Bcd, getAnimatedLedCurrent, getAnimatedSeg7Segments, getAnimatedBargraphSegments, getAnimatedVoltmeterVoltage, getIdealSeg7Display, getIdealBargraphDisplay, getAnimatedGroveLcdDisplay, isLedOvercurrent, quantizeVoltmeterReading } from './led-animation.js';
131 import { isSpeakerAudioPlaying } from './speaker-audio.js';
132 import { COLORS } from './theme.js';
133 function hc90SimCount(comp) {
134   if (!flags.isSimulating || !comp?.label) return null;
135   const bcdAnim = getAnimatedHc90Bcd(comp.label);
136   if (bcdAnim !== null) return bcdAnim;
137   let n = 0;
138   let any = false;
139   for (let i = 0; i < 4; i++) {
140     const lv = simulationResults.logicValues?.[`${comp.label}_Q${i}`];
141     if (lv && lv.logic === 1) {
142       n |= 1 << i;
143       any = true;
144     } else if (lv && lv.logic === 0) any = true;
145   }
146   return any ? n : null;
147 }
148
149 export function resizeCanvas() {
150   const container = canvas.parentElement;
151   const w = container?.clientWidth > 0
152     ? container.clientWidth
153     : Math.max(200, window.innerWidth - getArduinoPanelWidth());
154   const h = container?.clientHeight > 0
155     ? container.clientHeight
156     : Math.max(200, window.innerHeight - 48 - getBottomPanelHeight());
157   canvas.width = Math.max(200, Math.floor(w));

```

```

158     canvas.height = Math.max(200, Math.floor(h));
159     draw();
160 }
161
162 function drawUprightText(angle, fn) {
163     ctx.save();
164     if (angle) ctx.rotate(-angle * Math.PI / 180);
165     fn();
166     ctx.restore();
167 }
168
169 /** Texte horizontal à un point (x,y) du repère composant déjà pivoté. */
170 function drawUprightTextAt(angle, x, y, fn) {
171     ctx.save();
172     ctx.translate(x, y);
173     if (angle) ctx.rotate(-angle * Math.PI / 180);
174     fn();
175     ctx.restore();
176 }
177
178 /** Décalage en repère composant (tourne avec le symbole), puis texte horizontal. */
179 function drawUprightTextAtLocal(angle, x, y, localOffX, localOffY, fn) {
180     ctx.save();
181     ctx.translate(x, y);
182     ctx.translate(localOffX, localOffY);
183     if (angle) ctx.rotate(-angle * Math.PI / 180);
184     fn();
185     ctx.restore();
186 }
187
188 function drawOutlinedText(text) {
189     ctx.lineWidth = 3;
190     ctx.lineJoin = 'round';
191     ctx.strokeStyle = 'rgba(8, 8, 12, 0.92)';
192     ctx.strokeText(text, 0, 0);
193     ctx.fillStyle = COLORS.ink;
194     ctx.fillText(text, 0, 0);
195 }
196
197 function formatDisplayValue(val) {
198     if (val == null || val === '') return '';
199     return String(val).trim()
200         .replace(/Kohm/gi, 'K')
201         .replace(/Mohm/gi, 'M')
202         .replace(/ohm/gi, '');
203 }
204
205 function drawLabels(name, value, angle) {
206     const labelX = -38;
207     const nameY = -36;
208     drawUprightText(angle, () => {
209         if (name) {
210             ctx.fillStyle = COLORS.ink;
211             ctx.font = '12px Arial';
212             ctx.textAlign = 'left';
213             ctx.textBaseline = 'top';
214             ctx.fillText(name, labelX, nameY);
215         }
216         const display = formatDisplayValue(value);
217         if (display) {
218             ctx.fillStyle = COLORS.inkMuted;
219             ctx.font = '11px Arial';
220             ctx.textAlign = 'left';
221             ctx.textBaseline = 'top';
222             ctx.fillText(display, labelX, name ? nameY + 14 : nameY);
223         }
224     });
225 }
226
227 function formatMeterValue(num, decimals = 1) {
228     if (typeof num !== 'number' || !Number.isFinite(num)) return '0.0';
229     return num.toFixed(decimals);
230 }
231
232 function formatCurrentDisplay(amps) {
233     if (typeof amps !== 'number' || !Number.isFinite(amps)) return '0.0';
234     const abs = Math.abs(amps);
235     if (abs >= 1) return amps.toFixed(2);
236     if (abs >= 0.001) return (amps * 1000).toFixed(1);
237     return (amps * 1e6).toFixed(0);

```

```

238 }
239
240 function formatCurrentUnit(amps) {
241     if (typeof amps !== 'number' || !Number.isFinite(amps)) return 'A';
242     const abs = Math.abs(amps);
243     if (abs >= 1) return 'A';
244     if (abs >= 0.001) return 'mA';
245     return 'µA';
246 }
247
248 function formatResistanceDisplay(ohms) {
249     if (typeof ohms !== 'number' || !Number.isFinite(ohms)) return 'Ω';
250     if (ohms >= 1e6) return (ohms / 1e6).toFixed(1);
251     if (ohms >= 1e3) return (ohms / 1e3).toFixed(1);
252     if (ohms >= 10) return ohms.toFixed(0);
253     return ohms.toFixed(1);
254 }
255
256 function formatResistanceUnit(ohms) {
257     if (typeof ohms !== 'number' || !Number.isFinite(ohms)) return 'Ω';
258     if (ohms >= 1e6) return 'MΩ';
259     if (ohms >= 1e3) return 'kΩ';
260     return 'Ω';
261 }
262
263 /** « + » à côté de la borne droite (+), au-dessus du fil – pas sur le trait. */
264 function drawWirePlusLabel(wireEndX = 40, wireY = 0) {
265     ctx.font = 'bold 12px Arial';
266     ctx.textAlign = 'center';
267     ctx.textBaseline = 'bottom';
268     ctx.fillStyle = '#ff5252';
269     ctx.fillText('+', wireEndX, wireY - 6);
270 }
271
272 function drawMeterBody(color = '#00bcd4', showPlus = true) {
273     ctx.strokeStyle = color; ctx.lineWidth = 2; ctx.fillStyle = COLORS.meterFill;
274     ctx.beginPath(); ctx.arc(0, 0, 20, 0, Math.PI * 2); ctx.fill(); ctx.stroke();
275     ctx.strokeStyle = color;
276     ctx.beginPath(); ctx.moveTo(-40, 0); ctx.lineTo(-20, 0); ctx.moveTo(20, 0); ctx.lineTo(40, 0); ctx.stroke();
277     if (showPlus) drawWirePlusLabel(40, 0);
278 }
279
280 function drawOscilloscopeScreen(comp) {
281     const x0 = -42, y0 = -24, w = 84, h = 48;
282     ctx.fillStyle = COLORS.scopeBg;
283     ctx.fillRect(x0, y0, w, h);
284     ctx.strokeStyle = COLORS.scopeGrid;
285     ctx.lineWidth = 1;
286     const divW = w / 8;
287     const divH = h / 8;
288     for (let i = 1; i < 8; i++) {
289         ctx.beginPath(); ctx.moveTo(x0 + i * divW, y0); ctx.lineTo(x0 + i * divW, y0 + h); ctx.stroke();
290     }
291     for (let j = 1; j < 8; j++) {
292         ctx.beginPath(); ctx.moveTo(x0, y0 + j * divH); ctx.lineTo(x0 + w, y0 + j * divH); ctx.stroke();
293     }
294     ctx.fillStyle = COLORS.scopeLabel;
295     ctx.font = 'bold 11px Arial';
296     ctx.textAlign = 'center';
297     ctx.textBaseline = 'middle';
298     ctx.fillText('Osci', 0, 0);
299     ctx.strokeStyle = '#00bcd4';
300     ctx.lineWidth = 2;
301     ctx.strokeRect(x0, y0, w, h);
302     ctx.fillStyle = COLORS.ink;
303     ctx.font = '12px Arial';
304     ctx.textAlign = 'center';
305     ctx.textBaseline = 'bottom';
306     ctx.fillText(comp.label, 0, y0 - 6);
307 }
308
309 const SEG7_PIN_Y = [-60, -40, -20, 0, 20, 40, 60];
310 const SEG7_NAMES = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
311
312 function seg7SetFrom(segments) {
313     return new Set(Object.entries(segments).filter(([, on]) => on).map([k] => k));
314 }
315
316 function getSeg7LitSet(comp) {
317     if (!flags.isSimulating) return new Set();

```



```

318     const anim = getAnimatedSeg7Segments(comp.label);
319     if (anim?.segments) return seg7SetFrom(anim.segments);
320     const data = simulationResults.seg7?.[comp.label];
321     if (data?.segments) return seg7SetFrom(data.segments);
322     const idealArduino = getIdealSeg7Display(comp.label);
323     if (idealArduino?.segments && !idealArduino.blank) {
324         return seg7SetFrom(idealArduino.segments);
325     }
326     return new Set();
327 }
328
329 function drawSeg7Display(comp) {
330     const lit = getSeg7LitSet(comp);
331     const segColor = (name) => (lit.has(name) ? '#ff1744' : '#4a0808');
332     const pinYs = SEG7_PIN_Y;
333     const names = SEG7_NAMES;
334
335     // Style des fils/pattes : rouge comme une LED
336     const pinColor = '#ff1744';
337     const comX = 20;
338     const comY = 100;
339
340     // Boîtier plus large (droit)
341     // Centré sur x=20 pour aligner le fil de cathode et sa jonction.
342     const boxL = -18, boxR = 58, boxT = -78, boxB = 80;
343
344     // Pattes a..g : du point de connexion (pastille) jusqu'au bord gauche du boîtier (en rouge)
345     ctx.strokeStyle = pinColor;
346     ctx.lineWidth = 3;
347     for (let i = 0; i < 7; i++) {
348         ctx.beginPath();
349         ctx.moveTo(-40, pinYs[i]);
350         ctx.lineTo(boxL, pinYs[i]);
351         ctx.stroke();
352     }
353     // Patte commune : sort du bas vers la pastille commune (centrée + plus longue)
354     ctx.beginPath();
355     ctx.moveTo(comX, boxB);
356     ctx.lineTo(comX, comY);
357     ctx.stroke();
358
359     // Boîtier droit (non incliné)
360     ctx.strokeStyle = COLORS.componentStroke;
361     ctx.lineWidth = 2;
362     ctx.strokeRect(boxL, boxT, boxR - boxL, boxB - boxT);
363
364     // Segments du chiffre centrés dans le boîtier, légèrement inclinés
365     ctx.save();
366     ctx.transform(1, 0, -0.14, 1, 0, 0);
367
368     const segMarginX = 15;
369     const segMarginTop = 22;
370     const segMarginBottom = 24;
371     const leftX = boxL + segMarginX;
372     const rightX = boxR - segMarginX;
373     const topY = boxT + segMarginTop;
374     const botY = boxB - segMarginBottom;
375     const midY = (topY + botY) / 2;
376     ctx.lineCap = 'round';
377     ctx.lineWidth = 6;
378     const seg = (x1, y1, x2, y2, name) => {
379         ctx.strokeStyle = segColor(name);
380         ctx.beginPath();
381         ctx.moveTo(x1, y1);
382         ctx.lineTo(x2, y2);
383         ctx.stroke();
384     };
385     const hInset = 4;
386     seg(leftX + hInset, topY, rightX - hInset, topY, 'a'); // haut
387     seg(rightX, topY + 5, rightX, midY - 5, 'b'); // haut-droite
388     seg(rightX, midY + 5, rightX, botY - 5, 'c'); // bas-droite
389     seg(leftX + hInset, botY, rightX - hInset, botY, 'd'); // bas
390     seg(leftX, midY + 5, leftX, botY - 5, 'e'); // bas-gauche
391     seg(leftX, topY + 5, leftX, midY - 5, 'f'); // haut-gauche
392     seg(leftX + hInset, midY, rightX - hInset, midY, 'g'); // milieu
393     ctx.restore();
394
395     // Étiquettes des broches a..g, au-dessus de chaque patte près du boîtier
396     ctx.fillStyle = COLORS.ink;
397     ctx.font = 'bold 12px Arial';

```

```

398   ctx.textAlign = 'right';
399   ctx.textBaseline = 'bottom';
400   for (let i = 0; i < 7; i++) ctx.fillText(names[i], boxL - 4, pinYs[i] - 2);
401   // Étiquette de la borne commune C
402   ctx.textAlign = 'left';
403   ctx.textBaseline = 'middle';
404   ctx.fillText('C', comX + 8, comY);
405   // Nom du composant
406   ctx.textAlign = 'center';
407   ctx.textBaseline = 'bottom';
408   ctx.fillText(comp.label, 26, boxT - 6);
409 }
410
411 function bargraphLitSetFrom(segments) {
412   return new Set(Object.entries(segments).filter(([, on]) => on).map(([k]) => k));
413 }
414
415 function getBargraphLitSet(comp) {
416   if (!flags.isSimulating) return new Set();
417   const anim = getAnimatedBargraphSegments(comp.label);
418   if (anim?.segments) return bargraphLitSetFrom(anim.segments);
419   const data = simulationResults.bargraph?.[comp.label];
420   const spiceSet = data?.segments ? bargraphLitSetFrom(data.segments) : new Set();
421   const ideal = getIdealBargraphDisplay(comp.label);
422   const idealSet = ideal?.segments ? bargraphLitSetFrom(ideal.segments) : new Set();
423   if (spiceSet.size > 0) return spiceSet;
424   if (idealSet.size > 0) return idealSet;
425   return spiceSet;
426 }
427
428 function drawBargraphLabel(text, x, y, opts = {}) {
429   const {
430     font = 'bold 8px Arial',
431     align = 'center',
432     baseline = 'middle',
433     outlined = false,
434     fill = COLORS.ink,
435   } = opts;
436   ctx.font = font;
437   ctx.textAlign = align;
438   ctx.textBaseline = baseline;
439   if (outlined) drawOutlinedTextAt(text, x, y, { font, align, baseline });
440   else {
441     ctx.fillStyle = fill;
442     ctx.fillText(text, x, y);
443   }
444 }
445
446 function drawBargraphDc10h(comp) {
447   const lit = getBargraphLitSet(comp);
448   const pal = dc10hPalette(comp.barColor || 'red');
449   const segColor = (name) => (lit.has(name) ? pal.lit : pal.dim);
450   const pinYs = DC10H_PIN_Y;
451   const names = DC10H_SEG_NAMES;
452   const pinColor = pal.pin;
453   const flip = !!comp.flipX;
454
455   const boxL = DC10H_BOX_L;
456   const boxR = DC10H_BOX_R;
457   const boxT = DC10H_BOX_T;
458   const boxB = DC10H_BOX_B;
459   const comY = DC10H_COM_Y;
460
461   const juncX = flip ? DC10H_JUNC_R : DC10H_JUNC_L;
462   const boxPinX = flip ? DC10H_BOX_R : DC10H_BOX_L;
463   const comX = dc10hComX(flip);
464
465   ctx.strokeStyle = pinColor;
466   ctx.lineWidth = 2.5;
467   ctx.lineCap = 'round';
468   for (let i = 0; i < DC10H_SEG_COUNT; i++) {
469     ctx.beginPath();
470     ctx.moveTo(juncX, pinYs[i]);
471     ctx.lineTo(boxPinX, pinYs[i]);
472     ctx.stroke();
473   }
474   ctx.beginPath();
475   ctx.moveTo(comX, boxB);
476   ctx.lineTo(comX, comY);
477   ctx.stroke();

```

```

478
479   ctx.strokeStyle = COLORS.componentStroke;
480   ctx.lineWidth = 2;
481   ctx.strokeRect(boxL, boxT, boxR - boxL, boxB - boxT);
482
483   const barL = boxL + 6;
484   const barR = boxR - 6;
485   for (let i = 0; i < DC10H_SEG_COUNT; i++) {
486     const segName = names[i];
487     const y = dc10hBarTopY(i);
488     ctx.fillStyle = segColor(segName);
489     ctx.fillRect(barL, y, barR - barL, DC10H_BAR_H);
490     if (lit.has(segName)) {
491       ctx.fillStyle = 'rgba(255,255,255,0.15)';
492       ctx.fillRect(barL, y, barR - barL, DC10H_BAR_H * 0.35);
493     }
494   }
495
496   const cx = (boxL + boxR) / 2;
497   const pinLabelX = juncX + (flip ? -DC10H_PIN_LABEL_OFFSET_X : DC10H_PIN_LABEL_OFFSET_X);
498   const comLabelX = comX + (flip ? -DC10H_COM_LABEL_OFFSET_X : DC10H_COM_LABEL_OFFSET_X);
499
500   for (let i = 0; i < DC10H_SEG_COUNT; i++) {
501     drawBargraphLabel(names[i], pinLabelX, pinYs[i], {
502       font: 'bold 8px Arial',
503       align: flip ? 'left' : 'right',
504       baseline: 'middle',
505       outlined: true,
506     });
507   }
508
509   drawBargraphLabel('COM', comLabelX, (boxB + comY) / 2, {
510     font: 'bold 8px Arial',
511     align: flip ? 'right' : 'left',
512     baseline: 'middle',
513     outlined: true,
514   });
515
516   drawBargraphLabel(comp.label, cx, boxT - DC10H_COMP_LABEL_OFFSET, {
517     font: '11px Arial',
518     align: 'center',
519     baseline: 'bottom',
520     outlined: true,
521   });
522
523   drawBargraphLabel('DC10H', cx, comY + DC10H_TYPE_LABEL_OFFSET, {
524     font: '9px Arial',
525     align: 'center',
526     baseline: 'top',
527     fill: '#aaa',
528   });
529 }
530
531 function drawOutlinedTextAt(text, x, y, { font, align, baseline }) {
532   ctx.font = font;
533   ctx.textAlign = align;
534   ctx.textBaseline = baseline;
535   ctx.lineWidth = 3;
536   ctx.lineJoin = 'round';
537   ctx.strokeStyle = 'rgba(8, 8, 12, 0.92)';
538   ctx.strokeText(text, x, y);
539   ctx.fillStyle = COLORS.ink;
540   ctx.fillText(text, x, y);
541 }
542
543 function getGroveLcdDisplayState(comp) {
544   if (!comp?.label) {
545     return { lines: ['', ''], backlight: false, wired: false, blank: true };
546   }
547   if (!flags.isSimulating) {
548     const wired = isGroveLcdWiredToUno(
549       comp.label,
550       circuit.components,
551       circuit.wires,
552       circuit.autoJunctions
553     );
554     return { lines: ['', ''], backlight: false, wired, blank: true, rgb: null };
555   }
556   return getAnimatedGroveLcdDisplay(comp.label);
557 }

```

```

558
559 /** Grille native HD44780 : 16x5 = 80 px/ligne, 2x8 = 16 px haut → 1280 points. */
560 const LCD_NATIVE_H = HD44780_CHAR_H * GROVE_LCD_ROWS;
561 /** Marge 1 px natif (haut, gauche, droite) pour ne pas toucher le cadre vert. */
562 const LCD_MARGIN_L = 1;
563 const LCD_MARGIN_T = 1;
564 const LCD_MARGIN_R = 1;
565 const LCD_FRAME_W = HD44780_NATIVE_W + LCD_MARGIN_L + LCD_MARGIN_R;
566 const LCD_FRAME_H = LCD_NATIVE_H + LCD_MARGIN_T;
567
568 function groveLcdTextColor(state, screenOn) {
569   if (!screenOn) return state.wired ? '#1a2a1a' : '#6a8a6a';
570   const rgb = state.rgb;
571   if (rgb && rgb.r !== null) {
572     const lum = (0.299 * rgb.r + 0.587 * rgb.g + 0.114 * rgb.b) / 255;
573     return lum > 0.45 ? '#111111' : '#e8e8e8';
574   }
575   return '#111111';
576 }
577
578 function groveLcdBacklightFill(state, screenOn) {
579   if (!screenOn) return '#1e2a1e';
580   const rgb = state.rgb;
581   if (rgb && rgb.r !== null) {
582     return `rgb(${rgb.r | 0},${rgb.g | 0},${rgb.b | 0})`;
583   }
584   return '#c5d84a';
585 }
586
587 function drawGroveLcdCharGrid(scrL, scrT, scrW, scrH, lines, screenOn, state, flipX, fx) {
588   if (state.blank && state.wired) return;
589   const fg = groveLcdTextColor(state, screenOn);
590   const pxW = scrW / LCD_FRAME_W;
591   const pxH = scrH / LCD_FRAME_H;
592
593   ctx.save();
594   ctx.fillStyle = fg;
595   ctx.imageSmoothingEnabled = false;
596   for (let row = 0; row < GROVE_LCD_ROWS; row++) {
597     const line = String(lines[row] || '').padEnd(GROVE_LCD_COLS, ' ').slice(0, GROVE_LCD_COLS);
598     for (let col = 0; col < GROVE_LCD_COLS; col++) {
599       const glyph = getHd44780Glyph(line[col]);
600       const baseNx = col * HD44780_CHAR_W;
601       const baseNy = row * HD44780_CHAR_H;
602       for (let r = 0; r < HD44780_CHAR_H; r++) {
603         const bits = glyph[r] & 0x1f;
604         for (let c = 0; c < HD44780_CHAR_W; c++) {
605           if ((bits >> (4 - c)) & 1) {
606             const nx = baseNx + c;
607             const ny = baseNy + r;
608             const x = scrL + (LCD_MARGIN_L + nx) * pxW;
609             const y = scrT + (LCD_MARGIN_T + ny) * pxH;
610             const xa = fx(x);
611             const xb = fx(x + pxW);
612             ctx.fillRect(Math.min(xa, xb), y, Math.abs(pxW), pxH);
613           }
614         }
615       }
616     }
617   }
618   ctx.restore();
619 }
620
621 function drawGroveLcd16x2(comp) {
622   const state = getGroveLcdDisplayState(comp);
623   const flip = comp.flipX ? -1 : 1;
624   const fx = (x) => flip * x;
625   const fillRectFx = (x, y, w, h) => {
626     const xa = fx(x);
627     const xb = fx(x + w);
628     ctx.fillRect(Math.min(xa, xb), y, Math.abs(w), h);
629   };
630   const strokeRectFx = (x, y, w, h) => {
631     const xa = fx(x);
632     const xb = fx(x + w);
633     ctx.strokeRect(Math.min(xa, xb), y, Math.abs(w), h);
634   };
635   const drawTextFx = (text, x, y) => {
636     ctx.save();
637     if (comp.flipX) {

```

```

638         ctx.translate(fx(x), y);
639         ctx.scale(-1, 1);
640         ctx.fillText(text, 0, 0);
641     } else {
642         ctx.fillText(text, x, y);
643     }
644     ctx.restore();
645 };
646
647 const boxL = GROVE_LCD_BOX_L;
648 const boxR = GROVE_LCD_BOX_R;
649 const boxT = GROVE_LCD_BOX_T;
650 const boxB = GROVE_LCD_BOX_B;
651 const connL = GROVE_LCD_CONN_L;
652
653 const scrL = GROVE_LCD_SCREEN_L;
654 const scrT = GROVE_LCD_SCREEN_T;
655 const scrW = GROVE_LCD_SCREEN_R - GROVE_LCD_SCREEN_L;
656 const scrH = GROVE_LCD_SCREEN_B - GROVE_LCD_SCREEN_T;
657
658 // Connecteur Grove (colonne gauche, 1 pas)
659 ctx.fillStyle = '#141414';
660 fillRectFx(connL, boxT, GROVE_LCD_CONNECTOR_W, boxB - boxT);
661
662 // Cadre noir
663 ctx.fillStyle = '#1c1c1c';
664 ctx.strokeStyle = '#0a0a0a';
665 ctx.lineWidth = 2;
666 fillRectFx(boxL, boxT, boxR - boxL, boxB - boxT);
667 strokeRectFx(boxL, boxT, boxR - boxL, boxB - boxT);
668
669 // Écran vert 16x2 (couleur rétroéclairage jaune-vert)
670 const screenOn = flags.isSimulating
671     && state.wired
672     && state.backlight !== false
673     && !state.blank;
674 ctx.fillStyle = groveLcdBacklightFill(state, screenOn);
675 fillRectFx(scrL, scrT, scrW, scrH);
676
677 const lines = state.lines || ['', ''];
678 drawGroveLcdCharGrid(scrL, scrT, scrW, scrH, lines, screenOn, state, comp.flipX, fx);
679
680 // Fils/pattes vers les jonctions
681 ctx.strokeStyle = COLORS.strokeMuted;
682 ctx.lineWidth = 2.5;
683 ctx.lineCap = 'round';
684 for (let i = 0; i < GROVE_LCD_PINS.length; i++) {
685     const py = GROVE_LCD_PIN_Y[i];
686     ctx.beginPath();
687     ctx.moveTo(fx(GROVE_LCD_JUNC_X), py);
688     ctx.lineTo(fx(connL), py);
689     ctx.stroke();
690 }
691
692 const pinStep = GROVE_LCD_PIN_Y.length > 1
693     ? GROVE_LCD_PIN_Y[1] - GROVE_LCD_PIN_Y[0]
694     : 14;
695 const labelGap = 3;
696 const pinLabelFontSize = Math.min(9, Math.max(7, Math.floor(pinStep - labelGap - 2)));
697 const labelX = GROVE_LCD_PIN_LABEL_X;
698 ctx.font = `bold ${pinLabelFontSize}px Arial`;
699 ctx.textAlign = 'center';
700 ctx.textBaseline = 'bottom';
701 ctx.fillStyle = COLORS.ink;
702 for (let i = 0; i < GROVE_LCD_PINS.length; i++) {
703     const py = GROVE_LCD_PIN_Y[i];
704     drawTextFx(GROVE_LCD_PINS[i], labelX, py - labelGap);
705 }
706
707 ctx.fillStyle = COLORS.ink;
708 ctx.font = '11px Arial';
709 ctx.textAlign = 'center';
710 ctx.textBaseline = 'top';
711 drawTextFx(comp.label, 0, boxB + 10);
712 }
713
714 function drawGroveDht22SensorGrid(sx, sy, sw, sh, fx) {
715     const cols = 6;
716     const rows = 3;
717     const gap = 2;

```

```

718     const cellW = (sw - gap * (cols - 1)) / cols;
719     const cellH = (sh - gap * (rows - 1)) / rows;
720     ctx.fillStyle = '#f4f4f0';
721     ctx.strokeStyle = '#c8c8c0';
722     ctx.lineWidth = 1;
723     for (let r = 0; r < rows; r++) {
724         for (let c = 0; c < cols; c++) {
725             const x = sx + c * (cellW + gap);
726             const y = sy + r * (cellH + gap);
727             const xa = fx(x);
728             const xb = fx(x + cellW);
729             ctx.fillRect(Math.min(xa, xb), y, Math.abs(cellW), cellH);
730             ctx.strokeRect(Math.min(xa, xb), y, Math.abs(cellW), cellH);
731             const hx = x + cellW * 0.22;
732             const hy = y + cellH * 0.22;
733             const hw = cellW * 0.56;
734             const hh = cellH * 0.56;
735             ctx.fillStyle = '#d8d8d0';
736             const hxa = fx(hx);
737             const hxb = fx(hx + hw);
738             ctx.fillRect(Math.min(hxa, hxb), hy, Math.abs(hw), hh);
739             ctx.fillStyle = '#f4f4f0';
740         }
741     }
742 }
743
744 function drawGroveDht22(comp) {
745     const flip = comp.flipX ? -1 : 1;
746     const fx = (x) => flip * x;
747     const fillRectFx = (x, y, w, h) => {
748         const xa = fx(x);
749         const xb = fx(x + w);
750         ctx.fillRect(Math.min(xa, xb), y, Math.abs(w), h);
751     };
752     const strokeRectFx = (x, y, w, h) => {
753         const xa = fx(x);
754         const xb = fx(x + w);
755         ctx.strokeRect(Math.min(xa, xb), y, Math.abs(w), h);
756     };
757     const drawTextFx = (text, x, y) => {
758         ctx.save();
759         if (comp.flipX) {
760             ctx.translate(fx(x), y);
761             ctx.scale(-1, 1);
762             ctx.fillText(text, 0, 0);
763         } else {
764             ctx.fillText(text, x, y);
765         }
766         ctx.restore();
767     };
768
769     const boxL = GROVE_DHT22_BOX_L;
770     const boxR = GROVE_DHT22_BOX_R;
771     const boxT = GROVE_DHT22_BOX_T;
772     const boxB = GROVE_DHT22_BOX_B;
773     const connL = GROVE_DHT22_CONN_L;
774
775     ctx.fillStyle = '#141414';
776     fillRectFx(connL, boxT, GROVE_DHT22_CONNECTOR_W, boxB - boxT);
777
778     ctx.fillStyle = '#1a3d7a';
779     ctx.strokeStyle = '#0f2448';
780     ctx.lineWidth = 2;
781     fillRectFx(boxL, boxT, boxR - boxL, boxB - boxT);
782     strokeRectFx(boxL, boxT, boxR - boxL, boxB - boxT);
783
784     const sx = GROVE_DHT22_SENSOR_L;
785     const sy = GROVE_DHT22_SENSOR_T;
786     const sw = GROVE_DHT22_SENSOR_R - GROVE_DHT22_SENSOR_L;
787     const sh = GROVE_DHT22_SENSOR_B - GROVE_DHT22_SENSOR_T;
788     drawGroveDht22SensorGrid(sx, sy, sw, sh, fx);
789
790     ctx.strokeStyle = COLORS.strokeMuted;
791     ctx.lineWidth = 2.5;
792     ctx.lineCap = 'round';
793     for (let i = 0; i < GROVE_DHT22_PINS.length; i++) {
794         const py = GROVE_DHT22_PIN_Y[i];
795         ctx.beginPath();
796         ctx.moveTo(fx(GROVE_DHT22_JUNC_X), py);
797         ctx.lineTo(fx(connL), py);

```

```

798     ctx.stroke();
799 }
800
801 const pinStep = GROVE_DHT22_PIN_Y.length > 1
802     ? GROVE_DHT22_PIN_Y[1] - GROVE_DHT22_PIN_Y[0]
803     : 14;
804 const labelGap = 3;
805 const pinLabelFontSize = Math.min(8, Math.max(6, Math.floor(pinStep - labelGap - 2)));
806 ctx.font = `bold ${pinLabelFontSize}px Arial`;
807 ctx.textAlign = 'center';
808 ctx.textBaseline = 'bottom';
809 ctx.fillStyle = COLORS.ink;
810 for (let i = 0; i < GROVE_DHT22_PINS.length; i++) {
811     const py = GROVE_DHT22_PIN_Y[i];
812     drawTextFx(GROVE_DHT22_PINS[i], GROVE_DHT22_PIN_LABEL_X, py - labelGap);
813 }
814
815 ctx.fillStyle = COLORS.ink;
816 ctx.font = '10px Arial';
817 ctx.textAlign = 'center';
818 ctx.textBaseline = 'top';
819 drawTextFx('DHT22', 0, boxT - 12);
820 drawTextFx(comp.label, 0, boxB + 8);
821 }
822
823 function drawLogicGateSymbol(gateType) {
824     const inTopY = -20;
825     const inBottomY = 20;
826     const inMidY = 0;
827     const gateLeft = -20;
828     const gateArcR = 20;
829     const outBubbleR = (gateType === 'nand' || gateType === 'nor' || gateType === 'xnor' || gateType === 'not') ? 4 : 0;
830     const isOrFamily = gateType === 'or' || gateType === 'nor' || gateType === 'xor' || gateType === 'xnor';
831     const isXorFamily = gateType === 'xor' || gateType === 'xnor';
832     const isNot = gateType === 'not';
833
834     ctx.strokeStyle = '#00ca71';
835     ctx.lineWidth = 2;
836     ctx.fillStyle = COLORS.componentFill;
837
838     if (isNot) {
839         ctx.beginPath();
840         ctx.moveTo(-16, -18);
841         ctx.lineTo(14, 0);
842         ctx.lineTo(-16, 18);
843         ctx.closePath();
844         ctx.fill();
845         ctx.stroke();
846     } else if (isOrFamily) {
847         ctx.beginPath();
848         ctx.moveTo(-20, -20);
849         ctx.quadraticCurveTo(0, -20, 14, 0);
850         ctx.quadraticCurveTo(0, 20, -20, 20);
851         ctx.quadraticCurveTo(-8, 0, -20, -20);
852         ctx.closePath();
853         ctx.fill();
854         ctx.stroke();
855
856         if (isXorFamily) {
857             ctx.beginPath();
858             ctx.moveTo(-25, -20);
859             ctx.quadraticCurveTo(-13, 0, -25, 20);
860             ctx.stroke();
861         }
862     } else {
863         // Forme AND / NAND
864         ctx.beginPath();
865         ctx.moveTo(gateLeft, -20);
866         ctx.lineTo(0, -20);
867         ctx.arc(0, 0, gateArcR, -Math.PI / 2, Math.PI / 2);
868         ctx.lineTo(gateLeft, 20);
869         ctx.closePath();
870         ctx.fill();
871         ctx.stroke();
872     }
873
874     const outStemStart = 20 + outBubbleR;
875     if (isNot) {
876         // La pointe du triangle est à x=14
877         const notStemStart = 14 + outBubbleR;

```

```

878     ctx.beginPath();
879     ctx.moveTo(notStemStart, 0);
880     ctx.lineTo(40, 0);
881     ctx.stroke();
882 } else {
883     ctx.beginPath();
884     ctx.moveTo(outStemStart, 0);
885     ctx.lineTo(40, 0);
886     ctx.stroke();
887 }
888
889 if (isNot) {
890     ctx.beginPath();
891     ctx.moveTo(-40, 0);
892     ctx.lineTo(-16, 0);
893     ctx.stroke();
894 } else {
895     ctx.beginPath();
896     ctx.moveTo(-40, inTopY);
897     ctx.lineTo(gateLeft, inTopY);
898     ctx.moveTo(-40, inBottomY);
899     ctx.lineTo(gateLeft, inBottomY);
900     ctx.stroke();
901 }
902
903 if (outBubbleR > 0) {
904     const cx = isNot ? 14 + outBubbleR : 20 + outBubbleR;
905     ctx.beginPath();
906     ctx.arc(cx, 0, outBubbleR, 0, Math.PI * 2);
907     ctx.stroke();
908     ctx.fillStyle = COLORS.componentFill;
909     ctx.fill();
910 }
911
912 ctx.fillStyle = COLORS.inkMuted;
913 ctx.font = '10px Arial';
914 ctx.textAlign = 'left';
915 if (isNot) {
916     ctx.fillText('A', -34, inMidY - 4);
917 } else {
918     ctx.fillText('A', -34, inTopY - 4);
919     ctx.fillText('B', -34, inBottomY - 4);
920 }
921 }
922
923 function drawMeterDisplay(valuePart, unitPart, rot) {
924     const text = unitPart ? `${valuePart}${unitPart}` : String(valuePart);
925     ctx.save();
926     ctx.rotate(-rot * Math.PI / 180);
927     ctx.font = 'bold 7px monospace';
928     const textW = ctx.measureText(text).width;
929     const boxW = Math.max(28, Math.min(46, textW + 10));
930     ctx.fillStyle = COLORS.meterDisplayBg;
931     ctx.fillRect(-boxW / 2, -7, boxW, 14);
932     ctx.fillStyle = '#00ff66';
933     ctx.textAlign = 'center';
934     ctx.textBaseline = 'middle';
935     ctx.fillText(text, 0, 0);
936     ctx.restore();
937 }
938
939 function getLedCurrentAmps(comp) {
940     if (!flags.isSimulating) return null;
941     const animI = getAnimatedLedCurrent(comp.label);
942     if (animI !== null) return animI;
943     const ledMeasure = simulationResults.leds && simulationResults.leds[comp.label];
944     if (ledMeasure && typeof ledMeasure.current === 'number') return ledMeasure.current;
945     return null;
946 }
947
948 function drawLedSmoke(phase) {
949     ctx.save();
950     ctx.strokeStyle = COLORS.ledSmoke;
951     ctx.lineWidth = 1.5;
952     for (let i = 0; i < 3; i++) {
953         const xOff = -6 + i * 6 + Math.sin(phase + i * 1.3) * 3;
954         ctx.beginPath();
955         ctx.moveTo(xOff, -16);
956         ctx.quadraticCurveTo(
957             xOff + Math.sin(phase + i * 2) * 5, -26,

```



```

958         xOff + Math.cos(phase + i) * 4, -36 - i * 5
959     );
960     ctx.stroke();
961 }
962 ctx.restore();
963 }
964
965 /** Set/Reset : trait vertical uniquement à l'extérieur du boîtier ; jonction au bout ; S/R à gauche du fil. */
966 function drawFlipFlopSetReset(ctx, boxTopY, boxBottomY, stubOutside = 30) {
967     const setJuncY = boxTopY - stubOutside;
968     const resetJuncY = boxBottomY + stubOutside;
969     ctx.beginPath();
970     ctx.moveTo(0, setJuncY); ctx.lineTo(0, boxTopY);
971     ctx.moveTo(0, resetJuncY); ctx.lineTo(0, boxBottomY);
972     ctx.stroke();
973     ctx.fillStyle = COLORS.ink;
974     ctx.font = '10px Arial';
975     ctx.textBaseline = 'middle';
976     ctx.textAlign = 'right';
977     ctx.fillText('S', -10, (setJuncY + boxTopY) / 2);
978     ctx.fillText('R', -10, (resetJuncY + boxBottomY) / 2);
979 }
980
981 function drawComponentBody(comp) {
982     ctx.save(); ctx.translate(comp.x, comp.y);
983     const noRotate = comp.type === 'gimp' || comp.type === 'gsin' || comp.type === 'gsqr' || comp.type === 'oscilloscope' || comp.type === 'd
_flipflop' || comp.type === 'jk_flipflop' || comp.type === 'cd4511' || comp.type === 'ic_74hc90' || comp.type === 'arduino_uno' || comp.type
=== 'npn' || comp.type === 'opamp' || comp.type === 'seg7' || comp.type === 'bargraph_dc10h' || comp.type === 'grove_lcd16x2' || comp.type ==
= 'grove_dht22';
984     const rot = noRotate ? 0 : (comp.rotation || 0);
985     ctx.rotate(rot * Math.PI / 180);
986
987     if (interaction.selectedComponents.includes(comp)) {
988         ctx.strokeStyle = '#00bcd4'; ctx.lineWidth = 1.5;
989         if (comp.type === 'cd4511') {
990             ctx.strokeRect(-CD4511_HIT_DX, -CD4511_HIT_DY, CD4511_HIT_DX * 2, CD4511_HIT_DY * 2);
991         }
992         else if (comp.type === 'ic_74hc90') {
993             ctx.strokeRect(-IC90_HIT_DX, -IC90_HIT_DY, IC90_HIT_DX * 2, IC90_HIT_DY * 2);
994         }
995         else if (comp.type === 'jk_flipflop' || comp.type === 'd_flipflop') ctx.strokeRect(-45, -68, 90, 136);
996         else if (comp.type === 'oscilloscope') ctx.strokeRect(-50, -38, 100, 100);
997         else if (comp.type === 'npn') ctx.strokeRect(-42, -42, 64, 84);
998         else if (comp.type === 'opamp') ctx.strokeRect(-44, -40, 88, 80);
999         else if (comp.type === 'seg7') ctx.strokeRect(-52, -86, 124, 200);
1000         else if (comp.type === 'bargraph_dc10h') {
1001             ctx.strokeRect(DC10H_SEL_L, DC10H_SEL_T, DC10H_SEL_W, DC10H_SEL_H);
1002         }
1003         else if (comp.type === 'grove_lcd16x2') {
1004             const flip = comp.flipX ? -1 : 1;
1005             const fx = (x) => flip * x;
1006             const sell = Math.min(fx(GROVE_LCD_SEL_L), fx(GROVE_LCD_SEL_L + GROVE_LCD_SEL_W));
1007             ctx.strokeRect(sell, GROVE_LCD_SEL_T, GROVE_LCD_SEL_W, GROVE_LCD_SEL_H);
1008         }
1009         else if (comp.type === 'grove_dht22') {
1010             const flip = comp.flipX ? -1 : 1;
1011             const fx = (x) => flip * x;
1012             const sell = Math.min(fx(GROVE_DHT22_SEL_L), fx(GROVE_DHT22_SEL_L + GROVE_DHT22_SEL_W));
1013             ctx.strokeRect(sell, GROVE_DHT22_SEL_T, GROVE_DHT22_SEL_W, GROVE_DHT22_SEL_H);
1014         }
1015         else if (comp.type === 'logic_terminal') ctx.strokeRect(-14, -10, 24, 20);
1016         else if (comp.type !== 'logic_terminal') ctx.strokeRect(-45, -25, 90, 50);
1017     }
1018
1019     const railLead = GRID_SIZE;
1020     if (comp.type === 'gnd') {
1021         ctx.strokeStyle = COLORS.inkDim; ctx.lineWidth = 2;
1022         ctx.beginPath(); ctx.moveTo(-12, 0); ctx.lineTo(railLead, 0);
1023         ctx.moveTo(-12, -10); ctx.lineTo(-12, 10);
1024         ctx.moveTo(-17, -6); ctx.lineTo(-17, 6);
1025         ctx.moveTo(-22, -3); ctx.lineTo(-22, 3);
1026         ctx.stroke(); drawLabels(comp.label, "0V", rot);
1027     }
1028     else if (comp.type === 'vcc') {
1029         ctx.strokeStyle = '#ff3d00'; ctx.lineWidth = 2;
1030         ctx.beginPath(); ctx.moveTo(-12, 0); ctx.lineTo(railLead, 0);
1031         ctx.moveTo(-12, 0); ctx.lineTo(-4, -7); ctx.lineTo(-4, 7); ctx.closePath(); ctx.fillStyle = '#ff3d00'; ctx.fill();
1032         ctx.stroke();
1033         let val = comp.value !== undefined ? comp.value + "V" : "5V";
1034         drawLabels(comp.label, val, rot);

```

```

1035 }
1036 else if (comp.type === 'logic_terminal') {
1037   if (interaction.selectedComponents.includes(comp)) {
1038     ctx.strokeStyle = '#00bcd4'; ctx.lineWidth = 1.5;
1039     ctx.strokeRect(-14, -10, 24, 20);
1040   }
1041   ctx.strokeStyle = '#9c27b0'; ctx.lineWidth = 2;
1042   ctx.beginPath(); ctx.moveTo(8, 0); ctx.lineTo(railLead, 0);
1043   ctx.stroke();
1044   ctx.fillStyle = COLORS.componentFill; ctx.fillRect(-12, -8, 20, 16); ctx.strokeRect(-12, -8, 20, 16);
1045   let state = comp.state || 0;
1046   ctx.fillStyle = state === 1 ? '#00e676' : '#ff1744';
1047   ctx.font = 'bold 11px Arial'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
1048   ctx.fillText(state, -2, 0);
1049 }
1050 else if (comp.type === 'battery') {
1051   ctx.strokeStyle = '#ff9800'; ctx.lineWidth = 2;
1052   // Grande barre (gauche, haute) = + ; petite barre (droite, basse) = -
1053   ctx.beginPath(); ctx.moveTo(-5, -15); ctx.lineTo(-5, 15); ctx.stroke();
1054   ctx.beginPath(); ctx.moveTo(5, -8); ctx.lineTo(5, 8); ctx.stroke();
1055   ctx.beginPath();
1056   ctx.moveTo(-40, 0); ctx.lineTo(-5, 0);
1057   ctx.moveTo(5, 0); ctx.lineTo(40, 0);
1058   ctx.stroke();
1059   drawWirePlusLabel(-40, 0);
1060   let val = comp.value !== undefined ? comp.value + "V" : "5V";
1061   drawLabels(comp.label, val, rot);
1062 }
1063 else if (comp.type === 'resistor') {
1064   ctx.strokeStyle = '#007acc'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1065   ctx.fillRect(-20, -10, 40, 20); ctx.strokeRect(-20, -10, 40, 20);
1066   ctx.beginPath(); ctx.moveTo(-40, 0); ctx.lineTo(-20, 0); ctx.moveTo(20, 0); ctx.lineTo(40, 0); ctx.stroke();
1067   drawLabels(comp.label, comp.value || '1K', rot);
1068 }
1069 else if (comp.type === 'potentiometer') {
1070   const pos = Math.min(100, Math.max(0, comp.position ?? 50));
1071   const wx = -20 + (pos / 100) * 40;
1072   ctx.strokeStyle = '#007acc'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1073   ctx.fillRect(-20, -10, 40, 20); ctx.strokeRect(-20, -10, 40, 20);
1074   ctx.beginPath();
1075   ctx.moveTo(-40, 0); ctx.lineTo(-20, 0);
1076   ctx.moveTo(20, 0); ctx.lineTo(40, 0);
1077   ctx.moveTo(wx, -10); ctx.lineTo(wx, -22);
1078   ctx.moveTo(0, -22); ctx.lineTo(wx, -22);
1079   ctx.stroke();
1080   ctx.fillStyle = '#ff9800';
1081   ctx.font = 'bold 13px Arial';
1082   ctx.textAlign = 'center';
1083   ctx.textBaseline = 'middle';
1084   ctx.fillText('◀', -9, 18);
1085   ctx.fillText('▶', 9, 18);
1086   const potValue = `${formatDisplayValue(comp.value || '10k')} (${Math.round(pos)}%)`;
1087   drawLabels(comp.label, potValue, rot);
1088 }
1089 else if (comp.type === 'switch_spdt') {
1090   const onA = (comp.state ?? 0) !== 1;
1091   ctx.strokeStyle = COLORS.componentStroke; ctx.lineWidth = 2;
1092   ctx.beginPath();
1093   ctx.moveTo(-40, -18); ctx.lineTo(-14, -18);
1094   ctx.moveTo(-40, 0); ctx.lineTo(-14, 0);
1095   ctx.moveTo(-40, 18); ctx.lineTo(-14, 18);
1096   ctx.stroke();
1097   ctx.fillStyle = COLORS.componentFill;
1098   ctx.fillRect(-12, -18, 24, 36);
1099   ctx.strokeStyle = COLORS.componentStroke;
1100   ctx.strokeRect(-12, -18, 24, 36);
1101   ctx.beginPath();
1102   ctx.moveTo(0, 0);
1103   if (onA) ctx.lineTo(0, -14);
1104   else ctx.lineTo(0, 14);
1105   ctx.stroke();
1106   ctx.fillStyle = COLORS.inkDim;
1107   ctx.font = '9px Arial';
1108   ctx.textAlign = 'right';
1109   ctx.textBaseline = 'middle';
1110   ctx.fillText('COM', -44, 0);
1111   ctx.fillText('B', -44, 18);
1112   drawLabels(comp.label, null, rot);
1113 }
1114 else if (comp.type === 'push_button') {

```

```

1115     const pressed = (comp.state ?? 0) === 1;
1116     ctx.strokeStyle = COLORS.componentStroke; ctx.lineWidth = 2;
1117     ctx.beginPath();
1118     ctx.moveTo(-40, 0); ctx.lineTo(-16, 0);
1119     ctx.moveTo(16, 0); ctx.lineTo(40, 0);
1120     ctx.stroke();
1121     ctx.beginPath();
1122     ctx.arc(-16, 0, 3, 0, Math.PI * 2);
1123     ctx.moveTo(16, 0);
1124     ctx.arc(16, 0, 3, 0, Math.PI * 2);
1125     ctx.fillStyle = COLORS.componentStroke;
1126     ctx.fill();
1127     const barY = pressed ? -3 : -9;
1128     ctx.strokeStyle = COLORS.componentStroke; ctx.lineWidth = 2;
1129     ctx.beginPath();
1130     ctx.moveTo(-16, barY); ctx.lineTo(16, barY);
1131     ctx.moveTo(0, barY); ctx.lineTo(0, barY - 8);
1132     ctx.stroke();
1133     ctx.fillStyle = pressed ? '#ef5350' : COLORS.componentFill;
1134     ctx.strokeStyle = COLORS.componentStroke; ctx.lineWidth = 2;
1135     ctx.beginPath();
1136     ctx.rect(-12, barY - 16, 24, 8);
1137     ctx.fill();
1138     ctx.stroke();
1139     if (comp.maintained) {
1140         ctx.fillStyle = COLORS.inkDim;
1141         ctx.font = '9px Arial';
1142         ctx.textAlign = 'center';
1143         ctx.textBaseline = 'bottom';
1144         ctx.fillText('M', 0, barY - 18);
1145     }
1146     drawLabels(comp.label, pressed ? 'ON' : 'OFF', rot);
1147 }
1148 else if (comp.type === 'capacitor') {
1149     ctx.strokeStyle = '#66bb6a'; ctx.lineWidth = 2;
1150     ctx.beginPath();
1151     ctx.moveTo(-40, 0); ctx.lineTo(-8, 0);
1152     ctx.moveTo(-8, -14); ctx.lineTo(-8, 14);
1153     ctx.moveTo(8, -14); ctx.lineTo(8, 14);
1154     ctx.moveTo(8, 0); ctx.lineTo(40, 0);
1155     ctx.stroke();
1156     drawLabels(comp.label, comp.value || '1u', rot);
1157 }
1158 else if (comp.type === 'inductor') {
1159     ctx.strokeStyle = '#ffa726'; ctx.lineWidth = 2;
1160     ctx.beginPath();
1161     ctx.moveTo(-40, 0); ctx.lineTo(-28, 0);
1162     for (let i = 0; i < 4; i++) {
1163         ctx.arc(-21 + i * 14, 0, 7, Math.PI, 0, false);
1164     }
1165     ctx.lineTo(40, 0);
1166     ctx.stroke();
1167     drawLabels(comp.label, comp.value || '1m', rot);
1168 }
1169 else if (comp.type === 'diode') {
1170     ctx.strokeStyle = '#ef5350'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1171     ctx.beginPath();
1172     ctx.moveTo(-40, 0); ctx.lineTo(-12, 0);
1173     ctx.moveTo(-12, -12); ctx.lineTo(12, 0); ctx.lineTo(-12, 12); ctx.closePath();
1174     ctx.fill(); ctx.stroke();
1175     ctx.beginPath();
1176     ctx.moveTo(12, -12); ctx.lineTo(12, 12);
1177     ctx.moveTo(12, 0); ctx.lineTo(40, 0);
1178     ctx.stroke();
1179     drawLabels(comp.label, comp.value || '1N4148', rot);
1180 }
1181 else if (comp.type === 'npn') {
1182     const bx = 0, by = 0;
1183     const cx = 20, cy = -20, cEnd = -40;
1184     const ex = 20, ey = 20, eEnd = 40;
1185     const fx = (x) => (comp.flipX ? -x : x);
1186     ctx.save();
1187     if (comp.flipX) ctx.scale(-1, 1);
1188     ctx.strokeStyle = COLORS.strokeLight;
1189     ctx.lineWidth = 3;
1190     ctx.beginPath();
1191     ctx.moveTo(bx, -20); ctx.lineTo(bx, 20);
1192     ctx.stroke();
1193     ctx.lineWidth = 2;
1194     ctx.beginPath();

```

```

1195     ctx.moveTo(-40, by); ctx.lineTo(bx, by);
1196     ctx.moveTo(bx, by); ctx.lineTo(cx, cy); ctx.lineTo(cx, cEnd);
1197     ctx.moveTo(bx, by); ctx.lineTo(ex, ey); ctx.lineTo(ex, eEnd);
1198     ctx.stroke();
1199     const ax = 13, ay = 13;
1200     const ux = 0.707, uy = 0.707, px = -uy, py = ux, wing = 4.5;
1201     const tipX = ax + ux * 5.5, tipY = ay + uy * 5.5;
1202     ctx.beginPath();
1203     ctx.moveTo(tipX, tipY);
1204     ctx.lineTo(ax - ux * 1.5 + px * wing, ay - uy * 1.5 + py * wing);
1205     ctx.moveTo(tipX, tipY);
1206     ctx.lineTo(ax - ux * 1.5 - px * wing, ay - uy * 1.5 - py * wing);
1207     ctx.stroke();
1208     ctx.restore();
1209     ctx.fillStyle = COLORS.inkMuted; ctx.font = '11px Arial'; ctx.textBaseline = 'middle';
1210     ctx.textAlign = 'left';
1211     ctx.fillText('b', fx(-34), -10);
1212     ctx.fillText('c', fx(4), -32);
1213     ctx.fillText('e', fx(4), 32);
1214     ctx.fillStyle = COLORS.ink; ctx.font = '12px Arial';
1215     ctx.fillText(comp.label, fx(-36), -24);
1216     ctx.textAlign = 'center';
1217     ctx.fillText('2N2222', fx(34), 2);
1218 }
1219 else if (comp.type === 'opamp') {
1220     const tLeft = -12, tTop = -30, tBot = 30, tApex = 28, inY = 20;
1221     const fx = (x) => (comp.flipX ? -x : x);
1222     const fy = (y) => (comp.flipY ? -y : y);
1223     ctx.save();
1224     if (comp.flipX) ctx.scale(-1, 1);
1225     if (comp.flipY) ctx.scale(1, -1);
1226     ctx.fillStyle = COLORS.opampFill;
1227     ctx.strokeStyle = COLORS.strokeMuted;
1228     ctx.lineWidth = 2.5;
1229     ctx.beginPath();
1230     ctx.moveTo(tLeft, tTop); ctx.lineTo(tLeft, tBot); ctx.lineTo(tApex, 0); ctx.closePath();
1231     ctx.fill(); ctx.stroke();
1232     ctx.strokeStyle = COLORS.strokeLight; ctx.lineWidth = 2;
1233     ctx.beginPath();
1234     ctx.moveTo(-40, -inY); ctx.lineTo(tLeft, -inY);
1235     ctx.moveTo(-40, inY); ctx.lineTo(tLeft, inY);
1236     ctx.moveTo(tApex, 0); ctx.lineTo(40, 0);
1237     ctx.stroke();
1238     ctx.restore();
1239     ctx.fillStyle = COLORS.ink; ctx.font = 'bold 10px Arial';
1240     ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
1241     ctx.fillText('+', fx(-8), fy(-inY));
1242     ctx.fillText('-', fx(-8), fy(inY));
1243     ctx.fillStyle = COLORS.ink; ctx.font = '12px Arial';
1244     ctx.fillText(comp.label, fx(8), fy(-38));
1245     ctx.fillStyle = COLORS.inkMuted;
1246     ctx.fillText(comp.value || 'uA741', fx(8), fy(42));
1247 }
1248 else if (['not', 'and', 'nand', 'or', 'nor', 'xor', 'xnor'].includes(comp.type)) {
1249     drawLogicGateSymbol(comp.type);
1250     drawLabels(comp.label, null, rot);
1251 }
1252 else if (comp.type === 'd_flipflop') {
1253     ctx.strokeStyle = '#00bcd4'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1254     ctx.fillRect(-30, -30, 60, 60); ctx.strokeRect(-30, -30, 60, 60);
1255     ctx.beginPath();
1256     ctx.moveTo(-40, -20); ctx.lineTo(-30, -20); // D
1257     ctx.moveTo(-40, 20); ctx.lineTo(-30, 20); // CLK
1258     ctx.moveTo(30, -20); ctx.lineTo(40, -20); // Q
1259     ctx.moveTo(30, 20); ctx.lineTo(40, 20); // /Q
1260     ctx.stroke();
1261     drawFlipFlopSetReset(ctx, -30, 30);
1262     ctx.fillStyle = COLORS.ink; ctx.font = '10px Arial'; ctx.textBaseline = 'middle';
1263     ctx.textAlign = 'left'; ctx.fillText('D', -27, -20); ctx.fillText('>', -27, 20);
1264     ctx.textAlign = 'right'; ctx.fillText('Q', 27, -20); ctx.fillText('Q', 27, 20);
1265     ctx.beginPath(); ctx.moveTo(20, 13); ctx.lineTo(27, 13); ctx.stroke();
1266     ctx.font = '12px Arial'; ctx.fillStyle = COLORS.ink; ctx.fillText(comp.label, 0, -62);
1267 }
1268 else if (comp.type === 'jk_flipflop') {
1269     ctx.strokeStyle = '#ff9800'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1270     ctx.fillRect(-30, -35, 60, 70); ctx.strokeRect(-30, -35, 60, 70);
1271     ctx.beginPath();
1272     ctx.moveTo(-40, -20); ctx.lineTo(-30, -20); // J
1273     ctx.moveTo(-40, 0); ctx.lineTo(-30, 0); // CLK
1274     ctx.moveTo(-40, 20); ctx.lineTo(-30, 20); // K

```

```

1275     ctx.moveTo(30, -20); ctx.lineTo(40, -20);    // Q
1276     ctx.moveTo(30, 20); ctx.lineTo(40, 20);      // /Q
1277     ctx.stroke();
1278     drawFlipFlopSetReset(ctx, -35, 35);
1279     ctx.fillStyle = COLORS.ink; ctx.font = '10px Arial'; ctx.textBaseline = 'middle';
1280     ctx.textAlign = 'left'; ctx.fillText('J', -27, -20); ctx.fillText('>', -27, 0); ctx.fillText('K', -27, 20);
1281     ctx.textAlign = 'right'; ctx.fillText('Q', 27, -20); ctx.fillText('Q', 27, 20);
1282     ctx.beginPath(); ctx.moveTo(20, 13); ctx.lineTo(27, 13); ctx.stroke();
1283     ctx.font = '12px Arial'; ctx.fillStyle = COLORS.ink; ctx.fillText(comp.label, 0, -64);
1284 }
1285 else if (comp.type === 'cd4511') {
1286     const inLbl = ['A', 'B', 'C', 'D', 'LE', 'BI', 'LT'];
1287     const outLbl = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
1288     ctx.strokeStyle = '#ab47bc'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1289     ctx.fillRect(CD4511_BOX_L, CD4511_BOX_T, CD4511_BOX_R - CD4511_BOX_L, CD4511_BOX_B - CD4511_BOX_T);
1290     ctx.strokeRect(CD4511_BOX_L, CD4511_BOX_T, CD4511_BOX_R - CD4511_BOX_L, CD4511_BOX_B - CD4511_BOX_T);
1291     ctx.beginPath();
1292     CD4511_PIN_Y.forEach((y) => {
1293         ctx.moveTo(CD4511_JUNC_L, y);
1294         ctx.lineTo(CD4511_BOX_L, y);
1295     });
1296     CD4511_PIN_Y.forEach((y) => {
1297         ctx.moveTo(CD4511_BOX_R, y);
1298         ctx.lineTo(CD4511_JUNC_R, y);
1299     });
1300     ctx.stroke();
1301     ctx.fillStyle = COLORS.ink; ctx.font = 'bold 9px Arial'; ctx.textBaseline = 'middle';
1302     ctx.textAlign = 'left';
1303     inLbl.forEach((t, i) => ctx.fillText(t, CD4511_LABEL_L, CD4511_PIN_Y[i]));
1304     ctx.textAlign = 'right';
1305     outLbl.forEach((t, i) => ctx.fillText(t, CD4511_LABEL_R, CD4511_PIN_Y[i]));
1306     ctx.font = '9px Arial'; ctx.textAlign = 'center';
1307     ctx.fillText('CD4511', 0, -4);
1308     ctx.font = '8px Arial';
1309     ctx.fillText('BCD→7', 0, 6);
1310     ctx.font = '11px Arial';
1311     ctx.fillText(comp.label, 0, CD4511_BOX_T - 12);
1312 }
1313 else if (comp.type === 'ic_74hc90') {
1314     const leftLbl = ['CP1', 'MR1', 'MR2', '', 'VCC', 'MS1', 'MS2'];
1315     const rightLbl = ['Q0', 'Q1', 'Q2', 'Q3', '', 'GND', 'CP0'];
1316     ctx.strokeStyle = '#26a69a'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1317     ctx.fillRect(IC90_BOX_L, IC90_BOX_T, IC90_BOX_R - IC90_BOX_L, IC90_BOX_B - IC90_BOX_T);
1318     ctx.strokeRect(IC90_BOX_L, IC90_BOX_T, IC90_BOX_R - IC90_BOX_L, IC90_BOX_B - IC90_BOX_T);
1319     ctx.beginPath();
1320     IC90_LEFT_PIN_Y.forEach((y, i) => {
1321         if (!leftLbl[i]) return;
1322         ctx.moveTo(IC90_JUNC_L, y);
1323         ctx.lineTo(IC90_BOX_L, y);
1324     });
1325     IC90_RIGHT_PIN_Y.forEach((y, i) => {
1326         if (!rightLbl[i]) return;
1327         ctx.moveTo(IC90_BOX_R, y);
1328         ctx.lineTo(IC90_JUNC_R, y);
1329     });
1330     ctx.stroke();
1331     ctx.fillStyle = COLORS.ink; ctx.font = '8px Arial'; ctx.textBaseline = 'middle';
1332     ctx.textAlign = 'left';
1333     leftLbl.forEach((t, i) => { if (t) ctx.fillText(t, IC90_LABEL_L, IC90_LEFT_PIN_Y[i]); });
1334     ctx.textAlign = 'right';
1335     rightLbl.forEach((t, i) => { if (t) ctx.fillText(t, IC90_LABEL_R, IC90_RIGHT_PIN_Y[i]); });
1336     ctx.font = 'bold 9px Arial'; ctx.textAlign = 'center';
1337     ctx.fillText('74HC90', 0, -4);
1338     ctx.font = '8px Arial';
1339     ctx.fillText('décade', 0, 6);
1340     const count = hc90SimCount(comp);
1341     if (count != null) {
1342         ctx.fillStyle = '#76ff03';
1343         ctx.font = 'bold 13px Arial';
1344         ctx.fillText(String(count), 0, 20);
1345         const qPins = IC90_Q_STACK_INDICES.map((qi) => ({
1346             qi,
1347             y: IC90_RIGHT_PIN_Y[qi],
1348         }));
1349         qPins.forEach(({ qi, y }) => {
1350             const on = (count >> qi) & 1;
1351             ctx.beginPath();
1352             ctx.fillStyle = on ? '#76ff03' : '#455a64';
1353             ctx.arc(IC90_JUNC_R - 8, y, 4, 0, Math.PI * 2);
1354             ctx.fill();

```

```

1355     });
1356   }
1357   ctx.fillStyle = COLORS.ink;
1358   ctx.font = '11px Arial';
1359   ctx.fillText(comp.label, 0, IC90_BOX_T - 12);
1360 }
1361 else if (comp.type === 'arduino_uno') {
1362   ctx.fillStyle = '#00979d';
1363   ctx.strokeStyle = '#006064';
1364   ctx.lineWidth = 2;
1365   ctx.fillRect(UNO_BOX_L, UNO_BOX_T, UNO_BOX_R - UNO_BOX_L, UNO_BOX_B - UNO_BOX_T);
1366   ctx.strokeRect(UNO_BOX_L, UNO_BOX_T, UNO_BOX_R - UNO_BOX_L, UNO_BOX_B - UNO_BOX_T);
1367   ctx.beginPath();
1368   UNO_LEFT_PIN_Y.forEach((y) => {
1369     ctx.moveTo(UNO_JUNC_L, y);
1370     ctx.lineTo(UNO_BOX_L, y);
1371   });
1372   UNO_RIGHT_PIN_Y.forEach((y) => {
1373     ctx.moveTo(UNO_BOX_R, y);
1374     ctx.lineTo(UNO_JUNC_R, y);
1375   });
1376   ctx.strokeStyle = '#004d40';
1377   ctx.stroke();
1378   ctx.fillStyle = 'ffffff';
1379   ctx.font = '6px Arial';
1380   ctx.textBaseline = 'middle';
1381   ctx.textAlign = 'left';
1382   UNO_LEFT_PINS.forEach((t, i) => ctx.fillText(formatUnoPinLabel(t), UNO_LABEL_L, UNO_LEFT_PIN_Y[i]));
1383   ctx.textAlign = 'right';
1384   UNO_RIGHT_PINS.forEach((t, i) => ctx.fillText(formatUnoPinLabel(t), UNO_LABEL_R, UNO_RIGHT_PIN_Y[i]));
1385   ctx.font = 'bold 10px Arial';
1386   ctx.textAlign = 'center';
1387   ctx.fillStyle = 'ffffff';
1388   ctx.fillText('UNO', 0, 0);
1389   ctx.font = '8px Arial';
1390   ctx.fillText('ATmega328P', 0, 12);
1391   if (flags.isSimulating && comp.label) {
1392     UNO_DIGITAL_PINS.forEach((pinName) => {
1393       const lv = simulationResults.logicValues?.[`${comp.label}_${pinName}`]
1394         ?? simulationResults.logicValues?.[`${comp.label}/${pinName}`];
1395       if (!lv) return;
1396       const idx = UNO_RIGHT_PINS.indexOf(pinName);
1397       if (idx < 0) return;
1398       const y = UNO_RIGHT_PIN_Y[idx];
1399       ctx.beginPath();
1400       ctx.fillStyle = lv.logic === 1 ? '#76ff03' : '#455a64';
1401       ctx.arc(UNO_JUNC_R - 8, y, 3, 0, Math.PI * 2);
1402       ctx.fill();
1403     });
1404   }
1405   if (comp.lastCompileOk === true) {
1406     ctx.fillStyle = '#76ff03';
1407     ctx.font = '7px Arial';
1408     ctx.textAlign = 'left';
1409     ctx.fillText('✓ compile', UNO_BOX_L + 4, UNO_BOX_B - 6);
1410   } else if (comp.lastCompileOk === false) {
1411     ctx.fillStyle = '#ff5252';
1412     ctx.font = '7px Arial';
1413     ctx.textAlign = 'left';
1414     ctx.fillText('✗ compile', UNO_BOX_L + 4, UNO_BOX_B - 6);
1415   }
1416   ctx.fillStyle = COLORS.ink;
1417   ctx.font = '11px Arial';
1418   ctx.textAlign = 'center';
1419   ctx.fillText(comp.label, 0, UNO_BOX_T - 12);
1420 }
1421 else if (comp.type === 'seg7') {
1422   drawSeg7Display(comp);
1423 }
1424 else if (comp.type === 'bargraph_dc10h') {
1425   drawBargraphDc10h(comp);
1426 }
1427 else if (comp.type === 'grove_lcd16x2') {
1428   drawGroveLcd16x2(comp);
1429 }
1430 else if (comp.type === 'grove_dht22') {
1431   drawGroveDht22(comp);
1432 }
1433 else if (comp.type === 'led') {
1434   const ledCurrent = getLedCurrentAmps(comp);

```

```

1435     const isBurnt = ledCurrent !== null && isLedOvercurrent(ledCurrent);
1436     let isLit = false;
1437     if (flags.isSimulating && !isBurnt) {
1438         if (ledCurrent !== null) {
1439             isLit = ledCurrent > 1e-4;
1440         } else {
1441             let vAnode = getVoltageAtJunction(`${comp.label}_in`);
1442             let vCathode = getVoltageAtJunction(`${comp.label}_out`);
1443             if ((vAnode - vCathode) >= 1.5) isLit = true;
1444         }
1445     }
1446     const ledBodyColor = isBurnt ? '#3a3a3a' : (isLit ? '#ff1744' : '#4a030a');
1447     ctx.strokeStyle = isBurnt ? '#666666' : (isLit ? '#ff6b81' : '#ff1744');
1448     ctx.lineWidth = 2; ctx.fillStyle = ledBodyColor;
1449     if (isLit) {
1450         ctx.save(); ctx.shadowBlur = 20; ctx.shadowColor = '#ff1744';
1451         ctx.beginPath(); ctx.moveTo(-15, -15); ctx.lineTo(10, 0); ctx.lineTo(-15, 15); ctx.closePath(); ctx.fill(); ctx.restore();
1452     } else {
1453         ctx.beginPath(); ctx.moveTo(-15, -15); ctx.lineTo(10, 0); ctx.lineTo(-15, 15); ctx.closePath(); ctx.fill();
1454     }
1455     ctx.beginPath(); ctx.moveTo(10, -15); ctx.lineTo(10, 15); ctx.stroke();
1456     ctx.beginPath(); ctx.moveTo(-40, 0); ctx.lineTo(-15, 0); ctx.moveTo(10, 0); ctx.lineTo(40, 0); ctx.stroke();
1457     if (!isBurnt) {
1458         ctx.save();
1459         ctx.strokeStyle = isLit ? '#ff8a9a' : '#730510';
1460         ctx.beginPath(); ctx.moveTo(-5, -18); ctx.lineTo(5, -28); ctx.moveTo(5, -28); ctx.lineTo(0, -28); ctx.moveTo(5, -28); ctx.lineTo
(5, -23);
1461         ctx.moveTo(5, -14); ctx.lineTo(15, -24); ctx.moveTo(15, -24); ctx.lineTo(10, -24); ctx.moveTo(15, -24); ctx.lineTo(15, -19); ctx.
stroke();
1462         ctx.restore();
1463     }
1464     if (isBurnt) {
1465         drawLedSmoke(performance.now() / 350);
1466     }
1467     drawLabels(comp.label, isBurnt ? 'GRILLÉE' : null, rot);
1468 }
1469 else if (comp.type === 'gimp') {
1470     ctx.save();
1471     if (comp.flipX) ctx.scale(-1, 1);
1472     ctx.strokeStyle = '#ab47bc'; ctx.lineWidth = 2;
1473     ctx.beginPath();
1474     ctx.moveTo(8, 0); ctx.lineTo(40, 0);
1475     ctx.moveTo(0, 18); ctx.lineTo(0, 40);
1476     ctx.stroke();
1477     ctx.fillStyle = COLORS.componentFill; ctx.fillRect(-18, -18, 26, 36); ctx.strokeRect(-18, -18, 26, 36);
1478     ctx.fillStyle = '#ce93d8'; ctx.font = 'bold 11px Arial'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
1479     ctx.fillText('G', -5, 0);
1480     ctx.restore();
1481     drawLabels(comp.label, null, 0);
1482 }
1483 else if (comp.type === 'gsin') {
1484     ctx.strokeStyle = '#26c6da'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1485     ctx.fillRect(-18, -14, 36, 28); ctx.strokeRect(-18, -14, 36, 28);
1486     ctx.strokeStyle = '#4dd0e1'; ctx.lineWidth = 1.5;
1487     ctx.beginPath();
1488     ctx.moveTo(-14, 0);
1489     ctx.quadraticCurveTo(-7, -10, 0, 0);
1490     ctx.quadraticCurveTo(7, 10, 14, 0);
1491     ctx.stroke();
1492     ctx.lineWidth = 2; ctx.strokeStyle = '#26c6da';
1493     ctx.beginPath();
1494     ctx.moveTo(18, 0); ctx.lineTo(40, 0);
1495     ctx.moveTo(0, 14); ctx.lineTo(0, 40);
1496     ctx.stroke();
1497     drawWirePlusLabel(40, 0);
1498     drawLabels(comp.label, null, 0);
1499 }
1500 else if (comp.type === 'gsqr') {
1501     ctx.strokeStyle = '#29b6f6'; ctx.lineWidth = 2; ctx.fillStyle = COLORS.componentFill;
1502     ctx.fillRect(-18, -14, 36, 28); ctx.strokeRect(-18, -14, 36, 28);
1503     ctx.strokeStyle = '#4fc3f7'; ctx.lineWidth = 1.5;
1504     ctx.beginPath();
1505     ctx.moveTo(-14, 6); ctx.lineTo(-14, -6); ctx.lineTo(-2, -6); ctx.lineTo(-2, 6); ctx.lineTo(10, 6); ctx.lineTo(10, -6); ctx.lineTo(14,
-6);
1506     ctx.stroke();
1507     ctx.lineWidth = 2; ctx.strokeStyle = '#29b6f6';
1508     ctx.beginPath();
1509     ctx.moveTo(18, 0); ctx.lineTo(40, 0);
1510     ctx.moveTo(0, 14); ctx.lineTo(0, 40);
1511     ctx.stroke();

```

```

1512     drawWirePlusLabel(40, 0);
1513     drawLabels(comp.label, null, 0);
1514 }
1515 else if (comp.type === 'oscilloscope') {
1516     ctx.strokeStyle = '#00bcd4'; ctx.lineWidth = 2;
1517     ctx.beginPath();
1518     ctx.moveTo(-60, -20); ctx.lineTo(-42, -20);
1519     ctx.moveTo(-60, 20); ctx.lineTo(-42, 20);
1520     ctx.moveTo(0, 24); ctx.lineTo(0, 60);
1521     ctx.stroke();
1522     ctx.fillStyle = '#8899aa'; ctx.font = '9px Arial'; ctx.textAlign = 'center'; ctx.textBaseline = 'bottom';
1523     ctx.fillText('CH1', -51, -22);
1524     ctx.fillText('CH2', -51, 18);
1525     drawOscilloscopeScreen(comp);
1526 }
1527 else if (comp.type === 'voltmeter') {
1528     let displayValue = flags.isSimulating ? '-' : '0.0';
1529     let rawV = null;
1530     if (flags.isSimulating) {
1531         const animV = getAnimatedVoltmeterVoltage(comp.label);
1532         if (animV !== null && Number.isFinite(animV)) {
1533             rawV = animV;
1534         } else if (simulationResults.voltmeters && simulationResults.voltmeters[comp.label] !== undefined) {
1535             let measureData = simulationResults.voltmeters[comp.label];
1536             if (measureData && typeof measureData === 'object' && measureData.voltage !== undefined) {
1537                 rawV = measureData.voltage;
1538             } else if (typeof measureData === 'number') { rawV = measureData; }
1539         }
1540     }
1541     if (rawV !== null && Number.isFinite(rawV)) {
1542         displayValue = formatMeterValue(quantizeVoltmeterReading(rawV));
1543     }
1544     drawMeterBody('#00bcd4');
1545     drawMeterDisplay(displayValue, 'V', rot);
1546     drawLabels(comp.label, null, rot);
1547 }
1548 else if (comp.type === 'bode_analyzer') {
1549     drawMeterBody('#7cff6b');
1550     drawMeterDisplay('Bode', '', rot);
1551     drawLabels(comp.label, null, rot);
1552 }
1553 else if (comp.type === 'speaker') {
1554     const playing = flags.isSimulating && isSpeakerAudioPlaying();
1555     ctx.strokeStyle = playing ? '#ffeb3b' : '#ab47bc';
1556     ctx.lineWidth = 2;
1557     ctx.fillStyle = '#2a3b4c';
1558     ctx.beginPath();
1559     ctx.arc(0, 0, 20, 0, Math.PI * 2);
1560     ctx.fill();
1561     ctx.stroke();
1562     ctx.strokeStyle = playing ? '#ffeb3b' : '#ab47bc';
1563     ctx.beginPath();
1564     ctx.moveTo(-40, 0);
1565     ctx.lineTo(-20, 0);
1566     ctx.moveTo(20, 0);
1567     ctx.lineTo(40, 0);
1568     ctx.stroke();
1569     if (playing) {
1570         ctx.strokeStyle = '#ffeb3b';
1571         ctx.lineWidth = 1.5;
1572         for (let i = 0; i < 3; i++) {
1573             const r = 10 + i * 5;
1574             ctx.beginPath();
1575             ctx.arc(22, 0, r, -Math.PI / 4, Math.PI / 4);
1576             ctx.stroke();
1577         }
1578     }
1579     drawWirePlusLabel(40, 0);
1580     const z = comp.value ? `${comp.value}Ω` : '8Ω';
1581     drawMeterDisplay(playing ? '}' : 'HP', playing ? '' : z, rot);
1582     drawLabels(comp.label, null, rot);
1583 }
1584 else if (comp.type === 'ammeter') {
1585     let displayValue = '0.0';
1586     let unit = 'A';
1587     if (flags.isSimulating && simulationResults.ammeters && simulationResults.ammeters[comp.label] !== undefined) {
1588         const measureData = simulationResults.ammeters[comp.label];
1589         const i = (measureData && typeof measureData === 'object') ? measureData.current : measureData;
1590         displayValue = formatCurrentDisplay(i);
1591         unit = formatCurrentUnit(i);

```



```

1592     }
1593     drawMeterBody('#ff9800');
1594     drawMeterDisplay(displayValue, unit, rot);
1595     drawLabels(comp.label, null, rot);
1596   }
1597   else if (comp.type === 'ohmmeter') {
1598     let displayValue = '0';
1599     let unit = 'Ω';
1600     if (flags.isSimulating && simulationResults.ohmmeters && simulationResults.ohmmeters[comp.label] !== undefined) {
1601       const measureData = simulationResults.ohmmeters[comp.label];
1602       const r = (measureData && typeof measureData === 'object') ? measureData.resistance : measureData;
1603       displayValue = formatResistanceDisplay(r);
1604       unit = formatResistanceUnit(r);
1605     }
1606     drawMeterBody('#ce93d8', false);
1607     drawMeterDisplay(displayValue, unit, rot);
1608     drawLabels(comp.label, null, rot);
1609   }
1610   ctx.restore();
1611
1612   getComponentJunctions(comp).forEach(j => {
1613     if (!isJunctionConnected(j.id)) {
1614       ctx.save(); const isHovered = interaction.hoverJunction && interaction.hoverJunction.id === j.id;
1615       ctx.fillStyle = isHovered ? '#ff5722' : '#ff3333';
1616       ctx.beginPath(); ctx.arc(j.x, j.y, isHovered ? 6 : 4, 0, Math.PI * 2); ctx.fill(); ctx.restore();
1617     }
1618   });
1619 }
1620
1621 function drawWires() {
1622   ctx.save();
1623   circuit.wires.forEach(w => {
1624     if (w === interaction.selectedWire) { ctx.strokeStyle = '#ff9800'; ctx.lineWidth = 4.0; }
1625     else if (w === interaction.hoveredWire) { ctx.strokeStyle = '#ffeb3b'; ctx.lineWidth = 3.5; }
1626     else { ctx.strokeStyle = COLORS.wireDefault; ctx.lineWidth = 2.5; }
1627     ctx.beginPath(); ctx.moveTo(w.points[0].x, w.points[0].y);
1628     for (let i = 1; i < w.points.length; i++) ctx.lineTo(w.points[i].x, w.points[i].y);
1629     ctx.stroke();
1630   });
1631   if (interaction.activeWire) {
1632     ctx.strokeStyle = '#ffeb3b'; ctx.lineWidth = 2.5; ctx.setLineDash([4, 4]);
1633     ctx.beginPath(); ctx.moveTo(interaction.activeWire.points[0].x, interaction.activeWire.points[0].y);
1634     for (let i = 1; i < interaction.activeWire.points.length; i++) ctx.lineTo(interaction.activeWire.points[i].x, interaction.activeWire.
points[i].y);
1635     ctx.stroke();
1636   }
1637   ctx.restore();
1638
1639   circuit.autoJunctions.forEach(aj => {
1640     ctx.save(); const isHovered = interaction.hoverJunction && interaction.hoverJunction.id === aj.id;
1641     const isSelected = interaction.selectedAutoJunctions.includes(aj);
1642     if (isSelected) { ctx.fillStyle = '#00bcd4'; ctx.strokeStyle = COLORS.junctionStroke; ctx.lineWidth = 2; }
1643     else { ctx.fillStyle = COLORS.wireDefault; ctx.strokeStyle = COLORS.junctionStroke; ctx.lineWidth = isHovered ? 2 : 1; }
1644     ctx.beginPath(); ctx.arc(aj.x, aj.y, (isHovered || isSelected) ? 6 : 4.5, 0, Math.PI * 2); ctx.fill(); ctx.restore();
1645   });
1646 }
1647
1648 function drawGrid() {
1649   ctx.strokeStyle = COLORS.grid; ctx.lineWidth = 1 / scale.value;
1650   const startLeft = Math.floor(-pan.x / scale.value / GRID_SIZE) * GRID_SIZE;
1651   const startTop = Math.floor(-pan.y / scale.value / GRID_SIZE) * GRID_SIZE;
1652   const endRight = startLeft + canvas.width / scale.value + GRID_SIZE;
1653   const endBottom = startTop + canvas.height / scale.value + GRID_SIZE;
1654   ctx.beginPath();
1655   for (let x = startLeft; x < endRight; x += GRID_SIZE) { ctx.moveTo(x, startTop); ctx.lineTo(x, endBottom); }
1656   for (let y = startTop; y < endBottom; y += GRID_SIZE) { ctx.moveTo(startLeft, y); ctx.lineTo(endRight, y); }
1657   ctx.stroke();
1658 }
1659
1660 export function draw() {
1661   ctx.fillStyle = COLORS.canvasBg;
1662   ctx.fillRect(0, 0, canvas.width, canvas.height);
1663   ctx.save();
1664   ctx.translate(pan.x, pan.y);
1665   ctx.scale(scale.value, scale.value);
1666   drawGrid();
1667   drawWires();
1668   circuit.components.forEach(comp => drawComponentBody(comp));
1669   if (flags.isSelectingZone) {

```

```
1670     ctx.save(); ctx.strokeStyle = 'rgba(0, 188, 212, 0.7)'; ctx.fillStyle = 'rgba(0, 188, 212, 0.15)'; ctx.lineWidth = 1.5; ctx.setLineDash([4, 4]);
1671     ctx.fillRect(zone.start.x, zone.start.y, zone.end.x - zone.start.x, zone.end.y - zone.start.y);
1672     ctx.strokeRect(zone.start.x, zone.start.y, zone.end.x - zone.start.x, zone.end.y - zone.start.y); ctx.restore();
1673 }
1674 if (flags.isDraggingFromMenu && menuDrag.draggedComponentType) {
1675     ctx.globalAlpha = 0.5;
1676     const preview = {
1677         type: menuDrag.draggedComponentType,
1678         x: snapToGrid(menuDrag.x),
1679         y: snapToGrid(menuDrag.y),
1680         label: '',
1681         rotation: 0,
1682         state: 0,
1683         flipX: false,
1684     };
1685     if (preview.type === 'grove_lcd16x2') preview.i2cAddress = 0x3e;
1686     drawComponentBody(preview);
1687     ctx.globalAlpha = 1.0;
1688 }
1689 ctx.restore();
1690 }
```

```

1 // scope-animation.js – affichage des courbes CH1/CH2 sur l'oscilloscope
2 import { circuit } from './state.js';
3 import { getSimulationElapsedSec } from './led-animation.js';
4 import {
5     findArduinoDriveForScopeChannel,
6     synthesizeArduinoScopeTrace,
7 } from './Engine/scope-arduino-ideal.mjs';
8
9 export const SCOPE_H_DIVS = 8;
10 export const SCOPE_V_DIVS = 8;
11
12 let scopePlots = {};
13 let scopeAnimStartMs = 0;
14 let rafId = null;
15 let redraw = () => {};
16 let popupRedraw = () => {};
17
18 export function bindScopeAnimationRedraw(fn) {
19     redraw = typeof fn === 'function' ? fn : () => {};
20 }
21
22 export function bindScopePopupRedraw(fn) {
23     popupRedraw = typeof fn === 'function' ? fn : () => {};
24 }
25
26 function hasArduinoScopeChannels() {
27     for (const comp of circuit.components) {
28         if (comp.type !== 'oscilloscope') continue;
29         if (
30             findArduinoDriveForScopeChannel(
31                 comp.label,
32                 'CH1',
33                 circuit.components,
34                 circuit.wires,
35                 circuit.autoJunctions
36             ) ||
37             findArduinoDriveForScopeChannel(
38                 comp.label,
39                 'CH2',
40                 circuit.components,
41                 circuit.wires,
42                 circuit.autoJunctions
43             )
44         ) {
45             return true;
46         }
47     }
48     return false;
49 }
50
51 function getScopeElapsedSec() {
52     const ledElapsed = getSimulationElapsedSec();
53     if (ledElapsed > 0) return ledElapsed;
54     if (scopeAnimStartMs > 0) return (performance.now() - scopeAnimStartMs) / 1000;
55     return 0;
56 }
57
58 export function startScopeAnimation(plots) {
59     stopScopeAnimation();
60     scopePlots = plots && typeof plots === 'object' ? plots : {};
61     if (!Object.keys(scopePlots).length && !hasArduinoScopeChannels()) return;
62     scopeAnimStartMs = performance.now();
63     const tick = () => {
64         popupRedraw();
65         rafId = requestAnimationFrame(tick);
66     };
67     rafId = requestAnimationFrame(tick);
68 }
69
70 export function stopScopeAnimation() {
71     if (rafId !== null) cancelAnimationFrame(rafId);
72     rafId = null;
73     scopePlots = {};
74     scopeAnimStartMs = 0;
75 }
76
77 export function hasScopeAnimation() {
78     return rafId !== null && (Object.keys(scopePlots).length > 0 || hasArduinoScopeChannels());

```

```

79 }
80
81 function detectChannelPeriodSec(channel, threshold = 2.5) {
82     const time = channel?.time;
83     const voltage = channel?.voltage;
84     if (!time?.length || !voltage?.length) return 0;
85     const rising = [];
86     for (let i = 1; i < voltage.length; i++) {
87         if (voltage[i] > threshold && voltage[i - 1] <= threshold) rising.push(time[i]);
88     }
89     if (rising.length >= 2) return rising[1] - rising[0];
90     let toggles = 0;
91     for (let i = 1; i < voltage.length; i++) {
92         if ((voltage[i] > threshold) !== (voltage[i - 1] > threshold)) toggles++;
93     }
94     const span = time[time.length - 1] - time[0];
95     if (toggles >= 2 && span > 0) return (2 * span) / toggles;
96     return 0;
97 }
98
99 function interpolateChannelVoltage(channel, tSec) {
100     const time = channel?.time;
101     const voltage = channel?.voltage;
102     if (!time?.length || !voltage?.length) return 0;
103     if (tSec <= time[0]) return voltage[0];
104     const tLast = time[time.length - 1];
105     if (tSec >= tLast) return voltage[voltage.length - 1];
106     let lo = 0;
107     let hi = time.length - 1;
108     while (hi - lo > 1) {
109         const mid = (lo + hi) >> 1;
110         if (time[mid] <= tSec) lo = mid;
111         else hi = mid;
112     }
113     const t0 = time[lo];
114     const t1 = time[hi];
115     const v0 = voltage[lo];
116     const v1 = voltage[hi];
117     if (t1 <= t0) return v0;
118     return v0 + ((tSec - t0) / (t1 - t0)) * (v1 - v0);
119 }
120
121 /** Échantillons SPICE sur la dernière fenêtre, ou phase verrouillée si périodique. */
122 function sampleChannelWindow(channel, windowSec, options = {}) {
123     if (!channel?.time?.length || !channel?.voltage?.length) return [];
124     const timeOffsetSec = Number.isFinite(options.timeOffsetSec) ? options.timeOffsetSec : 0;
125     const period = options.periodSec ?? detectChannelPeriodSec(channel);
126     const synced = options.synced !== false && period > 0;
127
128     if (synced) {
129         const n = Math.max(64, Math.min(800, Math.ceil(windowSec * 4000)));
130         const out = [];
131         for (let i = 0; i < n; i++) {
132             const tScreen = (i / (n - 1)) * windowSec;
133             const tPhase = ((timeOffsetSec + tScreen) % period + period) % period;
134             const tSample = channel.time[0] + tPhase;
135             const v = interpolateChannelVoltage(channel, tSample);
136             if (Number.isFinite(v)) out.push({ t: tScreen, v });
137         }
138         return out;
139     }
140
141     const tEnd = channel.time[channel.time.length - 1] + timeOffsetSec;
142     const tStart = Math.max(channel.time[0], tEnd - windowSec);
143     const out = [];
144     for (let i = 0; i < channel.time.length; i++) {
145         const t = channel.time[i];
146         if (t < tStart) continue;
147         const v = channel.voltage[i];
148         if (Number.isFinite(t) && Number.isFinite(v)) out.push({ t: t - tStart, v });
149     }
150     return out;
151 }
152
153 /**
154  * Fenêtre temporelle + échantillons pour les deux voies (superposées à l'affichage).
155  * @param {{ label: string; timeDivSec?: number; ch1VoltsPerDiv?: number; ch2VoltsPerDiv?: number }} comp
156  */
157 export function getScopeTraceWindow(comp) {
158     const timeDiv = comp.timeDivSec > 0 ? comp.timeDivSec : 1e-3;

```

```

159     const windowSec = timeDiv * SCOPE_H_DIVS;
160     const timeOffsetSec = (Number.isFinite(comp.timePositionDiv) ? comp.timePositionDiv : 0) * timeDiv;
161     const elapsed = getScopeElapsedSec();
162     const plot = scopePlots[comp.label];
163     const sampleOpts = { timeOffsetSec };
164
165     const ch1Drive = findArduinoDriveForScopeChannel(
166         comp.label,
167         'CH1',
168         circuit.components,
169         circuit.wires,
170         circuit.autoJunctions
171     );
172     const ch2Drive = findArduinoDriveForScopeChannel(
173         comp.label,
174         'CH2',
175         circuit.components,
176         circuit.wires,
177         circuit.autoJunctions
178     );
179
180     const ch1 = ch1Drive
181         ? synthesizeArduinoScopeTrace(ch1Drive.uno, ch1Drive.pinLabel, windowSec, elapsed, {}, timeOffsetSec)
182         : plot
183         ? sampleChannelWindow(plot.ch1, windowSec, sampleOpts)
184         : [];
185     const ch2 = ch2Drive
186         ? synthesizeArduinoScopeTrace(ch2Drive.uno, ch2Drive.pinLabel, windowSec, elapsed, {}, timeOffsetSec)
187         : plot
188         ? sampleChannelWindow(plot.ch2, windowSec, sampleOpts)
189         : [];
190
191     if (!ch1.length && !ch2.length) return null;
192
193     return {
194         ch1,
195         ch2,
196         windowSec,
197         timeDiv,
198         timeOffsetSec,
199         ch1Vdiv: comp.ch1VoltsPerDiv > 0 ? comp.ch1VoltsPerDiv : 1,
200         ch2Vdiv: comp.ch2VoltsPerDiv > 0 ? comp.ch2VoltsPerDiv : 1,
201         ch1PosDiv: Number.isFinite(comp.ch1PositionDiv) ? comp.ch1PositionDiv : 0,
202         ch2PosDiv: Number.isFinite(comp.ch2PositionDiv) ? comp.ch2PositionDiv : 0,
203         ch1Digital: !!ch1Drive || isLikelyDigitalTrace(ch1),
204         ch2Digital: !!ch2Drive || isLikelyDigitalTrace(ch2),
205     };
206 }
207
208 /** Signal logique 0/5 V (I²C, GPIO) – affichage en créneaux, pas en diagonales. */
209 export function isLikelyDigitalTrace(points) {
210     if (!points || points.length < 8) return false;
211     let min = Infinity;
212     let max = -Infinity;
213     for (const pt of points) {
214         if (!Number.isFinite(pt.v)) continue;
215         if (pt.v < min) min = pt.v;
216         if (pt.v > max) max = pt.v;
217     }
218     const span = max - min;
219     if (span < 1.5) return false;
220     const margin = span * 0.12;
221     let binary = 0;
222     let n = 0;
223     for (const pt of points) {
224         if (!Number.isFinite(pt.v)) continue;
225         n++;
226         if (pt.v <= min + margin || pt.v >= max - margin) binary++;
227     }
228     return n >= 8 && binary / n > 0.88;
229 }
230
231 /**
232  * Trace CH1/CH2 sur un canvas (mode créneau pour signaux numériques).
233  */
234 export function drawScopeTrace(ctx, points, windowSec, vdiv, posDiv, x0, y0, w, h, color, options = {}) {
235     if (!points?.length || windowSec <= 0 || vdiv <= 0) return;
236     const pxPerDivY = h / SCOPE_V_DIVS;
237     const midY = y0 + h / 2;
238     const pos = Number.isFinite(posDiv) ? posDiv : 0;

```

```
239     const digital = options.digital ?? isLikelyDigitalTrace(points);
240     const toY = (v) => midY - ((v / vdiv) + pos) * pxPerDivY;
241     const toX = (t) => x0 + (t / windowSec) * w;
242
243     ctx.save();
244     ctx.beginPath();
245     ctx.rect(x0, y0, w, h);
246     ctx.clip();
247     ctx.strokeStyle = color;
248     ctx.lineWidth = options.lineWidth ?? 2;
249     ctx.lineJoin = 'miter';
250     ctx.beginPath();
251
252     let prevX = toX(points[0].t);
253     let prevY = toY(points[0].v);
254     ctx.moveTo(prevX, prevY);
255
256     for (let i = 1; i < points.length; i++) {
257         const x = toX(points[i].t);
258         const y = toY(points[i].v);
259         if (digital) {
260             ctx.lineTo(x, prevY);
261             ctx.lineTo(x, y);
262         } else {
263             ctx.lineTo(x, y);
264         }
265         prevX = x;
266         prevY = y;
267     }
268     ctx.stroke();
269     ctx.restore();
270 }
271
```

```

1 // scope-panel.js - réglages oscilloscope (panneau bas)
2 import { flags, saveState } from './state.js';
3 import { draw, resizeCanvas } from './renderer.js';
4 import { clearSourceSelection } from './source-panel.js';
5 import { openScopePopup, refreshScopePopup, closeScopePopup, isScopePopupOpen } from './scope-popup.js';
6
7 let activeScope = null;
8
9 const panel = () => document.getElementById('source-panel');
10 const titleEl = () => document.getElementById('source-panel-title');
11 const scopeFields = () => document.getElementById('scope-fields');
12 const timeDivEl = () => document.getElementById('scope-time-div');
13 const timePosEl = () => document.getElementById('scope-time-pos');
14 const ch1VdivEl = () => document.getElementById('scope-ch1-vdiv');
15 const ch2VdivEl = () => document.getElementById('scope-ch2-vdiv');
16 const ch1PosEl = () => document.getElementById('scope-ch1-pos');
17 const ch2PosEl = () => document.getElementById('scope-ch2-pos');
18 const timePosVal = () => document.getElementById('scope-time-pos-val');
19 const ch1PosVal = () => document.getElementById('scope-ch1-pos-val');
20 const ch2PosVal = () => document.getElementById('scope-ch2-pos-val');
21 const openWindowBtn = () => document.getElementById('scope-open-window');
22
23 function formatPosDiv(v) {
24     const n = Number.isFinite(v) ? v : 0;
25     const s = n.toFixed(1).replace('.', ',');
26     return `${n >= 0 ? '+' : ''}${s} div`;
27 }
28
29 export function isScopePanelOpen() {
30     return activeScope != null && panel() && !panel().classList.contains('hidden');
31 }
32
33 export function getScopePanelHeight() {
34     return isScopePanelOpen() && panel() ? panel().offsetHeight : 0;
35 }
36
37 export function getActiveScope() {
38     return activeScope;
39 }
40
41 export function refreshScopePanelFields() {
42     if (activeScope) syncFieldsFromComponent(activeScope);
43 }
44
45 function syncFieldsFromComponent(comp) {
46     if (!comp) return;
47     if (titleEl()) titleEl().textContent = `${comp.label} - Oscilloscope`;
48     if (timeDivEl()) timeDivEl().value = String(comp.timeDivSec ?? 0.001);
49     if (ch1VdivEl()) ch1VdivEl().value = String(comp.ch1VoltsPerDiv ?? 1);
50     if (ch2VdivEl()) ch2VdivEl().value = String(comp.ch2VoltsPerDiv ?? 1);
51     const tp = comp.timePositionDiv ?? 0;
52     const p1 = comp.ch1PositionDiv ?? 0;
53     const p2 = comp.ch2PositionDiv ?? 0;
54     if (timePosEl()) timePosEl().value = String(tp);
55     if (ch1PosEl()) ch1PosEl().value = String(p1);
56     if (ch2PosEl()) ch2PosEl().value = String(p2);
57     if (timePosVal()) timePosVal().textContent = formatPosDiv(tp);
58     if (ch1PosVal()) ch1PosVal().textContent = formatPosDiv(p1);
59     if (ch2PosVal()) ch2PosVal().textContent = formatPosDiv(p2);
60     updateOpenWindowButton();
61 }
62
63 function updateOpenWindowButton() {
64     const btn = openWindowBtn();
65     if (!btn) return;
66     btn.textContent = isScopePopupOpen() ? '🔍 Fenêtre scope ouverte' : '🔍 Afficher fenêtre scope';
67 }
68
69 function applyFieldsToComponent() {
70     if (!activeScope) return;
71     activeScope.timeDivSec = parseFloat(timeDivEl()?.value) || 0.001;
72     activeScope.ch1VoltsPerDiv = parseFloat(ch1VdivEl()?.value) || 1;
73     activeScope.ch2VoltsPerDiv = parseFloat(ch2VdivEl()?.value) || 1;
74     activeScope.timePositionDiv = parseFloat(timePosEl()?.value) || 0;
75     activeScope.ch1PositionDiv = parseFloat(ch1PosEl()?.value) || 0;
76     activeScope.ch2PositionDiv = parseFloat(ch2PosEl()?.value) || 0;
77     if (timePosVal()) timePosVal().textContent = formatPosDiv(activeScope.timePositionDiv);
78     if (ch1PosVal()) ch1PosVal().textContent = formatPosDiv(activeScope.ch1PositionDiv);

```

```

79     if (ch2PosVal()) ch2PosVal().textContent = formatPosDiv(activeScope.ch2PositionDiv);
80     refreshScopePopup();
81     draw();
82 }
83
84 /** Ouvre le panneau bas (sans forcer la popup). */
85 export function openScopePanel(comp, { openPopup = false } = {}) {
86     if (!comp || comp.type !== 'oscilloscope') return;
87     clearSourceSelection();
88     activeScope = comp;
89     syncFieldsFromComponent(comp);
90     const sf = scopeFields();
91     if (sf) {
92         sf.classList.remove('hidden');
93         sf.style.display = 'flex';
94     }
95     panel()?.classList.remove('hidden');
96     if (openPopup) openScopePopup(comp);
97     resizeCanvas();
98 }
99
100 export function closeScopePanelFully() {
101     activeScope = null;
102     const sf = scopeFields();
103     if (sf) { sf.classList.add('hidden'); sf.style.display = 'none'; }
104     panel()?.classList.add('hidden');
105     closeScopePopup(true);
106     resizeCanvas();
107 }
108
109 export function onScopePopupClosed() {
110     updateOpenWindowButton();
111 }
112
113 export function onScopeRemoved(comp) {
114     if (activeScope === comp) closeScopePanelFully();
115 }
116
117 export function initScopePanel() {
118     const apply = () => {
119         if (!activeScope) return;
120         applyFieldsToComponent();
121     };
122     const onChange = () => {
123         if (!activeScope) return;
124         if (!flags.isSimulating) saveState();
125         applyFieldsToComponent();
126     };
127
128     timeDivEl()?.addEventListener('change', onChange);
129     timePosEl()?.addEventListener('change', onChange);
130     ch1VdivEl()?.addEventListener('change', onChange);
131     ch2VdivEl()?.addEventListener('change', onChange);
132     ch1PosEl()?.addEventListener('change', onChange);
133     ch2PosEl()?.addEventListener('change', onChange);
134     timeDivEl()?.addEventListener('input', apply);
135     timePosEl()?.addEventListener('input', apply);
136     ch1VdivEl()?.addEventListener('input', apply);
137     ch2VdivEl()?.addEventListener('input', apply);
138     ch1PosEl()?.addEventListener('input', apply);
139     ch2PosEl()?.addEventListener('input', apply);
140
141     openWindowBtn()?.addEventListener('click', () => {
142         if (!activeScope) return;
143         if (isScopePopupOpen()) {
144             closeScopePopup();
145         } else {
146             openScopePopup(activeScope);
147         }
148         updateOpenWindowButton();
149     });
150 }
151

```



```

1 // scope-popup.js - fenêtre oscilloscope flottante (CH1 + CH2 superposées)
2 import { flags } from './state.js';
3 import { getScopeTraceWindow, drawScopeTrace, SCOPE_H_DIVS, SCOPE_V_DIVS } from './scope-animation.js';
4 import { draw } from './renderer.js';
5
6 let popupComp = null;
7 let onCloseCallback = () => {};
8 let onViewChanged = () => {};
9 let popupCanvas = null;
10 let popupCtx = null;
11
12 let floatX = 120;
13 let floatY = 70;
14 let dragging = false;
15 let dragOffX = 0;
16 let dragOffY = 0;
17 let panning = false;
18 let panStartX = 0;
19 let panStartPosDiv = 0;
20
21 const TIME_DIV_STEPS = [0.00001, 0.00005, 0.0001, 0.0002, 0.0005, 0.001, 0.002, 0.005, 0.01, 0.05, 0.1];
22
23 const popupEl = () => document.getElementById('scope-popup');
24 const titleEl = () => document.getElementById('scope-popup-title');
25
26 export function setScopePopupCloseCallback(fn) {
27   onCloseCallback = typeof fn === 'function' ? fn : () => {};
28 }
29
30 export function setScopeViewChangeCallback(fn) {
31   onViewChanged = typeof fn === 'function' ? fn : () => {};
32 }
33
34 function applyFloatPosition() {
35   const el = popupEl();
36   if (!el) return;
37   el.style.left = `${floatX}px`;
38   el.style.top = `${floatY}px`;
39 }
40
41 export function isScopePopupOpen() {
42   const el = popupEl();
43   return popupComp != null && el && !el.classList.contains('hidden');
44 }
45
46 export function renderScopePopup() {
47   if (!popupComp || !popupCtx || !popupCanvas) return;
48   const w = popupCanvas.width;
49   const h = popupCanvas.height;
50   const margin = { l: 48, r: 16, t: 24, b: 32 };
51   const plotW = w - margin.l - margin.r;
52   const plotH = h - margin.t - margin.b;
53   const x0 = margin.l;
54   const y0 = margin.t;
55
56   popupCtx.fillStyle = '#050a0e';
57   popupCtx.fillRect(0, 0, w, h);
58
59   popupCtx.strokeStyle = '#1a3a4a';
60   popupCtx.lineWidth = 1;
61   const divW = plotW / SCOPE_H_DIVS;
62   const divH = plotH / SCOPE_V_DIVS;
63   for (let i = 0; i <= SCOPE_H_DIVS; i++) {
64     popupCtx.beginPath();
65     popupCtx.moveTo(x0 + i * divW, y0);
66     popupCtx.lineTo(x0 + i * divW, y0 + plotH);
67     popupCtx.stroke();
68   }
69   for (let j = 0; j <= SCOPE_V_DIVS; j++) {
70     popupCtx.beginPath();
71     popupCtx.moveTo(x0, y0 + j * divH);
72     popupCtx.lineTo(x0 + plotW, y0 + j * divH);
73     popupCtx.stroke();
74   }
75   popupCtx.strokeStyle = '#2a5a6a';
76   popupCtx.lineWidth = 1.5;
77   popupCtx.beginPath();
78   popupCtx.moveTo(x0, y0 + plotH / 2);

```

```

79     popupCtx.lineTo(x0 + plotW, y0 + plotH / 2);
80     popupCtx.moveTo(x0 + plotW / 2, y0);
81     popupCtx.lineTo(x0 + plotW / 2, y0 + plotH);
82     popupCtx.stroke();
83
84     if (flags.isSimulating) {
85         const win = getScopeTraceWindow(popupComp);
86         if (win && (win.ch1.length || win.ch2.length)) {
87             drawScopeTrace(popupCtx, win.ch1, win.windowSec, win.ch1Vdiv, win.ch1PosDiv, x0, y0, plotW, plotH, '#ffeb3b', { digital: win.ch1D
igital });
88             drawScopeTrace(popupCtx, win.ch2, win.windowSec, win.ch2Vdiv, win.ch2PosDiv, x0, y0, plotW, plotH, '#00e5ff', { digital: win.ch2D
igital });
89             popupCtx.fillStyle = '#8899aa';
90             popupCtx.font = '11px Arial';
91             popupCtx.textAlign = 'right';
92             const td = win.timeDiv >= 0.001
93                 ? `${(win.timeDiv * 1000).toFixed(win.timeDiv >= 0.01 ? 1 : 2)} ms/div`
94                 : `${(win.timeDiv * 1e6).toFixed(0)} µs/div`;
95             const tpos = Number.isFinite(popupComp?.timePositionDiv) ? popupComp.timePositionDiv : 0;
96             const tposStr = `${tpos >= 0 ? '+' : ''}${tpos.toFixed(1)} div`;
97             popupCtx.fillText(
98                 `${td} | pos ${tposStr} | CH1: ${win.ch1Vdiv} V/div | CH2: ${win.ch2Vdiv} V/div`,
99                 w - margin.r,
100                 h - 10
101             );
102         } else {
103             popupCtx.fillStyle = '#556677';
104             popupCtx.font = '14px Arial';
105             popupCtx.textAlign = 'center';
106             popupCtx.textBaseline = 'middle';
107             popupCtx.fillText('Aucun signal – vérifiez les connexions CH1/CH2', w / 2, h / 2);
108         }
109     } else {
110         popupCtx.fillStyle = '#556677';
111         popupCtx.font = '14px Arial';
112         popupCtx.textAlign = 'center';
113         popupCtx.textBaseline = 'middle';
114         popupCtx.fillText('Lancez la simulation pour voir les signaux', w / 2, h / 2);
115     }
116
117     popupCtx.fillStyle = '#ffeb3b';
118     popupCtx.font = '12px Arial';
119     popupCtx.textAlign = 'left';
120     popupCtx.fillText('CH1', margin.l, 16);
121     popupCtx.fillStyle = '#00e5ff';
122     popupCtx.fillText('CH2', margin.l + 36, 16);
123 }
124
125 export function refreshScopePopup() {
126     if (isScopePopupOpen()) renderScopePopup();
127 }
128
129 export function openScopePopup(comp) {
130     if (!comp || comp.type !== 'oscilloscope') return;
131     popupComp = comp;
132     popupCanvas = document.getElementById('scope-popup-canvas');
133     popupCtx = popupCanvas?.getContext('2d') ?? null;
134     if (titleEl()) titleEl().textContent = `${comp.label} – Oscilloscope`;
135     applyFloatPosition();
136     popupEl()?.classList.remove('hidden');
137     renderScopePopup();
138 }
139
140 export function closeScopePopup(clearComp = false) {
141     popupEl()?.classList.add('hidden');
142     if (clearComp) popupComp = null;
143     onCloseCallback();
144 }
145
146 function clampFloatPosition() {
147     const el = popupEl();
148     if (!el) return;
149     const w = el.offsetWidth || 680;
150     const h = el.offsetHeight || 420;
151     floatX = Math.max(4, Math.min(window.innerWidth - w - 4, floatX));
152     floatY = Math.max(48, Math.min(window.innerHeight - h - 4, floatY));
153 }
154
155 function stepTimeDiv(cur, zoomIn) {
156     const idx = TIME_DIV_STEPS.findIndex((v) => Math.abs(v - cur) < cur * 0.01);

```

```

157     const base = idx >= 0 ? idx : TIME_DIV_STEPS.findIndex((v) => v >= cur);
158     const i = base < 0 ? 0 : base;
159     if (zoomIn) return TIME_DIV_STEPS[Math.max(0, i - 1)];
160     return TIME_DIV_STEPS[Math.min(TIME_DIV_STEPS.length - 1, i + 1)];
161 }
162
163 export function initScopePopup() {
164     document.getElementById('scope-popup-close')?.addEventListener('click', (e) => {
165         e.stopPropagation();
166         closeScopePopup();
167     });
168
169     const handle = document.getElementById('scope-popup-drag-handle');
170     handle?.addEventListener('mousedown', (e) => {
171         if (e.button !== 0) return;
172         if (e.target.closest('#scope-popup-close')) return;
173         dragging = true;
174         const rect = popupEl()?.getBoundingClientRect();
175         if (rect) {
176             dragOffX = e.clientX - rect.left;
177             dragOffY = e.clientY - rect.top;
178         }
179         e.preventDefault();
180     });
181
182     popupCanvas = document.getElementById('scope-popup-canvas');
183     popupCanvas?.addEventListener('wheel', (e) => {
184         if (!popupComp) return;
185         e.preventDefault();
186         const next = stepTimeDiv(popupComp.timeDivSec ?? 0.001, e.deltaY < 0);
187         popupComp.timeDivSec = next;
188         onViewChanged(popupComp);
189         renderScopePopup();
190     }, { passive: false });
191
192     popupCanvas?.addEventListener('mousedown', (e) => {
193         if (e.button !== 0 || !popupComp) return;
194         panning = true;
195         panStartX = e.clientX;
196         panStartPosDiv = popupComp.timePositionDiv ?? 0;
197         e.preventDefault();
198     });
199
200     window.addEventListener('mousemove', (e) => {
201         if (panning && popupComp && popupCanvas) {
202             const plotW = popupCanvas.width - 64;
203             const timeDiv = popupComp.timeDivSec > 0 ? popupComp.timeDivSec : 1e-3;
204             const pxPerDiv = plotW / SCOPE_H_DIVS;
205             const deltaDiv = (e.clientX - panStartX) / pxPerDiv;
206             popupComp.timePositionDiv = Math.max(-4, Math.min(4, panStartPosDiv + deltaDiv));
207             renderScopePopup();
208             return;
209         }
210         if (!dragging) return;
211         floatX = e.clientX - dragOffX;
212         floatY = e.clientY - dragOffY;
213         clampFloatPosition();
214         applyFloatPosition();
215     });
216
217     window.addEventListener('mouseup', () => {
218         if (panning) {
219             panning = false;
220             onViewChanged(popupComp);
221             draw();
222         }
223         dragging = false;
224     });
225
226     window.addEventListener('resize', () => {
227         if (isScopePopupOpen()) {
228             clampFloatPosition();
229             applyFloatPosition();
230         }
231     });
232 }
233

```

```

1  /** Moniteur série – UART hardware UNO (TX=D1, RX=D0). */
2  import { circuit, flags } from './state.js';
3  import {
4    getUnoSerialOutput,
5    sendUnoSerialInput,
6    clearUnoSerialOutput,
7    listUnoBoardLabels,
8  } from './led-animation.js';
9
10 let selectedLabel = null;
11 let floatX = 140;
12 let floatY = 90;
13 let dragging = false;
14 let dragOffX = 0;
15 let dragOffY = 0;
16 let lastOutputLen = 0;
17 let popupMounted = false;
18
19 const popupEl = () => document.getElementById('serial-monitor-popup');
20 const titleEl = () => document.getElementById('serial-monitor-title');
21 const outputEl = () => document.getElementById('serial-monitor-output');
22 const inputEl = () => document.getElementById('serial-monitor-input');
23 const boardSel = () => document.getElementById('serial-monitor-board');
24 const baudEl = () => document.getElementById('serial-monitor-baud');
25 const statusEl = () => document.getElementById('serial-monitor-status');
26 const dragHandle = () => document.getElementById('serial-monitor-drag-handle');
27
28 /** Déplace la popup à la racine du document (évite overflow / z-index des conteneurs). */
29 function ensurePopupMounted() {
30   const el = popupEl();
31   if (!el || popupMounted) return;
32   if (el.parentElement !== document.body) {
33     document.body.appendChild(el);
34   }
35   popupMounted = true;
36 }
37
38 function applyFloatPosition() {
39   const el = popupEl();
40   if (!el) return;
41   el.style.left = `${floatX}px`;
42   el.style.top = `${floatY}px`;
43 }
44
45 function syncFloatFromDom() {
46   const el = popupEl();
47   if (!el || el.classList.contains('hidden')) return;
48   const rect = el.getBoundingClientRect();
49   if (rect.width > 0) {
50     floatX = rect.left;
51     floatY = rect.top;
52   }
53 }
54
55 function refreshBoardList() {
56   const sel = boardSel();
57   if (!sel) return;
58   const labels = listUnoBoardLabels();
59   const prev = selectedLabel || sel.value;
60   sel.innerHTML = '';
61   if (labels.length === 0) {
62     const opt = document.createElement('option');
63     opt.value = '';
64     opt.textContent = '(aucune carte UNO)';
65     sel.appendChild(opt);
66     selectedLabel = null;
67     return;
68   }
69   for (const label of labels) {
70     const opt = document.createElement('option');
71     opt.value = label;
72     opt.textContent = label;
73     sel.appendChild(opt);
74   }
75   if (labels.includes(prev)) {
76     sel.value = prev;
77     selectedLabel = prev;
78   } else {

```

```

79     sel.value = labels[0];
80     selectedLabel = labels[0];
81   }
82 }
83
84 function activeUno() {
85   if (!selectedLabel) return null;
86   return circuit.components.find((c) => c.type === 'arduino_uno' && c.label === selectedLabel) || null;
87 }
88
89 export function isSerialMonitorOpen() {
90   const el = popupEl();
91   return el && !el.classList.contains('hidden');
92 }
93
94 export function refreshSerialMonitor() {
95   if (!isSerialMonitorOpen()) return;
96   refreshBoardList();
97   const out = outputEl();
98   const uno = activeUno();
99   if (!out) return;
100
101   const text = selectedLabel ? getUnoSerialOutput(selectedLabel) : '';
102   if (text.length !== lastOutputLen || out.textContent !== text) {
103     out.textContent = text;
104     lastOutputLen = text.length;
105     out.scrollTop = out.scrollHeight;
106   }
107
108   if (baudEl()) {
109     const baud = uno?.serialBaud;
110     baudEl().textContent = baud ? `${baud} baud` : '– baud';
111   }
112   if (statusEl()) {
113     if (!flags.isSimulating) {
114       statusEl().textContent = 'Simulation arrêtée – lancez la simulation pour voir la sortie série.';
115     } else if (!selectedLabel) {
116       statusEl().textContent = 'Ajoutez une carte Arduino UNO au schéma.';
117     } else if (!uno?.serialBegun) {
118       statusEl().textContent = 'Appelez Serial.begin(9600) dans le sketch – aucun fil sur D0/D1 n’est nécessaire pour le moniteur.';
119     } else {
120       statusEl().textContent = 'Moniteur virtuel (pas de câblage D0/D1 requis). TX logiciel → affichage ci-dessus.';
121     }
122   }
123 }
124
125 export function openSerialMonitor(preferredLabel) {
126   ensurePopupMounted();
127   refreshBoardList();
128   if (preferredLabel && listUnoBoardLabels().includes(preferredLabel)) {
129     selectedLabel = preferredLabel;
130     if (boardSel()) boardSel().value = preferredLabel;
131   }
132   if (titleEl()) {
133     titleEl().textContent = selectedLabel
134       ? `Moniteur série – ${selectedLabel}`
135       : 'Moniteur série';
136   }
137   const el = popupEl();
138   if (!el) return;
139   el.classList.remove('hidden');
140   syncFloatFromDom();
141   clampFloatPosition();
142   applyFloatPosition();
143   lastOutputLen = -1;
144   refreshSerialMonitor();
145   inputEl()?.focus({ preventScroll: true });
146 }
147
148 export function closeSerialMonitor() {
149   popupEl()?.classList.add('hidden');
150   dragging = false;
151 }
152
153 function clampFloatPosition() {
154   const el = popupEl();
155   if (!el) return;
156   const w = el.offsetWidth || 520;
157   const h = el.offsetHeight || 360;
158   floatX = Math.max(4, Math.min(window.innerWidth - w - 4, floatX));

```

```

159     floatY = Math.max(52, Math.min(window.innerHeight - h - 4, floatY));
160 }
161
162 function sendInputLine() {
163     const inp = inputEl();
164     if (!inp || !selectedLabel) return;
165     const line = inp.value;
166     if (!line && line !== '') return;
167     sendUnoSerialInput(selectedLabel, line + '\n');
168     inp.value = '';
169     refreshSerialMonitor();
170 }
171
172 export function initSerialMonitor() {
173     ensurePopupMounted();
174
175     document.getElementById('serial-monitor-close')?.addEventListener('click', (e) => {
176         e.stopPropagation();
177         closeSerialMonitor();
178     });
179     document.getElementById('serial-monitor-clear')?.addEventListener('click', () => {
180         if (selectedLabel) clearUnoSerialOutput(selectedLabel);
181         lastOutputLen = -1;
182         refreshSerialMonitor();
183     });
184     document.getElementById('serial-monitor-send')?.addEventListener('click', sendInputLine);
185     boardSel()?.addEventListener('change', () => {
186         selectedLabel = boardSel()?.value || null;
187         if (titleEl() && selectedLabel) titleEl().textContent = `Moniteur série - ${selectedLabel}`;
188         lastOutputLen = -1;
189         refreshSerialMonitor();
190     });
191     inputEl()?.addEventListener('keydown', (e) => {
192         if (e.key === 'Enter' && !e.shiftKey) {
193             e.preventDefault();
194             sendInputLine();
195         }
196     });
197
198     const handle = dragHandle();
199     handle?.addEventListener('pointerdown', (e) => {
200         if (e.button !== 0) return;
201         if (e.target.closest('#serial-monitor-close')) return;
202         dragging = true;
203         const rect = popupEl()?.getBoundingClientRect();
204         if (rect) {
205             dragOffX = e.clientX - rect.left;
206             dragOffY = e.clientY - rect.top;
207             floatX = rect.left;
208             floatY = rect.top;
209         }
210         handle.setPointerCapture(e.pointerId);
211         e.preventDefault();
212     });
213     handle?.addEventListener('pointermove', (e) => {
214         if (!dragging) return;
215         floatX = e.clientX - dragOffX;
216         floatY = e.clientY - dragOffY;
217         clampFloatPosition();
218         applyFloatPosition();
219     });
220     const endDrag = () => {
221         dragging = false;
222     };
223     handle?.addEventListener('pointerup', endDrag);
224     handle?.addEventListener('pointercancel', endDrag);
225
226     window.addEventListener('resize', () => {
227         if (isSerialMonitorOpen()) {
228             clampFloatPosition();
229             applyFloatPosition();
230         }
231     });
232 }
233
234

```

```

1 // simulation.js
2 import { circuit, flags, simulationResults, GRID_SIZE } from './state.js';
3 import { cd4511JunctionToTerminalKey } from './cd4511-layout.js';
4 import { ic74hc90JunctionToTerminalKey } from './ic74hc90-layout.js';
5 import { arduinoUnoJunctionToTerminalKey } from './arduino-uno-layout.js';
6 import { draw } from './renderer.js';
7 import { startLedAnimation, stopLedAnimation, startBurntLedSmokeLoop, isLedOvercurrent, hasLedAnimation, hasVoltmeterAnimation, ensureArduino
  LedAnimation, hasArduinoStaticIdealDisplay, resetArduinoRuntimes } from './led-animation.js';
8 import { startScopeAnimation, stopScopeAnimation, SCOPE_H_DIVS } from './scope-animation.js';
9 import { openScopePanel, isScopePanelOpen, getActiveScope, refreshScopePanelFields } from './scope-panel.js';
10 import { isScopePopupOpen, refreshScopePopup } from './scope-popup.js';
11 import { isSerialMonitorOpen, refreshSerialMonitor } from './serial-monitor-popup.js';
12 import { isBodePopupOpen, openBodePopup, refreshBodePopup } from './bode-popup.js';
13 import { startSpeakerAudio, stopSpeakerAudio } from './speaker-audio.js';
14
15 let liveSimTimer = null;
16 let simulationInFlight = false;
17 let simulationQueuedSilent = false;
18 /** @type {AbortController | null} */
19 let simulationAbortController = null;
20 let simulationGeneration = 0;
21
22 /** Relance ngspice pendant une simulation active (valeurs composants / sources). */
23 export function requestLiveSimulation() {
24   if (!flags.isSimulating) return;
25   clearTimeout(liveSimTimer);
26   liveSimTimer = setTimeout(() => {
27     if (flags.isSimulating) triggerSimulation(true);
28   }, 400);
29 }
30
31 import { prepareArduinoForSimulation } from './arduino-editor.js';
32 import {
33   arduinoUnoMinPulsePeriodSec,
34   applyArduinoSketchToComponent,
35   arduinoGpioIsTimeVarying,
36   sketchHasLoop,
37 } from './Engine/arduino-sketch-parse.mjs';
38 import { arduinoUartScopePeriodSec } from './Engine/arduino-uart-wave.mjs';
39 import { annotateUnoI2cBusEngine } from './Engine/i2c-bus-ideal.mjs';
40
41 /** Base de temps adaptée à l'I²C (100 kHz) si le scope est encore en ms/div. */
42 function autoTuneScopeForI2c() {
43   const simWires = circuit.wires
44     .filter((w) => w.fromJunctionId && w.toJunctionId)
45     .map((w) => ({
46       solid: true,
47       fromKey: junctionIdToTerminalKey(w.fromJunctionId),
48       toKey: junctionIdToTerminalKey(w.toJunctionId),
49       points: [],
50     })))
51     .filter((w) => w.fromKey && w.toKey);
52   let hasI2c = false;
53   for (const comp of circuit.components) {
54     if (comp.type !== 'arduino_uno') continue;
55     annotateUnoI2cBusEngine(comp, circuit.components, simWires);
56     if (comp.i2cBus?.active) hasI2c = true;
57   }
58   if (!hasI2c) return;
59   for (const osc of circuit.components) {
60     if (osc.type !== 'oscilloscope') continue;
61     if ((osc.timeDivSec ?? 0.001) > 0.0001) osc.timeDivSec = 0.00005;
62   }
63 }
64
65 /** Base de temps adaptée au clignotement Arduino (delay) sur l'oscilloscope. */
66 function autoTuneScopeForArduino() {
67   for (const comp of circuit.components) {
68     if (comp.type === 'arduino_uno') applyArduinoSketchToComponent(comp);
69   }
70   const uartPeriod = arduinoUartScopePeriodSec(circuit.components);
71   const gpioPeriod = arduinoUnoMinPulsePeriodSec(circuit.components);
72   const period = uartPeriod > 0 ? uartPeriod : gpioPeriod;
73   if (!period || period <= 0) return;
74   const idealDiv = Math.min(0.1, period / SCOPE_H_DIVS);
75   for (const osc of circuit.components) {
76     if (osc.type !== 'oscilloscope') continue;
77     const cur = osc.timeDivSec ?? 0.001;

```

```

78     if (cur < idealDiv * 0.5) osc.timeDivSec = idealDiv;
79   }
80 }
81
82 const COMPONENT_TYPE_TO_ENGINE = {
83   battery: 'vsource', vcc: 'vterm', logic_terminal: 'logic_state',
84   gnd: 'ground',
85   not: 'logic_not', and: 'logic_and', nand: 'logic_nand', or: 'logic_or', nor: 'logic_nor', xor: 'logic_xor', xnor: 'logic_xnor',
86   d_flipflop: 'logic_dff', jk_flipflop: 'logic_jk', cd4511: 'logic_cd4511', ic_74hc90: 'ic_74hc90', arduino_uno: 'arduino_uno', led: 'diode',
   _led', seg7: 'seg7', bargraph_dc10h: 'bargraph_dc10h', grove_lcd16x2: 'grove_lcd16x2', grove_dht22: 'grove_dht22',
87   gimp: 'vpulse', gsin: 'vsin', gsqr: 'vsquare',
88 };
89
90 function formatGsinValue(comp) {
91   const a = comp.peakAmplitude ?? 5;
92   const f = comp.frequency ?? 1000;
93   const o = comp.offset ?? 0;
94   const fStr = f >= 1000 && f % 1000 === 0 ? `${f / 1000}kHz` : `${f}Hz`;
95   return `${a}V ${fStr} ${o}V`;
96 }
97
98 function formatGimpValue(comp) {
99   const v = comp.voltageRail ?? 5;
100   const f = comp.frequency ?? 2;
101   const d = comp.dutyCycle ?? 50;
102   let fStr;
103   if (f > 0 && f < 1) {
104     const period = 1 / f;
105     fStr = `${period >= 10 ? Math.round(period * 1000) / 1000 : period}s`;
106   } else if (f >= 1000 && f % 1000 === 0) {
107     fStr = `${f / 1000}kHz`;
108   } else {
109     fStr = `${f}Hz`;
110   }
111   return `${v}V ${fStr} ${d}%`;
112 }
113
114 function formatGsqrValue(comp) {
115   const a = comp.peakAmplitude ?? 5;
116   const f = comp.frequency ?? 1000;
117   const o = comp.offset ?? 0;
118   const fStr = f >= 1000 && f % 1000 === 0 ? `${f / 1000}kHz` : `${f}Hz`;
119   return `${a}V ${fStr} offset ${o}V`;
120 }
121
122 function jonctionIdToTerminalKey(jonctionId) {
123   const junc = circuit.autoJunctions.find((j) => j.id === junctionId);
124   if (junc) return `__t${junc.x}#${junc.y}`;
125   for (const comp of circuit.components) {
126     const id = comp.label; if (!id) continue;
127     if (comp.type === 'not') {
128       if (junctionId === `${id}_inA`) return `${id}#0`;
129       if (junctionId === `${id}_out`) return `${id}#1`;
130     } else if (['and', 'nand', 'or', 'nor', 'xor', 'xnor'].includes(comp.type)) {
131       if (junctionId === `${id}_inA`) return `${id}#0`;
132       if (junctionId === `${id}_inB`) return `${id}#1`;
133       if (junctionId === `${id}_out`) return `${id}#2`;
134     } else if (comp.type === 'd_flipflop') {
135       if (junctionId === `${id}_D`) return `${id}#0`;
136       if (junctionId === `${id}_CLK`) return `${id}#1`;
137       if (junctionId === `${id}_Q`) return `${id}#2`;
138       if (junctionId === `${id}_Qbar`) return `${id}#3`;
139       if (junctionId === `${id}_SET`) return `${id}#4`;
140       if (junctionId === `${id}_RESET`) return `${id}#5`;
141     } else if (comp.type === 'jk_flipflop') {
142       if (junctionId === `${id}_J`) return `${id}#0`;
143       if (junctionId === `${id}_K`) return `${id}#1`;
144       if (junctionId === `${id}_CLK`) return `${id}#2`;
145       if (junctionId === `${id}_Q`) return `${id}#3`;
146       if (junctionId === `${id}_Qbar`) return `${id}#4`;
147       if (junctionId === `${id}_SET`) return `${id}#5`;
148       if (junctionId === `${id}_RESET`) return `${id}#6`;
149     } else if (['gnd', 'vcc', 'logic_terminal'].includes(comp.type)) {
150       if (junctionId === `${id}_out`) return `${id}#0`;
151     } else if (comp.type === 'gimp') {
152       if (junctionId === `${id}_in`) return `${id}#1`;
153       if (junctionId === `${id}_out`) return `${id}#0`;
154     } else if (comp.type === 'gsin') {
155       if (junctionId === `${id}_in`) return `${id}#0`;
156       if (junctionId === `${id}_out`) return `${id}#1`;

```



```

157     } else if (comp.type === 'gsqr') {
158         if (junctionId === `${id}_in`) return `${id}#0`;
159         if (junctionId === `${id}_out`) return `${id}#1`;
160     } else if (comp.type === 'oscilloscope') {
161         if (junctionId === `${id}_CH1`) return `${id}#0`;
162         if (junctionId === `${id}_CH2`) return `${id}#1`;
163         if (junctionId === `${id}_GND`) return `${id}#2`;
164     } else if (comp.type === 'npn') {
165         if (junctionId === `${id}_B`) return `${id}#0`;
166         if (junctionId === `${id}_C`) return `${id}#1`;
167         if (junctionId === `${id}_E`) return `${id}#2`;
168     } else if (comp.type === 'opamp') {
169         if (junctionId === `${id}_plus`) return `${id}#0`;
170         if (junctionId === `${id}_minus`) return `${id}#1`;
171         if (junctionId === `${id}_out`) return `${id}#2`;
172     } else if (comp.type === 'seg7') {
173         const segs = ['a', 'b', 'c', 'd', 'e', 'f', 'g'];
174         for (let i = 0; i < segs.length; i++) {
175             if (junctionId === `${id}_${segs[i]}`) return `${id}#${i}`;
176         }
177         if (junctionId === `${id}_COM`) return `${id}#7`;
178     } else if (comp.type === 'bargraph_dc10h') {
179         for (let i = 0; i < 10; i++) {
180             if (junctionId === `${id}_s${i + 1}`) return `${id}#${i}`;
181         }
182         if (junctionId === `${id}_COM`) return `${id}#10`;
183     } else if (comp.type === 'grove_lcd16x2') {
184         const pins = ['SDA', 'SCL', 'VCC', 'GND'];
185         for (let i = 0; i < pins.length; i++) {
186             if (junctionId === `${id}_${pins[i]}`) return `${id}#${i}`;
187         }
188     } else if (comp.type === 'grove_dht22') {
189         const pins = ['DATA', 'VCC', 'NC', 'GND'];
190         for (let i = 0; i < pins.length; i++) {
191             if (junctionId === `${id}_${pins[i]}`) return `${id}#${i}`;
192         }
193     } else if (comp.type === 'cd4511') {
194         const key = cd4511JunctionToTerminalKey(id, junctionId);
195         if (key) return key;
196     } else if (comp.type === 'ic_74hc90') {
197         const key = ic74hc90JunctionToTerminalKey(id, junctionId);
198         if (key) return key;
199     } else if (comp.type === 'arduino_uno') {
200         const key = arduinoUnoJunctionToTerminalKey(id, junctionId);
201         if (key) return key;
202     } else if (comp.type === 'potentiometer') {
203         if (junctionId === `${id}_in`) return `${id}#0`;
204         if (junctionId === `${id}_wip`) return `${id}#1`;
205         if (junctionId === `${id}_out`) return `${id}#2`;
206     } else if (comp.type === 'switch_spdt') {
207         if (junctionId === `${id}_com`) return `${id}#0`;
208         if (junctionId === `${id}_a`) return `${id}#1`;
209         if (junctionId === `${id}_b`) return `${id}#2`;
210     } else {
211         if (junctionId === `${id}_in`) return `${id}#0`;
212         if (junctionId === `${id}_out`) return `${id}#1`;
213     }
214 }
215 return null;
216 }
217
218 function buildSimulationState() {
219     const simComponents = circuit.components.map((comp) => {
220         const id = comp.label;
221         const engineType = COMPONENT_TYPE_TO_ENGINE[comp.type] || comp.type;
222         const out = { id, type: engineType, x: comp.x, y: comp.y, rotation: comp.rotation || 0 };
223         if (comp.type === 'logic_terminal') {
224             out.value = comp.state === 1 ? '1' : '0';
225             out.logicRail = comp.highVoltage !== undefined ? comp.highVoltage : 5;
226         } else if (comp.type === 'vcc') {
227             out.value = comp.value !== undefined ? String(comp.value) : '5';
228         } else if (engineType === 'vsource') {
229             out.value = comp.value || '5';
230         } else if (comp.type === 'gimp') {
231             out.value = formatGimpValue(comp);
232         } else if (comp.type === 'gsin') {
233             out.value = formatGsinValue(comp);
234         } else if (comp.type === 'gsqr') {
235             out.value = formatGsqrValue(comp);
236         }

```

```

237     if (comp.type === 'resistor') out.value = comp.value || '1k';
238     if (comp.type === 'potentiometer') {
239         out.value = comp.value || '10k';
240         out.position = comp.position ?? 50;
241     }
242     if (comp.type === 'switch_spdt') out.state = comp.state ?? 0;
243     if (comp.type === 'push_button') out.state = comp.state ?? 0;
244     if (comp.type === 'capacitor') out.value = comp.value || '1u';
245     if (comp.type === 'inductor') out.value = comp.value || '1m';
246     if (comp.type === 'diode') out.value = comp.value || '1N4148';
247     if (comp.type === 'speaker') out.value = comp.value || '8';
248     if (comp.type === 'npn') out.value = comp.value || '2N2222';
249     if (comp.type === 'opamp') {
250         out.value = comp.value || 'uA741';
251         out.vp = comp.vp ?? 15;
252         out.vn = comp.vn ?? -15;
253     }
254     if (comp.type === 'oscilloscope') {
255         out.timeDivSec = comp.timeDivSec ?? 0.001;
256         out.ch1VoltsPerDiv = comp.ch1VoltsPerDiv ?? 1;
257         out.ch2VoltsPerDiv = comp.ch2VoltsPerDiv ?? 1;
258         out.ch1PositionDiv = comp.ch1PositionDiv ?? 0;
259         out.ch2PositionDiv = comp.ch2PositionDiv ?? 0;
260         out.timePositionDiv = comp.timePositionDiv ?? 0;
261     }
262     if (comp.type === 'arduino_uno') {
263         applyArduinoSketchToComponent(comp);
264         out.sketch = comp.sketch || '';
265         out.fqbn = comp.fqbn || 'arduino:avr:uno';
266         out.pinModes = comp.pinModes || {};
267         out.pinLevels = comp.pinLevels || {};
268         out.pinPulses = comp.pinPulses || {};
269         out.pinPhases = comp.pinPhases || [];
270         out.avrRegisters = comp.avrRegisters || null;
271     }
272     return out;
273 });
274 const simWires = circuit.wires
275     .filter((w) => w.fromJunctionId && w.toJunctionId)
276     .map((w) => {
277         const fromKey = junctionIdToTerminalKey(w.fromJunctionId);
278         const toKey = junctionIdToTerminalKey(w.toJunctionId);
279         const pts = Array.isArray(w.points) && w.points.length >= 2
280             ? w.points
281             : [{ x: 0, y: 0 }, { x: 0, y: 0 }];
282         return { solid: true, fromKey, toKey, points: pts };
283     })
284     .filter((w) => w.fromKey && w.toKey);
285 const dropped = circuit.wires.length - simWires.length;
286 if (dropped > 0) {
287     console.warn(
288         `[Simulation] ${dropped} fil(s) ignoré(s) : jonction non reconnue ou extrémité manquante. Vérifiez le câblage (CD4511 : broches
A...LT, a...g).`
289     );
290 }
291 return { components: simComponents, wires: simWires };
292 }
293
294 /** URL de POST /api/simulate (serveur Node, même machine que la page). */
295 function resolveSimulationApiBaseUrl() {
296     const { protocol, hostname, port, pathname } = window.location;
297     const isSimulateurH = new URLSearchParams(window.location.search).get('app') === 'h';
298     if (protocol === 'file:') {
299         return isSimulateurH ? 'http://127.0.0.1:43721' : 'http://127.0.0.1:3000';
300     }
301     if (pathname.startsWith('/Simulateur')) return window.location.origin;
302     const p = port || (protocol === 'https:' ? '443' : '80');
303     if (p === '3000' || p === '80' || p === '443') return window.location.origin;
304     const host =
305         hostname === 'localhost' || hostname === '127.0.0.1' ? '127.0.0.1' : hostname;
306     return `${protocol}://${host}:${p}`;
307 }
308
309 function formatSimulationNetworkError(err, baseUrl) {
310     const msg = err?.message || String(err);
311     if (/Cannot GET/i.test(msg)) {
312         return (
313             `Le serveur Node ne gère pas cette requête (réponse « Cannot GET »).\n\n` +
314             `1. Dans le dossier du projet : npm start\n` +
315             `2. Ouvrez http://localhost:3000/Simulateur/ (pas le fichier HTML en double-clic)\n\n` +

```

```

316     `URL tentée : ${baseUrl}/api/simulate`
317   );
318 }
319 if (/Failed to fetch|NetworkError|ERR_CONNECTION|fetch/i.test(msg)) {
320   return (
321     `Impossible de joindre le serveur (${baseUrl}).\n\n` +
322     `Lancez « npm start », puis ouvrez http://localhost:3000/Simulateur/`
323   );
324 }
325 if (/EPERM|operation not permitted|bloqué|blocked|antivirus/i.test(msg)) {
326   return (
327     `ngspice bloqué par l'antivirus Windows (EPERM).\n\n` +
328     `Les circuits 74HC90 / CD4511 nécessitent ngspice_con.exe dans Simulateur/bin/.\n` +
329     `Demandez une exception pour ce dossier, ou testez : npm run check-ngspice`
330   );
331 }
332 return msg;
333 }
334
335 function formatSpiceEngineErrors(result) {
336   if (Array.isArray(result?.errors) && result.errors.length) {
337     return result.errors.map(String).filter((e) => e.trim()).join('\n');
338   }
339   return result?.error || result?.message || 'Vérifiez les masses et le câblage (VCC, GND, BI/LT du CD4511, MR du 74HC90).';
340 }
341
342 function warnDroppedWiresForLogicCircuits(dropped, isSilentUpdate) {
343   if (isSilentUpdate || dropped <= 0) return;
344   const hasLogicIc = circuit.components.some((c) => c.type === 'cd4511' || c.type === 'ic_74hc90');
345   if (!hasLogicIc) return;
346   alert(
347     `${dropped} fil(s) ignoré(s) – le schéma n'est pas entièrement transmis au moteur SPICE.\n\n` +
348     `Vérifiez le câblage (74HC90 : CP0, VCC, GND, MR1/MR2 à 0 V, Q0→CP1, Q0..Q3→CD4511 A..D ; ` +
349     `CD4511 : BI et LT au +5 V, LE à 0 V, a..g → afficheur).`
350   );
351 }
352
353 function showSimulationWarnings(warnings, isSilentUpdate) {
354   if (isSilentUpdate || !Array.isArray(warnings) || !warnings.length) return;
355   const text = warnings.map(String).filter((w) => w.trim()).join('\n');
356   if (!text) return;
357   const critical = warnings.some((w) =>
358     /CD4511|74HC90|XSPICE|digital\.cm|BI relié|LT relié|Q0 relié|fréquentielle|Analyse fréquentielle|Bode/i.test(String(w))
359   );
360   if (critical) alert(`Avertissement simulation : \n${text}`);
361 }
362
363 /** Démarre l'animation GPIO avant la fin du calcul SPICE (bargraph / LED Arduino). */
364 function shouldStartArduinoLiveDisplayEarly() {
365   let hasUno = false;
366   for (const comp of circuit.components) {
367     if (comp.type !== 'arduino_uno') continue;
368     hasUno = true;
369     applyArduinoSketchToComponent(comp);
370     if (sketchHasLoop(comp.sketch || '') || arduinoGpioIsTimeVarying(comp)) return true;
371   }
372   if (!hasUno) return false;
373   return circuit.components.some((c) =>
374     c.type === 'bargraph_dc10h' || c.type === 'seg7' || c.type === 'grove_lcd16x2'
375   );
376 }
377
378 export async function triggerSimulation(isSilentUpdate = false) {
379   if (simulationInFlight) {
380     if (isSilentUpdate) simulationQueuedSilent = true;
381     return;
382   }
383   simulationInFlight = true;
384   simulationGeneration += 1;
385   const myGeneration = simulationGeneration;
386   if (simulationAbortController) simulationAbortController.abort();
387   simulationAbortController = new AbortController();
388   const { signal } = simulationAbortController;
389   prepareArduinoForSimulation();
390   if (!isSilentUpdate) {
391     autoTuneScopeForI2c();
392     autoTuneScopeForArduino();
393     refreshScopePanelFields();
394   }
395   const baseUrl = resolveSimulationApiBaseUrl();
396   const payload = { state: buildSimulationState(), gridStep: GRID_SIZE };

```

```

396 const wiredCount = circuit.wires.filter((w) => w.fromJunctionId && w.toJunctionId).length;
397 warnDroppedWiresForLogicCircuits(wiredCount - payload.state.wires.length, isSilentUpdate);
398 const btnSim = document.getElementById('btn-simulate');
399 const btnStop = document.getElementById('btn-stop');
400 if (btnSim && !isSilentUpdate) { btnSim.innerText = " ⚡ Calculs SPICE..."; btnSim.style.background = "#ff9800"; }
401 if (!isSilentUpdate && shouldStartArduinoLiveDisplayEarly()) {
402     flags.isSimulating = true;
403     resetArduinoRuntimes();
404     ensureArduinoLedAnimation();
405     if (btnStop) btnStop.classList.remove('disabled');
406     draw();
407 }
408 try {
409     const response = await fetch(`${baseUrl}/api/simulate`, {
410         method: 'POST',
411         headers: { 'Content-Type': 'application/json' },
412         credentials: 'include',
413         body: JSON.stringify(payload),
414         signal,
415     });
416     if (myGeneration !== simulationGeneration) return;
417     if (!response.ok) {
418         let backendError = "";
419         try {
420             const errJson = await response.json();
421             backendError =
422                 errJson.error ||
423                 errJson.message ||
424                 (Array.isArray(errJson.errors) ? errJson.errors.join('\n') : '');
425         } catch (e) {
426             backendError = await response.text();
427         }
428         throw new Error(formatSimulationNetworkError({ message: backendError }, baseUrl));
429     }
430     const result = await response.json();
431     if (myGeneration !== simulationGeneration) return;
432     if (result.ok) {
433         showSimulationWarnings(result.warnings, isSilentUpdate);
434         simulationResults.voltmeters = result.voltmeterValues || {};
435         simulationResults.ammeters = result.ammeterValues || {};
436         simulationResults.ohmmeters = result.ohmmeterValues || {};
437         simulationResults.leds = result.ledValues || {};
438         simulationResults.scopePlots = result.scopePlots || {};
439         simulationResults.bodePlots = result.bodePlots || {};
440         simulationResults.seg7 = result.seg7Values || {};
441         simulationResults.bargraph = result.bargraphValues || {};
442         simulationResults.logicValues = result.logicValues || {};
443         const scopePlotKeys = Object.keys(simulationResults.scopePlots);
444         const hasOsc = circuit.components.some((c) => c.type === 'oscilloscope');
445         if (scopePlotKeys.length > 0 || hasOsc) {
446             startScopeAnimation(simulationResults.scopePlots);
447         } else {
448             stopScopeAnimation();
449         }
450         const osc = circuit.components.find((c) => c.type === 'oscilloscope');
451         if (osc) {
452             if (!isSilentUpdate) {
453                 openScopePanel(osc, { openPopup: true });
454             } else {
455                 const keepPopup = isScopePopupOpen();
456                 const keepPanel = isScopePanelOpen();
457                 if (keepPanel || keepPopup) {
458                     openScopePanel(getActiveScope() || osc, { openPopup: keepPopup });
459                 }
460                 refreshScopePopup();
461             }
462         }
463         if (isSerialMonitorOpen()) refreshSerialMonitor();
464         const bodeComp = circuit.components.find((c) => c.type === 'bode_analyzer');
465         if (bodeComp && (result.analysisAc || Object.keys(simulationResults.bodePlots).length > 0)) {
466             if (!isSilentUpdate) {
467                 openBodePopup(bodeComp);
468             } else if (isBodePopupOpen()) {
469                 refreshBodePopup();
470             }
471         }
472         if (result.analysisTran) {
473             const vmPlots = result.voltmeterTranPlots || {};
474             const ledPlots = result.ledTranPlots || {};
475             const seg7Plots = result.seg7TranPlots || {};

```

```

476     const bargraphPlots = result.bargraphTranPlots || {};
477     const logicGateTranPlots = result.logicGateTranPlots || {};
478     const arduinoIdeal = hasArduinoStaticIdealDisplay();
479     if (
480         Object.keys(vmPlots).length ||
481         Object.keys(ledPlots).length ||
482         Object.keys(seg7Plots).length ||
483         Object.keys(bargraphPlots).length ||
484         Object.keys(logicGateTranPlots).length
485     ) {
486         startLedAnimation(ledPlots, vmPlots, seg7Plots, logicGateTranPlots, {
487             keepClock: isSilentUpdate,
488             bargraphPlots,
489         });
490     } else if (!arduinoIdeal) {
491         stopLedAnimation();
492     }
493     const speakerPlots = result.speakerTranPlots || {};
494     if (Object.keys(speakerPlots).length > 0) {
495         startSpeakerAudio(speakerPlots);
496     } else {
497         stopSpeakerAudio();
498     }
499     } else {
500         stopLedAnimation(true);
501         stopSpeakerAudio();
502     }
503     const hasBurntLed = Object.values(simulationResults.leds).some((m) => {
504         const i = m && typeof m === 'object' ? m.current : m;
505         return isLedOvercurrent(i);
506     });
507     if (hasBurntLed && !hasLedAnimation() && !hasVoltmeterAnimation()) startBurntLedSmokeLoop();
508     flags.isSimulating = true;
509     resetArduinoRuntimes();
510     ensureArduinoLedAnimation();
511     if (btnSim) { btnSim.innerText = "▶ Simulation Live"; btnSim.style.background = "#00bcd4"; }
512     if (btnStop) btnStop.classList.remove('disabled');
513     draw();
514     } else {
515         if (!isSilentUpdate) alert("Erreur du moteur SPICE :\n" + formatSpiceEngineErrors(result));
516         stopSimulation();
517     }
518 } catch (err) {
519     if (err?.name === 'AbortError' || myGeneration !== simulationGeneration) return;
520     if (!isSilentUpdate) alert(`Erreur réseau :\n${formatSimulationNetworkError(err, baseUrl)}`);
521     stopSimulation();
522 } finally {
523     simulationInFlight = false;
524     if (myGeneration !== simulationGeneration) return;
525     if (simulationQueuedSilent) {
526         simulationQueuedSilent = false;
527         triggerSimulation(true);
528     }
529 }
530 }
531
532 export function stopSimulation() {
533     simulationGeneration += 1;
534     simulationQueuedSilent = false;
535     if (simulationAbortController) {
536         simulationAbortController.abort();
537         simulationAbortController = null;
538     }
539     simulationInFlight = false;
540     clearTimeout(liveSimTimer);
541     liveSimTimer = null;
542     stopLedAnimation();
543     stopScopeAnimation();
544     stopSpeakerAudio();
545     flags.isSimulating = false;
546     simulationResults.voltmeters = {}; simulationResults.ammeters = {}; simulationResults.ohmmeters = {}; simulationResults.leds = {}; simulationResults.scopePlots = {}; simulationResults.bodePlots = {}; simulationResults.seg7 = {}; simulationResults.bargraph = {}; simulationResults.logicValues = {};
547     const btnSim = document.getElementById('btn-simulate'); const btnStop = document.getElementById('btn-stop');
548     if (btnSim) { btnSim.innerText = "⚡ Lancer Simulation"; btnSim.style.background = "#00ca71"; }
549     if (btnStop) btnStop.classList.add('disabled');
550     draw();
551 }

```

```

1 // source-panel.js - panneau de réglage Gimp et générateurs
2 import { flags, saveState } from './state.js';
3 import { draw, resizeCanvas } from './renderer.js';
4 import { requestLiveSimulation } from './simulation.js';
5
6 let activeSource = null;
7
8 const panel = () => document.getElementById('source-panel');
9 const titleEl = () => document.getElementById('source-panel-title');
10 const gimpFields = () => document.getElementById('gimp-fields');
11 const gsinFields = () => document.getElementById('gsin-fields');
12 const gsqrFields = () => document.getElementById('gsqr-fields');
13
14 const gimpVoltageEl = () => document.getElementById('gimp-voltage');
15 const gimpFreqEl = () => document.getElementById('gimp-freq');
16 const gimpDutyEl = () => document.getElementById('gimp-duty');
17 const gimpClockPresetEl = () => document.getElementById('gimp-clock-preset');
18
19 const gsinAmpEl = () => document.getElementById('gsin-amplitude');
20 const gsinFreqEl = () => document.getElementById('gsin-freq');
21 const gsinOffsetEl = () => document.getElementById('gsin-offset');
22
23 const gsqrAmpEl = () => document.getElementById('gsqr-amplitude');
24 const gsqrFreqEl = () => document.getElementById('gsqr-freq');
25 const gsqrOffsetEl = () => document.getElementById('gsqr-offset');
26
27 export function isSourcePanelOpen() {
28   const p = panel();
29   return p && !p.classList.contains('hidden');
30 }
31
32 export function getSourcePanelHeight() {
33   const p = panel();
34   return isSourcePanelOpen() && p ? p.offsetHeight : 0;
35 }
36
37 /** Hauteur du panneau bas (source ou oscilloscope). */
38 export function getBottomPanelHeight() {
39   const p = panel();
40   return p && !p.classList.contains('hidden') ? p.offsetHeight : 0;
41 }
42
43 export function clearSourceSelection() {
44   activeSource = null;
45   if (gimpFields()) {
46     gimpFields().classList.add('hidden');
47     gimpFields().style.display = 'none';
48   }
49   if (gsinFields()) {
50     gsinFields().classList.add('hidden');
51     gsinFields().style.display = 'none';
52   }
53   if (gsqrFields()) {
54     gsqrFields().classList.add('hidden');
55     gsqrFields().style.display = 'none';
56   }
57 }
58
59 /** @deprecated compat */
60 export const isGimpPanelOpen = isSourcePanelOpen;
61 export const getGimpPanelHeight = getSourcePanelHeight;
62
63 export function getActiveSource() {
64   return activeSource;
65 }
66
67 function showFieldsForType(type) {
68   const g = gimpFields();
69   const s = gsinFields();
70   const q = gsqrFields();
71   if (g) { g.classList.toggle('hidden', type !== 'gimp'); g.style.display = type === 'gimp' ? 'flex' : 'none'; }
72   if (s) { s.classList.toggle('hidden', type !== 'gsin'); s.style.display = type === 'gsin' ? 'flex' : 'none'; }
73   if (q) { q.classList.toggle('hidden', type !== 'gsqr'); q.style.display = type === 'gsqr' ? 'flex' : 'none'; }
74 }
75
76 function syncFieldsFromComponent(comp) {
77   if (!comp) return;
78   showFieldsForType(comp.type);

```

```

79     if (comp.type === 'gimp') {
80         if (titleEl()) titleEl().textContent = `${comp.label} – Générateur d'impulsions`;
81         if (gimpVoltageEl()) gimpVoltageEl().value = String(comp.voltageRail ?? 5);
82         if (gimpFreqEl()) gimpFreqEl().value = String(comp.frequency ?? 1000);
83         if (gimpDutyEl()) gimpDutyEl().value = String(comp.dutyCycle ?? 10);
84     } else if (comp.type === 'gsin') {
85         if (titleEl()) titleEl().textContent = `${comp.label} – Générateur sinusoïdal`;
86         if (gsinAmpEl()) gsinAmpEl().value = String(comp.peakAmplitude ?? 5);
87         if (gsinFreqEl()) gsinFreqEl().value = String(comp.frequency ?? 1000);
88         if (gsinOffsetEl()) gsinOffsetEl().value = String(comp.offset ?? 0);
89     } else if (comp.type === 'gsqr') {
90         if (titleEl()) titleEl().textContent = `${comp.label} – Générateur carré`;
91         if (gsqrAmpEl()) gsqrAmpEl().value = String(comp.peakAmplitude ?? 5);
92         if (gsqrFreqEl()) gsqrFreqEl().value = String(comp.frequency ?? 1000);
93         if (gsqrOffsetEl()) gsqrOffsetEl().value = String(comp.offset ?? 0);
94     }
95 }
96
97 function applyFieldsToComponent() {
98     if (!activeSource) return;
99     if (activeSource.type === 'gimp') {
100         activeSource.voltageRail = parseFloat(gimpVoltageEl()?.value) || 5;
101         activeSource.frequency = Math.max(1 / 86400, parseFloat(gimpFreqEl()?.value) || 2);
102         activeSource.dutyCycle = Math.min(99, Math.max(1, parseFloat(gimpDutyEl()?.value) || 10));
103     } else if (activeSource.type === 'gsin') {
104         activeSource.peakAmplitude = Math.max(0, parseFloat(gsinAmpEl()?.value) || 5);
105         activeSource.frequency = Math.max(0.1, parseFloat(gsinFreqEl()?.value) || 1000);
106         activeSource.offset = parseFloat(gsinOffsetEl()?.value) || 0;
107     } else if (activeSource.type === 'gsqr') {
108         activeSource.peakAmplitude = Math.max(0, parseFloat(gsqrAmpEl()?.value) || 5);
109         activeSource.frequency = Math.max(0.1, parseFloat(gsqrFreqEl()?.value) || 1000);
110         activeSource.offset = parseFloat(gsqrOffsetEl()?.value) || 0;
111     }
112     draw();
113     if (flags.isSimulating) requestLiveSimulation();
114 }
115
116 export function openSourcePanel(comp) {
117     if (!comp || (comp.type !== 'gimp' && comp.type !== 'gsin' && comp.type !== 'gsqr')) return;
118     activeSource = comp;
119     syncFieldsFromComponent(comp);
120     const sf = document.getElementById('scope-fields');
121     if (sf) { sf.classList.add('hidden'); sf.style.display = 'none'; }
122     panel()?.classList.remove('hidden');
123     resizeCanvas();
124 }
125
126 export function openGimpPanel(comp) {
127     openSourcePanel(comp);
128 }
129
130 export function closeSourcePanel() {
131     activeSource = null;
132     if (gimpFields()) { gimpFields().classList.add('hidden'); gimpFields().style.display = 'none'; }
133     if (gsinFields()) { gsinFields().classList.add('hidden'); gsinFields().style.display = 'none'; }
134     if (gsqrFields()) { gsqrFields().classList.add('hidden'); gsqrFields().style.display = 'none'; }
135     const sf = document.getElementById('scope-fields');
136     if (!sf || sf.classList.contains('hidden')) {
137         panel()?.classList.add('hidden');
138     }
139     resizeCanvas();
140 }
141
142 export function closeGimpPanel() {
143     closeSourcePanel();
144 }
145
146 export function onSourceRemoved(comp) {
147     if (activeSource === comp) closeSourcePanel();
148 }
149
150 export function onGimpRemoved(comp) {
151     onSourceRemoved(comp);
152 }
153
154 function formatFreqLabel(f) {
155     if (!(f > 0)) return '?';
156     if (f >= 1000 && f % 1000 === 0) return `${f / 1000} kHz`;
157     if (f < 1) {
158         const p = 1 / f;

```

```

159         if (p >= 3600 && Math.abs(p % 3600) < 0.01) return `1/${p / 3600} h`;
160         if (p >= 60 && Math.abs(p % 60) < 0.01) return `1/${p / 60} min`;
161         return `${p >= 10 ? Math.round(p) : p.toFixed(2)} s`;
162     }
163     return `${f} Hz`;
164 }
165
166 export function formatGimpLabel(comp) {
167     const v = comp.voltageRail ?? 5;
168     const f = comp.frequency ?? 2;
169     const d = comp.dutyCycle ?? 10;
170     return `0-${v}V ${formatFreqLabel(f)} ${d}%`;
171 }
172
173 export function formatGsinLabel(comp) {
174     const a = comp.peakAmplitude ?? 5;
175     const f = comp.frequency ?? 1000;
176     const o = comp.offset ?? 0;
177     const fStr = f >= 1000 && f % 1000 === 0 ? `${f / 1000}kHz` : `${f}Hz`;
178     return `${a}V crête ${fStr} offset ${o}V`;
179 }
180
181 export function initSourcePanel() {
182     const apply = () => {
183         if (!activeSource) return;
184         applyFieldsToComponent();
185     };
186     const onChange = () => {
187         if (!activeSource) return;
188         if (!flags.isSimulating) saveState();
189         applyFieldsToComponent();
190     };
191
192     gimpVoltageEl()?.addEventListener('change', onChange);
193     gimpFreqEl()?.addEventListener('change', onChange);
194     gimpDutyEl()?.addEventListener('change', onChange);
195     gimpClockPresetEl()?.addEventListener('change', () => {
196         const preset = gimpClockPresetEl()?.value;
197         if (!preset || !activeSource || activeSource.type !== 'gimp') return;
198         const f = parseFloat(preset);
199         if (!Number.isFinite(f) || f <= 0) return;
200         if (!flags.isSimulating) saveState();
201         activeSource.frequency = f;
202         if (gimpFreqEl()) gimpFreqEl().value = String(f);
203         applyFieldsToComponent();
204     });
205     gsinAmpEl()?.addEventListener('change', onChange);
206     gsinFreqEl()?.addEventListener('change', onChange);
207     gsinOffsetEl()?.addEventListener('change', onChange);
208     gsqrAmpEl()?.addEventListener('change', onChange);
209     gsqrFreqEl()?.addEventListener('change', onChange);
210     gsqrOffsetEl()?.addEventListener('change', onChange);
211
212     gimpVoltageEl()?.addEventListener('input', apply);
213     gimpFreqEl()?.addEventListener('input', apply);
214     gimpDutyEl()?.addEventListener('input', apply);
215     gsinAmpEl()?.addEventListener('input', apply);
216     gsinFreqEl()?.addEventListener('input', apply);
217     gsinOffsetEl()?.addEventListener('input', apply);
218     gsqrAmpEl()?.addEventListener('input', apply);
219     gsqrFreqEl()?.addEventListener('input', apply);
220     gsqrOffsetEl()?.addEventListener('input', apply);
221 }
222
223 export function initGimpPanel() {
224     initSourcePanel();
225 }
226

```



```

1 // speaker-audio.js – restitution carte son à partir des courbes .tran du haut-parleur
2 import { circuit } from './state.js';
3 import { reachableJunctions } from './Engine/hc90-cascade.mjs';
4
5 /** Tension crête de référence du circuit (plein volume numérique). */
6 export const SPEAKER_MAX_CIRCUIT_VOLTS = 5;
7
8 const AUDIO_HEADROOM = 0.88;
9 const TARGET_SAMPLE_RATE = 44100;
10 const FADE_IN_MS = 30;
11 const FADE_OUT_MS = 25;
12
13 const AC_SOURCE_TYPES = new Set(['gsin', 'gsqr', 'gimp']);
14
15 let audioCtx = null;
16 /** @type {AudioBufferSourceNode|null} */
17 let sourceNode = null;
18 /** @type {OscillatorNode|null} */
19 let oscNode = null;
20 let gainNode = null;
21 let filterNode = null;
22 let playing = false;
23 let animRaf = null;
24 let lastPlaySig = '';
25 let redraw = () => {};
26
27 export function bindSpeakerAudioRedraw(fn) {
28   redraw = typeof fn === 'function' ? fn : () => {};
29 }
30
31 function ensureAudioContext() {
32   if (!audioCtx) audioCtx = new (window.AudioContext || window.webkitAudioContext)();
33   if (audioCtx.state === 'suspended') audioCtx.resume().catch(() => {});
34   return audioCtx;
35 }
36
37 function speakerNet() {
38   const sp = circuit.components.find((c) => c.type === 'speaker' && c.label);
39   if (!sp) return new Set();
40   const wires = circuit.wires || [];
41   const auto = circuit.autoJunctions || [];
42   const net = new Set([
43     ...reachableJunctions(`${sp.label}_in`, wires, auto),
44     ...reachableJunctions(`${sp.label}_out`, wires, auto),
45   ]);
46   return net;
47 }
48
49 function acSourceOnSpeakerNet(comp) {
50   if (!comp?.label) return false;
51   return speakerNet().has(`${comp.label}_out`);
52 }
53
54 function plotPeakVolts(plot) {
55   if (!plot?.voltage?.length) return 0;
56   let peak = 0;
57   for (const v of plot.voltage) {
58     const a = Math.abs(v);
59     if (a > peak) peak = a;
60   }
61   return peak;
62 }
63
64 function voltsToGain(peakVolts) {
65   if (!peakVolts > 1e-4) return AUDIO_HEADROOM * 0.5;
66   return Math.min(1, (peakVolts / SPEAKER_MAX_CIRCUIT_VOLTS) * AUDIO_HEADROOM);
67 }
68
69 function buildDriveFromComp(comp, plot) {
70   const freq = comp.frequency > 0 ? comp.frequency : 440;
71   let wave = 'sine';
72   if (comp.type === 'gsqr' || comp.type === 'gimp') wave = 'square';
73
74   let nominalPeak = 5;
75   if (comp.type === 'gsin') nominalPeak = comp.peakAmplitude ?? 5;
76   else if (comp.type === 'gsqr') nominalPeak = comp.peakAmplitude ?? 5;
77   else if (comp.type === 'gimp') nominalPeak = comp.voltageRail ?? 5;
78

```

```

79     const peak = plotPeakVolts(plot);
80     const amplitude = peak > 1e-3 ? voltsToGain(peak) : voltsToGain(nominalPeak);
81     return { wave, frequency: freq, amplitude, sourceLabel: comp.label };
82 }
83
84 /**
85  * Générateur Sin/Carré/Impulsions → OscillatorNode (pas de boucle .tran).
86  * @param {{ time?: number[]; voltage?: number[] }} plot
87  */
88 function detectOscillatorDrive(plot) {
89     const speakers = circuit.components.filter((c) => c.type === 'speaker');
90     const acSources = circuit.components.filter((c) => AC_SOURCE_TYPES.has(c.type));
91     if (!speakers.length || !acSources.length) return null;
92
93     for (const comp of acSources) {
94         if (acSourceOnSpeakerNet(comp)) return buildDriveFromComp(comp, plot);
95     }
96
97     // Montage simple : un HP + un générateur (même si fils en T non détectés)
98     if (acSources.length === 1 && speakers.length >= 1) {
99         return buildDriveFromComp(acSources[0], plot);
100     }
101
102     // Priorité au sinus si présent avec un HP
103     const gsin = acSources.find((c) => c.type === 'gsin');
104     if (gsin) return buildDriveFromComp(gsin, plot);
105
106     return null;
107 }
108
109 /** Signature stable (sans amplitude SPICE qui varie à chaque .tran). */
110 function driveSignature(drive) {
111     if (!drive) return '';
112     return `osc|${drive.wave}|${drive.frequency}|${drive.sourceLabel}`;
113 }
114
115 function interpolateVoltage(time, voltage, t) {
116     if (!time?.length || !voltage?.length) return 0;
117     if (t <= time[0]) return voltage[0];
118     if (t >= time[time.length - 1]) return voltage[voltage.length - 1];
119     for (let i = 1; i < time.length; i++) {
120         if (t <= time[i]) {
121             const t0 = time[i - 1];
122             const t1 = time[i];
123             const v0 = voltage[i - 1];
124             const v1 = voltage[i];
125             if (t1 <= t0) return v0;
126             const u = (t - t0) / (t1 - t0);
127             return v0 + u * (v1 - v0);
128         }
129     }
130     return voltage[voltage.length - 1];
131 }
132
133 function estimatePeriodSec(time, voltage) {
134     if (!time?.length || time.length < 8) return null;
135     const mean = voltage.reduce((s, v) => s + v, 0) / voltage.length;
136     const crossings = [];
137     for (let i = 1; i < voltage.length; i++) {
138         const a = voltage[i - 1] - mean;
139         const b = voltage[i] - mean;
140         if (a < 0 && b >= 0) crossings.push(time[i]);
141     }
142     if (crossings.length < 3) return null;
143     let sum = 0;
144     for (let i = 1; i < crossings.length; i++) sum += crossings[i] - crossings[i - 1];
145     return (sum / (crossings.length - 1)) * 2;
146 }
147
148 function buildPeriodicLoopBuffer(plot, sampleRate) {
149     const time = plot.time;
150     const voltage = plot.voltage;
151     const t0 = time[0];
152     const tEnd = time[time.length - 1];
153     const fullDuration = tEnd - t0;
154     if (!Number.isFinite(fullDuration) || fullDuration <= 1e-9) return null;
155
156     const period = estimatePeriodSec(time, voltage);
157     if (!(period > 1e-9)) return null;
158

```

```

159     const periodsInSpan = Math.max(2, Math.floor(fullDuration / period));
160     const loopDuration = periodsInSpan * period;
161     const loopT0 = t0 + fullDuration - loopDuration;
162     if (loopDuration <= 1e-9) return null;
163
164     const numSamples = Math.max(256, Math.round(loopDuration * sampleRate));
165     const data = new Float32Array(numSamples);
166     const scale = AUDIO_HEADROOM / SPEAKER_MAX_CIRCUIT_VOLTS;
167
168     let sum = 0;
169     for (let i = 0; i < numSamples; i++) {
170         const t = loopT0 + (i / numSamples) * loopDuration;
171         const v = interpolateVoltage(time, voltage, t);
172         const s = Number.isFinite(v) ? v * scale : 0;
173         data[i] = s;
174         sum += s;
175     }
176     const mean = sum / numSamples;
177     for (let i = 0; i < numSamples; i++) data[i] -= mean;
178
179     const fadeSamples = Math.min(Math.floor(0.01 * sampleRate), Math.floor(numSamples / 8));
180     if (fadeSamples >= 4) {
181         for (let i = 0; i < fadeSamples; i++) {
182             const w = i / fadeSamples;
183             const tail = numSamples - fadeSamples + i;
184             data[tail] = data[tail] * (1 - w) + data[i] * w;
185         }
186     }
187     data[numSamples - 1] = data[0];
188
189     for (let i = 0; i < numSamples; i++) {
190         data[i] = Math.max(-1, Math.min(1, data[i]));
191     }
192     return { data };
193 }
194
195 function plotSignature(plot) {
196     const t = plot?.time;
197     const v = plot?.voltage;
198     if (!t?.length || !v?.length) return '';
199     const n = t.length;
200     return `buf|${n}|${t[n - 1].toFixed(6)}`;
201 }
202
203 function connectOutputChain(ctx, outputNode, amplitude, useFilter) {
204     let chain = outputNode;
205     filterNode = null;
206
207     if (useFilter) {
208         filterNode = ctx.createBiquadFilter();
209         filterNode.type = 'lowpass';
210         filterNode.frequency.value = 6000;
211         filterNode.Q.value = 0.5;
212         chain.connect(filterNode);
213         chain = filterNode;
214     }
215
216     gainNode = ctx.createGain();
217     gainNode.gain.setValueAtTime(0, ctx.currentTime);
218     gainNode.gain.linearRampToValueAtTime(amplitude, ctx.currentTime + FADE_IN_MS / 1000);
219     chain.connect(gainNode);
220     gainNode.connect(ctx.destination);
221 }
222
223 function updateOscillatorLive(drive) {
224     if (!oscNode || !gainNode || !audioCtx) return false;
225     const t = audioCtx.currentTime;
226     oscNode.frequency.setTargetAtTime(drive.frequency, t, 0.02);
227     gainNode.gain.setTargetAtTime(drive.amplitude, t, 0.02);
228     return true;
229 }
230
231 function startOscillatorPlayback(ctx, drive) {
232     oscNode = ctx.createOscillator();
233     oscNode.type = drive.wave;
234     oscNode.frequency.value = drive.frequency;
235     connectOutputChain(ctx, oscNode, drive.amplitude, drive.wave === 'square');
236     oscNode.start(0);
237 }
238

```

```

239 function startBufferPlayback(ctx, plot) {
240   const pcm = buildPeriodicLoopBuffer(plot, TARGET_SAMPLE_RATE);
241   if (!pcm) return false;
242
243   const buffer = ctx.createBuffer(1, pcm.data.length, TARGET_SAMPLE_RATE);
244   buffer.getChannelData(0).set(pcm.data);
245
246   sourceNode = ctx.createBufferSource();
247   sourceNode.buffer = buffer;
248   sourceNode.loop = true;
249   sourceNode.loopStart = 0;
250   sourceNode.loopEnd = buffer.duration;
251
252   connectOutputChain(ctx, sourceNode, voltsToGain(plotPeakVolts(plot)), false);
253   sourceNode.start(0);
254   return true;
255 }
256
257 /**
258  * @param {Record<string, { time?: number[]; voltage?: number[] }>} speakerPlots
259  */
260 export function startSpeakerAudio(speakerPlots) {
261   if (!speakerPlots || typeof speakerPlots !== 'object') return;
262
263   const plots = Object.values(speakerPlots).filter(
264     (p) => p?.time?.length >= 2 && p?.voltage?.length >= 2
265   );
266   if (!plots.length) return;
267
268   const plot = plots[0];
269   const drive = detectOscillatorDrive(plot);
270
271   if (drive) {
272     const sig = driveSignature(drive);
273     if (playing && oscNode && sig === lastPlaySig) return;
274     if (playing && oscNode && updateOscillatorLive(drive)) {
275       lastPlaySig = sig;
276       return;
277     }
278   } else if (playing && sourceNode) {
279     const sig = plotSignature(plot);
280     if (sig === lastPlaySig) return;
281   }
282
283   stopSpeakerAudio({ immediate: true });
284
285   const ctx = ensureAudioContext();
286   try {
287     if (drive) {
288       startOscillatorPlayback(ctx, drive);
289       lastPlaySig = driveSignature(drive);
290     } else if (!startBufferPlayback(ctx, plot)) {
291       return;
292     } else {
293       lastPlaySig = plotSignature(plot);
294     }
295     playing = true;
296     redraw();
297     const tick = () => {
298       if (!playing) return;
299       redraw();
300       animRaf = requestAnimationFrame(tick);
301     };
302     animRaf = requestAnimationFrame(tick);
303   } catch {
304     playing = false;
305     sourceNode = null;
306     oscNode = null;
307     gainNode = null;
308     filterNode = null;
309     lastPlaySig = '';
310   }
311 }
312
313 export function stopSpeakerAudio(opts = {}) {
314   if (animRaf !== null) cancelAnimationFrame(animRaf);
315   animRaf = null;
316
317   const immediate = opts.immediate === true;
318   const ctx = audioCtx;

```

```

319     const src = sourceNode;
320     const osc = oscNode;
321     const gain = gainNode;
322     const filt = filterNode;
323
324     sourceNode = null;
325     oscNode = null;
326     gainNode = null;
327     filterNode = null;
328     playing = false;
329     lastPlaySig = '';
330
331     if (!src && !osc) return;
332
333     const finalize = () => {
334         if (src) {
335             try { src.stop(); } catch { /* */ }
336             try { src.disconnect(); } catch { /* */ }
337         }
338         if (osc) {
339             try { osc.stop(); } catch { /* */ }
340             try { osc.disconnect(); } catch { /* */ }
341         }
342         try { filt?.disconnect(); } catch { /* */ }
343         try { gain?.disconnect(); } catch { /* */ }
344     };
345
346     if (immediate || !ctx || !gain) {
347         finalize();
348         return;
349     }
350
351     const t = ctx.currentTime;
352     try {
353         gain.gain.cancelScheduledValues(t);
354         gain.gain.setValueAtTime(gain.gain.value, t);
355         gain.gain.linearRampToValueAtTime(0, t + FADE_OUT_MS / 1000);
356     } catch {
357         finalize();
358         return;
359     }
360     setTimeout(finalize, FADE_OUT_MS + 10);
361 }
362
363 export function isSpeakerAudioPlaying() {
364     return playing;
365 }
366

```

```

1 // state.js
2 // Remplacez les deux premières lignes de state.js par :
3 export const canvas = document.getElementById('circuitCanvas') || document.createElement('canvas');
4 export const ctx = canvas.getContext ? canvas.getContext('2d') : null;
5
6 export const GRID_SIZE = 20;
7 export let scale = { value: 1.0 };
8 export let pan = { x: 0, y: 0 };
9
10 export let flags = {
11     isPanning: false,
12     startX: 0,
13     startY: 0,
14     isDraggingComponent: false,
15     isSimulating: false,
16     isSelectingZone: false,
17     isShiftPressed: false,
18     isDraggingFromMenu: false
19 };
20
21 export let simulationResults = { voltmeters: {}, ammeters: {}, ohmmeters: {}, leds: {}, scopePlots: {}, bodePlots: {}, seg7: {}, bargraph: {}, logicValues: {} };
22
23 export let menuDrag = {
24     draggedComponentType: null,
25     x: 0,
26     y: 0
27 };
28
29 export let file = { handle: null };
30
31 export let counters = { battery: 0, resistor: 0, potentiometer: 0, switch_spdt: 0, push_button: 0, capacitor: 0, inductor: 0, diode: 0, npn: 0, opamp: 0, not: 0, and: 0, nand: 0, or: 0, nor: 0, xor: 0, xnor: 0, d_flipflop: 0, jk_flipflop: 0, cd4511: 0, ic_74hc90: 0, arduino_uno: 0, led: 0, seg7: 0, bargraph_dc10h: 0, grove_lcd16x2: 0, grove_dht22: 0, voltmeter: 0, ammeter: 0, ohmmeter: 0, oscilloscope: 0, bode_analyzer: 0, speaker: 0, junction: 0, gnd: 0, vcc: 0, logic_terminal: 0, gimp: 0, gsin: 0, gsqr: 0 };
32 export let circuit = {
33     components: [],
34     wires: [],
35     autoJunctions: []
36 };
37
38 export let interaction = {
39     activeWire: null,
40     hoverJunction: null,
41     hoveredComponent: null,
42     hoveredWire: null,
43     selectedWire: null,
44     selectedComponents: [],
45     selectedAutoJunctions: []
46 };
47
48 export let zone = {
49     start: { x: 0, y: 0 },
50     end: { x: 0, y: 0 }
51 };
52
53 export let clipboard = { data: null };
54 export let undoStack = [];
55 export let redoStack = [];
56
57 export const emptyDragImage = new Image();
58 emptyDragImage.src = 'data:image/gif;base64,R0lGODlhAQABAIAAAAAAAP///yH5BAEAAAAALAAAAABAAEAAAIBRAA7';
59
60 // Utilitaires de base liés à la grille
61 export function snapToGrid(val) { return Math.round(val / GRID_SIZE) * GRID_SIZE; }
62 export function toGridCoords(clientX, clientY) {
63     const rect = canvas.getBoundingClientRect();
64     return {
65         x: (clientX - rect.left - pan.x) / scale.value,
66         y: (clientY - rect.top - pan.y) / scale.value
67     };
68 }
69
70 // Système d'historique
71 export function saveState() {
72     undoStack.push(JSON.stringify({ components: circuit.components, wires: circuit.wires, autoJunctions: circuit.autoJunctions, counters }));
73     redoStack.length = 0;
74 }

```

```

1 :root {
2   --bg-color: #121212;
3   --panel-bg: #1e1e1e;
4   --accent-color: #ff5722;
5   --text-color: #ffffff;
6   --text-muted: #b8b8b8;
7   --border-color: #333333;
8   --canvas-bg: #121212;
9   --sim-run: #0d9668;
10  --sim-run-hover: #0a7d56;
11  --sim-stop: #c0392b;
12  --sim-stop-hover: #a93226;
13 }
14
15 body.theme-light {
16   --bg-color: #f0f0f0;
17   --panel-bg: #ffffff;
18   --text-color: #1a1a1a;
19   --text-muted: #333333;
20   --border-color: #cccccc;
21   --canvas-bg: #ffffff;
22 }
23
24 body.theme-light .dropdown-item:hover {
25   color: var(--text-color);
26 }
27
28 body.theme-light .modal-content {
29   color: var(--text-color);
30 }
31
32 body.theme-light .close-modal:hover {
33   color: var(--text-color);
34 }
35
36 * {
37   box-sizing: border-box;
38   margin: 0;
39   padding: 0;
40   user-select: none;
41 }
42
43 input,
44 textarea,
45 select,
46 [contenteditable="true"] {
47   user-select: text;
48   -webkit-user-select: text;
49 }
50
51 body {
52   font-family: "Segoe UI", system-ui, -apple-system, sans-serif;
53   background-color: var(--bg-color);
54   color: var(--text-color);
55   overflow: hidden;
56   height: 100vh;
57   display: flex;
58   flex-direction: column;
59   -webkit-font-smoothing: antialiased;
60 }
61
62 .navbar {
63   position: fixed;
64   top: 0;
65   left: 0;
66   right: 0;
67   height: 48px;
68   flex-shrink: 0;
69   background-color: var(--panel-bg);
70   border-bottom: 1px solid var(--border-color);
71   display: flex;
72   align-items: center;
73   padding: 0 12px;
74   z-index: 100000;
75   box-shadow: 0 1px 0 rgba(0, 0, 0, 0.35);
76   pointer-events: auto;
77 }
78

```

```

79 .app-root {
80     margin-top: 48px;
81     flex: 1;
82     min-height: 0;
83     display: flex;
84     flex-direction: column;
85     overflow: hidden;
86 }
87
88 .menu-item {
89     position: relative;
90     padding: 10px 14px;
91     cursor: pointer;
92     display: flex;
93     align-items: center;
94     gap: 8px;
95     font-size: 13px;
96     font-weight: 600;
97     letter-spacing: 0.01em;
98     transition: background 0.15s;
99 }
100
101 .menu-item:hover {
102     background-color: var(--border-color);
103 }
104
105 .menu-icon {
106     font-size: 14px;
107     opacity: 0.85;
108 }
109
110 .menu-icon--sources::before {
111     content: "⚡";
112     font-size: 15px;
113 }
114
115 .simulation-controls {
116     margin-left: auto;
117     display: flex;
118     align-items: center;
119     gap: 12px;
120     min-width: 0;
121 }
122
123 .circuit-name {
124     font-size: 13px;
125     font-weight: 600;
126     color: var(--text-muted);
127     max-width: min(280px, 28vw);
128     overflow: hidden;
129     text-overflow: ellipsis;
130     white-space: nowrap;
131     padding-right: 12px;
132     border-right: 1px solid var(--border-color);
133     flex-shrink: 1;
134 }
135
136 body.theme-light .circuit-name {
137     color: #555;
138 }
139
140 .simulation-controls button {
141     border: 1px solid transparent;
142     padding: 7px 16px;
143     font-size: 12px;
144     font-weight: 600;
145     letter-spacing: 0.02em;
146     border-radius: 6px;
147     cursor: pointer;
148     color: white;
149     transition: background 0.15s, border-color 0.15s, opacity 0.15s;
150 }
151
152 #btn-simulate {
153     background-color: var(--sim-run);
154 }
155
156 #btn-simulate:hover {
157     background-color: var(--sim-run-hover);
158 }

```



```
159
160 #btn-stop {
161     background-color: var(--sim-stop);
162 }
163
164 #btn-stop:hover:not(.disabled) {
165     background-color: var(--sim-stop-hover);
166 }
167
168 .disabled {
169     background-color: #444444 !important;
170     color: #888888 !important;
171     cursor: not-allowed !important;
172     opacity: 0.6;
173 }
174
175 .dropdown {
176     display: none;
177     position: absolute;
178     top: 100%;
179     left: 0;
180     background-color: var(--panel-bg);
181     border: 1px solid var(--border-color);
182     box-shadow: 0 8px 24px rgba(0, 0, 0, 0.45);
183     z-index: 501;
184     min-width: 200px;
185     border-radius: 0 0 8px 8px;
186     padding: 4px 0;
187 }
188
189 .dropdown-item {
190     color: var(--text-muted);
191     padding: 9px 16px;
192     text-decoration: none;
193     display: block;
194     font-size: 13px;
195     font-weight: 500;
196     white-space: nowrap;
197 }
198
199 .dropdown-item:hover {
200     background-color: var(--border-color);
201     color: white;
202 }
203
204 .menu-item:hover > .dropdown,
205 .menu-item.open > .dropdown {
206     display: block;
207 }
208
209 .dropdown-submenu {
210     position: relative;
211 }
212
213 .dropdown-submenu > .submenu {
214     display: none;
215     position: absolute;
216     left: 100%;
217     top: 0;
218     min-width: 120px;
219     border-radius: 4px;
220     background-color: var(--panel-bg);
221     border: 2px solid var(--border-color);
222     box-shadow: 0px 8px 16px 0px rgba(0,0,0,0.5);
223     z-index: 2;
224 }
225
226 .dropdown-submenu.open > .submenu {
227     display: block;
228 }
229
230 .submenu-title {
231     cursor: pointer;
232     color: var(--text-muted);
233     user-select: none;
234 }
235
236 .submenu .dropdown-item {
237     padding: 10px 14px;
238     text-align: left;
```

```

239     min-width: 0;
240 }
241
242 .submenu .dropdown-item::before {
243     content: none !important;
244 }
245
246 .dropdown-submenu > .submenu-title::after {
247     content: '▸';
248     float: right;
249     opacity: 0.55;
250     margin-left: 12px;
251 }
252
253 /* Menus avec sous-menus : texte seul, aligné */
254 .menu-item:has(.dropdown-submenu) > .dropdown > .dropdown-item,
255 .submenu .dropdown-item {
256     padding-left: 14px;
257     padding-right: 14px;
258 }
259
260 .panel-field-range {
261     flex-wrap: wrap;
262     align-items: center;
263     gap: 8px;
264 }
265
266 .panel-field-range input[type="range"] {
267     flex: 1;
268     min-width: 120px;
269     accent-color: #00bcd4;
270 }
271
272 .panel-field-range output {
273     min-width: 56px;
274     font-size: 12px;
275     color: var(--text-muted);
276     text-align: right;
277 }
278
279 .canvas-container {
280     flex: 1;
281     min-width: 0;
282     min-height: 0;
283     position: relative;
284     background-color: var(--bg-color);
285     overflow: hidden;
286 }
287
288 #circuitCanvas {
289     display: block;
290     width: 100%;
291     height: 100%;
292     background-color: var(--canvas-bg);
293 }
294
295 .sim-workspace {
296     display: flex;
297     flex: 1;
298     min-height: 0;
299     overflow: hidden;
300 }
301
302 .right-panel {
303     width: min(420px, 42vw);
304     flex-shrink: 0;
305     display: flex;
306     flex-direction: column;
307     background-color: var(--panel-bg);
308     border-left: 2px solid var(--border-color);
309     z-index: 20;
310     min-height: 0;
311 }
312
313 .right-panel.hidden {
314     display: none;
315 }
316
317 .panel-header--arduino {
318     color: #00979d;

```

```
319 }
320
321 .arduino-panel-header-actions {
322     display: flex;
323     align-items: center;
324     gap: 6px;
325 }
326
327 .arduino-panel-body {
328     flex: 1;
329     display: flex;
330     flex-direction: column;
331     padding: 12px;
332     min-height: 0;
333     overflow-y: auto;
334     overflow-x: hidden;
335 }
336
337 .arduino-editor-label {
338     display: block;
339     font-size: 12px;
340     font-weight: 700;
341     color: #00979d;
342     margin-bottom: 6px;
343 }
344
345 .arduino-editor-section {
346     flex: 1 1 280px;
347     display: flex;
348     flex-direction: column;
349     min-height: 280px;
350     margin-bottom: 10px;
351 }
352
353 .arduino-sketch-wrap {
354     flex: 1;
355     min-height: 240px;
356     overflow: auto;
357     border: 1px solid var(--border-color);
358     border-radius: 4px;
359     background: var(--canvas-bg);
360     scrollbar-gutter: stable;
361 }
362
363 .arduino-sketch-grid {
364     display: grid;
365     min-height: 100%;
366 }
367
368 .arduino-sketch-highlight,
369 .arduino-sketch-input {
370     grid-area: 1 / 1;
371     box-sizing: border-box;
372     width: 100%;
373     min-height: 240px;
374     margin: 0;
375     padding: 10px;
376     border: none;
377     border-radius: 0;
378     font-family: Consolas, "Courier New", monospace;
379     font-size: 13px;
380     font-weight: normal;
381     font-style: normal;
382     line-height: 1.45;
383     tab-size: 4;
384     -moz-tab-size: 4;
385     white-space: pre;
386     word-wrap: normal;
387     overflow-wrap: normal;
388     word-break: normal;
389     letter-spacing: normal;
390     font-variant-ligatures: none;
391 }
392
393 .arduino-sketch-highlight {
394     z-index: 0;
395     pointer-events: none;
396     overflow: visible;
397     background: transparent;
398     color: var(--text-color, #d4d4d4);
```

```
399 }
400
401 .arduino-sketch-input {
402     z-index: 1;
403     resize: none;
404     overflow: hidden;
405     background: transparent;
406     color: transparent;
407     -webkit-text-fill-color: transparent;
408     caret-color: var(--text-color, #d4d4d4);
409     user-select: text;
410     -webkit-user-select: text;
411     outline: none;
412 }
413
414 .arduino-sketch-input:focus {
415     outline: none;
416 }
417
418 .arduino-sketch-input::selection {
419     background: rgba(0, 151, 157, 0.35);
420     color: transparent;
421     -webkit-text-fill-color: transparent;
422 }
423
424 .ard-hl-kw { color: #569cd6; }
425 .ard-hl-type { color: #4ec9b0; }
426 .ard-hl-fn { color: #dcdcaa; }
427 .ard-hl-str { color: #ce9178; }
428 .ard-hl-cmt { color: #6a9955; }
429 .ard-hl-pre { color: #9b6e4a; }
430 .ard-hl-lib { color: #6a9fb5; }
431 .ard-hl-num { color: #b5cea8; }
432 .ard-hl-macro { color: #c586c0; }
433
434 .dropdown-item--action {
435     cursor: pointer;
436     font-weight: 600;
437 }
438
439 .dropdown-item--action:hover {
440     background: rgba(0, 151, 157, 0.2);
441 }
442
443 .serial-monitor-floating {
444     width: min(520px, calc(100vw - 32px));
445     z-index: 100010;
446 }
447
448 .serial-monitor-floating .scope-floating-header {
449     touch-action: none;
450 }
451
452 .serial-monitor-toolbar {
453     display: flex;
454     align-items: center;
455     gap: 10px;
456     margin-bottom: 8px;
457     flex-wrap: wrap;
458     font-size: 12px;
459 }
460
461 .serial-monitor-board-label {
462     display: flex;
463     align-items: center;
464     gap: 6px;
465 }
466
467 .serial-monitor-board-select {
468     font-size: 12px;
469     padding: 2px 6px;
470     border-radius: 4px;
471     border: 1px solid var(--border-color);
472     background: var(--canvas-bg);
473     color: inherit;
474 }
475
476 .serial-monitor-baud {
477     color: var(--text-muted, #8899aa);
478     font-family: Consolas, monospace;
```

```
479 }
480
481 .serial-monitor-output {
482     display: block;
483     width: 100%;
484     box-sizing: border-box;
485     min-height: 200px;
486     max-height: 280px;
487     overflow: auto;
488     margin: 0 0 8px;
489     padding: 10px;
490     font-family: Consolas, "Courier New", monospace;
491     font-size: 12px;
492     line-height: 1.4;
493     background: #0a0a0a;
494     color: #c5e8c5;
495     border: 1px solid #1a3a1a;
496     border-radius: 4px;
497     white-space: pre-wrap;
498     word-break: break-word;
499 }
500
501 .serial-monitor-status {
502     font-size: 11px;
503     color: var(--text-muted, #8899aa);
504     margin: 0 0 8px;
505     min-height: 1.2em;
506 }
507
508 .serial-monitor-input-row {
509     display: flex;
510     gap: 8px;
511 }
512
513 .serial-monitor-input {
514     flex: 1;
515     font-family: Consolas, monospace;
516     font-size: 12px;
517     padding: 6px 8px;
518     border: 1px solid var(--border-color);
519     border-radius: 4px;
520     background: var(--canvas-bg);
521     color: inherit;
522 }
523
524 .arduino-panel-actions {
525     display: flex;
526     align-items: center;
527     gap: 8px;
528     margin-bottom: 8px;
529     flex-shrink: 0;
530     flex-wrap: wrap;
531 }
532
533 .arduino-compile-status {
534     font-size: 0.85rem;
535     font-weight: 600;
536     margin-left: auto;
537 }
538
539 .arduino-compile-status--ok {
540     color: #76ff03;
541 }
542
543 .arduino-compile-status--busy {
544     color: var(--text-muted);
545 }
546
547 .arduino-btn--errors {
548     margin-left: auto;
549     background: #c62828;
550     border-color: #8e0000;
551     color: #fff;
552 }
553
554 .arduino-compile-log {
555     flex: 0 1 auto;
556     min-height: 60px;
557     max-height: 160px;
558     overflow: auto;
```

```
559 font-family: Consolas, monospace;
560 font-size: 11px;
561 padding: 8px;
562 background: #0a0a0a;
563 border: 1px solid var(--border-color);
564 border-radius: 4px;
565 white-space: pre-wrap;
566 margin: 0;
567 }
568
569 .arduino-compile-log.hidden {
570     display: none;
571 }
572
573 .modal-content--prompt {
574     max-width: 420px;
575     width: 90%;
576 }
577
578 .value-prompt-label {
579     display: block;
580     margin-bottom: 12px;
581     font-size: 1rem;
582     line-height: 1.4;
583 }
584
585 .value-prompt-input {
586     width: 100%;
587     box-sizing: border-box;
588     padding: 8px 10px;
589     margin-bottom: 14px;
590     border: 1px solid var(--border-color);
591     border-radius: 4px;
592     background: var(--canvas-bg);
593     color: inherit;
594     font-size: 1rem;
595 }
596
597 .value-prompt-actions {
598     display: flex;
599     gap: 10px;
600     justify-content: flex-end;
601 }
602
603 .value-prompt-btn {
604     padding: 8px 16px;
605     border: 1px solid var(--border-color);
606     border-radius: 4px;
607     background: var(--panel-bg);
608     color: inherit;
609     cursor: pointer;
610 }
611
612 .value-prompt-btn--ok {
613     background: #2563eb;
614     border-color: #1d4ed8;
615     color: #fff;
616 }
617
618 .value-prompt-btn:hover {
619     filter: brightness(1.08);
620 }
621
622 .modal-content--arduino {
623     max-width: 720px;
624     width: 92%;
625 }
626
627 .arduino-cli-status {
628     font-size: 0.85rem;
629     margin: 0 0 10px;
630     padding: 6px 8px;
631     border-radius: 4px;
632     background: rgba(255, 255, 255, 0.06);
633 }
634
635 .arduino-cli-status--ok { color: #76ff03; }
636 .arduino-cli-status--warn { color: #ffb74d; }
637
638 .arduino-lib-hint {
```

```
639     font-size: 11px;
640     line-height: 1.35;
641     color: var(--text-muted);
642     margin: 0 0 8px;
643 }
644
645 .arduino-lib-hint code {
646     font-size: 10px;
647     color: #80cbc4;
648 }
649
650 .arduino-lib-details {
651     flex-shrink: 0;
652     margin-top: 8px;
653     border: 1px solid var(--border-color);
654     border-radius: 6px;
655     background: rgba(0, 0, 0, 0.12);
656 }
657
658 .arduino-lib-summary {
659     cursor: pointer;
660     padding: 8px 10px;
661     font-size: 12px;
662     font-weight: 700;
663     color: #00979d;
664     list-style: none;
665 }
666
667 .arduino-lib-summary::-webkit-details-marker {
668     display: none;
669 }
670
671 .arduino-lib-details[open] .arduino-lib-summary {
672     border-bottom: 1px solid var(--border-color);
673 }
674
675 .arduino-lib-section {
676     flex-shrink: 0;
677     margin-bottom: 0;
678     padding: 8px 10px 10px;
679     border: none;
680     border-radius: 0;
681     max-height: none;
682     overflow: visible;
683     background: transparent;
684 }
685
686 .arduino-lib-header {
687     display: flex;
688     align-items: center;
689     justify-content: space-between;
690     margin-bottom: 6px;
691 }
692
693 .arduino-lib-title {
694     font-size: 12px;
695     font-weight: 700;
696     color: #00979d;
697 }
698
699 .arduino-lib-search-row {
700     display: flex;
701     gap: 6px;
702     margin-bottom: 6px;
703 }
704
705 .arduino-lib-search-input {
706     flex: 1;
707     min-width: 0;
708     padding: 5px 8px;
709     font-size: 12px;
710     border: 1px solid var(--border-color);
711     border-radius: 4px;
712     background: var(--canvas-bg);
713     color: inherit;
714 }
715
716 .arduino-lib-subtitle {
717     font-size: 11px;
718     font-weight: 600;
```

```
719     color: var(--text-muted);
720     margin: 8px 0 4px;
721 }
722
723 .arduino-lib-list {
724     list-style: none;
725     margin: 0;
726     padding: 0;
727 }
728
729 .arduino-lib-list:empty {
730     display: none;
731 }
732
733 .arduino-lib-item {
734     display: flex;
735     align-items: flex-start;
736     gap: 6px;
737     padding: 5px 0;
738     border-bottom: 1px solid rgba(255, 255, 255, 0.06);
739     font-size: 11px;
740     line-height: 1.3;
741 }
742
743 .arduino-lib-item:last-child {
744     border-bottom: none;
745 }
746
747 .arduino-lib-item-info {
748     flex: 1;
749     min-width: 0;
750 }
751
752 .arduino-lib-item-name {
753     font-weight: 600;
754     color: #80deea;
755     background: none;
756     border: none;
757     padding: 0;
758     margin: 0;
759     font: inherit;
760     text-align: left;
761     cursor: pointer;
762     text-decoration: underline;
763     text-underline-offset: 2px;
764 }
765
766 .arduino-lib-item-name:hover {
767     color: #b2ebf2;
768 }
769
770 .arduino-lib-floating {
771     width: min(520px, calc(100vw - 24px));
772     max-height: min(85vh, 640px);
773 }
774
775 .arduino-lib-popup-body {
776     overflow: auto;
777     max-height: min(70vh, 520px);
778     padding-right: 4px;
779     font-size: 13px;
780     line-height: 1.45;
781     color: #cfd8dc;
782 }
783
784 .arduino-lib-doc-note {
785     margin: 0 0 12px;
786     color: #90a4ae;
787 }
788
789 .arduino-lib-doc-code {
790     margin: 0 0 12px;
791     padding: 8px 10px;
792     background: #0a1218;
793     border: 1px solid #1a3a4a;
794     border-radius: 4px;
795     font-size: 12px;
796     color: #a5d6a7;
797     overflow-x: auto;
798 }
```



```
799
800 .arduino-lib-doc-table {
801     width: 100%;
802     border-collapse: collapse;
803     margin-bottom: 10px;
804     font-size: 12px;
805 }
806
807 .arduino-lib-doc-table th,
808 .arduino-lib-doc-table td {
809     border: 1px solid #2a3a4a;
810     padding: 6px 8px;
811     vertical-align: top;
812     text-align: left;
813 }
814
815 .arduino-lib-doc-table th {
816     background: #0f1a22;
817     color: #b0bec5;
818 }
819
820 .arduino-lib-doc-sig {
821     display: block;
822     margin-bottom: 4px;
823     color: #ffeb3b;
824     white-space: pre-wrap;
825     word-break: break-word;
826 }
827
828 .arduino-lib-doc-badge {
829     display: inline-block;
830     font-size: 10px;
831     padding: 1px 6px;
832     border-radius: 3px;
833     background: #37474f;
834     color: #b0bec5;
835     vertical-align: middle;
836 }
837
838 .arduino-lib-doc-badge--sim {
839     background: #1b5e20;
840     color: #c8e6c9;
841 }
842
843 .arduino-lib-doc-legend {
844     margin: 8px 0 0;
845     font-size: 11px;
846     color: #78909c;
847 }
848
849 .arduino-lib-item-meta {
850     color: var(--text-muted);
851     font-size: 10px;
852 }
853
854 .arduino-lib-msg {
855     font-size: 10px;
856     color: var(--text-muted);
857     margin: 6px 0 0;
858     min-height: 14px;
859 }
860
861 .arduino-lib-msg--ok { color: #76ff03; }
862 .arduino-lib-msg--err { color: #ffb74d; }
863
864 .arduino-btn--small {
865     padding: 4px 8px;
866     font-size: 11px;
867     flex-shrink: 0;
868 }
869
870 .arduino-btn--install {
871     background: #00979d;
872     border-color: #006064;
873     color: #fff;
874 }
875
876 .arduino-btn--uninstall {
877     background: transparent;
878     border-color: var(--border-color);
```

```
879     color: var(--text-muted);
880 }
881
882
883 .arduino-modal-actions {
884     display: flex;
885     flex-wrap: wrap;
886     gap: 8px;
887     margin-bottom: 10px;
888 }
889
890 .arduino-btn {
891     padding: 8px 14px;
892     border: 1px solid var(--border-color);
893     border-radius: 4px;
894     background: var(--panel-bg);
895     color: inherit;
896     cursor: pointer;
897 }
898
899 .arduino-btn--primary {
900     background: #00979d;
901     border-color: #006064;
902     color: #fff;
903 }
904
905
906 .modal {
907     display: none !important;
908     position: fixed;
909     z-index: 100001;
910     left: 0;
911     top: 0;
912     width: 100%;
913     height: 100%;
914     background-color: rgba(0,0,0,0.7);
915     pointer-events: none !important;
916     visibility: hidden;
917 }
918
919 .modal.is-open {
920     display: block !important;
921     pointer-events: auto !important;
922     visibility: visible;
923 }
924
925 .modal-content {
926     background-color: var(--panel-bg);
927     margin: 10% auto;
928     padding: 20px;
929     border: 2px solid var(--border-color);
930     width: 50%;
931     max-width: 500px;
932     border-radius: 6px;
933     color: white;
934 }
935
936 .modal-content--doc {
937     max-width: 640px;
938     width: 92%;
939     max-height: 85vh;
940     overflow-y: auto;
941 }
942
943 .modal-body--doc h3 {
944     margin: 16px 0 8px;
945     font-size: 14px;
946     color: #ce93d8;
947 }
948
949 .modal-body--doc .doc-list {
950     margin: 0 0 10px 18px;
951     font-size: 13px;
952     line-height: 1.45;
953 }
954
955 .modal-body--doc .doc-hint {
956     margin-top: 14px;
957     font-size: 12px;
958     color: #888;
```

```
959     border-bottom: none;
960 }
961
962 .doc-pin-table {
963     width: 100%;
964     border-collapse: collapse;
965     font-size: 12px;
966     margin-bottom: 8px;
967 }
968
969 .doc-pin-table th,
970 .doc-pin-table td {
971     border: 1px solid #3a3a3a;
972     padding: 6px 8px;
973     text-align: left;
974 }
975
976 .doc-pin-table th {
977     background: #2a2a2a;
978     color: #ce93d8;
979 }
980
981 .modal-body--doc code {
982     font-size: 11px;
983     color: #80cbc4;
984 }
985
986 .bargraph-color-picker {
987     display: flex;
988     flex-wrap: wrap;
989     gap: 10px;
990     margin: 8px 0 16px;
991 }
992
993 .bargraph-color-btn {
994     display: flex;
995     align-items: center;
996     gap: 6px;
997     padding: 6px 12px;
998     border: 2px solid #444;
999     border-radius: 6px;
1000     background: #1e1e1e;
1001     color: #ddd;
1002     cursor: pointer;
1003     font-size: 13px;
1004 }
1005
1006 .bargraph-color-btn:hover {
1007     border-color: #888;
1008 }
1009
1010 .bargraph-color-btn.is-active {
1011     border-color: #80cbc4;
1012     box-shadow: 0 0 1px #80cbc4;
1013 }
1014
1015 .bargraph-color-swatch {
1016     width: 18px;
1017     height: 18px;
1018     border-radius: 3px;
1019     border: 1px solid rgba(255, 255, 255, 0.3);
1020 }
1021
1022 .close-modal {
1023     color: #aaa;
1024     float: right;
1025     font-size: 28px;
1026     font-weight: bold;
1027     cursor: pointer;
1028 }
1029
1030 .close-modal:hover {
1031     color: white;
1032 }
1033
1034 .modal-body {
1035     margin-top: 15px;
1036 }
1037
1038 .modal.hidden {
```

```

1039     display: none !important;
1040 }
1041
1042 .scope-floating {
1043     position: fixed;
1044     left: 120px;
1045     top: 70px;
1046     z-index: 100010;
1047     background-color: var(--panel-bg);
1048     border: 2px solid var(--border-color);
1049     border-radius: 8px;
1050     padding: 0 12px 12px;
1051     box-shadow: 0 8px 28px rgba(0, 0, 0, 0.55);
1052     pointer-events: auto;
1053 }
1054
1055 .scope-floating.hidden {
1056     display: none !important;
1057 }
1058
1059 .scope-floating-header {
1060     display: flex;
1061     align-items: center;
1062     justify-content: space-between;
1063     padding: 8px 0 10px;
1064     color: #fff;
1065     font-weight: bold;
1066     font-size: 14px;
1067     cursor: move;
1068     user-select: none;
1069 }
1070
1071 .scope-floating-header .panel-close {
1072     cursor: pointer;
1073     flex-shrink: 0;
1074 }
1075
1076 #scope-popup-canvas,
1077 #bode-popup-canvas {
1078     display: block;
1079     background: #050a0e;
1080     border: 1px solid #1a3a4a;
1081     border-radius: 4px;
1082     max-width: min(640px, calc(100vw - 40px));
1083     height: auto;
1084 }
1085
1086 #scope-popup-canvas {
1087     cursor: grab;
1088 }
1089
1090 #scope-popup-canvas:active {
1091     cursor: grabbing;
1092 }
1093
1094 .bode-popup-info {
1095     padding: 4px 12px 8px;
1096     font-size: 12px;
1097     color: #ff9800;
1098     min-height: 1.2em;
1099 }
1100
1101 .modal-body p {
1102     margin-bottom: 10px;
1103     font-size: 13px;
1104     line-height: 1.4;
1105     border-bottom: 1px solid #2a2a2a;
1106     padding-bottom: 5px;
1107 }
1108
1109 /* Panneau bas – réglages Gimp */
1110 .bottom-panel {
1111     background-color: var(--panel-bg);
1112     border-top: 2px solid var(--border-color);
1113     padding: 0;
1114     z-index: 20;
1115     flex-shrink: 0;
1116 }
1117
1118 .bottom-panel.hidden,

```

```
1119 .hidden {
1120     display: none;
1121 }
1122
1123 .panel-header {
1124     display: flex;
1125     align-items: center;
1126     justify-content: space-between;
1127     padding: 8px 16px;
1128     border-bottom: 1px solid var(--border-color);
1129     font-size: 14px;
1130     font-weight: bold;
1131     color: #00bcd4;
1132 }
1133
1134 .panel-close {
1135     background: none;
1136     border: none;
1137     color: #aaa;
1138     font-size: 22px;
1139     cursor: pointer;
1140     line-height: 1;
1141     padding: 0 4px;
1142 }
1143
1144 .panel-close:hover {
1145     color: #fff;
1146 }
1147
1148 .panel-body {
1149     display: flex;
1150     flex-wrap: wrap;
1151     gap: 20px 32px;
1152     padding: 12px 16px 14px;
1153     align-items: center;
1154 }
1155
1156 .panel-field {
1157     display: flex;
1158     align-items: center;
1159     gap: 10px;
1160     font-size: 13px;
1161     color: var(--text-muted);
1162 }
1163
1164 .panel-field span:first-child {
1165     min-width: 110px;
1166     color: #bbb;
1167 }
1168
1169 .panel-field select,
1170 .panel-field input[type="number"] {
1171     background: #2a2a2a;
1172     border: 1px solid #444;
1173     color: #fff;
1174     padding: 5px 10px;
1175     border-radius: 4px;
1176     font-size: 13px;
1177     width: 120px;
1178 }
1179
1180 .panel-field input[type="number"] {
1181     width: 100px;
1182 }
1183
1184 .panel-unit {
1185     color: #888;
1186     font-size: 12px;
1187 }
1188
1189 .panel-action-btn {
1190     margin-left: 12px;
1191     padding: 6px 14px;
1192     background: #006978;
1193     color: #fff;
1194     border: 1px solid #00bcd4;
1195     border-radius: 4px;
1196     font-size: 13px;
1197     cursor: pointer;
1198     align-self: center;
```

```
1199 }  
1200  
1201 .panel-action-btn:hover {  
1202     background: #00838f;  
1203 }
```

```

1  /** Thème de l'éditeur de schéma (sombre / clair). */
2
3  const PALETTES = {
4    dark: {
5      canvasBg: '#121212',
6      grid: '#262626',
7      ink: '#ffffff',
8      inkMuted: '#aaaaaa',
9      inkDim: '#9e9e9e',
10     componentFill: '#1e1e1e',
11     componentStroke: '#bdbdbd',
12     strokeLight: '#cccccc',
13     strokeMuted: '#666666',
14     opampFill: '#d8d8d8',
15     junctionStroke: '#ffffff',
16     meterFill: '#2a3b4c',
17     meterDisplayBg: '#0d1b1e',
18     scopeBg: '#050a0e',
19     scopeGrid: '#1a3a4a',
20     scopeLabel: '#3a5a6a',
21     wireDefault: '#00fffa',
22     ledSmoke: 'rgba(170, 170, 170, 0.75)',
23   },
24   light: {
25     canvasBg: '#ffffff',
26     grid: '#c8c8c8',
27     ink: '#1a1a1a',
28     inkMuted: '#555555',
29     inkDim: '#666666',
30     componentFill: '#ffffff',
31     componentStroke: '#333333',
32     strokeLight: '#333333',
33     strokeMuted: '#555555',
34     opampFill: '#e8e8e8',
35     junctionStroke: '#1a1a1a',
36     meterFill: '#e8eef2',
37     meterDisplayBg: '#e8f5e9',
38     scopeBg: '#f5f5f5',
39     scopeGrid: '#bbbbbb',
40     scopeLabel: '#666666',
41     wireDefault: '#007755',
42     ledSmoke: 'rgba(80, 80, 80, 0.75)',
43   },
44 };
45
46 export const COLORS = { ...PALETTES.dark };
47 export let editorTheme = 'dark';
48
49 function applyCssTheme() {
50   document.body.classList.toggle('theme-light', editorTheme === 'light');
51   document.body.classList.toggle('theme-dark', editorTheme === 'dark');
52   document.documentElement.style.setProperty('--canvas-bg', COLORS.canvasBg);
53 }
54
55 function updateThemeMenuMarks() {
56   const darkBtn = document.getElementById('btn-theme-dark');
57   const lightBtn = document.getElementById('btn-theme-light');
58   if (darkBtn) darkBtn.textContent = editorTheme === 'dark' ? 'Sombre ✓' : 'Sombre';
59   if (lightBtn) lightBtn.textContent = editorTheme === 'light' ? 'Clair ✓' : 'Clair';
60 }
61
62 export function setEditorTheme(theme) {
63   editorTheme = theme === 'light' ? 'light' : 'dark';
64   Object.assign(COLORS, PALETTES[editorTheme]);
65   try { localStorage.setItem('sim-editor-theme', editorTheme); } catch (_) { /* ignore */ }
66   applyCssTheme();
67   updateThemeMenuMarks();
68 }
69
70 export function initEditorTheme() {
71   let saved = 'dark';
72   try { saved = localStorage.getItem('sim-editor-theme') || 'dark'; } catch (_) { /* ignore */ }
73   setEditorTheme(saved === 'light' ? 'light' : 'dark');
74 }
75

```

```

1  /** Boîte de saisie modale (remplace window.prompt, compatible Electron). */
2  import { showModal, hideModal } from './modal-ui.js';
3
4  /** @type {(value: string | null) => void} | null */
5  let pendingResolve = null;
6
7  function getModal() {
8      return document.getElementById('value-prompt-modal');
9  }
10
11  /**
12   * @param {string} message
13   * @param {string} [defaultValue]
14   * @returns {Promise<string | null>}
15   */
16  export function showValuePrompt(message, defaultValue = '') {
17      return new Promise((resolve) => {
18          const modal = getModal();
19          const input = document.getElementById('value-prompt-input');
20          const label = document.getElementById('value-prompt-message');
21          if (!modal || !input || !label) {
22              resolve(typeof window.prompt === 'function' ? window.prompt(message, defaultValue) : null);
23              return;
24          }
25          if (pendingResolve) pendingResolve(null);
26          pendingResolve = resolve;
27          label.textContent = message;
28          input.value = defaultValue ?? '';
29          showModal(modal);
30          input.focus();
31          input.select();
32      });
33  }
34
35  export function initValuePrompt() {
36      const modal = getModal();
37      const input = document.getElementById('value-prompt-input');
38      const okBtn = document.getElementById('value-prompt-ok');
39      const cancelBtn = document.getElementById('value-prompt-cancel');
40      const closeBtn = document.getElementById('close-value-prompt');
41
42      function finish(value) {
43          if (!pendingResolve) return;
44          const resolve = pendingResolve;
45          pendingResolve = null;
46          hideModal(modal);
47          resolve(value);
48      }
49
50      okBtn?.addEventListener('click', () => finish(input?.value ?? null));
51      cancelBtn?.addEventListener('click', () => finish(null));
52      closeBtn?.addEventListener('click', () => finish(null));
53      input?.addEventListener('keydown', (e) => {
54          if (e.key === 'Enter') {
55              e.preventDefault();
56              finish(input.value);
57          }
58          if (e.key === 'Escape') {
59              e.preventDefault();
60              finish(null);
61          }
62      });
63      modal?.addEventListener('click', (e) => {
64          if (e.target === modal) finish(null);
65      });
66  }
67

```



```

1 "use strict";
2
3 const { app, BrowserWindow, shell } = require("electron");
4 const path = require("path");
5
6 const DEFAULT_PORT = 43721;
7 let mainWindow = null;
8 /** @type {import("http").Server | null} */
9 let httpServer = null;
10
11 function isDevMode() {
12     return !app.isPackaged;
13 }
14
15 /** Racine contenant Simulateur/ et tools/ (dev : parent du dossier SimulateurH). */
16 function resolveRepoRoot() {
17     if (isDevMode()) {
18         return path.join(__dirname, "..");
19     }
20     return path.join(process.resourcesPath, "simulator-app");
21 }
22
23 function resolveStandaloneServerModule() {
24     return require(path.join(__dirname, "simulate-server.cjs"));
25 }
26
27 async function startBackend() {
28     const { startStandaloneSimulateServer } = resolveStandaloneServerModule();
29     const repoRoot = resolveRepoRoot();
30     const port = Number(process.env.SIMULATEUR_H_PORT) || DEFAULT_PORT;
31     const { server, url } = await startStandaloneSimulateServer({
32         repoRoot,
33         port,
34         host: "127.0.0.1",
35     });
36     httpServer = server;
37     return `${url}?app=h`;
38 }
39
40 function createWindow(startUrl) {
41     mainWindow = new BrowserWindow({
42         title: "Simulateur H",
43         width: 1280,
44         height: 860,
45         minWidth: 960,
46         minHeight: 640,
47         autoHideMenuBar: true,
48         webPreferences: {
49             contextIsolation: true,
50             nodeIntegration: false,
51             sandbox: true,
52         },
53     });
54
55     mainWindow.loadURL(startUrl);
56
57     mainWindow.webContents.setWindowOpenHandler(({ url }) => {
58         if (url.startsWith("http://127.0.0.1") || url.startsWith("http://localhost")) {
59             return { action: "allow" };
60         }
61         shell.openExternal(url);
62         return { action: "deny" };
63     });
64
65     mainWindow.on("closed", () => {
66         mainWindow = null;
67     });
68 }
69
70 function stopBackend() {
71     if (httpServer) {
72         try {
73             httpServer.close();
74         } catch {
75             /* ignore */
76         }
77         httpServer = null;
78     }

```

```
79 }
80
81 const gotLock = app.requestSingleInstanceLock();
82 if (!gotLock) {
83   app.quit();
84 } else {
85   app.on("second-instance", () => {
86     if (mainWindow) {
87       if (mainWindow.isMinimized()) mainWindow.restore();
88       mainWindow.focus();
89     }
90   });
91
92   app.whenReady().then(async () => {
93     try {
94       const startUrl = await startBackend();
95       createWindow(startUrl);
96     } catch (err) {
97       console.error("[Simulateur H] Démarrage impossible :", err);
98       const { dialog } = require("electron");
99       dialog.showErrorBox(
100         "Simulateur H – erreur de démarrage",
101         (err && err.message) || String(err)
102       );
103       app.exit(1);
104     }
105   });
106
107   app.on("window-all-closed", () => {
108     stopBackend();
109     app.quit();
110   });
111
112   app.on("before-quit", () => {
113     stopBackend();
114   });
115 }
116
```

```

1 "use strict";
2 /**
3  * Serveur minimal Simulateur H : fichiers statiques + POST /api/simulate.
4  * Fichier dans l'asar Electron (express disponible). repoRoot = dossier avec Simulateur/ et tools/.
5  */
6 const express = require("express");
7 const path = require("path");
8 const os = require("node:os");
9 const { execFile } = require("node:child_process");
10 const { copyFile, mkdtemp, readFile: readFileAsync, rm, writeFile } = require("node:fs/promises");
11 const { pathToFileURL } = require("node:url");
12
13 const XSPICE_DIGITAL_CM_PLACEHOLDER = "__XSPICE_DIGITAL_CM__";
14 const SIM_ENGINE_BUILD_TAG = "v2-reset-2026-05-09";
15
16 function createSimulateStandaloneApp(repoRoot) {
17     const {
18         resolveNgspiceForServer,
19         applyPathPrepend,
20         isNgspiceWrongPlatformBinary,
21         resolveDigitalCmSourcePath,
22     } = require(path.join(repoRoot, "tools", "ngspice-bundle.cjs"));
23
24     const app = express();
25     app.use(express.json({ limit: "20mb" }));
26     app.use(express.urlencoded({ extended: true }));
27
28     const dirSimulateur = path.join(repoRoot, "Simulateur");
29
30
31     const ngspiceXspiceProbeModuleUrl = pathToFileURL(
32         path.join(repoRoot, "Simulateur", "Engine", "ngspice-xspice-probe.mjs")
33     ).href;
34     /** @type {{ exe: string | null; envKey: string | null; value: boolean | null }} */
35     let cachedNgspiceXspice = { exe: null, envKey: null, value: null };
36
37     function envPathKey(env) {
38         return env && (env.PATH || env.Path) ? String(env.PATH || env.Path) : "";
39     }
40
41     function forceBsourceFromEnv() {
42         const v = process.env.FORCE_BSOURCE;
43         return v === "1" || String(v || "").toLowerCase() === "true";
44     }
45
46     /**
47      * Détecte si le binaire ngspice du serveur supporte XSPICE (résultat mis en cache).
48      * @param {string} exe
49      * @param {NodeJS.ProcessEnv} env
50      */
51     async function serverNgspiceHasXspice(exe, env) {
52         const envKey = envPathKey(env);
53         if (
54             cachedNgspiceXspice.exe === exe &&
55             cachedNgspiceXspice.envKey === envKey &&
56             cachedNgspiceXspice.value !== null
57         ) {
58             return cachedNgspiceXspice.value;
59         }
60         try {
61             const mod = await import(ngspiceXspiceProbeModuleUrl);
62             const value = mod.ngspiceHasXspice(exe, env);
63             cachedNgspiceXspice = { exe, envKey, value };
64             return value;
65         } catch {
66             cachedNgspiceXspice = { exe, envKey, value: false };
67             return false;
68         }
69     }
70
71     /**
72      * Binaire ngspice : NGSPICE / NGSPICE_PATH, sinon Simulateur/bin/ngspice(.exe)
73      * si vous y copiez le bundle (bin, lib, share).
74      */
75     function ngspiceExecutablePath() {
76         return resolveNgspiceForServer(repoRoot).exe;
77     }
78

```

```

79 /**
80  * @param {{ message?: string; code?: string | number }} error
81  */
82 function isNgspiceMissingError(error) {
83     if (!error) return false;
84     if (error.code === "ENOENT") return true;
85     const msg = `${error.message || ""}`;
86     // Ne pas matcher seul « spawn » : spawn EPERM = blocage sécurité, pas « introuvable ».
87     return /not recognized|introuvable|pas reconnu|cannot find|No such file|est pas une commande|n'est pas reconnu|n'est pas reconnu|command
not found/i.test(
88         msg
89     );
90 }
91
92 /**
93  * @param {{ message?: string; code?: string | number }} error
94  * @param {string} [exeTried]
95  */
96 function isLikelySecuritySoftwareBlock(error, exeTried) {
97     if (!error || isNgspiceWrongPlatformBinary(exeTried || "")) return false;
98     if (!isWin32Platform()) return false;
99     const c = error.code;
100     if (c === "EPERM" || c === "EACCES") return true;
101     const msg = `${error.message || ""}`;
102     return /operation not permitted|accès refusé|access denied|bloqué|blocked|not permitted by/i.test(msg);
103 }
104
105 function isWin32Platform() {
106     return process.platform === "win32";
107 }
108
109 // --- Moteur SPICE (modules ESM chargés une seule fois) ---
110 const {
111     loadSimEngineModules,
112     preloadSimEngineModules,
113 } = require(path.join(repoRoot, "tools", "simulate-engine-loader.cjs"));
114
115 /** buildNgspiceDeck est async (ESM) ; tolère aussi une version synchrone ancienne. */
116 async function invokeBuildNgspiceDeck(buildFn, state, opts) {
117     const result = buildFn(state, opts);
118     if (result != null && typeof result.then === "function") return await result;
119     return result;
120 }
121
122 /** Délai exec ngspice : circuits 74HC90 / CD4511 (.tran jusqu'à ~120 s simulées). */
123 function ngspiceExecTimeoutMs(netlistText) {
124     const text = String(netlistText || "");
125     if (!/\.tran\b/i.test(text)) return 30000;
126     const m = text.match(/\.tran\s+\S+(\S+)/i);
127     if (!m) return 90000;
128     const tstop = parseFloat(m[1]);
129     if (!Number.isFinite(tstop) || tstop <= 0) return 90000;
130     return Math.min(180000, Math.max(15000, Math.round(tstop * 3000 + 10000)));
131 }
132
133 function runNgspice(netlistPath, outputPath, opts = {}) {
134     const { exe, prependPath } = resolveNgspiceForServer(repoRoot);
135     const env = applyPathPrepend(process.env, prependPath);
136     const cwd = opts.cwd || repoRoot;
137     const timeoutMs = opts.timeoutMs ?? 90000;
138     // Linux (Docker/Render) : -i charge digital.cm copié dans le répertoire de travail.
139     // Windows : pas de -i/-f (ngspice_con charge spinit du bundle).
140     const args = ["-b"];
141     if (opts.xspiceRc && process.platform !== "win32") {
142         args.push("-i", opts.xspiceRc);
143     }
144     args.push("-o", outputPath, netlistPath);
145     return new Promise((resolve, reject) => {
146         execFile(
147             exe,
148             args,
149             {
150                 windowsHide: true,
151                 timeout: timeoutMs,
152                 maxBuffer: 8 * 1024 * 1024,
153                 env,
154                 cwd,
155             },
156             (error, stdout, stderr) => {
157                 if (error) {

```

```

158         reject({
159             message: error.message,
160             code: error.code,
161             stdout: stdout || "",
162             stderr: stderr || "",
163             ngspiceExe: exe,
164         });
165         return;
166     }
167     resolve({ stdout: stdout || "", stderr: stderr || "" });
168 }
169 );
170 });
171 }
172
173 app.get("/api/simulate", (req, res) => {
174     res.status(405).json({
175         ok: false,
176         phase: "method",
177         errors: [
178             "Cette URL accepte uniquement POST (pas GET dans la barre d'adresse).",
179             "Lancez « npm start », puis ouvrez http://localhost:3000/Simulateur/ et cliquez sur « Lancer Simulation ».",
180         ],
181     });
182 });
183
184 app.post("/api/simulate", async (req, res) => {
185     const state = req.body?.state;
186     if (
187         !state ||
188         typeof state !== "object" ||
189         !Array.isArray(state.components) ||
190         !Array.isArray(state.wires)
191     ) {
192         return res.status(400).json({
193             ok: false,
194             phase: "build",
195             errors: [
196                 "Schéma non reçu par le serveur. Rechargez le simulateur (Ctrl+F5), puis réessayez.",
197             ],
198             warnings: [],
199             netlist: "",
200         });
201     }
202     let eng;
203     try {
204         eng = await loadSimEngineModules(repoRoot);
205     } catch (error) {
206         res.status(500).json({
207             ok: false,
208             phase: "init",
209             errors: ["Modules ngspice (netlist ou parseur resultats) indisponibles sur le serveur."],
210             details: { message: error?.message || "" }
211         });
212         return;
213     }
214     const {
215         buildNgspiceDeck,
216         mergeVoltmeterMeasurements,
217         mergeAmmeterMeasurements,
218         mergeLedMeasurements,
219         mergeLedTranPlotsFromWrdata,
220         mergeLedValuesFromTranPlots,
221         mergeLogicGateMeasurements,
222         mergeLogicGateTranFromWrdata,
223         mergeLogicGateTranPlotsFromWrdata,
224         mergeOhmmeterMeasurements,
225         mergeOscilloscopeMeasurements,
226         deriveOscilloscopeValuesFromScopePlots,
227         mergeScopePlotsFromTranWrdata,
228         mergeBodePlotsFromAcWrdata,
229         mergeVoltmeterRmsFromTranWrdata,
230         mergeAmmeterRmsFromTranWrdata,
231         mergeVoltmeterFromTranWrdata,
232         mergeAmmeterFromTranWrdata,
233         mergeOhmmeterFromTranWrdata,
234         mergeVoltmeterTranPlotsFromWrdata,
235         mergeSeg7Measurements,
236         mergeSeg7FromTranWrdata,
237         mergeSeg7TranPlotsFromWrdata,

```

```

238     mergeBargraphMeasurements,
239     mergeBargraphFromTranWrdata,
240     mergeBargraphTranPlotsFromWrdata,
241 } = eng;
242
243 const gs = Number(req.body?.gridStep);
244 const { exe: ngspiceExe, prependPath } = resolveNgspiceForServer(repoRoot);
245 const ngspiceEnv = applyPathPrepend(process.env, prependPath);
246 const linuxServer = process.platform !== "win32";
247 const hasXspice = await serverNgspiceHasXspice(ngspiceExe, ngspiceEnv);
248 const forceAllBsource = forceBsourceFromEnv() || !hasXspice;
249 const deckOpts = {
250     repoRoot: repoRoot,
251     ngspiceExe,
252     ngspiceEnv,
253     // CD4511 : d_genlut plante sur ngspice apt (Render) → sources B obligatoires sous Linux.
254     // Bascules D : XSPICE si le binaire le supporte ; repli B sinon.
255     forceBsourceCd4511: linuxServer || forceAllBsource,
256     forceBsourceDff: forceAllBsource,
257     // 74HC90 : bascules JK XSPICE si disponibles (repli B automatique sinon).
258     quickTran: linuxServer,
259 };
260 if (Number.isFinite(gs) && gs > 0) deckOpts.gridStep = gs;
261 const built = await invokeBuildNgspiceDeck(buildNgspiceDeck, state, deckOpts);
262 if (!built || typeof built !== "object") {
263     return res.status(500).json({
264         ok: false,
265         phase: "build",
266         errors: [
267             "Réponse netlist invalide du moteur SPICE. Redéployez le serveur (server.js + Simulateur/Engine/).",
268         ],
269         warnings: [],
270         netlist: "",
271     });
272 }
273 if (!built.ok) {
274     const errs = Array.isArray(built.errors)
275         ? built.errors.filter((e) => e !== null && String(e).trim()).map(String)
276         : [];
277     if (!errs.length) errs.push("Impossible de générer la netlist SPICE à partir du schéma.");
278     res.status(400).json({
279         ok: false,
280         phase: "build",
281         errors: errs,
282         warnings: built.warnings || [],
283         netlist: built.netlist || "",
284     });
285     return;
286 }
287
288 const tempDir = await mkdtemp(path.join(os.tmpdir(), "sim-ngspice-"));
289 const netlistPath = path.join(tempDir, "circuit.cir");
290 const outputPath = path.join(tempDir, "ngspice.log");
291 const wavePathFs = path.join(tempDir, "tran_waves.txt");
292 const acWavePathFs = path.join(tempDir, "ac_waves.txt");
293
294 try {
295     let deckText = built.netlist;
296     // Toujours chemins relatifs : ngspice tourne dans tempDir (évite espaces dans le chemin du projet).
297     if (built.analysisTran && typeof deckText === "string") {
298         deckText = deckText.split("__TRAN_WAVE_PATH__").join("tran_waves.txt");
299     }
300     if (built.analysisAc && typeof deckText === "string") {
301         deckText = deckText.split("__AC_WAVE_PATH__").join("ac_waves.txt");
302     }
303     let xspiceRc;
304     if (typeof deckText === "string" && deckText.includes(XSPICE_DIGITAL_CM_PLACEHOLDER)) {
305         const digitalSrc = resolveDigitalCmSourcePath(repoRoot);
306         if (!digitalSrc) {
307             return res.status(500).json({
308                 ok: false,
309                 phase: "run",
310                 errors: [
311                     "digital.cm introuvable (Simulateur/lib/ngspice/digital.cm). Voir Simulateur/lib/ngspice/README.txt.",
312                 ],
313                 warnings: built.warnings,
314                 netlist: built.netlist,
315             });
316         }
317         await copyFile(digitalSrc, path.join(tempDir, "digital.cm"));

```

```

318     xspiceRc = path.join(tempDir, "ngspice-xspice.rc");
319     await writeFile(xspiceRc, "codemodel digital.cm\n", "utf8");
320     xspiceRc = path.basename(xspiceRc);
321 }
322 await writeFile(netlistPath, deckText, "utf8");
323 const spiceTimeoutMs = ngspiceExecTimeoutMs(deckText);
324 const runResult = await runNgspice("circuit.cir", "ngspice.log", {
325     cwd: tempDir,
326     xspiceRc,
327     timeoutMs: spiceTimeoutMs,
328 });
329 let log = "";
330 try {
331     log = await readFileAsync(outputPath, "utf8");
332 } catch {
333     // Certaines variantes/runtime d'ngspice peuvent ne pas ecrire le log attendu.
334 }
335 const combinedLog = [log, runResult.stdout || "", runResult.stderr || ""].
336     .filter((part) => typeof part === "string" && part.trim().length > 0)
337     .join("\n");
338 const tran = !!(/* @type {{ analysisTran?: boolean }} */ (built).analysisTran);
339 const ac = !!(/* @type {{ analysisAc?: boolean }} */ (built).analysisAc);
340 let voltmeterValues = mergeVoltmeterMeasurements(combinedLog, built.voltmeters, built.nodeMeasures || []);
341 let ammeterValues = mergeAmmeterMeasurements(combinedLog, built.ammeters || []);
342 let ledValues = mergeLedMeasurements(combinedLog, built.leds || []);
343 let logicValues = mergeLogicGateMeasurements(combinedLog, built.logicGates || []);
344 let ohmmeterValues = {};
345 let voltmeterRmsValues = {};
346 let ammeterRmsValues = {};
347 let scopePlots = {};
348 let bodePlots = {};
349 let seg7Values = mergeSeg7Measurements(combinedLog, built.seg7Displays || []);
350 let bargraphValues = mergeBargraphMeasurements(combinedLog, built.bargraphDisplays || []);
351 let seg7TranPlots = {};
352 let bargraphTranPlots = {};
353 let logicGateTranPlots = {};
354 let ledTranPlots = {};
355 let voltmeterTranPlots = {};
356 let speakerTranPlots = {};
357 let waveDiag = "";
358 if (tran) {
359     let waveTxt = "";
360     try {
361         waveTxt = await readFileAsync(wavePathFs, "utf8");
362     } catch (waveErr) {
363         waveTxt = "";
364         waveDiag = `[wrdData] Fichier courbes introuvable : ${waveErr?.message || waveErr}`;
365     }
366     const meta = Array.isArray(built.scopesTranMeta) ? built.scopesTranMeta : [];
367     scopePlots = mergeScopePlotsFromTranWrdData(waveTxt, meta);
368     const ledsMeta = Array.isArray(built.ledsTranMeta) ? built.ledsTranMeta : [];
369     ledTranPlots = mergeLedTranPlotsFromWrdData(waveTxt, ledsMeta);
370     const fromTran = mergeLedValuesFromTranPlots(ledTranPlots);
371     if (Object.keys(fromTran).length > 0) ledValues = fromTran;
372     const lgMeta = Array.isArray(built.logicGatesTranMeta) ? built.logicGatesTranMeta : [];
373     logicGateTranPlots = mergeLogicGateTranPlotsFromWrdData(waveTxt, lgMeta);
374     const fromTranLg = mergeLogicGateTranFromWrdData(waveTxt, lgMeta);
375     if (Object.keys(fromTranLg).length > 0) logicValues = fromTranLg;
376     const metersMeta = built.metersTranMeta || {};
377     voltmeterValues = mergeVoltmeterFromTranWrdData(
378         waveTxt,
379         Array.isArray(metersMeta.voltmeters) ? metersMeta.voltmeters : []
380     );
381     voltmeterTranPlots = mergeVoltmeterTranPlotsFromWrdData(
382         waveTxt,
383         Array.isArray(metersMeta.voltmeters) ? metersMeta.voltmeters : []
384     );
385     speakerTranPlots = mergeVoltmeterTranPlotsFromWrdData(
386         waveTxt,
387         Array.isArray(metersMeta.speakers) ? metersMeta.speakers : []
388     );
389     ammeterValues = mergeAmmeterFromTranWrdData(
390         waveTxt,
391         Array.isArray(metersMeta.ammeters) ? metersMeta.ammeters : []
392     );
393     voltmeterRmsValues = mergeVoltmeterRmsFromTranWrdData(
394         waveTxt,
395         Array.isArray(metersMeta.voltmetersRms) ? metersMeta.voltmetersRms : []
396     );
397     ammeterRmsValues = mergeAmmeterRmsFromTranWrdData(

```

```

398         waveTxt,
399         Array.isArray(metersMeta.ammetersRms) ? metersMeta.ammetersRms : []
400     );
401     const seg7Meta = Array.isArray(built.seg7TranMeta) ? built.seg7TranMeta : [];
402     const fromTranSeg7 = mergeSeg7FromTranWrdData(waveTxt, seg7Meta);
403     if (Object.keys(fromTranSeg7).length > 0) seg7Values = fromTranSeg7;
404     seg7TranPlots = mergeSeg7TranPlotsFromWrdData(waveTxt, seg7Meta);
405     const bargraphMeta = Array.isArray(built.bargraphTranMeta) ? built.bargraphTranMeta : [];
406     const fromTranBargraph = mergeBargraphFromTranWrdData(waveTxt, bargraphMeta);
407     if (Object.keys(fromTranBargraph).length > 0) bargraphValues = fromTranBargraph;
408     bargraphTranPlots = mergeBargraphTranPlotsFromWrdData(waveTxt, bargraphMeta);
409     /* Diagnostic visible dans Vérification → Journal */
410     const linesCnt = waveTxt ? waveTxt.split("\n").length : 0;
411     const plotKeys = Object.keys(scopePlots);
412     const ledPlotKeys = Object.keys(ledTranPlots);
413     waveDiag = [
414         `[wrdData] Fichier courbes : ${waveTxt.length} octets, ${linesCnt} lignes`,
415         `[wrdData] Premières données : ${waveTxt.slice(0, 300).replace(/\r/g, "")} || "(vide)"`,
416         `[wrdData] Oscilloscopes : ${plotKeys.length ? plotKeys.join(", ") : "(aucun)"}`,
417         `[wrdData] LED (courant i(VIL_*)) : ${ledPlotKeys.length ? ledPlotKeys.join(", ") : "(aucun)"}`
418     ].join("\n");
419     const oscList = Array.isArray(built.oscilloscopes) ? built.oscilloscopes : [];
420     if (oscList.length > 0 && plotKeys.length === 0) {
421         const miss = oscList.map((o) => o.id).join(", ");
422         built.warnings = built.warnings || [];
423         built.warnings.push(
424             `[wrdData] Oscilloscope(s) ${miss} : aucune courbe dans tran_waves.txt. Reliez CH1, CH2 et la masse (borne du bas) ; vérifiez qu'il
y a un générateur sinus/carré.`
425         );
426     }
427 }
428 if (ac) {
429     let acWaveTxt = "";
430     try {
431         acWaveTxt = await readFileAsync(acWavePathFs, "utf8");
432     } catch (acErr) {
433         acWaveTxt = "";
434         waveDiag = `[wrdData ac] Fichier courbes introuvable : ${acErr?.message || acErr}`;
435     }
436     const bodeMeta = Array.isArray(built.bodeAcMeta) ? built.bodeAcMeta : [];
437     bodePlots = mergeBodePlotsFromAcWrdData(acWaveTxt, bodeMeta);
438     const bodeList = Array.isArray(built.bodeAnalyzers) ? built.bodeAnalyzers : [];
439     if (bodeList.length > 0 && Object.keys(bodePlots).length === 0) {
440         const miss = bodeList.map((b) => b.id).join(", ");
441         built.warnings = built.warnings || [];
442         built.warnings.push(
443             `[wrdData] Analyse fréquentielle ${miss} : aucune courbe dans ac_waves.txt. Vérifiez le câblage et le générateur sinus.`
444         );
445     }
446 }
447 const ohmList = Array.isArray(built.ohmmeters) ? built.ohmmeters : [];
448 const ohmIsoNetlist = built.ohmmeterIsolationNetlist;
449 if (ohmList.length > 0 && typeof ohmIsoNetlist === "string" && ohmIsoNetlist.trim()) {
450     try {
451         const ohmLogPath = path.join(tempDir, "ngspice_ohm.log");
452         await writeFile(path.join(tempDir, "circuit_ohm.cir"), ohmIsoNetlist, "utf8");
453         const ohmRun = await runNgspice("circuit_ohm.cir", "ngspice_ohm.log", {
454             cwd: tempDir,
455             xspiceRc,
456             timeoutMs: spiceTimeoutMs,
457         });
458         let ohmLog = "";
459         try {
460             ohmLog = await readFileAsync(ohmLogPath, "utf8");
461         } catch {
462             /* pas de log ohmmètre */
463         }
464         const ohmCombined = [ohmLog, ohmRun.stdout || "", ohmRun.stderr || ""]
465             .filter((part) => typeof part === "string" && part.trim().length > 0)
466             .join("\n");
467         ohmmeterValues = mergeOhmmeterMeasurements(ohmCombined, ohmList);
468     } catch (ohmErr) {
469         built.warnings = built.warnings || [];
470         built.warnings.push(
471             `Ohmmètre : mesure en mode isolation impossible – vérifiez le câblage ou retirez l'ohmmètre du circuit alimenté.`
472         );
473         ohmmeterValues = mergeOhmmeterMeasurements(combinedLog, ohmList);
474     }
475 }
476 const oscilloscopeValues = tran

```



```

477         ? deriveOscilloscopeValuesFromScopePlots(scopePlots)
478         : mergeOscilloscopeMeasurements(combinedLog, built.oscilloscopes || []);
479     res.json({
480         ok: true,
481         warnings: built.warnings,
482         netlist: deckText,
483         log: [combinedLog || log, waveDiag].filter(Boolean).join("\n\n--- DIAGNOSTIC COURBES ---\n"),
484         voltmeterValues,
485         voltmeterIds: Array.isArray(built.voltmeters) ? built.voltmeters.map((v) => v.id) : [],
486         voltmeterNodes: Array.isArray(built.voltmeters) ? built.voltmeters : [],
487         ammeterValues,
488         ammeterIds: Array.isArray(built.ammeters) ? built.ammeters.map((a) => a.id) : [],
489         ammeterBranches: Array.isArray(built.ammeters) ? built.ammeters : [],
490         ledValues,
491         ledIds: Array.isArray(built.leds) ? built.leds.map((l) => l.id) : [],
492         ledBranches: Array.isArray(built.leds) ? built.leds : [],
493         ledTranPlots,
494         voltmeterTranPlots,
495         speakerTranPlots,
496         logicValues,
497         logicIds: Array.isArray(built.logicGates) ? built.logicGates.map((g) => g.id) : [],
498         logicGates: Array.isArray(built.logicGates) ? built.logicGates : [],
499         voltmeterRmsValues,
500         voltmeterRmsIds: Array.isArray(built.voltmetersRms)
501             ? built.voltmetersRms.map((v) => v.id)
502             : [],
503         voltmeterRmsNodes: Array.isArray(built.voltmetersRms) ? built.voltmetersRms : [],
504         ammeterRmsValues,
505         ammeterRmsIds: Array.isArray(built.ammetersRms) ? built.ammetersRms.map((a) => a.id) : [],
506         ammeterRmsBranches: Array.isArray(built.ammetersRms) ? built.ammetersRms : [],
507         ohmmeterValues,
508         ohmmeterIds: Array.isArray(built.ohmmeters) ? built.ohmmeters.map((o) => o.id) : [],
509         ohmmeterNodes: Array.isArray(built.ohmmeters) ? built.ohmmeters : [],
510         oscilloscopeValues,
511         oscilloscopeIds: Array.isArray(built.oscilloscopes)
512             ? built.oscilloscopes.map((o) => o.id)
513             : [],
514         oscilloscopeNodes: Array.isArray(built.oscilloscopes) ? built.oscilloscopes : [],
515         analysisTran: tran,
516         analysisAc: ac,
517         scopePlots,
518         bodePlots,
519         seg7Values,
520         seg7TranPlots,
521         bargraphValues,
522         bargraphTranPlots,
523         logicGateTranPlots,
524         seg7Displays: Array.isArray(built.seg7Displays) ? built.seg7Displays : [],
525         bargraphDisplays: Array.isArray(built.bargraphDisplays) ? built.bargraphDisplays : [],
526     });
527 } catch (error) {
528     const exeTried = /** @type {{ ngspiceExe?: string }} */ (error)?.ngspiceExe || ngspiceExecutablePath();
529     const wrongPlatform = isNgspiceWrongPlatformBinary(exeTried);
530     const missing = isNgspiceMissingError(error);
531     const securityBlock = isLikelySecuritySoftwareBlock(error, exeTried);
532     let logTail = "";
533     try {
534         logTail = await readFileAsync(outputPath, "utf8");
535     } catch {
536         /* pas de ngspice.log */
537     }
538     const tailOut = [logTail, error?.stderr, error?.stdout, error?.message]
539         .filter((x) => typeof x === "string" && x.trim())
540         .join("\n")
541         .trim()
542         .slice(-4000);
543     let primary;
544     if (wrongPlatform) {
545         primary =
546             `Binaire ngspice incompatible (${exeTried}) : un exécutable Windows (.exe) ne peut pas tourner sur ${process.platform}. ` +
547             `Sur Render / Docker / Linux : définir la variable d'environnement NGSPICE=/usr/bin/ngspice (voir Dockerfile) ou installer ng
548             spice via le système ; ne pas déployer Simulateur/bin/ngspice.exe sur le serveur. ` +
549             `En local Windows : utiliser ngspice.exe dans Simulateur/bin/ ou NGSPICE vers votre installation.`;
550     } else if (missing) {
551         const exeBase = path.basename(String(exeTried || "").toLowerCase().replace(/\.exe$/i, ""));
552         const probe = exeBase === "ngspice_con" ? "ngspice_con -v" : "ngspice -v";
553         const hint = isWin32Platform()
554             ? "sur Windows : place ngspice_con.exe dans Simulateur/bin/ (livré avec le projet), supprime une éventuelle variable NGSPICE=
555             ngspice mal définie, ou pointe NGSPICE vers le chemin complet du .exe"

```

```

554      : "sur linux : installe ngspice (apt install ngspice) puis exporte NGSPICE=/usr/bin/ngspice – ne déploie pas les .exe Windows
sur le serveur";
555      primary = `ngspice introuvable ou non exécutable (essayé : ${exeTried}). ${hint}, puis relance npm start depuis le même terminal
où « ${probe} » fonctionne.`;
556    } else if (securityBlock) {
557      primary =
558      `Exécution de ngspice bloquée (${exeTried}). Un antivirus ou une stratégie « logiciel inconnu » (ex. Trend Micro) peut empêch
er ngspice.exe : seul un administrateur informatique peut créer une exception.` +
559      `En attendant : utiliser le site hébergé en ligne (Docker / Render) où ngspice tourne sur le serveur, pas sur le PC du visite
ur.`;
560    } else {
561      primary = `Echec d'exécution ngspice (${exeTried}).${tailOut ? `\n\n${tailOut}` : ""}`;
562    }
563    const attachTail = tailOut && (missing || securityBlock || wrongPlatform);
564    res.status(500).json({
565      ok: false,
566      phase: "run",
567      errors: [primary + (attachTail ? `\n\n${tailOut}` : "")],
568      warnings: built.warnings,
569      netlist: built.netlist,
570      details: {
571        message: error?.message || "",
572        stdout: error?.stdout || "",
573        stderr: error?.stderr || "",
574        ngspiceExe: exeTried,
575      },
576    });
577  } finally {
578    await rm(tempDir, { recursive: true, force: true });
579  }
580 });
581
582 app.get("/api/version", (req, res) => {
583   res.json({
584     ok: true,
585     service: "simulateur-h",
586     simEngineBuildTag: SIM_ENGINE_BUILD_TAG,
587     ngspiceExecutable: ngspiceExecutablePath(),
588     pid: process.pid,
589   });
590 });
591
592 const { mountArduinoRoutes } = require(path.join(repoRoot, "tools", "arduino-routes.cjs"));
593 mountArduinoRoutes(app);
594
595 app.use("/Simulateur", express.static(dirSimulateur));
596 app.get("/", (req, res) => res.redirect("/Simulateur/"));
597 return app;
598 }
599
600 function startStandaloneSimulateServer(options = {}) {
601   const repoRoot = options.repoRoot;
602   if (!repoRoot) throw new Error("repoRoot requis");
603   const port = options.port ?? 43721;
604   const host = options.host ?? "127.0.0.1";
605   const app = createSimulateStandaloneApp(repoRoot);
606   const { preloadSimEngineModules: preloadEngine } = require(path.join(
607     repoRoot,
608     "tools",
609     "simulate-engine-loader.cjs"
610   ));
611   preloadEngine(repoRoot);
612   const { resolveNgspiceForServer } = require(path.join(repoRoot, "tools", "ngspice-bundle.cjs"));
613   return new Promise((resolve, reject) => {
614     const server = app.listen(port, host, () => {
615       const { exe: ngspiceExe } = resolveNgspiceForServer(repoRoot);
616       console.log("[Simulateur H] ngspice : " + ngspiceExe);
617       console.log("[Simulateur H] http://" + host + ":" + port + "/Simulateur/");
618       resolve({ server, port, host, url: "http://" + host + ":" + port + "/Simulateur/" });
619     });
620     server.on("error", reject);
621   });
622 }
623
624 module.exports = { createSimulateStandaloneApp, startStandaloneSimulateServer, SIM_ENGINE_BUILD_TAG };
625

```

```

1 "use strict";
2 /**
3  * Intégration arduino-cli : compilation de sketches Arduino UNO.
4  * Variable d'environnement ARDUINO_CLI = chemin vers arduino-cli(.exe).
5  */
6 const { execFile } = require("node:child_process");
7 const fs = require("node:fs");
8 const path = require("node:path");
9 const os = require("node:os");
10 const { mkdir, mkdtemp, readFile, rm, writeFile } = require("node:fs/promises");
11
12 const DEFAULT_FQBN = "arduino:avr:uno";
13
14 function cleanEnvExecutable(s) {
15   let x = String(s || "").trim();
16   if ((x.startsWith('\'') && x.endsWith('\'')) || (x.startsWith('"') && x.endsWith('"'))) {
17     x = x.slice(1, -1).trim();
18   }
19   return x;
20 }
21
22 function resolveArduinoCliPath() {
23   const fromEnv = cleanEnvExecutable(process.env.ARDUINO_CLI);
24   if (fromEnv) {
25     if (path.isAbsolute(fromEnv) && fs.existsSync(fromEnv)) return fromEnv;
26     const fromCwd = path.resolve(process.cwd(), fromEnv);
27     if (fs.existsSync(fromCwd)) return fromCwd;
28     // Chemin ARDUINO_CLI invalide (ex. build Docker incomplet) - retomber sur le PATH
29   }
30   if (process.platform === "win32") {
31     const desktop = path.join(process.env.USERPROFILE || process.env.HOME || "", "Desktop");
32     const candidates = [
33       path.join(desktop, "Arduino-cli", "arduino-cli.exe"),
34       path.join(desktop, "arduino-cli", "arduino-cli.exe"),
35       path.join(process.env.ProgramFiles || "", "Arduino CLI", "arduino-cli.exe"),
36       path.join(process.env["ProgramFiles(x86)"] || "", "Arduino CLI", "arduino-cli.exe"),
37     ];
38     for (const c of candidates) {
39       if (c && fs.existsSync(c)) return c;
40     }
41   }
42   return "arduino-cli";
43 }
44
45 function runCli(args, opts = {}) {
46   const exe = opts.exe || resolveArduinoCliPath();
47   const timeoutMs = opts.timeoutMs ?? 120000;
48   return new Promise((resolve, reject) => {
49     execFile(
50       exe,
51       args,
52       {
53         windowsHide: true,
54         timeout: timeoutMs,
55         maxBuffer: 4 * 1024 * 1024,
56         env: process.env,
57       },
58       (error, stdout, stderr) => {
59         resolve({
60           ok: !error,
61           exe,
62           stdout: stdout || "",
63           stderr: stderr || "",
64           code: error?.code,
65           message: error?.message || "",
66         });
67       }
68     );
69   });
70 }
71
72 function sanitizeSketchFolderName(name) {
73   const base = String(name || "sketch")
74     .replace(/^[^a-zA-Z0-9_]/g, "_")
75     .replace(/^_+/g, "")
76     .slice(0, 40);
77   return base.match(/^[a-zA-Z_]/) ? base : `sketch_${base || "uno"}`;
78 }

```

```

79
80 /** Dossiers racine où l'utilisateur peut déposer des bibliothèques Arduino. */
81 function resolveUserLibraryRoots() {
82     const roots = [];
83     const envPaths = String(process.env.ARDUINO_LIBRARIES || "")
84         .split(/[;|/]/)
85         .map((p) => p.trim())
86         .filter(Boolean);
87     for (const p of envPaths) {
88         const abs = path.isAbsolute(p) ? p : path.resolve(process.cwd(), p);
89         if (fs.existsSync(abs)) roots.push(abs);
90     }
91     for (const rel of ["arduino-libraries", path.join("Simulateur", "arduino-libraries")]) {
92         const abs = path.resolve(process.cwd(), rel);
93         if (fs.existsSync(abs) && !roots.includes(abs)) roots.push(abs);
94     }
95     return roots;
96 }
97
98 function isArduinoLibraryDir(dir) {
99     if (!fs.existsSync(dir) || !fs.statSync(dir).isDirectory()) return false;
100     if (fs.existsSync(path.join(dir, "library.properties"))) return true;
101     const entries = fs.readdirSync(dir);
102     if (entries.some((f) => f.endsWith(".h"))) return true;
103     const src = path.join(dir, "src");
104     return fs.existsSync(src) && fs.statSync(src).isDirectory()
105         && fs.readdirSync(src).some((f) => f.endsWith(".h"));
106 }
107
108 /** Chemins des bibliothèques locales (--library arduino-cli). */
109 function resolveUserLibraryPaths() {
110     const paths = [];
111     for (const root of resolveUserLibraryRoots()) {
112         for (const name of fs.readdirSync(root)) {
113             const libDir = path.join(root, name);
114             if (isArduinoLibraryDir(libDir)) paths.push(libDir);
115         }
116     }
117     return paths;
118 }
119
120 function parseSketchIncludes(sketch) {
121     const headers = new Set();
122     for (const m of String(sketch || "").matchAll(/#include\s*["']([^\s"]+)[>]/g)) {
123         headers.add(m[1].trim());
124     }
125     return [...headers];
126 }
127
128 /** Correspondance en-tête → nom Library Manager (installable via arduino-cli lib install). */
129 const HEADER_LIB_REGISTRY = {
130     "LiquidCrystal_I2C.h": "LiquidCrystal I2C",
131     "rgb_lcd.h": "Grove - LCD RGB Backlight",
132 };
133
134 function findLocallibraryForHeader(header, libraryPaths) {
135     for (const libDir of libraryPaths) {
136         if (fs.existsSync(path.join(libDir, header))) return libDir;
137         if (fs.existsSync(path.join(libDir, "src", header))) return libDir;
138     }
139     return null;
140 }
141
142 async function ensureRegistryLibraries(headers, libraryPaths) {
143     const installed = [];
144     const skipped = [];
145     for (const header of headers) {
146         if (findLocallibraryForHeader(header, libraryPaths)) {
147             skipped.push(header);
148             continue;
149         }
150         const libName = HEADER_LIB_REGISTRY[header];
151         if (!libName) continue;
152         const result = await installArduinoLibrary(libName);
153         if (result.ok) installed.push(libName);
154     }
155     return { installed, skipped };
156 }
157
158 function tryParseCliJson(text) {

```

```

159     const t = String(text || "").trim();
160     if (!t) return null;
161     try {
162         return JSON.parse(t);
163     } catch {
164         return null;
165     }
166 }
167
168 function normalizeLibraryEntry(entry) {
169     if (!entry || typeof entry !== "object") return null;
170     if (entry.library && typeof entry.library === "object") {
171         return normalizeLibraryEntry(entry.library);
172     }
173     const name = entry.name || entry.Name;
174     if (!name) return null;
175     return {
176         name: String(name),
177         author: String(entry.author || entry.maintainer || entry.Author || ""),
178         version: String(entry.version || entry.Version || ""),
179         description: String(entry.sentence || entry.paragraph || entry.Sentence || "").slice(0, 160),
180     };
181 }
182
183 function dedupeLibrariesByName(items) {
184     const seen = new Set();
185     const out = [];
186     for (const item of items) {
187         const key = item.name.toLowerCase();
188         if (seen.has(key)) continue;
189         seen.add(key);
190         out.push(item);
191     }
192     return out;
193 }
194
195 /** @returns {Promise<{ ok: boolean; libraries: object[]; log?: string; errors?: string[] }>} */
196 async function searchArduinoLibraries(query) {
197     const q = String(query || "").trim();
198     if (!q) {
199         return { ok: false, libraries: [], errors: ["Saisissez un nom de bibliothèque à rechercher."] };
200     }
201     const run = await runCli(["lib", "search", q, "--format", "json"], { timeoutMs: 120000 });
202     const log = [run.stdout, run.stderr].filter(Boolean).join("\n").trim();
203     const data = tryParseCliJson(run.stdout);
204     let raw = [];
205     if (Array.isArray(data)) raw = data;
206     else if (Array.isArray(data?.libraries)) raw = data.libraries;
207     const libraries = dedupeLibrariesByName(
208         raw.map(normalizeLibraryEntry).filter(Boolean)
209     );
210     if (!run.ok) {
211         return {
212             ok: false,
213             libraries,
214             log,
215             errors: ["Recherche Library Manager échouée.", run.message, log].filter(Boolean),
216         };
217     }
218     return { ok: true, libraries, log };
219 }
220
221 /** Bibliothèques installées (Library Manager + dossier arduino-libraries). */
222 async function listInstalledArduinoLibraries() {
223     const run = await runCli(["lib", "list", "--format", "json"], { timeoutMs: 90000 });
224     const log = [run.stdout, run.stderr].filter(Boolean).join("\n").trim();
225     const data = tryParseCliJson(run.stdout);
226     let raw = [];
227     if (Array.isArray(data)) raw = data;
228     else if (Array.isArray(data?.installed_libraries)) raw = data.installed_libraries;
229     const registry = dedupeLibrariesByName(
230         raw.map(normalizeLibraryEntry).filter(Boolean)
231     ).map((lib) => ({ ...lib, source: "registry" }));
232
233     const local = resolveUserLibraryPaths().map((p) => ({
234         name: path.basename(p),
235         author: "",
236         version: "local",
237         description: "Bibliothèque locale (arduino-libraries/)",
238         source: "local",

```

```

239     path: p,
240   });
241
242   const libraries = dedupeLibrariesByName([...registry, ...local]);
243   return {
244     ok: run.ok || local.length > 0,
245     libraries,
246     log,
247     errors: run.ok ? [] : [run.message, log].filter(Boolean),
248   };
249 }
250
251 /** @returns {Promise<{ ok: boolean; name: string; log?: string; errors?: string[] }>} */
252 async function installArduinoLibrary(name) {
253   const libName = String(name || "").trim();
254   if (!libName) {
255     return { ok: false, name: "", errors: ["Nom de bibliothèque manquant."] };
256   }
257   const run = await runCli(["lib", "install", libName], { timeoutMs: 30000 });
258   const log = [run.stdout, run.stderr].filter(Boolean).join("\n").trim();
259   return {
260     ok: run.ok,
261     name: libName,
262     log,
263     errors: run.ok ? [] : ["Installation échouée.", log || run.message].filter(Boolean),
264   };
265 }
266
267 /** @returns {Promise<{ ok: boolean; name: string; log?: string; errors?: string[] }>} */
268 async function uninstallArduinoLibrary(name) {
269   const libName = String(name || "").trim();
270   if (!libName) {
271     return { ok: false, name: "", errors: ["Nom de bibliothèque manquant."] };
272   }
273   const run = await runCli(["lib", "uninstall", libName], { timeoutMs: 12000 });
274   const log = [run.stdout, run.stderr].filter(Boolean).join("\n").trim();
275   return {
276     ok: run.ok,
277     name: libName,
278     log,
279     errors: run.ok ? [] : ["Désinstallation échouée.", log || run.message].filter(Boolean),
280   };
281 }
282
283 async function updateArduinoLibraryIndex() {
284   const run = await runCli(["lib", "update-index"], { timeoutMs: 12000 });
285   const log = [run.stdout, run.stderr].filter(Boolean).join("\n").trim();
286   return {
287     ok: run.ok,
288     log,
289     errors: run.ok ? [] : ["Mise à jour de l'index échouée.", log || run.message].filter(Boolean),
290   };
291 }
292
293 function buildCompileArgs(fqbn, outDir, sketchDir, libraryPaths) {
294   const args = ["compile", "--fqbn", fqbn, "--output-dir", outDir];
295   for (const libDir of libraryPaths) {
296     args.push("--library", libDir);
297   }
298   args.push(sketchDir);
299   return args;
300 }
301
302 async function getArduinoCliStatus() {
303   const exe = resolveArduinoCliPath();
304   const versionRun = await runCli(["version"], { exe });
305   const version = [versionRun.stdout, versionRun.stderr].join("\n").trim();
306   const libraryPaths = resolveUserLibraryPaths();
307   return {
308     ok: versionRun.ok,
309     exe,
310     version: version || null,
311     fqbnDefault: DEFAULT_FQBN,
312     libraryRoots: resolveUserLibraryRoots(),
313     libraryPaths,
314     hint: versionRun.ok
315       ? null
316       : process.env.ARDUINO_CLI && !fs.existsSync(cleanEnvExecutable(process.env.ARDUINO_CLI))
317         ? `ARDUINO_CLI pointe vers un fichier absent (${process.env.ARDUINO_CLI}). Reconstituez l'image Docker ou corrigez la variabl
e.`

```

```

318         : "Installez arduino-cli (https://arduino.github.io/arduino-cli/) puis définissez ARDUINO_CLI ou ajoutez-le au PATH.",
319     };
320 }
321
322 /**
323  * @param {{ sketch: string; sketchName?: string; fqbn?: string }} opts
324  */
325 async function compileArduinoSketch(opts) {
326     const sketch = String(opts?.sketch || "");
327     if (!sketch.trim()) {
328         return { ok: false, errors: ["Sketch vide."], log: "" };
329     }
330     const fqbn = String(opts?.fqbn || DEFAULT_FQBN);
331     const folderName = sanitizeSketchFolderName(opts?.sketchName || "sketch");
332     const tmpRoot = await mkdtemp(path.join(os.tmpdir(), "sim-arduino-"));
333     const sketchDir = path.join(tmpRoot, folderName);
334     await mkdir(sketchDir, { recursive: true });
335     await writeFile(path.join(sketchDir, `${folderName}.ino`), sketch, "utf8");
336     const outDir = path.join(tmpRoot, "build");
337     await mkdir(outDir, { recursive: true });
338
339     let libraryPaths = resolveUserLibraryPaths();
340     const headers = parseSketchIncludes(sketch);
341     let registryInstall = await ensureRegistryLibraries(headers, libraryPaths);
342     if (registryInstall.installed.length > 0) {
343         libraryPaths = resolveUserLibraryPaths();
344     }
345
346     try {
347         let compileRun = await runCli(
348             buildCompileArgs(fqbn, outDir, sketchDir, libraryPaths),
349             { timeoutMs: 180000 }
350         );
351         let log = [compileRun.stdout, compileRun.stderr].filter(Boolean).join("\n").trim();
352
353         if (!compileRun.ok && /No such file or directory/i.test(log)) {
354             registryInstall = await ensureRegistryLibraries(headers, libraryPaths);
355             if (registryInstall.installed.length > 0) {
356                 libraryPaths = resolveUserLibraryPaths();
357                 compileRun = await runCli(
358                     buildCompileArgs(fqbn, outDir, sketchDir, libraryPaths),
359                     { timeoutMs: 180000 }
360                 );
361                 log = [compileRun.stdout, compileRun.stderr].filter(Boolean).join("\n").trim();
362             }
363         }
364
365         if (!compileRun.ok) {
366             const libHint = libraryPaths.length
367                 ? `Bibliothèques locales : ${libraryPaths.map((p) => path.basename(p)).join(", ")}`
368                 : "Ajoutez vos bibliothèques dans le dossier arduino-libraries/ (un sous-dossier par lib).";
369             return {
370                 ok: false,
371                 errors: [
372                     "Compilation Arduino échouée.",
373                     compileRun.message ? `(${compileRun.message})` : "",
374                     libHint,
375                     log ? log.slice(-6000) : "Vérifiez arduino-cli core install arduino:avr.",
376                 ].filter(Boolean),
377                 log,
378                 fqbn,
379                 exe: compileRun.exe,
380                 libraryPaths,
381             };
382         }
383         let hexPath = null;
384         try {
385             const files = fs.readdirSync(outDir);
386             const hex = files.find((f) => f.endsWith(".hex"));
387             if (hex) hexPath = path.join(outDir, hex);
388         } catch {
389             /* ignore */
390         }
391         return {
392             ok: true,
393             log: log || "Compilation OK.",
394             fqbn,
395             exe: compileRun.exe,
396             hexPath,
397             buildDir: outDir,

```

```
398     sketchDir,
399     tmpRoot,
400     libraryPaths,
401   };
402   } catch (err) {
403     await rm(tmpRoot, { recursive: true, force: true }).catch(() => {});
404     return {
405       ok: false,
406       errors: [err?.message || String(err)],
407       log: "",
408     };
409   }
410 }
411
412 module.exports = {
413   DEFAULT_FQBN,
414   resolveArduinoCliPath,
415   resolveUserLibraryRoots,
416   resolveUserLibraryPaths,
417   parseSketchIncludes,
418   searchArduinoLibraries,
419   listInstalledArduinoLibraries,
420   installArduinoLibrary,
421   uninstallArduinoLibrary,
422   updateArduinoLibraryIndex,
423   getArduinoCliStatus,
424   compileArduinoSketch,
425   sanitizeSketchFolderName,
426 };
427
```



```

1 "use strict";
2 const {
3   getArduinoCliStatus,
4   compileArduinoSketch,
5   resolveUserLibraryRoots,
6   resolveUserLibraryPaths,
7   searchArduinoLibraries,
8   listInstalledArduinoLibraries,
9   installArduinoLibrary,
10  uninstallArduinoLibrary,
11  updateArduinoLibraryIndex,
12  DEFAULT_FQBN,
13 } = require("./arduino-api.cjs");
14
15 /**
16  * Monte les routes REST Arduino sur une app Express.
17  * @param {import("express").Express} app
18  */
19 function mountArduinoRoutes(app) {
20   app.get("/api/arduino/status", async (_req, res) => {
21     try {
22       const status = await getArduinoCliStatus();
23       res.json({ ok: true, ...status });
24     } catch (err) {
25       res.status(500).json({
26         ok: false,
27         errors: [err?.message || String(err)],
28       });
29     }
30   });
31
32   app.get("/api/arduino/libraries", async (_req, res) => {
33     try {
34       const result = await listInstalledArduinoLibraries();
35       res.json({
36         ok: result.ok,
37         roots: resolveUserLibraryRoots(),
38         libraries: result.libraries,
39         log: result.log || "",
40         errors: result.errors || [],
41       });
42     } catch (err) {
43       res.status(500).json({ ok: false, errors: [err?.message || String(err)] });
44     }
45   });
46
47   app.get("/api/arduino/lib/search", async (req, res) => {
48     const q = req.query?.q || req.query?.query || "";
49     try {
50       const result = await searchArduinoLibraries(q);
51       if (!result.ok) {
52         return res.status(400).json(result);
53       }
54       res.json(result);
55     } catch (err) {
56       res.status(500).json({ ok: false, libraries: [], errors: [err?.message || String(err)] });
57     }
58   });
59
60   app.post("/api/arduino/lib/install", async (req, res) => {
61     const name = req.body?.name || req.body?.library;
62     try {
63       const result = await installArduinoLibrary(name);
64       if (!result.ok) {
65         return res.status(400).json(result);
66       }
67       res.json(result);
68     } catch (err) {
69       res.status(500).json({ ok: false, errors: [err?.message || String(err)] });
70     }
71   });
72
73   app.post("/api/arduino/lib/uninstall", async (req, res) => {
74     const name = req.body?.name || req.body?.library;
75     try {
76       const result = await uninstallArduinoLibrary(name);
77       if (!result.ok) {
78         return res.status(400).json(result);

```

```

79     }
80     res.json(result);
81   } catch (err) {
82     res.status(500).json({ ok: false, errors: [err?.message || String(err)] });
83   }
84 });
85
86 app.post("/api/arduino/lib/update-index", async (_req, res) => {
87   try {
88     const result = await updateArduinoLibraryIndex();
89     if (!result.ok) {
90       return res.status(400).json(result);
91     }
92     res.json(result);
93   } catch (err) {
94     res.status(500).json({ ok: false, errors: [err?.message || String(err)] });
95   }
96 });
97
98 app.post("/api/arduino/compile", async (req, res) => {
99   const sketch = req.body?.sketch;
100   const sketchName = req.body?.sketchName || req.body?.label || "sketch";
101   const fqbn = req.body?.fqbn || DEFAULT_FQBN;
102   try {
103     const result = await compileArduinoSketch({ sketch, sketchName, fqbn });
104     if (!result.ok) {
105       return res.status(400).json(result);
106     }
107     res.json({
108       ok: true,
109       log: result.log,
110       fqbn: result.fqbn,
111       exe: result.exe,
112       hexPath: result.hexPath || null,
113     });
114   } catch (err) {
115     res.status(500).json({
116       ok: false,
117       errors: [err?.message || String(err)],
118       log: "",
119     });
120   }
121 });
122 }
123
124 module.exports = { mountArduinoRoutes };
125

```

```

1 "use strict";
2 /**
3  * Vérifie en local : modules Engine (ESM) + exécutable ngspice (même logique que server.js).
4  * Usage : npm run check-ngspice
5  * Optionnel : NGSPICE ou NGSPICE_PATH. Sinon : Simulateur/bin/ngspice(.exe) + PATH bin/lib.
6  */
7 const { execFile } = require("node:child_process");
8 const path = require("node:path");
9 const { pathToFileURL } = require("node:url");
10 const { resolveNgspiceForServer, applyPathPrepend } = require("./ngspice-bundle.cjs");
11
12 async function loadNgspiceHasXspice() {
13     const url = pathToFileURL(
14         path.join(__dirname, "..", "Simulateur", "Engine", "ngspice-xspice-probe.mjs")
15     ).href;
16     const mod = await import(url);
17     return mod.ngspiceHasXspice;
18 }
19
20 function runNgspiceVersion(bin, env) {
21     return new Promise((resolve, reject) => {
22         execFile(
23             bin,
24             ["-v"],
25             { windowsHide: true, timeout: 15000, maxBuffer: 2 * 1024 * 1024, env },
26             (err, stdout, stderr) => {
27                 if (err) reject(err);
28                 else resolve({ stdout: stdout || "", stderr: stderr || "" });
29             }
30         );
31     });
32 }
33
34 async function main() {
35     const root = path.join(__dirname, "..");
36     const { exe, prependPath } = resolveNgspiceForServer(root);
37     const env = applyPathPrepend(process.env, prependPath);
38     console.log("Binaire ngspice attendu :", exe);
39     if (prependPath.length) console.log("PATH préfixé (portable) :", prependPath.join(path.delimiter));
40
41     const deckUrl = pathToFileURL(
42         path.join(root, "Simulateur", "Engine", "spice-netlist-v2.mjs")
43     ).href;
44     const mod = await import(deckUrl);
45     if (typeof mod.buildNgspiceDeck !== "function") {
46         throw new Error("buildNgspiceDeck introuvable dans spice-netlist-v2.mjs");
47     }
48
49     const rpUrl = pathToFileURL(
50         path.join(root, "Simulateur", "Engine", "v2", "result-parser.mjs")
51     ).href;
52     const rp = await import(rpUrl);
53     if (typeof rp.mergeVoltmeterMeasurements !== "function") {
54         throw new Error("mergeVoltmeterMeasurements introuvable dans result-parser.mjs");
55     }
56     console.log("Modules Simulateur/Engine : OK");
57
58     try {
59         const ngspiceHasXspice = await loadNgspiceHasXspice();
60         const { stdout, stderr } = await runNgspiceVersion(exe, env);
61         const out = (stdout || stderr || "").trim();
62         console.log("ngspice -v : OK");
63         if (out) console.log("---\n" + out);
64         const xspiceInBanner = /\bxspice\b/i.test(out);
65         const xspice = ngspiceHasXspice(exe, env);
66         const digitalCm = path.join(root, "Simulateur", "lib", "ngspice", "digital.cm");
67         const fs = require("node:fs");
68         const hasCm = fs.existsSync(digitalCm);
69         console.log("XSPICE dans le binaire :", xspice ? "oui" : "non");
70         if (xspice && !xspiceInBanner) {
71             console.log(
72                 "    (devhelp d_dff OK - certaines builds ngspice-46 n'affichent pas « XSPICE » dans ngspice -v)"
73             );
74         }
75         console.log("digital.cm (Simulateur/lib/ngspice/) :", hasCm ? "present" : "absent");
76         if (!xspice) {
77             console.warn(
78                 "\nBascules D : mode XSPICE desactive (devhelp d_dff indisponible). " +

```

```

79         "Verifiez ngspice_con.exe dans Simulateur/bin/ ou une build compilee avec XSPICE."
80     );
81 } else if (!hasCm) {
82     console.warn("\nBascules D : copiez digital.cm dans Simulateur/lib/ngspice/ pour activer d_dff.");
83 }
84 } catch (e) {
85     console.error("ngspice -v : ÉCHEC -", e && e.message ? e.message : e);
86     console.error(
87         process.platform === "win32"
88         ? "\nPlace ngspice.exe ou ngspice_con.exe dans Simulateur/bin/ et les DLL dans Simulateur/lib/, " +
89           "ou définis NGSPICE / NGSPICE_PATH, ou ajoute ngspice au PATH système."
90         : "\nSur Linux : apt install ngspice puis NGSPICE=/usr/bin/ngspice, " +
91           "ou place le binaire ELF dans Simulateur/bin/ngspice (pas ngspice.exe).");
92     );
93     process.exitCode = 1;
94 }
95 }
96
97 main().catch((e) => {
98     console.error(e);
99     process.exit(1);
100 });
101

```

```

1 "use strict";
2 /**
3  * Résolution ngspice : variables NGSPICE / NGSPICE_PATH, sinon bundle portable
4  * dans Simulateur/bin (+ lib pour les DLL Windows).
5  * Sur Linux/macOS : ngspice (ELF), jamais ngspice.exe même s'il est présent dans bin/.
6  */
7 const fs = require("node:fs");
8 const path = require("node:path");
9
10 function isWin32() {
11     return process.platform === "win32";
12 }
13
14 function isPortableWindowsExe(exePath) {
15     return /\.exe$/i.test(String(exePath || ""));
16 }
17
18 function cleanEnvExecutable(s) {
19     let x = String(s).trim();
20     if ((x.startsWith('"') && x.endsWith('"')) || (x.startsWith("'") && x.endsWith("'")))
21         x = x.slice(1, -1).trim();
22     return x;
23 }
24
25 function resolveNgspiceCandidate(p, appRoot) {
26     const t = cleanEnvExecutable(p);
27     if (!t) return "ngspice";
28     if (path.isAbsolute(t)) return path.normalize(t);
29     const fromCwd = path.resolve(process.cwd(), t);
30     if (fs.existsSync(fromCwd)) return fromCwd;
31     const fromApp = path.resolve(appRoot, t);
32     if (fs.existsSync(fromApp)) return fromApp;
33     return t;
34 }
35
36 /** Chemin explicite (absolu ou relatif) vers un fichier exécutable existant. */
37 function ngspiceCandidateExists(exe, appRoot) {
38     if (!exe) return false;
39     if (path.isAbsolute(exe)) return fs.existsSync(exe);
40     if (/[\.\.\/]/.test(exe)) {
41         return (
42             fs.existsSync(path.resolve(process.cwd(), exe)) ||
43             fs.existsSync(path.resolve(appRoot, exe))
44         );
45     }
46     return false;
47 }
48
49 function pathsEqual(a, b) {
50     return path.normalize(a).toLowerCase() === path.normalize(b).toLowerCase();
51 }
52
53 /** Noms des exécutables Windows dans Simulateur/bin (ordre de préférence). */
54 const WIN_BUNDLE_EXE_NAMES = ["ngspice_con.exe", "ngspice.exe"];
55
56 function isBareNgspiceCommand(exe) {
57     const base = path.basename(String(exe || "")).toLowerCase().replace(/\.exe$/i, "");
58     return base === "ngspice" || base === "ngspice_con";
59 }
60
61 /**
62  * @param {string} binDir
63  * @param {string} libDir
64  * @returns {string[]}
65  */
66 function prependPathForSimulateurBundle(binDir, libDir) {
67     const segments = [];
68     if (fs.existsSync(binDir)) segments.push(binDir);
69     if (isWin32() && fs.existsSync(libDir)) segments.push(libDir);
70     const libNgspice = path.join(libDir, "ngspice");
71     if (isWin32() && fs.existsSync(libNgspice)) segments.push(libNgspice);
72     return segments;
73 }
74
75 /**
76  * @param {string} binDir
77  * @returns {string | null}
78  */

```

```

79 function findWindowsBundledExe(binDir) {
80     for (const name of WIN_BUNDLE_EXE_NAMES) {
81         const candidate = path.join(binDir, name);
82         if (fs.existsSync(candidate)) return path.normalize(candidate);
83     }
84     const noExt = path.join(binDir, "ngspice");
85     if (fs.existsSync(noExt)) return path.normalize(noExt);
86     return null;
87 }
88
89 /**
90  * Binaire portable selon l'OS (sans variables d'environnement).
91  * @param {string} binDir
92  * @param {string} libDir
93  * @returns {{ exe: string, prependPath: string[] } | null}
94  */
95 function resolveBundledNgspice(binDir, libDir) {
96     const unixExe = path.join(binDir, "ngspice");
97     const prepend = prependPathForSimulateurBundle(binDir, libDir);
98
99     if (isWin32()) {
100         const win = findWindowsBundledExe(binDir);
101         if (win) return { exe: win, prependPath: prepend };
102     } else if (fs.existsSync(unixExe)) {
103         return { exe: path.normalize(unixExe), prependPath: prepend };
104     }
105
106     const winCon = path.join(binDir, "ngspice_con.exe");
107     if (!isWin32() && fs.existsSync(winCon)) {
108         return { exe: path.normalize(winCon), prependPath: prepend };
109     }
110
111     return null;
112 }
113
114 /**
115  * @param {string} exe Chemin ou nom du binaire ngspice tenté
116  * @returns {boolean} true si .exe sur Linux/macOS (ne peut pas s'exécuter)
117  */
118 function isNgspiceWrongPlatformBinary(exe) {
119     return !isWin32() && isPortableWindowsExe(exe);
120 }
121
122 /**
123  * @param {string} appRoot Racine du projet (typiquement __dirname de server.js)
124  * @returns {{ exe: string, prependPath: string[] }}
125  */
126 function resolveNgspiceForServer(appRoot) {
127     const simRoot = path.join(appRoot, "Simulateur");
128     const binDir = path.join(simRoot, "bin");
129     const libDir = path.join(simRoot, "lib");
130
131     const prependForSimulateurBin = () => prependPathForSimulateurBundle(binDir, libDir);
132
133     const bundled = resolveBundledNgspice(binDir, libDir);
134
135     const raw = process.env.NGSPICE || process.env.NGSPICE_PATH;
136     if (typeof raw === "string" && raw.trim().length > 0) {
137         const exe = resolveNgspiceCandidate(raw, appRoot);
138         if (
139             bundled &&
140             (isBareNgspiceCommand(exe) || !ngspiceCandidateExists(exe, appRoot))
141         ) {
142             return bundled;
143         }
144         const exeDir = path.dirname(path.resolve(exe));
145         if (pathsEqual(exeDir, path.resolve(binDir))) {
146             return { exe, prependPath: prependForSimulateurBin() };
147         }
148         return { exe, prependPath: [] };
149     }
150
151     if (bundled) return bundled;
152
153     const winFallback = isWin32() ? findWindowsBundledExe(binDir) : null;
154     if (winFallback) {
155         return { exe: winFallback, prependPath: prependForSimulateurBin() };
156     }
157
158     // Aucun bundle trouvé : tenter le nom le plus courant sur la plateforme.

```

```

159 // Windows : ngspice_con (console build, sans dépendance X). Linux/macOS : ngspice.
160 return { exe: isWin32() ? "ngspice_con" : "ngspice", prependPath: [] };
161 }
162
163 /**
164  * @param {NodeJS.ProcessEnv} base
165  * @param {string[]} prependPath absolute dirs to prepend to PATH
166  */
167 function applyPathPrepend(base, prependPath) {
168     if (!prependPath.length) return base;
169     const env = { ...base };
170     const sep = path.delimiter;
171     const prefix = prependPath.filter((d) => fs.existsSync(d)).join(sep);
172     if (!prefix) return env;
173     env.PATH = prefix + sep + (env.PATH || "");
174     return env;
175 }
176
177 /**
178  * Chemin source de digital.cm dans le dépôt (pour copie vers dossier temp ngspice).
179  * @param {string} appRoot
180  * @returns {string | null}
181  */
182 function resolveDigitalCmSourcePath(appRoot) {
183     const candidates = [
184         path.join(appRoot, "Simulateur", "lib", "ngspice", "digital.cm"),
185         path.join(appRoot, "Simulateur", "share", "ngspice", "lib", "ngspice", "digital.cm"),
186     ];
187     for (const p of candidates) {
188         if (fs.existsSync(p)) return path.normalize(p);
189     }
190     return null;
191 }
192
193 module.exports = {
194     resolveNgspiceForServer,
195     applyPathPrepend,
196     isNgspiceWrongPlatformBinary,
197     isPortableWindowsExe,
198     resolveDigitalCmSourcePath,
199 };
200

```

```
1 "use strict";
2
3 const fs = require("fs");
4
5 function parseJsonText(raw) {
6   if (raw == null) {
7     throw new Error("Fichier vide.");
8   }
9   const text = String(raw).replace(/^\uFEFF/, "").trim();
10  if (!text) {
11    throw new Error("Fichier vide.");
12  }
13  return JSON.parse(text);
14 }
15
16 /**
17  * Lit un fichier JSON (UTF-8, BOM toléré). Retourne defaultValue si absent ou illisible.
18  * @param {string} filePath
19  * @param {*} defaultValue
20  */
21 function readJsonFileSafe(filePath, defaultValue) {
22   try {
23     if (!fs.existsSync(filePath)) return defaultValue;
24     return parseJsonText(fs.readFileSync(filePath, "utf8"));
25   } catch (err) {
26     console.warn(`[JSON] Lecture échouée (${filePath}) : ${err?.message || err}`);
27     return defaultValue;
28   }
29 }
30
31 module.exports = { parseJsonText, readJsonFileSafe };
32
```



```

1  /**
2   * Chargement unique (cache) des modules ESM du moteur SPICE.
3   * Évite ~25 import() par requête POST /api/simulate (lent sur Render).
4   */
5   const path = require("path");
6   const { pathToFileURL } = require("url");
7
8   /** @type {Promise<Record<string, Function>> | null} */
9   let loadPromise = null;
10
11  const PARSER_EXPORTS = [
12    "mergeVoltmeterMeasurements",
13    "mergeAmmeterMeasurements",
14    "mergeLedMeasurements",
15    "mergeSeg7Measurements",
16    "mergeSeg7FromTranWrdata",
17    "mergeSeg7TranPlotsFromWrdata",
18    "mergeBargraphMeasurements",
19    "mergeBargraphFromTranWrdata",
20    "mergeBargraphTranPlotsFromWrdata",
21    "mergeLedTranPlotsFromWrdata",
22    "mergeLedValuesFromTranPlots",
23    "mergeLogicGateMeasurements",
24    "mergeLogicGateTranFromWrdata",
25    "mergeLogicGateTranPlotsFromWrdata",
26    "mergeOhmmeterMeasurements",
27    "mergeOscilloscopeMeasurements",
28    "mergeScopePlotsFromTranWrdata",
29    "mergeBodePlotsFromAcWrdata",
30    "deriveOscilloscopeValuesFromScopePlots",
31    "mergeVoltmeterRmsFromTranWrdata",
32    "mergeAmmeterRmsFromTranWrdata",
33    "mergeVoltmeterFromTranWrdata",
34    "mergeAmmeterFromTranWrdata",
35    "mergeOhmmeterFromTranWrdata",
36    "mergeVoltmeterTranPlotsFromWrdata",
37  ];
38
39  /**
40   * @param {string} repoRoot répertoire racine du dépôt (contient Simulateur/)
41   * @returns {Promise<Record<string, Function>>}
42   */
43  async function loadSimEngineModules(repoRoot) {
44    if (loadPromise) return loadPromise;
45    loadPromise = (async () => {
46      const deckPath = path.join(repoRoot, "Simulateur", "Engine", "spice-netlist-v2.mjs");
47      const parserPath = path.join(repoRoot, "Simulateur", "Engine", "v2", "result-parser.mjs");
48      const [deckMod, parserMod] = await Promise.all([
49        import(pathToFileURL(deckPath).href),
50        import(pathToFileURL(parserPath).href),
51      ]);
52      if (typeof deckMod.buildNgspiceDeck !== "function") {
53        throw new Error("Module buildNgspiceDeck introuvable.");
54      }
55      const out = { buildNgspiceDeck: deckMod.buildNgspiceDeck };
56      for (const name of PARSER_EXPORTS) {
57        if (typeof parserMod[name] !== "function") {
58          throw new Error(`Module ${name} introuvable.`);
59        }
60        out[name] = parserMod[name];
61      }
62      return out;
63    })();
64    loadPromise.catch(() => {
65      loadPromise = null;
66    });
67    return loadPromise;
68  }
69
70  /** Pré-charge au démarrage du serveur (Render : 1er clic simulation plus rapide). */
71  function preloadSimEngineModules(repoRoot) {
72    loadSimEngineModules(repoRoot).catch((err) => {
73      console.error("[Simulateur] préchargement moteur SPICE :", err?.message || err);
74    });
75  }
76
77  module.exports = { loadSimEngineModules, preloadSimEngineModules };
78

```